

Tab 1

8 Data Scientist:

Machine

Learning

Specialist

CODECAMPY

Hypothesis Test | ML

Welcome to the Data Scientist: Machine Learning Specialist Career Path

Your data science journey begins here!

Machine Learning is at the forefront of data science. It's what drives many of the technologies that we use every day, such as recommendation systems, disease identification, detecting credit card fraud, route optimization, and so much more. Collectively, these technologies represent a shift in how data affects our daily life. If you want to be a part of this movement to leverage data at scale, this Career Path is for you.

About the Data Scientist Career Paths

The Data Scientist Career Paths are designed for you to gain the knowledge, skills, and abilities (KSAs) you need to earn a data scientist job. You will develop the statistical knowledge, programming skills, and visualization techniques to work with data responsibly. You will also build a portfolio along the way to showcase your abilities to future employers.

You are currently enrolled in the Data Scientist: Machine Learning Specialist Career Path. Data Scientists who focus on machine learning specialize in supervised and unsupervised machine learning, feature engineering, model evaluation, and answering data problems at scale. As a Machine Learning based Data Scientist, you might work on a data science, product, marketing, or engineering team. You will often work to create predictive algorithms or identify anomalous activity.

This Career Path is divided into 2 parts. The first section covers all the basics every Data Scientist needs. We call this the Foundations of Data Science. The second section is the specialized KSAs you need to go from being a data generalist to a Data Scientist, Machine Learning Specialist.

If, after learning a bit more about data science, you decide that you'd like to change your specialty, your progress will transfer, allowing you to pick up where you left off. To learn more about the specialties, check out [this blog post](#).

For a quick summary:

Machine Learning Specialists focus on Machine Learning, predictive analytics, creating models and algorithms, and answering questions at scale.

Analytics Specialists focus on answering questions with data, communicating results, and driving decision making.

Inference Specialists focus on finding out why something happened with causal inference, conducting hypothesis tests, and A/B tests, and validating results statistically.

Natural Language Processing Specialists focus on getting meaning out of texts, generating human-like text, and interacting with computers for Artificial Intelligence.

Overview

In the first half of this Career Path, you will learn about Python, SQL, statistics, and data best practices. You will start with a code-free introduction to working with data in the Data Literacy Track and then dive into *SQL* and *Python* in the SQL Fundamentals and Python Fundamentals Tracks. SQL and Python are essential tools that will give you the technical foundation you need to work with data programmatically.

Then you will dive into Data Manipulation with Python Pandas. Pandas is a Python library at the heart of Data Science. It is an essential tool for working with data and has dozens of useful built-in functions. Once you have mastered *Pandas*, you will apply your new programming skills to Exploratory Data Analysis where you will learn to summarize datasets before mastering the Fundamentals of *Statistics* where you will learn to spot trends and patterns.

Next, you will learn how to transform those numbers and analyses into meaningful *Data Visualizations*. Then you will apply your new skills to Wrangle and Tidy Datasets and prepare real data for analysis. Finally, you will bring it all together to Communicate Data Science Findings to various groups and stakeholders.

At this point in your journey, you will have completed the Data Foundations and be ready to move into your specialty.

Machine Learning Specialty

In the Machine Learning Specialty, you will learn about the most commonly used machine learning algorithms, when to apply them, and how to optimize them. You'll be able to leverage data to create predictive models, and analyze large datasets to derive new insights. You can apply your skills to solve a variety of business, medical, educational, and financial problems – machine learning is at the forefront of a wide variety of industries and organizations.

But, if you decide that you want to switch specialties after you learn more about Data Science, you can pick from any of our four specialties – and your progress will transfer!

By the end of the Data Scientist: Machine Learning Specialist Career Path, you will be able to:

- Create programs with Python 3
- Query and manipulate data with SQL and Python pandas
- Create data visualizations with Python matplotlib and seaborn
- Summarize and analyze datasets
- Conduct hypothesis testing
- Design experiments
- Clean and tidy datasets
- Build, train, and test supervised and unsupervised machine learning models
- Evaluate model performance
- Decide when to apply a given model
- Conduct Feature Engineering
- Leverage Jupyter Notebooks for experimentation and communication

Along the way, you will build a portfolio of projects to showcase your data science KSA's, and prove your job-ready skills!

Structure

Throughout this Path, you will see lessons, quizzes, articles, and projects. Lessons and articles introduce new tools and concepts, quizzes give you a chance to quickly check your understanding, and projects give you a chance to apply your new skills.

There are a lot of projects, but pay special attention to Portfolio Projects and Cumulative Projects because they give you a chance to apply your skills and build up your portfolio.

The content in this Career Path is cumulative, and we recommend that you take the courses in order. However, if you are familiar with a given technology or idea, feel free to jump ahead, but be aware that skipping modules will affect your completion percentage.

Finally, we are constantly updating our content to be sure it is up to date and of the highest quality. Keep an eye on the notification bells, which we will use to alert you to new content. Sometimes these updates might affect your completion percentage. We will always do our best to let you know with a notification bell

Welcome to the Machine Learning Specialty

Welcome to the second half of your Career Path: The specialization!!

You are on your way to becoming a Data Scientist with a Machine Learning specialty. Now that you have a foundation in the general skills a Data Scientist needs, you are ready to complete your Machine Learning education.

In this section of the Career Path, we will focus on the specialized skills that you need. We will return to some topics such as statistics and math. We will also explore new topics such as supervised and unsupervised algorithms, feature engineering, hyperparameter tuning, and neural networks.

We will dive back into mathematics to develop a foundation in Linear Algebra and Calculus that will prepare you to understand more complex machine learning algorithms. We will also revisit statistics with hypothesis testing and experimental design.

Along the way, you'll complete challenge projects to build your skills, and capstone and portfolio projects to build your portfolio. By the end of this Data Scientist: Machine Learning Specialist Career Path, you will be ready to start your career in Data Science.

But the most successful candidates don't wait until the end to start creating a portfolio. It is a good idea to start thinking about it now. The best place to start is by getting the projects you are already working on portfolio-ready. But more on that in the next article. See you there!

How to Showcase your Data Science Skills

And start building your data science portfolio.

If a tree falls in the forest and no one is around to hear it, does it make a sound?

– Philosophy students everywhere

Hiring managers have their own version of that question: **If a data scientist doesn't have a portfolio, do they really have the skills?**

You need to have a portfolio. A portfolio is evidence that you have the technical skills and knowledge that you say that you do. But what goes into a portfolio and what does that mean in data science?

What Goes Into a Portfolio

Ultimately, exactly what you put in your portfolio is up to you. This [project at the end of our Interview Prep Skill Path](#) will help guide you through making sure that you've covered all of the major categories, but the projects themselves are completely up to you.

We recommend that you demonstrate the ability to:

Throughout this Career Path, you will encounter Capstone and Portfolio projects. These can absolutely be put into your portfolio. However, thousands of people take this Career Path. To really make yourself stand out, you will want to add projects about things you are interested in.

If you like sports, use sports data, if you like science, use scientific data, etc. Showing a little bit of your interests and what you are curious about will not only make the project more fun for you, but also let you showcase your own domain knowledge or curiosity. Interest and enthusiasm in your projects will help set you apart from other candidates when you are applying and interviewing.

There are a lot of places to look for datasets. We have an entire lesson on where to find data later in this Career Path. For right now, the best thing you can do is be curious about the world around you, and keep track of some things that you might want to explore as you build your portfolio.

What a Data Science Portfolio Looks Like

A data science portfolio can take a lot of shapes, you could create your own professional website, upload documents to your LinkedIn or other social media site, or create a repository on GitHub. We highly recommend GitHub. It is simple, lightweight, and by far the most popular place to house a portfolio.

You have a few choices in terms of what kinds of files you want to upload. A general rule is the simpler, the better. You can upload [Jupyter notebooks](#) of your projects, `.Rmd` or Python files of your code, or PDFs, or presentations of summary reports.

The kind of files you upload will depend on the kind of Data Scientist role you are applying for. A role that is more programming-focused (i.e., a Machine Learning specialization) should have a more programming-heavy portfolio. A role that is more focused on analysis and communication should have at least one summary report.

The Culture of Portfolios

There are a few unspoken rules about portfolios that you should keep in mind. Throughout the Career Path, we've tried to be explicit about when these things come up, and we have presented best practices throughout. However, we want to call out some of the potential pitfalls and make sure you avoid them.

Datasets

There are a few datasets that are used for teaching a lot. Specifically, the [iris dataset](#) and the [titanic dataset](#) are the most common. We use them in the Path, and you will likely find them in the wild. These are so popular that almost everyone who works in data science has seen them. These do not make good portfolio projects because employers might read them as highly scaffolded student projects or as simply replicating a tutorial, and they won't set you apart from other candidates.

Formatting

It is important to pay close attention to presentation. Using the right file types, and presenting content in the right format is essential to getting that first interview.

For example, if you have written some code to analyze a dataset, it should be stored as a `.py`, `.ipynb`, `.R`, or `.Rmd` file. We teach you how to do this in the Career Paths, but it is worth mentioning again that code should almost never be stored in a text document (e.g., `.txt`, `.doc`, etc.) unless it is formatted as a code snippet and used as an example or evidence.

Another example is Tableau (or PowerBI) visualizations and dashboards. There are two ways to store these that demonstrate proficiency with these tools. First, both platforms offer repositories. [Tableau Public](#) is both a great place to show off your work and see what other people are making. The other way is to embed them into a website with an `iframe`.

You should **not** take screenshots, download them as jpegs, pdfs, or otherwise transform the rich, interactive dashboards you've made into static images.

You can (and should) embed static visualizations (that you make in Python, R, or even Excel or Tableau) into written reports, but the skill you are demonstrating is communicating visually about data (not proficiency in Tableau).

Finally, reports and summaries can be woven into your Jupyter Notebooks, presented as written documents (and stored as PDFs), or compiled into slide decks (also stored as PDFs).

Code Hygiene

It takes a long time and a lot of practice to write clear, well-structured code. However, you can take massive steps in that direction by writing well-commented and organized code. Just like writing an essay, writing good code often requires editing. Not all of your projects need to be edited. However, if you choose to highlight one or two (which you can do by pinning them in GitHub), you'll want to be sure that you've written in some comments and reorganized your code for clarity as much as possible.

Grammar

Just like in your resume and cover letter, you want to proofread your portfolio projects to be sure they are free from grammatical or spelling errors. Unlike in your resume/cover letter, a couple typos probably won't disqualify you. However, every Data Scientist needs to communicate about data, and you want to demonstrate that you can do that clearly. Even simply running any reports you write through a spellchecker will help in this respect.

Conclusion

Ultimately, your portfolio is how you verify that you have all of the skills you've worked so hard to acquire. Have fun with it by including some projects that excite you, and be sure to be ready to talk about those projects with recruiters when you get that first interview!

What is Machine Learning?

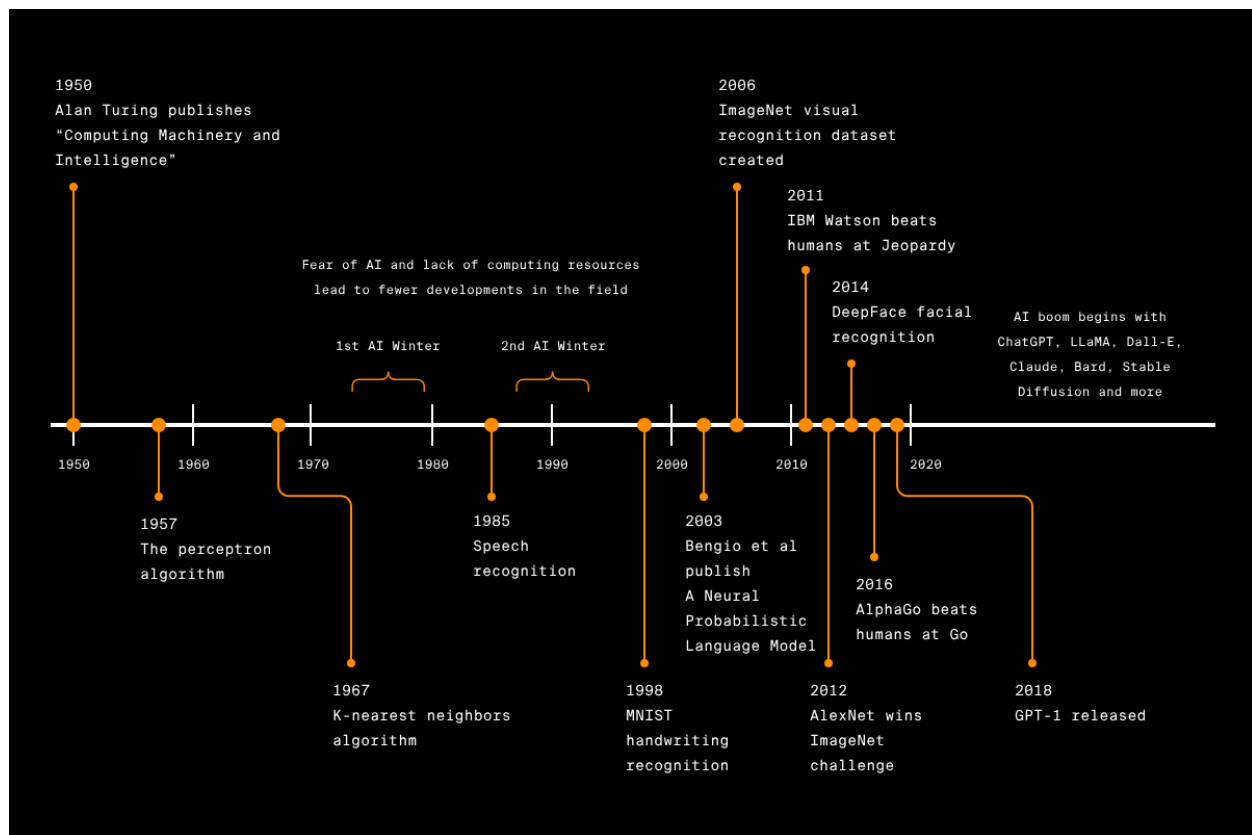
While at IBM, [Arthur Samuel](#) developed a program that learned how to play checkers (1959). He called it:

"The field of study that gives computers the ability to learn without being explicitly programmed."
What does this mean?

As programmers, we often approach problems in a methodical, logic-based way. We try to determine what our desired outputs should be, and then create the proper rules that will transform our inputs into those outputs.

Machine learning flips the script. We want the program itself to learn the rules that describe our data the best, by finding patterns in what we know and applying those patterns to what we don't know.

These algorithms are able to *learn*. Their performance gets better and better with each iteration, as it uncovers more hidden trends in the data.



Supervised Learning

Machine learning can be branched out into the following categories:

Supervised Learning
Unsupervised Learning

[Supervised Learning](#) is where the data is labeled and the program learns to predict the output from the input data. For instance, a supervised learning

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

for credit card fraud detection would take as input a set of recorded transactions. For each
Preview: Docs Loading link description

[transaction](#)

, the program would predict if it is fraudulent or not.

Supervised learning problems can be further grouped into regression and classification problems.

Regression:

In regression problems, we are trying to predict a continuous-valued output. Examples are:

What is the housing price in Neo York?
What is the value of cryptocurrencies?

Classification:

In classification problems, we are trying to predict a discrete number of values. Examples are:

Is this a picture of a human or a picture of an AI?
Is this email spam?

For a quick preview, we will show you an example of supervised learning.

```
from texts import text_counter, text_training
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

intercepted_text = "This hot dog was awful!"

text_counts = text_counter.transform([intercepted_text])

text_classifier = MultinomialNB()

text_labels = [0] * 1000 + [1] * 1000

text_classifier.fit(text_training, text_labels)

final_pos = text_classifier.predict_proba(text_counts)[0][1]

final_neg = text_classifier.predict_proba(text_counts)[0][0]

if final_pos > final_neg:
    print("The text is positive.")
else:
    print("The text is negative.")
```

```
from texts import text_counter, text_training
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

intercepted_text = "I am saotimonean, who love everything about this country
features as food, culture, people, etc."

text_counts = text_counter.transform([intercepted_text])

text_classifier = MultinomialNB()

text_labels = [0] * 1000 + [1] * 1000

text_classifier.fit(text_training, text_labels)
```

```

final_pos = text_classifier.predict_proba(text_counts)[0][1]
final_neg = text_classifier.predict_proba(text_counts)[0][0]
if final_pos > final_neg:
    print("The text is positive.")
else:
    print("The text is negative.")

```

Unsupervised Learning

[Unsupervised Learning](#) is a type of

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

where the program learns the inherent structure of the data based on unlabeled examples.

Clustering is a common unsupervised machine learning approach that finds patterns and structures in unlabeled data by grouping them into clusters.

Some examples:

- Social networks clustering topics in their news feed
- Consumer sites clustering users for recommendations
- Search engines to group similar objects in one cluster

For a quick preview, we will show you an example of unsupervised learning.

```

#import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np

from os.path import join, dirname, abspath
from mpl_toolkits.mplot3d import Axes3D

from sklearn.cluster import KMeans
from sklearn import datasets

iris = datasets.load_iris()

x = iris.data
y = iris.target

fignum = 1

# Plot the ground truth

fig = plt.figure(fignum, figsize=(4, 3))

ax = Axes3D(fig, rect=[0, 0, .95, 1], elev=48, azim=134)

for name, label in [('Robots', 0),
                    ('Cyborgs', 1),
                    ('Humans', 2)]:
    ax.text3D(x[y == label, 3].mean(),
              x[y == label, 0].mean(),
              x[y == label, 2].mean() + 2, name,
              horizontalalignment='center',

```

```

        bbox=dict(alpha=.2, edgecolor='w', facecolor='w'))

# Reorder the labels to have colors matching the cluster results
y = np.choose(y, [1, 2, 0]).astype(np.float)
ax.scatter(x[:, 3], x[:, 0], x[:, 2], c=y, edgecolor='k')

ax.w_xaxis.set_ticklabels([])
ax.w_yaxis.set_ticklabels([])
ax.w_zaxis.set_ticklabels([])

ax.set_xlabel('Time to Heal')
ax.set_ylabel('Reading Speed')
ax.set_zlabel('EQ')

ax.set_title('')
ax.dist = 12

plt.show()

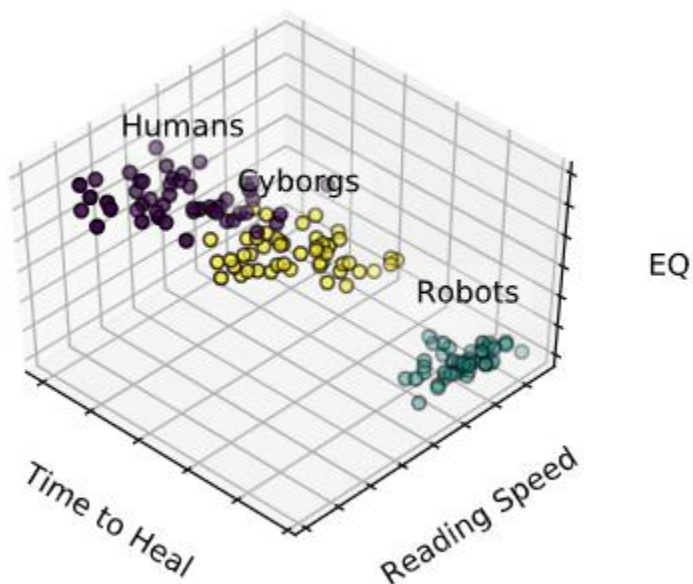
```

NYBD wants to determine how humans and cyborgs differ from each other in terms of:

- The speed of recovering from wounds
- Emotional intelligence (EQ)
- Words per minute (WPM) reading speed

They have taken measurements of these things for the New York population and have plotted them on 3 axes.

Press run to see the clusters!



Begin Your Journey

DING DING DING!

It was but a dream and there's still time. Understanding

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[Machine Learning](#)

will help you change this bleak future, or be ready for it.

As you begin your journey through our Machine Learning curriculum, you will learn the different applications of this powerful field. You will build your own models, test them, and try to improve them. You will analyze bigger, and more complex data and deliver faster, more accurate results — even on a very large scale using:

Supervised Learning

Unsupervised Learning

Throughout your exploration of ML, we hope you gain an understanding of how to harness predictive models to do good, and enhance human life rather than replace it!

The Machine Learning Process

Learn the general structure of how to approach Machine Learning problems in a methodical way.

The Process

When people think of Machine Learning, they often think of a program that is taking in data and spitting out predictions and insights. The process of performing Machine Learning often requires many more steps before and after the *predictive analytics*.

We try to think of the Machine Learning process as:

Formulating a Question

Finding and Understanding the Data

Cleaning the Data and Feature Engineering

Choosing a Model

Tuning and Evaluating

Using the Model and Presenting Results

1. Formulating a Question

What is it that we want to find out? How will we reach the success criteria that we set?

Let's say we are performing machine learning for a high-traffic fast-casual restaurant chain, and our goal is to improve the customer experience. We can serve this goal in many ways. When we're thinking about creating a model, we have to narrow down to one measurable, specific task. For example, we might say we want to predict the wait times for customers' food orders within 2 minutes, so that we can give them an accurate time estimate.

2. Finding and Understanding the Data

Arguably the largest chunk of time in any machine learning process is finding the relevant data to help answer your question, and getting it into the format necessary for performing predictive analysis.

We know that for supervised learning, we need *labeled* datasets, or datasets that have clear labels of what their ground truth is. For an example like the restaurant wait time, this would mean we would need many examples of past orders, tagged with how long the wait time was. Maybe the restaurant already tracks this data, but we might need to augment the data collection with a timer that starts when the customer orders, stops when the customer receives their food, and records that information.

Creating this system of recording data, as well as gathering enough data to be able to train our model will take time.

Once you have your data, you want to understand it so that you will know what model to apply and what the outputs will mean. First, you will want to examine the summary statistics:

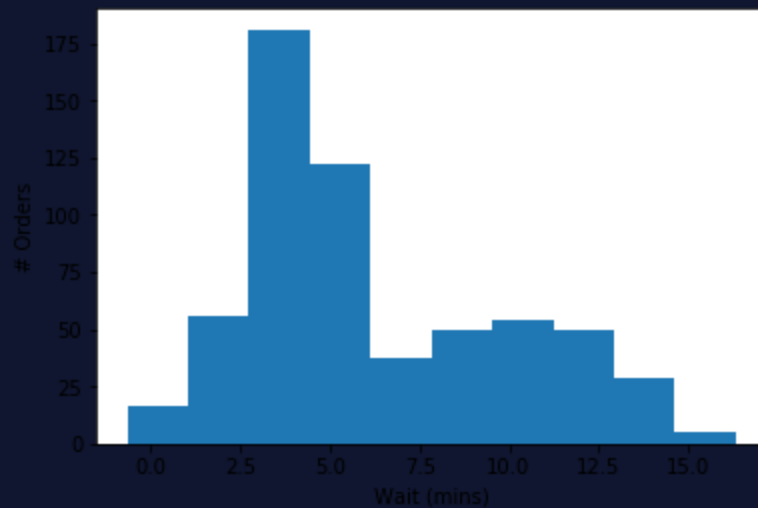
- Calculate means and medians to understand the distribution

- Calculate percentiles

- Find correlations that indicate relationships

You may also want to visualize the data, perhaps using box plots to identify outliers, histograms to show the basic structure of the data, and scatter plots to examine relationships between variables.

Let's say we're examining the existing distribution of wait times. We see that the overall average is 6.25 minutes per order. But we also produce this histogram:



We might glean from this that there are two main groups of orders. One group seems to cluster around 4 minutes, while another, smaller, group seems to cluster around 11 mins. We could use this to modify our question and build a model that will classify whether or not an order will be in this “short” timeframe, or in the “long” timeframe. Is it dependent on the food that it ordered? The time of day of the order?

Perhaps we just become aware of the bimodality of our data. If our model consistently predicts a wait time of around 6 or 7 minutes, then we are not taking into account the true structure of our data.

3. Cleaning the Data and Feature Engineering

Real data is messy! Data may have errors. Some columns may be empty. The features we’re interested in might require string manipulation to extract. Cleaning the data refers to the process by which we address missing values and outliers, among other things that may affect our insights.

We may see that we have a group of orders that took over 20 minutes, due to an emergency in the kitchen one afternoon. This is pushing our average wait time up, and may skew our predictions. If we want to model the more general functioning of the restaurant, we may want to remove these values.

Feature Engineering refers to the process by which we choose the important features (or columns) to look at, and make the appropriate transformations to prepare our data for our model.

We might try:

- Normalizing or standardizing the data

- Augmenting the data by adding new columns

- Removing unnecessary columns

After we test our model on the data we have, we might go back and reengineer features to see if we get a better result.

4. Choosing a Model

Once we understand our dataset and know the problem we are trying to solve, we can begin to choose a model that will help us tackle our problem.

If we are attempting to find a continuous output, like predicting the number of minutes someone should wait for their order, we would use a regression algorithm.

If we are attempting to classify an input, like determining if an order will take under 5 minutes or over 10 mins, then we would use a classification algorithm.

The different classification and regression algorithms work better on different types of datasets. We use different models on categorical and numerical data, and different models on datasets with many features and datasets with few features. Our models also have different levels of interpretability — how easy is it for us to see what these results mean and what led to them? When we teach the models, we will discuss the tradeoffs of using each one.

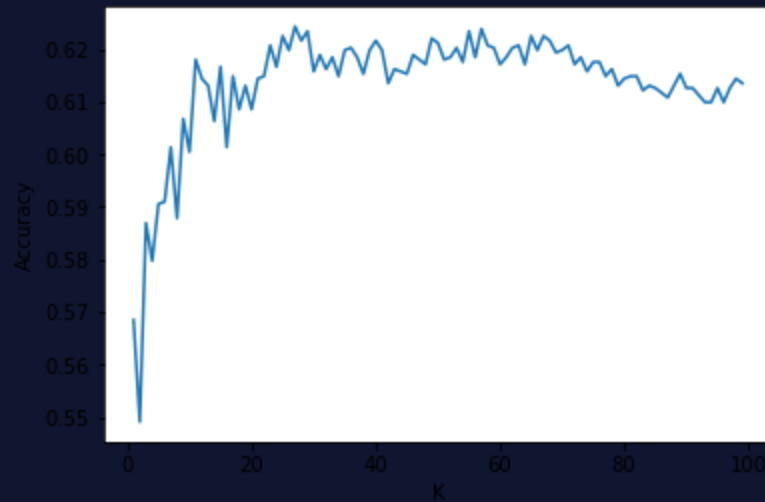
5. Tuning and Evaluating

We often want to set a metric of success, so that we know the model we've chosen is good enough. Are we looking for accuracy? Precision? Some combination of the two? We discuss this in our lesson on Precision and Accuracy.

Each model has a variety of parameters that change how it makes decisions. We can adjust these and compare the chosen evaluation metrics of the different variants to find the most accurate model.

For example, let's say we're using a K-Nearest Neighbors regression algorithm to solve the wait time prediction problem. This algorithm uses a parameter k , which you will learn about in the KNN lesson. We can adjust k to get different results.

Is it ideal to compare against 3 nearest neighbors? 10? 1? We can try many different values of k and see which one gives us the highest level of accuracy:



From this analysis, we would set our k to be 26, which got the highest level of accuracy.

6. Using the Model and Presenting Results

When you achieve the level of accuracy you want on your training set, you can use the model on the data you actually care about analyzing.

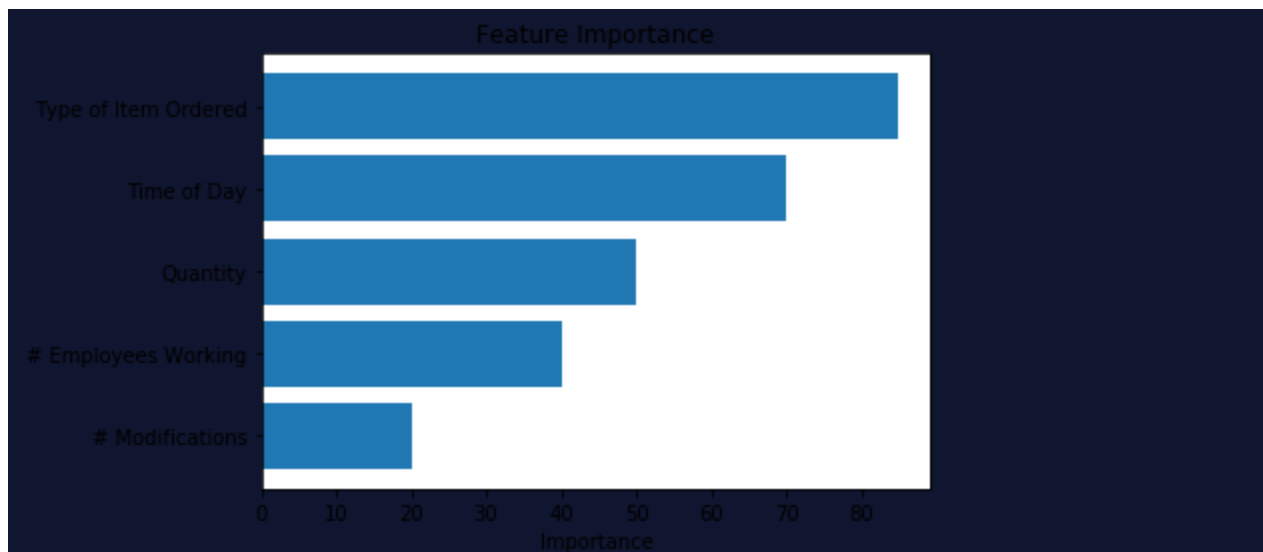
For our example, we can now start inputting new orders. The input could be an order, with features like:

- the type of item ordered
- the quantity
- the time of day
- the number of employees working

The output would be how long the order is expected to take. This information could be displayed to users.

An important step is being able to convey what you've learned and created, so that people can use it in the future.

Sometimes you learn more about your data by looking at the model. For example, using Multiple Learning Regression can give you insights into the importance of each feature. We can create a feature importance graph to visualize this for those unfamiliar with our model:



Your Process

The process we have outlined is a fairly standard process for performing machine learning. As you get experience going through this process on your own, with your own problems, you will start to form your own process. The steps may not be linear! As you clean your data, you may uncover a better question to ask. As you tune your model, you may realize you need more data, and go back to the collection step.

The important part is to stay curious, and to keep iterating until you find a model that works the best!

The 10 types of clustering that all data scientists need to know.

Let's dive in:

1. K-Means Clustering:

This is a centroid-based algorithm, where the goal is to minimize the sum of distances between points and their respective cluster centroid.

2. Hierarchical Clustering:

This method creates a tree of clusters. It is subdivided into Agglomerative (bottom-up approach) and Divisive (top-down approach).

3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

This algorithm defines clusters as areas of high density separated by areas of low density.

4. Mean Shift Clustering:

It is a centroid-based algorithm, which updates candidates for centroids to be the mean of points within a given region.

5. Gaussian Mixture Models (GMM):

This method uses a probabilistic model to represent the presence of subpopulations within an overall population without requiring to assign each data point to a cluster.

6. Spectral Clustering:

It uses the eigenvalues of a similarity matrix to reduce dimensionality before applying a clustering algorithm, typically K-means.

7. OPTICS (Ordering Points To Identify the Clustering Structure):

Similar to DBSCAN, but creates a reachability plot to determine clustering structure.

8. Affinity Propagation:

It sends messages between pairs of samples until a set of exemplars and corresponding clusters gradually emerges.

9. BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies):

Designed for large datasets, it incrementally and dynamically clusters incoming multi-dimensional metric data points.

10. CURE (Clustering Using Representatives):

It identifies clusters by shrinking each cluster to a certain number of representative points rather than the centroid.

EVERY DATA SCIENTIST NEEDS TO LEARN AI IN 2025.

- 99% of data scientists are overlooking AI.
- I want to help.
- Want to become a Generative AI Data Scientist in 2025?
- On Wednesday, July 9th, I'm sharing one of my best AI Projects
- How I built an AI Customer Segmentation Agent with Python:
 - - Scikit Learn
 - - LangChain
 - - OpenAI

Learn the general structure of how to approach Machine Learning problems in a methodical way.

The Process

When people think of Machine Learning, they often think of a program that is taking in data and spitting out predictions and insights. The process of performing Machine Learning often requires many more steps before and after the predictive analytics.

We try to think of the Machine Learning process as:

- Formulating a Question
- Finding and Understanding the Data
- Cleaning the Data and Feature Engineering
- Choosing a Model
- Tuning and Evaluating
- Using the Model and Presenting Results

Scikit-Learn Cheatsheet

Explore key Scikit-Learn commands for machine learning, covering regression, classification, clustering, and model validation in Python.



[Scikit-learn](#) is a library in Python that provides many unsupervised and supervised learning algorithms. It's built upon some of the technology you might already be familiar with, like NumPy, pandas, and Matplotlib!

As you build robust Machine Learning programs, it's helpful to have all the `sklearn` commands all in one place in case you forget.

Linear regression in Scikit-Learn

Import and create the model:

```
from sklearn.linear_model import LinearRegression  
  
your_model = LinearRegression()
```

Fit:


```
your_model.fit(x_training_data, y_training_data)
```

`.coef_`: contains the coefficients
`.intercept_`: contains the intercept

Predict:

```
predictions = your_model.predict(your_x_data)
```

`.score()`: returns the coefficient of determination R^2

Naive Bayes classification in Scikit-Learn

Import and create the model:

```
from sklearn.naive_bayes import MultinomialNB  
  
your_model = MultinomialNB()
```

Fit:

```
your_model.fit(x_training_data, y_training_data)
```

Predict:

```
# Returns a list of predicted classes - one prediction for every data point  
predictions = your_model.predict(your_x_data)  
  
# For every data point, returns a list of probabilities of each class  
probabilities = your_model.predict_proba(your_x_data)
```

K-nearest neighbors (KNN) in Scikit-Learn

Import and create the model:

```
from sklearn.neighbors import KNeighborsClassifier  
  
your_model = KNeighborsClassifier()
```

Fit:

```
your_model.fit(x_training_data, y_training_data)
```

Predict:

```
# Returns a list of predicted classes - one prediction for every data point  
predictions = your_model.predict(your_x_data)  
  
# For every data point, returns a list of probabilities of each class  
probabilities = your_model.predict_proba(your_x_data)
```

K-means clustering in Scikit-Learn

Import and create the model:

```
from sklearn.cluster import KMeans

your_model = KMeans(n_clusters=4, init='random')
```

`n_clusters`: number of clusters to form and number of centroids to generate
`init`: method for initialization
 k-means++: K-Means++ [default]
 random: K-Means
`random_state`: the seed used by the random number generator [optional]

Fit:

```
your_model.fit(x_training_data)
```

Predict:

```
predictions = your_model.predict(your_x_data)
```

Validating a machine learning model

Import and print accuracy, recall, precision, and F1 score:

```
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score

print(accuracy_score(true_labels, guesses))
print(recall_score(true_labels, guesses))
print(precision_score(true_labels, guesses))
print(f1_score(true_labels, guesses))
```

Import and print the confusion matrix:

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(true_labels, guesses))
```

Splitting data into training and test sets

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8,
test_size=0.2)
```

`train_size`: the proportion of the dataset to include in the train split
`test_size`: the proportion of the dataset to include in the test split
`random_state`: the seed used by the random number generator [optional]

Conclusion

Scikit-Learn provides a powerful and user-friendly framework for implementing machine learning models in Python. From regression and classification to clustering and model validation, it simplifies complex tasks with efficient built-in functions. Whether you're training models, making predictions, or evaluating performance, Scikit-Learn equips you with the tools needed to build robust machine learning applications.

If you're interested in learning more about Scikit-Learn and its applications in machine learning, please check out our [AI Catalog of articles!](#)

Virtual Environment

Pipenv

- <https://www.codecademy.com/paths/data-science/tracks/dsml-python-fundamentals-part-iii/modules/dsml-python-modules/videos/learn-python3-pipenv>
- https://www.youtube.com/watch?v=BVr-6Ki96XM&ab_channel=Codecademy

SETUP VIRTUAL ENRONMENT PYTHON - PIPENV ON WINDOWS PC

```
> cmd
> cd dir_path #setup directory path
> pip --version
> pip install --user pipenv #install in user level, not global
> pipenv
= warning? The scripts pipenv-resolver.exe and pipenv.exe are installed in 'dir_path/Python/Python37/Scripts' which
is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

--> copy this: dir_path/Python/Python37/Scripts
      then go to > windows > type environment > edit the system environment variables > environment variables >
path > edit > new > paste > save>save>save

> pipenv --version

It is DONE
NOW OPEN ANY editor - PyCharm, Visual Studio, ATOM, etc. and start working in
Create or go to directory
> cd dir_path
> pipenv --three #python 3
> pipenv install numpy #ny version of numpy --> numpy = "*" but install the latest version of module
> pipenv install requests==2.18.1
```

https://www.youtube.com/watch?time_continue=391&v=BVr-6Ki96XM&embeds_referring_euri=https%3A%2F%2Fwww.codecademy.com%2F&embeds_referring_origin=https%3A%2F%2Fwww.codecademy.com&source_ve_path=Mjq2NjY

SETUP VIRTUAL ENRONMENT PYTHON - PIPENV ON MAC PC

```
> cmd #command line
> pip --version
> pip install --user pipenv
```

/* run pipenv -> if says like: ...C:/../bin which is not on PATH. consider adding directory PATH

```
If >  
> pipenv > command not found
```

```
Then >  
> vi ~/.bash_profile > enter button
```

```
ADD TO open Notebook  
export PATH="/Users/file_directory/Library/3.7/bin:$PATH"
```

```
AT the end > write
```

```
> :wq! > enter_button
```

```
> more ~/.bash_profile
```

```
RESTART CMD
```

```
>pipenv --version
```

Introduction: Linear Algebra

Find out what you'll learn about linear algebra and why it's important.

Welcome to Linear Algebra

In this unit, we will cover fundamental linear algebra operations for vectors and matrices—including how to represent and solve linear systems of equations.

Why is this important?

Vectors and matrices are fundamental building blocks of data science as they are used to organize and manipulate data we feed through learning models. Whether you are just starting your journey in data science or have already played around with machine learning models, learning linear algebra fundamentals is key to your success in understanding advanced data science material.

There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

Efficient computation of linear transformation on large quantities of data. Computer programs and hardware like GPUs have been optimized around these linear algebra data structures to enable much of the high-performance computing advances.

Computer graphics, especially video games, rely heavily on linear algebra to render images quickly and efficiently.

Machine learning and advances in deep learning, whose foundations are matrix and vector multiplication and addition.

What will you learn?

At the end of this module, you will be able to:

Perform basic mathematical operations between scalars, vectors, and matrices.

Use vectors and matrices to represent and solve linear systems of equations.

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Linear Algebra?

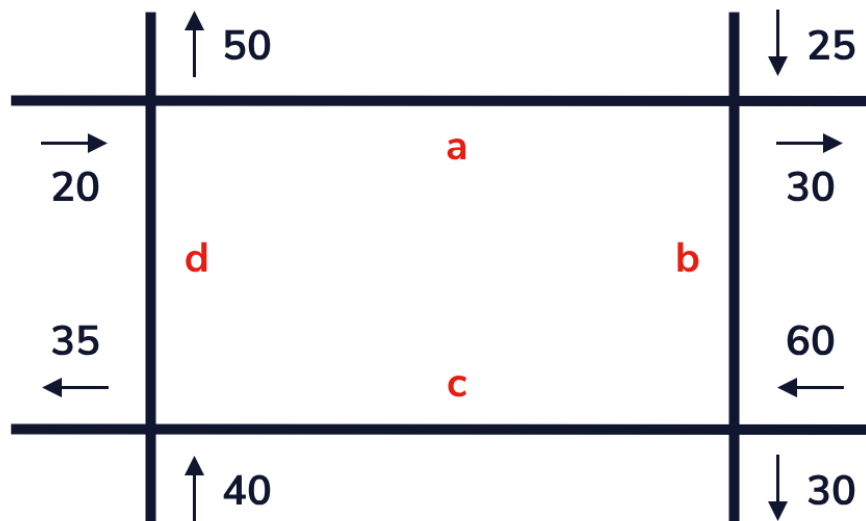
Learn about what linear algebra is and how it is relevant for data science!

Linear algebra focuses on the mathematics surrounding linear operations and solving systems of linear equations. While most of us learned about basic linear operations in algebra classes in high school and middle school, "linear algebra" extends these lessons to apply to multi-dimensional data.

The reality is that many engineering optimization problems can be reduced to systems of linear equations, albeit systems with many equations and many variables. By simplifying problems, we can solve for a solution or set of solutions.

Let's imagine a problem where we can do this. We have a section of town containing four roads and four intersections. We are trying to find the number of cars along the roads in between the intersections and have data about the number of cars departing intersections outside this road system. How can we find out our missing car counts?

To solve the problem mathematically, we can realize that this system can be modeled as a system of linear equations since the number of cars entering an intersection must equal the number leaving an intersection. We can visualize the data on the diagram below.



This creates a straightforward system, written out below— one that we can solve without linear algebra techniques.

$$20+d=50+a \quad 25+a=30+b \quad 60+b=30+c \quad 40+c=35+d$$

2

Now, what if we extend this problem to the entire city of Manhattan? And now only provide inconsistent intersection counts, requiring us to solve for many unknown road segment car counts? In this case, we can still set the problem up as an extensive system of linear equations, but we quickly realize that solving for those unknown car counts by hand is impossible. This is where linear algebra will come in and allow us to solve these complex problems.

Coming Up

In the upcoming content, we will look at two important linear algebra data structures that can store high-dimensional data. This data can be actual numerical data like one might see on a spreadsheet or representative data like if we need to represent a large system of equations. We will learn how to perform various operations between these data structures and use these techniques to solve systems of linear equations.

As mentioned earlier, linear algebra is fundamental because it allows us to mathematically operate on large amounts of data, which is vital to modern data science techniques. There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

- Solving systems of linear equations, especially those with many variables and cannot be reasonably solved by hand.

- Efficient computation of linear transformation on large quantities of data. Throughout this lesson, we will go into great detail on data structures that hold large amounts of data (vectors and matrices) and their various operations that allow us to apply linear transforms. Computer programs and hardware like GPU's have been optimized around these linear algebra data structures to enable much of the high-performance computing advances.

- Computer graphics, especially video games, rely heavily on linear algebra to render images quickly and efficiently.

- Machine learning and advances in deep learning, whose foundations are matrix and vector multiplication and addition.

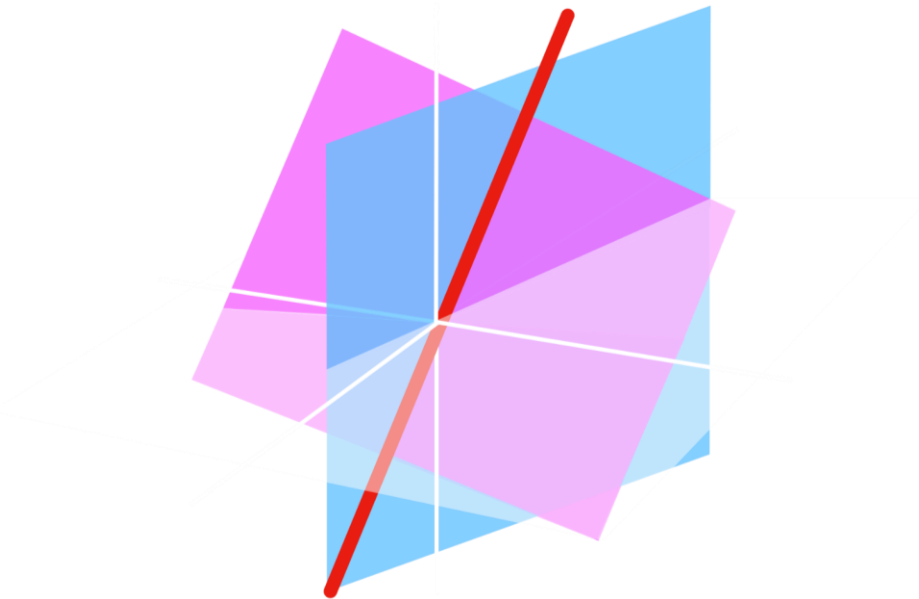
Introduction

Linear algebra is a branch of mathematics that studies linear transformations, including the use of vectors and matrices. Linear algebra enables us to solve problems like linear regression, as well as computational problems that concern large amounts of data (machine learning, computer graphics, etc.).

In this lesson, we will learn about the following topics:

- Vectors and their basic operations
- The dot product between vectors
- Matrices and their basic operations
- Matrix multiplication and special matrices
- Gauss-Jordan Elimination to solve systems of linear equations
- Gauss-Jordan Elimination to solve for inverse matrices

These topics will give us a solid foundation in linear algebra, providing a strong base to solve more complicated problems like linear regression in future content.



Vectors

The fundamental building blocks of linear algebra are *vectors*. Vectors are defined as quantities having both direction and magnitude, compared to *scalar* quantities that only have magnitude. In order to have direction and magnitude, vector quantities consist of two or more elements of data. The *dimensionality* of a vector is determined by the number of numerical elements in that vector. For example, a vector with four elements would have a dimensionality of four.

Let's take a look at examples of a scalar versus a vector. A car driving at a speed of 40mph is a scalar quantity. Describing the car driving 40mph to the east would represent a two-dimensional vector quantity since it has a magnitude in both the x and y directions.

Vectors can be represented as a series of numbers enclosed in parentheses, angle brackets, or square brackets. In this lesson, we will use square brackets for consistency. For example, a three-dimensional vector is written as:

$$\mathbf{v}$$

The magnitude (or length) of a vector, $\|\mathbf{v}\|$, can be calculated with the following formula:

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$$

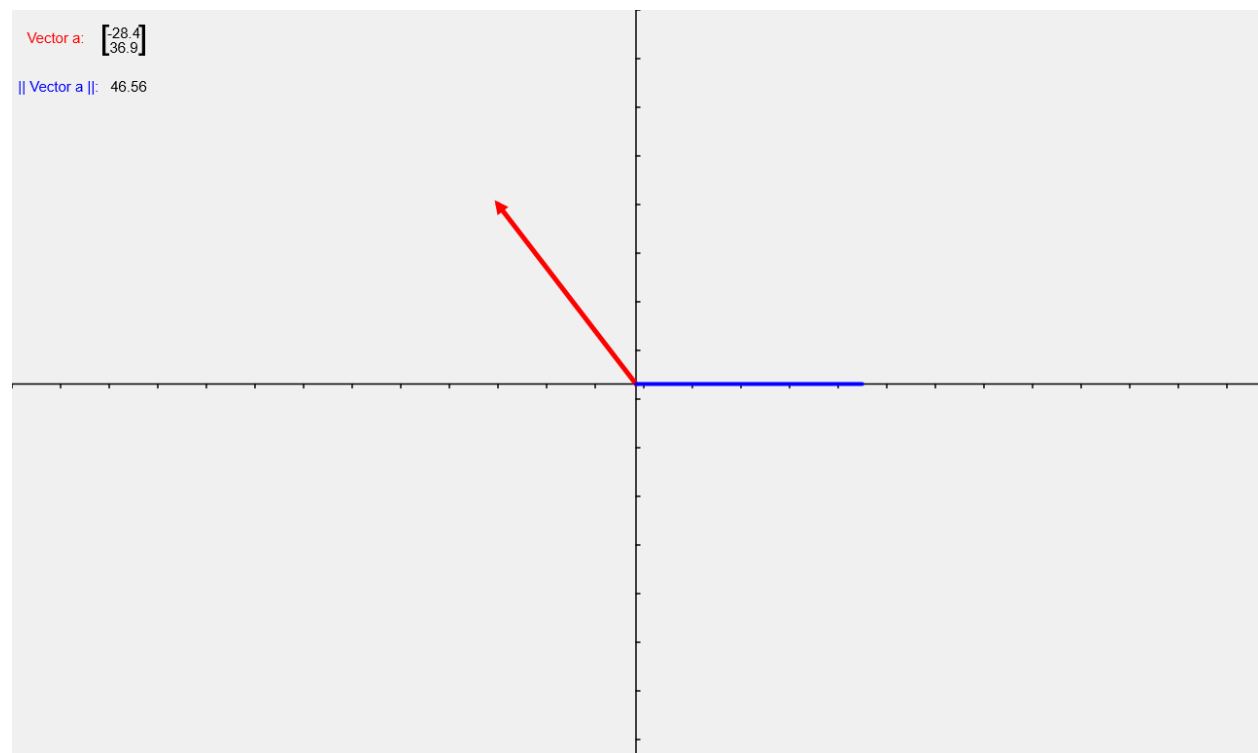
This formula translates to the sum of each vector component squared, which can be also written out as:

$$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

Let's look at an example. We are told that a ball is traveling through the air and given the velocities of that ball in its x , y , and z directions in a standard Cartesian coordinate system. The velocity component values are:

$$\begin{aligned} x &= -12 \\ y &= 8 \\ z &= -2 \end{aligned}$$

Convert the velocities into a vector, and find the total speed of the ball. (Hint: the speed of the ball is the magnitude of the velocity vector!)



Basic Vector Operations

Now that we know what a vector is and how to represent one, we can begin to perform various operations on vectors and between different vectors. As we've previously discussed, the basis of linear algebra is the linear combinations we can generate between vectors (and matrices).

Scalar Multiplication

Any vector can be multiplied by a scalar, which results in every element of that vector being multiplied by that scalar individually.

$$k \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kz \end{bmatrix}$$

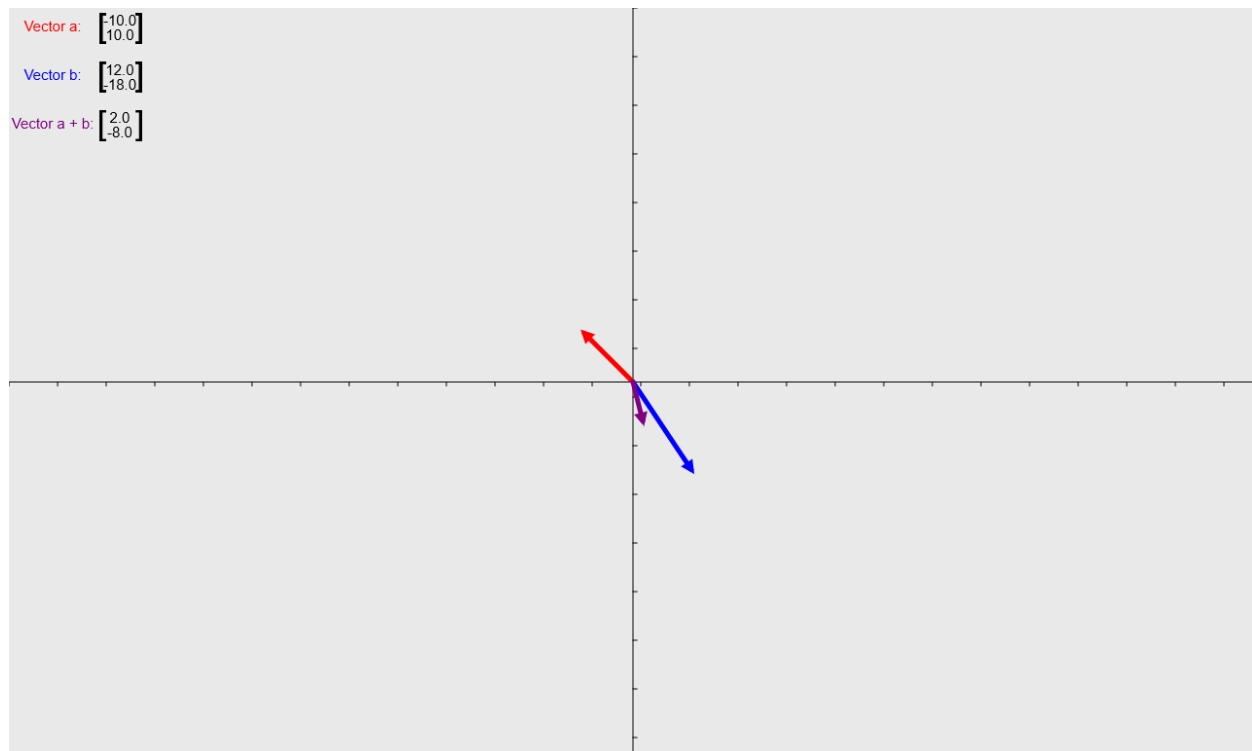
Multiplying vectors by scalars is an *associative operation*, meaning that rearranging the parentheses in the expression does not change the result. For example, we can say $2(a3) = (2a)3$.

Vector Addition and Subtraction

Vectors can be added and subtracted from each other when they are of the same dimension (same number of components). Doing so adds or subtracts corresponding elements, resulting in a new vector of the same dimension as the two being summed or subtracted. Below is an example of three-dimensional vectors being added and subtracted together.

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} + 2 \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix} - 3 \begin{bmatrix} x_3 \\ y_3 \\ z_3 \end{bmatrix} = \begin{bmatrix} x_1 + 2x_2 - 3x_3 \\ y_1 + 2y_2 - 3y_3 \\ z_1 + 2z_2 - 3z_3 \end{bmatrix}$$

Vector addition is *commutative*, meaning the order of the terms does not matter. For example, we can say $(a+b = b+a)$. Vector addition is also *associative*, meaning that $(a + (b+c) = (a+b) + c)$.



Vector Dot Products

An important vector operation in linear algebra is the *dot product*. A dot product takes two equal dimension vectors and returns a single scalar value by summing the products of the vectors' corresponding components. This can be written out formulaically as:

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

The dot product operation is both commutative ($a \cdot b = b \cdot a$) and distributive ($a \cdot (b+c) = a \cdot b + a \cdot c$).

The resulting scalar value represents how much one vector "goes into" the other vector. If two vectors are perpendicular (or orthogonal), their dot product is equal to 0, as neither vector "goes into the other."

Let's take a look at an example dot product. Consider the following two vectors:

$$a = \begin{bmatrix} 3 \\ 2 \\ -3 \end{bmatrix}, b = \begin{bmatrix} 0 \\ -3 \\ -6 \end{bmatrix}$$

To find the dot product between these two vectors, we do the following:

$$a \cdot b = 3 * 0 + 2 * -3 + -3 * -6 = 12$$

The dot product can also be used to find the magnitude of a vector and the angle between two vectors. To find the magnitude, we can reference Exercise 2 and see that the magnitude of a vector is simply the square root of a vector's dot product with itself.

$$||a|| = (a \cdot a)^{1/2}$$

$$||a|| = \sqrt{}$$

To find the angle between two vectors, we rely on the dot product between the two vectors and use the following equation.

$$\theta = \arccos \frac{a \cdot b}{||a|| ||b||}$$

Let's look at the same two vectors from above:

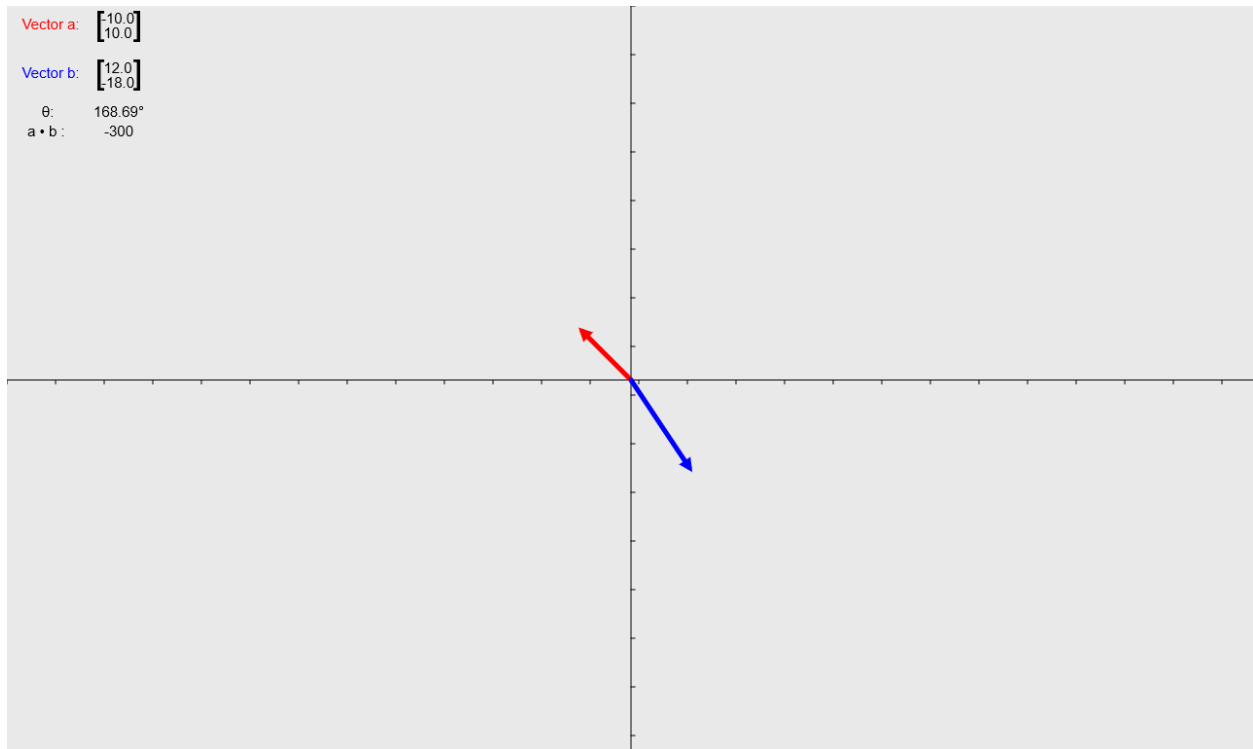
$$a = \begin{bmatrix} 3 \\ 2 \\ -3 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ -3 \\ -6 \end{bmatrix}$$

To find the angle between these two vectors, we do the following:

$$\theta = \arccos \frac{3 * 0 + 2 * -3 + -3 * -6}{\sqrt{(3)^2 + (2)^2 + (-3)^2} * \sqrt{(0)^2 + (-3)^2 + (-6)^2}}$$

Solving this, we end up with:

$$\theta = 67.58^\circ$$



Matrices

Now that we've learned about vector quantities, we can expand upon those lessons and focus on matrices. A *matrix* is a quantity with m rows and n columns of data. For example, we can combine multiple vectors into a matrix where each column of that matrix is one of the vectors.

To the right, you can see a comparison between scalars, vectors, and matrices for context.

We can also think of vectors as single-column matrices in their own right. Matrices are helpful because they allow us to perform operations on large amounts of data, such as representing entire systems of equations in a single matrix quantity.

Matrices can be represented by using square brackets that enclose the rows and columns of data (elements). The shape of a matrix is said to be $m \times n$, where there are m rows and n columns. When representing matrices as a variable, we denote the matrix with a capital letter and a particular matrix element as the matrix variable with an " m, n " determined by the element's location. Let's look at an example of this. Consider the matrix below.

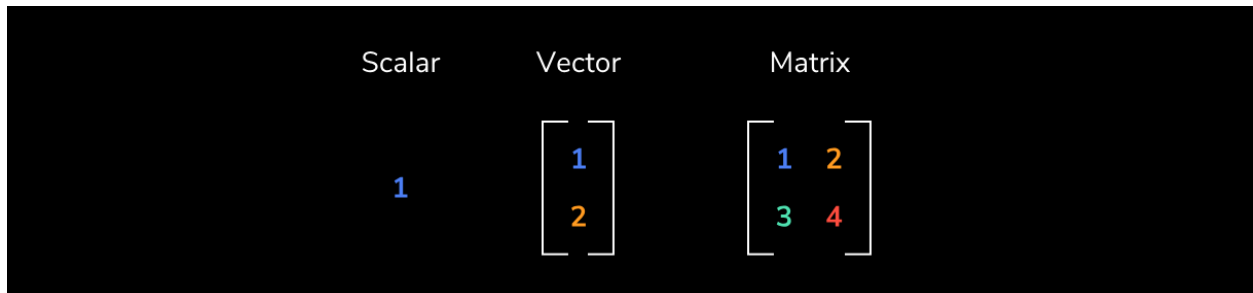
$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

The value corresponding to the first row and second column is b .

$$A_{1,2} = b$$

What is the value corresponding to the second row and third column?

$$A_{2,3} = f$$



Matrix Operations

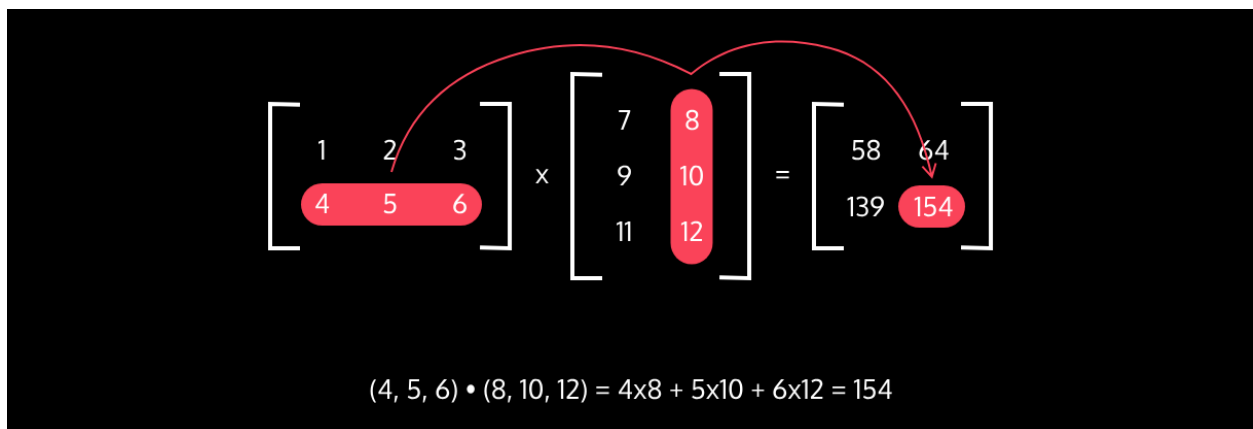
Like with vectors, there are fundamental operations we can perform on matrices that enable the linear transformations needed for linear algebra. We can again both multiply entire matrices by a scalar value, as well as add or subtract matrices with equal shapes.

$$2 \begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix} + 3 \begin{bmatrix} c_1 & d_1 \\ c_2 & d_2 \end{bmatrix} = \begin{bmatrix} 2a_1 + 3c_1 & 2b_1 + 3d_1 \\ 2a_2 + 3c_2 & 2b_2 + 3d_2 \end{bmatrix}$$

A new and important operation we can now perform is matrix multiplication. Matrix multiplication works by computing the dot product between each row of the first matrix and each column of the second matrix. For example, in computing $AB = C$, element (1,2) of the matrix product C will be the dot product of row 1 of matrix A and column 2 of matrix B . The animation to the right walks through the multiplication of two matrices showing the dot product between each row and column.

An important rule about matrix multiplication is that the shapes of the two matrices AB must be such that the number of columns in A is equal to the number of rows in B . With this condition satisfied, the resulting matrix product will be the shape of $rowsA \times columnsB$. For example, in the animation to the right, the product of a 2x3 matrix and a 3x2 matrix ends up being a 2x2 matrix.

Based on how we compute the product matrix with dot products and the importance of having correctly shaped matrices, we can see that matrix multiplication is not commutative, $AB \neq BA$. However, we can also see that matrix multiplication is associative, $A(BC) = (AB)C$.



Identity matrix

The *identity matrix* is a square matrix of elements equal to 0 except for the elements along the diagonal that are equal to 1. Any matrix multiplied by the identity matrix, either on the left or right side, will be equal to itself.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transpose matrix

The *transpose* of a matrix is computed by swapping the rows and columns of a matrix. The transpose operation is denoted by a superscript uppercase "T" (A^T).

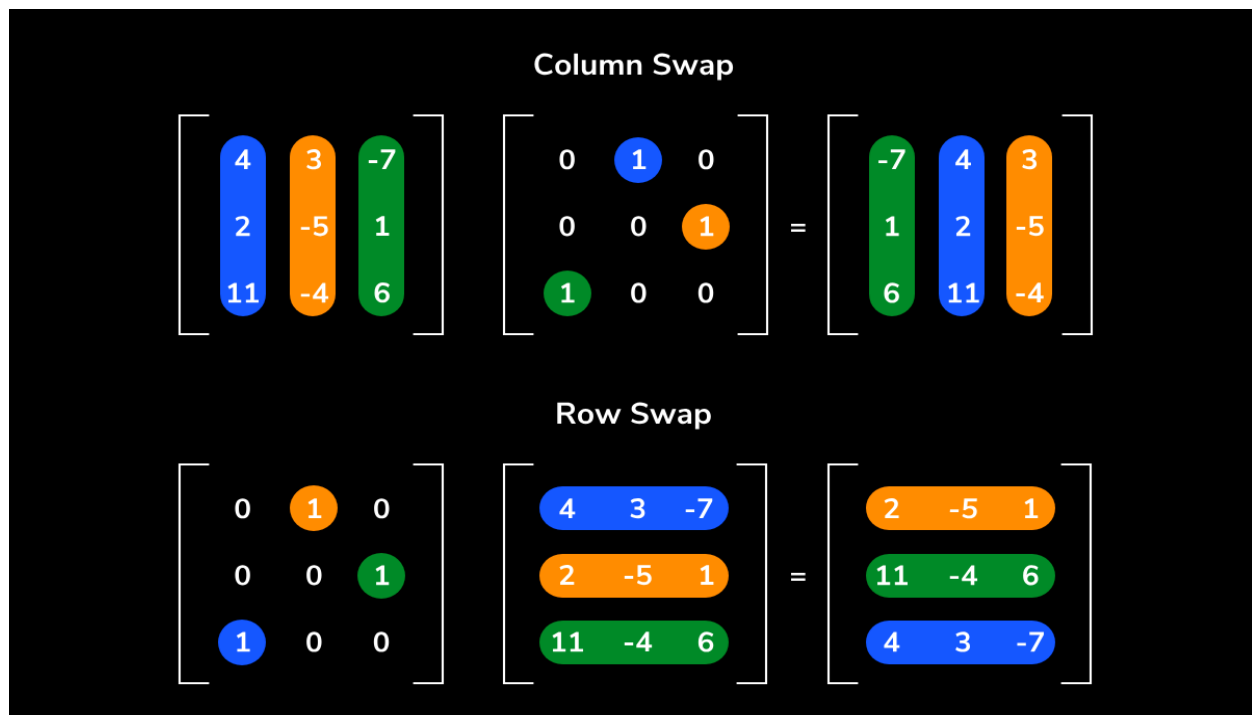
$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}^T = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}$$

Permutation Matrix

A *permutation matrix* is a square matrix that allows us to flip rows and columns of a separate matrix. Similar to the identity matrix, a permutation matrix is made of elements equal to 0, except for one element in each row or column that is equal to 1. In order to flip rows in matrix A , we multiply a permutation matrix P on the left (PA). To flip columns, we multiply a permutation matrix P on the right (AP).

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \begin{bmatrix} d & e & f \\ g & h & i \\ a & b & c \end{bmatrix}$$

$$AP = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} c & a & b \\ f & d & e \\ i & g & h \end{bmatrix}$$



Linear Systems in Matrix Form

An extremely useful application of matrices is for solving systems of linear equations. Consider the following system of equations in its algebraic form.

$$\begin{aligned} a_1x + b_1y + c_1z &= d_1 \\ a_2x + b_2y + c_2z &= d_2 \\ a_3x + b_3y + c_3z &= d_3 \end{aligned}$$

This system of equations can be represented using vectors and their linear combination operations that we discussed in the previous exercise. We combine the coefficients of the same unknown variables, as well as the equation solutions, into vectors. These vectors are then scalar multiplied by their unknown variable and summed.

$$x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

Our final goal is going to be to represent this system in form $Ax = b$, using matrices and vectors. As we learned earlier, we can combine vectors to create a matrix, which we'll do with the coefficient vectors to create matrix A . We can also convert the unknown variables into vector x . We end up with the following $Ax = b$ form:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

Click below if you would like to see how these two expressions are equivalent:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Click below if you would like to see how these two expressions are equivalent:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

► Click to see proof

We can also write $Ax = b$ in the following form:

$$Ax = b \rightarrow [A \mid b]$$

This means we can write our $Ax = b$ equation above as:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} \rightarrow \begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$$

This is what is known as an *augmented matrix*, and we will use these in the following exercises to solve for linear systems!

Gauss-Jordan Elimination

Now that we have our system of linear equations in augmented matrix form, we can solve for the unknown [variables](#) using a technique called *Gauss-Jordan Elimination*. In regular algebra, we may try to solve the system by combining equations to eliminate variables until we can solve for a single one. Having one variable solved for then allows us to solve for a second variable, and we can continue that process until all variables are solved for.

The same goal can be used for solving for the unknown variables when in the matrix representation. We start with forming our augmented matrix.

$$\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$$

To solve for the system, we want to put our augmented matrix into something called *row echelon form* where all elements below the diagonal are equal to zero. This looks like the following:

$$\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & b'_1 & c'_1 & d'_1 \\ 0 & 1 & c'_2 & d'_2 \\ 0 & 0 & 1 & d'_3 \end{bmatrix}$$

Note that the values with apostrophes in the *row echelon form* matrix mean that they have been changed in the process of updating the matrix. Once in this form we can rewrite our original equation as:

$$\begin{aligned} x + b'_1 y + c'_1 z &= d'_1 \\ y + c'_2 z &= d'_2 \\ z &= d'_3 \end{aligned}$$

This allows us to solve the equations directly using simple algebra. But how do we get to this form?

To get to row echelon form we swap rows and/or add or subtract rows against other rows. A typical strategy is to add or subtract row 1 against all rows below in order to make all elements in column 1 equal to 0 under the diagonal. Once this is achieved, we can do the same with row 2 and all rows below to make all elements below the diagonal in column 2 equal to 0.

Once all elements below the diagonal are equal to 0, we can simply solve for the variable values, starting at the bottom of the matrix and working our way up.

It's important to realize that not all systems of equations are solvable! For example, we can perform Gauss-Jordan Elimination and end up with the following augmented matrix.

Inverse Matrices

The *inverse of a matrix*, A^{-1} , is one where the following equation is true:

$$AA^{-1} = A^{-1}A = I$$

This means that the product of a matrix and its inverse (in either order) is equal to the identity matrix.

The inverse matrix is useful in many contexts, one of which includes solving equations with matrices. Imagine we have the following equation:

$$xA = BC$$

If we are trying to solve for x , we need to find a way to isolate that variable. Since we cannot divide matrices, we can multiply both right sides of the equation by the inverse of A , resulting in the following:

$$xAA^{-1} = BCA^{-1} \rightarrow x = BCA^{-1}$$

An important consideration to keep in mind is that not all matrices have inverses. Those matrices that do not have an inverse are referred to as *singular matrices*.

To find the inverse matrix, we can again use Gauss-Jordan elimination. Knowing that $AA^{-1} = I$, we can create the augmented matrix $[A \mid I]$, where we attempt to perform row operations such that $[A \mid I] \rightarrow [I \mid A^{-1}]$.

One method to find the necessary row operations to convert $A \rightarrow I$ is to start by creating an upper triangular matrix first, and then work on converting the elements above the diagonal afterward (starting from the bottom right side of the matrix). If the matrix is invertible, this should lead to a matrix with elements equal to 0 except along the diagonal. Each row can then be multiplied by a scalar that leads to the diagonal elements equaling 1 to create the identity matrix.

Cancel out the first entry in row 2

$R_2 - R_1 \rightarrow$

-1	1	2	0	0	1
0	-3	-2	0	1	-1
0	2	1	1	0	0

Using NumPy Arrays

The core of using NumPy effectively for linear algebra is using NumPy arrays. NumPy arrays are n-dimensional

Preview: Docs Loading link description

[array](#)

Preview: Docs Loading link description

[data structures](#)

that can be used to represent both vectors (1-dimensional array) and matrices (2-dimensional arrays).

A NumPy array is initialized using the `np.array()` function, and including a Python list

Preview: Docs Loading link description

[argument](#)

or Python nested list argument for arrays with more than one dimension.

For example, the following creates a NumPy array representation of a vector:

```
v = np.array([1, 2, 3, 4, 5, 6])
```

Copy to Clipboard

We can also create a matrix, which is the equivalent of a two-dimensional NumPy array, using a nested Python list:

```
A = np.array([[1,2],[3,4]])
```

Copy to Clipboard

If we print this NumPy array, it will look like the following:

```
[[1 2]
 [3 4]]
```

Copy to Clipboard

Matrices can also be created by combining existing vectors using the `np.column_stack()` function:

```
v = np.array([-2,-2,-2,-2])
u = np.array([0,0,0,0])
w = np.array([3,3,3,3])

A = np.column_stack((v, u, w))
print(A)
```

Copy to Clipboard

Output:

```
[[ -2  0  3]
 [ -2  0  3]
 [ -2  0  3]
 [ -2  0  3]]
```

Copy to Clipboard

To access the shape of a matrix or vector once it's been created as a NumPy array, we call the `.shape` attribute of the array variable:

```
A = np.array([[1,2],[3,4]])
print(A.shape)
```

Copy to Clipboard

Here, the output will be:

```
(2, 2)
```

Copy to Clipboard

since there are two rows and two columns.

To access individual elements in a NumPy array, we can

Preview: Docs Loading link description

[index](#)

the array using square brackets. Unlike regular Python lists, we can index into all dimensions in a single square bracket, separating the dimension indices with commas.

Thus in order to index the element equal to 2 in matrix A, we can do the following:

```
A = np.array([[1,2],[3,4]])
print(A[0,1])
```

Copy to Clipboard

The first index accesses the specific row, while the second index accesses the specific column. Note that rows and columns are zero-indexed. In this example, the output will be the following:

```
2
```

Copy to Clipboard

We can also select a subset or entire dimension of a NumPy array using a colon. For example, if we want the entire second column of a matrix, we can index the second column and use an empty colon to select every row as such:

```
A = np.array([[1,2],[3,4]])
print(A[:,1])
```

Copy to Clipboard

Output:

```
[2 4]
```

Copy to Clipboard

Note: `[2 4]` is a column vector, but it outputs in the

Preview: Docs A terminal is a command-line interface (CLI) that allows users to interact with a computer's operating system by executing text-based commands.

[terminal](#)

horizontally.

```
import numpy as np

# Given vectors
vector_1 = np.array([-2,-6,2,3])
vector_2 = np.array([4,1,-3,8])
vector_3 = np.array([5,-7,9,0])
```

```
import numpy as np

# Given vectors
vector_1 = np.array([-2,-6,2,3])
vector_2 = np.array([4,1,-3,8])
vector_3 = np.array([5,-7,9,0])

# Answer 1
matrix = np.column_stack((vector_1, vector_2, vector_3))
print(matrix)
```

```
# Answer 2
print(matrix.shape)

# Answer 3
second_column = matrix[:, 2]
print(second_column)
```

Using NumPy for Linear Algebra Operations

Now that we know how to use NumPy arrays to create vectors and matrices, we can learn how to perform various linear algebra operations using Python.

To multiply a vector or matrix by a scalar, we use inbuilt Python multiplication between the NumPy [array](#) and the scalar:

```
A = np.array([[1,2],[3,4]])
4 * A
```

```
import numpy as np

# Given
# 2 x 3 matrix
A = np.array([[2,3,-4], [-2, 1, -3]])
# 2 x 3 matrix
B = np.array([[1,-1,4], [3,-3,3]])
# 3 x 2 matrix
C = np.array([[1, 2], [3, 4], [5, 6]])

# Calculate D = 4A - 2B
D = 4*A - 2*B
print(D)

# Calculate E = AC
E = A@C
print(E)

# Calculate F = CA
F = C@A
print(F)
```

Special Matrices

In addition to having built-in support for many linear algebra-related operations, Let's see how NumPy can create special matrices, such as the identity matrix.

An identity matrix can be constructed using the `np.eye()` functions, which takes an integer

Preview: Docs An argument is the actual value of a variable passed into a function.
[argument](#)
that determines the $n \times n$ size of the square identity matrix.

```
# 4x4 identity matrix
identity = np.eye(4)
```

Copy to Clipboard

In the output

Preview: Docs A terminal is a command-line interface (CLI) that allows users to interact with a computer's operating system by executing text-based commands.

[terminal](#)

,identity renders as:

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
```

Copy to Clipboard

A matrix or vector of all zeros can be constructed using the `np.zeros()` function, which takes in a

Preview: Docs Loading link description

[tuple](#)

argument for the shape of the NumPy

Preview: Docs Loading link description

[array](#)

filled with zeros.

```
# 5-element vector of zeros
zero_vector = np.zeros((5))
```

```
import numpy as np

# Given
A = np.array([[1,-1,1], [0,1,0], [-1,2,1]])
B = np.array([[0.5,1.5,-0.5], [0,1,0], [0.5,-0.5,0.5]])

# Find the matrix product AB and BA, and determine what that product says about A and B.
print(np.matmul(A,B))
print(B@A)

# Find the transpose of matrix A and matrix B
print(A.T)
print(B.T)
```

Additional Linear Algebra Operations

Moving past special matrices, there are some more advanced linear algebra operations we can perform using NumPy. To start, we will be using the `numpy.linalg` sublibrary to perform the following operations:

The “norm” (or length/magnitude) of a vector can be found using `np.linalg.norm()`:

```
v = np.array([2,-4,1])
v_norm = np.linalg.norm(v)
```

Copy to Clipboard

This outputs `v_norm` as:

```
4.58257569495584
```

Copy to Clipboard

The inverse of a square matrix, if one exists, can be found using `np.linalg.inv()`:

```
A = np.array([[1,2],[3,4]])
print(np.linalg.inv(A))
```

Copy to Clipboard

This outputs:

```
[[-2.  1. ]
 [ 1.5 -0.5]]
```

Copy to Clipboard

Finally, we can actually solve for unknown

Preview: Docs Loading link description

[variables](#)

in a system on linear equations in $Ax=b$ form using `np.linalg.solve()`, which takes in the A and b parameters. Given:

```
x + 4y - z = -1
-x - 3y + 2z = 2
2x - y - 2z = -2
```

Copy to Clipboard

We convert to $Ax=b$ form and solve.

```
# each array in A is an equation from the above system of equations
A = np.array([[1,4,-1],[-1,-3,2],[2,-1,-2]])
# the solution to each equation
b = np.array([-1,2,-2])
# solve for x, y, and z
x,y,z = np.linalg.solve(A,b)
```

Copy to Clipboard

If we `print((x, y, z))`, we see the following output:

```
(0, 0, 1.0)
```

Copy to Clipboard

showing that the solution to the system of equations is:

```
x = 0
y = 0
z = 1
import numpy as np
```

```
# Represent the following system in NumPy matrix/vector form, then solve for x, y, and z

# Given
'''
4x + z = 2
-y + 2z - 3x = 0
.5y - x - 1.5z = -4
'''

# Answer 1
A = np.array([[4, 0, 1], [-3, -1, 2], [-1, 0.5, -1.5]])

# Answer 2
b = np.array([2, 0, -4])

# Answer 3
x,y,z = np.linalg.solve(A,b)
print((x,y,z))
```

```
(6.0, -62.0, -22.0)
```

Introduction: Differential Calculus

Find out what you'll learn about differential calculus and why it's important.

Welcome to Differential Calculus

In this unit, we will cover fundamentals of differential calculus and investigate how we can use limits and derivatives to analyze functions.

Why is this important?

Differential calculus deals with the study of rates of change. It allows us to quantify how one variable relates to another. In the field of data science, this is invaluable information to understand. Learning models are built upon quantifying how variables relate to each other to make predictions or understand patterns. Whether you are just starting on your data science journey or have worked with machine learning models, understanding the principles of calculus is an invaluable asset to add to your toolbelt!

What will you learn?

At the end of this module, you will be able to:

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#) and on [Discord](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Calculus?

Learn about what calculus is and how it is important in the field of data science.

The core task of a data scientist is to make data-driven recommendations on the best way to solve a problem. Along the way, data scientists build and fit statistical models to support their claims, make predictions, and more. In this article, we will look at what calculus is and why it is vital in the data science process.

Rate of Change

Differential calculus is the field that studies rates of change. By quantifying how variables relate to each other, we can better understand our statistical model or physical system. For example, as a data scientist at a t-shirt company, we may want to know how a change in the marketing budget impacts the company's total profit. Spend too little, and our product won't be well known; spend too much, and we might reach a point of diminishing returns where our money would be better spent elsewhere. How can we find the optimal size of our marketing budget?

Optimization

The last question above is the sort of questions asked and answered by the field of *mathematical optimization*, which intertwines closely with calculus. Optimization problems involve looking for the best solution to a problem. Perhaps that's finding the 'optimal' size of the marketing budget for our company, the point where we maximize our profits with a budget that allows us to reach a broad audience without paying too much. Perhaps that means finding the statistical model that 'best' fits our data according to some criteria that we define ourselves— mathematical optimization gives us a way to decide among all our possible options. It is a powerful field that underpins much of the modern machine learning revolution and it is built upon the fundamentals of calculus we will investigate.

Infinitesimal Analysis

One of the fundamental paradigms in calculus is to think via *infinitesimals* (extremely small quantities) to say things about instantaneous rates of change. We will discuss this at length in the upcoming lesson. To give a rough overview, however, the idea is that by 'zooming in' enough and considering how a variable changes over an infinitesimal time frame, we can learn how that variable is changing instantaneously, at a specific moment in time.

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE
EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

Limits

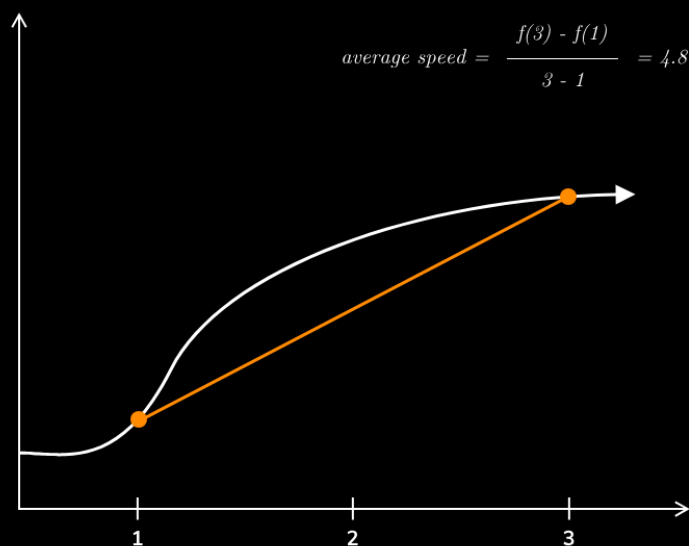
The first concept we will look at is something called a *limit*. Limits quantify what happens to the values of a function as we approach a given point. This can be defined notationally as:



Limit Definition of a Derivative

Suppose we wanted to measure a runner's instantaneous speed using a stopwatch. By instantaneous speed, we mean their speed at an exact moment in time.

Let's define $f(t)$ as the runner's distance from the start time at time t . How could we find the runner's instantaneous speed at 1 second after the race? We could record their positions at 1 second and some time after 1 second and then do some calculations. Let's say we measure their distance from the start at $f(1)$ and $f(3)$. The change in position is $f(3) - f(1)$, and the time between 1 second and 3 seconds is 2 seconds

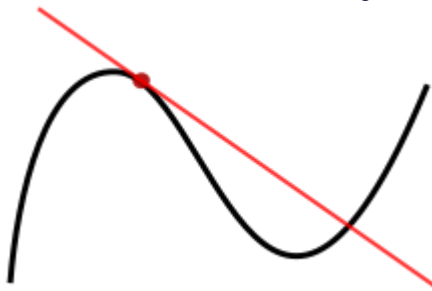


The Derivative Function

In the previous exercise, we discussed the idea of the derivative at a point as the instantaneous rate of change at that point. The idea behind the definition is that by finding how the function changes over increasingly small intervals starting at a point, we can understand how a function changes exactly at that point. The formal expression of this concept is written as:

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

An equivalent interpretation is that the derivative is the slope of the tangent line at a point. A *tangent line* at a point is the line that touches the function at that point. Just as the slope of a line describes how a line changes, the slope of the tangent line describes how a "curvier" function changes at a specific point. The visual below shows what a



tangent line looks like:



Properties of the Derivative Function

Derivative functions contain important information. Let's look at a couple of examples. Going back to our running example, $f(t)$ describes a runner's position as a function of time; therefore, the derivative function $f'(t)$ describes the runner's speed at any time t . Let's say we have a function $s(t)$ that describes the real-time sales of a particular product; the corresponding derivative function $s'(t)$ describes how those sales are changing in real-time.

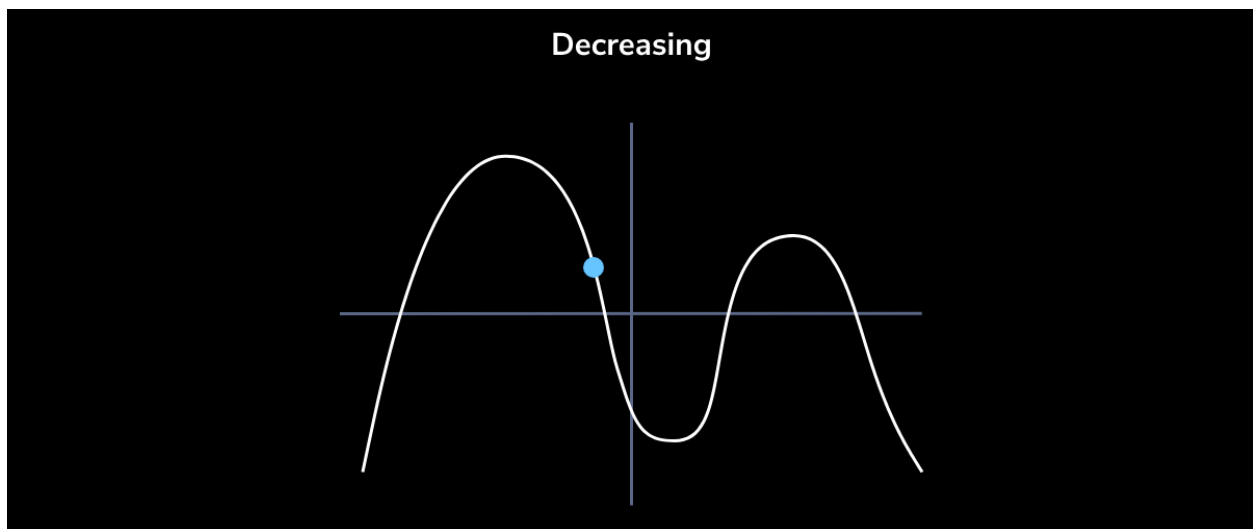
Let's look at the general information we can get as well. If $f'(x) > 0$, the corresponding function $f(x)$ is increasing. Keep in mind that $f(x)$ itself may be negative even though the derivative $f'(x)$ is positive. The derivative says nothing about the value of the original function, only about how it is changing. Similarly, if $f'(x) < 0$, then the original function is decreasing.

If $f'(x) = 0$, then the function is not changing. This can mean one of a few things.

It may mean that the function has reached a *local maximum* (or maxima). A local maximum is a value of x where $f'(x)$ changes from positive to negative and thus hits 0 along the way. In $f(x)$, the local maximum is higher than all the points around it.

It may also mean that the function has reached what is called a *local minimum*. A local minimum is lower than the points around it. When $f'(x)$ goes from negative values to 0 to positive values, a local minimum forms.

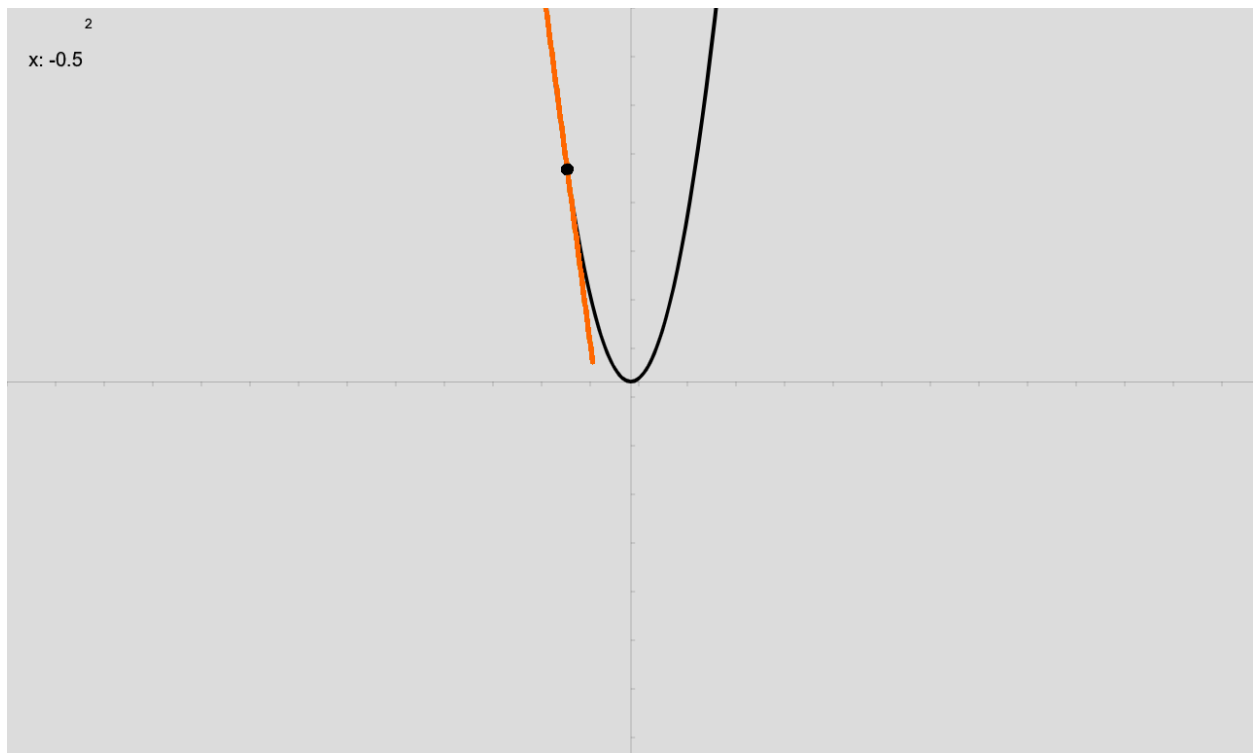
It may be an *inflection point*. This is a point where a function has a change in the direction of curvature. For example, the curve of the function goes from "facing down" to "facing up." Finding inflection points involves a second derivative test, which we will not get to in this lesson.



Calculating Derivatives

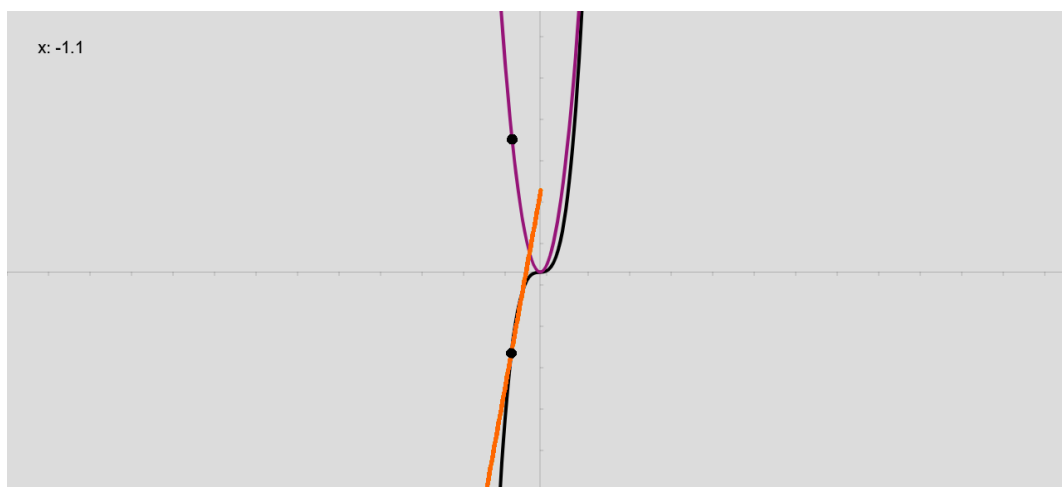
We have talked at length about derivatives abstractly. But how do we actually find derivative functions? We will outline this in this exercise and the next one.

The answer is through a series of "rules" that we'll introduce in this exercise and the next. These rules are building blocks, and by combining rules, we can find and plot the derivatives of many common functions.



More on Calculating Derivatives

So we now have an idea on where to start differentiating (that is, take the derivative of) things
lik



Calculating Derivatives in Python

We've discussed how to compute derivatives by hand. However, oftentimes when working on a computer, we have functions given to us in array form rather than an explicit equation. In this exercise, we will investigate how to calculate derivatives using Python.

For example, let's say we have an array that represents the following function:

$$f(x)=x^2+3$$

$$f'(x)=2x$$

$$2$$

$$+3$$

We can define this function in two sets of arrays using the following code.

```
from math import pow
# dx is the "step" between each x value
dx = 0.05
def f(x):
    # to calculate the y values of the function
    return pow(x, 2) + 3
# x values
f_array_x = [x for x in np.arange(0,4,dx)]
# y values
f_array_y = [f(x) for x in np.arange(0,4,dx)]
```

Copy to Clipboard

The `pow()` function within the `f(x)` function we defined allows us to calculate the values of x^2 , and we use a list comprehension to create a list of the y-values in a variable called `f_array_y`.

To compute the derivative of `f_array`, we use a NumPy function called `gradient()`.

```
f_array_deriv = np.gradient(f_array_y, dx)
```

Copy to Clipboard

`gradient()` takes in an array (in this case, a one-dimensional array) as its first argument. We also need to specify the distance between the x values, which is `dx`. For a one-dimensional array, `gradient()` cal

```
# Turning our function into an array
import numpy as np
from math import pow
from math import sin
import codecademylib3
import matplotlib.pyplot as plt

# our change in x value
dx = 0.01
def f(x):
    return sin(x)
sin_y = [f(x) for x in np.arange(0,20,dx)]
sin_x = [x for x in np.arange(0,20,dx)]

# Define your derivative here
sin_deriv = np.gradient(sin_y,dx)

plt.plot(sin_x, sin_y)
plt.plot(sin_x, sin_deriv)
plt.show()
```

Introduction: Statistics Fundamentals Part II

Discover what you will learn in Statistics Fundamentals Part II

The 'Science' part of Data Science comes from using the scientific method to get results. A lot of what Data Scientists do is conduct experiments with data that already exists. This presents some unique challenges compared to creating a completely new data set.

For starters, the data might not address exactly what you want it to, it might be incidental data from multiple sources that you have to reconcile. Whatever the case may be, it's the responsibility of the Data Scientist to still analyze that data with the same rigor as they might analyze the results of a lab experiment.

This mostly comes down to testing hypotheses and designing experiments. Sometimes those experiments are in the shape of A/B tests - like when evaluating two variables. Sometimes they are testing the strength of a relationship between two variables.

Though you've had an introduction to Hypothesis testing already, to work as an Inferential or Machine Learning Data Scientist, we will go a little deeper to develop a more sophisticated toolset in this area.

What will Statistics Fundamentals Part II cover?

After this unit, you will be able to:

- Test the strength of an association between two variables
- Design experiments
- Conduct A/B Tests
- Calculate the necessary sample size for an experiment
- Apply your hypothesis testing skills to new projects

Introduction

In this lesson, we'll use hypothesis tests to make inference about population-level associations between two

Preview: Docs Loading link description

[variables](#)

.

We will cover four different hypothesis tests:

Two Sample T-Tests (for an association between a quantitative variable and a

Preview: Docs Loading link description

[binary](#)

categorical variable)

ANOVA and Tukey Tests (for an association between a quantitative variable and a non-binary categorical variable)

Chi-Square Tests (for an association between two categorical variables)

Hypothesis Testing: Associations

Two-Sample T-Test

Suppose that a company is considering a new color-scheme for their website. They think that visitors will spend more time on the site if it is brightly colored. To test this theory, the company shows the old and new versions of the website to 50 site visitors, each – and finds that, on average, visitors spent 2 minutes longer on the new version compared to the old. Will this be true of future visitors as well? Or could this have happened by random chance among the 100 people in this sample?

One way of testing this is with a *2-sample t-test*. The null hypothesis for this test is that average length of a visit does not differ based on the color of the website. In other words, if we could observe all site visitors in two alternate universes (one where they see each version of the site), the average visiting times in these universes would be equal.

We can use SciPy's `ttest_ind()` function to perform a 2-sample t-test. It takes the values for each group as inputs and returns the t-statistic (not covered in this course) and a p-value:

```
from scipy.stats import ttest_ind
tstat, pval = ttest_ind(times_version1, times_version2)
```

Copy to Clipboard

By default, `ttest_ind()` runs a two-sided test.

QUESTIONS:

The company randomly sampled 100 site visitors. They showed the old version of their website to half of their sample and the new version to the other half. The amount of time (in minutes) that each visitor spent on the website was recorded.

An overlaid histogram showing the amount of time spent on the website by visitors to the new and old versions has already been created for you in **script.py**. Press “Run” and inspect the histograms. Based on this picture, do you think there is a significant association between the version of the website a visitor saw and how long they spent on the site?

Stuck? Get a hint

Checkpoint 2 Passed

2.

The data from this study has already been saved for you in **script.py**: the time spent by the 50 visitors to the old version is saved as `old`; the time spent by visitors to the new version is saved as `new`. Run a two-sample t-test comparing these groups and save the p-value as `pval`, then print it out.

Stuck? Get a hint

Checkpoint 3 Passed

3.

Using a significance threshold of 0.05, is there a significant difference between the average amount of time visitors are spending on the old and new versions of the website? In **script.py** set the value of `significant` equal to `True` if there is a significant difference and `False` if not.

SOLUTION

```
import pandas as pd
import matplotlib.pyplot as plt
import codecademylib3
from scipy.stats import ttest_ind
```



```

data = pd.read_csv('version_time.csv')

#separate out times for two versions
old = data.time_minutes[data.version=='old']
new = data.time_minutes[data.version=='new']

#run the t-test here:
tstat, pval = ttest_ind(old, new)
print(pval)

#determine significance
significant = True

#plot overlapping histograms
plt.hist(old, alpha=.8, label='old')
plt.hist(new, alpha=.8, label='new')
plt.legend()
plt.show()

```

Multiple Tests

In the previous exercise, we used a two-sample t-test to investigate an association between a quantitative variable (time spent on a website) and a

Preview: Docs Binary is a number system of 1s and 0s which serves as a textual representation method for giving instructions to a computers CPU. In computing and telecommunications, binary codes are used for various methods of encoding data such as turning the character strings of source code, into bit strings or instruction sets for the CPU to execute.

[binary](#)

categorical variable (an old color scheme or a new color scheme).

In some circumstances, we might instead care about an association between a quantitative variable and a **non-binary** categorical variable (non-binary means more than two categories).

For example, suppose that we own a chain of stores that sell ants, called VeryAnts. There are three different locations: A, B, and C. We want to know whether customers are spending a significantly different amount per order at any of the locations.

There are three different comparisons we could make: A vs. B, B vs. C, and A vs. C. One way to answer our question is to simply run three separate 2-sample t-tests.

QUESTIONS:

We have created samples `a`, `b`, and `c`, representing the amount (in U.S.D) spent on orders at VeryAnts at locations A, B, and C, respectively. We want to see if there's a significant difference in the average spending per order at the three locations.

Code has been provided for you to generate side by side box plots of the sales at each of these stores.

Based on this visualization, are there any stores where customers appear to be spending more or less money?

Stuck? Get a hint

Checkpoint 2 Passed

2.

Perform a 2-Sample T-test between each pair of location data.

Store the p-values in variables called `a_b_pval`, `a_c_pval`, and `b_c_pval`. Print them to the console.

Note that you may see numbers in scientific notation in the print out of one or more p-values. If you see

something like `e-05` at the end of a number, that means that the preceding number is multiplied by $10^{(-5)}$. In other words, `2.5134230524e-05` is equal to `0.000025134230524`.

Stuck? Get a hint

Checkpoint 3 Passed

3.

Inspect the p-values that you printed out. Using a significance level of 0.05, for which pairs of stores did you find a significant difference between the average sale price? Assign the values of `a_b_significant`, `a_c_significant`, `b_c_significant` to `True` if the p-value indicates a significant difference and `False` if the p-value does not indicate a significant difference.

SOLUTIONS:

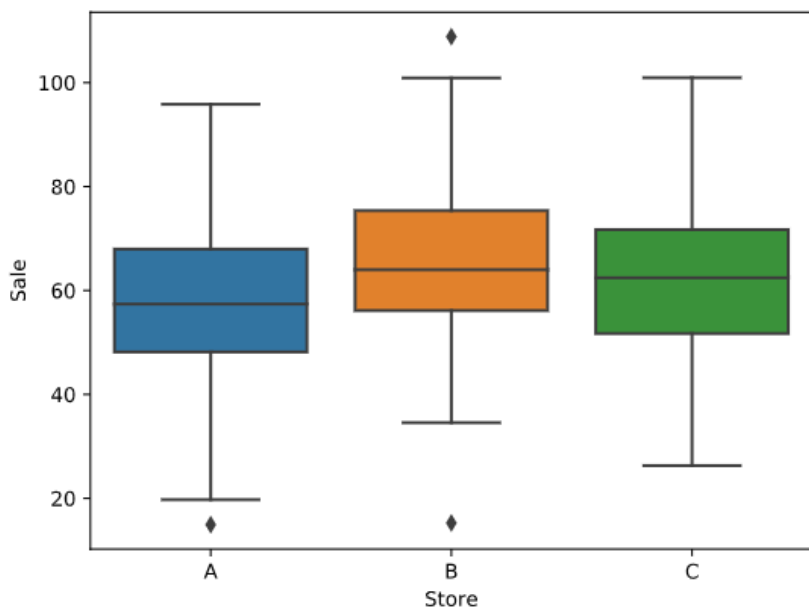
```
from scipy.stats import ttest_ind
import codecademylib3
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')
a = veryants.Sale[veryants.Store == 'A']
b = veryants.Sale[veryants.Store == 'B']
c = veryants.Sale[veryants.Store == 'C']

# run t-tests
a_b_tval, a_b_pval = ttest_ind(a,b)
a_c_tval, a_c_pval = ttest_ind(a,c)
b_c_tval, b_c_pval = ttest_ind(b,c)
print("A to B comparison:", a_b_pval)
print("A to C comparison:", a_c_pval)
print("B to C comparison:", b_c_pval)

# determine significance
a_b_significant = True
a_c_significant = True
b_c_significant = False

# create plot
sns.boxplot(data=veryants, x='Store', y='Sale')
plt.show()
```



ANOVA

In the last exercise, we ran three separate 2-sample t-tests to investigate an association between a quantitative variable (amount spent per sale) and a non-binary categorical variable (location of VeryAnts visited, with options A, B, and C). The problem with this approach is that it inflates our probability of a type I error; the more tests we run, the worse the problem becomes!

In this situation, one approach is to instead use *ANOVA* (Analysis of Variance). ANOVA tests the null hypothesis that all groups have the same population mean (eg., the true average price of a sale is the same at every location of VeryAnts).

In Python, we can use the SciPy function `f_oneway()` to perform an ANOVA. `f_oneway()` has two outputs: the F-statistic (not covered in this course) and the p-value. If we were comparing scores on a video-game for math majors, writing majors, and psychology majors, we could run an ANOVA test with this line:

```
from scipy.stats import f_oneway
fstat, pval = f_oneway(scores_mathematicians, scores_writers, scores_psychologists)
```

Copy to Clipboard

If the p-value is below our significance threshold, we can conclude that at least one pair of our groups earned significantly different scores on average; however, we won't know which pair until we investigate further!

QUESTIONS:

The same data from the previous exercise is available to you in the workspace: costs of sales made at three locations of VeryAnts (saved as `a`, `b`, and `c`).

Perform an ANOVA test on `a`, `b`, and `c` and store the p-value in a variable called `pval`, then print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

At a .05 significance level, does this p-value lead you to reject the null hypothesis (and conclude that at least one pair of stores have significantly different average sales)?

Change the value of `significant` to `True` if the p-value indicates at least one pair of stores have significantly different sales and `False` otherwise.

SOLUTIONS:

```
from scipy.stats import f_oneway
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')
a = veryants.Sale[veryants.Store == 'A']
b = veryants.Sale[veryants.Store == 'B']
c = veryants.Sale[veryants.Store == 'C']

# run ANOVA
Fstat, pval = f_oneway(a, b, c)
print(pval)

# determine significance
significant = True
```

Tukey's Range Test

Let's say that we have performed an ANOVA to compare sales at the three VeryAnts stores. We calculated a p-value less than 0.05 and concluded that there is a significant difference between at least one pair of stores.

Now, we want to find out **which** pair of stores are different. This is where Tukey's range test comes in handy!

In Python, we can perform Tukey's range test using the `statsmodels` function `pairwise_tukeyhsd()`. For example, suppose we are again comparing video-game scores for math majors, writing majors, and psychology majors. We have a dataset named `data` with two columns: `score` and `major`. We could run Tukey's range test with a type I error rate of 0.05 as follows:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey_results = pairwise_tukeyhsd(data.score, data.major, 0.05)
print(tukey_results)
```

Copy to Clipboard

Output:

```
Multiple Comparison of Means - Tukey HSD,FWER=0.05
=====
group1 group2 meandiff lower upper reject
-----
math psych 3.32 -0.11 6.74 False
math write 5.23 2.03 8.43 True
psych write -2.12 -5.25 1.01 False
=====
```

Tukey's range test is similar to running three separate 2-sample t-tests, except that it runs all of these tests simultaneously in order to preserve the type I error rate.

The function output is a table, with one row per pair-wise comparison. For every comparison where `reject` is `True`, we "reject the null hypothesis" and conclude there *is* a significant difference between those two groups. For example, in the output above, we would conclude that there is a significant difference between scores for math and writing majors, but no significant difference in scores for the other comparisons.

QUESTIONS:

The `veryants` dataset is provided for you once again in `script.py`. The `Store` column represents the store that a sale was made at ('A', 'B', or 'C') and the `Sale` column represents the cost of a sale in U.S.D. Run Tukey's range test with a type I error rate of 0.05 to determine whether average sales are different at any pair of two stores and save the result as `tukey_results`, then print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Inspect the output from the test you just ran. For which pairs of stores did you find a significant difference in average sales?

Assign the values of `a_b_significant`, `a_c_significant`, `b_c_significant` to `True` if the test indicates a significant difference in sales at the indicated pair of stores and `False` if not.

Recall that when we ran three t-tests, we found significant differences for the A vs. B and A vs. C comparisons. Do we get the same result with this test?

Stuck? Get a hint

SOLUTIONS:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')

# run tukey's test
tukey_results = pairwise_tukeyhsd(veryants.Sale, veryants.Store, 0.05)
print(tukey_results)

# determine significance
a_b_significant = True
a_c_significant = False
b_c_significant = False
```

RESULT:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj lower upper reject
-----
A    B    7.2763 0.001 3.2263 11.3264  True
A    C    4.011 0.0529 -0.0391  8.0611  False
B    C   -3.2653 0.141 -7.3154  0.7847  False
-----
```

Assumptions of T-Tests, ANOVA, and Tukey

Before we use a two sample t-test, ANOVA, or Tukey's range test, we need to be sure that the following things are true:

1. The observations should be independently randomly sampled from the population

Suppose the population we are interested in is all visitors to a website. Random sampling will help ensure that our sample is representative of the population we care about. For example, if we only sample site visitors on Halloween, those visitors may behave differently from the general population. In practice, this can be a challenging assumption to meet, but it's important to be aware of.

2. The standard deviations of the groups should be equal

For example, if we're comparing time spent on a website for two versions of a homepage, we first want to make sure that the standard deviation of time spent on version 1 is roughly equal to the standard deviation of time spent on version 2. To check this assumption, it is normally sufficient to divide one standard deviation by the other and see if the ratio is "close" to 1. Generally, a ratio between 0.9 and 1.1 should suffice.

That said, there is also a way to run a 2-sample t-test without assuming equal standard deviations — for example, by setting the `equal_var`

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

[parameter](#)

in the `scipy.stats.ttest_ind()` function equal to `False`. Running the test in this way has some disadvantages (it essentially makes it harder to reject the null hypothesis even when there is a true difference between groups), so it's important to check for equal standard deviations before running the test.

3. The data should be normally distributed...ish

Data analysts in the real world often still perform these tests on data that are not normally distributed. This is usually not a problem if sample size is large, but it depends on how non-normal the data is. In general, the bigger the sample size, the safer you are!

4. The groups created by the categorical variable must be independent

Here are some examples where the groups are **not** independent:

the number of goals scored per soccer player before, during, and after undergoing a rigorous training regimen (not independent because the same players are measured in each category)

years of schooling completed by a group of adults compared to their parents (not independent because kids and their parents can influence one another)

QUESTIONS:

Suppose that we want to run a two-sample t-test to compare the sample means for two groups (saved as `dist_1` and `dist_2` in the workspace).

Before running the test, we want to check whether the ratio of standard deviations for these groups is equal. Use the NumPy `std()` function to divide the standard deviation of `dist_1` by the standard deviation of `dist_2` and save the result as `ratio`, then print it out.

Are the standard deviations roughly equal?

Stuck? Get a hint

Checkpoint 2 Passed

2.

Code has already been provided for you in the workspace to create an overlaid histogram of these two distributions. Inspect the distributions. Are they approximately normal? If so, set the value of `normal_assumption` equal to `True` in **script.py**; otherwise set `normal_assumption` equal to `False`.

SOLUTIONS:

```
import codecademylib3
import numpy as np
import matplotlib.pyplot as plt

dist_1 = np.genfromtxt("1.csv")
dist_2 = np.genfromtxt("2.csv")

#calculate ratio of standard deviations:
ratio = np.std(dist_1)/np.std(dist_2)
print(ratio)

#check normality assumption
normal_assumption = True

#plot histograms of each distribution
plt.hist(dist_1, alpha = .8, normed = True, label = 'dist 1')
plt.hist(dist_2, alpha = .8, normed = True, label = 'dist 2')
plt.legend()
plt.show()
```

Chi-Square Test

If we want to understand whether the outcomes of two categorical

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

[variables](#)

are associated, we can use a Chi-Square test. It is useful in situations like:

An A/B test where half of users were shown a green submit button and the other half were shown a purple submit button. Was one group more likely to click the submit button?

People under and over age 40 were given a survey asking “Which of the following three products is your favorite?” Did these age groups have significantly different preferences?

In `SciPy`, we can use the function `chi2_contingency()` to perform a Chi-Square test. The input to `chi2_contingency` is a contingency table, which can be created using the `pandas crosstab()` function as follows:

```
#create table:
import pandas as pd
table = pd.crosstab(variable_1, variable_2)

#run the test:
from scipy.stats import chi2_contingency
```

```
chi2, pval, dof, expected = chi2_contingency(table)
```

For example, suppose we want to know whether gender is associated with the probability of a website visitor making a purchase. The null hypothesis is that there's no association between the variables (eg. males, females, and non-binary people are all equally likely to make a purchase on the website, so gender and purchase-status are not associated). If the p-value is below our chosen threshold (often 0.05), we reject the null hypothesis and can conclude there is a statistically significant association between the two variables (eg. men, women, and non-binary people appear to have different probabilities of making a purchase, so gender is associated with purchase-status).

QUESTIONS:

The management at the VeryAnts ant store wants to know if their two most popular species of ants, the Leaf Cutter and the Harvester, vary in popularity between 1st, 2nd, and 3rd graders. We have provided a dataset named `ants` with a sample of 108 sales to 1st, 2nd, and 3rd grade teachers. The dataset has two columns: `Grade` (equal to '1st', '2nd', or '3rd') and `Ant` (equal to 'Leaf Cutter' or 'Harvester').

Use this data to create a contingency table of the `Grade` and `Ant` columns, and save the table as `table`.

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `chi2_contingency()` function from SciPy to run a Chi-Square test using the contingency table you just created (saved as `table`). Save the p-value as `pval` and print it out.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Are certain types of ants more popular among specific grades (is there an association between grade and ant type)? Using a significance threshold of 0.05, indicate your answer by changing the value of `significant` to `True` if there is a significant association between these variables and `False` otherwise.

SOLUTIONS:

```
import pandas as pd
from scipy.stats import chi2_contingency

# read in and print data
ants = pd.read_csv("ants_grade.csv")
print(ants.head())

# create contingency table
table = pd.crosstab(ants.Grade, ants.Ant)
print(table)

# run Chi-Square test and print p-value
chi2, pval, dof, expected = chi2_contingency(table)
print(pval)

# determine significance
significant = False
```

RESULTS:

Output-only Terminal

```
Grade      Ant
0  1st      harvester
1  2nd      harvester
2  2nd      leaf cutter
3  1st      harvester
4  3rd      leaf cutter
Ant      harvester  leaf cutter
Grade
1st      28          8
2nd      31          5
3rd      23          13
0.08356116834982279
```

Assumptions of a Chi-Square Test

Before we use a Chi-Square test, we need to be sure that the following things are true:

1. The observations should be independently randomly sampled from the population

This is also true of 2-sample t-tests, ANOVA, and Tukey. The purpose of this assumption is to ensure that the sample is representative of the population of interest.

2. The categories of both

Preview: Docs Loading link description

[variables](#)

must be mutually exclusive

In other words, individual observations should only fall into one category per variable. This means that categorical variables like “college major”, where students can have multiple different college majors, would not be appropriate for a Chi-Square test.

3. The groups should be independent

Similar to 2-sample t-tests, ANOVA, and Tukey, a Chi-Square test also shouldn’t be used if either of the categorical variables splits observations into groups that can influence one another. For example, a Chi-Square test would **not** be appropriate if one of the variables represents three different time points.

QUESTIONS:

Researchers are running a study to test a new vaccine for Covid-19 in adults. A sample of 1000 adults (you can assume that they are randomly sampled adults, or at least representative of the population) are randomly split into two groups: half get a vaccine, while the other half get a placebo. Everyone is monitored for six months to see if they develop symptoms of Covid-19. The first few rows of the resulting dataset looks like this:

Group

Outcome

vaccine	not sick
---------	----------

vaccine	not sick
---------	----------

placebo	sick
---------	------

placebo	not sick
---------	----------

The researchers want to use this data to determine whether their vaccine will be effective at preventing illness in the general population of adults (eg., is whether or not someone got a vaccine associated with whether or not they got sick?).

Is a Chi-Square test appropriate to address this question? Change the value of `checkpoint_1` to `True` if a Chi-Square test is appropriate and `False` if it is not.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Researchers are interested in studying the effect of a 10 minute yoga regimen on self-reported mood in adults. In order to test this, a representative sample of 1000 adults are asked to complete a survey where they rate their current happiness level as “very low”, “low”, “neutral”, “high”, or “very high”. Each person then completes a 10 minute yoga regimen, then responds to the same survey once again. The first few rows of data from this study look like this:

Person ID	Time	Happiness
1	before yoga	low
1	after yoga	neutral
2	before yoga	neutral
2	after yoga	high

The researchers want to know if 10 minutes of yoga can help improve self-reported mood for adults in the general population (eg., is whether or not someone has just completed 10 minutes of yoga associated with their self-reported happiness level?).

Is a Chi-Square test appropriate to address this question? Change the value of `checkpoint_2` to `True` if a Chi-Square test is appropriate and `False` if it is not.

Stuck? Get a hint

SOLUTIONS

```
checkpoint_1 = True  
checkpoint_2 = False
```

Review

In this lesson, we have reviewed a few different ways to run a hypothesis test for an association between two variables:

Two Sample T-Tests (for an association between a quantitative variable and a

Preview: Docs Loading link description

[binary](#)

categorical variable)

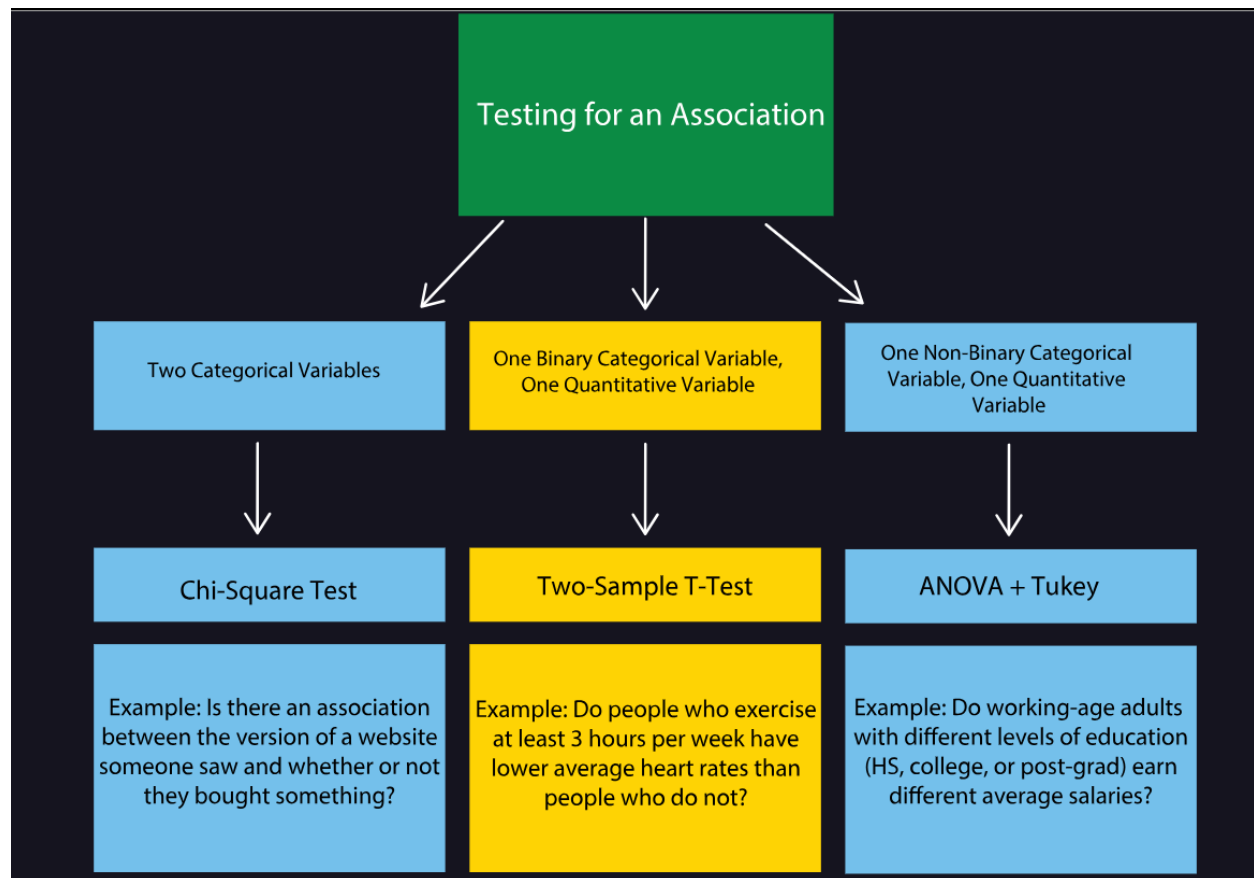
ANOVA and Tukey Tests (for an association between a quantitative variable and a non-binary categorical variable)

Chi-Square Tests (for an association between two categorical variables)

You are now equipped to use and interpret the results of each of these tests!

Hypothesis Testing for an Association - VIDEO- YouTube

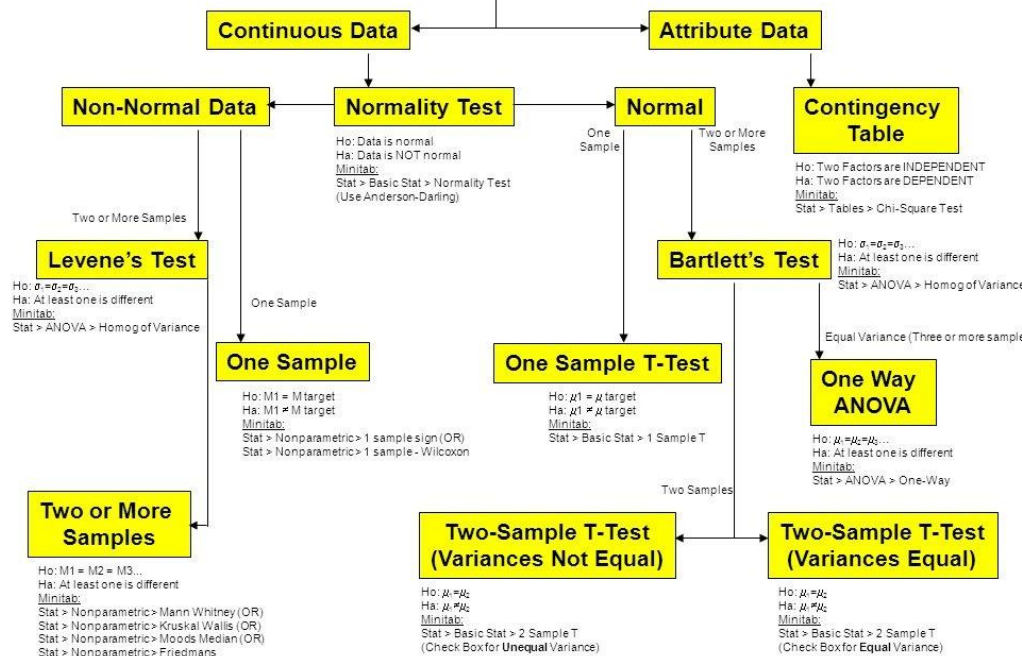
- <https://www.youtube.com/watch?v=w-7dSKL0UHs>
- <https://www.codecademy.com/paths/data-science/tracks/dsml-statistics-fundamentals-part-ii/modules/dsaly-hypothesis-testing-testing-an-association/informationals/live-stats-hp-association>
-



For all tests:
 $p > 0.05$ Fail to Reject H_0 (Null)
 $p < 0.05$ Reject H_0

Hypothesis Testing

“Roadmap”



How to Choose a Hypothesis Test

Learn how to choose an appropriate hypothesis test for your data and questions.

Introduction

In this module we'll cover some of the key considerations for designing an experiment. One of the first things to consider is **which statistical test** is appropriate for the question you are trying to answer. Sometimes this will depend on the data you have available, and sometimes you will have the opportunity to decide this before starting to collect data.

There are many different types of hypothesis tests that should be used in different situations. In this article, we are going to briefly review some of them and when they are appropriate.

Here is a summary table of some of the hypothesis tests we'll cover:

	Comparing a sample statistic to a hypothesized population value for a...	Testing an association between a binary categorical variable (2 categories) and a...	Testing an association between a categorical variable with 3 or more categories and a...
Quantitative Variable	<i>One sample t-test</i>	<i>Two-Sample t-test</i>	<i>ANOVA with Tukey's Range Test</i>
Categorical Variable	<i>Binomial Test</i>	<i>Chi-Square Test</i>	

Comparing a sample statistic to a hypothesized population value

In some situations, we may want to compare a **sample statistic** to a **hypothesized population value**. In this scenario, we have a sample of data from some population and have calculated a sample statistic (eg., a sample mean, frequency, or proportion). Based on this observed sample statistic, we then want to know whether the sample was likely to be drawn from a population with some hypothesized or target value of that same statistic.

For example, suppose we hypothesize that 5% of all people who access a website will buy a product. In order to understand if this is the case, we might find a sample of people who accessed that website and calculate that 4.3% of our sample bought a product. Finally, we can use a hypothesis test to determine the likelihood that, if we could observe ALL people who ever access the website, 5% of them will buy a product (even though only 4.3% of our sample did).

If we have a sample of quantitative data, such as height, weight, or amount spent, we should use a one-sample t-test. If we have a sample of binary data, such as whether or not someone made a purchase or clicked a link, we should use a binomial test.

Example: one sample t-test

Suppose we want to compare exam scores for students who attended a test prep program to the global average score of 35 points. Do students who attend this program score higher than 35 points? The global average is the **hypothesized population value** and the average of the exam scores of students who attended the program is the **sample statistic** (in this case, sample **mean**).

Below is the code to run a one-sample t-test to address the above question. In this example the **alternative hypothesis** is that the sample mean is significantly different than 35, and the **null hypothesis** is that the sample mean is 35.

```
from scipy.stats import ttest_1samp

global_average_score = 35
sample_scores = [12, 42, 37, 18, 23, 39, 45, ... , 52]

t_stat, p_value = ttest_1samp(sample_scores, global_average_score)
```

Example: binomial test

If we instead have a sample of **binary data** and want to compare a sample proportion/frequency to an **underlying probability (population value)**, a binomial test is appropriate. The classic example of a binomial test is tossing a coin to determine if it's fair (fair means that the probability of either heads or tails is exactly 50%).

For example, suppose that you collect sample data from a coin by tossing it 100 times, and find that 45 flips result in heads. Based on this sample, what is the probability that the coin is actually fair (if you flipped it infinitely many times, exactly half those flips would be heads)? The following code runs the binomial test to answer this question:

```
from scipy.stats import binom_test

p_value = binom_test(45, 100, p = 0.50)
```

The alternative hypothesis for this test is that the probability is **different** than $p = 0.50$, and the null is that it is equal to 0.50 .

Here are some other examples of situations where a binomial test would be useful

Fill in the blank

Questions

Quality Assurance at the Dairy Culture yogurt company

Code

You are a data analyst on the quality assurance team of a yogurt company, "The Dairy Culture". The team studies samples of its yogurt on a regular basis to make sure it meets its nutritional facts and collects data on the number of discarded yogurts during production. Their nutritional facts declare that a 6oz yogurt has 20g of Carbohydrates. The Dairy Culture also has a goal of discarding bad yogurts at a rate of 4% or lower.

To test both of these, you take a sample of 1000 6oz yogurts and find that those yogurts have an average of 23g of Carbohydrates. You also find that 48 of the 1000 yogurts you sampled were discarded (or 4.8%).

To check the claim that 6oz yogurts made by The Dairy Culture have 20g of carbohydrates on average, you would run a

blank 1

with the null hypothesis that all yogurts (the "population") have an average of

blank 2

of carbohydrates.

To check whether The Dairy Culture met their discarding goal of 4% or lower, you would run a

blank 3

with the null hypothesis that the discarding goal is less than

blank 4

for all yogurts (the population).

Answer Choices

6g

two-sample t-test

20%

4.8%

one sample t-test

4%

binomial test

20g

Click or drag and drop to fill in the blank

Testing for an association between two variables at the population level

When we have a sample of data with two variables, and want to know if there is an association between those variables at the population level, we'll need a different set of hypothesis tests. For example, we might be interested in the relationship between:

In each of these examples, we cannot observe the entire population of interest (eg., the entire wild population of each species of bear). Instead, we can collect data for a smaller sample and then use a hypothesis test to understand the likelihood that a similar association exists among the population we care about.

Example: two-sample t-test

A two-sample t-test is used to investigate an association between a quantitative variable and a binary categorical variable. For example, suppose we want to test if there is an association between claw size (quantitative) and species: black or grizzly bear (binary categorical). To answer this question, we could sample a selection of black bears and grizzly bears, then calculate the average claw size for each species. Then, we can use a two-sample t-test to determine the probability that the claw sizes for these two species are significantly different (among the entire population of black and grizzly bears). The data might look like this:

Species		Claw size (in)
0	black	1.48

1	grizzly	2.01
2	grizzly	1.89
3	black	1.65
4	black	1.58

And the code to run the two-sample t-test will look like this:

```
from scipy.stats import ttest_ind

#separate out claw lengths for two species
grizzly_bear = data.claw_length[data.species=='grizzly']
black_bear = data.claw_length[data.species=='black']

#run the t-test here:
tstat, pval = ttest_ind(grizzly_bear, black_bear)
```

Other examples of two-sample t-tests include studies like drug trials or psychology studies with a control and experimental group or A/B Testing with quantitative data like “time spent on a website”.

Example: ANOVA and Tukey's range test

In cases similar to the two-sample t-test, but when the categorical variable has three or more categories, an ANOVA can be used to see if there is a significant difference between *any* of the groups. Then, if at least one pair of groups are significantly different, Tukey's range test can be used to determine **which** groups are different. This is better than running multiple two-sample t-tests because it leads to a lower probability of making a type I error.

For example, if we want to compare the heights of **three different tree species**, in order to test the hypothesis that average tree heights vary by species, we can use an ANOVA. Then, if the p-value from the ANOVA is below our significance threshold, we can run Tukey's range test to determine which tree species have significantly different heights.

The code to run these tests is as follows:

```
# ANOVA Test
from scipy.stats import f_oneway
fstat, pval = f_oneway(heights_pine, heights_oak, heights_spruce)

# Tukey's Range Test
from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey_results = pairwise_tukeyhsd(tree_data.height, tree_data.species, 0.05)
```

Example: Chi-Square test

When looking at the relationship between two categorical variables, we can run a Chi-Square Test to see if there is a significant association between the variables. Both variables can have any number of categories. Some examples include testing an association between:

For example, suppose we are interested in understanding whether there is an association between the version of a website someone sees and whether or not they subscribe. The data may look something like this:

Web Version		Subscribed?
0	A	yes
1	A	yes
2	B	yes
3	B	no
4	A	no

These variables can then be summarized using a contingency table:

Subscribed		no	yes
Web Version			
A		23	27
B		16	34

Finally, a Chi-Square test evaluates whether the observed contingency table is significantly different from the table that would be expected if there were no association between the variables. The code to create the contingency table and run the Chi-Square test looks something like this:

```
import pandas as pd
from scipy.stats import chi2_contingency

# create contingency table
ab_contingency = pd.crosstab(data.Web_Version, data.Subscribed)

# run a Chi-Square test
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
```

Finding a representative sample

Beyond choosing a hypothesis test, it is important to understand whether the data you have meets the assumptions of the test you want to run. Each hypothesis test has a unique set of assumptions, which we won't cover in detail in this article; however, there is one assumption that all hypothesis tests share: the data was randomly sampled from the population of interest.

This is important because random sampling ensures that the sample is representative of the population in terms of observed (and unobserved) characteristics. Unfortunately, there may be situations where random sampling is impossible, but it is important to understand how this can bias results of a test.

For example, let's return to the example with the yogurt company "The Dairy Culture". Let's say the company had multiple factories, but the quality assurance team only collected yogurts from one specific factory. The data is thus **not** randomly sampled from the entire population that we care about (all factories), and could be biased if the quality of yogurt differs at each one.

There can also be ethical issues that arise when a sample is not representative of a population. When developing and testing a vaccine, for example, researchers must make sure to find volunteers from an appropriate proportion of genders, races, age ranges, pre-existing conditions, and so on to test efficacy for the entire population that the vaccine will be used on. If the vaccine manufacturers test on a sample that doesn't include sufficient data for one race, there is a risk that there could be reduced (if during the initial research phase) or unknown efficacy for that group.

It can often be challenging to find a representative sample or even to recognize when there is biased data, but it is essential to think about when designing an experiment.

A/B Testing: Sample Size Calculators

Baseline Conversion Rate

A/B tests usually compare an option that we're currently using to a new option that we suspect might be better. In order to compare the two options, we need a metric. Often, our metric will be the percent of users who take a certain action after interacting with one of our options. For instance:

The percent of customers who **buy a t-shirt** after visiting one of two versions of a website
The percent of users who **click** on one of two versions of an ad

In the t-shirt example above, the *baseline conversion rate* is our estimate for the percent of people who will buy a shirt under the current website design.

We can generally calculate a baseline by looking at historical data for the option that we're currently using. For example, suppose that 2000 people visited a website over the past three months and 320 of those visitors purchased a shirt. We could estimate the baseline rate as follows:

```
baseline = 320/2000*100  
print(baseline) #output: 16.0
```

Copy to Clipboard

This number may be written as a proportion (eg., 0.16) or a percent (eg., 16%).

QUESTIONS:

Suppose that an online media company wants to run an A/B test to see if a re-designed subscribe button will make site visitors more likely click “subscribe”. Some historical data is saved for you in **script.py**:

`number_of_site_visitors` is the number of people who visited the site over the past month
`number_of_converted_visitors` is the number who clicked “subscribe”.

Use these two variables to calculate the baseline conversion rate and save the result as a variable named `baseline_rate`, then print it out. Report the percentage, not the proportion.

Hint

Divide the number of converted visitors by the number of total visitors and multiply by 100.

SOLUTIONS:

```
number_of_site_visitors = 2000.0
number_of_converted_visitors = 1300.0

# calculate baseline_rate and print it out:
baseline_rate = number_of_converted_visitors/number_of_site_visitors*100

print(baseline_rate)
```

Minimum Detectable Effect

Suppose we’re running an A/B Test to find out if a new website layout drives more subscriptions than the current one. If the new layout is only a tiny percent better, would we really care?

In order to detect precise differences, we need a very large sample size. In order to choose a sample size, we need to know the smallest difference that we actually care to measure. This “smallest difference” is our desired *minimum detectable effect*. This is also sometimes referred to as *desired lift*.

Minimum detectable effect or lift is generally expressed as a percent of the baseline conversion rate. Suppose that 6% of customers currently subscribe to our website (that’s our baseline conversion rate). Changing a website layout is hard, so we only think that it’s worth doing if at least 8% of our customers would subscribe with the new layout. To calculate this as a percentage of our baseline:

```
baseline = 6
new = 8
min_detectable_effect = (new - baseline) / baseline * 100
print(min_detectable_effect) #output: 33.0
```

Copy to Clipboard

Our minimum detectable effect/desired lift is 33%.

Instructions

Checkpoint 1 Passed

1.

Suppose you have a baseline conversion rate of 8% and want to implement a new website design if it will increase the conversion rate to 12%. What is your minimum detectable effect? Save that number into a variable called `mde` and print it out.

Significance Threshold

When we run an A/B test, we usually want to use the results of the test to make a decision: use version A or B? In order to make that decision, many data scientists use a pre-determined significance threshold for their hypothesis test. For example, if we set a significance threshold of 0.05 (a commonly chosen value), we'll "reject the null hypothesis" and conclude that the conversion rate for version B is significantly different from version A if we get a p-value less than 0.05.

It turns out that this significance threshold is the *false positive rate* for the test: the probability of finding a significant difference when there really is none. As a business owner, we don't want to make this kind of mistake, because then we might invest money in a change that doesn't actually make a difference!

Unfortunately, there's a trade-off between false positives and false negatives. A false negative occurs when there is a difference between version A and B, but the test doesn't detect it. This is a potential missed opportunity for a business owner!

Most A/B test sample size calculators estimate the sample size needed for a 20% false negative rate; while a data scientist needs to choose the false positive rate they are comfortable with. The lower the false positive rate, the larger the sample size will need to be!

Don't Interfere With Your Tests

Suppose that a Product Manager is running an A/B Test for a redesign of a landing page. Before starting the test, she used a sample size calculator to determine the sample size: 2,200 total website visitors. After reaching 2,200 visits, she ran a Chi-Square Test. The new website design performed slightly better, but the results were not statistically significant.

It might be tempting to run the test for another week to see if the difference becomes significant, but that would be a big mistake! By choosing to extend the A/B test past the original sample size, the project manager would introduce personal bias to the results of the test; she will be more likely to get the results she wants, regardless if these results reflect reality.

Here are two important rules for making sure that A/B tests remain unbiased:

- Don't continue to run the test after the predetermined sample size, until "significant" results are found.
- Don't stop a test before reaching the predetermined sample size, just because your results reach significance early (unless there are ethical reasons that require you to stop, like a prescription drug trial).

Test data is sensitive to changes in sample size, which is why it is important to calculate beforehand.

Sample Size Determination with Simulation

Introduction

In this lesson, we will use simulation to understand some of the considerations for setting up an A/B test: sample size, power, and the false positive rate. But before we think about designing an A/B test, let's first remind ourselves how to conduct the test itself, **after** planning and collecting data.

Suppose that a media company currently has a weekly newsletter email and wants to see if using the recipient's first name in the email subject will cause more people to open the email (ie. "Bob! Checkout this week's updates" vs "Checkout this week's updates"). They randomly assign a group of 100 recipients to receive one of the two email subjects and record whether or not each recipient opened the email. The first few rows of their data might look something like this:

Email	Opened
name	yes
name	no
control	yes
control	yes
name	no

In order to run a hypothesis test to decide whether there is a significant difference in the open rate for these emails, we would run a Chi-Square test. To accomplish this, we would first create a contingency table for the `Email` and `Opened`

Preview: Docs Loading link description

[variables](#)

in the above table:

```
X = pd.crosstab(data.Email, data.Opened)
print(X)
```

Copy to Clipboard

Output:

Opened	no	yes
Email		
control	23	27
name	16	34

We would then use this table to run a Chi-Square test and get a p-value:

```
chi2, pval, dof, expected = chi2_contingency(X)
print(pval) #Output: 0.2186
```

Copy to Clipboard

Based on the p-value, we would make a decision about which email to use; a small p-value would provide evidence that the open rates are significantly different for the two groups, while a large p-value would suggest no significant difference.

QUESTIONS

Run the code in **script.py** to see the first five rows of data.

Checkpoint 2 Passed

2.

Suppose that you are running an A/B test comparing two versions of a checkout page (version A or version B) to see whether there is a significantly different purchase rate for one version compared to the other. Data from this experiment has been loaded for you in **script.py** as a dataframe named `data`. Use this data to create a contingency table and save the result as `ab_contingency`, then print out the result.

Hint

Use the `pd.crosstab()` function with the two columns of the dataframe `Web_Version` and `Purchased` as the two input variables.

Checkpoint 3 Passed

3.

Use `ab_contingency` to run a Chi-Square test using `chi2_contingency()` and save the p-value as a variable named `pval`. Print out `pval`.

SOLUTIONS

```
import pandas as pd

from scipy.stats import chi2_contingency

data = pd.read_csv("ab_data.csv")
print(data.head())

# calculate contingency table here
ab_contingency = pd.crosstab(data.Web_Version, data.Purchased)
print(ab_contingency)

# run your chi square test here
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
print('P-value')
print(pval)
```

Simulating Data for a Chi-Square test

In the last exercise, we used some data from an A/B test to run a Chi-Square test. In the next few exercises, we'll build up a simulation to understand the considerations that go into choosing a sample size for that test.

Again consider the A/B test example from the previous exercise, comparing email subjects with and without the recipient's first name. Suppose we know that visitors have a 50% chance of opening the control email and a 65% chance of opening the name email (30% lift!).

Here we use **lift** to refer to the inherent difference in the distributions of our two groups of data. In the *A/B Testing: Sample Size Calculators* lesson, we learned that **minimum detectable effect** is the smallest size of the difference between the two groups that we want our test to be able to detect. If we set up our experiment with a minimum detectable effect of at least 20%, our statistical test should detect a difference with a "lift" or "effect" of 20% or greater. In this lesson we are going to simulate data that has a lift of 30% to demonstrate how the inherent lift impacts the power of our statistical test.

We can use the aforementioned probabilities to simulate a dataset of 100 email recipients as follows:

```
sample_control = np.random.choice(['yes', 'no'], size=50, p=[.5, .5])
sample_name = np.random.choice(['yes', 'no'], size=50, p=[.65, .35])
```

Copy to Clipboard

This gives us two simulated samples, of 50 recipients each, who hypothetically saw the name or control email subject. Each one looks something like `['yes' 'no' 'no' 'no' 'yes' 'yes' ...]`, where 'yes' corresponds to an opened email.

Next, we can assemble these arrays into a data frame that looks a lot like the one we saw in exercise 1:

```
group = ['control']*50 + ['name']*50
outcome = list(sample_control) + list(sample_name)
sim_data = {"Email": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)
print(sim_data.head())
```

QUESTIONS

1.

In **script.py**, you'll see the code from the narrative, which can be used to simulate a dataset for a Chi-Square test. You'll notice that we've replaced all hard-coded numbers with the following variables: `sample_size`, `control_rate`, and `name_rate` (which is calculated using `control_rate` and `lift`).

Change the sample size to 4 and press "Run". Inspect the output. Does it look as expected?

Stuck? Get a hint

Checkpoint 2 Enabled

2.

Press "Run" a few more times and notice how the data changes each time even though you haven't changed the code. This happens because we've provided probabilities for the outcomes; (opened or not), rather than specific values.

SOLUTIONS

```
import numpy as np
import pandas as pd

sample_size = 4
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate,1-control_rate])
sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])

group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
outcome = list(sample_control) + list(sample_name)
sim_data = {"Button": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)
print(sim_data)
```

Determining Significance

Now that we've practiced simulating data for an A/B test, let's actually run a Chi-Square test for each simulated dataset and consider the decision we would make based on the outcome.

If we were really running this test, we would want to use the data to make a decision about whether to use the control (old) or name (new) email subject. To make that decision, we can use a significance threshold. For example, if we're using a significance threshold of 0.05, we'll "reject the null hypothesis" for any p-value less than 0.05. In this context, rejecting the null would mean that we conclude that there **is** a significant difference between the open rates for the two email subjects and therefore we **should** switch to the email subject that uses the recipient's first name.

We can use the following Python statement to record whether a particular p-value is significant or not, based on a threshold of 0.05:

```
result = ('significant' if pval < 0.05 else 'not significant')
print(result)
```

QUESTIONS:

The code from the previous exercises is provided for you in **script.py**. This code generates a simulated dataset named `sim_data` and then runs a Chi-Square test for that data, saving the p-value as `pval`. An additional variable named `significance_threshold` has been defined for you in **script.py**, which is equal to the significance threshold for the test. After the p-value calculation, add a line of code that uses `significance_threshold` to determine whether the p-value is 'significant' or 'not significant'. Save the result as `result` and print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Press "Run" a few times until you see both a 'significant' and a 'not significant' result. Note that it is possible to get different results every time you sample a new group of 100 recipients.

SOLUTIONS

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# pre-set values
significance_threshold = 0.05
```



```

sample_size = 100
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

# simulate a dataset
sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate,1-control_rate])
sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])

group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
outcome = list(sample_control) + list(sample_name)
sim_data = {"Email": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)

# run a chi-square test
ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
chi2, pval, dof, expected = chi2_contingency(ab_contingency, correction=False)
print("P Value:")
print(pval)

# determine significance here:
result = ('significant' if pval < significance_threshold else 'not significant')

print("Result:")
print(result)

```

Estimating Power

In the last exercise, we learned how to simulate a dataset for a Chi-Square test, run the test, and then output a result: 'significant' or 'not significant'. In this exercise, we'll repeat that process many times so that we can inspect the relative frequency of each outcome.

To do this, we'll start by creating an empty list to store the results of our repeated experiments. Next, we'll move all of our simulation code (to create a sample dataset, run a Chi-Square test, and determine a result) inside of a for-loop. In each iteration of the loop, we'll append the outcome to our results list so that we can inspect it later.

The outline of the code looks something like this:

```

Set the sample size and subscription probabilities
Create an empty list named `results`

```

```

Repeat 100 times in a for-loop:
    Simulate a dataset
    Run a Chi-Square test
    Use the p-value to determine significance
    Append the result ('significant' or 'not significant') to `results`

```

Finally, we can inspect `results` by calculating the proportion of simulated tests where the result was `'significant'`:

```

results = np.array(results)
print(np.sum(results == 'significant')/100)

```

1.

In **script.py**, we've copied over the code from the previous exercise and moved the simulation inside a for-loop as described in the narrative. We've also initialized an empty list named `results`.

Below the determination of `result`, but still inside the for-loop, add a line of code to append `result` onto `results`.

Hint

`results` is a list, so you can use the `.append()` method to append a value to it. Make sure to keep this inside the for-loop by indenting to keep it inline with the rest of the code in the for-loop.

Checkpoint 2 Passed

2.

Outside of the for-loop, create a variable named, `sig_prop` that calculates the proportion of `results` that are 'significant'. Make sure to divide it by the total number of simulations to make it a proportion!

Print the results, then press "Run" a few times (note: you'll see slightly different numbers each time because this is a random process). Approximately what proportion of the results were significant (would have led us to switch to the new, name email subject)?

Hint

To find the proportion of results that were significant, you can add the 1s and divide by the total number of simulations. The code is similar to what we used above:

```
results = np.array(results)
sig_prop = np.sum(results == 'significant')/100
print(sig_prop)
```

Copy to Clipboard

Don't forget to convert the list into an array before summing.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 100
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list here:
    results.append(result)

# calculate proportion of significant results here:
print("Proportion of significant results:")
results = np.array(results)
sig_prop = np.sum(results == 'significant')/100
print(sig_prop)
```

False Positives and True Positives

In the previous exercise, we simulated 1,000 datasets and ran a Chi-Square test for each one, recording whether the results were 'significant' or 'not significant'. This allowed us to estimate the proportion of simulated datasets that led to a 'significant' result.

In general, we hope that the test reflects reality. We therefore want the result to be 'significant' if there really **is** a significant difference in the probability of an open for the two email subjects ($\text{lift} > 0$). In that case, the proportion of significant results is the true positive rate, also called the *power* of the test. Most sample size calculators aim for a power of 80%.

On the other hand, if there's no difference in the probability of an email being opened for the two email subjects ($\text{lift} = 0$), a 'significant' result would be a false-positive (also called a type I error). This would lead us to invest time and resources into adding first names into email subjects when there's no real pay-off in the long run.

Instructions

Checkpoint 1 Enabled

1.

The simulation code from the previous exercises is loaded for you in **script.py**. We've included the code to print out the proportion of tests where a significant result was recorded. Currently, the simulation is set up so that there **is** a difference in the probability of a subscription for the two buttons.

Press "Run" a few times and inspect the proportion of significant tests (printed to the output terminal) each time. If we ran a test with the provided sample size (100), baseline conversion rate (50%) and lift (30%), approximately what percent of the time would we correctly observe a significant result? Note that this is the "power" of the test.

Hint

When we run this code, we see that approximately 28% of the tests result in a significant p-value. Every time we run the simulation, we'll get slightly different numbers, but they're all roughly between .20 and .36.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, change the value of `lift` so that the proportion of significant tests is equal to the **false positive rate** and press "Run" once more.

Note that the proportion of significant tests should be approximately equal to the significance threshold if you've done this correctly.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 100
lift = 0
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
    p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
    name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)
```

```
# run the test
ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
result = ('significant' if pval < significance_threshold else 'not significant')

# append the result to our results list:
results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)
```

Trade Offs

At this point, let's return to the point of view of a product manager who is actually planning this A/B test. Suppose that the product manager wants to be able to accurately detect a lift of 30% (or higher), but also wants to avoid false positives (they don't want to change the email subjects unless there's actually a difference between them). To plan their test, the product manager needs to consider the following:

Increasing the sample size increases the power of the test (the probability of detecting a difference if there **is** one); however, larger sample sizes require more time and resources.

Increasing the significance threshold also increases the power of the test; however, it simultaneously increases the false positive rate (the probability of detecting a difference when there **isn't** one).

Finally, if the project manager chooses a larger minimum detectable effect/lift, then they'll be able to decrease the sample size without decreasing power. However, if they set up their test to detect a minimum lift of 30% (for example), they may not be able to detect smaller differences that are still meaningful.

Instructions

Checkpoint 1 Enabled

1.

The simulation code from the previous exercises is provided for you in **script.py**. Currently, the simulation is set up to use an open rate of 50% for the control email, and a lift of 30% for the name email subject. Set the sample size of 100 and press "Run" and make note of the proportion of significant results (which is the power of the test).

Hint

Make sure the sample size variable, `sample_size`, is set to 100.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now increase the sample size to 500 and press "Run" again. Note that the power of the test also increases.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, increase the significance threshold to 0.10. Note that the power of the test increases even more.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Finally, increase the lift to 40%. Note that again, the power of the test increases.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.10
sample_size = 500
```

```

lift = .4
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list:
    results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)

```

Review

Congratulations! You've now learned how to use simulation to investigate the trade-offs for an A/B test sample-size calculation. As a recap, this lesson covered the following:

- The significance threshold for a test is equal to the false positive rate
- The power of a test is the probability of correctly detecting a significant result
- Increasing sample size increases the power of a test
- Increasing the significance threshold increases power, but also increases the false positive rate
- Larger sample sizes are needed to detect smaller effect sizes

Two notes about the terminology in the sample size calculator:

Baseline conversion rate is equivalent to our `control_rate` in the code.

Minimum detectable effect (MDE) is the smallest effect size (or `lift`) that we want our test to be able to detect. If the MDE is *larger* than our true `lift`, power will decrease because our sample size might not be large enough to detect the difference between the two groups.

```

import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 180#change the sample size here
lift = .3

```

```

control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list:
    results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)

```

Introduction to Machine Learning Foundations for Data Science

Get ready to dive into the world of machine learning!

All the pieces are in place, and you are equipped with a solid understanding of Python, math, and statistics, and ready to dive into the world of machine learning. It's a huge accomplishment to get to this point, and we're sure you are excited, so let's see what you are going to learn about in this unit!

What will Machine Learning Foundations cover?

After this unit, you will be able to:

- Explain what machine learning is (and isn't)
- Distinguish between supervised and unsupervised classification
- Analyze features in light of their role in machine learning

- Transform datasets and prepare them for machine learning

Introduction to Machine Learning

The year is 2049...

Neo York is overrun by bots and web crawlers. The capabilities of

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[Machine Learning](#)

have reached new heights and the world as we know it will never be the same.

Facial recognition technology that helps users tag and share photos of friends can now tag *future* friends; night drones are on the prowl.

Machine learning powered self-driving cars (and flying cars) are now massively available to consumers. The steering wheel has become a thing of the past.

Recommendation engines that suggest what VR shows to watch and what products to buy will now display a different environment for each user group.

At the dawn of a new age, you can't help but wonder, what is Machine Learning and how did it pivot our world so drastically?

What is Machine Learning?

While at IBM, [Arthur Samuel](#) developed a program that learned how to play checkers (1959). He called it:

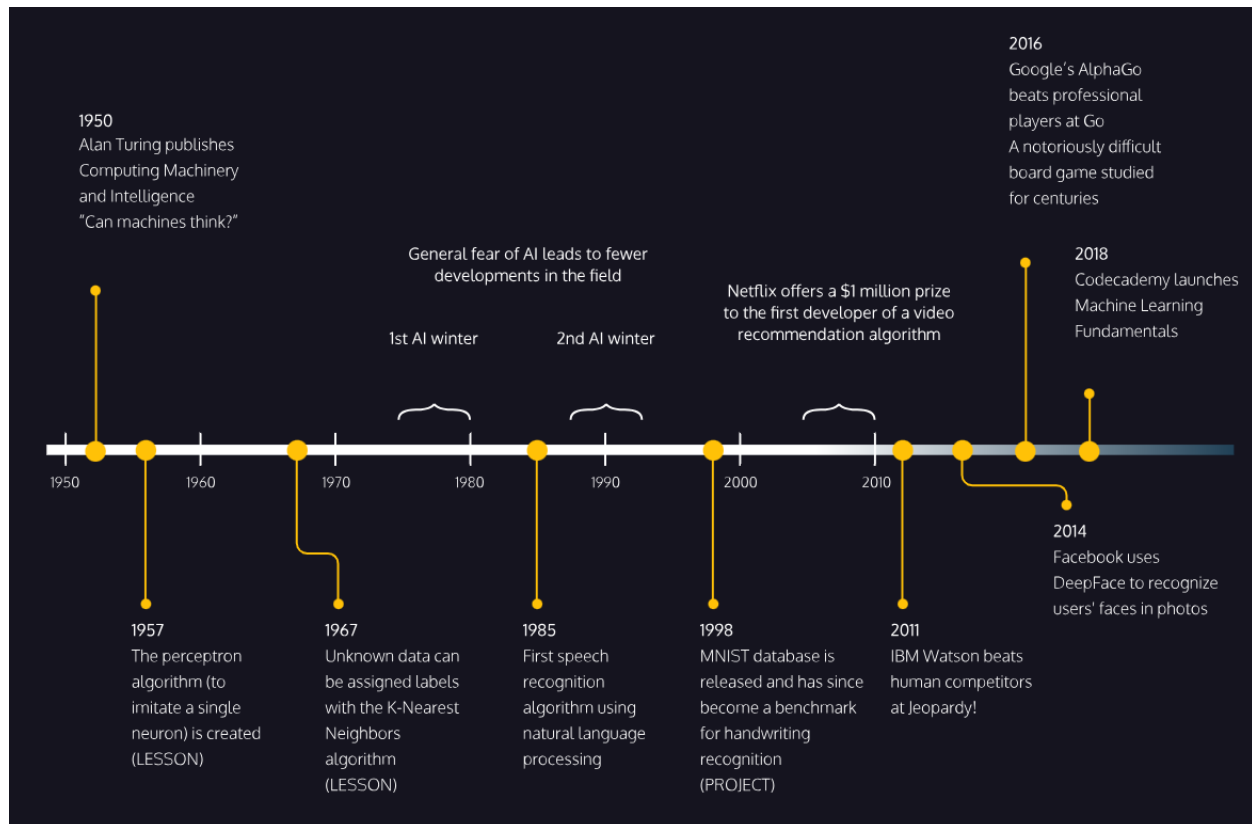
"The field of study that gives computers the ability to learn without being explicitly programmed."

What does this mean?

As programmers, we often approach problems in a methodical, logic-based way. We try to determine what our desired outputs should be, and then create the proper rules that will transform our inputs into those outputs.

Machine learning flips the script. We want the program itself to learn the rules that describe our data the best, by finding patterns in what we know and applying those patterns to what we don't know.

These algorithms are able to *learn*. Their performance gets better and better with each iteration, as it uncovers more hidden trends in the data.



Supervised Learning: Regression

Machine learning can be branched out into the following categories:

- Supervised Learning
- Unsupervised Learning

Supervised Learning is where the data is labeled and the program learns to predict the output from the input data. For instance, a supervised learning

Preview: Docs Loading link description
[algorithm](#)

for credit card fraud detection would take as input a set of recorded transactions. For each
 Preview: Docs Refers to an encapsulated set of instructions sent to a database that must
 happen as a unit.

[transaction](#)

, the program would predict if it is fraudulent or not.

Supervised learning problems can be further grouped into regression and classification problems.

Regression:

In regression problems, we are trying to predict a continuous-valued output. Examples are:

What is the housing price in New York?
What is the value of cryptocurrencies?

Classification:

In classification problems, we are trying to predict a discrete number of values. Examples are:

Is this a picture of a human or a picture of a cyborg?
Is this email spam?

For a quick preview, here's an example of a regression problem.

A real estate company wants to analyze housing costs in New York. They built a linear regression model to predict rent prices from two variables: the square footage of each apartment and the number of burglaries in the apartment's neighborhood during the past year.

Instructions

Checkpoint 1 Enabled

1.

Run the code to train the model and predict the monthly rent for a 950-square-foot apartment in a neighborhood with 2 burglaries in the last year.

Hint

This code is already written for you in **script.py**.

We will learn about linear regression in a later lesson!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Change the variables `square_footage` and `number_of_burglaries` to different values. Run the code again. How do those changes affect the predicted rent?

If you want to experiment more you can change the values to other numbers and see how that affects the predicted rent.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load the data
housing_data = pd.read_csv('housing_data.csv')
X = housing_data[['Sq ft', 'Burglaries']]
y = housing_data['Rent']

# Create the model
reg = LinearRegression()

# Train the model
reg.fit(X, y)

square_footage = 950
number_of_burglaries = 1

y_pred = reg.predict(np.array([square_footage, number_of_burglaries]).reshape(1, 2))

print(y_pred)
```

Supervised Learning: Classification

Now that you've seen a regression example, let's take a look at a classification example.

An exclusive nightclub in Neo York doesn't want to serve robots, but technology has advanced so far that it's hard for bouncers to tell humans from robots just by looking. To help the bouncers, the nightclub created a model that uses the k-nearest neighbors

Preview: Docs Loading link description

[algorithm](#)

to distinguish between humans and robots based on how long it takes them identify blurry pictures or traffic lights.

Instructions

Checkpoint 1 Enabled

1.

Run the code to train the model and see whether someone who identifies a picture in 5 seconds can pass as a human.

Hint

This code is already written for you in **script.py**.

We will learn about k-nearest neighbors in a later lesson!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Change the variable `time_to_identify_picture` to a different value. Run the code again. Does the model still think that you're a human?

If you want to experiment more you can change the value again and see how that affects the outcome.

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

# Load the data
photo_id_times = pd.read_csv('photo_id_times.csv')

# Separate the data into independent and dependent variables
X = np.array(photo_id_times['Time to id photo']).reshape(-1, 1)
y = photo_id_times['Class']

# Create a model and fit it to the data
neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(X, y)

time_to_identify_picture = 6.5

# Make a prediction based on how long it takes to identify a picture
y_pred = neigh.predict(np.array(time_to_identify_picture).reshape(1, -1))

if y_pred == 1:
    print("We think you're a robot.")
else:
    print("Welcome, human!")
```

Unsupervised Learning

[Unsupervised Learning](#) is a type of

Preview: Docs Loading link description

[machine learning](#)

where the program learns the inherent structure of the data based on unlabeled examples.

Clustering is a common unsupervised machine learning approach that finds patterns and structures in unlabeled data by grouping them into clusters.

Some examples:

- Social networks clustering topics in their news feed
- Consumer sites clustering users for recommendations
- Search engines to group similar objects in one cluster

For a quick preview, here's an example of unsupervised learning.

A social media platform wants to separate their users into categories based on what kind of content they engage with. They have collected three pieces of data from a sample of users:

- Number of hours per week spent reading posts
- Number of hours per week spent watching videos
- Number of hours per week spent in virtual reality

The company is using an

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

called k-means clustering to sort users into three different groups.

Begin Your Journey

DING DING DING!

It was but a dream and there's still time. Understanding

Preview: Docs Loading link description

[Machine Learning](#)

will help you change this bleak future, or be ready for it.

As you begin your journey through our Machine Learning curriculum, you will learn the different applications of this powerful field. You will build your own models, test them, and try to improve them. You will analyze bigger, and more complex data and deliver faster, more accurate results — even on a very large scale using:

- Supervised Learning**
 - Regression**
 - Classification**
- Unsupervised Learning**

Throughout your exploration of ML, we hope you gain an understanding of how to harness predictive models to do good, and enhance human life rather than replace it!

If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) then join us here!

Supervised vs. Unsupervised Learning: what's the difference?

Learn the key differences between supervised and unsupervised learning in machine learning, with real-world examples.

As humans, we have many different ways we learn things. The way you learned calculus, for example, is probably not the same way you learned to stack blocks. The way you learned the alphabet is probably wildly different from the way you learned how to tell if objects are approaching you or going away from you. The latter you might not even realize you learned at all!

Similarly, when we think about making programs that can learn, we have to think about these programs learning in different ways. Two main ways that we can approach machine learning are **Supervised Learning** and **Unsupervised Learning**. Both are useful for different situations or kinds of data available.

What is supervised learning?

Let's imagine you're first learning about different genres in music. Your music teacher plays you an indie rock song and says "This is an indie rock song". Then, they play you a K-pop song and tell you "This is a K-pop song". Then, they play you a techno track and say "This is techno". You go through many examples of these genres.

The next time you're listening to the radio, and you hear techno, you may think "This is similar to the 5 techno tracks I heard in class today. This must be techno!"

Even though the teacher didn't tell you about *this* techno track, she gave you enough examples of songs that were techno, so you could recognize more examples of it.

When we explicitly tell a program what we expect the output to be, and let it learn the rules that produce expected outputs from given inputs, we are performing supervised learning.

A common example of this is **image classification**. Often, we want to build systems that will be able to describe a picture. To do this, we normally show a program thousands of examples of pictures, with labels that describe them. During this process, the program adjusts its internal parameters. Then, when we show it a new example of a photo with an unknown description, it should be able to produce a reasonable description of the photo.

When you complete a [Captcha](#) and identify the images that have cars, you're labeling images! A supervised machine learning algorithm can now use those pictures that you've tagged to make it's car-image predictor more accurate.

What is unsupervised learning?

Let's say you are an alien who has been observing the meals people eat. You've embedded yourself into the body of an employee at a typical tech startup, and you see people eating breakfasts, lunches, and snacks. Over the course of a couple weeks, you surmise that for breakfast people mostly eat foods like:

- Cereals
- Bagels
- Granola bars

Lunch is usually a combination of:

- Some sort of protein
- Some sort of grain

Snacks are usually a piece of fruit or a handful of nuts. No one explicitly *told* you what kinds of foods go with each meal, but you learned from natural observation and put the patterns together. In unsupervised learning, we don't tell the program anything about what we expect the output to be. The program itself analyzes the data it encounters and tries to pick out patterns and group the data in meaningful ways.

An example of this includes **clustering** to create segments in a business's user population. In this case, an unsupervised learning algorithm would probably create groups (or clusters) based on parameters that a human may not even consider.

Conclusion

We have gone over the difference between supervised and unsupervised learning:

Supervised Learning: Data is labeled, and the program learns to predict the output from the input data

Unsupervised Learning: Data is unlabeled, and the program learns to recognize the inherent structure in the input data

EDA Prior To Fitting a Regression Model

Learn about recommended EDA steps before fitting a regression model.

Introduction

Before fitting any model, it is often important to conduct an exploratory data analysis (EDA) in order to check assumptions, inspect the data for anomalies (such as missing, duplicated, or mis-coded data), and inform feature selection/transformation. In this article, we will use [pandas](#) to explore some of the EDA techniques that are generally employed prior to fitting a regression model.

The data

For our example analysis, we've downloaded a dataset from [Kaggle](#) which contains data on Major League Baseball (MLB) games from the 2016 season. We've saved this data as a [DataFrame](#) named `bb`. Suppose that we want to fit a linear regression to predict `attendance` using the following predictors:

`game_type` — is the game during the day or at night?
`day_of_week` — what day of the week did the game occur?
`temperature` — average game temperature (Fahrenheit).
`sky` — description of sky condition at the time of the game.
`total_runs` — total runs scored in the game.

Preview the dataset

Any EDA process will probably begin by inspecting a subset of data. For a pandas [DataFrame](#), this can be done by using the `.head()` method:

```
bb.head()
```

	attendance	game_type	day_of_week	temperature	sky	total_runs
0	40030.0	Night Game	Sunday	74	Sunny	7
1	21621.0	Night Game	Wednesday	55	Overcast	5
2	12622.0	Night Game	Wednesday	48	Unknown	6
3	18531.0	Night Game	Wednesday	65	Cloudy	4
4	18572.0	Day Game	Wednesday	77	In Dome	7

By looking at the first few rows of the data, we can often figure out what kind of data we have (eg., discrete or continuous) and get a sense of how they are coded. For example, we can see that `attendance`, `temperature`, and `total_runs` are numbers, while `game_type`, `day_of_week`, and `sky` appear to be text.

After our initial inspection, we'll want to dig deeper to investigate the following:

The data type of each variable.
How discrete/categorical data is coded (and whether we need to make any changes).
How the data are scaled.

Whether there is missing data and how it is coded.
Whether there are outliers.
The distributions of continuous features.
The relationships between pairs of features.

Data types

It is important to check the data type for each feature. The quantitative variables should be read in as numbers — either `int64` or `float64` — and categorical variables should be stored as strings (columns of `strings` have a `dtype` of `object` because of how they are stored in Python). We can check data types of columns in a pandas `DataFrame` using the `.dtypes` property.

```
bb.dtypes
```

The output will look like this:

```
attendance    float64
game_type     object
day_of_week   object
temperature   object
sky           object
total_runs    object
dtype: object
```

From this output, we can see that `temperature` and `total_runs` are both quantitative variables but they were read in as `object` dtypes. This can happen when there is a non-numeric character — such as a letter or punctuation symbol — in the same column. We would need to explore further in order to figure out what's going on. For example, we might inspect a different set of rows and see the following:

	attendance	game_type	day_of_week	temperature	sky	total_runs
5	12757.0	Night Game	Tuesday	72	In Dome	5
6	28329.0	Night Game	Tuesday	70	Unknown	-
7	26049.0	Night Game	Tuesday	Unknown	Sunny	11
8	10478.0	Night Game	Tuesda	70	NaN	9
9	47820.0	Day Game	Tuesday	36	Sunny	8

We note that `temperature` has missing data coded as `Unknown` rather than `NaN`. If we fit a regression on this data as is, we will end up treating temperature as a categorical variable and therefore fitting separate slopes for every value of temperature; instead, we probably want a single slope. To fix this, we'll need to replace every `Unknown` with some other value (or remove them from the data altogether) and recode the temperature column as an `int`.

Categorical encoding

EDA is also important during the feature engineering process in order to inform decisions around categorical encoding. This is important because categorical features with many levels are “expensive” to include in a regression model (we need to calculate a separate slope for each level). If one of the levels has only a few observations, we might want to delete those records from the data before fitting the model. We can check this using `.value_counts()`:

```
bb['game_type'].value_counts(dropna=False)
```

Output:

```
Night Game    1664
Day Game       799
Name: game_type, dtype: int64
```

Based on the output, we can see here that there are two levels for `game_type`; about one-third of games are day games and two-thirds are night games.

The `.value_counts()` accessor can also illuminate other issues. For example, in the following output, we notice that one instance of `'Tuesday'` was miscoded as `Tuesda`. This can either be corrected or removed before proceeding with a regression model.

```
bb['day_of_week'].value_counts(dropna=False)
```

Output:

```
Saturday    396
Friday      394
Sunday      392
Wednesday   379
Tuesday     375
Monday      278
Thursday    248
Tuesda       1
Name: day_of_week, dtype: int64
```

There are a few different options for how we might want to code the `day_of_week` variable. If attendance increases approximately linearly throughout the week, we might argue that `day_of_week` is ordinal and code it as an `int` in our model. However, attendance goes up and down throughout the week, we're better off leaving it as an unordered category (`str`). Finally, if we see that games on Friday-Sunday simply have higher attendance than other days of the week, we might re-code this feature to only have two levels: `Weekend` and `Weekday`. We can check this by using boxplots:

We can see here that attendance on Friday, Saturday, and Sunday is on average higher than the other days of the week. Therefore it may be beneficial to re-code this feature to either `Weekend` or `Weekday`.

Scaling

For quantitative features, it is important to think about how each feature is scaled. Some features will be on vastly different scales than others just based on the nature of what the feature is measuring. For example, let's look at `temperature` and `total_runs` using the `.describe()` method:

```
bb.describe()
```

The output will look like this:

```
count      attendance      temperature      total_runs
mean      30380.462352      73.834959      8.949187
std       9874.626652      10.567219      4.579542
min       8766.000000      31.000000      1.000000
25%      22437.000000      67.000000      6.000000
50%      30628.000000      74.000000      8.000000
75%      38412.000000      81.000000     12.000000
max       54449.000000     101.000000     60.000000
```

These two features are on different scales because what they are measuring are different (`temperature` is in degrees Fahrenheit, `total_runs` is the number of runs scored in a game). Because of this, the ranges of values and the standard deviations for each are very different from one another. We can see here that `temperature` has a standard deviation of about 10.57, while `total_runs` has a standard deviation of about 4.58.

When working with features with largely differing scales, it is often a good idea to *standardize* the features so that they all have a mean of 0 and a standard deviation of 1.

A feature without any values close to zero may also make it more difficult to estimate and interpret the intercept of a regression model. Standardizing or otherwise re-scaling the feature can fix this issue.

Missing data

When we initially inspected the data, we saw some evidence that missing data is coded in a few different ways:

	attendance	game_type	day_of_week	temperature	sky	total_runs
5	12757.0	Night Game	Tuesday	72	In Dome	5
6	28329.0	Night Game	Tuesday	70	Unknown	-
7	26049.0	Night Game	Tuesday	Unknown	Sunny	11
8	10478.0	Night Game	Tuesda	70	NaN	9
9	47820.0	Day Game	Tuesday	36	Sunny	8

For example, `temperature` uses the term `Unknown`, `sky` uses both `Unknown` and `NaN`, and `total_runs` has `-` to represent a missing value. The observations with missing values will either have to be removed or replaced (with an imputed value or missing data type that Python can recognize, such as `np.NaN`) in order to proceed with fitting a regression model.

Outliers

In our EDA, it is important to check for outliers and skew in the data. One way to check for outliers is to use scatter plots:

```
bb.plot.scatter(x = 'total_runs', y = 'attendance')
```

We can see here that there is one instance where the total runs in a single game is about 60, which is much larger than in the other games. Depending on the situation, we may first want to verify that this value is correct, then we can decide whether or not to remove it prior to fitting the model.

Distributions and associations

Prior to fitting a linear regression model, it can be important to inspect the distributions of quantitative features and investigate the relationships between features. We can visually inspect both of these by using a pair plot:

Looking at the histograms along the diagonal, `total_runs` appears to be somewhat right-skewed. This indicates that we may want to transform this feature to make it more normally distributed.

We can explore the relationships between pairs of features by looking at the scatterplots off of the diagonal. This is useful for a few different reasons. For example, if we see non-linear associations between any of the predictors and the outcome variable, that might lead us to test out polynomial terms in our model. We can also get a sense for which features are most highly related to our outcome variable and check for collinearity. In this example, there appears to be a slight positive linear association between temperature and the total number of runs. We can further investigate this using a heat map of the correlation matrix:

There is a correlation of 0.35 between temperature and the total number of runs. This is not large enough to cause concern; however, if two or more predictors are highly correlated, we may consider leaving only one in our analysis. On the other hand, features that are highly correlated with our outcome variable are especially important to include in the model.

Conclusion

Let's review the ways we were able to explore this data set in preparation for a regression model:

- We previewed the first few rows of the data set using the `.head()` method.

- We checked the data type of each variable in the data set using `.dtypes` and corrected variables with incorrect data types.

- We investigated our categorical data to inform categorical encoding.

- We investigated the scale of our quantitative variables and considered whether standardizing/scaling might be appropriate.

- We investigated missing data.

- We checked for outliers.

We inspected the distributions of our quantitative variables.

We looked at the relationships between pairs of features using both scatter plots and box plots.

By going through these steps, we are more prepared to make decisions about feature selection/engineering and have learned valuable information about how to build a more accurate predictive model.

EDA Prior to Fitting a Classification Model

Learn about recommended EDA steps before fitting a classification model.

Introduction

Similar to regression models, it is important to conduct EDA before fitting a classification model. An EDA should check the assumptions of the classification model, inspect how the data are coded, and check for strong relationships between features. In this article, we will explore some of the EDA techniques that are generally employed prior to fitting a classification model.

Data

Suppose we want to build a model to predict whether a patient has heart disease or not based on other characteristics about them. We have downloaded a dataset from the [UCI Machine Learning Repository](#) about heart disease which contains patient information such as:

age

sex

cp

trestbps

chol

fbs

restecg

thalach

exang

oldpeak

```
slope
ca
thal
```

The response variable for this analysis will be `heart_disease`, which we have condensed down to either 0 (if the patient does not have heart disease) or 1 (the patient does have heart disease).

EDA is extremely useful to better understand which patient attributes are highly related to heart disease, and ultimately to build a classification model that can accurately predict whether someone has heart disease based on their measurements. By exploring the data, we may be able to see which variables — or which combination of variables — provide the most information about whether or not the patient has heart disease.

Preview the data

Similar to EDA prior to a regression model, it is good to begin EDA with inspecting the first few rows of data:

```
print(heart.head())
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	heart_disease
0	63	1	1	145	233	1	2	150	0	2.3	3	0	6	0
1	67	1	4	160	286	0	2	108	1	1.5	2	3	3	2
2	67	1	4	120	229	0	2	129	1	2.6	2	2	7	1
3	37	1	3	130	250	0	0	187	0	3.5	3	0	3	0

```

4      4      0      2      130      20      0      2      172      0      1.4      1      0      3      0
1      1

```

By looking at the first rows of data, we can note that all of the columns appear to contain numbers. We can quickly check for missing values and data types by using `.info()`:

```
print(heart.info())
```

Output:

```

Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  -
0   age                 303 non-null    int64
1   sex                 303 non-null    int64
2   cp                  303 non-null    int64
3   trestbps            303 non-null    int64
4   chol                303 non-null    int64
5   fbs                 303 non-null    int64
6   restecg             303 non-null    int64
7   thalach              303 non-null    int64
8   exang               303 non-null    int64
9   oldpeak             303 non-null    float64
10  slope               303 non-null    int64
11  ca                  303 non-null    object
12  thal                303 non-null    object
13  heart_disease       303 non-null    int64
dtypes: float64(1), int64(11), object(2)

```

We can see that all columns have a count of “303 non-null” values, meaning there are no blank spaces in the dataset. However, there can still be other ways that missing data can be hiding in the data. For example, `ca` and `thal` are `object` data types, indicating that there is at least one character in each of these columns which are preventing the variable from being read as a numeric data type. This could be either an input mistake (such as the letter “o” in place of a “0”), or it can be an indication of how missing data were handled. Depending on which model program is used, you may have to find and remove the observations with characters before proceeding with the model.

We also want to make sure to check how categorical data is encoded before proceeding with model fitting. For example, `cp` is the patient's chest pain type and is indicated by a number between 1 and 4. These numbers are intended to be treated as groups, so this variable should be changed into an object before continuing into the analysis.

Pair plot

We can explore the relationships between the different numeric variables using a pair plot. If we also color the observations based on heart disease status, we can simultaneously get a sense for a) which features are most associated with heart disease status and b) whether there are any pairs of features that are jointly useful for determining heart disease status:

In this pair plot, we are looking for patterns between the two color groups. Looking at the density plots along the diagonal, there are no features that cleanly separate the groups (age has the most separation). However, looking at the scatterplot for `age` and `thalach` (maximum heart rate from an exercise test), there is more clear separation. It appears that patients who are old and have low `thalach` are more likely to be diagnosed with heart disease than patients who are young and have high `thalach`. This suggests that we want to make sure both of these features are included in our model.

Correlation heat map

Similar to linear regression, some classification models assume no multicollinearity in the data, meaning that two highly correlated predictors should not be included in the model. We can check this assumption by looking at a correlation heat map:

There is no set value for what counts as “highly correlated”, however a general rule is a correlation of 0.7 (or -0.7). There are no pairs of features with a correlation of 0.7 or higher, so we do not need to consider leaving any features out of our model based on multicollinearity.

Further exploration

You can use more complex visualizations to examine the relationships between 2 or more features and the response variable at the same time. For example, the following boxplots show the relationship between `oldpeak`, `slope`, and `heart_disease`:



In this boxplot, we can see a pretty distinct difference between those with heart disease and those without at slope level 3. Seeing this distinction indicates that on average `oldpeak` is connected to heart disease at different `slope` levels. This gives insight that it might be beneficial to include an interaction term between `oldpeak` and `slope` in a linear regression model.

Classification model results

After this EDA, we can run a principal component analysis (PCA), which attempts to identify which features (or combination of features) are highly related to heart disease. Ideal results of a PCA show one or more pairs of principal components with some separation between the colored groups:

We can see here that there are not any clear separations, which would indicate that this is not an effective analysis. However, we might use the weights of the components to further explore relationships between features and use that in other analyses.

Conclusion

Exploring the data in the ways outlined above will help prepare you to build an effective classification model. These steps ensure that the data is properly coded and can be useful for both feature selection and model tuning.

EDA Prior to Fitting a Classification Model

Learn about recommended EDA steps before fitting a classification model.

Introduction

Similar to regression models, it is important to conduct EDA before fitting a classification model. An EDA should check the assumptions of the classification model, inspect how the data are coded, and check for strong relationships between features. In this article, we will explore some of the EDA techniques that are generally employed prior to fitting a classification model.

Data

Suppose we want to build a model to predict whether a patient has heart disease or not based on other characteristics about them. We have downloaded a dataset from the [UCI Machine Learning Repository](#) about heart disease which contains patient information such as:

age: age in years
sex: male (1) or female (0)
cp: chest pain type
trestbps: resting blood pressure (mm Hg)
chol: cholesterol level
fbs: fasting blood sugar level (normal or not)
restecg: resting electrocardiograph results
thalach: maximum heart rate from an exercise test
exang: presence of exercise-induced angina
oldpeak: ST depression induced by exercise relative to rest
slope: slope of peak exercise ST segment
ca: number of vessels colored by flourosopy (0 through 3)
thal: type of defect (3, 6 or 7)

The response variable for this analysis will be `heart_disease`, which we have condensed down to either 0 (if the patient does not have heart disease) or 1 (the patient does have heart disease).

EDA is extremely useful to better understand which patient attributes are highly related to heart disease, and ultimately to build a classification model that can accurately predict whether someone has heart disease based on their measurements. By exploring the data, we may be able to see which variables — or which combination of variables — provide the most information about whether or not the patient has heart disease.

Preview the data

Similar to EDA prior to a regression model, it is good to begin EDA with inspecting the first few rows of data:

```
print(heart.head())
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	heart_disease
0	63	1	1	145	233	1	2	150	0	2.3	3	0	6	0
1	67	1	4	160	286	0	2	108	1	1.5	2	3	3	2
2	67	1	4	120	229	0	2	129	1	2.6	2	2	7	1
3	37	1	3	130	250	0	0	187	0	3.5	3	0	3	0
4	41	0	2	130	204	0	2	172	0	1.4	1	0	3	0

By looking at the first rows of data, we can note that all of the columns appear to contain numbers. We can quickly check for missing values and data types by using `.info()`:

```
print(heart.info())
```

Output:

```
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  -
0   age                 303 non-null    int64
1   sex                 303 non-null    int64
2   cp                  303 non-null    int64
3   trestbps            303 non-null    int64
4   chol                303 non-null    int64
5   fbs                 303 non-null    int64
6   restecg             303 non-null    int64
7   thalach             303 non-null    int64
8   exang               303 non-null    int64
9   oldpeak             303 non-null    float64
10  slope                303 non-null    int64
```

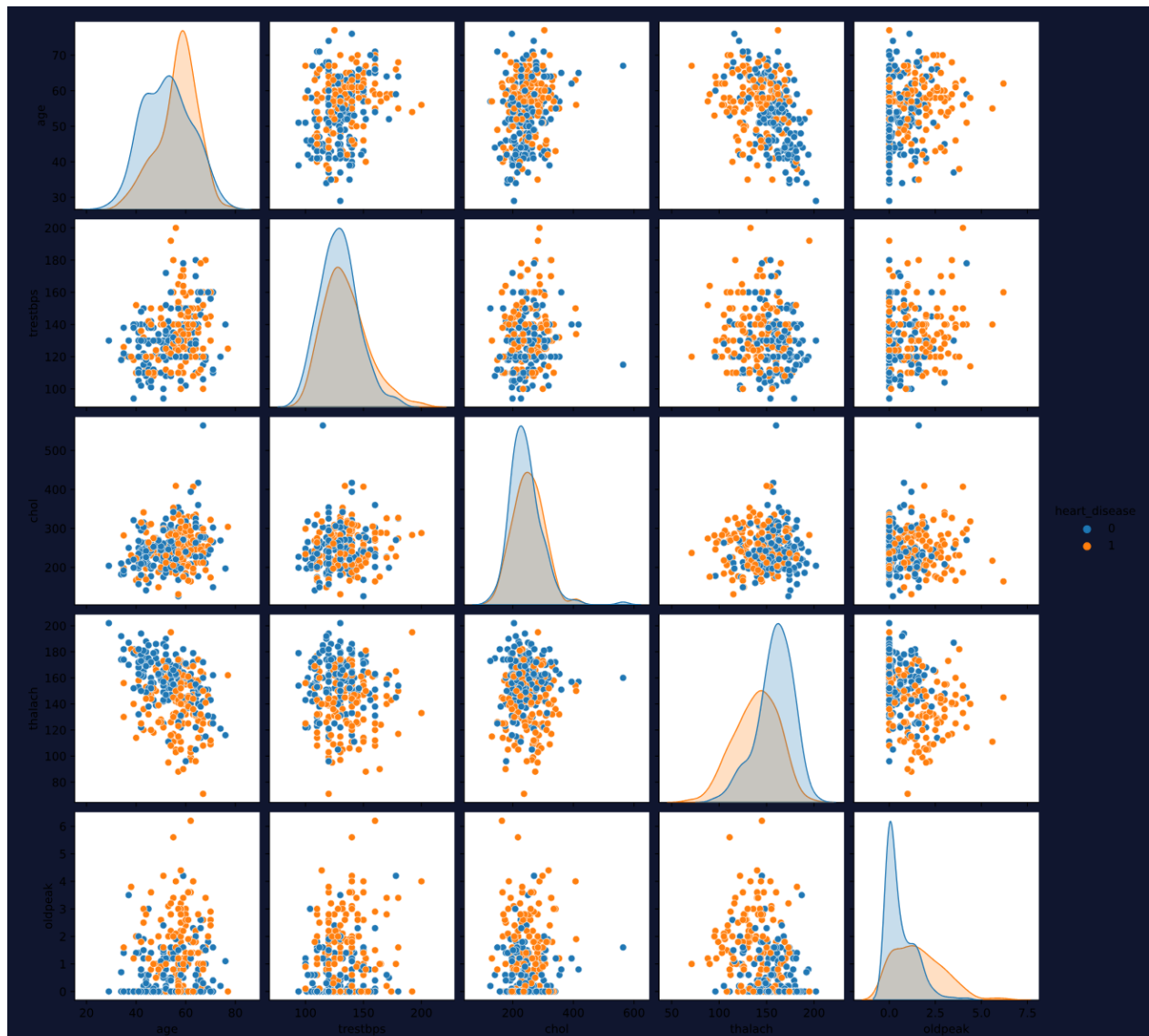
```
11  ca                303 non-null    object
12  thal              303 non-null    object
13  heart_disease     303 non-null    int64
dtypes: float64(1), int64(11), object(2)
```

We can see that all columns have a count of “303 non-null” values, meaning there are no blank spaces in the dataset. However, there can still be other ways that missing data can be hiding in the data. For example, `ca` and `thal` are `object` data types, indicating that there is at least one character in each of these columns which are preventing the variable from being read as a numeric data type. This could be either an input mistake (such as the letter “o” in place of a “0”), or it can be an indication of how missing data were handled. Depending on which model program is used, you may have to find and remove the observations with characters before proceeding with the model.

We also want to make sure to check how categorical data is encoded before proceeding with model fitting. For example, `cp` is the patient’s chest pain type and is indicated by a number between 1 and 4. These numbers are intended to be treated as groups, so this variable should be changed into an object before continuing into the analysis.

Pair plot

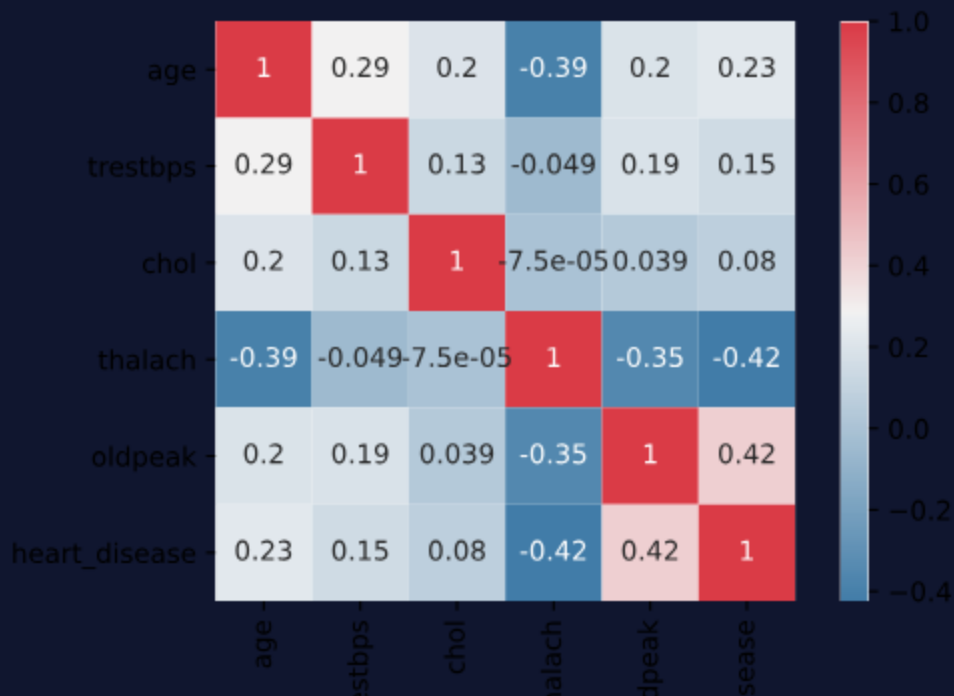
We can explore the relationships between the different numeric variables using a pair plot. If we also color the observations based on heart disease status, we can simultaneously get a sense for a) which features are most associated with heart disease status and b) whether there are any pairs of features that are jointly useful for determining heart disease status:



In this pair plot, we are looking for patterns between the two color groups. Looking at the density plots along the diagonal, there are no features that cleanly separate the groups (age has the most separation). However, looking at the scatterplot for `age` and `thalach` (maximum heart rate from an exercise test), there is more clear separation. It appears that patients who are old and have low `thalach` are more likely to be diagnosed with heart disease than patients who are young and have high `thalach`. This suggests that we want to make sure both of these features are included in our model.

Correlation heat map

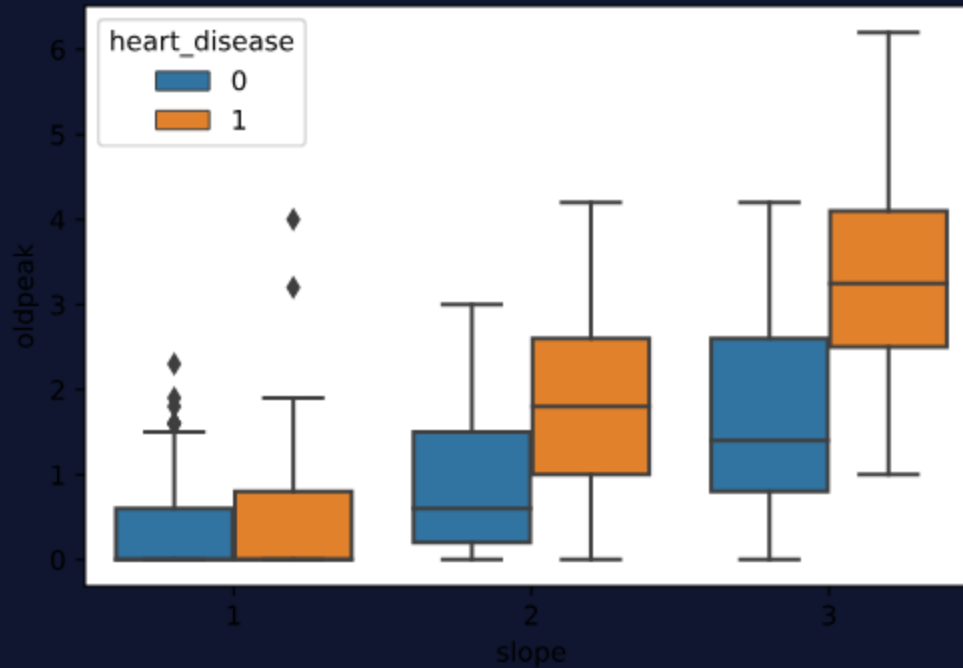
Similar to linear regression, some classification models assume no multicollinearity in the data, meaning that two highly correlated predictors should not be included in the model. We can check this assumption by looking at a correlation heat map:



There is no set value for what counts as “highly correlated”, however a general rule is a correlation of 0.7 (or -0.7). There are no pairs of features with a correlation of 0.7 or higher, so we do not need to consider leaving any features out of our model based on multicollinearity.

Further exploration

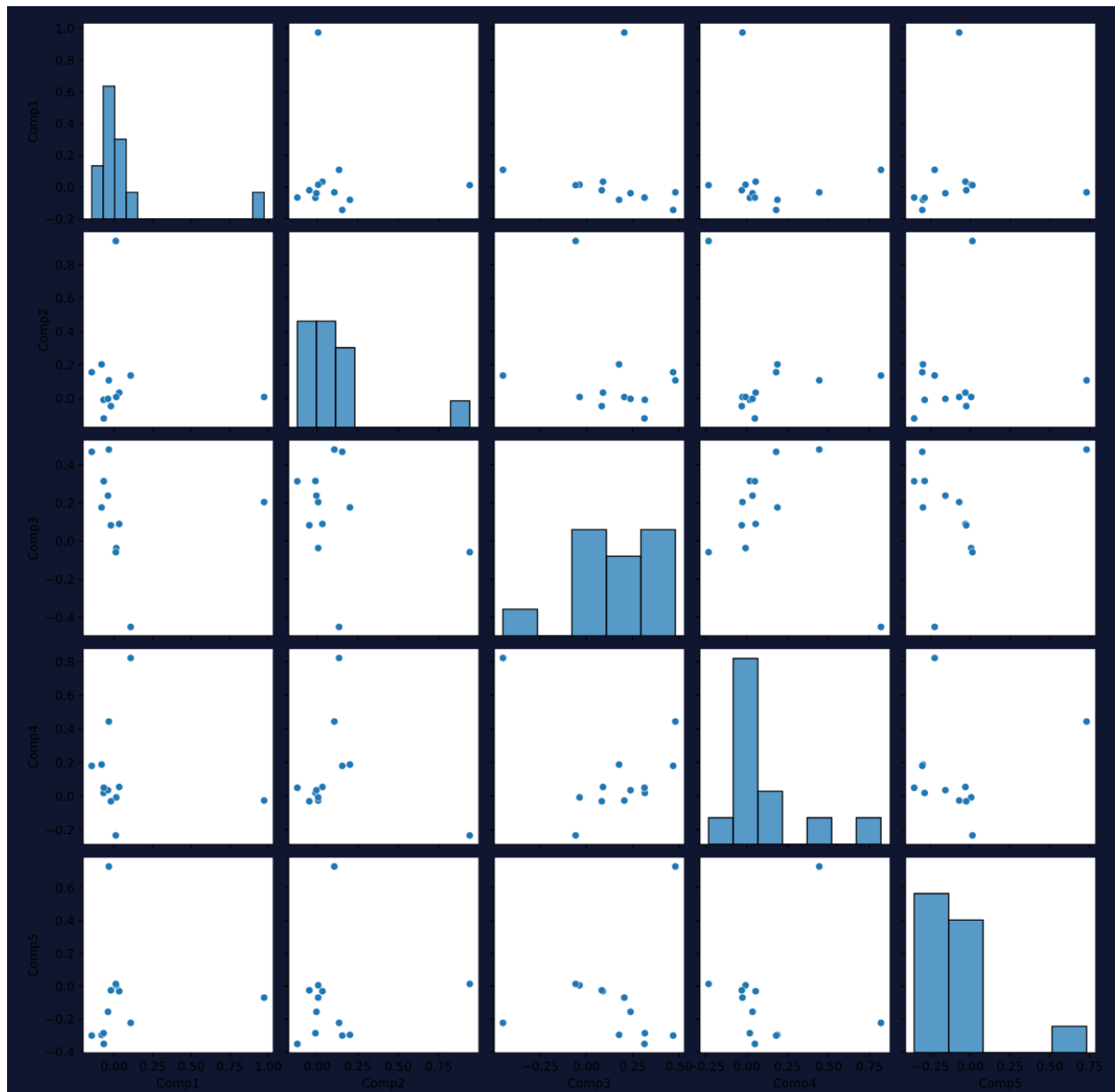
You can use more complex visualizations to examine the relationships between 2 or more features and the response variable at the same time. For example, the following boxplots show the relationship between `oldpeak`, `slope`, and `heart_disease`:



In this boxplot, we can see a pretty distinct difference between those with heart disease and those without at slope level 3. Seeing this distinction indicates that on average `oldpeak` is connected to heart disease at different `slope` levels. This gives insight that it might be beneficial to include an interaction term between `oldpeak` and `slope` in a linear regression model.

Classification model results

After this EDA, we can run a principal component analysis (PCA), which attempts to identify which features (or combination of features) are highly related to heart disease. Ideal results of a PCA show one or more pairs of principal components with some separation between the colored groups:



We can see here that there are not any clear separations, which would indicate that this is not an effective analysis. However, we might use the weights of the components to further explore relationships between features and use that in other analyses.

Conclusion

Exploring the data in the ways outlined above will help prepare you to build an effective classification model. These steps ensure that the data is properly coded and can be useful for both feature selection and model tuning.

EDA Prior to Unsupervised Clustering

Learn the EDA steps that can be helpful prior to creating an unsupervised clustering model.

Introduction

When we want to understand underlying groups for a set of observations but don't know what the group labels should be, we often turn to unsupervised clustering. If we are planning to fit an unsupervised machine learning model, we often want to explore questions such as:

How many groups are there?
How might those groups differ?

In this article, we will demonstrate an exploratory process for beginning to address these questions.

Data

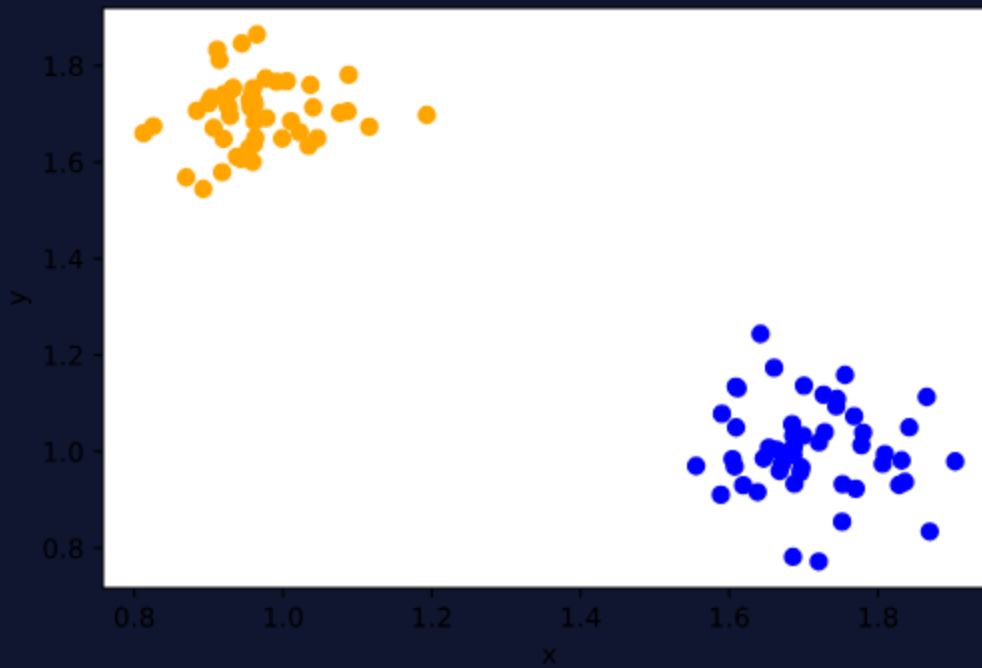
Let's say that you are opening a restaurant and want to be sure to offer a "wide variety" of wines. You get this dataset of chemical differences between types of wines from the [UCI Machine Learning Repository](#) with traits such as:

Alcohol
Malic acid
Ash
Alcalinity of ash
Magnesium
Total phenols
Flavanoids
Nonflavanoid phenols
Proanthocyanins
Color intensity
Hue
OD280/OD315 of diluted wines
Proline

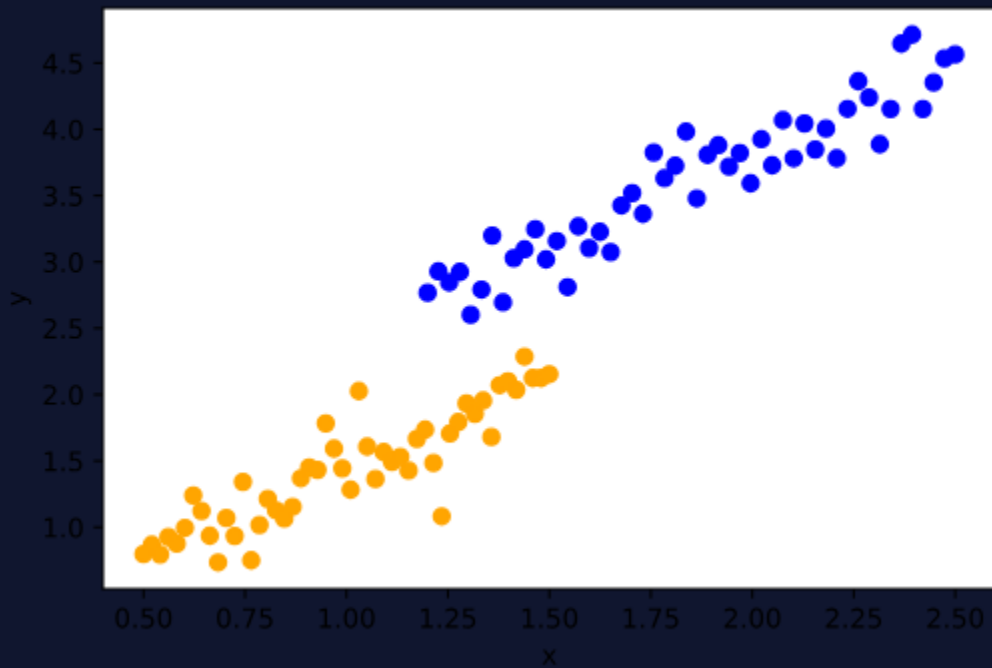
You want to use this information to try to categorize the wine into different groups to ensure that the wines that you decide to buy for the restaurant have a good amount of variety. You want the wines within each group to be similar to each other, and wines in different groups to be less similar. You have no idea how many groups there are or what the group labels should be. So instead, you want to see if the given information has natural groupings based on the characteristics that you have collected data about.

In order to get started, you might need to answer some of the following questions:

For some unsupervised clustering algorithms, you'll need to specify the number of groups ahead of time. Also, different types of algorithms can handle different kinds of groupings more efficiently, so it can be helpful to visualize the shapes of the clusters. For example, k-means algorithms are good at identifying data groups with spherical shapes because they are based on principles of equal variance and distance between data points:



On the other hand, Principal Component Analysis algorithms and BIRCH methods are better at identifying elongated groupings:



There is no method that is the best in every situation. It takes some investigating to know which method will be best for a given set of data.

Prepare the data

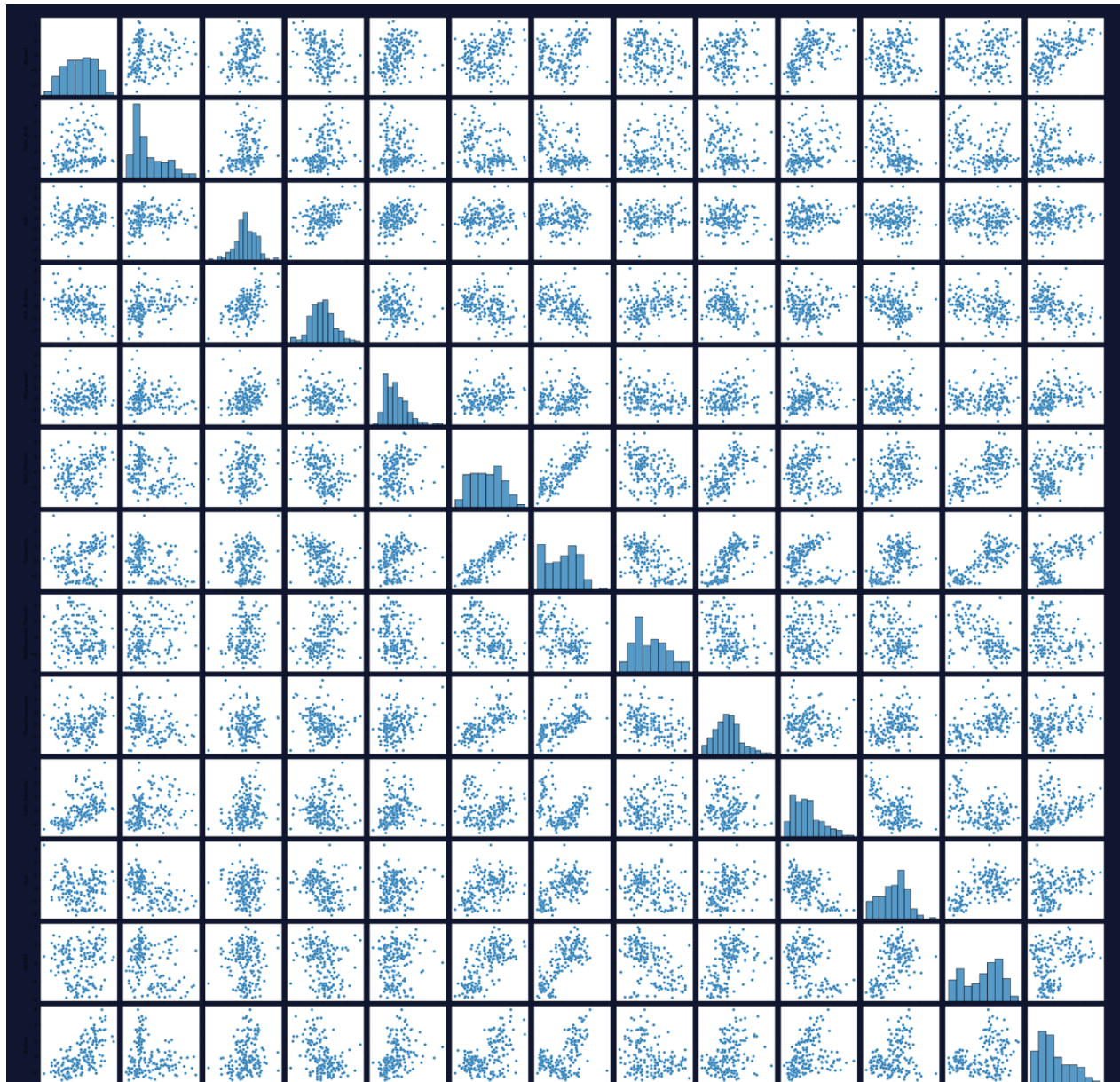
As usual, before we analyze the data, we should go through the process of previewing the data:

	Alc oho l	Mali c_Ac id	A s h	Ash_A lcani ty	Magn esiu m	Total Phenol s	Flav anoi ds	Nonflavan oid_Phen ols	Proanth ocyanin s	Color_I ntensit y	H u e	OD 28 0	Pro lin e
0	14. 23	1.71	2 . 4 3	15.6	127	2.80	3.06	0.28	2.29	5.64	1 . 0 4	3. 92	106 5
1	13. 20	1.78	2 . 1 4	11.2	100	2.65	2.76	0.26	1.28	4.38	1 . 0 5	3. 40	105 0
2	13. 16	2.36	2 . 6 7	18.6	101	2.80	3.24	0.30	2.81	5.68	1 . 0 3	3. 17	118 5
3	14. 37	1.95	2 . 5 0	16.8	113	3.85	3.49	0.24	2.18	7.80	0 . 8 6	3. 45	148 0
4	13. 24	2.59	2 . 8 7	21.0	118	2.80	2.69	0.39	1.82	4.32	1 . 0 4	2. 93	735

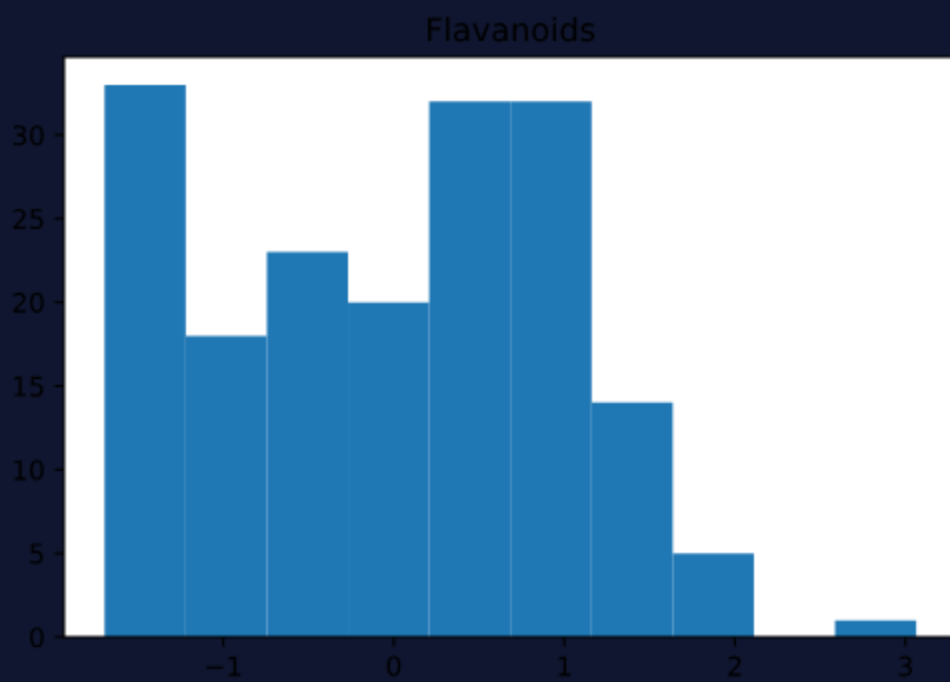
Before continuing with our analysis, we will want to clean and standardize the data, as well as ensure that the variables are coded appropriately as numerical values.

Pairplot to look for clusters

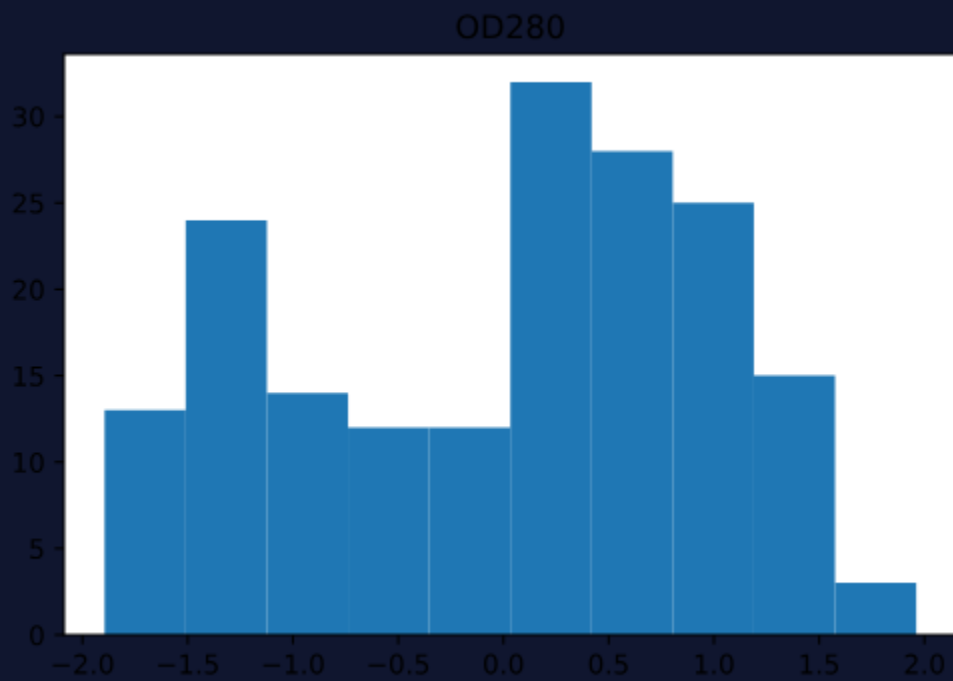
A pairplot of the variables is a good way to look for univariate and bivariate clusters.



Looking at the histograms along the diagonal, some variables have what appear to be bimodal distributions, as indicated by the two peaks. Let's take a closer look at a couple of them:

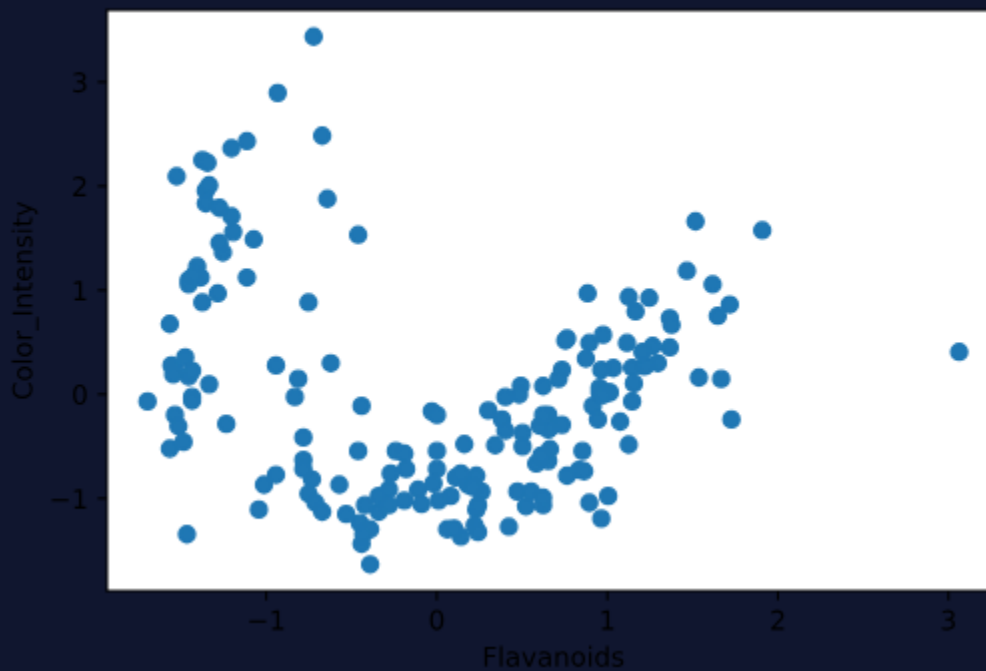


In this histogram of `Flavanoids`, we can see a peak at the very left-hand side as well as between 0 and 1. This is evidence that there could be two groups of wines with respect to `Flavanoids` (low and high flavanoid wines).

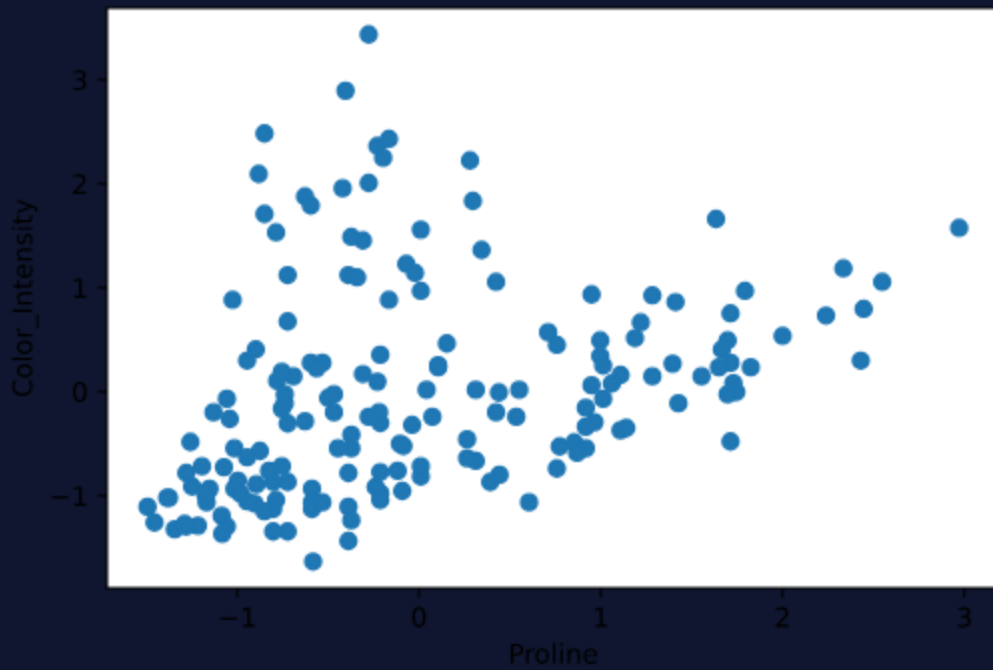


Similarly, the variable of `od280` has two distinct peaks, one between -1.5 and -1, and another between 0 and 0.5. Thus, we have evidence of at least two wine groups based on `od280` as well.

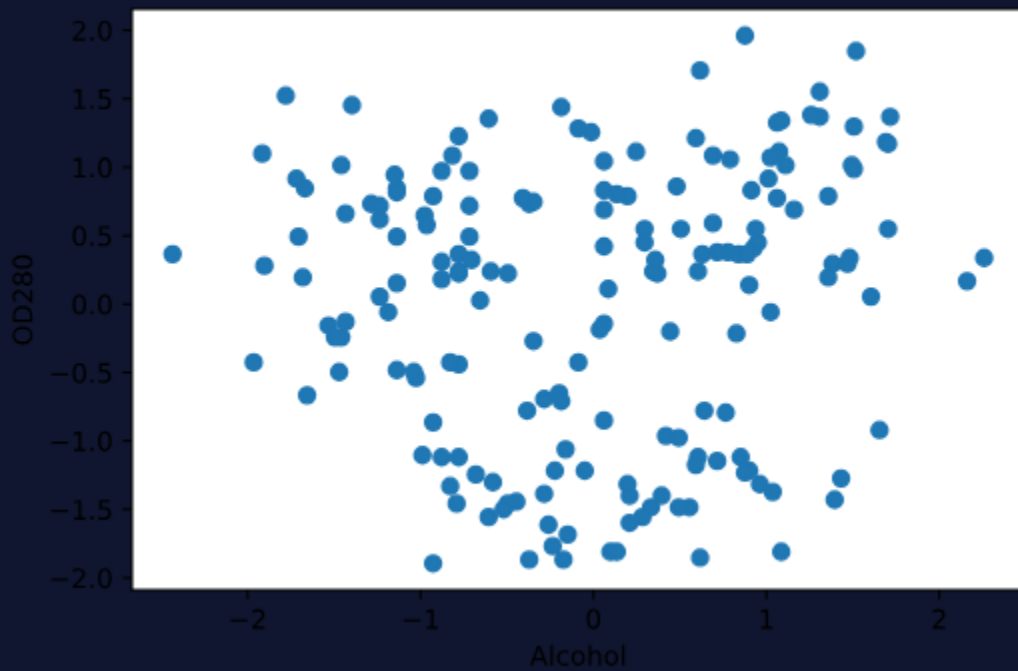
Looking at the rest of the pair plot, it is difficult to see relationships in the scatterplots when there are this many variables. The task of looking at each scatterplot can be tedious, so we have selected three bivariate plots to highlight:



This first scatterplot is between `Flavanoids` and `Color_Intensity`. In this plot, we see at least two clear elongated clusters.



This second scatterplot is between `Proline` and `Color_Intensity` and appears to have at least three different clusters.

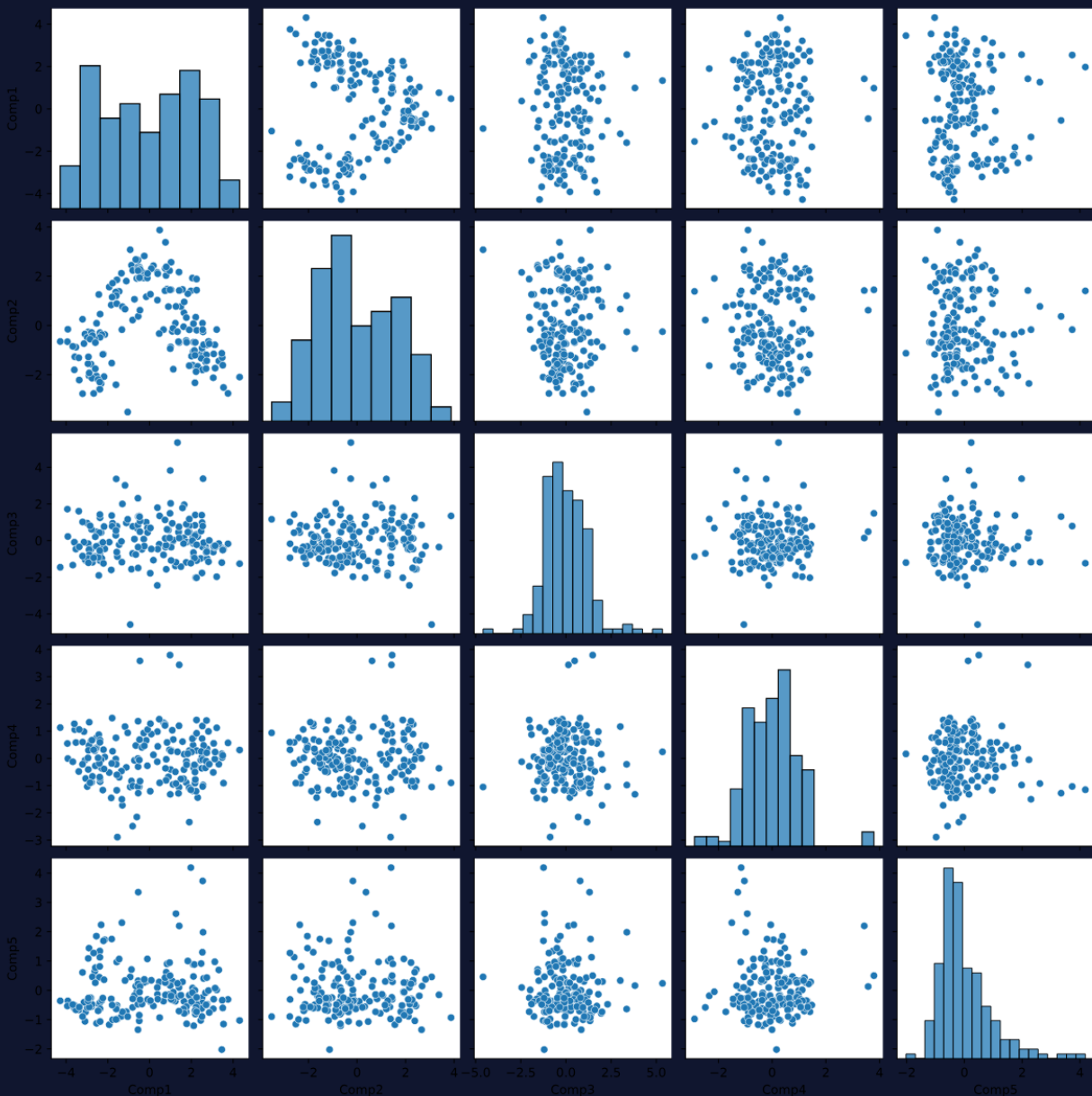


This final scatterplot is between `Alcohol` and `OD280` and contains three distinct round blobs of points.

Based on these plots we can conclude that there are probably at least three different types of wine in this dataset and that `Flavanoids`, `Color_Intensity`, `Alcohol`, and `OD280` may be particularly important in distinguishing those groups.

Feature reduction for EDA

The first pair plot with all of the variables was difficult to inspect, but we can reduce the number of dimensions by transforming our data using PCA. Let's look at the pair plot of the first few principal components:



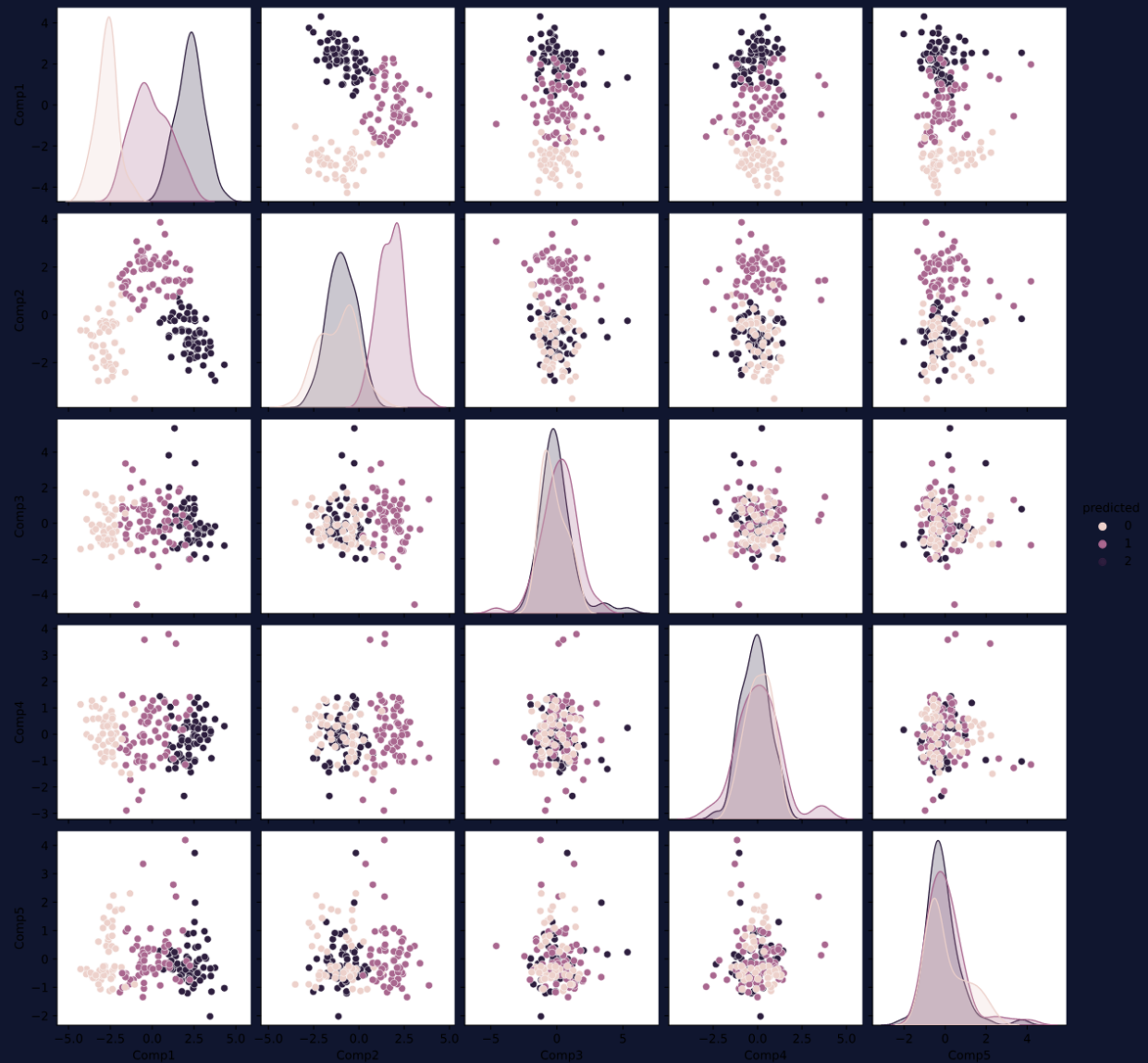
Now, instead looking at a 13 by 13 pair plot, we can zoom in on a 5 by 5 plot that includes all of the original features. In fact, a single plot can be used to visualize relationships between all of the original features at once, not just two at a time. We can see that there is some distinction between groups in clusters 1 and 2, as well as 2 and 4. We also see three fairly clear groups in the plot of component 1 vs. 2. But what does this mean about our original features?

We can look at the weights for each of these components and see which features were most highly weighted in components 1 and 2 (which seem to cluster into three clear groups):

	Comp1	Comp2	Comp3	Comp4	Comp5
Alcohol	0.144329	-0.483652	-0.207383	-0.017856	-0.265664
Malic_Acid	-0.245188	-0.224931	0.089013	0.536890	0.035214
Ash	-0.002051	0.626224	0.626224	-0.214176	-0.143025
Ash_Alcanity	-0.239320	0.010591	0.612080	0.060859	0.066103
Magnesium	0.141992	-0.299634	0.130757	-0.351797	0.727049
Total_Phenols	0.394661	-0.065040	0.146179	0.198068	-0.149318
Flavanoids	0.422934	0.003360	0.150682	0.152295	-0.109026
Nonflavanoid_Phenols	-0.298533	-0.0287790	0.170368	-0.203301	-0.500703
Proanthocyanins	0.313429	-0.039302	0.149454	0.399057	0.136860
Color_Intensity	-0.088617	-0.529996	-0.137306	0.065926	-0.076437
Hue	0.296715	0.279235	0.085222	-0.427771	-0.173615
OD280	0.376167	0.164496	0.166005	0.184121	-0.101161
Proline	0.286752	-0.364903	-0.126746	-0.232071	-0.157869

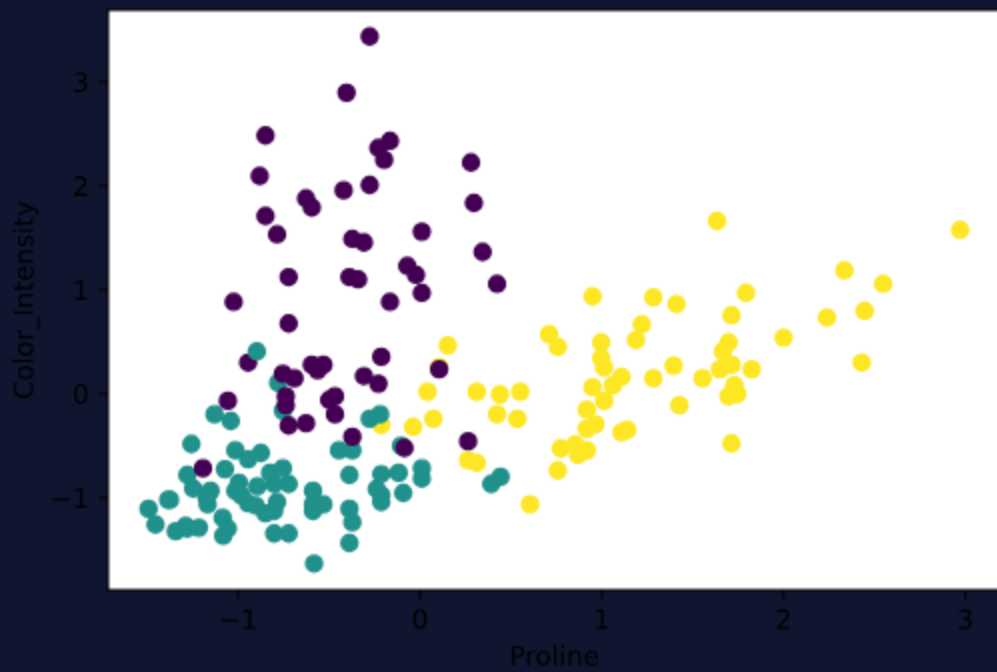
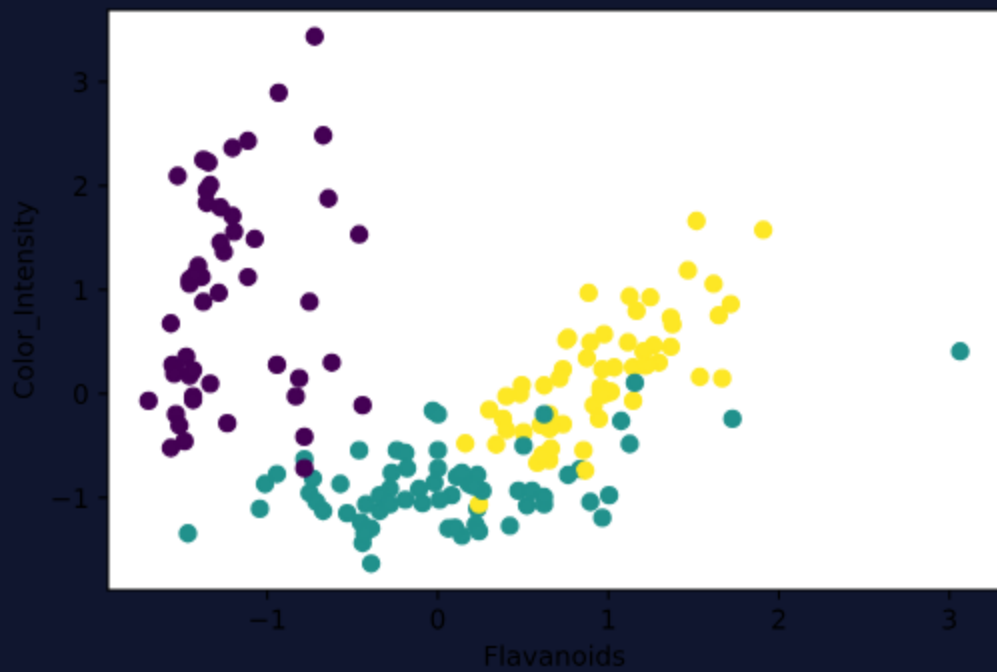
We are looking for the highest weighted feature for each component of interest. For component 1, this would be `Flavanoids`, component 2, `Color_Intensity`, and so on. These may be particularly important features to include in our model.

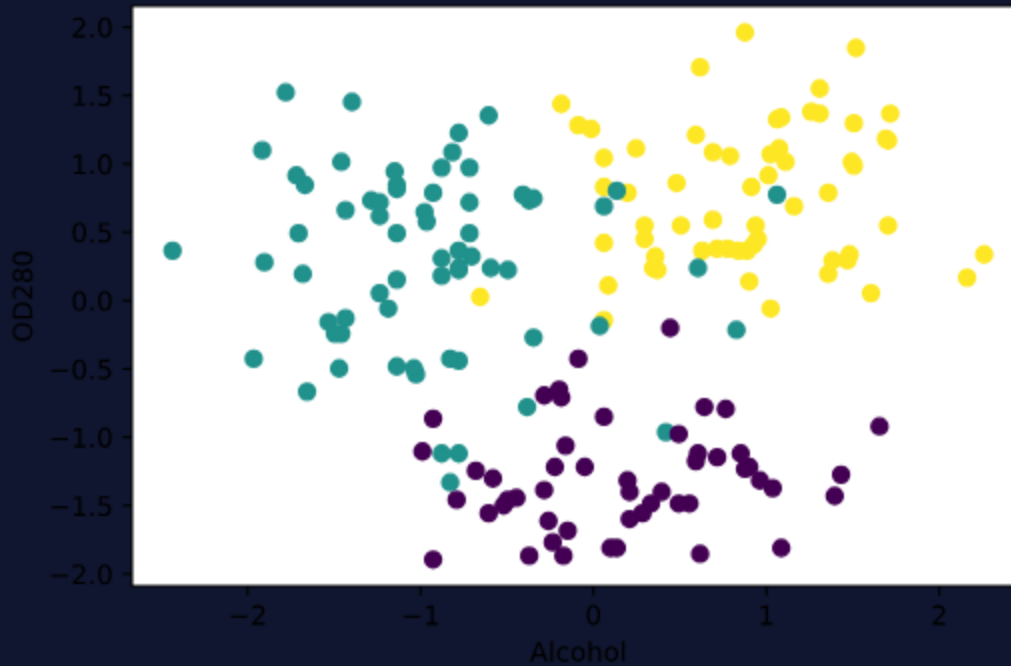
We can also use the transformed data to visually inspect the groups that are produced by other supervised machine learning methods. For example, here is the same pair plot of the transformed data, but the points are colored by the outcome of a k-means analysis with 3 groups:



We can see here that the groupings we saw in the PCA-transformed data align with those produced by the k-means model. Specifically, we see that the pair plot of components 1 and 2 separates the k-means clusters particularly well.

We can also look at the same bivariate scatterplots from earlier, with the added k-means results:





We suspected that these features would be useful in clustering our wines, and now we see that the k-means model is producing groups based on these features. This is also useful for explaining the k-means model to potential stakeholders. For example, we can say that the “purple” group produced by our k-means model is characterized by lower than average amounts of flavanoids and higher than average color intensity.

Conclusion

EDA before and after fitting unsupervised clustering algorithms is extremely helpful for checking model assumptions, choosing an algorithm, determining the number of groups, and explaining the results to potential stakeholders.

Feature Engineering

Introduction to Feature Engineering

Learn about feature engineering and its role in machine learning.

Introduction

Feature engineering is an integral part of building and implementing machine learning models. In this article, you'll learn about what feature engineering is, why we need it and where it fits in within the machine learning workflow. Additionally, you'll get a brief overview of the many data science tools involved in feature engineering and how they assist data scientists in model diagnostics.

What is feature engineering?

Before we can talk about feature engineering, we have to define what we mean by 'features'. A feature is a measurable property in a dataset, and a feature can be used as an input to a machine learning model. One way to think about features is as predictor variables that go into a model to predict the outcome variable. For example, if we want a model that predicts precipitation at a given place and time, we might use temperature, humidity, month, altitude, etc. as inputs. These are our features.

Often, when presented with a dataset, it might not be clear what features we should use for a specific model (i.e., should we use 'tree density' as an input to our precipitation model?). Similarly, in large datasets, there might be too many features to manually decide which features to use. Some features might be highly correlated with one another, some might not vary much with the outcome variable, and some might be in the wrong form to be a model input and so on. It is not uncommon that a data scientist might not realize any of this until they begin diagnosing a model that is performing poorly.

Feature engineering is a way to address these challenges. It is an umbrella term for the techniques we use to help us make decisions about features in machine learning model implementations.

Why do we need feature engineering?

To paraphrase Tolstoy, "All functional model implementations resemble one another but every dysfunctional model implementation is dysfunctional in its own way."

There are a lot of different ways that a model implementation can be dysfunctional. So the question is: What would make a model implementation functional or dysfunctional? Consider the following four attributes:

1. Performance:

We would like our machine learning model to perform "well" on our data. If it is not able to predict the outcome variable (to a reasonable degree of accuracy) on known data, it would be unwise to use it to predict outcomes on unknown data.

2. Runtime:

Suppose a model has excellent performance but takes a really long time to run. For a data scientist, depending on their available computational resources, such a model might be impractical for production.

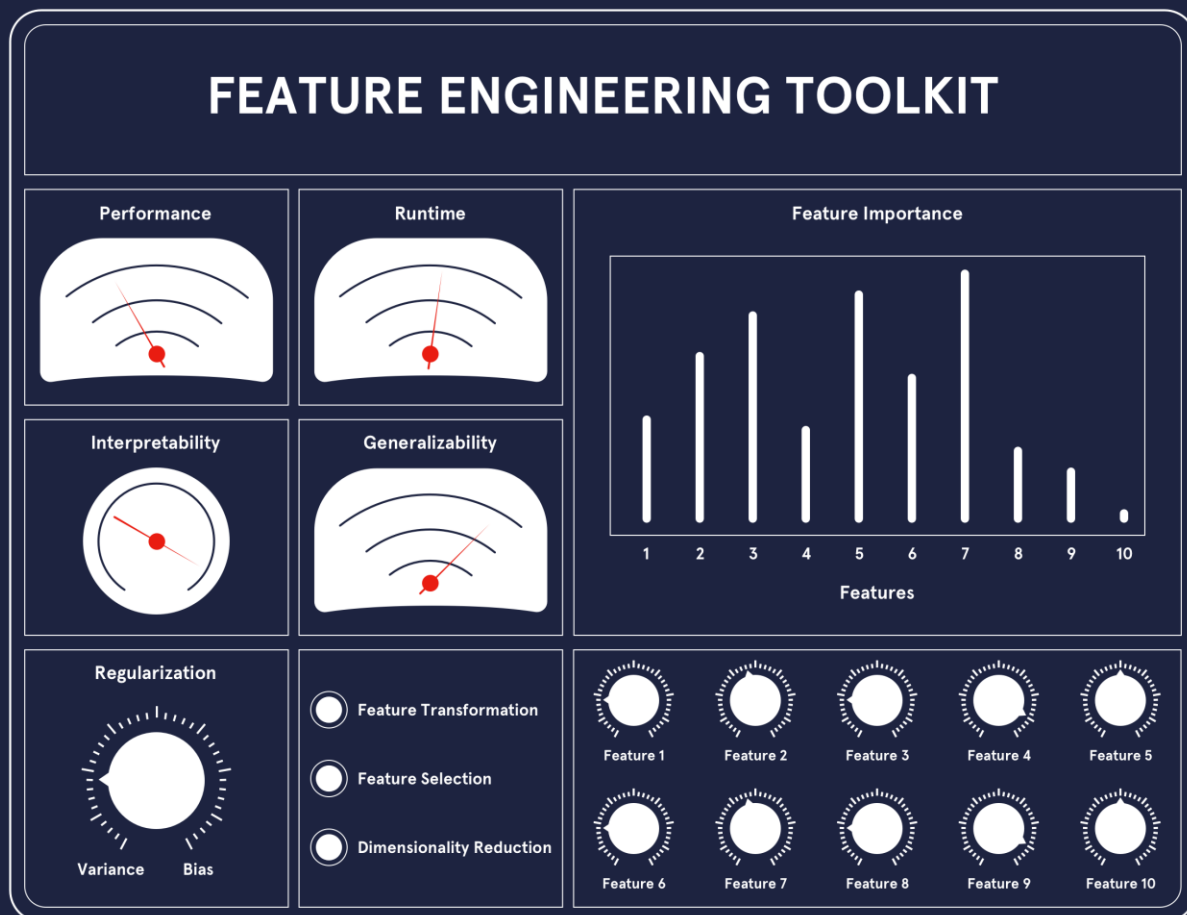
3. Interpretability:

A model is only as good as the insights it helps us glean from the data. Data scientists are often tasked with finding out what factors drive different outcomes. A well-performing model would not be of much help if it's opaque and uninterpretable.

4. Generalizability:

We would like our model to generalize well to unseen data. Often data scientists work with streaming data and need their model to be flexible with new and unknown data..

Feature engineering can be thought of as a toolkit of techniques to use when a model is missing one or more of the above attributes. If we imagine a model diagnostic machine (kind of like a modern version of this one) with meters representing the attributes, feature engineering would be akin to turning knobs and pushing buttons until we arrive upon a model that meets all of our attributes satisfactorily.



Feature Engineering and the Machine Learning Workflow

So where does feature engineering fall within the machine learning workflow? Does it go before modeling, after modeling or alongside modeling? The answer is: all of the above! Feature engineering is often introduced as an intermediate step between exploratory data analysis and implementing a machine learning algorithm. In reality, these distinctions are fuzzy and the process is not exactly linear.

Broadly, we can divide feature engineering techniques into three categories:

1. Feature Transformation methods:

These involve numerical transformations methods and ways to encode non-numerical variables. These techniques are applied *before* implementing a machine learning model. They include and are not limited to: scaling, binning, logarithmic transformations, hashing and one hot encoding. These methods typically improve performance, runtime and interpretability of a model.

2. Dimensionality Reduction methods:

Dimensionality of a dataset refers to the number of features within a dataset and reducing dimensionality allows for faster runtimes and (often) better performance. This is an extremely powerful tool in working with datasets with “high dimensionality”. For instance, a hundred-feature problem can be reduced to less than ten modified features, saving a lot of computational time and resources while maintaining or even improving performance. Typically, dimensionality reduction methods are machine learning algorithms themselves, such as Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), etc.

These techniques transform the existing feature space into a new subset of features that are ordered by decreasing importance. Since they “extract” new features from high dimensional data they’re also referred to as **Feature Extraction** methods. The transformed features do not directly relate to anything in the real world anymore. Rather, they are mathematical objects that are related to the original features. However, these mathematical objects are often difficult to interpret. The lack of interpretability is one of the drawbacks of dimensionality reduction.

3. Feature Selection methods

Feature selection methods are a set of techniques that allow us to choose among the pool of features available. Unlike dimensionality reduction, these retain the features *as they are* which makes them highly interpretable. They usually belong to one of these three categories:

i. Filter methods:

These are statistical techniques used to “filter” out useful features. Filter methods are completely model agnostic (meaning they can be used with any model) and are useful sanity checks for a data scientist to carry out before deciding on a model. They include and are not limited to: correlation coefficients (Pearson, Spearman, etc) , χ^2 , ANOVA, and Mutual Information calculations.

ii. Wrapper methods:

Wrapper methods search for the best set of features by using a “greedy search strategy”. They pick a subset of features, train a model, evaluate it, try a different subset of features, train a model again, and so on until the best possible set of features and most optimal performance is reached. As this could potentially go on forever, a stopping criterion based on number of features or a model performance metric is typically used. Forward Feature Selection, Backward Feature Elimination and Sequential Floating are some examples of wrapper method algorithms.

iii. Embedded methods:

Embedded methods are implemented *during* the model implementation step. Regularization techniques such as Lasso or Ridge tweak the model to get it to generalize better to new and unknown data. Tree-based feature importance is another widely used embedded method. This method provides insight into the features that are most relevant to the outcome variable while fitting decision trees to the data.

Summary

Here’s a brief summary of the feature engineering methods we’ve covered in this article, the attributes they seek to improve, and where they fit in within the machine learning workflow:

Feature Engineering Method	What	Why	Where
1. Feature Transformation	Transforms Numerical and non-numerical data	Performance, Runtime, Interpretability	Before model implementation
2. Dimensionality Reduction	Extracts a new feature subset	Runtime, Performance	Before model implementation
3. Feature Selection	Filter Methods	Interpretability, Performance	Before model implementation
	Wrapper Methods	Performance, Interpretability	Iterates over model implementation (i.e., before and after)
	Embedded Method: Regularization	Generalizability, Interpretability	Implemented with model implementation
	Embedded Method: Feature Importance	Interpretability, Performance	Implemented with model implementation

We're now ready to learn about these in-depth!

Data Transformations

Introduction: Data Transformations

Find out what you will learn in Data Transformations for Feature Analysis!

In this module you will learn how to prepare your data and features prior to running a machine learning model. This is a very essential step that, if skipped, can cause some grief down the line. It is usually performed right after or alongside Exploratory Data Analysis (EDA).

Why did we build this?

Let's take a moment to consider the consequence of not doing data transformations before a model implementation. An obvious one might be that the model code errors out. This is still a benign possibility as it tells you that something is wrong. The more insidious possibility is one where it doesn't cause a problem until much later. Multiple diagnoses of many poorly performing models later, it might become apparent that the data might've needed to be transformed in some way at the beginning! Data Transformations are thus an irreplaceable step for data scientists in their workflows.

What will you learn?

You're going to learn about:

- numerical data transformations such as standardizing, binning, scaling, etc.
- different ways of encoding categorical and non-numerical data.

when, where and how to apply these methods when presented with a dataset

Numerical Transformation Introduction

We've all heard it; a lot of work goes into getting your data just right for the task you have at hand. Perhaps you are trying to find insight into customer behavior or predicting the best time to send a marketing email. Whatever problem you may be trying to solve - you have probably already spent some time wrangling your data and getting it just right, and now you are at the stage where you need to prepare your data for machine learning.

For example, some machine learning models, like logistic regression and neural networks, can only handle numbers. Then random forest and decision tree models can take both numbers and text. We will call these text features categorical data, but more on that later. We also need to understand the spread of our data, as some models have a tough time handling data that has extreme outliers.

This process is called *numerical transformation*, when we take our numerical data and change it into another numerical value. This is meant to change the scale of our values or even adjust the skewness of our data. You may be thinking, "we already have our data in numbers. Why would we want to change those?" Well, first of all, that is a great question. We'll dive deep into the "why we do this" throughout this lesson. To put it simply, we do this to help our model better compare features and, most importantly, improve our model's accuracy and interpretability. That sounds like some good reasons to put the time and effort into numerical transformations if I do say so myself.

We'll focus on the following numerical transformations:

Centering
Standard Scaler
Min and Max Scaler
Binning
Log transformations

Let's get to know the data frame we will be using. This dataset has just over 100 responses from customers where they were asked about a recent Starbucks experience. You will soon notice that we have a mix of numerical and categorical data, but we'll focus only on the numerical features for this lesson.

Instructions

Checkpoint 1 Failed, try again

1.

We have provided you with a csv file under the name `starbucks_customers.csv` and have already imported pandas for you. Import the file and set it to a variable called `coffee`.

Hint

```
coffee = pd.read_csv('FILENAME.csv')
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Examine the features in your new data frame by printing the columns.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Look at each feature within your data frame by printing `.info()`

```
import pandas as pd

## add code below
## set up dataframe
coffee = pd.read_csv('starbucks_customers.csv')

## print the column names
print(coffee.columns)

## get information on your data
print(coffee.info())
```

Centering Your Data

Data centering involves subtracting the mean of a data set from each data point so that the new mean is 0. This process helps us understand how far above or below each of our data points is from the mean.

We can look at the `nearest_starbucks` column.

```
distance = coffee['nearest_starbucks']
```

Copy to Clipboard

We will find the mean of our feature, create one line of code to center our data, and then plot the centered data. Here's what it will look like in Python.

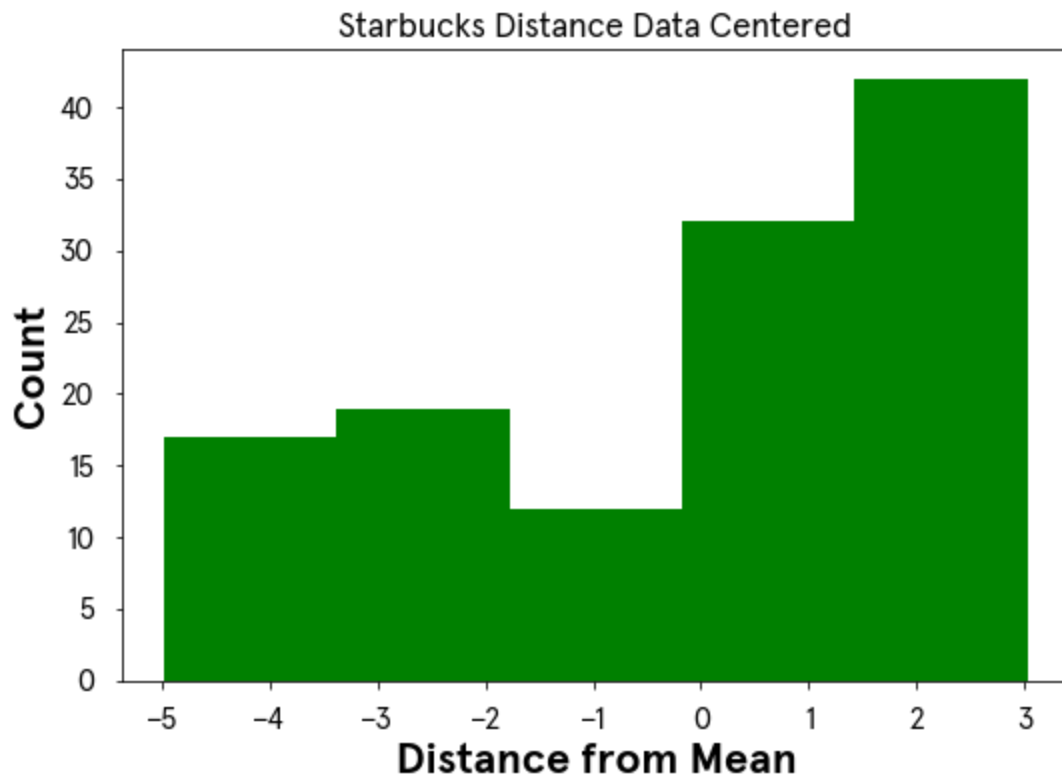
```
#get the mean of your feature
mean_dis = np.mean(distance)

#take our distance array and subtract the mean_dis, this will create a new series with
the results
centered_dis = distance - mean_dis

#visualize your new list
plt.hist(centered_dis, bins = 5, color = 'g')

#label our visual
plt.title('Starbucks Distance Data Centered')
plt.xlabel('Distance from Mean')
plt.ylabel('Count')
plt.show();
```

Copy to Clipboard



Now, look at that! Our data is centered at 0 and we can see the spread of data, many of our customers who took the survey tend to live fairly close to a Starbucks.

Let's try centering our age feature to see what happens there.

1. Start by setting your `age` feature to a variable called `ages`

Hint

```
ages = df['column_name']
```

Copy to Clipboard

Checkpoint 2 Passed

2. It's helpful to know both the minimum and maximum age in our feature. First, create a variable called `min_age` and print the value

Hint

```
min_age = np.min(variable)
print(min_age)
```

Copy to Clipboard

Checkpoint 3 Passed

3. Now find the maximum age and set it to a variable called `max_age` and print the value.

Hint

```
max_value = np.max(feature)
print(max_value)
```

Copy to Clipboard

Checkpoint 4 Passed

- 4.

We want to better understand the spread of our data, so print the difference between `min_age` and `max_age`

Hint

Subtract the `min_age` from the `max_age`

Checkpoint 5 Passed

5.

We will need the mean age of our feature to center our data around, so create a variable called `mean_age` and print that value.

Hint

```
np.mean(variable)
```

Copy to Clipboard

Checkpoint 6 Passed

6.

Let's center our data! Set the result to a variable called `centered_ages` and print the results.

Hint

```
centered_dis = distance - mean_dis
```

Copy to Clipboard

Checkpoint 7 Passed

7.

Plot your centered data as a histogram.

Hint

```
plt.hist(centered_data)
plt.show()
```

Copy to Clipboard

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3 seaborn

coffee = pd.read_csv('starbucks_customers.csv')

## add code below
## create ages variable
ages = coffee['age']

## get min and print
min_age = np.min(ages)
print(min_age)

## get max and print
max_age = np.max(ages)
print(max_age)

## print the range
print(max_age - min_age)

## find the mean
mean_age = np.mean(ages)
print(mean_age)

## center ages
centered_ages = ages - mean_age
print(centered_ages)

## graph it
plt.hist(centered_ages)
plt.show();

plt.hist(coffee.age)
plt.show();
```

Standardizing our Data

Excellent work with centering the age feature from our dataset! Now we'll take that concept one step further and discuss standardizing our data. *Standardization* (also known as *Z-Score normalization*) is when we center our data, then divide it by the standard deviation. Once we do that, our entire data set will have a mean of zero and a standard deviation of one. This allows all of our features to be on the same scale. How cool is that?

This step is critical because some

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

models will treat all features equally regardless of their scale. You'll definitely want to standardize your data in the following situations:

- Before Principal Component Analysis

- Before using any clustering or distance based

- Preview: Docs Loading link description

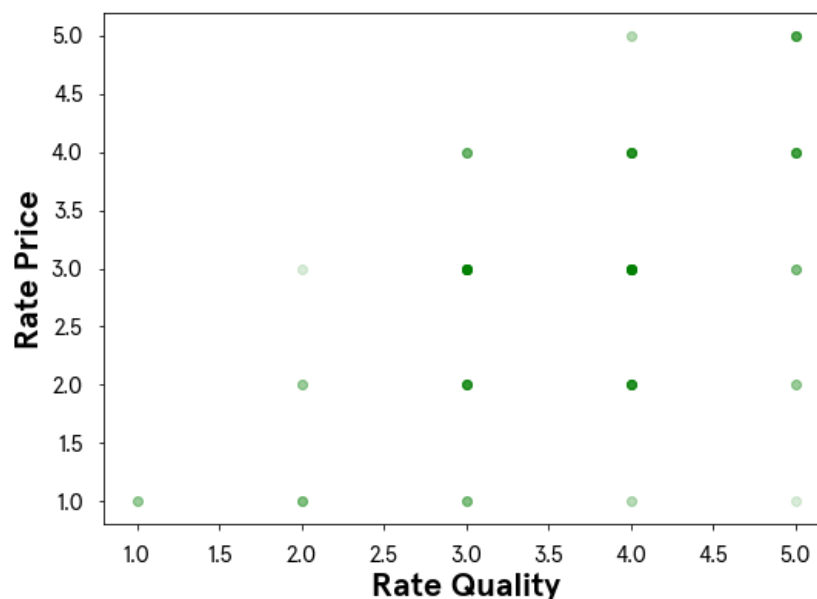
- [algorithm](#)

- (think KMeans or DBSCAN)

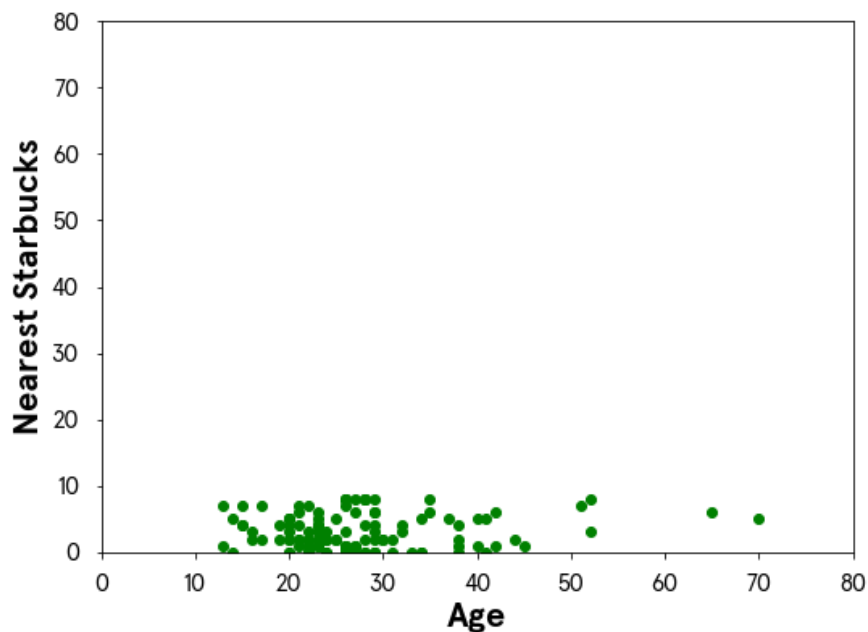
- Before KNN

- Before performing regularization methods like LASSO and Ridge

If I wanted to see how customers rated quality vs. price, I could graph a scatter plot of those two features and easily see that customers tend to score those two questions closely. Notice the darker spots (meaning more data points are laying on top of one another) around the 3.0 for "Rate Quality" and 3.0 for "Rate Price" as an example. This insight was easy to see since our features are on the same one to five scale.



Now, what happens if I am working with features on two totally different scales? Perhaps the "customer age" and "how far they live from a Starbucks" feature? Let's take a look.



Woah! Looking at this, it is much more challenging to gain insight or even identify patterns within our data. This will be a similar experience for our machine learning models. That's why when we standardize our entire dataset, we tend to see a more robust model performance if all features are on the same scale.

Let's examine one feature to learn the mathematics that goes on when we standardize our data. The mathematical formula will look like this:

$$z = (\text{value} - \text{mean}) / \text{stdev}$$

We'll look at just one feature, where customers were asked how close they are to their nearest Starbucks, and follow that formula above. First, we will set our `nearest_starbucks` feature to its own variable and then find the mean and standard deviation. Then we can quickly standard our list following the formula above.

```
distance = coffee['nearest_starbucks']

#find the mean of our feature
distance_mean = np.mean(distance)

#find the standard deviation of our feature
distance_std_dev = np.std(distance)

#this will take each data point in distance subtract the mean, then divide by the
standard deviation
distance_standardized = (distance - distance_mean) / distance_std_dev
```

Copy to Clipboard

We now have our distance feature standardized! Let's double-check by seeing what the mean and standard deviation of our [array](#) is.

```
# print what type distance_standardized is
print(type(distance_standardized))
#output = <class 'pandas.core.series.Series'>

#print the mean
print(np.mean(distance_standardized))
```

```
#output = 7.644158530205996e-17

#print the standard deviation
print(np.std(distance_standardized))
#output = 1.00000000000000013
```

Copy to Clipboard

Our outputs are basically mean = 0 and standard deviation = 1. Fantastic! Let's see what our `age` feature looks like standardized.

Instructions

Checkpoint 1 Passed

1.

Find the average of your variable `ages` and set the result to a variable called `mean_age`

Hint

```
average = np.mean(feature)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Find the standard deviation of your variable `ages` and set the result to a variable called `std_dev_age`

Hint

```
standard_dev = np.std(feature)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Standardize our age column. Set the result to a variable called `ages_standardized`

Hint

```
distance_standardized = (distance - mean_distance) / std_dev_distance
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Print the mean and standard deviation of `ages_standardized`.

Hint

```
print(np.mean(lst))
print(np.std(lst))
```

```
import pandas as pd
import numpy as np

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## set up your variables
mean_age = np.mean(ages)
std_dev_age = np.std(ages)

## standardize ages
ages_standardized = (ages - mean_age) / std_dev_age

## print the results
print(np.mean(ages_standardized))
print(np.std(ages_standardized))
```

Standardizing our Data with Sklearn

Now that we understand the mathematics behind a standard scaler let's quickly implement it with the sklearn library. We will begin by importing our `StandardScaler` library from `sklearn.preprocessing`.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

Copy to Clipboard

We instantiate the `StandardScaler` by setting it to a variable called `scaler` which we can then use to transform our feature. The next step is to reshape our distance

Preview: Docs Loading link description

[array](#)

. `StandardScaler` must take in our array as 1 column, so we'll reshape our distance array using the `.reshape(-1,1)`

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

. This numpy method says to take our data and give it back to us as 1 column, represented in the second value. The -1 asks numpy to figure out the exact number of rows to create based on our data. Pretty cool, right?

```
reshaped_distance = np.array(distance).reshape(-1,1)
distance_scaler = scaler.fit_transform(reshaped_distance)
```

Copy to Clipboard

Just like we learned in the last lesson, we do this so our data has a mean of 0 and standard deviation of 1. Let's print to see how the `StandardScaler` did.

```
print(np.mean(distance_scaler))
#output = -9.464196275493137e-17
print(np.std(distance_scaler))
#output = 0.9999999999999997
```

Copy to Clipboard

You'll notice the `e-17` at the end of our output for the mean, and that is our number written in exponential notation. Written in standard notation our answer is `-0.000000000000000009464196275493137`. Which basically has the mean at 0 and our standard deviation = 1. Great work! Now you can try to standardize the age column with the new knowledge you just gained.

Instructions

Checkpoint 1 Passed

1.

Instantiate `StandardScaler` by creating a variable named `scaler`

Hint

```
scaler = StandardScaler()
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Reshape our `ages` feature and set the result to a variable named `ages_resaped`

Hint

```
reshaped = np.array(feature).reshape(-1,1)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Now we are ready to perform a `.fit_transform` on our `ages_resaped` feature and set the results to the variable named `ages_scaled`

Hint

```
feature_scaled = scaler.fit_transform(reshaped_feature)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Print the mean and standard deviation of your new variable `ages_scaled`

Hint

```
np.mean(feature_scaled)
np.std(feature_scaled)
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## import standard scaler
scaler = StandardScaler()

## reshape
ages_resaped = np.array(ages).reshape(-1,1)

## fit_transform
ages_scaled = scaler.fit_transform(ages_resaped)

## print mean and standard deviation
print(np.mean(ages_scaled))
print(np.std(ages_scaled))
```

Min-Max Normalization

Another form of scaling your data is to use a min-max normalization process. The name says it all, we find the minimum and maximum data point in our entire data set and set each of those to 0 and 1, respectively. Then the rest of the data points will transform to a number between 0 and 1, depending on its distance between the minimum and maximum number. We find that transformed number by taking the data point subtracting it from the minimum point, then dividing by the value of our maximum minus minimum.

Mathematically a min-max normalization looks like this:

$$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$$

One thing to note about min-max normalization is that this transformation does not work well with data that has extreme outliers. You will want to perform a min-max normalization if the range between your min and max point is not too drastic.

The reason we would want to normalize our data is very similar to why we would want to standardize our data - getting everything on the same scale.

We'd do this process to our entire data set, but let's look at our distance to a nearest Starbucks feature. We already know from earlier in this lesson that our range in this feature is not too large, so it is a good candidate for a min-max normalization. We'll write a function that will perform the mathematics needed to transform the entire column.

```
distance = coffee['nearest_starbucks']

#find the min value in our feature
distance_min = np.min(distance)

#find the max value in our feature
distance_max = np.max(distance)

#normalize our feature by following the formula
```

```
distance_normalized = (distance - distance_min) / (distance_max - distance_min)
```

Copy to Clipboard

Excellent! Now if I were to print all the unique numbers in `distance_norm` here is what we will see.

```
{0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0}
```

Copy to Clipboard

We can see that all numbers are between 0 and 1. Amazing!

Instructions

Checkpoint 1 Passed

1.

We will now work with the `spent` feature in our data frame. Take just that column and set it equal to a new variable called `spent`.

Hint

```
new_variable = coffee['column_name']
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Find the maximum amount someone spent and set that to a variable called `max_spent`

Hint

```
max_spent = np.max(variable)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Find the minimum amount someone spent and set that to a variable called `min_spent`

Hint

```
min_spent = np.min(variable)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Find the difference between `max_spent` and `min_spent` and set that to a variable called `spent_range`. Print the results.

Hint

```
spent_range = max_spent - min_spent  
print(spent_range)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Normalize your `spent` column. Set the results to a variable called `spent_normalized`.

Hint

```
normalized = (feature - min_feature) / feature_range
```

Copy to Clipboard

Checkpoint 6 Passed

6.

Print your `spent_normalized` array to see how your numbers transformed.

Hint

```
print(array)
```

```
import pandas as pd  
import numpy as np
```

```

coffee = pd.read_csv('starbucks_customers.csv')

## add code below
## get spent feature
spent = coffee['spent']

#find the max spent
max_spent = np.max(spent)

#find the min spent
min_spent = np.min(spent)

#find the difference
spent_range = max_spent - min_spent
print(spent_range)

#normalize your spent feature
spent_normalized = (spent - min_spent) / spent_range

#print your results
print(spent_normalized)

```

Min-Max Normalization with Sklearn

Happy to see you now have a strong understanding of how a Min-Max Normalization works, let's try implementing it with the sklearn library.

We will start by importing our `MinMaxScaler` library from `sklearn.preprocessing`. Just like we covered in the `StandardScaler` exercise, we start by instantiating the `MinMaxScaler` by setting it to a variable called `mmscaler` which we can then use to transform our feature.

```

from sklearn.preprocessing import MinMaxScaler
mmscaler = MinMaxScaler()

```

Copy to Clipboard

The next step is to import our distance feature and reshape it so it is ready for our `mmscaler`.

```

#get our distance feature
distance = coffee['nearest_starbucks']

#reshape our array to prepare it for the mmscaler
reshaped_distance = np.array(distance).reshape(-1,1)

#.fit_transform our reshaped data
distance_norm = mmscaler.fit_transform(reshaped_distance)

#see unique values
print(set(np.unique(distance_norm)))
#output = {0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0}

```

Copy to Clipboard

Excellent. We now have a normalized feature within our dataset using the sklearn library `MinMaxScaler`!

Instructions

Checkpoint 1 Enabled

1.

Reshape the array, and set it to a variable called `spent_reshaped`

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Instantiate MinMaxScaler and set the variable equal to `mmscaler`

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now let's try using the MinMaxScaler from the imported sklearn library to perform a `.fit_transform` on our `spent_rescaled` feature and set the results to the variable `reshaped_scaled`

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print the minimum and maximum number from your new normalized array nam

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

coffee = pd.read_csv('starbucks_customers.csv')
spent = coffee['spent']

## add code below
## reshape array
spent_rescaled = np.array(spent).reshape(-1, 1)

## instantiate mmscaler
mmscaler = MinMaxScaler()

## .fit_transform
reshaped_scaled = mmscaler.fit_transform(spent_rescaled)

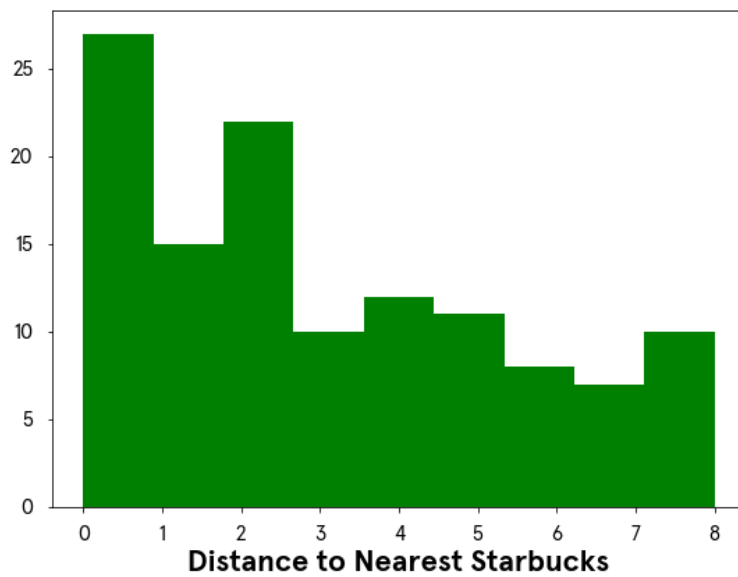
## print min and max
print(np.min(reshaped_scaled))
print(np.max(reshaped_scaled))
```

Binning our Data

Binning data is the process of taking numerical or categorical data and breaking it up into groups. We could decide to bin our data to help capture patterns in noisy data. There isn't a clean and fast rule about how to bin your data, but like so many things in machine learning, you need to be aware of the trade-offs.

You want to make sure that your bin ranges aren't so small that your model is still seeing it as noisy data. Then you also want to make sure that the bin ranges are not so large that your model is unable to pick up on any pattern. It is a delicate decision to make and will depend on the data you are working with.

Going back to our customer data and looking at our distance feature, let's look at the data with a histogram.



We can easily see that a lot of customers who completed this survey live fairly close to a Starbucks, and our data has a range of 0 km to 8km. I wonder how our data would transform if we were to bin our data in the following way:

```
distance < 1km
1.1km <= distance < 3km
3.1km <= distance < 5km
5.1km <= distance
```

First, we'll set the upper boundaries of what we listed above.

```
bins = [0, 1, 3, 5, 8.1]
```

Copy to Clipboard

Now you may be asking yourself 'Why end at 8.1? Isn't our max value 8?' That is true! We have 8.1 and not 8 because the pandas function we will use `pd.cut()` has a parameter where it will include the lower bound, and excludes the upper bound. Let me show you what I mean.

```
coffee['binned_distance'] = pd.cut(coffee['nearest_starbucks'], bins, right = False)
print(coffee[['binned_distance', 'nearest_starbucks']].head(3))
```

```
#output
#  binned_distance  nearest_starbucks
#0      [5.0, 8.1)                  8
#1      [5.0, 8.1)                  8
#2      [5.0, 8.1)                  8
```

Copy to Clipboard

We can see that those who marked 8 km now live in the `[5.0, 8.1)` bucket. The bracket `[` tells us 5.0 is included, and the parenthesis `)` tells us that 8.1 is excluded. We could write it as an inequality statement like this: `5 <= distance < 8.1` this allows our customers who marked 8 to belong to the 'Lives greater than 5.1 km' bin. Now let's have a look at our newly binned data.

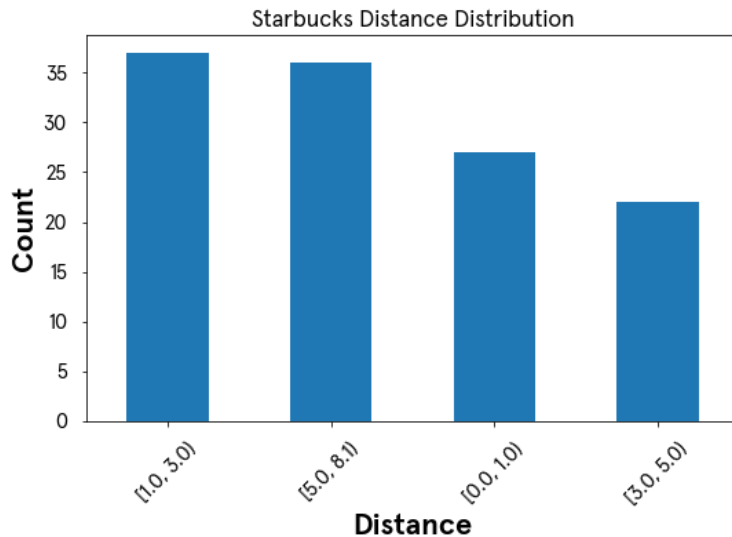
```
# Plot the bar graph of binned distances
coffee['binned_distance'].value_counts().plot(kind='bar')

# Label the bar graph
plt.title('Starbucks Distance Distribution')
```

```
plt.xlabel('Distance')
plt.ylabel('Count')

# Show the bar graph
plt.show()
```

Copy to Clipboard



Use this new skill to bin our age feature.

Instructions

Checkpoint 1 Passed

1.

Print the age features min and max values.

Hint

```
print(np.min(feature))
print(np.max(feature))
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Based on our results from above, let's set the bin boundaries. Store it as a variable called `age_bins` and use the following limits [12, 20, 30, 40, 71]

Hint

Set the list provided to a variable called `age_bins`

Checkpoint 3 Passed

3.

Create new binned column with `pd.cut()`. Name the new column `binned_ages`

Hint

```
df['new_column'] = pd.cut(df['original_col'], new_bins, right = False)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Print the first 10 rows of the new column `binned_ages`

Hint

```
df['column_name'].head(10)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Plot a bar graph of new binned data.

Hint

```
df['binned_col'].value_counts().plot(kind='bar')
```

```
plt.show()
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3_seaborn

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## print min and max
print(np.min(ages))
print(np.max(ages))

## set bins
age_bins = [12, 20, 30, 40, 71]

## new column
coffee['binned_ages'] = pd.cut(ages, age_bins, right = False)

## print first 10 rows
print(coffee['binned_ages'].head(10))

## graph it
coffee['binned_ages'].value_counts().plot(kind='bar')
plt.show()
```

Natural Log Transformation

We've covered plenty of transformations! We have one last transformation we need to cover, and that is log transformations. Logarithms are an essential tool in statistical analysis and

Preview: Docs Loading link description

[machine learning](#)

preparation. This transformation works well for right-skewed data and data with large outliers. After we log transform our data, one large benefit is that it will allow the data to be closer to a "normal" distribution. It also changes the scale so our data points will drastically reduce the range of their values.

For example, let's explore a whole new data set from Kaggle around used car prices. Take a look at this histogram plot of 100,000 used car odometers.

```
import pandas as pd

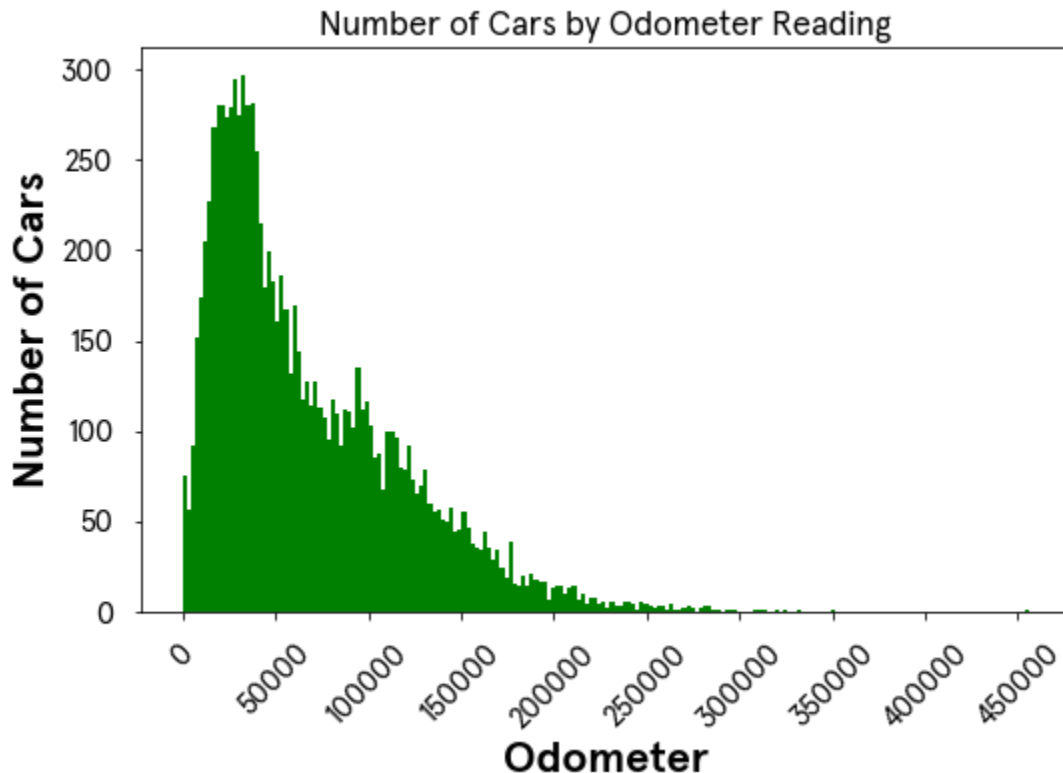
#import our dataframe
cars = pd.read_csv('cars.csv')

#set our variable
odometer = cars['odometer']
```

```
#graph our odometer readings
plt.hist(odometer, bins = 200, color = 'g')

#add labels
plt.xticks(rotation = 45)
plt.title('Number of Cars by Odometer Reading')
plt.ylabel('Number of Cars')
plt.xlabel('Odometer')
plt.show();
```

Copy to Clipboard



This histogram is right-skewed, where the majority of our data is located on the left side of our graph. If we were to provide this feature to our machine learning model it will see a lot of different cars with odometer readings off on the left of our graph. It will not see a lot of examples with very high odometer readings. This may cause issues with our model, as it may struggle to pick up on patterns that are within those examples off on the right side of our histogram.

We'll perform a log transformation using numpy to see how our data will transform.

```
import numpy as np

#perform the log transformation
log_car = np.log(cars['odometer'])

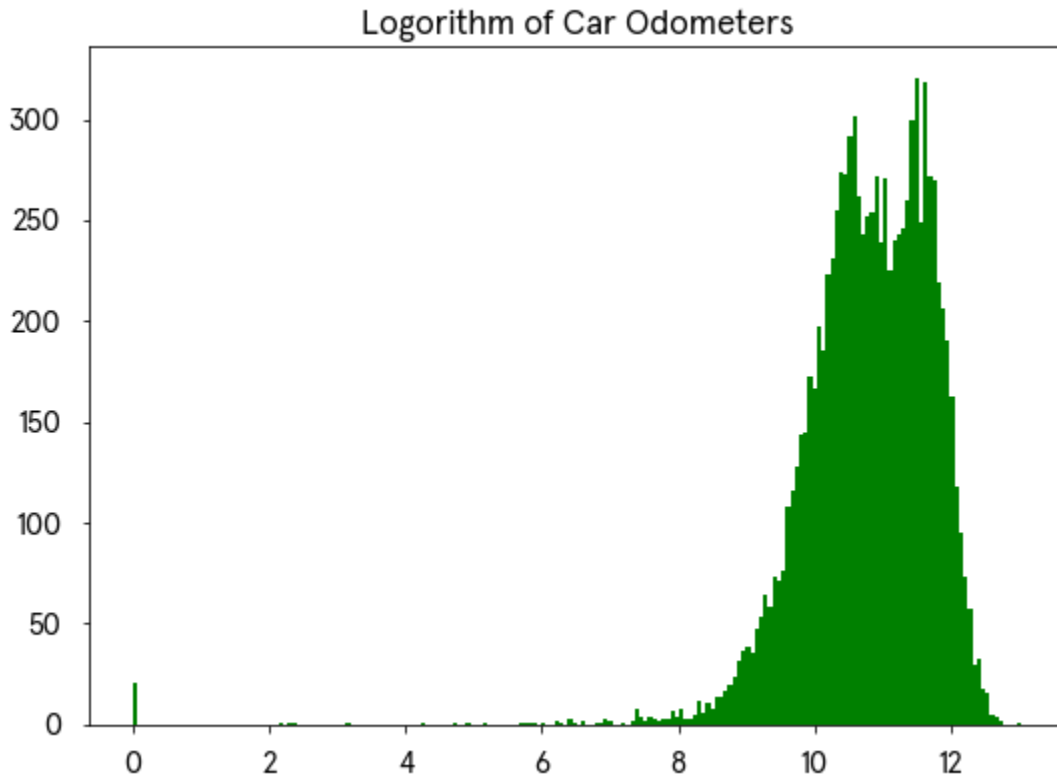
#graph our transformation
plt.hist(log_car, bins = 200, color = 'g')

#rotate the x labels so we can read it easily
plt.xticks(rotation = 45)
```



```
#provide a title
plt.title('Logarithm of Car Odometers')
plt.show();
```

Copy to Clipboard



Our data looks much closer to a normal distribution! If we were to look at a sample of five different cars with varying odometer readings, let's examine how the log transformation changed their values.

make	odometer	odometer_logged
Altima	10126	9.222862
Jetta	34042	10.435350
Camry	56762	10.946622
Civic	100103	11.513955

F-150	145695	11.889271
-------	--------	-----------

Saturn	151687	11.929574
--------	--------	-----------

If we compare the Altima with 10,126 miles to the Saturn with 151,687 miles those two cars have a huge difference in odometer readings. Yet, once we log transform the data we see the range from 9.22 to 11.93 is much smaller. Compressing the range of our data can help our model perform better!

There is so much more to add about log transformation. For the purpose of this exercise we just want to give a high overview and demonstrate how to log transform your data. Before we have you start testing your new skills let's quickly cover two major topics with log transformation:

Using a log transformation in a machine learning model will require some extra interpretation. For example, if you were to log transform your data in a linear regression model, our independent variable has a multiplication relationship with our dependent variable instead of the usual additive relationship we would have if our data was not log-transformed.

Keep in mind, just because your data is skewed does not mean that a log transformation is the best answer. You would not want to log transform your feature if:

- You have values less than 0. The natural logarithm (which is what we've been talking about) of a negative number is undefined.

- You have left-skewed data. That data may call for a square or cube transformation.

- You have non-parametric data

Instructions

Checkpoint 1 Passed

1.

We've imported all the necessary libraries, now you need to bring in the `cars.csv` file set it to a variable named `cars`.

Hint

```
variable = pd.read_csv('FILENAME.csv')
```

Copy to Clipboard

Checkpoint 2 Passed

2.

We will be working with the `sellingprice` feature in this data set, so set that column to a variable called `prices`.

Hint

```
variable = df['col_name']
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Let's look at a histogram of `prices` to check to see if it is right-skewed. Set your histogram to show 150 bins so we can see more values in our graph.

Hint

```
plt.hist(feature, bins = 150)
plt.show()
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Looks good! Now perform a log transformation on that variable `prices` and set the results to a variable called `log_prices`.

Hint

```
new_variable = np.log(variable)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Plot your newly log-transformed feature. Be sure to use `bins = 150`.

Hint

```
plt.hist(feature, bins = 150)
plt.show()
```

Encoding Categorical Variables

Encoding Categorical Variables

Encoding categorical variables with Python.

Introduction

Categorical data is data that has more than one category. When working with that type of data we have two types, nominal and ordinal. *Nominal data* is data that has no particular order or hierarchy to it, and *ordinal data* is categorical data where the categories have order, but the differences between the categories are not important or unclear.

We will be working with a dataset of used cars for this article to truly understand and demonstrate how to work with categorical data. Let's explore it and see what type of data we are working with.

```
import pandas as pd

# import data
cars = pd.read_csv('cars.csv')

# check variable types
print(cars.dtypes)
## OUTPUT
# year          int64
```

```
# make      object
# model     object
# trim      object
# body      object
# transmission object
# vin       object
# state     object
# condition object
# odometer  float64
# color     object
# interior  object
# seller    object
# mmr       int64
# sellingprice int64
# saledate  object
```

We can see from the output that we have a lot of features that are `dtype = object` and that tells us those features could be text or a mix of text and numerical values. For our encoding examples, we will explore a few of those object features and transform those values so we can have a data frame ready for machine learning.

The reason we put our time into this level of encoding is that there are many machine learning models that cannot handle text and will only work with numbers. Our data must be encoded into numbers before we even begin to train, test, or evaluate a model.

Ordinal encoding

We mentioned already that *ordinal data* is data that does have order and a hierarchy between its values. Let us take a look at the `condition` feature from our data frame and perform a `value_counts` to see how many times each label is listed in our feature.

```
print(cars['condition'].value_counts())
# #OUTPUT
# New      2881
# Like New 2860
# Good     2027
# Fair     753
# Excellent 186
```

This is definitely an example of ordinal data: the condition of the used cars can easily be put in order of those in the “best” condition to the cars in the “worst” condition. The output printed the labels with the highest counts, but we can assume the following hierarchy:

We need to convert these labels into numbers, and we can do this with two different approaches. First, we can do this by creating a dictionary where every label is the key and the new numeric number is the value. ‘Excellent’ will get the highest score and ‘Fair’ will be our lowest score. Then we will map each label from the `condition` column to the numeric value and create a new column called `condition_rating`.

```
# create dictionary of label:values in order
rating_dict = {'Excellent':5, 'New':4, 'Like New':3, 'Good':2, 'Fair':1}

#create a new column
cars['condition_rating'] = cars['condition'].map(rating_dict)
```

The second approach we will show is how to utilize the `sklearn.preprocessing` library `OrdinalEncoder`. We follow a similar approach: we set our categories as a list, and then we will `.fit_transform` the values in our feature `condition`. We need to make sure we adhere to the shape requirements of a 2-D array, so you'll notice the method `.reshape(-1,1)`.

We'll also note, this method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
# using scikit-learn
from sklearn.preprocessing import OrdinalEncoder

# create encoder and set category order
encoder = OrdinalEncoder(categories=[['Excellent', 'New', 'Like New', 'Good', 'Fair']])

# reshape our feature
condition_resaped = cars['condition'].values.reshape(-1,1)

# create new variable with assigned numbers
cars['condition_rating'] = encoder.fit_transform(condition_resaped)
```

Label Encoding

Now, we can talk about *nominal data*, and we have to approach this type of data differently than what we did with *ordinal data*. Our `color` feature has a lot of different labels, but here are the top five colors that appear in our data frame.

```
print(cars['color'].nunique())
# #OUTPUT
# 19

print(cars['color'].value_counts()[:5])
# #OUTPUT
# black      2015
# white      1931
# gray       1506
# silver     1503
# blue       869
```

To prepare this feature, we still need to convert our text to numbers, so let's do just that. We will demonstrate two different approaches, with the first one showing how to convert the feature from an `object` type to a `categories` type.

```
# convert feature to category type
cars['color'] = cars['color'].astype('category')

# save new version of category codes
cars['color'] = cars['color'].cat.codes

# print to see transformation
print(cars['color'].value_counts()[:5])
# #OUTPUT
# 2      2015
```

```
# 18      1931
# 8        1506
# 15       1503
# 3         869
```

Comparing our newly transformed data to the original top 5 list, we can see Black was transformed to number 2, White was transformed to 18, and so on.

However, we have created a problem for ourselves and potentially our model. We can see that 'Blue' cars now have a value of 3, and our model will assume that 'Blue' has lower precedence over the 'Black' car, whose color has a value of 2. Since 'Blue' cars = 3 and 'White' cars = 18, our model could actually give 'White' cars 6 times more weight than a 'Blue' car simply because of the way we encoded this feature. To combat this ordinal assumption our model will make, we should *one-hot encode* our nominal data, which we will cover in the next section.

One more way we can transform this feature is by using `sklearn.preprocessing` and the `LabelEncoder` library. This method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
from sklearn.preprocessing import LabelEncoder

# create encoder
encoder = LabelEncoder()

# create new variable with assigned numbers
cars['color'] = encoder.fit_transform(cars['color'])
```

One-hot Encoding

One-hot encoding is when we create a dummy variable for each value of our categorical feature, and a *dummy variable* is defined as a numeric variable with two values: 1 and 0. We will continue to talk about our `color` feature from our used car dataset.

Looking at this visual below, we can see we have ten cars in four different colors. In place of the single `color` column, we create four dummy variables - one new column for each color. Then the values that go into that column are binary, indicating if the car in that row is the color of the column name (1) or not (0).

Cars	Yellow	Green	Pink	Blue
	1	0	0	0
	0	1	0	0
	1	0	0	0
	1	0	0	0
	0	0	0	1
	0	1	0	0
	0	0	0	1
	0	0	1	0
	1	0	0	0
	0	1	0	0

This approach is great for our color feature and will allow the model to see each category as its own feature and not try to create order between a “Black car” and a “Red car”. Here is how we can implement this in Python:

```
import pandas as pd
# use pandas .get_dummies method to create one new column for each color
ohe = pd.get_dummies(cars['color'])

# join the new columns back onto our cars dataframe
cars = cars.join(ohe)
```

A downside to this approach is that it can create a lot of features which can then create a very sparse matrix.

Questions

One-hot encode the feature `body` from our cars dataset.

Hint

Binary encoding

If we find the need to one-hot encode a lot of categorical features which would, in turn, create a sparse matrix and may cause problems for our model, a strong alternative to this issue is performing a *binary encoder*. A *binary encoder* will find the number of

unique categories and then convert each category to its binary representation. Let us take a quick review of binary numbers and keep using our `color` feature. We know that we have 19 unique colors, so the way to represent the numbers from 1 to 19 in binary format is as follows:

Number	Binary Number
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100

13	1101
14	1110
15	1111
16	10000
17	10001
18	10010
19	10011

We can easily see that our highest number 19 is 5 digits long, so our binary encoder will need 5 columns to be able to represent all digits. Here is a sample of how our `color` column will transform each color if we were to perform a binary encoder.

color_1	color_2	color_3	color_4	color_5	color_text
0	0	0	0	1	blue
0	0	0	1	0	—
0	0	0	1	1	gold
0	0	1	0	0	white
0	0	1	0	1	red
0	0	1	1	0	silver
0	0	1	1	1	black
0	1	0	0	0	gray
0	1	0	0	1	burgundy
0	1	0	1	0	brown
0	1	0	1	1	turquoise
0	1	1	0	0	orange
0	1	1	0	1	green
0	1	1	1	0	beige
0	1	1	1	1	yellow
1	0	0	0	0	purple
1	0	0	0	1	off-white
1	0	0	1	0	charcoal
1	0	0	1	1	pink

Our 19th color, pink, has transformed to be represented in the binary form 10011. If we were to utilize this process instead of the traditional one-hot encoder we would have 5 numerical features instead of 19, reducing our features by about 75%!

To make this happen with Python we'll use a library called `category_encoders` and import `BinaryEncoder`. We will determine which column to transform and set `drop_invariant` to `True` so it will keep the five binary columns. If it is set to the default 0, then we would have an additional column full of zeros.

```
from category_encoders import BinaryEncoder

#this will create a new data frame with the color column removed and replaced with our 5 new
binary feature columns
colors = BinaryEncoder(cols = ['color'], drop_invariant = True).fit_transform(cars)
```

Hashing

Another option we have available to us is an encoding technique called *hashing*. This process is similar to one-hot encoding where it will create new binary columns, but within the parameters, you can decide how many features to output. A huge advantage is reduced dimensionality, but a large disadvantage is that some categories will be mapped to the same values. That is called *collision*.

For example, we have 19 different colored cars. If I were to use the hash encoder and set the number of features to be 5, I will definitely have a few colors with the same hash values.

	col_0	col_1	col_2	col_3
color				
beige	0	1	0	0
black	0	0	0	1
blue	1	0	0	0
brown	0	0	1	0
burgundy	0	1	0	0
charcoal	0	0	1	0
gold	1	0	0	0
gray	0	1	0	0
green	0	0	0	0
off-white	0	0	1	0
orange	1	0	0	0
pink	1	0	0	0
purple	0	0	0	1
red	0	0	1	0
silver	0	1	0	0
turquoise	0	1	0	0
white	0	0	1	0
yellow	0	0	0	0
—	0	1	0	0

We can easily see that brown and charcoal colors have the same hash values. Meaning, we've lost some information and our model won't be able to see the difference between those two colors.

Here is how we can make this work with Python. Our final result of `hash_results` will produce a data frame of just 5 columns, so we will want to concatenate this new data onto our original data frame.

```
from category_encoders import HashingEncoder
```

```
# instantiate our encoder
encoder = HashingEncoder(cols='color', n_components=5)

# do a fit transform on our color column and set to a new variable
hash_results = encoder.fit_transform(cars['color'])
```

Now you may be thinking, when would I use this if I'm going to lose information and my model will see brown and charcoal (or some other color combo with the same hash value) as the same thing? Well, this could be a solution to your project and dataset if you are not as interested in assessing the impact of any particular categorical value.

For this example, maybe you aren't interested in knowing which color car had an impact on your final prediction, but you want to be able to get the best performance from your model. This encoding solution may be a good approach.

Target encoding

Target encoding is a Bayesian encoder used to transform categorical features into hashed numerical values and is sometimes called the *mean encoder*. This encoder can be utilized for data sets that are being prepared for regression-based supervised learning, as it needs to take into consideration the mean of the target variable and its correlation between each individual category of our feature. In fact, the numerical values of each category is replaced with a blend of the posterior probability of the target given a particular categorical value and the prior probability of the target over all the training data.

Woah, now that was a lot of Bayesian buzzwords. How would it work with our specific `color` feature? It replaces each color with a blend of the mean price of that car color and the mean price of all the cars. Had it been predicting something categorical, it would've used a Bayesian target statistic.

Some drawbacks to this approach are overfitting and unevenly distributed values that could lead to extremes. Let's review how to implement this in Python and check out what type of numerical values it will return. Again, we'll continue with our color feature - hope you are not yet tired of it!

Say we are preparing our dataset for a regression-based supervised learning algorithm that is trying to predict the selling price.

```
from category_encoders import TargetEncoder

# instantiate our encoder
encoder = TargetEncoder(cols = 'color')

# set the results of our fit_transform to a variable
# the output will be its own pandas series
encoder_results = encoder.fit_transform(cars['color'], cars['sellingprice'])

print(encoder_results.head())
#   color
# 0 11761.881473
# 1 18007.276995
# 2  8458.251232
# 3 14769.292595
# 4 12691.099747
```

We can examine all the different values our `encoder_results` holds, and if we look at the output from below we can see our min is about 3,054 and our max is about 18,048. That is quite a big difference!

```
# print all 19 unique values
print(np.sort(encoder_results['color'].unique()))
# OUTPUT
# [ 3054.12209927  8088.87434555  8458.25123153  9276.78571429
#   9867.50002121  9885.8093167   11043.90243902  11247.82608763
#  11761.88147296  11805.06187625  12124.83443709  12376.19047882
#  12691.09974747  13912.83399734  14769.29259451  15496.72704715
#  17174.36440678  17176.25931731  18007.27699531  18048.52540833]
```

Encoding date-time variables

Every data analyst or scientist will have to work with date-time objects at some point in their career. There is so much information to be gained from date-time objects. We will work with the `saledate` feature - yay, a new column to explore! The very first thing we need to do is convert this to a date-time object.

```
print(cars['saledate'].dtypes)
# # OUTPUT
# dtype('O')

cars['saledate'] = pd.to_datetime(cars['saledate'])
# #OUTPUT
# datetime64[ns, tzlocal()]
```

Now that we have our feature set up as a datetime object, let's demonstrate some methods we can utilize to get additional information.

```
# create new variable for month
cars['month'] = cars['saledate'].dt.month

# create new variable for day of the week
cars['dayofweek'] = cars['saledate'].dt.day

# create new variable for difference between cars model year and year sold
cars['yearbuild_sold'] = cars['saledate'].dt.year - cars['year']
```

Below are other options available through pandas `.dt`

Syntax	Description & Output
<code>df['col'].dt.year</code>	Outputs the year
<code>df['col'].dt.day</code>	Outputs the day number
<code>df['col'].dt.hour</code>	Outputs the hour from the time

`df['col'].dt.minute` Outputs the minute from the time

`df['col'].dt.second` Outputs the seconds from the time

`df['col'].dt.week` Outputs the week ordinal of the year

`df['col'].dt.dayofweek` Outputs the day of the week with Monday = 0 & Sunday = 6

Additional methods can be found [here](#).

Review

Wow, we covered a lot in this article! You have so many options when it comes to transforming categorical data into numerical values. We touched in:

- **Ordinal encoding**
- **Label encoding**
- **One-hot encoding**
- **Binary encoding**
- **Hashing**
- **Target encoding**
- **Date-time encoding**

We hope you can see the importance of getting your categorical features just right before you begin to train and test a model

Supervised Learning

Introduction to Supervised Learning Part I

Supervised machine learning fun!

Supervised learning is at the heart of Machine Learning. It typically depends on human-generated training data that leverages computational approaches to finding patterns and making generalizations. In many ways, supervised learning is a way of asking computers to find patterns about the world that we simply cannot spot. Let's dive in and see what it's all about!

What will Supervised Learning for Data Science (Part I) Cover?

After this unit, you will be able to use and apply:

- *Simple Linear Regression*
- *Multiple Linear Regression*
- *Logistic Regression*
- *the K Nearest Neighbors Algorithm*
- *Decision Trees Algorithms*
- *Evaluation Metrics to assess the performance of your models.*

Regression vs. Classification

Learn about the two types of Supervised Learning algorithms.

Machine Learning is a set of many different techniques that are each suited to answering different types of questions.

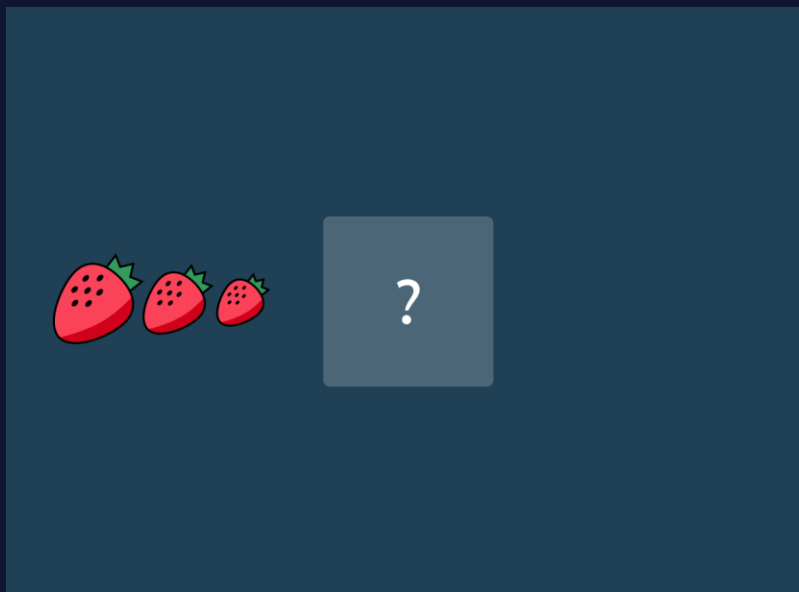
One way of categorizing machine learning algorithms is by using the kind output they produce. In terms of output, two main types of machine learning models exist: those for regression and those for classification.

Regression

Regression is used to predict outputs that are *continuous*. The outputs are quantities that can be flexibly determined based on the inputs of the model rather than being confined to a set of possible labels.

For example:

Predict the height of a potted plant from the amount of rainfall
Predict salary based on someone's age and availability of high-speed internet
Predict a car's MPG (miles per gallon) based on size and model year



Linear regression is the most popular regression algorithm. It is often underrated because of its relative simplicity. In a business setting, it could be used to predict the likelihood that a customer will churn or the revenue a customer will generate. More complex models may fit this data better, at the cost of losing simplicity.

Classification

Classification is used to predict a *discrete label*. The outputs fall under a finite set of possible outcomes. Many situations have only two possible outcomes. This is called *binary classification* (True/False, 0 or 1, 🍌 / not 🍌).

For example:

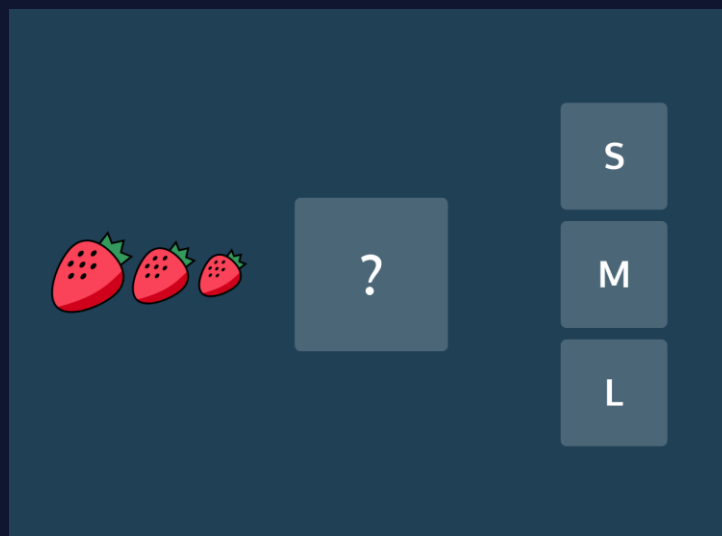
Predict whether an email is spam or not
Predict whether it will rain or not
Predict whether a user is a power user or a casual user

There are also two other common types of classification: *multi-class classification* and *multi-label classification*.

Multi-class classification has the same idea behind binary classification, except instead of two possible outcomes, there are three or more.

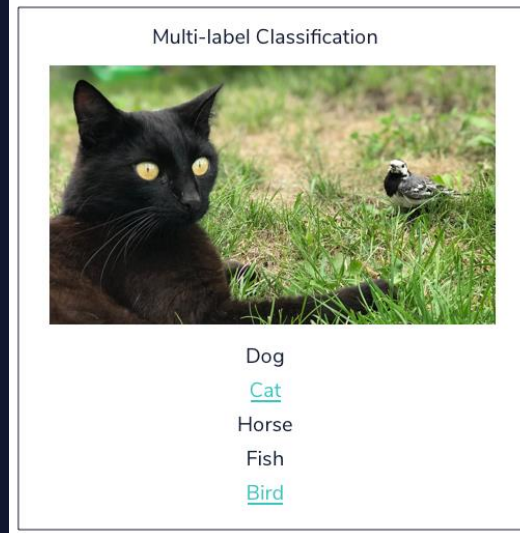
For example:

Predict whether a photo contains a pear, apple, or peach
Predict what letter of the alphabet a handwritten character is
Predict whether a piece of fruit is small, medium, or large



An important note about binary and multi-class classification is that in both, each outcome has one specific label. However, in multi-label classification, there are multiple possible labels for each outcome. This is useful for customer segmentation, image categorization, and sentiment analysis for understanding text. To perform these classifications, we use models like Naïve Bayes, K-Nearest Neighbors, SVMs, as well as various deep learning models.

An example of multi-label classification is shown below. Here, a cat and a bird are both identified in a photo showing a classification model with more than one label as a result.



Choosing a model is a critical step in the [Machine Learning process](#). It is important that the model fits the question at hand. When you choose the right model, you are already one step closer to getting meaningful and interesting results.

Introduction to Multiple Linear Regression

Linear regression is useful when we want to predict the values of a variable from its relationship with other

Preview: Docs Loading link description

[variables](#)

. There are two different types of linear regression models ([simple linear regression](#) and multiple linear regression). In predicting the price of a home, one factor to consider is the size of the home. The relationship between those two variables, price and size, is important, but there are other variables that factor in to pricing a home: location, air quality, demographics, parking, and more. When making predictions for price, our *dependent variable*, we'll want to use multiple *independent variables*. To do this, we'll use Multiple Linear Regression.

Multiple Linear Regression uses two or more independent variables to predict the values of the dependent variable. It is based on the following equation that we'll explore later on:

$$y=b+m_1x_1+m_2x_2+...+m_nx_n$$

StreetEasy Dataset

You'll learn multiple linear regression by performing it on this dataset. It contains information about apartments in New York.

Instructions

Checkpoint 1 Passed

1.

Before we start digging into the StreetEasy data, add this line at the end of **script.py**:

```
plt.show()
```

Copy to Clipboard

And then press run to see the graph!

In this example, we used size (ft²) and building age (years) as independent variables to predict the rent (\$).

When we have two independent variables, we can create a linear regression plane. We can now guess what the rent is by plugging in the independent variables and finding where they lie on the plane.

Hint

Don't worry if you don't quite understand the model just yet!

When we are looking at relationships, the **independent variables** are what you change. The **dependent variable** is what you measure. So, if we are looking at the way soil quality and watering frequency affects the height of a house plant, the independent variables would be:

Soil quality

Watering frequency

The dependent variable would be height.

```
import codecademylib3_seaborn

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from mpl_toolkits.mplot3d import Axes3D

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

streeteasy = pd.read_csv("https://raw.githubusercontent.com/sonnynomnom/Codecademy-Machine-Learning-Fundamentals/master/StreetEasy/manhattan.csv")

df = pd.DataFrame(streeteasy)

x = df[['size_sqft', 'building_age_yrs']]
y = df[['rent']]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.8, test_size = 0.2, random_state=6)

ols = LinearRegression()

ols.fit(x_train, y_train)

# Plot the figure

fig = plt.figure(1, figsize=(6, 4))
plt.clf()

elev = 43.5
azim = -110

ax = Axes3D(fig, elev=elev, azim=azim)

ax.scatter(x_train[['size_sqft']], x_train[['building_age_yrs']], y_train, c='k', marker='+')

ax.plot_surface(np.array([[0, 0], [4500, 4500]]), np.array([[0, 140], [0, 140]]),
ols.predict(np.array([[0, 0, 4500, 4500], [0, 140, 0, 140]]).T).reshape((2, 2)), alpha=.7)

ax.set_xlabel('Size (ft$^2$)')
ax.set_ylabel('Building Age (Years)')
ax.set_zlabel('Rent ($)')

ax.w_xaxis.set_ticklabels([])
ax.w_yaxis.set_ticklabels([])
ax.w_zaxis.set_ticklabels([])

# Add the code below:
plt.show()
```

StreetEasy Dataset



[StreetEasy](#) is New York City's leading real estate marketplace — from studios to high-rises, Brooklyn Heights to Harlem.

In this lesson, you will be working with a dataset that contains a sample of 5,000 rentals listings in [Manhattan](#), [Brooklyn](#), and [Queens](#), active on StreetEasy in June 2016.

It has the following columns:

```
rental_id: rental ID
rent: price of rent in dollars
bedrooms: number of bedrooms
bathrooms: number of bathrooms
size_sqft: size in square feet
min_to_subway: distance from subway station in minutes
floor: floor number
building_age_yrs: building's age in years
no_fee: does it have a broker fee? (0 for fee, 1 for no fee)
has_roofdeck: does it have a roof deck? (0 for no, 1 for yes)
has_washer_dryer: does it have washer/dryer in unit? (0/1)
has_doorman: does it have a doorman? (0/1)
has_elevator: does it have an elevator? (0/1)
has_dishwasher: does it have a dishwasher (0/1)
has_patio: does it have a patio? (0/1)
has_gym: does the building have a gym? (0/1)
neighborhood: (ex: Greenpoint)
borough: (ex: Brooklyn)
```

More information about this dataset can be found in the [StreetEasy Dataset](#) article.

Let's start by doing exploratory data analysis to understand the dataset better. We have broken the dataset for you into:

[manhattan.csv](#)
[brooklyn.csv](#)
[queens.csv](#)

Instructions

Checkpoint 1 Passed

1.

First, pick a borough out of the three (Manhattan, Brooklyn, and Queens) that you are most interested in!

We are going to import the dataset and store it in a variable called `df`.

To import, we will need to run this snippet:

```
pd.read_csv("path")
```

Copy to Clipboard

Replace `path` with one of the three URL's above.

Hint

For example, if you want to take a look at the Manhattan data:

```
df =  
pd.read_csv("https://raw.githubusercontent.com/Codecademy/datasets/master/streeteasy/manhattan.csv")
```

Copy to Clipboard

Each dataset is going to be very different from the others. In this lesson, you are going to pick one of them!

And now we have a DataFrame named `df`.

A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. You can think of it like a spreadsheet or a SQL table. It is generally the most commonly used `pandas` object.

The image you are
requesting does not exist
or is no longer available.

imgur.com

Checkpoint 2 Passed

2.

Let's take a look at the first few rows using `df.head()`:

How far is the apartment in the third row from a subway station?

Which neighborhood is it in?

Hint

```
print(df.head())
```

Copy to Clipboard

A table should appear on the right panel with column names and 5 rows of apartments.

The answers to these questions depend on the borough that you picked!

Training Set vs. Test Set

As with most

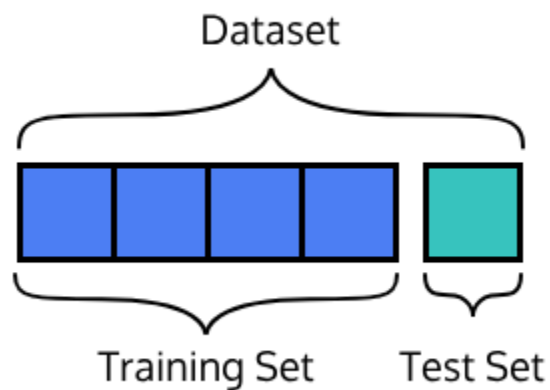
Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

algorithms, we have to split our dataset into:

Training set: the data used to fit the model

Test set: the data partitioned away at the very start of the experiment (to provide an unbiased evaluation of the model)



In general, putting 80% of your data in the training set and 20% of your data in the test set is a good place to start.

Suppose you have some values in `x` and some values in `y`:

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8,
                                                    test_size=0.2)
```

Copy to Clipboard

Here are the parameters:

`train_size`: the proportion of the dataset to include in the train split (between 0.0 and 1.0)
`test_size`: the proportion of the dataset to include in the test split (between 0.0 and 1.0)
`random_state`: the seed used by the random number generator [optional]

1. Import `train_test_split` from `sklearn.model_selection`.

Stuck? Get a hint

Checkpoint 2 Passed

2. Create a DataFrame `x` that selects the following columns from the main `df` DataFrame:

```
'bedrooms'
'bathrooms'
```

```
'size_sqft'
'min_to_subway'
'floor'
'building_age_yrs'
'no_fee'
'has_roofdeck'
'has_washer_dryer'
'has_doorman'
'has_elevator'
'has_dishwasher'
'has_patio'
'has_gym'
```

Create a DataFrame `y` that selects the `rent` column from the main `df` DataFrame.
These are the columns we want to use for our regression model.

Stuck? Get a hint

3. Use scikit-learn's `train_test_split()` method to split `x` into 80% training set and 20% testing set and generate:

```
x_train
x_test
y_train
y_test
```

Set the `random_state` to 6.

Stuck? Get a hint

4. Let's take a look at the shapes of `x_train`, `x_test`, `y_train`, and `y_test` to see we got the proportion we wanted.

We have 14 features that we're looking for for each apartment, and 1 label we're looking for for each apartment.

```
import pandas as pd

from sklearn.model_selection import train_test_split

streeteasy = pd.read_csv("https://raw.githubusercontent.com/sonnynomnom/Codecademy-Machine-Learning-Fundamentals/master/StreetEasy/manhattan.csv")

df = pd.DataFrame(streeteasy)

x = df[['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher', 'has_patio', 'has_gym']]

y = df[['rent']]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.8, test_size =
0.2, random_state=6)

print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Multiple Linear Regression: Scikit-Learn

Now we have the training set and the test set, let's use scikit-learn to build the linear regression model!

The steps for multiple linear regression in scikit-learn are identical to the steps for simple linear regression. Just like simple linear regression, we need to import `LinearRegression` from the `linear_model` module:

```
from sklearn.linear_model import LinearRegression
```

Copy to Clipboard

Then, create a `LinearRegression` model, and then fit it to your `x_train` and `y_train` data:

```
mlr = LinearRegression()

mlr.fit(x_train, y_train)
# finds the coefficients and the intercept value
```

Copy to Clipboard

We can also use the `.predict()` function to pass in x-values. It returns the y-values that this plane would predict:

```
y_predicted = mlr.predict(x_test)
# takes values calculated by `.fit()` and the `x` values, plugs them into the multiple
linear regression equation, and calculates the predicted y values.
```

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

streeteasy = pd.read_csv("https://raw.githubusercontent.com/sonnynomnom/Codecademy-Machine-
Learning-Fundamentals/master/StreetEasy/manhattan.csv")

df = pd.DataFrame(streeteasy)

x = df[['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher', 'has_patio', 'has_gym']]

y = df[['rent']]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.8, test_size =
0.2, random_state=6)

# Add the code here:

mlr = LinearRegression()

mlr.fit(x_train, y_train)

y_predict = mlr.predict(x_test)

# Sonny doesn't have an elevator so the 11th item in the list is a 0
sonny_apartment = [[1, 1, 620, 16, 1, 98, 1, 0, 1, 0, 0, 1, 1, 0]]

predict = mlr.predict(sonny_apartment)

print("Predicted rent: $%.2f" % predict)
```

Visualizing Results with Matplotlib

You've performed Multiple Linear Regression, and you also have the predictions in `y_predict`. However, we don't have insight into the data, yet. In this exercise, you'll create a 2D scatterplot to see how the independent

Preview: Docs Loading link description

[variables](#)

impact prices.

How do you create 2D graphs?

Graphs can be created using Matplotlib's `pyplot` module. Here is the code with inline comments explaining how to plot using Matplotlib's `.scatter()`:

```
# Create a scatter plot
plt.scatter(x, y, alpha=0.4)

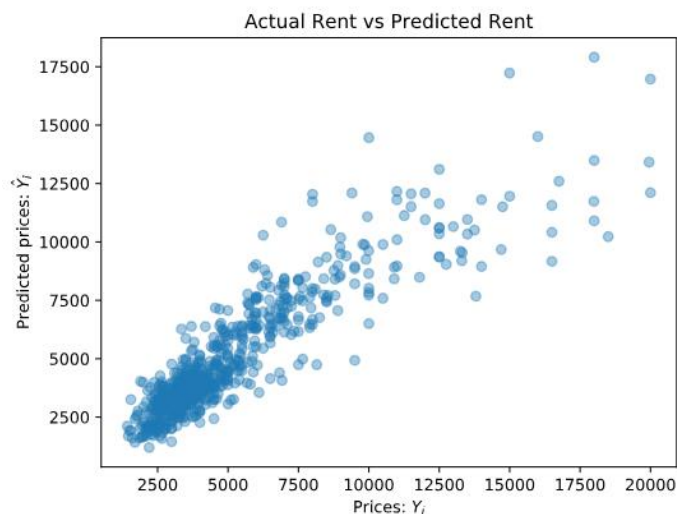
# Create x-axis label and y-axis label
plt.xlabel("the x-axis label")
plt.ylabel("the y-axis label")

# Create a title
plt.title("title!")

# Show the plot
plt.show()
```

Copy to Clipboard

We want to create a scatterplot like this:



Instructions

1. Create a 2D scatter plot using `y_test` and `y_predict`.

The x-axis should represent actual rent prices and the y-axis should represent predicted rent prices.

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Add appropriate x-axis labels and y-axis labels, as well as a title.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3. Show the plot using `plt.show()`.

Multiple Linear Regression Equation

Now that we have implemented Multiple Linear Regression, we will learn how to tune and evaluate the model. Before we do that, however, it's essential to learn the equation behind it.

Equation 6.1 The equation for multiple linear regression that uses two independent variables is this:

$$y=b+m_1x_1+m_2x_2$$

Equation 6.2 The equation for multiple linear regression that uses three independent variables is this:

$$y=b+m_1x_1+m_2x_2+m_3x_3$$

Equation 6.3 As a result, since multiple linear regression can use any number of independent variables, its general equation becomes:

$$y=b+m_1x_1+m_2x_2+...+m_nx_n$$

Here, $m_1, m_2, m_3, \dots, m_n$ refer to the **coefficients**, and b refers to the **intercept** that you want to find. You can plug these values back into the equation to compute the predicted y values.

Remember, with `sklearn`'s `LinearRegression()` method, we can get these values with ease.

The `.fit()` method gives the model two variables that are useful to us:

`.coef_`, which contains the coefficients
`.intercept_`, which contains the intercept

After performing multiple linear regression, you can print the coefficients using `.coef_`.

Coefficients are most helpful in determining which independent variable carries more weight. For example, a coefficient of -1.345 will impact the rent more than a coefficient of 0.238, with the former impacting prices negatively and latter positively.

```
x = df[['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher', 'has_patio', 'has_gym']]

y = df[['rent']]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.8, test_size =
0.2, random_state=6)

mlr = LinearRegression()

model = mlr.fit(x_train, y_train)

y_predict = mlr.predict(x_test)

# Input code here:

print(mlr.coef_)
```

Correlations

In our Manhattan model, we used 14

Preview: Docs Loading link description
[variables](#)

, so there are 14 coefficients:

```
[ -302.73009383  1199.3859951  4.79976742
 -24.28993151  24.19824177 -7.58272473
 -140.90664773  48.85017415 191.4257324
 -151.11453388  89.408889  -57.89714551
 -19.31948556  -38.92369828 ]
```

Copy to Clipboard

```
bedrooms - number of bedrooms
bathrooms - number of bathrooms
size_sqft - size in square feet
min_to_subway - distance from subway station in minutes
floor - floor number
building_age_yrs - building's age in years
no_fee - has no broker fee (0 for fee, 1 for no fee)
has_roofdeck - has roof deck (0 for no, 1 for yes)
has_washer_dryer - has in-unit washer/dryer (0/1)
has_doorman - has doorman (0/1)
has_elevator - has elevator (0/1)
has_dishwasher - has dishwasher (0/1)
has_patio - has patio (0/1)
has_gym - has gym (0/1)
```

To see if there are any features that don't affect price linearly, let's graph the different features against `rent`.

Interpreting graphs

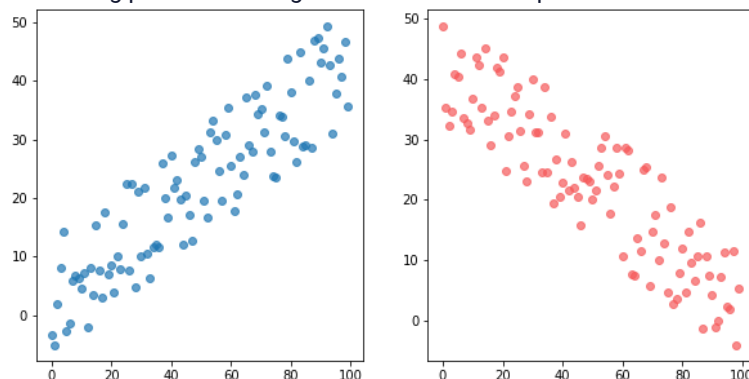
In regression, the independent variables will either have a positive linear relationship to the dependent variable, a negative linear relationship, or no relationship. A negative linear relationship means that as X values *increase*, Y values will *decrease*. Similarly, a positive linear relationship means that as X values *increase*, Y values will also *increase*.

Graphically, when you see a downward trend, it means a negative linear relationship exists. When you find an upward trend, it indicates a positive linear relationship. Here are two

Preview: Docs Graphs are a way of modeling systems based on a node and edge structure for representing the relationships between items.

[graphs](#)

indicating positive and negative linear relationships:



Instructions

Checkpoint 1 Passed

1.

Create a scatterplot of `size_sqft` and `rent`:

```
plt.scatter(df[['size_sqft']], df[['rent']], alpha=0.4)
```

Copy to Clipboard
Is there a strong correlation?

Stuck? Get a hint

Checkpoint 2 Passed

2.

Create a scatterplot of `min_to_subway` and `rent`:

```
plt.scatter(df[['min_to_subway']], df[['rent']], alpha=0.4)
```

Copy to Clipboard
Is there a strong correlation?

Stuck? Get a hint

Checkpoint 3 Passed

3.

Do the same for a few others and write down the ones that don't have strong correlations.

```
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

streeteasy = pd.read_csv("https://raw.githubusercontent.com/sonnynomnom/Codecademy-Machine-Learning-Fundamentals/master/StreetEasy/manhattan.csv")

df = pd.DataFrame(streeteasy)

# Input code here:

plt.scatter(df[['size_sqft']], df[['rent']], alpha=0.4)
# plt.scatter(df[['floor']], df[['rent']])
# plt.scatter(df[['min_to_subway']], df[['rent']])

plt.show()
```

Evaluating the Model's Accuracy

When trying to evaluate the accuracy of our multiple linear regression model, one technique we can use is **Residual Analysis**.

The difference between the actual value y , and the predicted value $\hat{y}$ is the **residual e** . The equation is:

$$e = y - \hat{y}$$

In the StreetEasy dataset, y is the actual rent and the $\hat{y}$ is the predicted rent. The real y values should be pretty close to these predicted y values.

`sklearn's` `linear_model.LinearRegression` comes with a `.score()`

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

that returns the coefficient of determination R^2 of the prediction.
The coefficient R^2 is defined as:

$$1 - \frac{u}{v}$$

where u is the residual sum of squares:

```
((y - y_predict) ** 2).sum()
```

Copy to Clipboard

and v is the total sum of squares (TSS):

```
((y - y.mean()) ** 2).sum()
```

Copy to Clipboard

The TSS tells you how much variation there is in the y variable.

R^2 is the percentage variation in y explained by all the x

Preview: Docs Loading link description

[variables](#)

together.

For example, say we are trying to predict `rent` based on the `size_sqft` and the `bedrooms` in the apartment and the R^2 for our model is 0.72 — that means that all the x variables (square feet and number of bedrooms) together explain 72% variation in y (`rent`).

Now let's say we add another x variable, building's age, to our model. By adding this third relevant x variable, the R^2 is expected to go up. Let say the new R^2 is 0.95. This means that square feet, number of bedrooms and age of the building *together* explain 95% of the variation in the rent.

The best possible R^2 is 1.00 (and it can be negative because the model can be arbitrarily worse). Usually, a R^2 of 0.70 is considered good.

Instructions

Checkpoint 1 Failed, try again

1.

Use the `.score()` method from `LinearRegression` to find the coefficient of determination (R^2) for the **training set**.

Write that number down.

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `.score()` method from `LinearRegression` to find the coefficient of determination (R^2) for the **testing set**.

Write that number down.

```
df = pd.DataFrame(streeteasy)

x = df[['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher', 'has_patio', 'has_gym']]

y = df[['rent']]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.8, test_size =
0.2, random_state=6)

mlr = LinearRegression()
```

```

model=mlr.fit(x_train, y_train)

y_predict = mlr.predict(x_test)

# Input code here:
print("Train score:")
print(mlr.score(x_train, y_train))

print("Test score:")
print(mlr.score(x_test, y_test))

```

Rebuild the Model

Now let's rebuild the model using the new features as well as evaluate the new model to see if we improved!

For `Manhattan`, the scores returned:

```

Train score: 0.772546055982
Test score: 0.805037197536

```

Copy to Clipboard

For `Brooklyn`, the scores returned:

```

Train score: 0.613221453798
Test score: 0.584349923873

```

Copy to Clipboard

For `Queens`, the scores returned:

```

Train score: 0.665836031009
Test score: 0.665170319781

```

Copy to Clipboard

For whichever borough you used, let's see if we can improve these scores!

Instructions

Checkpoint 1 Passed

1.

Print the coefficients again to see which ones are strongest.

Checkpoint 2 Passed

2.

Currently the `x` should look something like:

```

x = df[['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor',
'building_age_yrs', 'no_fee', 'has_roofdeck', 'has_washer_dryer',
'has_doorman', 'has_elevator', 'has_dishwasher', 'has_patio',
'has_gym']]

```

Copy to Clipboard

Remove some of the features that don't have strong correlations and see if your scores improved!

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-i/modules/dsml-multiple-linear-regression/lessons/multiple-linear-regression-streeteasy/exercises/review>

Ordinary Least Squares to Gradient Descent

Solving a Regression Problem: Ordinary Least Squares to Gradient Descent

Learn about how the linear regression problem is solved analytically (using Ordinary Least Squares) and algorithmically (using Gradient Descent) and how these two methods are connected!

Linear regression finds a linear relationship between one or more predictor variables and an outcome variable. This article will explore two different ways of finding linear relationships: *Ordinary Least Squares* and *Gradient Descent*.

Ordinary Least Squares

To understand the method of least squares, let's take a look at how to set up the linear regression problem with linear equations. We'll use the [Diabetes dataset](#) as an example. The outcome variable, Y , is a measure of disease progression. There are 10 predictor variables in this dataset, but to simplify things let's take a look at just one of them: BP (average blood pressure). Here are the first five rows of data:

BP	Y
32.1	151
21.6	75

0.5

141

25.3

206

23

135

We can fit the data with the following simple linear regression model with slope m and intercept b :

$$Y = m * BP + b + \text{error}$$

$$Y = m * BP + b + \text{error}$$

This equation is actually short-hand for a large number of equations — one for each patient in our dataset. The first five equations (corresponding to the first five rows of the dataset) are:

$$\begin{aligned} 151 &= m * 32.1 + b + \text{error}_1 \\ 175 &= m * 21.6 + b + \text{error}_2 \\ 141 &= m * 0.5 + b + \text{error}_3 \\ 206 &= m * 25.3 + b + \text{error}_4 \\ 135 &= m * 23 + b + \text{error}_5 \end{aligned}$$

When we fit this linear regression model, we are trying to find the values of m and b such that the sum of the squared error terms above (e.g., $\text{error}_1^2 + \text{error}_2^2 + \text{error}_3^2 + \text{error}_4^2 + \text{error}_5^2 + \dots$) is minimized.

We can create a column matrix of Y (the outcome variable), a column matrix of BP (the predictor variable), and a column matrix of the errors and rewrite the five equations above as one matrix equation:

Using the rules of matrix addition and multiplication, it is possible to simplify this to the following.

In total we have 4 matrices in this equation:

A one-column matrix on the left hand side of the equation containing the outcome variable values that we will call Y

A two-column matrix on the right hand side that contains a column of 1's and a column of the predictor variable values (BP here) that we will call X .

A one-column matrix containing the intercept b and the slope m , i.e, the solution matrix that we will denote by the Greek letter β . The goal of the regression problem is to find this matrix.

A one-column matrix of the residuals or errors, the error matrix. The regression problem can be solved by minimizing the sum of the squares of the elements of this matrix. The error matrix will be denoted by the Greek letter ϵ .

Using these shorthands, the matrix representation of the regression equation is thus:

$$Y = X\beta + \epsilon$$

$$Y = X\beta + \epsilon$$

Ordinary Least Squares gives us an explicit formula for β . Here's the formula:

A couple of reminders: X^T is the *transpose* of X . M^{-1} is the *inverse* of a matrix. We won't review these terms here, but if you want to know more you can check the Wikipedia articles for the [transpose](#) and [invertible matrices](#).

This looks like a fairly simple formula. In theory, you should be able to plug in X and Y , do the computations, and get β . But it's not always so simple.

First of all, it's possible that the matrix XX^T might not even have an inverse. This will be the case if there happens to be an exact linear relationship between some of the columns of X . If there is such a relationship between the columns of X , we say that X is *multicollinear*. For example, your data set might contain temperature readings in both Fahrenheit and Celsius. Those columns would be linearly related, and thus XX^T would not have an inverse.

In practice, you also have to watch out for data that is almost multicollinear. For example, a data set might have Fahrenheit and Celsius readings that are rounded to the nearest degree. Due to rounding error, those columns would not be perfectly correlated. In that case it would still be possible to compute the inverse of XX^T , but it would lead to other problems. Dealing with situations like that is beyond the scope of this article, but you should be aware that multicollinearity can be troublesome.

Another drawback of the OLS equation for β is that it can take a long time to compute. Matrix multiplication and matrix inversion are both computationally intensive operations. Data sets with a large number of predictor variables and rows can make these computations impractical.

Gradient Descent

Gradient descent is a numerical technique that can determine regression parameters without resorting to OLS. It's an iterative process that uses calculus to get closer and closer to the exact coefficients one step at a time. To introduce the concept, we'll look at a simple example: linear regression with one predictor variable. For each row of data, we have the following equation:

This is the loss function. It depends on two variables: m and b . [Here's](#) an interactive plot where you can tune the parameters m and b and see how it affects the loss.

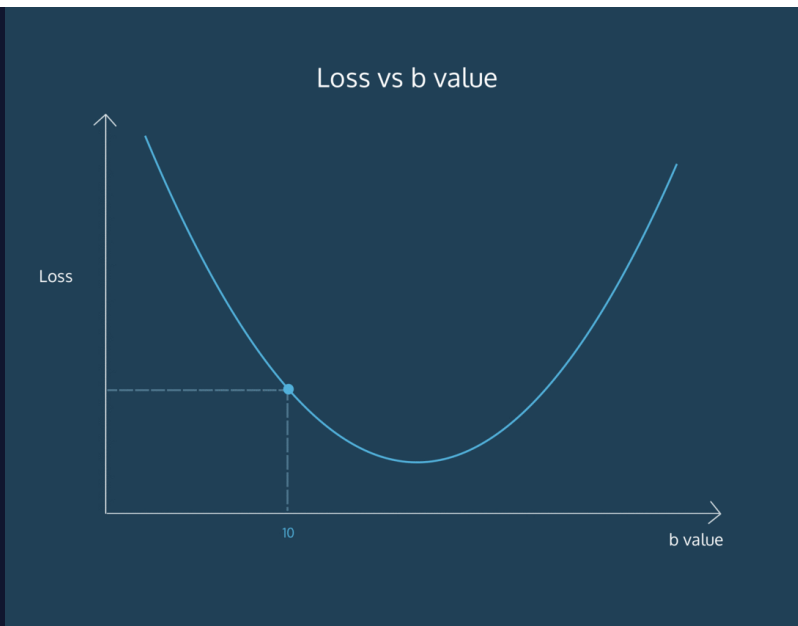
Try changing m and b and observe how those changes affect the loss. You can also try to come up with an algorithm for how to adjust m and b in order to minimize the loss.

As you adjust the sliders and try to minimize the loss, you might notice that there is a sweet spot where changing either m or b too far in either direction will increase the loss. Get too far away from that sweet spot, and small changes in m or b will result in bigger changes to the loss.

If playing with the sliders made you think about rates of change, derivatives, and calculus, then you're well on your way toward understanding gradient descent. The *gradient* of a function is a calculus concept that's very similar to the derivative of a function. We won't cover the technical definition here, but you can think of it as a vector that points uphill in the steepest direction. The steeper the slope, the larger the gradient. If you step in the direction opposite of the gradient, you will move downhill.

That's where the *descent* part of the gradient descent comes in. Imagine you're standing on some undulating terrain with peaks and valleys. Now take a step in the opposite direction of the gradient. If the gradient is large (in other words, if the slope you're on is steep), take a big step. If the gradient is small, take a small step. If you repeat this process enough times, you'll probably end up at the bottom of a valley. By going against the gradient, you've minimized your elevation!

Here's an example of how this works in two dimensions.



Let's take a closer look at how gradient descent works in the case of linear regression with one predictor variable. The process always starts by making starting guesses for m and b . The initial guesses aren't important. You could make random guesses, or just start with $m=0$ and $b=0$. The initial guesses will be adjusted by using gradient formulas.

Here's the gradient formula for b . This formula can be obtained by differentiating the average of the squared error terms with respect to b .

In this formula,

Here's the gradient formula for m . Again, this can be obtained by differentiating the average of the squared error terms with respect to m .

The next step of gradient descent is to adjust the current guesses for m and b by subtracting a number proportional the gradient.

Our new guess for b is

Our new guess for m is

For now, η is just a constant. We'll explain it in the next section. Once we get new guesses for m and b , we recompute the gradient and continue the process.

Multiple choice

Questions

Which of the following statements is FALSE?

Answer Choices

Gradient descent can be preferable to Ordinary Least Squares regression when dealing with a very large number of variables.

Ordinary Least Squares regression uses a matrix formula to find parameters that minimize loss.

Gradient descent is an iterative process.

In the context of linear regression, Ordinary Least Squares and gradient descent are two different techniques that minimize two different loss functions.

Learning Rate and Convergence

How big should your steps be when doing gradient descent? Imagine you're trying to get to the bottom of a valley, one step at a time. If you step one inch at time, it could take a very long time to get to the bottom. You might want to take bigger steps. On the other extreme, imagine you could cover a whole mile in a single step. You'd cover a lot of ground, but you might step over the bottom of the valley and end up on a mountain!

The size of the steps that you take during gradient descent depend on the gradient (remember that we take big steps when the gradient is steep, and small steps when the gradient is small). In order to further tune the size of the steps, machine learning algorithms multiply the gradient by a factor called the *learning rate*. If this factor is big, gradient descent will take bigger steps and hopefully reach the bottom of the valley faster. In other words, it "learns" the regression parameters faster. But if the learning rate is too big, gradient descent can overshoot the bottom of the valley and fail to converge.

How do you know when to stop doing gradient descent? Imagine trying to find the bottom of a valley if you were blindfolded. How would you know when you reached the lowest point?

If you're walking downhill (or doing gradient descent on a loss function), sooner or later you'll reach a point where everything flattens out and moving against the gradient will only reduce your elevation by a negligible amount. When that happens, we say that gradient descent *converges*. You might have noticed this when you were adjusting m and b on the interactive graph: when m and b are both near their sweet spot, small adjustments to m and b didn't affect the loss much.

To summarize what we've learned so far, here are the steps of gradient descent. We'll denote the learning rate by η .

If the algorithm fails to converge because the loss increases after some steps, the learning rate is probably too large. If the algorithm runs for a long time without converging, then the learning rate is probably too small.

Multiple choice

Questions

Gradient descent is being used to do linear regression. After each iteration of gradient descent, the loss gets larger. Which of the following should you try?

Answer Choices

Keep running the gradient descent algorithm until it converges.

Make the learning rate larger.

Make the learning rate smaller.

Implementation in sci-kit learn

Version 1.0.3 of the scikit-learn library has two different linear regression models: one that uses OLS and another that uses a variation of gradient descent.

The `LinearRegression` model uses OLS. For most applications this is a good approach. Even if a data set has hundreds of predictor variables or thousands of observations, your computer will have no problem computing the parameters using OLS. One advantage of OLS is that it is guaranteed to find the exact optimal parameters for linear regression. Another advantage is that you don't have to worry about what the learning rate is or whether the gradient descent algorithm will converge.

Here's some code that uses `LinearRegression`.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression

# Import the data set
X, y = load_diabetes(return_X_y=True)

# Create the OLS linear regression model
ols = LinearRegression()

# Fit the model to the data
ols.fit(X, y)

# Print the coefficients of the model
print(ols.coef_)

# Print R^2
print(ols.score(X, y))
```

Output:

```
[ -10.01219782 -239.81908937  519.83978679  324.39042769 -792.18416163
  476.74583782  101.04457032  177.06417623  751.27932109   67.62538639]
0.5177494254132934
```

Scikit-learn's `SGDRegressor` model uses a variant of gradient descent called *stochastic gradient descent* (or *SGD* for short). SGD is very similar to gradient descent, but instead of using the actual gradient it uses an approximation of the gradient that is more efficient to compute. This model is also sophisticated enough to adjust the learning rate as the SGD algorithm iterates, so in many cases you won't have to worry about setting the learning rate.

`SGDRegressor` also uses a technique called *regularization* that encourages the model to find smaller parameters. Regularization is beyond the scope of this article, but it's important to note that the use of regularization can sometimes result in finding different coefficients than OLS would have.

If your data set is simply too large for your computer to handle OLS, you can use `SGDRegressor`. It will not find the exact optimal parameters, but it will get close enough for all practical purposes and it will do so without using too much computing power. Here's an example.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import SGDRegressor

# Import the data set
X, y = load_diabetes(return_X_y=True)

# Create the SGD linear regression model
# max_iter is the maximum number of iterations of SGD to try before halting
sgd = SGDRegressor(max_iter = 10000)

# Fit the model to the data
sgd.fit(X, y)
```

```
# Print the coefficients of the model
print(sgd.coef_)

# Print R^2
print(sgd.score(X, y))
```

Output:

```
[ 12.19842555 -177.93853188  463.50601685  290.64175509 -33.34621692
 -94.62205923 -202.87056914  129.75873577  386.77536299  123.17079841]
0.5078357600233131
```

Gradient Descent in Other Machine Learning Algorithms

Gradient descent can be used for much more than just linear regression. In fact, it can be used to train any machine learning algorithm as long as the ML algorithm has a loss function that is a differentiable function of the ML algorithm's parameters. In more intuitive terms, gradient descent can be used whenever the loss function looks like smooth terrain with hills and valleys (even if those hills and valleys live in a space with more than 3 dimensions).

Gradient descent (or variations of it) can be used to find parameters in logistic regression models, support vector machines, neural networks, and other ML models. Gradient descent's flexibility makes it an essential part of machine learning.

Logistic Regression

Introduction

When an email lands in your inbox, how does your email service know whether it's real or spam? This evaluation is made billions of times per day, and one possible

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

is logistic regression.

Logistic regression is a supervised

Preview: Docs Loading link description

[machine learning](#)

Preview: Docs Loading link description

[algorithm](#)

that predicts the probability, ranging from 0 to 1, of a datapoint belonging to a specific category, or class. These probabilities can then be used to assign, or *classify*, observations to the more probable group.

For example, we could use a logistic regression model to predict the probability that an incoming email is spam. If that probability is greater than 0.5, we could automatically send it to a spam folder. This is called *binary classification* because there are only two groups (eg., spam or not spam).

Some other examples of problems that we could solve using logistic regression:

Disease identification — Is a tumor malignant?

Customer conversion — Will a customer arriving on a sign-up page enroll in a service?

In this lesson you will learn how to perform logistic regression and use it to make predictions!

If you are unfamiliar with linear regression, we recommend you review it before proceeding. Otherwise, let's dive in!

```
import codecademylib3
import pandas as pd
import matplotlib.pyplot as plt
codecademyU = pd.read_csv('codecademyU.csv')

# Scatter plot of exam passage vs number of hours studied
plt.scatter(x = 'hours_studied', y = 'passed_exam', data = codecademyU, color='black')
plt.ylabel('passed/failed')
plt.xlabel('hours studied')

plt.show()
```

Linear Regression Approach

With the data from Codecademy University, we want to predict whether each student will pass their final exam. Recall that in linear regression, we fit a line of the following form to the data:

$$y=b_0+b_1x_1+b_2x_2+\dots+b_nx_n$$

where

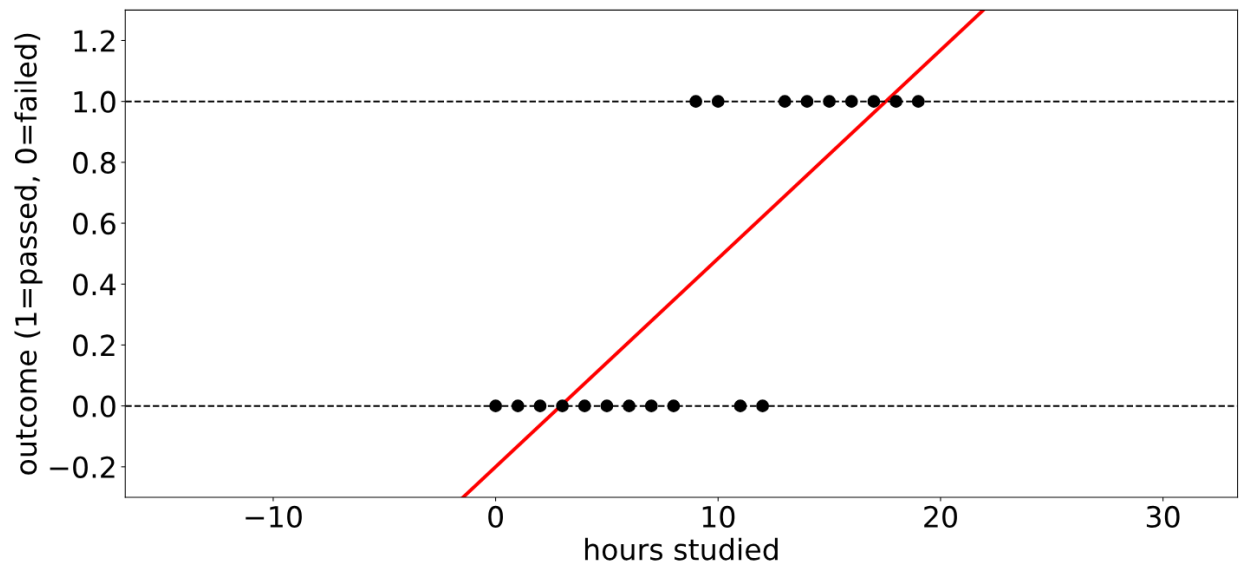
y is the value we are trying to predict
 b_0 is the intercept of the regression line
 $b_1, b_2, \dots, b_n$ are the coefficients
 $x_1, x_2, \dots, x_n$ are the predictors (also sometimes called *features*)

For our data, y is a

Preview: Docs Loading link description

[binary](#)

variable, equal to either 1 (passing), or 0 (failing). We have only one predictor (x_1): `num_hours_studied`. Below we've fitted a linear regression model to our data and plotted the results. The best fit line is in red.



We see that the linear model does not fit the data well. Our goal is to predict whether a student passes or fails; however, a best fit line allows predictions between negative and positive infinity.

Instructions

Checkpoint 1 Enabled

1.

We've provided you with the code to train a linear regression model on the Codecademy University data and plot the regression line. Run the code and observe the plot. Expand the plot to fullscreen for a larger view. Using the regression line, estimate the predicted outcomes (given by the line) for students who study 0 hour, 10 hours, and 30 hours, respectively. Save the results to `slacker`, `average`, and `studious`.

How would you use these numerical outcomes to determine whether a student is predicted to pass or fail?

Can you think of a threshold you might use?

Stuck? Get a hint

```
import codecademylib3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
codecademyU = pd.read_csv('codecademyU.csv')

# Define slacker, average, and studious below
slacker = -.2
average = 0.5
studious = 1.75

# Fit a linear model
```



```

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(codecademyU[['hours_studied']], codecademyU[['passed_exam']])

# Get predictions from the linear model
sample_x = np.linspace(-16.65, 33.35, 300).reshape(-1,1)
predictions = model.predict(sample_x)

# Plot the data
plt.scatter(x = 'hours_studied', y = 'passed_exam', data = codecademyU, color='black',
s=100)

# Plot the line
plt.plot(sample_x, predictions, color='red', linewidth=3)

# Customization for readability
plt.xticks(fontsize = 20)
plt.yticks(fontsize = 20)
plt.axhline(y=0, color='k', linestyle='--')
plt.axhline(y=1, color='k', linestyle='--')

# Label plot and set limits
plt.ylabel('outcome (1=passed, 0=failed)', fontsize = 15)
plt.xlabel('hours studied', fontsize = 15)
plt.xlim(-16.65, 33.35)
plt.ylim(-.3, 1.8)

# Show the plot
plt.tight_layout()
plt.show()

```

Logistic Regression

We saw that predicted outcomes from a linear regression model range from negative to positive infinity. These predictions don't really make sense for a classification problem. Step in **logistic regression**!

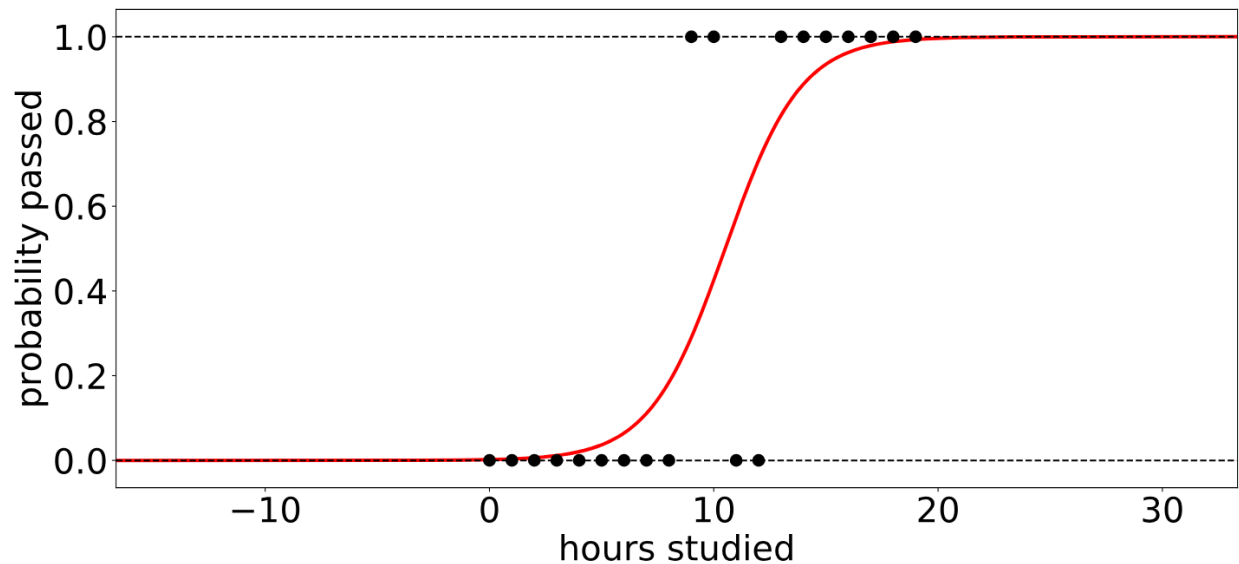
To build a logistic regression model, we apply a **logit link function** to the left-hand side of our linear regression function. Remember the equation for a linear model looks like this:

$$y=b_0+b_1x_1+b_2x_2+\dots+b_nx_n$$

When we apply the logit function, we get the following:

$$\ln(y1-y)=b_0+b_1x_1+b_2x_2+\dots+b_nx_n$$

For the Codecademy University example, this means that we are fitting the curve shown below to our data — instead of a line, like in linear regression:



Notice that the red line stays between 0 and 1 on the y-axis. It now makes sense to interpret this value as a probability of group membership; whereas that would have been non-sensical for regular linear regression.

Note that this is a pretty nifty trick for adapting a linear regression model to solve classification problems! There are actually many other kinds of link functions that we can use for different adaptations.

```
import codecademylib3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
codecademyU = pd.read_csv('codecademyU.csv')

# Define five_hour_studier below
five_hour_studier = 0.05

# Fit the logistic regression model
hours_studied = codecademyU[['hours_studied']]
passed_exam = codecademyU[['passed_exam']]
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(hours_studied, passed_exam)

# Plug sample data into fitted model
sample_x = np.linspace(-16.65, 33.35, 300).reshape(-1,1)
probability = model.predict_proba(sample_x)[:,-1]

# Plot exam data
plt.scatter(hours_studied, passed_exam, color='black', s=100)

# Plot logistic curve
plt.plot(sample_x, probability, color='red', linewidth=3)

# Customization for readability
plt.xticks(fontsize = 30)
plt.yticks(fontsize = 30)
plt.axhline(y=0, color='k', linestyle='--')
plt.axhline(y=1, color='k', linestyle='--')

# Label plot and set limits
plt.ylabel('probability passed', fontsize = 30)
```

```
plt.xlabel('hours studied', fontsize = 30)
plt.xlim(-1, 25)
plt.tight_layout()

# Show the plot
plt.show()
```

Log-Odds

So far, we've learned that the equation for a logistic regression model looks like this:

$$\ln(p / (1-p)) = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Note that we've replaced y with the letter p because we are going to interpret it as a probability (eg., the probability of a student passing the exam). The whole left-hand side of this equation is called **log-odds** because it is the natural logarithm ($\ln$) of odds ($p/(1-p)$). The right-hand side of this equation looks exactly like regular linear regression!

In order to understand how this link function works, let's dig into the interpretation of **log-odds** a little more. The *odds* of an event occurring is:

$$\text{Odds} = \frac{p}{1-p} = \frac{P(\text{event occurring})}{P(\text{event not occurring})}$$

$$\text{Odds} = \frac{p}{1-p} = \frac{P(\text{event not occurring})}{P(\text{event occurring})}$$

For example, suppose that the probability a student passes an exam is 0.7. That means the probability of failing is $1 - 0.7 = 0.3$. Thus, the odds of passing are:

$$\text{Odds of passing} = 0.7 / 0.3 = 2.33$$

This means that students are 2.33 times more likely to pass than to fail.

Odds can only be a positive number. When we take the natural log of odds (the log odds), we transform the odds from a positive value to a number between negative and positive infinity — which is exactly what we need! The logit function (log odds) transforms a probability (which is a number between 0 and 1) into a continuous value that can be positive or negative.

1.

Suppose that there is a 40% probability of rain today ($p = 0.4$). Calculate the odds of rain and save it as `odds_of_rain`. Note that the odds are less than 1 because the probability of rain is less than 0.5.

Feel free to print `odds_of_rain` to see the results.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Use the odds that you calculated above to calculate the log odds of rain and save it as `log_odds_of_rain`. You can calculate the natural log of a value using the `numpy.log()` function. Note that the log odds are negative because the probability of rain was less than 0.5.

Feel free to print `log_odds_of_rain` to see the results.

Stuck? Get a hint

Checkpoint 3 Passed

3.

Suppose that there is a 90% probability that my train to work arrives on-time. Calculate the odds of my train being on-time and save it as `odds_on_time`. Note that the odds are greater than 1 because the probability is greater than 0.5.

Feel free to print `odds_on_time` to see the results.

Checkpoint 4 Passed

4.

Use the odds that you calculated above to calculate the log odds of an on-time train and save it as `log_odds_on_time`. Note that the log odds are positive because the probability of an on-time train was greater than 0.5.

Feel free to print `log_odds_on_time` to see the results.

```
import numpy as np
from exam import hours_studied, calculated_coefficients, intercept

# Calculate odds_of_rain
odds_of_rain = .4/.6
print('odds of rain: ', odds_of_rain)

# Calculate log_odds_of_rain
log_odds_of_rain = np.log(.4/.6)
print('log odds of rain: ', log_odds_of_rain)

# Calculate odds_on_time
odds_on_time = .9/.1
print('odds of an on-time train: ', odds_on_time)

# Calculate log_odds_on_time
log_odds_on_time = np.log(.9/.1)
print('log odds of on-time train: ', log_odds_on_time)
```

Sigmoid Function

Let's return to the logistic regression equation and demonstrate how this works by fitting a model in sklearn. The equation is:

$$\ln(p/(1-p)) = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Suppose that we want to fit a model that predicts whether a visitor to a website will make a purchase. We'll use the number of minutes they spent on the site as a predictor. The following code fits the model:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(min_on_site, purchase)
```

Copy to Clipboard

Next, just like linear regression, we can use the right-hand side of our regression equation to make predictions for each of our original datapoints as follows:

```
log_odds = model.intercept_ + model.coef_ * min_on_site
print(log_odds)
```

Copy to Clipboard

Output:

```
[[-3.28394203]
 [-1.46465328]
 [-0.02039445]
 [ 1.22317391]
 [ 2.18476234]]
```

Copy to Clipboard

Notice that these predictions range from negative to positive infinity: these are log odds. In other words, for the first datapoint, we have:

$$\ln(p/(1-p)) = -3.28394203$$

We can turn log odds into a probability as follows:

$$\ln(p / (1-p)) = -3.28$$

$$P/(1-p) = e^{-3.28}$$

$$p = e^{-3.28} * (1-p)$$

$$p = e^{-3.28} - e^{-3.28} * p$$

$$P + e^{-3.28} * p = e^{-3.28}$$

$$P * (1 + e^{-3.28}) = e^{-3.28}$$

$$P = e^{-3.28} / (1 + e^{-3.28})$$

$$P = 0.04$$

(

In Python, we can do this simultaneously for all of the datapoints using NumPy (loaded as `np`):

```
np.exp(log_odds) / (1 + np.exp(log_odds))
```

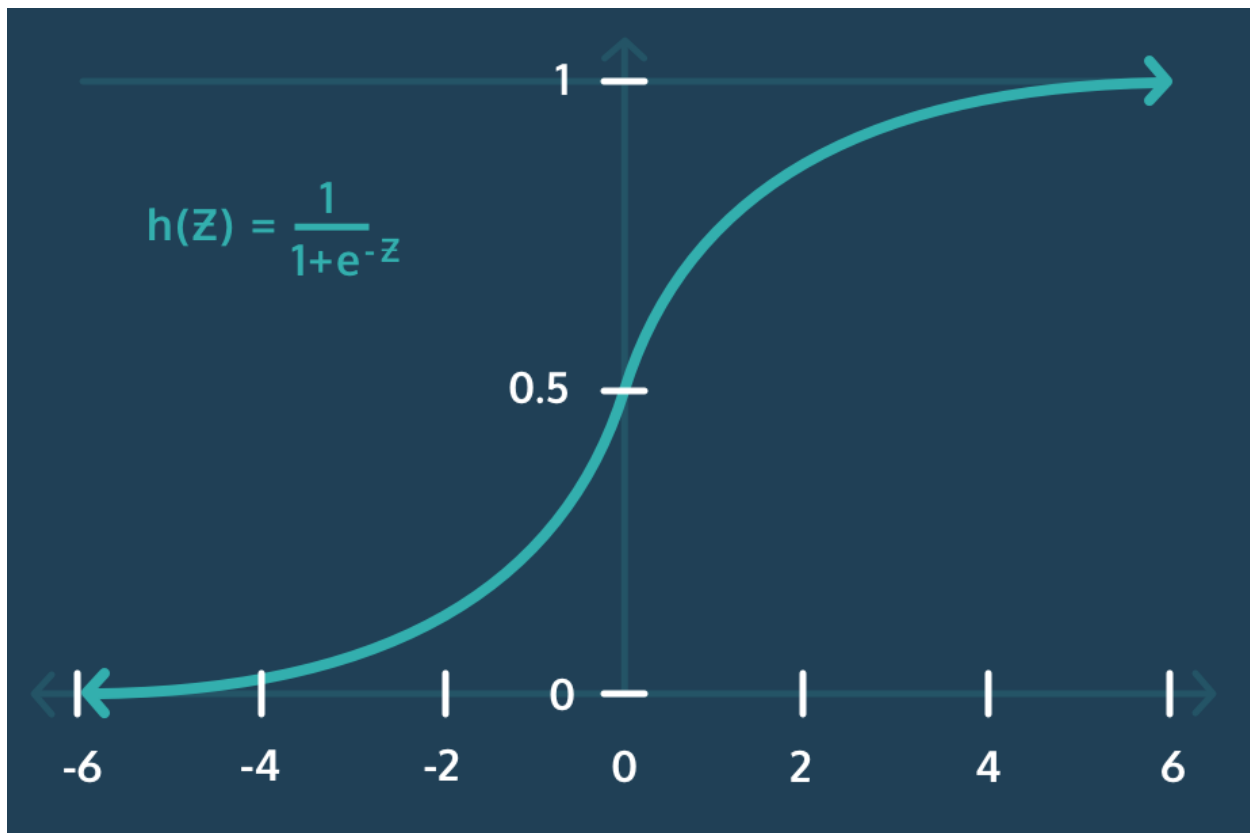
Copy to Clipboard

Output:

```
array([[0.0361262 ],
       [0.18775665],
       [0.49490156],
       [0.77262162],
       [0.89887279]])
```

Copy to Clipboard

The calculation that we just did required us to use something called the *sigmoid function*, which is the inverse of the logit function. The sigmoid function produces the S-shaped curve we saw previously:



Instructions

Checkpoint 1 Enabled

1.

In the workspace, we've fit a logistic regression on the Codecademy University data and saved the intercept and coefficient on `hours_studied` as `intercept` and `coef`, respectively.

For each student in the dataset, use the intercept and coefficient to calculate the log odds of passing the exam. Save the result as `log_odds`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, convert the predicted log odds for each student into a predicted probability of passing the exam. Save the predicted probabilities as `pred_probability_passing`.

```
# Import libraries and data
import pandas as pd
import numpy as np
codecademyU = pd.read_csv('codecademyU.csv')

# Fit the logistic regression model
hours_studied = codecademyU[['hours_studied']]
passed_exam = codecademyU[['passed_exam']]
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(hours_studied, passed_exam)

# Save intercept and coef
intercept = model.intercept_
coef = model.coef_

# Calculate log_odds here
log_odds = intercept + coef * hours_studied
print(log_odds)
```

```
# Calculate pred_probability_passing here
pred_probability_passing = np.exp(log_odds)/(1+ np.exp(log_odds))
print(pred_probability_passing)
```

Fitting a model in sklearn

Now that we've learned a little bit about how logistic regression works, let's fit a model using `sklearn`.

To do this, we'll begin by importing the `LogisticRegression` module and creating a `LogisticRegression` object:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
```

Copy to Clipboard

After creating the object, we need to fit our model on the data. We can accomplish this using the `.fit()`

Preview: Docs Loading link description

[method](#)

, which takes two parameters: a matrix of features and a matrix of class labels (the outcome we are trying to predict).

```
model.fit(features, labels)
```

Copy to Clipboard

Now that the model is trained, we can access a few useful attributes:

`model.coef_` is a vector of the coefficients of each feature

`model.intercept_` is the intercept

The coefficients can be interpreted as follows:

Large positive coefficient: a one unit increase in that feature is associated with a large **increase** in the log odds (and therefore probability) of a datapoint belonging to the positive class (the outcome group labeled as 1)

Large negative coefficient: a one unit increase in that feature is associated with a large **decrease** in the log odds/probability of belonging to the positive class.

Coefficient of 0: The feature is not associated with the outcome.

One important note is that `sklearn`'s logistic regression implementation requires the features to be standardized because regularization is implemented by default.

Instructions

Checkpoint 1 Passed

1.

We've pre-processed this data and split it into training and test sets as follows:

`X_train` is the feature matrix, containing standardized training data for hours studied and practice test score

`y_train` contains the outcome variable for the training data: whether or not each student passed the final exam (1 indicates passing, 0 indicates failing)

Create a `LogisticRegression` object named `cc_lr` and fit it to the provided training data.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Print out the coefficients and intercept for the model. Are the coefficients positive or negative and does this match your expectation? Which feature (hours studied or practice test score) is more strongly associated with students' probability of passing the final exam?

```
# Import pandas and the data
import pandas as pd
codecademyU = pd.read_csv('codecademyU_2.csv')

# Separate out X and y
X = codecademyU[['hours_studied', 'practice_test']]
y = codecademyU.passed_exam

# Transform X
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state = 27)

# Create and fit the logistic regression model here:
from sklearn.linear_model import LogisticRegression
cc_lr = LogisticRegression()
cc_lr.fit(X_train, y_train)

# Print the intercept and coefficients here:
print(cc_lr.intercept_)
print(cc_lr.coef_)
```

Predictions in sklearn

Using a trained model, we can predict whether new datapoints belong to the positive class (the group labeled as `1`) using the `.predict()`

Preview: Docs Loading link description

[method](#)

. The input is a matrix of features and the output is a vector of predicted labels, `1` or `0`.

```
print(model.predict(features))
# Sample output: [0 1 1 0 0]
```

Copy to Clipboard

If we are more interested in the predicted probability of group membership, we can use the `.predict_proba()` method. The input to `predict_proba()` is also a matrix of features and the output is an

Preview: Docs Loading link description

[array](#)

of probabilities, ranging from `0` to `1`:

```
print(model.predict_proba(features)[:,1])
# Sample output: [0.32 0.75 0.55 0.20 0.44]
```

Copy to Clipboard

By default, `.predict_proba()` returns the probability of class membership for both possible groups. In the example code above, we've only printed out the probability of belonging to the positive class. Notice that datapoints with predicted probabilities greater than 0.5 (the second and third datapoints in this example) were classified as `1`s by the

`.predict()` method. This is a process known as thresholding. As we can see here, sklearn sets the default classification threshold probability as 0.5.

Instructions

Checkpoint 1 Failed, try again

1.

In the workspace, we've fit the same logistic regression model on the `CodecademyU` training data. We've also created `x_test` and `y_test`, which contain the testing data.

Use the `.predict()` method to predict whether the students in the test dataset will pass the final exam, then print out the resulting vector of predictions.

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, use the `.predict_proba()` method to calculate the predicted probability that each student in the test dataset will pass the exam. Print out the results.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out `y_test` to see whether the students in the test dataset actually passed the exam. Did the model make accurate predictions? Looking at the probabilities, do the misclassification(s) make sense?

```
# Import pandas and the data
import pandas as pd
codecademyU = pd.read_csv('codecademyU_2.csv')

# Separate out X and y
X = codecademyU[['hours_studied', 'practice_test']]
y = codecademyU.passed_exam

# Transform X
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state = 27)

# Create and fit the logistic regression model here:
from sklearn.linear_model import LogisticRegression
cc_lr = LogisticRegression()
cc_lr.fit(X_train, y_train)

# Print out the predicted outcomes for the test data
print(cc_lr.predict(X_test))

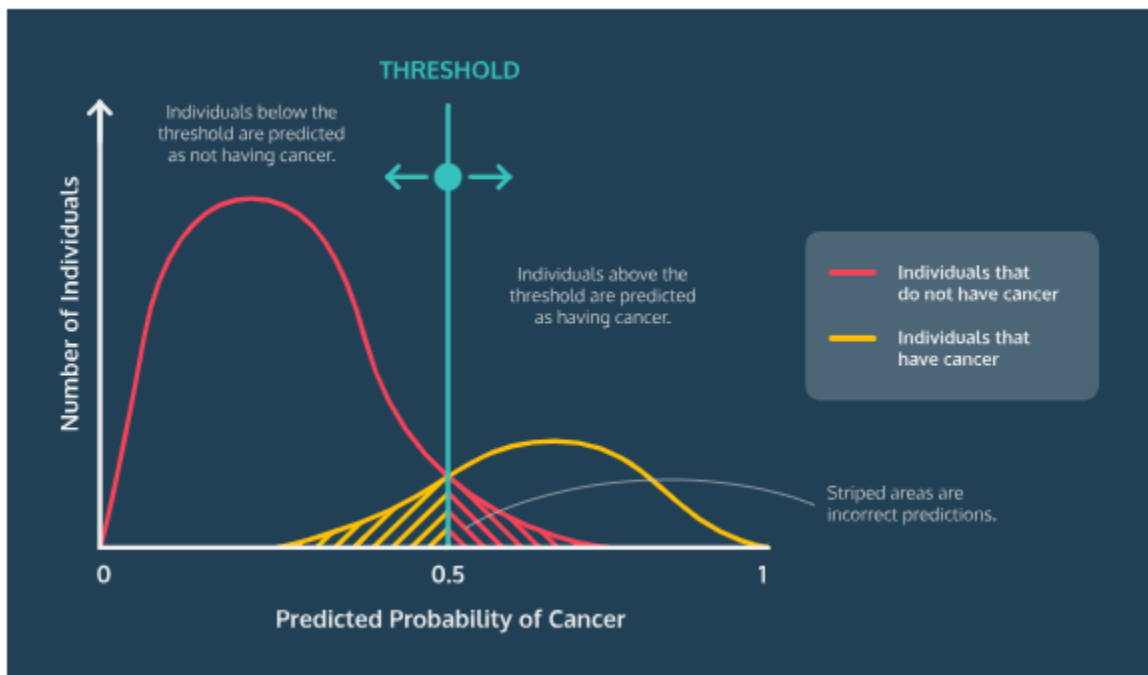
# Print out the predicted probabilities for the test data
print(cc_lr.predict_proba(X_test)[:,1])

# Print out the true outcomes for the test data
print(y_test)
```

Classification Thresholding

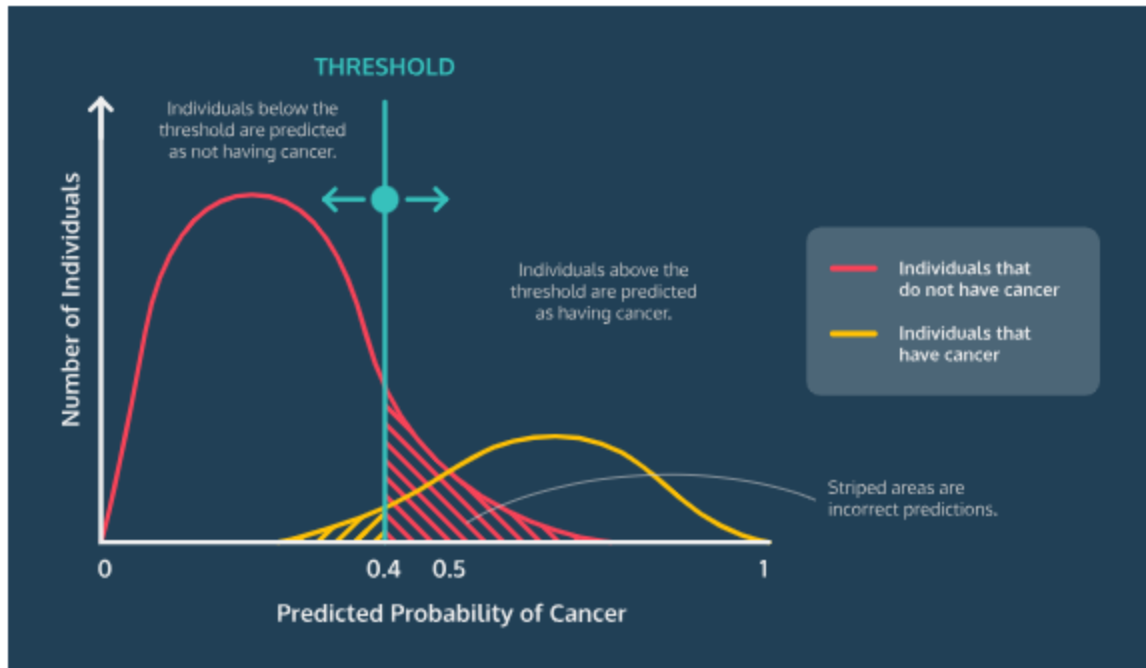
As we've seen, logistic regression is used to predict the probability of group membership. Once we have this probability, we need to make a decision about what class a datapoint belongs to. This is where the **classification threshold** comes in!

The default threshold for `sklearn` is 0.5. If the predicted probability of an observation belonging to the positive class is greater than or equal to the threshold, 0.5, the datapoint is assigned to the positive class.



We can choose to change the threshold of classification based on the use-case of our model. For example, if we are creating a logistic regression model that classifies whether or not an individual has cancer, we may want to be more sensitive to the positive cases. We wouldn't want to tell someone they don't have cancer when they actually do!

In order to ensure that most patients with cancer are identified, we can move the classification threshold down to 0.3 or 0.4, increasing the sensitivity of our model to predicting a positive cancer classification. While this might result in more overall misclassifications, we are now missing fewer of the cases we are trying to detect: actual cancer patients.



Instructions

Checkpoint 1 Enabled

1.

In the workspace, we've fit the same logistic regression model on the `CodecademyU` training data. We've also printed the predicted classes and true classes for the test data.

Take a look at the predicted probability of passing the exam for the mis-classified datapoint. The `.predict()` method uses a default threshold of 0.5 for predicting group membership. For this example, we could correctly classify all five datapoints in the test dataset using a different threshold.

Set the value of `alternative_threshold` to any value that would accomplish this.

Stuck? Get a hint

```
# Pick an alternative threshold here:
alternative_threshold = 0.6
# note: any value between 0.577 and 0.793 will work

# Import pandas and the data
import pandas as pd
codecademyU = pd.read_csv('codecademyU_2.csv')

# Separate out X and y
X = codecademyU[['hours_studied', 'practice_test']]
y = codecademyU.passed_exam

# Transform X
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state = 27)

# Create and fit the logistic regression model here:
from sklearn.linear_model import LogisticRegression
```

```
cc_lr = LogisticRegression()
cc_lr.fit(X_train,y_train)

# Print out the predicted outcomes for the test data
print(cc_lr.predict(X_test))

# Print out the predicted probabilities for the test data
print(cc_lr.predict_proba(X_test)[:,1])

# Print out the true outcomes for the test data
print(y_test)
```

Confusion matrix

When we fit a

Preview: Docs Loading link description

[machine learning](#)

model, we need some way to evaluate it. Often, we do this by splitting our data into training and test datasets. We use the training data to fit the model; then we use the test set to see how well the model performs with new data. As a first step, data scientists often look at a confusion matrix, which shows the number of true positives, false positives, true negatives, and false negatives.

For example, suppose that the true and predicted classes for a logistic regression model are:

```
y_true = [0, 0, 1, 1, 1, 0, 0, 1, 0, 1]
y_pred = [0, 1, 1, 0, 1, 0, 1, 1, 0, 1]
```

Copy to Clipboard

We can create a confusion matrix as follows:

```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_true, y_pred))
```

Copy to Clipboard

Output:

```
array([[3, 2],
       [1, 4]])
```

Copy to Clipboard

This output tells us that there are **3** true negatives, **1** false negative, **4** true positives, and **2** false positives. Ideally, we want the numbers on the main diagonal (in this case, **3** and **4**, which are the true negatives and true positives, respectively) to be as large as possible.

Instructions

Checkpoint 1 Enabled

1.

In the workspace, we've fit the same logistic regression model on the `codecademyU` training data and made predictions for the test data. `y_test` contains the true classes and `y_pred` contains the predicted classes. Create and print a confusion matrix for this data. How many incorrect classifications were there (false positives or false negatives)?

```
# Import pandas and the data
import pandas as pd
codecademyU = pd.read_csv('codecademyU_2.csv')

# Separate out X and y
X = codecademyU[['hours_studied', 'practice_test']]
y = codecademyU.passed_exam
```

```

# Transform X
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state =
27)

# Create and fit the logistic regression model here:
from sklearn.linear_model import LogisticRegression
cc_lr = LogisticRegression()
cc_lr.fit(X_train, y_train)

# Save and print the predicted outcomes
y_pred = cc_lr.predict(X_test)
print('predicted classes: ', y_pred)

# Print out the true outcomes for the test data
print('true classes: ', y_test)

# Print out the confusion matrix here
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, y_pred))

```

Accuracy, Recall, Precision, F1 Score

Once we have a confusion matrix, there are a few different

Preview: Docs Loading link description

[statistics](#)

we can use to summarize the four values in the matrix. These include accuracy, precision, recall, and F1 score. We won't go into much detail about these metrics here, but a quick summary is shown below (T = true, F = false, P = positive, N = negative). For all of these metrics, a value closer to 1 is better and closer to 0 is worse.

- **Accuracy** = $(TP + TN) / (TP + FP + TN + FN)$
- **Precision** = $TP / (TP + FP)$
- **Recall** = $TP / (TP + FN)$
- **F1 score**: weighted average of precision and recall

In `sklearn`, we can calculate these metrics as follows:

```

# accuracy:
from sklearn.metrics import accuracy_score
print(accuracy_score(y_true, y_pred))
# output: 0.7

# precision:
from sklearn.metrics import precision_score
print(precision_score(y_true, y_pred))
# output: 0.67

```

```
# recall:
from sklearn.metrics import recall_score
print(recall_score(y_true, y_pred))
# output: 0.8

# F1 score
from sklearn.metrics import f1_score
print(f1_score(y_true, y_pred))
# output: 0.73
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

In the workspace, we've fit the same logistic regression model on the `codecademyU` training data and made predictions for the test data. `y_pred` contains the predicted classes and `y_test` contains the true classes. Also, note that we've changed the train-test split (by using a different value for the `random_state` parameter, making the confusion matrix different from the one you saw in the previous exercise.

Calculate the accuracy for the model and print it out.

Checkpoint 2 Passed

2.

Calculate the F1 score for the model and print it out.

```
# Import pandas and the data
import pandas as pd
codecademyU = pd.read_csv('codecademyU_2.csv')

# Separate out X and y
X = codecademyU[['hours_studied', 'practice_test']]
y = codecademyU.passed_exam

# Transform X
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state = 51)

# Create and fit the logistic regression model here:
from sklearn.linear_model import LogisticRegression
cc_lr = LogisticRegression()
cc_lr.fit(X_train, y_train)

# Save and print the predicted outcomes
y_pred = cc_lr.predict(X_test)
print('predicted classes: ', y_pred)

# Print out the true outcomes for the test data
print('true classes: ', y_test)

# Print out the confusion matrix
from sklearn.metrics import confusion_matrix
print('confusion matrix: ')
print(confusion_matrix(y_test, y_pred))

# Print accuracy here:
from sklearn.metrics import accuracy_score

# Print F1 score here:
from sklearn.metrics import f1_score
```

Review

Congratulations! You just learned how a logistic regression model works and how to fit one to a dataset. Here are some of the things you learned:

Logistic regression is used to perform
Preview: Docs Loading link description

[binary](#)
classification.

Logistic regression is an extension of linear regression where we use a logit link function to fit a sigmoid curve to the data, rather than a line.

We can use the coefficients from a logistic regression model to estimate the log odds that a datapoint belongs to the positive class. We can then transform the log odds into a probability.

The coefficients of a logistic regression model can be used to estimate relative feature importance.

A classification threshold is used to determine the probabilistic cutoff for where a data sample is classified as belonging to a positive or negative class. The default cutoff in sklearn is `0.5`.

We can evaluate a logistic regression model using a confusion matrix or summary [statistics](#) such as accuracy, precision, recall, and F1 score.

Assumptions of Logistic Regression I

We're now ready to delve deeper into Logistic Regression! In this lesson, we will cover the different assumptions that go into logistic regression, model hyperparameters, how to evaluate a classifier, ROC curves, and what to do when there's a class imbalance in the classification problem we're working with.

For this lesson, we will be using the [Wisconsin Breast Cancer Data Set](#) (Diagnostic) to predict whether a tumor is benign (0) or malignant (1) based on characteristics of the cells, such as radius, texture, smoothness, etc. Like a lot of real-world data sets, the distribution of outcomes is uneven (benign diagnoses are more common than malignant) and there is a bias in terms of the importance of the outcomes (classifying all malignant cases correctly is of the utmost importance).

We're going to begin with the primary assumptions about the data that need to be checked before implementing a logistic regression model.

1. The target variable is

Preview: Docs Binary is a number system of 1s and 0s which serves as a textual representation method for giving instructions to a computers CPU. In computing and telecommunications, binary codes are used for various methods of encoding data such as turning the character strings of source code, into bit strings or instruction sets for the CPU to execute.

[binary](#)

One of the most basic assumptions of logistic regression is that the outcome variable needs to be binary, which means there are two possible outcomes. Multinomial logistic regression is an exception to this assumption and is beyond the

Preview: Docs Loading link description

[scope](#)

of this lesson.

2. Independent observations

While often overlooked, checking for independent observations in a data set is important for logistic regression. This can be violated if, in this case, patients are biopsied multiple times (repeated sampling of the same individual).

3. Large enough sample size

Since logistic regression is fit using maximum likelihood estimation instead of least squares minimization, there must be a large enough sample to get convergence. When a model fails to converge, this causes the estimates to be extremely inaccurate. Now, what does a “large enough” sample mean? Often a rule of thumb is that there should be at least 10 samples per feature for the smallest class in the outcome variable.

For example, if there were 100 samples and the outcome variable `diagnosis` had 60 benign tumors and 40 malignant tumors, then the max number of features allowed would be 4. To get 4 we took the smallest of the classes in the outcome variable, 40, and divided it by 10.

4. No influential outliers

Logistic regression is sensitive to outliers, so we must remove any extremely influential outliers for model building. Outliers are a broad topic with many different definitions – z-scores, scaler of the interquartile range, Cook’s distance/influence/leverage, etc – so there are many ways to identify them. But here, we will use visual tools to rule out obvious outliers.

Instructions

Checkpoint 1 Enabled

1.

We need to verify that there are two classes in `diagnosis`, making it binary.

Using `value_counts()` print the distinct `diagnosis` values and their frequency in the dataset.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now we need to make sure that there are no repeat samples. We can do this by making sure the number of unique `ids` is equal to the number of `ids`.

Use `.nunique()` to find the number of unique `id` values and use `.count()` to find the number of `id` values. Set these two values equal to each other and print the result.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

If classes are very imbalanced, it is important the smallest class still meets the rule of thumb. Based on this how many features should we have at maximum?

Using the outcome variable `diagnosis`, divide the smallest class size by 10. Save this variable as `max_features` and print the value.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We’re going to view a boxplot of the mean features (`predictor_var` has the list of these features) so that we can find and exclude any obvious outliers. Since the features are not normally distributed (since all the values must be greater than zero and are right-skewed), it is more informative to look at the boxplot of the log-transformed z-scores.

Uncomment the relevant lines here and press Run to see the output.

Which features have extreme outliers?

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

As mentioned earlier, Logistic Regressions are sensitive to extreme outliers causing them to distort your findings. The `fractal_dimension_mean` feature has many extreme outliers above the overall distribution of its data.

Uncomment the code listed under #5 and run the code to remove any samples that are above the 99 percentile.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Create another boxplot similar to step 4 (you can copy the code), but use the `df_filtered` data created in step 5.

How does this boxplot differ from the first one?

```
import pandas as pd
import numpy as np
```



```

import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler #not currently used
from scipy.stats import zscore #not needed but built in
#https://www.kaggle.com/uciml/breast-cancer-wisconsin-data
df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})

print(df.head())

#imports/load data
predictor_var = ['radius_mean', 'texture_mean', 'perimeter_mean',
                 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
                 'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean']
df.head()

#1.Print distinct diagnosis values and frequency in dataset
print(df.diagnosis.value_counts())

#2. Test if the number of unique IDs is equal to sample size, i.e. no repeated patients
unique_ids = df.id.nunique()==df.id.count()
print(unique_ids)

#3. At a maximum, there should be no more than the smallest class size divided by 10 number
of features.
max_features = min(df.diagnosis.value_counts()/10)
print(max_features)

#4. Uncomment the code to see which features have extreme outliers:
sns.boxplot(data=np.log(df[predictor_var]+.01).apply(zscore))
plt.xticks(rotation=45);
plt.show()
plt.close()

#5. Uncomment the code to remove the samples with extreme fractal_dimensions_mean values:
q_hi = df["fractal_dimension_mean"].quantile(0.99)
df_filtered = df[(df["fractal_dimension_mean"] < q_hi)]

#6. Run the boxplot again but with the filtered dataframe:
sns.boxplot(data=np.log(df_filtered[predictor_var]+.01).apply(zscore))
plt.xticks(rotation=45);
plt.show()
plt.close()

```

Assumptions of Logistic Regression II

1. Features linearly related to log odds

Similar to linear regression, the underlying assumption of logistic regression is that the features are linearly related to the logit of the outcome. To test this visually, we can use Seaborn's regplot, with the

Preview: Docs Loading link description

[parameter](#)

`logistic= True` and the x value as our feature of interest. If this condition is met, the fit model will resemble a sigmoidal curve (as is the case when `x=radius_mean`).

We've added code to create another plot using the feature `fractal_dimension_mean`. Press **Run** in the workspace. How do the curves compare?

2. Multicollinearity

Like in linear regression, one of the assumptions is that there is no multicollinearity in the data. Meaning the features should not be highly correlated. Multicollinearity can cause the coefficients and p-values to be inaccurate. With a correlation plot, we can see which features are highly correlated and then we can drop one of the features.

We're going to look at the "mean" features which are highly correlated with each other using a heatmap correlation plot.

Instructions

Checkpoint 1 Enabled

1.

Uncomment the relevant lines of code and press Run to see the heatmap.

There are two features that are highly positively correlated with `radius_mean`. Can you spot them?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Not including `radius_mean`, `perimeter_mean`, or `area_mean`, there is another pair of features that are highly correlated. Create an array named `correlated_pair` containing these two features.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score

#https://www.kaggle.com/uciml/breast-cancer-wisconsin-data
df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})
predictor_var = ['radius_mean', 'texture_mean', 'perimeter_mean','area_mean',
'smoothness_mean', 'compactness_mean','concavity_mean', 'symmetry_mean',
'fractal_dimension_mean']
x = df[predictor_var]

#Compare the curves
sns.regplot(x= 'radius_mean', y= 'diagnosis', data= df, logistic= True,)
plt.show()
plt.close()
sns.regplot(x= 'fractal_dimension_mean', y= 'diagnosis', data= df, logistic= True)
plt.show()
plt.close()

#1 Uncomment the heatmap and identify the two features that are highly correlated with
radius_mean.
plt.figure(figsize = (10,7))
```

```
sns.heatmap(x.corr(), annot=True)
plt.show()

#2. Identify the other highly correlated pair. Define an array called correlated_pair
containing the two features you've identified.
correlated_pair = ['compactness_mean', 'concavity_mean']
```

`scikit-learn` Implementation

Model Training and Hyperparameters

Now that we have checked the assumptions of Logistic Regression, we can train and predict a model using `scikit-learn`. We will first set the *hyperparameters* of our model. Hyperparameters are set before the model implementation step and tuned later to improve model performance. Conversely, parameters are the result of model implementation, such as the intercept and coefficients.

Note: Within `scikit-learn` these hyperparameters are often referred to as “parameters” which might cause some confusion. It is worth noting that the meaning within `scikit-learn` documentation refers to these being “parameters” of the function and not of the model itself.

Evaluation Metrics

Despite the name, logistic regression is being used as a classifier here, so any evaluation metrics for classification tasks will apply. The simplest metric is accuracy – how many correct predictions did we make out of the total? However, when classes are imbalanced, this can be a misleading metric for model performance. Similarly, if we care more about accurately predicting a certain class, other metrics may be more appropriate to use, such as precision, recall, or F1-score may be better to evaluate performance. All of these metrics are available in `scikit-learn`. Check out the [Evaluation Metrics](#) lesson if you’d like to brush up on the same.

```
Accuracy = (TP + TN) / Total

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 score = 2 * ((Precision * Recall) / (Precision + Recall))
```

Copy to Clipboard

Which metrics matter most?

For our breast cancer dataset, predicting ALL malignant cases as malignant is of the utmost importance – and even if there are some false positives (benign cases that are marked as malignant), these likely will be discovered by follow-up tests. Whereas missing a malignant case (classifying it as benign) could have deadly consequences. Thus, we want to minimize false negatives. This in turn will maximize the recall ratio (also known as the sensitivity or true positive rate).

Instructions

Checkpoint 1 Enabled

1.

Model Hyperparameters:

Using the mean predictor variables defined in the code editor, define a logistic regression classifier named `log_reg` using `scikit-learn` with no regularization (i.e. no penalty) and an intercept term. This will mean setting

the input argument `penalty` to `'none'` (the default penalty is `'l2'`).

the input argument `fit_intercept` to `True`

Print the hyperparameters using the `.get_params()` method.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Model parameters:

Fit the model to the training data and calculate the resulting model coefficients and intercept. Store them as variables `coefficients` and `intercept` respectively and print their values.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Evaluation metrics: Accuracy, Precision, Recall, F1

Now we will explore some of the most common evaluation metrics for classification.

First, use `.predict()` to predict the outcome variable and save it as `y_pred`. Then complete the following:

Calculate accuracy and store it as a variable named `accuracy`.

Calculate precision and store it as `precision`.

Calculate recall and store it as `recall`.

Calculate the f1-score and store it as `f1`.

Remove the comments from the code block to print the evaluation metrics you just stored. Which metric gives the highest and lowest values?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Evaluation Metrics: Confusion Matrix

Take a look at the confusion matrix by uncommenting the code lines listed in #4.

How many tumors were predicted correctly? How many people's tumors were predicted as benign but were actually malignant? And vice versa?

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score

#https://www.kaggle.com/uciml/breast-cancer-wisconsin-data
df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})
predictor_var = ['radius_mean', 'texture_mean',
                 'compactness_mean',
                 'symmetry_mean',]
outcome_var='diagnosis'
x_train, x_test, y_train, y_test = train_test_split(df[predictor_var], df[outcome_var],
random_state=0, test_size=0.3)

#1. Fit a Logistic Regression model with the specified hyperparameters
log_reg = LogisticRegression(penalty='none', fit_intercept=True)
print(log_reg.get_params())

#2. Fit the model to training data and obtain coefficient and intercept
log_reg.fit(x_train, y_train)
coefficients = log_reg.coef_
intercept = log_reg.intercept_

print('coefficients: ', coefficients)
print('intercept: ', intercept)

#3. Calculate the accuracy, precision, recall, and f1-score on the testing data
y_pred = log_reg.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```

print(f'Test set accuracy:\t{accuracy}')
print(f'Test set precision:\t{precision}')
print(f'Test set recall:\t{recall}')
print(f'Test set f1-score:\t{f1}')

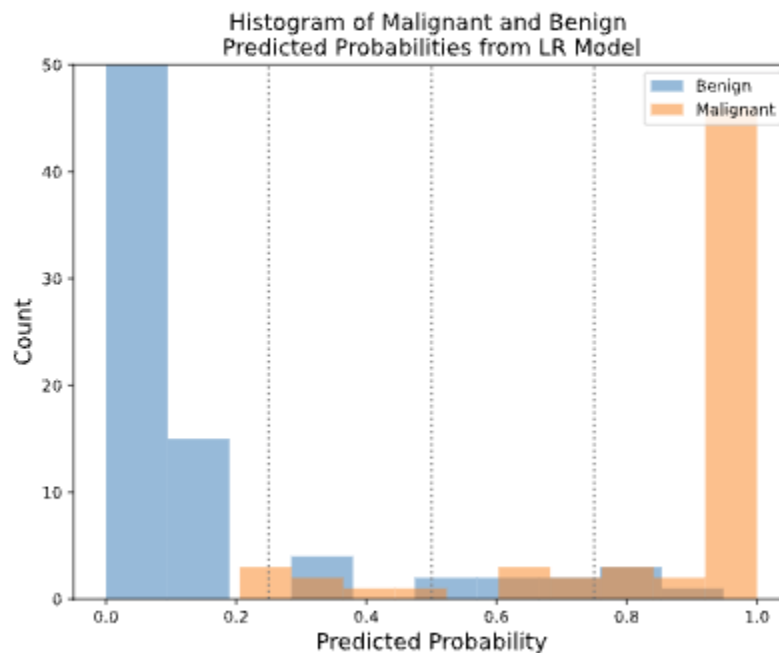
#4. Remove the comments from the following code block to print the confusion matrix
test_conf_matrix = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=['actual no', 'actual yes'],
    columns=['predicted no', 'predicted yes']
)

print(test_conf_matrix)

```

Prediction Thresholds

Logistic regression not only predicts the class of a sample, but also the probability of a sample belonging to each class. It provides us with a measure of certainty associated with each prediction. In the default implementation in `scikit-learn`, a probability greater than 50% means that the predicted outcome will belong to the positive class. This is referred to as a prediction threshold. If two samples have predicted probabilities of 51% and 99%, both will be considered positive with the default threshold. However, if the threshold is increased to 60%, a predicted probability of 51% will be assigned the negative class.



Consider the histogram of the predicted probabilities for the logistic regression classifier shown above. The benign (or negative class) is depicted in blue, and the malignant (or positive class) in orange for the breast cancer data set. The benign cases are heavily clustered around zero, which is good as they will be correctly classified as benign, whereas malignant cases are heavily clustered around one. The vertical lines depict hypothetical threshold values at

25%, 50%, and 75%. For the highest threshold, almost all the samples above 75% belong to the malignant class, but there will be some benign cases that are misdiagnosed as malignant (false positives). In addition, there are a number of malignant cases that are missed (false negatives). If instead the lowest threshold value is used, almost all the malignant cases are identified, but there are more false positives.

Therefore, the value of the threshold is an additional lever that can be used to tune a model's predictions. A higher value is generally associated with fewer false positives and more false negatives. Whereas a lower value is associated with fewer false negatives and more false positives.

Instructions

Checkpoint 1 Enabled

1.

In the code editor, we've trained a logistic regression model, fit it to training data and calculated the predicted class. The `predict_proba` method gets us the predicted probabilities, which are then saved as `y_pred_prob`:

```
y_pred_prob = log_reg.predict_proba(x_test)
```

Copy to Clipboard

We've created a new array `y_pred_class` that is 1 when `y_pred_prob > 0.5` and 0 otherwise. Let's check if this array is the same as `y_pred`. Luckily, there is a `numpy` method for this!

Use `np.array_equal()` and place `y_pred_class` and `y_pred` as the two arguments. Then set it equal to the variable `diff` and print it. If they are equal, it will return `True`. [Click here](#) to read the documentation.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We're now going to take a look at the confusion matrix for the default threshold value (probability of 0.5 or 50%). Recall that there are four possible categories that a prediction can fall under: True Positive (**TP**), True Negative (**TN**), False Positive (**FP**), False Negative (**FN**). In a confusion matrix the predicted classes are represented as columns and the actual classes are represented as rows.

	Predicted -	Predicted +
Actual -	TN	FP
Actual +	FN	TP

Uncomment the relevant lines in the code editor and press Run to view the confusion matrix.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We can now calculate the confusion matrices for alternative thresholds.

Calculate the confusion matrices for thresholds of 25% and 75% and store them as `cm_25` and `cm_75` respectively. Print them to examine how they're different from the confusion matrix for the default threshold value.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We're going to put what we've learned in this exercise together by figuring out how to set the correct threshold for a given problem. In our classification problem, we would like to minimize the number of false negatives, i.e., we'd like to correctly catch as many malignant cases as possible. Specifically, we would like to set a threshold such that no more than 2 malignancies are misidentified, i.e., no more than 2 false negatives are allowed.

We've created an array `thresh` that contains a range of threshold values between 0 and 1. The array `false_negatives` contains the corresponding number of false negatives that would occur for each threshold value. What is the lowest value of `thresh` corresponding to the occurrence of a false negative value of 2? Store your answer as a variable `thresh_choice` and print it.

```

import pandas as pd
import numpy as np

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})
predictor_var = ['radius_mean', 'texture_mean',
                 'compactness_mean',
                 'symmetry_mean',]
outcome_var='diagnosis'
x_train, x_test, y_train, y_test = train_test_split(df[predictor_var], df[outcome_var],
random_state=0, test_size=0.3)

log_reg = LogisticRegression(penalty='none',
fit_intercept=True,tol=0.0000001,solver='newton-cg')
log_reg.fit(x_train, y_train)
y_pred = log_reg.predict(x_test)
y_pred_prob = log_reg.predict_proba(x_test)

#Using the predicted probabilities to get the predicted class
y_pred_class = (y_pred_prob[:,1]>0.5)*1.0
#1 Check if it's the same as y_pred
diff = np.array_equal(y_pred_class,y_pred)
print(diff)

## 2. Print the confusion matrix
print("Confusion Matrix: Threshold 50%")
cm_50 = confusion_matrix( y_test, y_pred_class)
print(cm_50)

##3 Confusion matrices for thresholds of 0.25 and 0.75
print("Confusion Matrix: Threshold 25%")
cm_25 = confusion_matrix(y_test, (y_pred_prob[:,1]>0.25)*1.0)
print(cm_25)

print("Confusion Matrix: Threshold 75%")
cm_75 = confusion_matrix(y_test, (y_pred_prob[:,1]>0.75)*1.0)
print(cm_75)

#4. Choosing the right threshold for a question

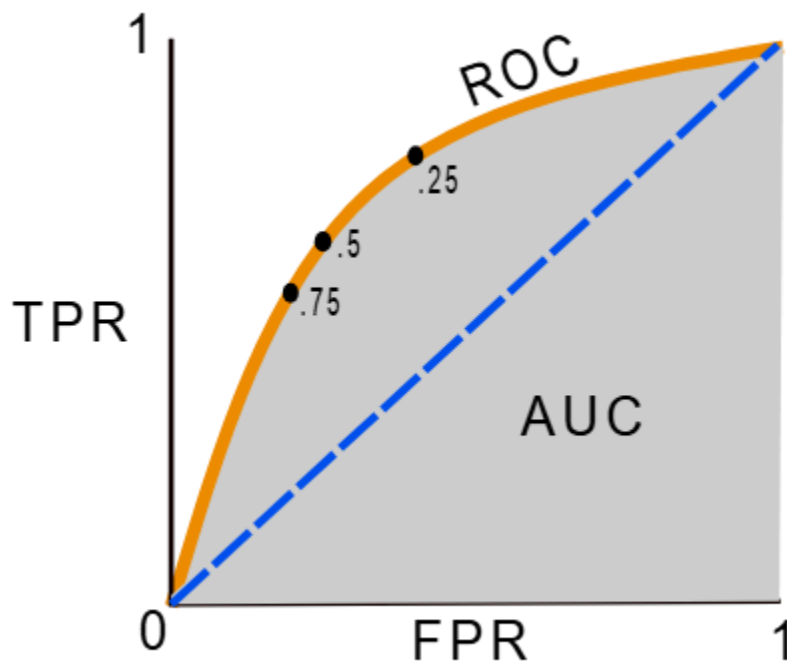
#Array of thresholds
thresh = np.linspace(0,1,100)
false_negatives = []

for t in thresh:
    cm = confusion_matrix(y_test, (y_pred_prob[:,1]>t)*1.0)
    false_negatives.append(cm[1][0])
thresh_choice = thresh[np.argmax(np.array(false_negatives)>=2)]
print(thresh_choice)

```

ROC Curve and AUC

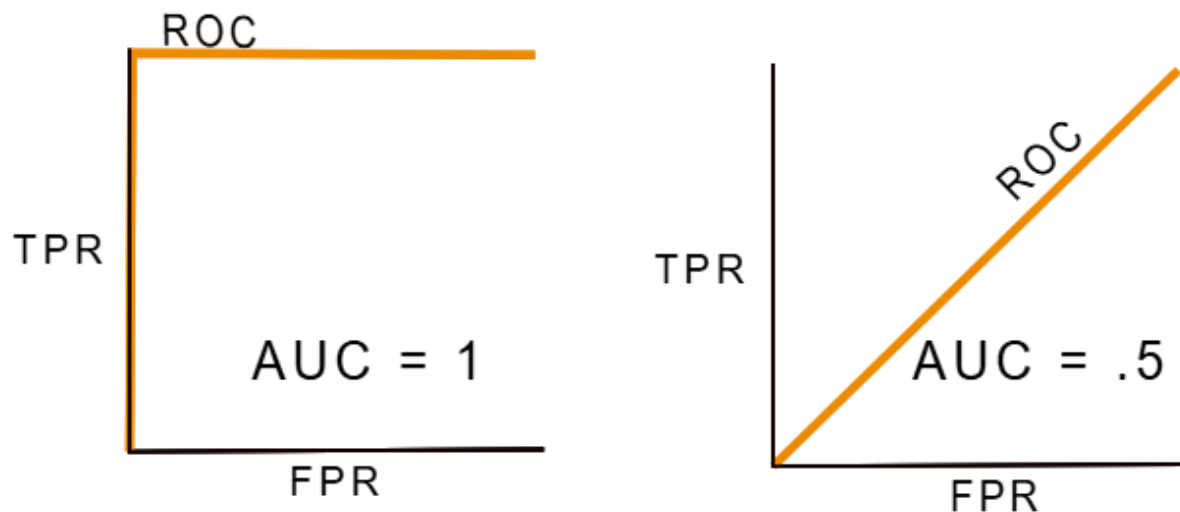
We have examined how changing the threshold can affect the logistic regression predictions. There is a continuum of predictions available in a single model by varying the threshold incrementally from zero to one. For each of these thresholds, the True Positive Rate (TPR) and the False Positive Rate (FPR) can be calculated and then plot. The resulting curve these points form is known as the Receiver Operating Characteristic (ROC) curve.



In the ROC curve plotted above, the True Positive Rate ($TPR = TP / TP + FN$) is on the y-axis and the False Positive Rate ($FPR = FP / TN + FP$) is on the x-axis. The ROC curve is the orange line and the dashed blue line is the Dummy Classification line, which is the equivalent of random guessing.

Notice there are three data points on the ROC curve, each labeled with their threshold values. The classification threshold of .5 will give us a TPR of about .65 with an FPR of about .28. For our specific data, we want a higher TPR so that we catch every malignant tumor. We might select a lower threshold of .25 so that our TPR is about .8, even though this may give us an FPR of about .4. The ROC curve can help us decide on a threshold that best fits our specific classification problem.

While the ROC curve measures the probabilities, the AUC (Area Under the Curve) gives us a single metric for separability. The AUC tells us how well our model can distinguish between the two classes. An AUC score close to 1 is a near-perfect classifier, whereas a value of 0.5 is equivalent to random guessing. To visualize different AUC scores, look at the ROC curve plots below:



Instructions

Checkpoint 1 Enabled

1.

In the code editor, we've plotted the ROC Curve for the dataset and model we've been working with throughout this lesson. We used `scikit-learn`'s `roc_curve()` function, where the inputs are the arrays `y_true` and `y_score`. The threshold values are not discernible from the curve alone, so we've labeled five thresholds for visibility. Additionally, we also plotted the ROC curve of a "dummy classifier", that predicts that all the data points belong to the more frequent class.

Hit Run to see the ROC curve plot.

Which threshold would be best for our classification problem?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the ROC AUC score using `scikit-learn`'s `roc_auc_score()` function and save the results as `roc_auc`. The two arguments in this function will be the true binary label and the predicted probability of the positive class. Print `roc_auc` to see what AUC score our model has.

Does our model have good separability?

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score

#https://www.kaggle.com/uciml/breast-cancer-wisconsin-data
df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})
predictor_var = ['radius_mean', 'texture_mean',
                  'compactness_mean',
                  'symmetry_mean',]
outcome_var='diagnosis'
x_train, x_test, y_train, y_test = train_test_split(df[predictor_var], df[outcome_var],
random_state=0, test_size=0.3)

log_reg = LogisticRegression(penalty='none',
fit_intercept=True,tol=0.0000001,solver='newton-cg')
log_reg.fit(x_train, y_train)
```

```

y_pred = log_reg.predict(x_test)
y_pred_prob = log_reg.predict_proba(x_test)
#####
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob[:,1])
plt.figure()
plt.plot(fpr, tpr, color='darkorange',
         label='ROC curve')
idx = list(range(len(thresholds)))[1::4]
for i in idx:
    plt.text(fpr[i], tpr[i], thresholds[i].round(2))

clf = DummyClassifier(strategy='most_frequent', random_state=0)
clf.fit(x_train, y_train)
roc_auc = roc_auc_score(y_test, clf.predict_proba(x_test)[:,1] )
fpr, tpr, thresholds = roc_curve(y_test, clf.predict_proba(x_test)[:,1])
plt.plot(fpr, tpr, color='navy', linestyle='--', label='Dummy Classifier (most frequent) \n
(area = %0.2f)' % roc_auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.grid()
plt.legend(loc="lower right")
plt.show()

#1. Run the code

#2. Find ROC_AUC score and print it

```

Class Imbalance

Class imbalance is when your

Preview: Docs Loading link description

[binary](#)

classes for the outcome variable are not evenly split. Technically, anything different from a 50/50 distribution would be imbalanced and need appropriate care. In the case of rare events, sometimes the positive class can be less than 1% of the total. If your classes are significantly imbalanced, this could create a bias towards the majority class since the model learns that it can have a higher accuracy if it predicts the majority class more often.

Positivity Rate

We can use the positivity rate to tell us how balanced our classes are. The positivity rate is the rate of occurrence for the positive class. With our breast cancer data, the formula is $\text{Positivity Rate} = \frac{\text{Total Malignant Cases}}{\text{Total Cases}}$. If our positivity rate is close to .5, then our classes are balanced.

Stratification

If your classes are imbalanced (more likely to happen with smaller datasets) then this difference can become even greater after you split your data into a training and testing dataset. One way to mitigate this is to randomly split using stratification on the class labels. Stratification is when the data is sorted into subgroups to ensure a nearly equal class distribution in your train and test sets. After using stratification, the training and testing datasets should have a

very similar positivity rate (but stratification does not necessarily cause the positivity rate of the dataset to reach closer to .5).

Undersampling/Oversampling

To bring the positivity rate of the dataset closer to .5, we can undersample the majority class or oversample the minority class. For oversampling, repeated samples (with replacement) are taken from the minority class until the size is equal to that of the majority class. This causes the same data to be used multiple times, giving a higher weight to these samples. Alternatively, undersampling leaves out some of the majority class data to have the same number of samples as the minority class, leaving fewer data to build the model. We will not be practicing undersampling/oversampling in this exercise even though it can be a useful way to balance the classes.

Balance the Class Weight

When training a model, it is the default for every sample to be weighted equally. However, in the case of class imbalance, this can result in poor predictive power for the smaller of the two classes. A way to counteract this in logistic regression is to use the

Preview: Docs Loading link description

parameter

`class_weight='balanced'`. This applies a weight inversely proportional to the class frequency, therefore supplying higher weight to misclassified instances in the smaller class. While overall accuracy may not increase, this can increase the accuracy for the smaller class (e.g. increase the number of malignant cases correctly diagnosed). Keep in mind that we want the recall score (also known as the True Positive Rate) to be as high as we can get it for our breast cancer data.

```
Recall = TP / TP + FN
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Stratified Sampling

In the code editor, we've written some code to execute a typical train-test split to calculate the positivity rate for the training and test data:

```
x_train, x_test, y_train, y_test = train_test_split(df[predictor_var],
df[outcome_var], random_state=6, test_size=0.3)
```

Copy to Clipboard

Create a stratified train-test sample with the following variable names `x_train_str`, `x_test_str`, `y_train_str`, `y_test_str`. Use `train_test_split()` again and copy the same parameters as before but also add the `stratify` parameter that is set to the class labels `df[outcome_var]`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Calculate the train and test positivity rates for the stratified sample and store them as

`str_train_positivity_rate` and `str_test_positivity_rate` respectively. Print them to see how they differ from the positivity rates with the previous (unstratified) sample.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Fit the logistic regression model to the stratified training data and get `y_pred` on the stratified test data.

Calculate the recall and accuracy scores on the stratified test data and store them as variables `recall_str` and `accuracy_str` respectively.

Did stratification increase the recall and accuracy scores?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Balancing Class Weights

To balance class weights, define a Logistic Regression classifier `log_reg_bal` that has the same parameters as the `log_reg` model that was given in the starting code:

```
log_reg = LogisticRegression(penalty='none', max_iter=1000,
fit_intercept=True, tol=0.000001)
```

Copy to Clipboard

And add a parameter `class_weight` set to `'balanced'`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Fit the balanced class Logistic Regression classifier to the training data and predict on the test data to obtain recall and accuracy scores. Store them as `recall_bal` and `accuracy_bal` respectively.

In the output there should be a recall and an accuracy score for each of the three models; the default `log_reg` model, the `log_reg` model with the stratified samples, and the balanced model `log_reg_bal`. Which model has the highest scores?

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score

#https://www.kaggle.com/uciml/breast-cancer-wisconsin-data
df = pd.read_csv('breast_cancer_data.csv')
#encode malignant as 1, benign as 0
df['diagnosis'] = df['diagnosis'].replace({'M':1,'B':0})
predictor_var = ['radius_mean', 'texture_mean',
                  'compactness_mean',
                  'symmetry_mean']
outcome_var='diagnosis'

x_train, x_test, y_train, y_test = train_test_split(df[predictor_var], df[outcome_var],
random_state=6, test_size=0.3)

print('Train positivity rate: ')
print(sum(y_train)/y_train.shape[0])
print('Test positivity rate: ')
print(sum(y_test)/y_test.shape[0])

log_reg = LogisticRegression(penalty='none', max_iter=1000, fit_intercept=True,
tol=0.000001)
log_reg.fit(x_train, y_train)
y_pred = log_reg.predict(x_test)

recall = recall_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print('Recall and Accuracy scores')
print(recall, accuracy)

## 1. Stratified Sampling
x_train_str, x_test_str, y_train_str, y_test_str = train_test_split(df[predictor_var],
df[outcome_var], random_state=6, test_size=0.3, stratify=df[outcome_var])

### 2. Stratified positivity rates
print('Stratified train positivity rate: ')
str_train_positivity_rate = sum(y_train_str)/y_train_str.shape[0]
print(str_train_positivity_rate)

print('Stratified test positivity rate: ')
str_test_positivity_rate = sum(y_test_str)/y_test_str.shape[0]
print(str_test_positivity_rate)

### 3. Model predictions after Stratified sampling
log_reg.fit(x_train_str, y_train_str)
y_pred = log_reg.predict(x_test_str)
```

```

recall_str = recall_score(y_test_str, y_pred)
accuracy_str = accuracy_score(y_test_str, y_pred)

print('Stratified Sampling: Recall and Accuracy scores')
print(recall_str, accuracy_str)

### 4. Balanced Class Weights
log_reg_bal = LogisticRegression(penalty='none', max_iter=1000, fit_intercept=True,
tol=0.000001, class_weight='balanced')

### 5. Model Predictions after balancing Class Weights
log_reg_bal.fit(x_train, y_train)
y_pred = log_reg_bal.predict(x_test)

recall_bal = recall_score(y_test, y_pred)
accuracy_bal = accuracy_score(y_test, y_pred)
print('Balanced Class Weights: Recall and Accuracy scores')
print(recall_bal)
print(accuracy_bal)

```

Logistic Regression II Review

Nice work! Let's review all the concepts you've learned in this lesson:

- The logistic regression primary assumptions include:
 - The target variable is binary
 - The observations are independent of one another
 - The sample size must be large enough
 - There should not be any extreme outliers in the data
- Additional assumptions of LR include:
 - Features are linearly related to the logit of the outcome (sigmoidal curve)
 - No multicollinearity (can cause the coefficients and p-values to be inaccurate)
- We can train and predict a model using `scikit-learn`.
- Hyperparameters are set before the model implementation step and tuned later to improve model performance.
- We can use metrics (accuracy, precision, recall, or F1-score) to evaluate our Logistic Regression model.
- The prediction threshold (a measure of certainty associated with each prediction) can be an additional lever to tune a model's predictions. In our breast cancer data, we wanted a lower threshold with fewer false negatives (a malignant tumor classified as benign).
- For each prediction threshold, the True Positive Rate (TPR) and the False Positive Rate (FPR) can be calculated and then plotted. The resulting curve these points form is known as the Receiver Operating Characteristic (ROC) curve.
- The AUC (Area Under the Curve) tells us how well our model can distinguish between the two classes. An AUC score close to 1 is a near-perfect classifier, whereas a value of 0.5 is equivalent to random guessing.
- If your classes are significantly imbalanced, this could create a bias towards the majority class since the model learns that it can have a higher accuracy if it predicts the majority class more often (not good).
- To bring the positivity rate of the dataset closer to .5, we can undersample the majority class, oversample the minority class, or balance the weights with `class_weight='balanced'`

K-Nearest Neighbors

Introduction

Before diving into the K-Nearest Neighbors

Preview: Docs Loading link description

[algorithm](#)

, let's first take a minute to think about an example.

Consider a dataset of movies. Let's brainstorm some features of a movie data point. A feature is a piece of information associated with a data point. Here are some potential features of movie data points:

the *length* of the movie in minutes.
the *budget* of a movie in dollars.

If you think back to the previous exercise, you could imagine movies being places in that two-dimensional space based on those numeric features. There could also be some

Preview: Docs Loading link description

[boolean](#)

features: features that are either true or false. For example, here are some potential boolean features:

Black and white. This feature would be `True` for black and white movies and `False` otherwise.
Directed by Stanley Kubrick. This feature would be `False` for almost every movie, but for the few movies that were directed by Kubrick, it would be `True`.

Finally, let's think about how we might want to classify a movie. For the rest of this lesson, we're going to be classifying movies as either good or bad. In our dataset, we've classified a movie as good if it had an IMDb rating of 7.0 or greater. Every "good" movie will have a class of `1`, while every bad movie will have a class of `0`.

To the right, we've created some movie data points where the first item in the list is the length, the second is the budget, and the third is whether the movie was directed by Stanley Kubrick.

Instructions

Checkpoint 1 Passed

1.

Add another movie to the code to your right. Add the variable `gone_with_the_wind`. This movie is 238 minutes long (wow!), had a budget of \$3,977,000, and was not directed by Stanley Kubrick.

`gone_with_the_wind` should be a list of three values. The last one should be `False`.

K-Nearest Neighbors

Distance Between Points - 2D

In the first exercise, we were able to visualize the dataset and estimate the k nearest neighbors of an unknown point. But a computer isn't going to be able to do that!

We need to define what it means for two points to be close together or far apart. To do this, we're going to use the [Distance Formula](#).

For this example, the data has two dimensions:

The length of the movie
The movie's release date

Consider *Star Wars* and *Raiders of the Lost Ark*. *Star Wars* is 125 minutes long and was released in 1977. *Raiders of the Lost Ark* is 115 minutes long and was released in 1981.

The distance between the movies is computed below:

$$\sqrt{(125 - 115)^2 + (1977 - 1981)^2} = 10.77$$

Instructions

Checkpoint 1 Enabled

1.

Write a function named `distance` that takes two lists named `movie1` and `movie2` as parameters. You can assume that each of these lists contains two numbers — the first number being the movie's runtime and the second number being the year the movie was released. The function should return the distance between the two lists.

Remember, in python, `x ** 0.5` will give you the square root of `x`.

Similarly, `x ** 2` will give you the square of `x`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Call the function on some of the movies we've given you.

Print the distance between *Star Wars* and *Raiders of the Lost Ark*.

Print the distance between *Star Wars* and *Mean Girls*.

Which movie is *Star Wars* more similar to?

```
star_wars = [125, 1977]
raiders = [115, 1981]
mean_girls = [97, 2004]

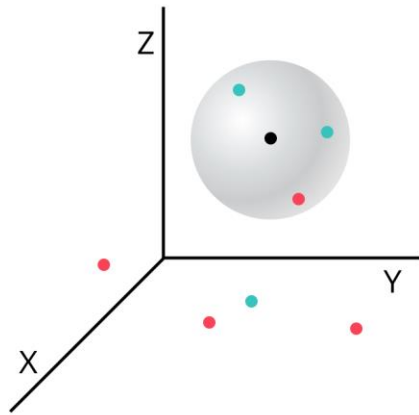
def distance(movie1, movie2):
    length_difference = (movie1[0] - movie2[0]) ** 2
    year_difference = (movie1[1] - movie2[1]) ** 2
    distance = (length_difference + year_difference) ** 0.5
    return distance

print(distance(star_wars, raiders))
print(distance(star_wars, mean_girls))
```

Distance Between Points - 3D

Making a movie rating predictor based on just the length and release date of movies is pretty limited. There are so many more interesting pieces of data about movies that we could use! So let's add another dimension.

Let's say this third dimension is the movie's budget. We now have to find the distance between these two points in three dimensions.



What if we're not happy with just three dimensions? Unfortunately, it becomes pretty difficult to visualize points in dimensions higher than 3. But that doesn't mean we can't find the distance between them.

The generalized distance formula between points A and B is as follows:

$$(A_1 - B_1)^2 + (A_2 - B_2)^2 + \dots + (A_n - B_n)^2$$

Here, $A_1 - B_1$ is the difference between the first feature of each point. $A_n - B_n$ is the difference between the last feature of each point.

Using this formula, we can find the K-Nearest Neighbors of a point in N-dimensional space! We now can use as much information about our movies as we want.

We will eventually use these distances to find the nearest neighbors to an unlabeled point.

Instructions

Checkpoint 1 Enabled

1.

Modify your `distance` function to work with any number of dimensions. Use a `for` loop to iterate through the dimensions of each movie.

Return the total distance between the two movies.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We've added a third dimension to each of our movies.

Print the new distance between *Star Wars* and *Raiders of the Lost Ark*.

Print the new distance between *Star Wars* and *Mean Girls*.

Which movie is *Star Wars* closer to now?

```
star_wars = [125, 1977, 11000000]
raiders = [115, 1981, 18000000]
mean_girls = [97, 2004, 17000000]

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
    return final_distance

print(distance(star_wars, raiders))
print(distance(star_wars, mean_girls))
```


Data with Different Scales: Normalization

In the next three lessons, we'll implement the three steps of the K-Nearest Neighbor Algorithm:

Normalize the data

Find the k nearest neighbors

Classify the new point based on those neighbors

When we added the dimension of budget, you might have realized there are some problems with the way our data currently looks.

Consider the two dimensions of release date and budget. The maximum difference between two movies' release dates is about 125 years (The Lumière Brothers were making movies in the 1890s). However, the difference between two movies' budget can be millions of dollars.

The problem is that the distance formula treats all dimensions equally, regardless of their scale. If two movies came out 70 years apart, that should be a pretty big deal. However, right now, that's exactly equivalent to two movies that have a difference in budget of 70 dollars. The difference in one year is exactly equal to the difference in one dollar of budget. That's absurd!

Another way of thinking about this is that the budget completely outweighs the importance of all other dimensions because it is on such a huge scale. The fact that two movies were 70 years apart is essentially meaningless compared to the difference in millions in the other dimension.

The solution to this problem is to [normalize the data](#) so every value is between 0 and 1. In this lesson, we're going to be using min-max normalization.

Instructions

1.

Write a function named `min_max_normalize` that takes a list of numbers named `lst` as a parameter (`lst` short for list).

Begin by storing the minimum and maximum values of the list in variables named `minimum` and `maximum`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create an empty list named `normalized`. Loop through each value in the original list.

Using min-max normalization, normalize the value and add the normalized value to the new list.

After adding every normalized value to `normalized`, return `normalized`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Call `min_max_normalize` using the given list `release_dates`. Print the resulting list.

What does the date 1897 get normalized to? Why is it closer to 0 than 1?

```
release_dates = [1897.0, 1998.0, 2000.0, 1948.0, 1962.0, 1950.0, 1975.0, 1960.0, 2017.0,
1937.0, 1968.0, 1996.0, 1944.0, 1891.0, 1995.0, 1948.0, 2011.0, 1965.0, 1891.0, 1978.0]
```

```
def min_max_normalize(lst):
    minimum = min(lst)
    maximum = max(lst)
    normalized = []

    for value in lst:
        normalized_num = (value - minimum) / (maximum - minimum)
        normalized.append(normalized_num)

    return normalized

print(min_max_normalize(release_dates))
```

Finding the Nearest Neighbors

The K-Nearest Neighbor Algorithm:

Normalize the data

Find the k nearest neighbors

Classify the new point based on those neighbors

Now that our data has been normalized and we know how to find the distance between two points, we can begin classifying unknown data!

To do this, we want to find the k nearest neighbors of the unclassified point. In a few exercises, we'll learn how to properly choose k , but for now, let's choose a number that seems somewhat reasonable. Let's choose 5.

In order to find the 5 nearest neighbors, we need to compare this new unclassified movie to every other movie in the dataset. This means we're going to be using the distance formula again and again. We ultimately want to end up with a sorted list of distances and the movies associated with those distances.

It might look something like this:

```
[
  [0.30, 'Superman II'],
  [0.31, 'Finding Nemo'],
  ...
  [0.38, 'Blazing Saddles']
]
```

Copy to Clipboard

In this example, the unknown movie has a distance of 0.30 to Superman II.

In the next exercise, we'll use the labels associated with these movies to classify the unlabeled point.

Instructions

Checkpoint 1 Passed

1.

Begin by running the program. We've imported and normalized a movie dataset for you and printed the data for the movie `Bruce Almighty`. Each movie in the dataset has three features:

- the normalized budget (dollars)
- the normalized duration (minutes)
- the normalized release year.

We've also imported the labels associated with every movie in the dataset. The label associated with `Bruce Almighty` is a 0, indicating that it is a bad movie. Remember, a bad movie had a rating less than 7.0 on IMDb.

Comment out the two print lines after you have run the program.

Checkpoint 2 Failed, try again

2.

Create a function called `classify` that has three parameters: the data point you want to classify named `unknown`, the dataset you are using to classify it named `dataset`, and `k`, the number of neighbors you are interested in.

For now put `pass` inside your function.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Inside the `classify` function remove `pass`. Create an empty list called `distances`.

Loop through every `title` in the dataset.

Access the data associated with every `title` by using `dataset[title]`.

Find the distance between `dataset[title]` and `unknown` and store this value in a variable called `distance_to_point`.

Add the list `[distance_to_point, title]` to `distances`.

Outside of the loop, return `distances`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We now have a list of distances and points. We want to sort this list by the distance (from smallest to largest). Before returning `distances`, use Python's built-in `sort()` function to sort `distances`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

The `k` nearest neighbors are now the first `k` items in `distances`. Create a new variable named `neighbors` and set it equal to the first `k` items of `distances`. You can use Python's built-in slice function.

For example, `lst[2:5]` will give you a list of the items at indices 2, 3, and 4 of `lst`.

Return `neighbors`.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Test the `classify` function and print the results. The three parameters you should use are:

```
[.4, .2, .9]
movie_dataset
5
```

Take a look at the 5 nearest neighbors. In the next exercise, we'll check to see how many of those neighbors are good and how many are bad.

```
from movies import movie_dataset, movie_labels

#print(movie_dataset['Bruce Almighty'])
#print(movie_labels['Bruce Almighty'])

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
    return final_distance

def classify(unknown, dataset, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    return neighbors

print(classify([.4, .2, .9], movie_dataset, 5))
```

Count Neighbors

The K-Nearest Neighbor Algorithm:

Normalize the data

Find the `k` nearest neighbors

Classify the new point based on those neighbors

We've now found the `k` nearest neighbors, and have stored them in a list that looks like this:

```
[
    [0.083, 'Lady Vengeance'],
    [0.236, 'Steamboy'],
    ...
    ...
    [0.331, 'Godzilla 2000']
]
```

Copy to Clipboard

Our goal now is to count the number of good movies and bad movies in the list of neighbors. If more of the neighbors were good, then the

Preview: Docs Loading link description

[algorithm](#)

will classify the unknown movie as good. Otherwise, it will classify it as bad.

In order to find the class of each of the labels, we'll need to look at our `movie_labels` dataset. For example, `movie_labels['Akira']` would give us `1` because Akira is classified as a good movie.

You may be wondering what happens if there's a tie. What if `k = 8` and four neighbors were good and four neighbors were bad? There are different strategies, but one way to break the tie would be to choose the class of the closest point.

Instructions

Checkpoint 1 Passed

1.

Our classify function now needs to have knowledge of the labels. Add a parameter named `labels` to `classify`. It should be the third parameter.

Checkpoint 2 Passed

2.

Continue writing your classify function.

Create two variables named `num_good` and `num_bad` and set them each at `0`. Use a for loop to loop through every `movie` in `neighbors`. Store their title in a variable called `title`.

Remember, every neighbor is a list of `[distance, title]` so the title can be found at index `1`.

For now, return `title` at the end of your function (outside of the loop).

Checkpoint 3 Passed

3.

Use `labels` and `title` to find the label of each movie:

If that label is a `0`, add one to `num_bad`.

If that label is a `1`, add one to `num_good`.

For now, return `num_good` at the end of your function.

Checkpoint 4 Passed

4.

We can finally classify our unknown movie:

If `num_good` is greater than `num_bad`, return a `1`.

Otherwise, return a `0`.

Checkpoint 5 Passed

5.

Call `classify` using the following parameters and print the result.

`[.4, .2, .9]` as the movie you're looking to classify.

`movie_dataset` the training dataset.

`movie_labels` as the training labels.

`k = 5`

Does the system predict this movie will be good or bad?

```
from movies import movie_dataset, movie_labels

def distance(movie1, movie2):
```

```

squared_difference = 0
for i in range(len(movie1)):
    squared_difference += (movie1[i] - movie2[i]) ** 2
final_distance = squared_difference ** 0.5
return final_distance

def classify(unknown, dataset, labels, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    num_good = 0
    num_bad = 0
    for neighbor in neighbors:
        title = neighbor[1]
        if labels[title] == 0:
            num_bad += 1
        elif labels[title] == 1:
            num_good += 1
    if num_good > num_bad:
        return 1
    else:
        return 0

print(classify([.4, .2, .9], movie_dataset, movie_labels, 5))

```

Classify Your Favorite Movie

Nice work! Your classifier is now able to predict whether a movie will be good or bad. So far, we've only tested this on a completely random point `[.4, .2, .9]`. In this exercise we're going to pick a real movie, normalize it, and run it through our classifier to see what it predicts!

In the instructions below, we are going to be testing our classifier using the 2017 movie *Call Me By Your Name*. Feel free to pick your favorite movie instead!

Instructions

Checkpoint 1 Enabled

1.

To begin, we want to make sure the movie that we want to classify isn't already in our database. This is important because we don't want one of the nearest neighbors to be itself!

You can do this by using the `in` keyword.

Begin by printing if the title of your movie is `in movie_dataset`. This should print `False`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Once you confirm your movie is not in your database, we need to make a datapoint for your movie. Create a variable named `my_movie` and set it equal to a list of three numbers. They should be:

- The movie's budget (dollars)
- The movie's runtime (minutes)
- The year the movie was released

Make sure to put the information in that order.

If you want to use *Call Me By Your Name*, the budget was 3,500,000 dollars, the runtime was 132 minutes, and the movie was released in 2017.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, we want to normalize this datapoint. We've included the function `normalize_point` which takes a datapoint as a parameter and returns the point normalized. Create a variable called `normalized_my_movie` and set it equal to the normalized value of `my_movie`. Print the result!

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Finally, call `classify` with the following parameters:

```
normalized_my_movie
movie_dataset
movie_labels
5
```

Print the result? Did your classifier think your movie was good or bad?

```
from movies import movie_dataset, movie_labels, normalize_point

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
    return final_distance

def classify(unknown, dataset, labels, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    num_good = 0
    num_bad = 0
    for neighbor in neighbors:
        title = neighbor[1]
        if labels[title] == 0:
            num_bad += 1
        elif labels[title] == 1:
            num_good += 1
    if num_good > num_bad:
        return 1
    else:
        return 0

print("Call Me By Your Name" in movie_dataset)
my_movie = [3500000, 132, 2017]
normalized_my_movie = normalize_point(my_movie)
print(normalized_my_movie)
print(classify(normalized_my_movie, movie_dataset, movie_labels, 5))
```

Training and Validation Sets

You've now built your first K Nearest Neighbors

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

capable of classification. You can feed your program a never-before-seen movie and it can predict whether its IMDb rating was above or below 7.0. However, we're not done yet. We now need to report how effective our algorithm is.

After all, it's possible our predictions are totally wrong!

As with most

Preview: Docs Loading link description

[machine learning](#)

algorithms, we have split our data into a training set and validation set.

Once these sets are created, we will want to use every point in the validation set as input to the K Nearest Neighbor algorithm. We will take a movie from the validation set, compare it to all the movies in the training set, find the K Nearest Neighbors, and make a prediction. After making that prediction, we can then peek at the real answer (found in the validation labels) to see if our classifier got the answer correct.

If we do this for every movie in the validation set, we can count the number of times the classifier got the answer right and the number of times it got it wrong. Using those two numbers, we can compute the validation accuracy.

Validation accuracy will change depending on what K we use. In the next exercise, we'll use the validation accuracy to pick the best possible K for our classifier.

Instructions

Checkpoint 1 Passed

1.

We've imported `training_set`, `training_labels`, `validation_set`, and `validation_labels`. Let's take a look at one of the movies in `validation_set`.

The movie "Bee Movie" is in `validation_set`. Print out the data associated with *Bee Movie*. Print *Bee Movie*'s label as well (which can be found in `validation_labels`).

Is *Bee Movie* a good or bad movie?

Checkpoint 2 Passed

2.

Let's have our classifier predict whether *Bee Movie* is good or bad using `k = 5`. Call the `classify` function using the following parameters:

Bee Movie's data

```
training_set  
training_labels  
5
```

Store the results in a variable named `guess` and print `guess`.

Checkpoint 3 Passed

3.

Let's check to see if our classification got it right. If `guess` is equal to *Bee Movie*'s real class (found in `validation_labels`), print "Correct!". Otherwise, print "Wrong!".

```
from movies import training_set, training_labels, validation_set, validation_labels  
  
def distance(movie1, movie2):  
    squared_difference = 0  
    for i in range(len(movie1)):  
        squared_difference += (movie1[i] - movie2[i]) ** 2  
    final_distance = squared_difference ** 0.5  
    return final_distance  
  
def classify(unknown, dataset, labels, k):  
    distances = []  
    #Looping through all points in the dataset  
    for title in dataset:  
        movie = dataset[title]  
        distance_to_point = distance(movie, unknown)  
        #Adding the distance and point associated with that distance  
        distances.append([distance_to_point, title])  
    distances.sort()  
    #Taking only the k closest points  
    neighbors = distances[0:k]  
    num_good = 0  
    num_bad = 0  
    for neighbor in neighbors:  
        title = neighbor[1]  
        if labels[title] == 0:  
            num_bad += 1  
        elif labels[title] == 1:  
            num_good += 1  
    if num_good > num_bad:  
        return 1  
    else:  
        return 0
```

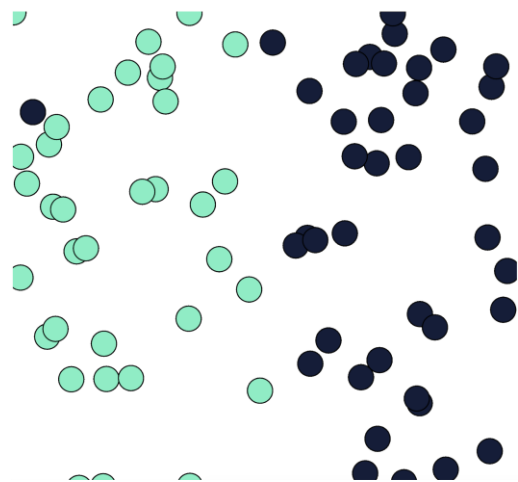
```
print(validation_set["Bee Movie"])
print(validation_labels["Bee Movie"])

guess=classify(validation_set["Bee Movie"], training_set, training_labels, 5)
print(guess)
if guess == validation_labels["Bee Movie"]:
    print("Correct!")
else:
    print("Wrong!")
```

Choosing K

In the previous exercise, we found that our classifier got one point in the training set correct. Now we can test every point to calculate the validation accuracy.

The validation accuracy changes as k changes. The first situation that will be useful to consider is when k is very small. Let's say $k = 1$. We would expect the validation accuracy to be fairly low due to *overfitting*. Overfitting is a concept that will appear almost any time you are writing a machine learning algorithm. Overfitting occurs when you rely too heavily on your training data; you assume that data in the real world will always behave exactly like your training data. In the case of K-Nearest Neighbors, overfitting happens when you don't consider enough neighbors. A single outlier could drastically determine the label of an unknown point. Consider the image below.



The dark blue point in the top left corner of the graph looks like a fairly significant outlier. When $k = 1$, all points in that general area will be classified as dark blue when it should probably be classified as green. Our classifier has relied too heavily on the small quirks in the training data.

On the other hand, if k is very large, our classifier will suffer from *underfitting*. Underfitting occurs when your classifier doesn't pay enough attention to the small quirks in the training set. Imagine you have 100 points in your training set and you set $k = 100$. Every single unknown point will be classified in the same exact way. The distances between the points don't matter at all! This is an extreme example, however, it demonstrates how the classifier can lose understanding of the training data if k is too big.

Instructions

Checkpoint 1 Passed

1.

Begin by creating a function called `find_validation_accuracy` that takes five parameters. The parameters should be `training_set`, `training_labels`, `validation_set`, `validation_labels`, and `k`.

Checkpoint 2 Passed

2.

Create a variable called `num_correct` and have it begin at `0.0`. Loop through the movies of `validation_set`, and call `classify` using each movie's data, the `training_set`, the `training_labels`, and `k`. Store the result in a variable called `guess`. For now, return `guess` outside of your loop.

Remember, the movie's data can be found by using `validation_set[title]`.

Checkpoint 3 Passed

3.

Inside the for loop, compare `guess` to the corresponding label in `validation_labels`. If they were equal, add `1` to `num_correct`. For now, outside of the for loop, return `num_correct`

Checkpoint 4 Passed

4.

Outside the for loop return the validation error. This should be `num_correct` divided by the total number of points in the validation set.

Checkpoint 5 Passed

5.

Call `find_validation_accuracy` with `k = 3`. Print the results The code should take a couple of seconds to run.

```
from movies import training_set, training_labels, validation_set, validation_labels

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
    return final_distance

def classify(unknown, dataset, labels, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    num_good = 0
    num_bad = 0
    for neighbor in neighbors:
        title = neighbor[1]
        if labels[title] == 0:
            num_bad += 1
        elif labels[title] == 1:
            num_good += 1
    if num_good > num_bad:
        return 1
    else:
        return 0

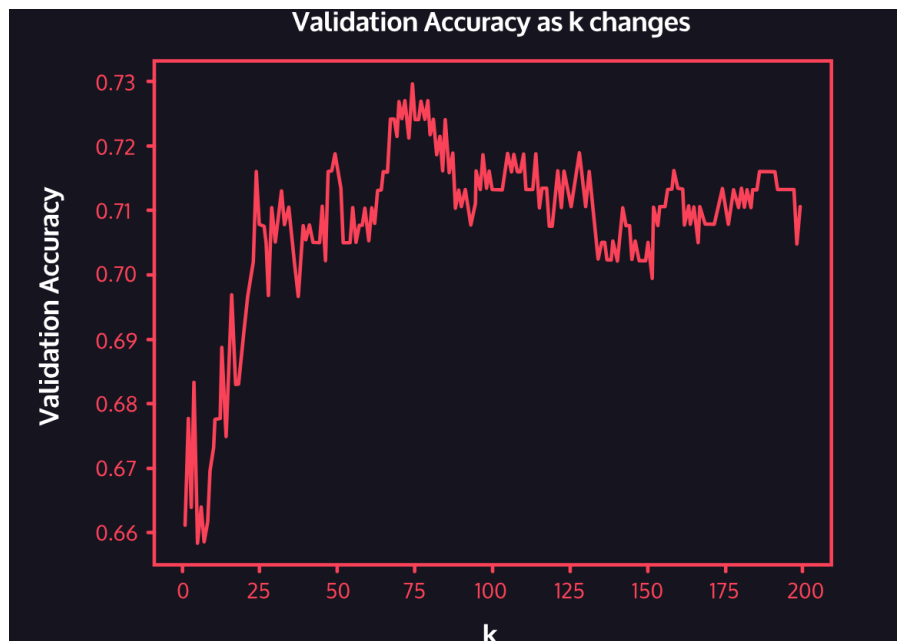
def find_validation_accuracy(training_set, training_labels, validation_set,
validation_labels, k):
    num_correct = 0.0
    for title in validation_set:
        guess = classify(validation_set[title], training_set, training_labels, k)
        if guess == validation_labels[title]:
            num_correct += 1
    return num_correct / len(validation_set)

print(find_validation_accuracy(training_set, training_labels, validation_set,
validation_labels, 3))
```

Graph of K

The graph to the right shows the validation accuracy of our movie classifier as k increases.

When k is small, overfitting occurs and the accuracy is relatively low. On the other hand, when k gets too large, underfitting occurs and accuracy starts to drop.



Using sklearn

You've now written your own K-Nearest Neighbor classifier from scratch! However, rather than writing your own classifier every time, you can use Python's `sklearn` library. `sklearn` is a Python library specifically used for

Preview: Docs Loading link description

[Machine Learning](#)

. It has an amazing number of features, but for now, we're only going to investigate its K-Nearest Neighbor classifier. There are a couple of steps we'll need to go through in order to use the library. First, you need to create a `KNeighborsClassifier` object. This object takes one

Preview: Docs Loading link description

[parameter](#)

- k . For example, the code below will create a classifier where $k = 3$

```
classifier = KNeighborsClassifier(n_neighbors = 3)
```

Copy to Clipboard

Next, we'll need to train our classifier. The `.fit()`

Preview: Docs Loading link description

[method](#)

takes two parameters. The first is a list of points, and the second is the labels associated with those points. So for our movie example, we might have something like this

```
training_points = [
    [0.5, 0.2, 0.1],
    [0.9, 0.7, 0.3],
    [0.4, 0.5, 0.7]
]

training_labels = [0, 1, 1]
classifier.fit(training_points, training_labels)
```

Copy to Clipboard

Finally, after training the model, we can classify new points. The `.predict()` method takes a list of points that you want to classify. It returns a list of its guesses for those points.

```
unknown_points = [
    [0.2, 0.1, 0.7],
    [0.4, 0.7, 0.6],
    [0.5, 0.8, 0.1]
]

guesses = classifier.predict(unknown_points)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

We've imported `sklearn` for you. Create a `KNeighborsClassifier` named `classifier` that uses `k=5`.

Checkpoint 2 Passed

2.

We've also imported some movie data. Train your classifier using `movie_dataset` as the training points and `labels` as the training labels.

Checkpoint 3 Passed

3.

Let's classify some movies. Classify the following movies: `[.45, .2, .5]`, `[.25, .8, .9]`, `[.1, .1, .9]`. Print the classifications!

Which movies were classified as good movies and which were classified as bad movies?

Remember, those three numbers associated with a movie are the normalized budget, run time, and year of release.

```
from movies import movie_dataset, labels
from sklearn.neighbors import KNeighborsClassifier

classifier = KNeighborsClassifier(n_neighbors = 5)
classifier.fit(movie_dataset, labels)
guess = classifier.predict([[.45, .2, .5], [.25, .8, .9], [.1, .1, .9]])
print(guess)
```

Review

Congratulations! You just implemented your very own classifier from scratch and used Python's `sklearn` library. In this lesson, you learned some techniques very specific to the K-Nearest Neighbor

Preview: Docs Loading link description

[algorithm](#)

, but some general

Preview: Docs Loading link description

[machine learning](#)

techniques as well. Some of the major takeaways from this lesson include:

- Data with n features can be conceptualized as points lying in n -dimensional space.
- Data points can be compared by using the distance formula. Data points that are similar will have a smaller distance between them.
- A point with an unknown class can be classified by finding the k nearest neighbors
- To verify the effectiveness of a classifier, data with known classes can be split into a training set and a validation set. Validation error can then be calculated.
- Classifiers have parameters that can be tuned to increase their effectiveness. In the case of K-Nearest Neighbors, k can be changed.
- A classifier can be trained improperly and suffer from overfitting or underfitting. In the case of K-Nearest Neighbors, a low k often leads to overfitting and a large k often leads to underfitting.
- Python's sklearn library can be used for many classification and machine learning algorithms.

To the right is an interactive visualization of K-Nearest Neighbors. If you move your mouse over the canvas, the location of your mouse will be classified as either green or blue. The nearest neighbors to your mouse are highlighted in yellow. Use the slider to change k to see how the boundaries of the classification change.

Nearest Neighbor Regressor

Regression

The K-Nearest Neighbors

Preview: Docs Loading link description

[algorithm](#)

is a powerful supervised

Preview: Docs Loading link description

[machine learning](#)

algorithm typically used for classification. However, it can also perform regression.

In this lesson, we will use the movie dataset that was used in the [K-Nearest Neighbors classifier lesson](#). However, instead of classifying a new movie as either good or bad, we are now going to predict its IMDb rating as a real number.

This process is almost identical to classification, except for the final step. Once again, we are going to find the k nearest neighbors of the new movie by using the distance formula. However, instead of counting the number of good and bad neighbors, the regressor averages their IMDb ratings.

For example, if the three nearest neighbors to an unrated movie have ratings of 5.0, 9.2, and 6.8, then we could predict that this new movie will have a rating of 7.0.

Instructions

Checkpoint 1 Failed, try again

1.

We've imported most of the K-Nearest Neighbor algorithm. Before we dive into finishing the regressor, let's refresh ourselves with the data.

At the bottom of your code, print `movie_dataset["Life of Pi"]`. You should see a list of three values. These values are the [normalized](#) values for the movie's budget, runtime, and release year.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the rating for "Life of Pi". This can be found in `movie_ratings`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We've included the majority of the K-Nearest Neighbor algorithm in the `predict()` function. Right now, the variable `neighbors` stores a list of `[distance, title]` pairs.

Loop through every neighbor and find its rating in `movie_ratings`. Add those ratings together and return that sum divided by the total number of neighbors.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Call `predict` with the following parameters:

```
[0.016, 0.300, 1.022]
movie_dataset
movie_ratings
5
```

Print the result.

Note that the list `[0.016, 0.300, 1.022]` is the normalized budget, runtime, and year of the movie *Incredibles 2*! The normalized year is larger than 1 because our training set only had movies that were released between 1927 and 2016 — *Incredibles 2* was released in 2018.

```
from movies import movie_dataset, movie_ratings

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
    return final_distance

def predict(unknown, dataset, movie_ratings, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    sum = 0
    for neighbor in neighbors:
        title = neighbor[1]
        sum += movie_ratings[title]
    return sum/len(neighbors)

print(movie_dataset["Life of Pi"])
print(movie_ratings["Life of Pi"])
print(predict([0.016, 0.300, 1.022], movie_dataset, movie_ratings, 5))
```

Weighted Regression

We're off to a good start, but we can be even more clever in the way that we compute the average. We can compute a *weighted* average based on how close each neighbor is.

Let's say we're trying to predict the rating of movie X and we've found its three nearest neighbors. Consider the following table:

Movie	Rating	Distance to movie X
A	5.0	3.2
B	6.8	11.5
C	9.0	1.1

If we find the mean, the predicted rating for X would be 6.93. However, movie X is most similar to movie C, so movie C's rating should be more important when computing the average. Using a weighted average, we can find movie X's rating:

$$\frac{\frac{5.0}{3.2} + \frac{6.8}{11.5} + \frac{9.0}{1.1}}{\frac{1}{3.2} + \frac{1}{11.5} + \frac{1}{1.1}} = 7.9$$

The numerator is the sum of every rating divided by their respective distances. The denominator is the sum of one over every distance. Even though the ratings are the same as before, the *weighted* average has now gone up to 7.9.

Instructions

Checkpoint 1 Failed, try again

1.

Let's redo our `predict()` function so it computes the weighted average.

Before you begin looping through the neighbors, create a variable named `numerator` and set it to 0. Loop through every neighbor and add the neighbor's rating (found in `movie_ratings`) divided by the neighbor's distance to `numerator`.

For now, return `numerator`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's now calculate the denominator of the weighted average. Before your loop, create a variable named `denominator` and set it equal to 0.

Inside your for loop, add 1 divided by the neighbor's distance to `denominator`.

Outside the loop, return `numerator/denominator`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Once again call your `predict` function using *Incredibles 2*'s features. Those features were `[0.016, 0.300, 1.022]`. Set `k = 5`. Print the results.

How did using a weighted average change the predicted rating? Remember, before calculating the weighted average the prediction was 6.86.

```
from movies import movie_dataset, movie_ratings

def distance(movie1, movie2):
    squared_difference = 0
    for i in range(len(movie1)):
        squared_difference += (movie1[i] - movie2[i]) ** 2
    final_distance = squared_difference ** 0.5
```

```

    return final_distance

def predict(unknown, dataset, movie_ratings, k):
    distances = []
    #Looping through all points in the dataset
    for title in dataset:
        movie = dataset[title]
        distance_to_point = distance(movie, unknown)
        #Adding the distance and point associated with that distance
        distances.append([distance_to_point, title])
    distances.sort()
    #Taking only the k closest points
    neighbors = distances[0:k]
    numerator = 0
    denominator = 0
    for neighbor in neighbors:
        rating = movie_ratings[neighbor[1]]
        distance_to_neighbor = neighbor[0]
        numerator += rating / distance_to_neighbor
        denominator += 1 / distance_to_neighbor
    return numerator / denominator

print(predict([0.016, 0.300, 1.022], movie_dataset, movie_ratings, 5))

```

Scikit-learn

Now that you've written your own K-Nearest Neighbor regression model, let's take a look at scikit-learn's implementation. The `KNeighborsRegressor` class is very similar to `KNeighborsClassifier`.

We first need to create the regressor. We can use the

Preview: Docs Loading link description

[parameter](#)

`n_neighbors` to define our value for `k`.

We can also choose whether or not to use a weighted average using the parameter `weights`. If `weights` equals "uniform", all neighbors will be considered equally in the average. If `weights` equals "distance", then a weighted average is used.

```
classifier = KNeighborsRegressor(n_neighbors = 3, weights = "distance")
```

Copy to Clipboard

Next, we need to fit the model to our training data using the `.fit()`

Preview: Docs Loading link description

[method](#)

`.fit()` takes two parameters. The first is a list of points, and the second is a list of values associated with those points.

```

training_points = [
    [0.5, 0.2, 0.1],
    [0.9, 0.7, 0.3],
    [0.4, 0.5, 0.7]
]

training_labels = [5.0, 6.8, 9.0]
classifier.fit(training_points, training_labels)

```

Copy to Clipboard

Finally, we can make predictions on new data points using the `.predict()` method. `.predict()` takes a list of points and returns a list of predictions for those points.

```
unknown_points = [
    [0.2, 0.1, 0.7],
    [0.4, 0.7, 0.6],
    [0.5, 0.8, 0.1]
]

guesses = classifier.predict(unknown_points)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Create a `KNeighborsRegressor` named `regressor` where `n_neighbors = 5` and `weights = "distance"`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We've also imported some movie data. Train your classifier using `movie_dataset` as the training points and `movie_ratings` as the training values.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's predict some movie ratings. Predict the ratings for the following movies:

```
[0.016, 0.300, 1.022],
[0.0004092981, 0.283, 1.0112],
[0.00687649, 0.235, 1.0112]
```

These three lists are the features for *Incredibles 2*, *The Big Sick*, and *The Greatest Showman*. Those three numbers associated with a movie are the normalized budget, runtime, and year of release.

Print the predictions!

```
from movies import movie_dataset, movie_ratings
from sklearn.neighbors import KNeighborsRegressor

regressor = KNeighborsRegressor(n_neighbors = 5, weights = "distance")
regressor.fit(movie_dataset, movie_ratings)
print(regressor.predict([[0.016, 0.300, 1.022], [0.0004092981, 0.283, 1.0112], [0.00687649, 0.235, 1.0112]]))
```

Review

Great work! Here are some of the major takeaways from this lesson:

The K-Nearest Neighbor

Preview: Docs Loading link description

[algorithm](#)

can be used for regression. Rather than returning a classification, it returns a number.

By using a weighted average, data points that are extremely similar to the input point will have more of a say in the final result.

scikit-learn has an implementation of a K-Nearest Neighbor regressor named `KNeighborsRegressor`.

In the browser, you'll find an example of a K-Nearest Neighbor regressor in action. Instead of the training data coming from IMDb ratings, you can create the training data yourself! Rate the movies that you have seen. Once you've rated more than `k` movies, a K-Nearest Neighbor regressor will train on those ratings. It will then make predictions for every movie that you haven't seen.

As you add more and more ratings, the predictor should become more accurate. After all, the regressor needs information from the user in order to make personalized recommendations. As a result, the system is somewhat useless to brand new users — it takes some time for the system to “warm up” and get enough data about a user. This conundrum is an example of the *cold start problem*.

Decision Trees

What is a Decision Tree?

Decision trees are

Preview: Docs Loading link description

[machine learning](#)

models that try to find patterns in the features of data points. Take a look at the tree on this page. This tree tries to predict whether a student will get an A on their next test.

By asking questions like “What is the student’s average grade in the class” the decision tree tries to get a better understanding of their chances on the next test.

In order to make a classification, this classifier needs a data point with four features:

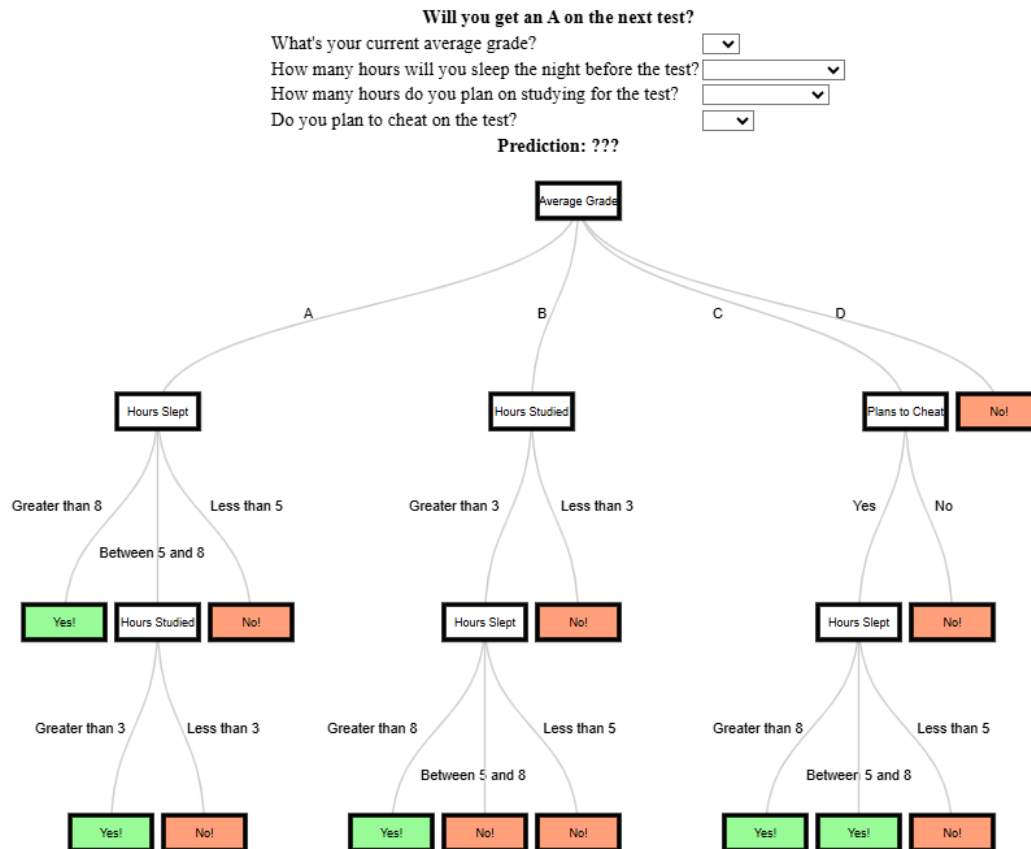
- The student’s average grade in the class.

- The number of hours the student plans on studying for the test.

- The number of hours the student plans on sleeping the night before the test.

- Whether or not the student plans on cheating.

For example, let’s say that somebody has a “B” average in the class, studied for more than 3 hours, slept less than 5 hours before the test, and doesn’t plan to cheat. If we start at the top of the tree and take the correct path based on that data, we’ll arrive at a *leaf node* that predicts the person will *not* get an A on the next test.



What is a Decision Tree? (Contd.)

If we're given this magic tree, it seems relatively easy to make classifications. But how do these trees get created in the first place? Decision trees are supervised

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

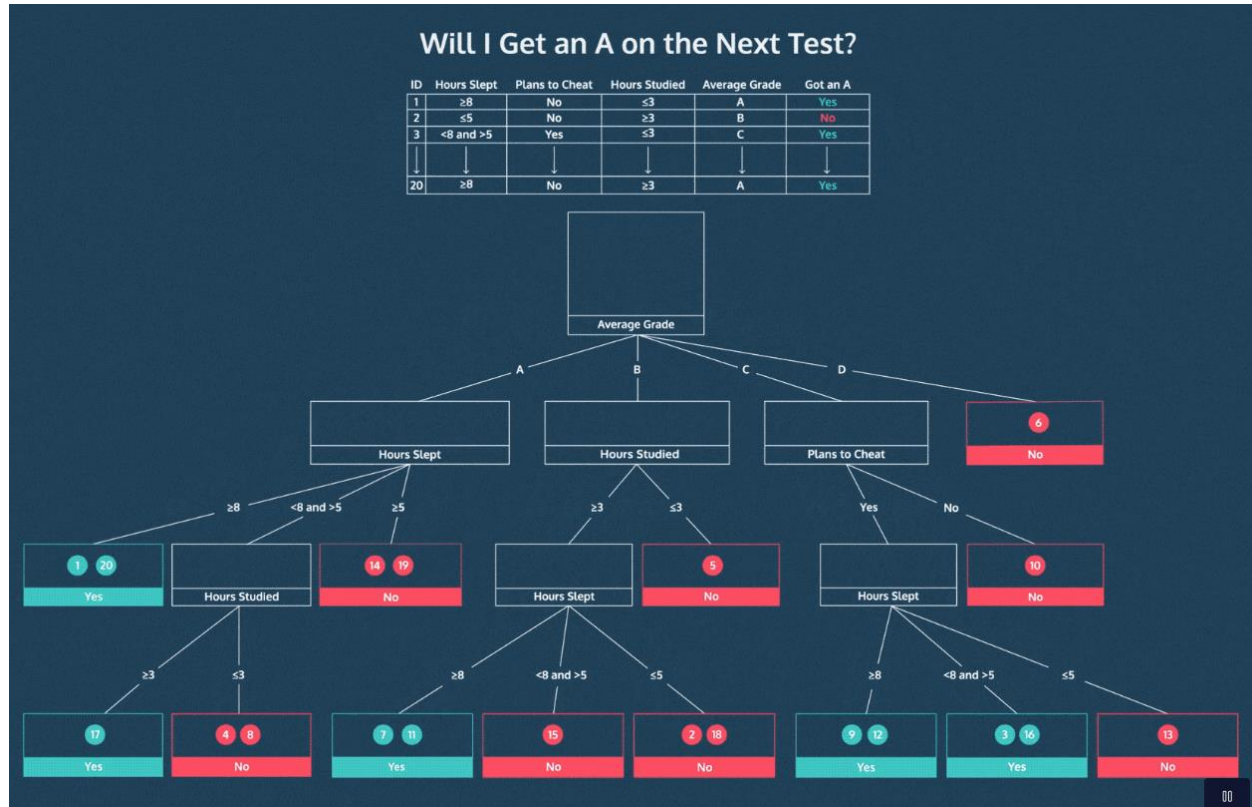
models, which means that they're created from a training set of labeled data. Creating the tree is where the *learning* in machine learning happens.

Take a look at the gif on this page. We begin with every point in the training set at the top of the tree. These training points have labels – the red points represent students that didn't get an A on a test and the green points represent students that did get an A on a test.

We then decide to split the data into smaller groups based on a feature. For example, that feature could be something like their average grade in the class. Students with an A average would go into one set, students with a B average would go into another subset, and so on.

Once we have these subsets, we repeat the process – we split the data in each subset again on a different feature. Eventually, we reach a point where we decide to stop splitting the data into smaller groups. We've reached a leaf of the tree. We can now count up the labels of the data in that leaf. If an unlabeled point reaches that leaf, it will be classified as the majority label.

We can now make a tree, but how did we know which features to split the data set with? After all, if we started by splitting the data based on the number of hours they slept the night before the test, we'd end up with a very different tree that would produce very different results. How do we know which tree is best? We'll tackle this question soon!



Implementing a Decision Tree

To answer the questions posed in the previous exercise, we're going to do things a bit differently in this lesson and work "backwards" (!!!): we're going to first fit a decision tree to a dataset and visualize this tree using `scikit-learn`. We're then going to systematically unpack the following: how to interpret the tree visualization, how `scikit-learn`'s implementation works, what is gini impurity, what are parameters and hyper-parameters of the decision tree model, etc.

We're going to use a dataset about cars with six features:

- The price of the car, `buying`, which can be "vhigh", "high", "med", or "low".
- The cost of maintaining the car, `maint`, which can be "vhigh", "high", "med", or "low".
- The number of doors, `doors`, which can be "2", "3", "4", "5more".
- The number of people the car can hold, `persons`, which can be "2", "4", or "more".
- The size of the trunk, `luggage`, which can be "small", "med", or "big".
- The safety rating of the car, `safety`, which can be "low", "med", or "high".

The question we will be trying to answer using decision trees is: when considering buying a car, what factors go into making that decision?

Instructions

Checkpoint 1 Passed

1.

We've imported the dataset in the workspace.

Take a look at the first five rows of the dataset by uncommenting `print(df.head())` and clicking Run.

We've created dummy features for the categorical values and set the predictor and target variables as `x` and `y` respectively. Uncomment the lines pertaining to this and press Run.

You can examine the new set of features using `print(X.columns)`

How many features are there now? Do they make sense?

Checkpoint 2 Enabled

2.

We can now perform a train-test split and fit a decision tree to our training data. We'll be using `scikit-learn`'s `train_test_split` function to do the split and the `DecisionTreeClassifier()` class to fit the data. Uncomment the lines that do the same and press Run.

(Don't worry about the input parameters to the tree classifier for the moment- we'll get to them in a bit!)

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We're now ready to visualize the decision tree! The `tree` module within `scikit-learn` has a plotting functionality that allows us to do this. Uncomment the lines relevant to this and press Run to view the tree visualization.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt

#Import models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

#Loading the dataset
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])

# 1a. Take a look at the dataset
print(df.head())

# 1b. Setting the target and predictor variables
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']

# 1c. Examine the new features
print(X.columns)
print(len(X.columns))

# 2a. Performing the train-test split
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.2)

# 2b.Fitting the decision tree classifier
dt = DecisionTreeClassifier(max_depth=3, ccp_alpha=0.01,criterion='gini')
dt.fit(x_train, y_train)

# 3.Plotting the Tree
plt.figure(figsize=(20,12))
tree.plot_tree(dt, feature_names = x_train.columns, max_depth=5, class_names = ['unacc',
'acc'], label='all', filled=True)
plt.tight_layout()
plt.show()
```

Interpreting a Decision Tree

We're now going to examine the decision tree we built for the car dataset. The image to the right of the

Preview: Docs Loading link description

[code editor](#)

is the exact plot you created in the previous exercise. Two important concepts to note here are the following:

The root node is identified as the top of the tree. This is notated already with the number of samples and the numbers in each class (i.e. unacceptable vs. acceptable) that was used to build the tree.

Splits occur with True to the left, False to the right. Note the right split is a `leaf node` i.e., there are no more branches. Any decision ending here results in the majority class. (The majority class here is `unacc.`)

(Note that there is a term called `gini` in each of the boxes that is immensely important for how the split is done - we will explore this in the following exercise!)

To interpret the tree, it's useful to keep in mind that the

Preview: Docs Loading link description

[variables](#)

we're looking at are categorical variables that correspond to:

`buying`: The price of the car which can be "vhigh", "high", "med", or "low".
`maint`: The cost of maintaining the car which can be "vhigh", "high", "med", or "low".
`doors`: The number of doors which can be "2", "3", "4", "5more".
`persons`: The number of people the car can hold which can be "2", "4", or "more".
`lugboot`: The size of the trunk which can be "small", "med", or "big".
`safety`: The safety rating of the car which can be "low", "med", or "high".

Instructions

Checkpoint 1 Passed

1.

The root node is identified as the top of the tree. This is notated already with the number of samples and the numbers in each class (i.e. unacceptable vs. acceptable) that was used to build the tree. We've created a variable `frac_acc` to print the fraction of acceptable cars the tree was trained on. Calculate this from the root node of the tree.

Checkpoint 2 Passed

2.

The first split occurs off the root node based on the boolean `safety_low <= 0.5`. As this is a dummy variable, a value of 1 indicates `safety='low'`, 0 indicates `safety<>'low'`. Splits occur with True to the left, False to the right. Note the right split is a `leaf node` (i.e. there are no more branches), so any decision ending here results in the majority class. What is the predicted class of any vehicle with LOW safety?

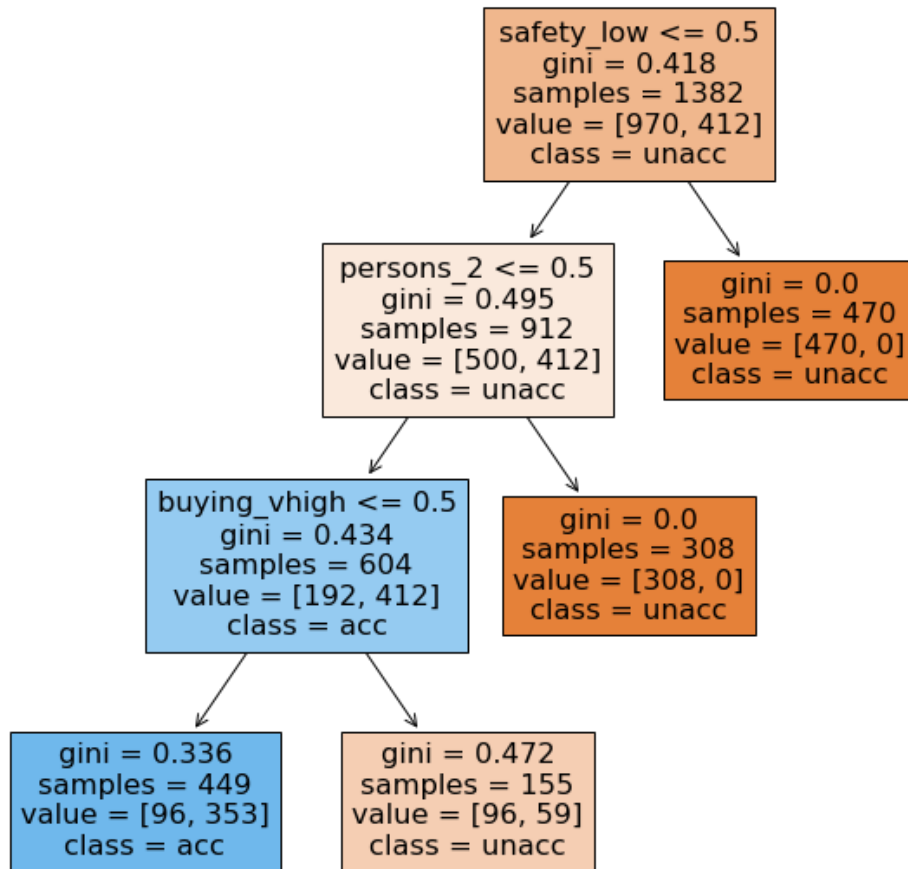
(We've created a variable `low_safety_class` to fill in the answer in the code editor.)

Checkpoint 3 Passed

3.

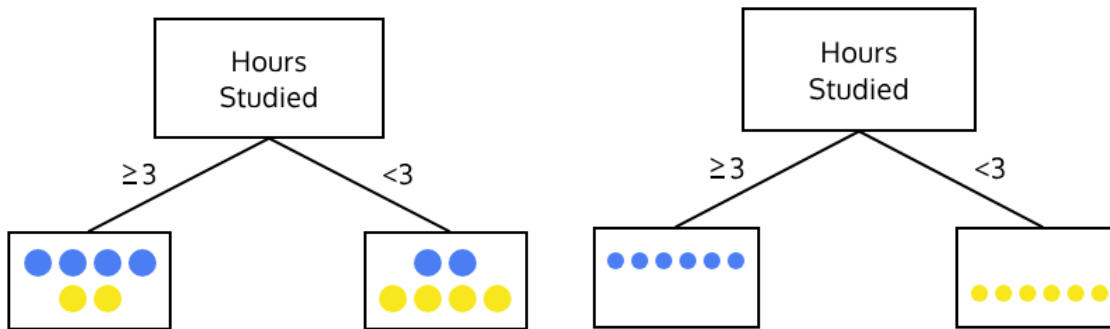
Identify the final outcome of the decision tree for the sample with `buying_vhigh=1, persons_2= 0, safety_low=0`. We've defined a variable called `sample_class` to fill in the correct answer.

```
frac_acc = 412/1382
print(f'Fraction of acceptable cars: {frac_acc}')
low_safety_class = 'unacc'
print(f'Cars with low safety: {low_safety_class}')
sample_class = 'unacc'
print(f'The class of the sample car: {sample_class}')
```



Gini Impurity

Consider the two trees below. Which tree would be more useful as a model that tries to predict whether someone would get an A in a class?



Let's say you use the top tree. You'll end up at a leaf node where the label is up for debate. The training data has labels from both classes! If you use the bottom tree, you'll end up at a leaf where there's only one type of label. There's no debate at all! We'd be much more confident about our classification if we used the bottom tree.

This idea can be quantified by calculating the *Gini impurity* of a set of data points. For two classes (1 and 2) with probabilities p_1 and p_2 respectively, the Gini impurity is:

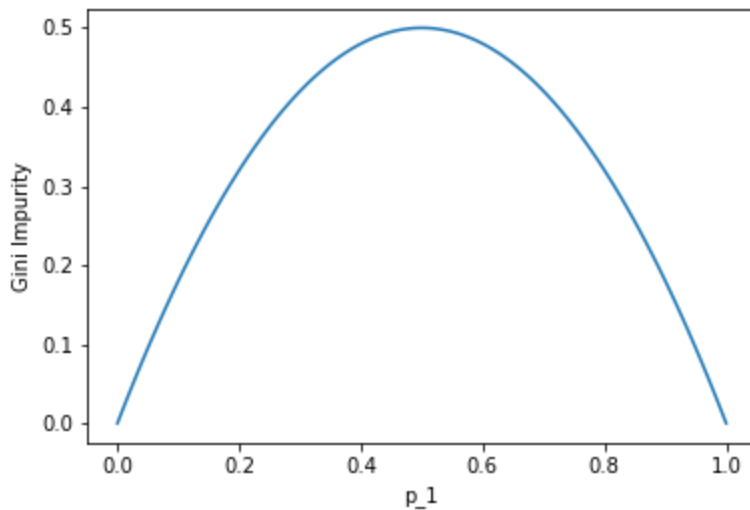
$$1 - (p_1^2 + p_2^2) = 1 - (p_1^2 + (1 - p_1)^2)$$

Mathematically, The Gini Index is represented by

$$Gini\ impurity = 1 - \sum (p(i)^2)$$

Another commonly used formula is:

$$Gini\ impurity = 1 - \sum (p(i) * (1 - p(i)))$$



The goal of a decision tree model is to separate the classes the best possible, i.e. minimize the impurity (or maximize the purity). Notice that if p_1 is 0 or 1, the Gini impurity is 0, which means there is only one class so there is perfect separation. From the graph, the Gini impurity is maximum at $p_1=0.5$, which means the two classes are equally balanced, so this is perfectly impure!

In general, the Gini impurity for C classes is defined as:

$$1 - \sum_{i=1}^C p_i^2$$

Instructions

Checkpoint 1 Enabled

1.

Using the same decision tree from the previous exercise, notice that each node is labeled with a `gini`, which is the Gini impurity for the samples at that node. For the root node, calculate the Gini impurity using the formula and confirm this matches the value listed on the node.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Confirm the Gini impurity for the bottom left node. Since this is a non-root node (either a leaf or a split node), the Gini impurity is only calculated for the samples passing through this node, not the entire dataset – hence the number of samples is 449, not 1382.

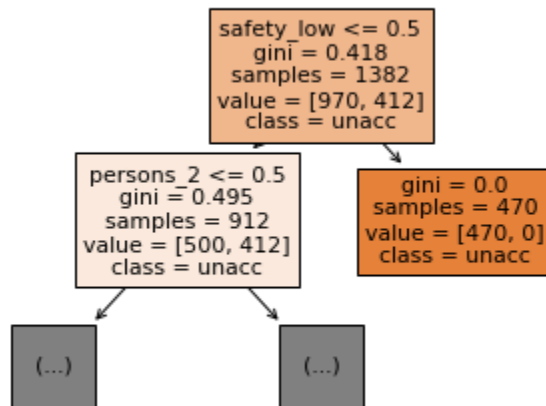
```
gini_root = 1 - ( (412/1382)**2 + (970/1382)**2 )
print(f'Gini impurity root node : {gini_root}')

gini_bottom_left = 1 - ( (96/449)**2 + (353/449)**2 )
print(f'Gini impurity bottom left node : {gini_bottom_left}')
```

Information Gain

We know that we want to end up with leaves with a low Gini Impurity, but we still need to figure out which features to split on in order to achieve this. To answer this question, we can calculate the *information gain* of splitting the data on a certain feature. Information gain measures the difference in the impurity of the data before and after the split.

For example, let's start with the root node of our car acceptability tree:



The initial Gini impurity (which we confirmed previously) is 0.418. The first split occurs based on the feature `safety_low <= 0.5`, and as this is a dummy variable with values 0 and 1, this split is pushing higher safety cars to the left (912 samples) and low safety cars to the right (470 samples). Before we discuss how we decided to split on this feature, let's calculate the information gain.

The new Gini impurities for these two split nodes are 0.495 and 0 (which is a pure leaf node!). All together, the now weighted Gini impurity after the split is:

$$912/1382 * (.495) + 470/1382 * (0) = 0.3267$$

$$912/1382 * (.495) + 470/1382 * (0) = 0.3267$$

Not bad! (Remember we *want* our Gini impurity to be lower!) This is lower than our initial Gini impurity, so by splitting the data in that way, we've gained some information about how the data is structured — the datasets after the split are purer than they were before the split.

Then the information gain (or reduction in impurity after the split) is

$$0.4185 - 0.3267 = 0.0918$$

$$0.4185 - 0.3267 = 0.0918$$

The higher the information gain the better — if information gain is 0, then splitting the data on that feature was useless!

Instructions

Checkpoint 1 Enabled

1.

Verify splitting on a pure node will result in an information gain of zero. For example, with the right-hand-side split of the root node, `safety_low <= 0.5` is false, consider a further split of this node.

The result will be each subsequent split will still have Gini impurity of zero. Fill this in the code editor and uncomment the relevant lines to calculate the gini information gain.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Calculate the information gain on the next split, on `persons_2` in the tree but reading the following:

The number of samples of the tree to input the split ratio (`r_persons_2`)

The initial gini value at the parent node `persons_2(initial_gini_persons_2)`

The gini values at the child nodes (`gini_left_node` and `gini_right_node`) Input these, uncomment the relevant lines of code and press Run to obtain the information gain at this split.

```
#1. Information gain at a pure node (i.e., node with no more branches!)
r = 0.5 # ratio of new split, could be anything
gini_pure_node = 0
gini_info_gain = r*gini_pure_node + (1-r)*gini_pure_node
print(f'Gini information gain pure node split safety_low >= .5 : {gini_info_gain}')

#2. Information gain at the 'persons_2' split
r_persons_2 = 604/912 # read ratio of the split from the tree!
gini_left_split = 0.434
gini_right_split = 0
initial_gini_persons_2 = 0.495

weighted_gini_persons_2 = r_persons_2*gini_left_split + (1-r_persons_2)*gini_right_split
gini_info_gain_persons_2 = initial_gini_persons_2 - weighted_gini_persons_2

print(f'Gini information gain node persons 2 : {gini_info_gain_persons_2}')
```

How a Decision Tree is Built (Feature Split)

We're ready to understand how the decision tree was built by `scikit-learn`. To recap:

The root node in the tree we've been using so far is split on the feature `safety_low`. When its value is 1, this corresponds to the right split (vehicles *with* low safety) and when its value is 0, this corresponds to the left split.

Information gain is the difference in the weighted gini impurity before and after performing a split at a node. We saw in the previous exercise that information gain for `safety_low` as root node was $0.4185 - 0.3267 = 0.0918$.

We now consider an important question: **How does one know that this is the best node to split on?!** To figure this out we're going to go through the process of calculating information gain for other possible root node choices and calculate the information gain values for each of these. This is precisely what is going on under the hood when one runs a `DecisionTreeClassifier()` in `scikit-learn`. By checking information gain values of all possible options at any given split, the algorithm decide on the best feature to split on at every node.

For starters, we will consider a different feature we could have split on first: `persons_2`. Recall that `persons_2` can take a binary value of 0 or 1 as well. Setting `persons_2` as the root node means that:

the left split will contain data corresponding to a `persons_2` value <0.5 .
the right split will contain data corresponding to a `persons_2` value >0.5 .

We've defined two functions in the code editor to help us calculate gini impurity (`gini`) and information gain (`info_gain`) in the code editor. Read through the functions to see if they reflect the formulas we've covered thusfar. Using these functions, we're going to calculate the information gain from splitting on `persons_2`. We can then follow this procedure for all other possible root node choices and truly check if `safety_low` is indeed our best choice for this!

Instructions

Checkpoint 1 Enabled

1.

Create two DataFrames `left` and `right` that represent the `y_train` values that correspond to `x_train['persons_2']` being 0 and 1 respectively. Calculate the length of these DataFrames, store them as `len_left` and `len_right`, and print them.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We're now going to calculate the gini impurities corresponding to the overall training data and the left and right split. To do so:

Uncomment the line pertaining to calculating the gini impurity in the overall training data. For the gini value of the left node, create a variable `gini_left` and use the `gini` function to calculate the value.

For the gini value of the right node, create a variable `gini_right` and use the `gini` function to calculate the value.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Before proceeding to calculate the information gain at this split, let's consolidate what we've calculated in the previous checkpoints:

There are 917 cars with a `persons_2` value of 0 and 465 cars with `persons_2` value of 1.

The overall gini impurity of the training data is 0.4185. The gini impurity for the left split was 0.4949 and the gini impurity of the right split is 0.

This means that the *weighted impurity* of this split is:

```
917/1382 (0.4949) + 465/1382 (0) = 0.3284
```

Copy to Clipboard

The information gain for tree whose root node is `persons_2` should be

```
0.4185 - 0.3284 = 0.0901
```

Copy to Clipboard

Use the `info_gain` function to calculate the information gain corresponding to this split and store it as `info_gain_persons_2`. Print it to check if it is indeed the expected value!

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We've now verified that splitting at a root node of `persons_2` gives us a lesser information gain than splitting at `safety_low` (0.0901 in comparison to 0.0918!). Verify the information gain is the highest at the root node using the function `info_gain` and looping through ALL the features. Uncomment the lines that pertain to this calculation to verify if the tree we've been working with so far has the best possible root node!

```
## The usual libraries, loading the dataset and performing the train-test split
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.2)

## Functions to calculate gini impurity and information gain

def gini(data):
    """calculate the Gini Impurity
    """
    data = pd.Series(data)
    return 1 - sum(data.value_counts(normalize=True)**2)

def info_gain(left, right, current_impurity):
    """Information Gain associated with creating a node/split data.
    Input: left, right are data in left branch, right branch, respectively
    current_impurity is the data impurity before splitting into left, right branches
```

```

"""
# weight for gini score of the left branch
w = float(len(left)) / (len(left) + len(right))
return current_impurity - w * gini(left) - (1 - w) * gini(right)

#### -----
## 1. Calculate sample sizes for a split on `persons_2`

left = y_train[x_train['persons_2']==0]
right = y_train[x_train['persons_2']==1]
len_left = len(left)
len_right = len(right)
print('No. of cars with persons_2 == 0:', len_left)
print('No. of cars with persons_2 == 1:', len_right)

## 2. Calculate gini impurities
gi = gini(y_train)
gini_left = gini(left)
gini_right = gini(right)

print('Left split gini impurity:', gini_left)
print('Right split gini impurity:', gini_right)

## 3. Information gain when using feature `persons_2`
info_gain_persons_2 = info_gain(left, right, gi)
print(f'Information gain for persons_2:', info_gain_persons_2)

## 4. Which feature split maximizes information gain?
info_gain_list = []
for i in x_train.columns:
    left = y_train[x_train[i]==0]
    right = y_train[x_train[i]==1]
    info_gain_list.append([i, info_gain(left, right, gi)])

info_gain_table = pd.DataFrame(info_gain_list).sort_values(1, ascending=False)
print(info_gain_table.iloc[0,:][0])
print(f'Greatest impurity gain at:{info_gain_table.iloc[0,:]}')
print(info_gain_table)

```

How a Decision Tree is Built (Recursion)

Now that we can find the best feature to split the dataset, we can repeat this process again and again to create the full tree. This is a recursive algorithm! We start with every data point from the training set, find the best feature to split the data, split the data based on that feature, and then recursively repeat the process again on each subset that was created from the split.

We'll stop the recursion when we can no longer find a feature that results in any information gain. In other words, we want to create a leaf of the tree when we can't find a way to split the data that makes purer subsets.

Instructions

Checkpoint 1 Enabled

1.

Update the variables `y_train_sub` and `x_train_sub` with a logical filter to represent the data at the left node after the root node split. Note, this will include only data that has been split at the root node to the left, i.e. `'safety_low<=.5'`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the previously defined functions `gini` and loop for `info_gain_list` to verify the next split after `'safety_low<=.5'` will be on `person_2`.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

def gini(data):
    """calculate the Gini Impurity
    """
    data = pd.Series(data)
    return 1 - sum(data.value_counts(normalize=True)**2)

def info_gain(left, right, current_impurity):
    """Information Gain associated with creating a node/split data.
    Input: left, right are data in left branch, right branch, respectively
    current_impurity is the data impurity before splitting into left, right branches
    """
    # weight for gini score of the left branch
    w = float(len(left)) / (len(left) + len(right))
    return current_impurity - w * gini(left) - (1 - w) * gini(right)

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.2)

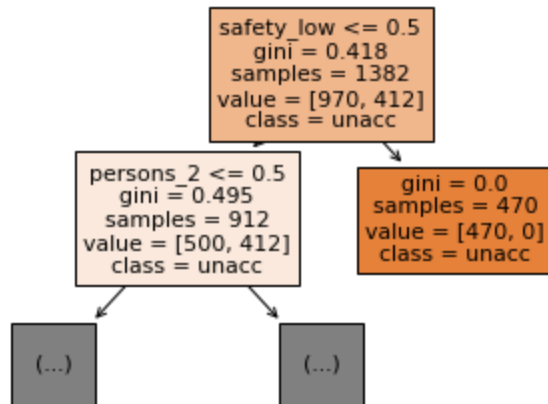
y_train_sub = y_train[x_train['safety_low']==0]
x_train_sub = x_train[x_train['safety_low']==0]

gi = gini(y_train_sub)
print(f'Gini impurity at root: {gi}')

info_gain_list = []
for i in x_train.columns:
    left = y_train_sub[x_train_sub[i]==0]
    right = y_train_sub[x_train_sub[i]==1]
    info_gain_list.append([i, info_gain(left, right, gi)])

info_gain_table = pd.DataFrame(info_gain_list).sort_values(1,ascending=False)
print(f'Greatest impurity gain at:{info_gain_table.iloc[0,:]}')

```



Train and Predict using `scikit-learn`

Now we'll finally build a decision tree ourselves! We will use `scikit-learn`'s tree module to create, train, predict, and visualize a decision tree classifier. The syntax is the same as other models in `scikit-learn`, so it should look very familiar. First, an instance of the model class is instantiated with `DecisionTreeClassifier()`. To use non-default hyperparameter values, you can pass them at this stage, such as `DecisionTreeClassifier(max_depth=5)`.

Then `.fit()` takes a list of data points followed by a list of the labels associated with that data and builds the decision tree model.

Finally, once we've made our tree, we can use it to classify new data points. The `.predict()`

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

takes an

Preview: Docs Stores elements of various data types in an ordered collection.

[array](#)

of data points and will return an array of classifications for those data points. `predict_proba()` can also be used to return class probabilities instead. Last, `.score()` can be used to generate the accuracy score for a new set of data and labels.

As with other sklearn models, only numeric data can be used (categorical

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

[variables](#)

and nulls must be handled prior to model fitting).

Instructions

Checkpoint 1 Enabled

1.

Create a decision tree classification model defined as `dtree` with default parameters. Print the parameters of the tree using `.get_params()` method.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Fit `dtree` using the training data set and labels. After the tree is fit, print the depth of the tree using `.get_depth()`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Predict the classes of the test data set (`x_test`) and save this as an array `y_pred`. Print the accuracy of model on the test set (either using `.score()` or `accuracy_score()`).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.metrics import accuracy_score

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.2)

## 1. Create a decision tree and print the parameters
dtree = DecisionTreeClassifier()
print(f'Decision Tree parameters: {dtree.get_params()}')

## 2. Fit decision tree on training set and print the depth of the tree
dtree.fit(x_train, y_train)
print(f'Decision tree depth: {dtree.get_depth()}')

## 3. Predict on test data and accuracy of model on test set
y_pred = dtree.predict(x_test)

print(f'Test set accuracy: {dtree.score(x_test, y_test)}') # or accuracy_score(y_test,
y_pred)
```

Visualizing Decision Trees

Great, we built a decision tree using `scikit-learn` and predicted new values with it! But what does the tree look like? What features are used to split? Two methods using only `scikit-learn/matplotlib` can help visualize the tree, the first using `tree.plot`, the second listing the rules as text. There are other libraries available with more advanced visualization (`graphviz` and `dtreeviz`, for example, but may require additional installation and won't be covered here).

Instructions

Checkpoint 1 Enabled

1.

Press Run on the existing code and inspect the current output of `tree.plot_tree()` (Use the arrows in the browser to zoom out for a closer view!). What seems to be missing? The features are labeled only by index rather than name. Pass the additional parameters `feature_names = x_train.columns`, `class_names = ['unacc','acc']`, `filled=True` to the plotting function and note the differences.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Maybe you want a more simplistic method of visualizing the tree in a text-based fashion. This can be done by printing the result of `tree.export_text()`. Fix the same issue as above by passing the parameter `feature_names` with a list of the columns to print out the names rather than index.

```
import codecademylib3
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

## Loading the data and setting target and predictor variables
```

```

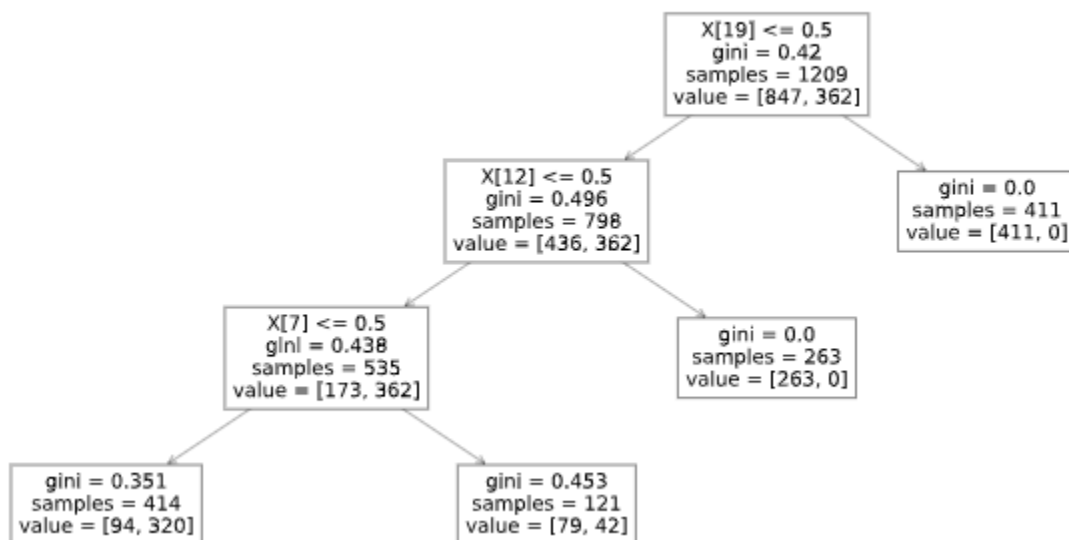
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']

## Train-test split and fitting the tree
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.3)
dtree = DecisionTreeClassifier(max_depth=3)
dtree.fit(x_train, y_train)

## Visualizing the tree
plt.figure(figsize=(27,12))
tree.plot_tree(dtree)
plt.tight_layout()
plt.show()

## Text-based visualization of the tree (View this in the Output terminal!)
print(tree.export_text(dtree))

```



Advantages and Disadvantages

As we have seen already, decision trees are easy to understand, fully explainable, and have a natural way to visualize the decision making process. In addition, often little modification needs to be made to the data prior to modeling (such as scaling, normalization, removing outliers) and decision trees are relatively quick to train and predict. However, now let's talk about some of their limitations.

One problem with the way we're currently making our decision trees is that our trees aren't always *globally optimal*. This means that there might be a better tree out there somewhere that produces better results. But wait, why did we go through all that work of finding information gain if it's not producing the best possible tree?

Our current strategy of creating trees is *greedy*. We assume that the best way to create a tree is to find the feature that will result in the largest information gain *right now* and split on that feature. We never consider the ramifications

of that split further down the tree. It's possible that if we split on a suboptimal feature right now, we would find even better splits later on. Unfortunately, finding a globally optimal tree is an extremely difficult task, and finding a tree using our greedy approach is a reasonable substitute.

Another problem with our trees is that they are prone to *overfit* the data. This means that the structure of the tree is too dependent on the training data and may not generalize well to new data. In general, larger trees tend to overfit the data more. As the tree gets bigger, it becomes more tuned to the training data and it loses a more generalized understanding of the real world data.

Instructions

Checkpoint 1 Enabled

1.

Fit two decision trees to the dataset we've been working with:

dtree1 with no restriction on the `max_depth`
dtree2 with a `max_depth` of 7, on the training data

Print the `max_depth` of each tree.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Evaluate the accuracy of each tree on the test set.

Does the tree with the greater depth or the tree with the lower depth have a higher accuracy?

Remember, a larger depth does not always mean better accuracy, as it can lead to overfitting.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6])
y = df['accep']

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.3)

## 1. Two decision trees
dtree1 = DecisionTreeClassifier()
dtree2 = DecisionTreeClassifier(max_depth = 7)

## Fit first decision tree
dtree1.fit(x_train, y_train)
dtree1_depth = dtree1.get_depth()
print(f'First Decision Tree depth: {dtree1_depth}')

## Fit second decision tree
dtree2.fit(x_train, y_train)
dtree2_depth = dtree2.get_depth()
print(f'Second Decision Tree depth: {dtree2_depth}')

## 2. Calculate accuracy scores on test data for both trees

dtree1_score = dtree1.score(x_test, y_test)
print(f'Test set accuracy tree no max depth: {dtree1_score}')# or accuracy_score(y_test,
y_pred)

dtree2_score = dtree2.score(x_test, y_test)
print(f'Test set accuracy tree max depth 7: {dtree2_score}')# or accuracy_score(y_test,
y_pred)
```

Decision Trees for Classification and Regression

Learn about decision trees, how they work and how they can be used for classification and regression tasks.

Introduction

Decision trees are a common type of machine learning model used for binary classification tasks. The natural structure of a binary tree lends itself well to predicting a “yes” or “no” target. It is traversed sequentially here by evaluating the truth of each logical statement until the final prediction outcome is reached. Some examples of classification tasks that can use decision trees are: predicting whether a student will pass or fail an exam, whether an email is spam or not, if transaction is fraudulent or legitimate, etc

Decision trees can also be used for regression tasks! Predicting the grade of a student on an exam, the number of spam emails per day, the amount of fraudulent transactions on a platform, etc. are all possible using decision trees. The algorithm works pretty much the same way, with modifications only to the splitting criteria and how the final output it computed. In this article, we will explore both a binary classification and regression model using decision trees with the [Indian Graduate Admissions dataset](#).

Dataset

The data contains features commonly used in determining admission to masters’ degree programs, such as GRE, GPA, and letters of recommendation. The complete list of features is summarized below:

- GRE Scores (out of 340)
- TOEFL Scores (out of 120)
- University Rating (out of 5)
- Statement of Purpose and Letter of Recommendation Strength (out of 5)
- Undergraduate GPA (out of 10)
- Research Experience (either 0 or 1)
- Chance of Admit (ranging from 0 to 1)

We’re going to begin by loading the dataset as a `pandas DataFrame`. Feel free to open up a `jupyter` notebook on the side to implement the code in the article!

```
import pandas as pd
df = pd.read_csv("Admission_Predict.csv")
df.columns = df.columns.str.strip().str.replace(' ','_').str.lower()
```

Decision Trees for Classification: A Recap

As a first step, we will create a binary class (1=admission likely, 0=admission unlikely) from the `chance_of_admit` – greater than 80% we will consider as likely. The remaining data columns will be used as predictors.

```
X = df.loc[:, 'gre_score': 'research']
y = df['chance_of_admit'] >= .8
```

Fitting and Predicting

We will use `scikit-learn`'s tree module to create, train, predict, and visualize a decision tree classifier. The syntax is the same as other models in `scikit-learn`, once an instance of the model class is instantiated with `dt = DecisionTreeClassifier()`, `.fit()` can be used to fit the model on the training set. After fitting, `.predict()` (and `.predict_proba()`) and `.score()` can be called to generate predictions and score the model on the test data.

As with other `scikit-learn` models, only numeric data can be used (categorical variables and nulls must be handled prior to model fitting). In this case, our categorical features have already been transformed and no missing values are present in the data set.

```
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0,
test_size=0.2)
dt = DecisionTreeClassifier(max_depth=2, ccp_alpha=0.01, criterion='gini')
dt.fit(x_train, y_train)
y_pred = dt.predict(x_test)
print(dt.score(x_test, y_test))
print(accuracy_score(y_test, y_pred))
```

Output:

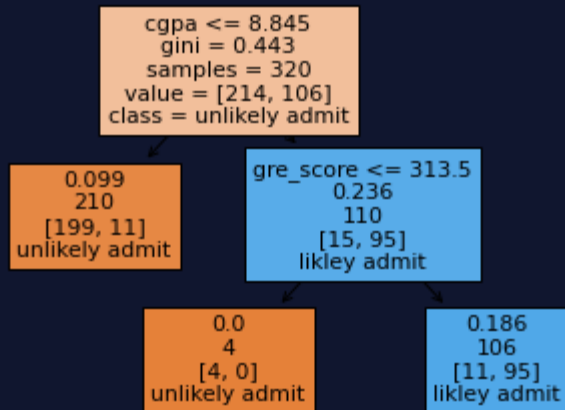
```
0.925
0.925
```

Two methods are available to visualize the tree within the tree module – the first is using `tree.plot` to graphically represent the decision tree. The second uses `export_text` to list the rules behind the splits in the decision tree. There are many other packages available for more visualization options – such as `graphviz`, but may require additional installations and will not be covered here.

```
tree.plot_tree(dt, feature_names = x_train.columns,
               max_depth=3, class_names = ['unlikely admit', 'likely admit'],
               label='root', filled=True)
print(tree.export_text(dt, feature_names = X.columns.tolist()))
```

Output:

```
|--- cgpa <= 8.85
|   |--- class: False
|--- cgpa > 8.85
|   |--- gre_score <= 313.50
|   |   |--- class: False
|   |   |--- gre_score > 313.50
|   |       |--- class: True
```



Output:

Split Criteria

For a classification task, the default split criteria is Gini impurity – this gives us a measure of how “impure” the groups are. At the root node, the first split is then chosen as the one that maximizes the information gain, i.e. decreases the Gini impurity the most. Our tree has already been built for us, but how was the split `cgpa<=8.845` determined? `cgpa` is a continuous variable, which adds an extra complication, as the split can occur for ANY value of `cgpa`.

To verify, we will use the defined functions `gini` and `info_gain`. By running `gini(y_train)`, we get the same Gini impurity value as printed in the tree at the root node, `0.443`.

```
def gini(data):
    """Calculate the Gini Impurity Score
    """
    data = pd.Series(data)
    return 1 - sum(data.value_counts(normalize=True)**2)

gi = gini(y_train)
print(f'Gini impurity at root: {round(gi,3)}')
```

Output:

Gini impurity at root: 0.443

Next, we are going to verify how the split on `cgpa` was determined, i.e. where did the `8.845` value come from:

```
def info_gain(left, right, current_impurity):
    """Information Gain associated with creating a node/split data.
    Input: left, right are data in left branch, right branch, respectively
    current_impurity is the data impurity before splitting into left, right branches
    """
    # weight for gini score of the left branch
    w = float(len(left)) / (len(left) + len(right))
    return current_impurity - w * gini(left) - (1 - w) * gini(right)
```

We will use `info_gain` over ALL values of `cgpa` to determine the information gain when split on each value. This is stored in a table and sorted, and voila, the top value for the split is `cgpa<=8.845`! This is also done for every other feature (and for those continuous ones, every value), to find the top split overall.

```
info_gain_list = []
for i in x_train.cgpa.unique():
    left = y_train[x_train.cgpa<=i]
    right = y_train[x_train.cgpa>i]
    info_gain_list.append([i, info_gain(left, right, gi)])

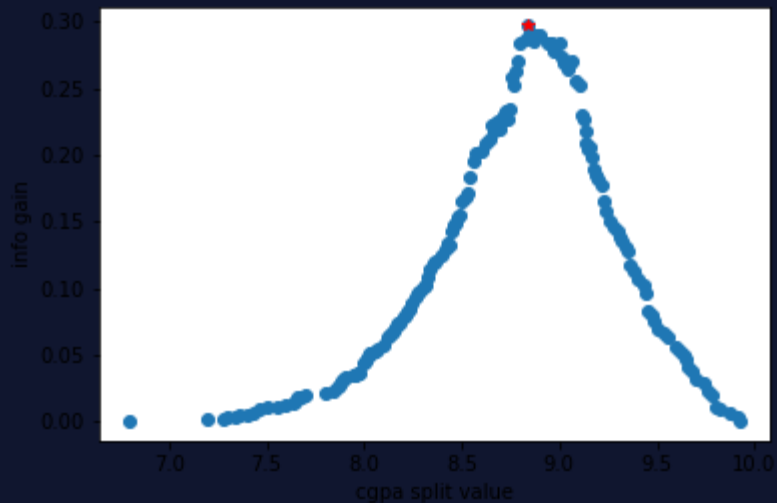
ig_table = pd.DataFrame(info_gain_list, columns=['split_value',
'info_gain']).sort_values('info_gain',ascending=False)
ig_table.head(10)
```

Output:

	split_value	info_gain
10	8.84	0.296932
124	8.85	0.291464
139	8.88	0.290704
18	8.90	0.290054
98	8.83	0.287810
110	8.87	0.286050
152	8.94	0.284714
57	8.96	0.284210
96	8.80	0.283371
21	9.00	0.283364

Plot the results:

```
plt.plot(ig_table['split_value'], ig_table['info_gain'],'o')
plt.plot(ig_table['split_value'].iloc[0], ig_table['info_gain'].iloc[0], 'r*')
plt.xlabel('cgpa split value')
plt.ylabel('info gain')
```



Output:

After this process is repeated, and there is no further info gain by splitting, the tree is finally built. Last to evaluate, any sample traverses through tree and appropriate splits until it reaches a leaf node, and then assigned the majority class of that leaf (or weighted majority).

Regression

For the regression problem, we will use the unaltered `chance_of_admit` target, which is a floating point value between 0 and 1.

```
X = df.loc[:, 'gre_score': 'research']
y = df['chance_of_admit']
```

Fitting and Predicting

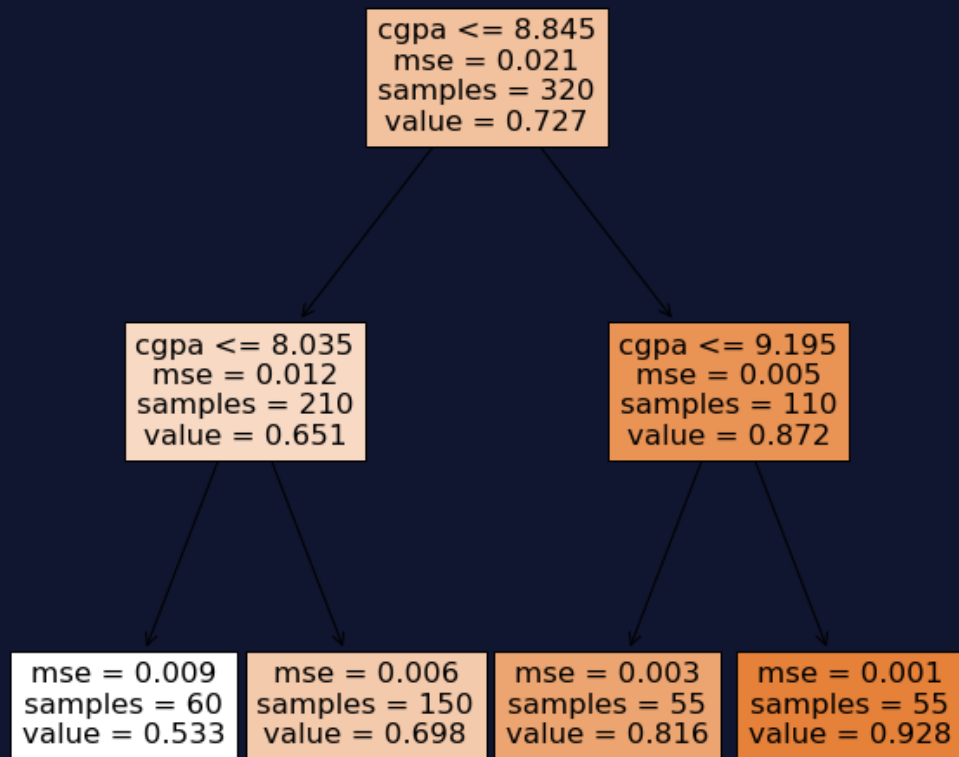
The syntax is identical as the decision tree classifier, except the target, `y`, must be real-valued and the model used must be `DecisionTreeRegressor()`. As far as the model hyperparameters go, almost all are the same, except for the split criterion. The split criterion now needs to be suitable for a regression task – the default for regression is Mean Squared Error (or MSE). Let's investigate this:

```
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0,
test_size=0.2)
dt = DecisionTreeRegressor(max_depth=3, ccp_alpha=0.001)
dt.fit(x_train, y_train)
y_pred = dt.predict(x_test)
print(dt.score(x_test, y_test))
```

Similarly, the tree can be visualized using `tree.plot_tree` – keeping in mind the splitting criteria is `mse` and the `value` is the average `chance_of_admit` of all samples in that leaf.

```
plt.figure(figsize=(10,10))
tree.plot_tree(dt, feature_names = x_train.columns,
               max_depth=2, filled=True);
```

Output:



Split Criteria

Unlike the classification problem, there are no longer classes to split the tree by. Instead, at each level, the value is the average of all samples that fit the logical criteria. In terms of evaluating the split, the default method is MSE. For example, the root node, the average target value is 0.727 (verify `y_train.mean()`). Then the MSE (mean-squared error) if we were to use 0.727 as the value for all samples, would be:

```
np.mean((y_train - y_train.mean())**2) = 0.02029
```

Now to determine the split, for each value of `cgpa`, the information gain, or decrease in MSE after the split, is calculated and then values are sorted. Like before, we can modify our functions for the regression version, and see the best split is again `cgpa <= 8.84`.

The below code walks you through the details – in the regression version, instead of Gini impurity, MSE is used, and the information gain function is modified to `mse_gain`.

```
def mse(data):
    """Calculate the MSE of a data set
    """
    return np.mean((data - data.mean())**2)

def mse_gain(left, right, current_mse):
    """Information Gain (MSE) associated with creating a node/split data based on MSE.
    Input: left, right are data in left branch, right branch, respectively
    current_impurity is the data impurity before splitting into left, right branches
    """
    # weight for gini score of the left branch
    w = float(len(left)) / (len(left) + len(right))
    return current_mse - w * mse(left) - (1 - w) * mse(right)

m = mse(y_train)
print(f'MSE at root: {round(m,3)}')

mse_gain_list = []
for i in x_train.cgpa.unique():
    left = y_train[x_train.cgpa<=i]
    right = y_train[x_train.cgpa>i]
    mse_gain_list.append([i, mse_gain(left, right, m)])

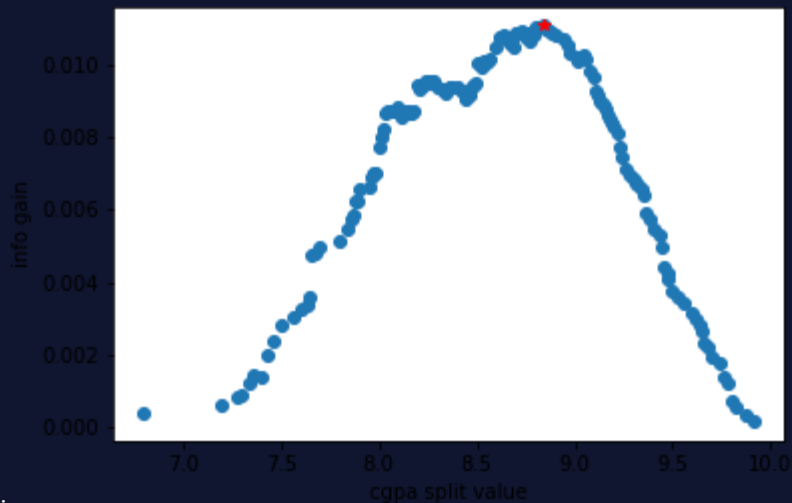
mse_table = pd.DataFrame(mse_gain_list, columns=['split_value',
'info_gain']).sort_values('info_gain', ascending=False)
print(mse_table.head(10))
```

Output:

```
MSE at root: 0.021
   split_value  info_gain
10         8.84   0.011065
96         8.80   0.011037
98         8.83   0.011023
124        8.85   0.010985
125        8.73   0.010939
110        8.87   0.010932
139        8.88   0.010895
1         8.70   0.010894
17         8.76   0.010858
140        8.74   0.010850
```

Plot the results:

```
plt.plot(mse_table['split_value'], mse_table['info_gain'], 'o')
plt.plot(mse_table['split_value'].iloc[0], mse_table['info_gain'].iloc[0], 'r*')
plt.xlabel('cgpa split value')
plt.ylabel('info gain')
```

Output:

Again, the process will continue until there is no increase in information gain by splitting. Now that the tree has been built, evaluation occurs in pretty much the same way. Any sample traverses through the tree until it reaches a leaf node and is then assigned the average value of the samples in leaf. Depending on the depth of the tree, *the predicted values can be limited*. In this example, only four unique predicted values are possible, which we can verify. This is something to be aware of when using a decision tree regressor, unlike linear/logistic regression, not all output values may be possible.

```
np.unique(dt.predict(x_train))
```

Output:

```
array([0.          , 0.00588235, 0.28571429, 0.32         , 0.63636364,
        0.96428571])
```

Summary

We've seen how decision trees can be used for both classification and regression tasks.

- The fundamental difference is that for classification, splits are based on Gini impurity error calculations whereas for regression, Mean Squared Error minimization is used.
- Tree traversal based on information gain and evaluation works pretty much the same way for both tasks.
- Decision tree regressors differ from other regressors in that all output values may not be possible and it depends on the depth of the tree.

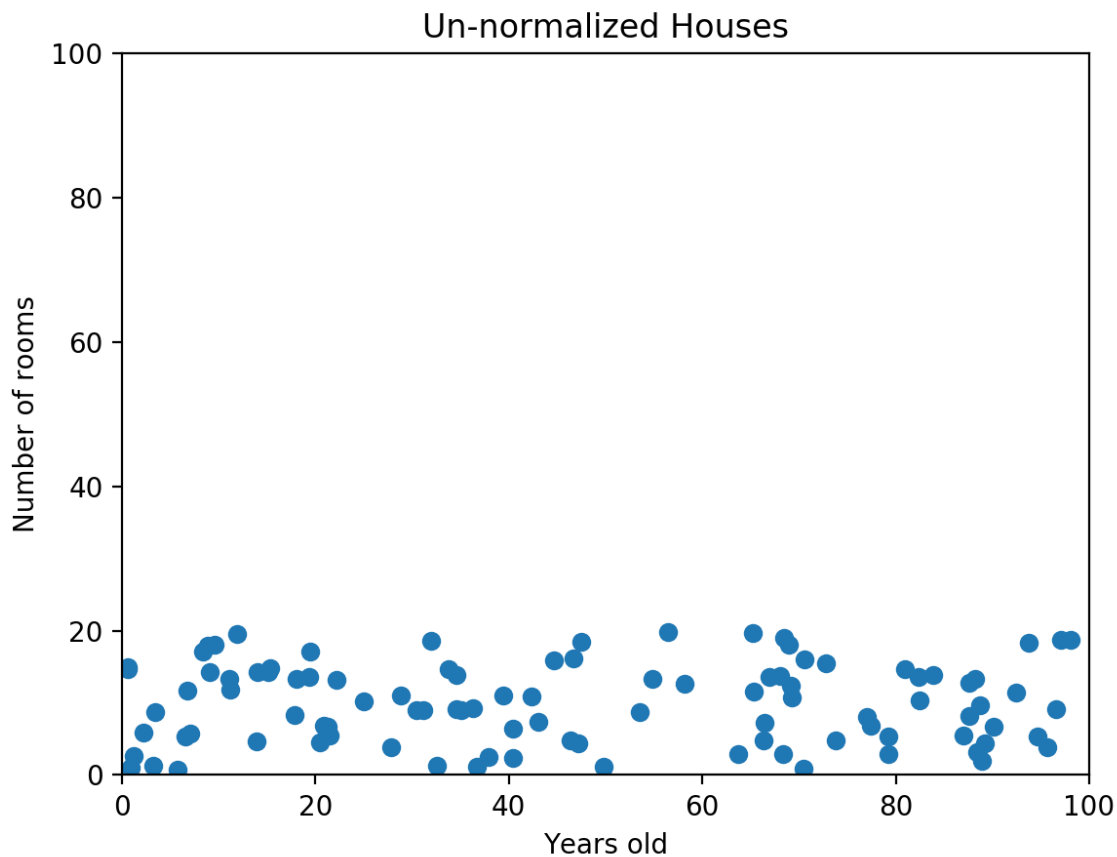
Normalization

This article describes why normalization is necessary. It also demonstrates the pros and cons of min-max normalization and z-score normalization.

Why Normalize?

Many machine learning algorithms attempt to find trends in the data by comparing features of data points. However, there is an issue when the features are on drastically different scales.

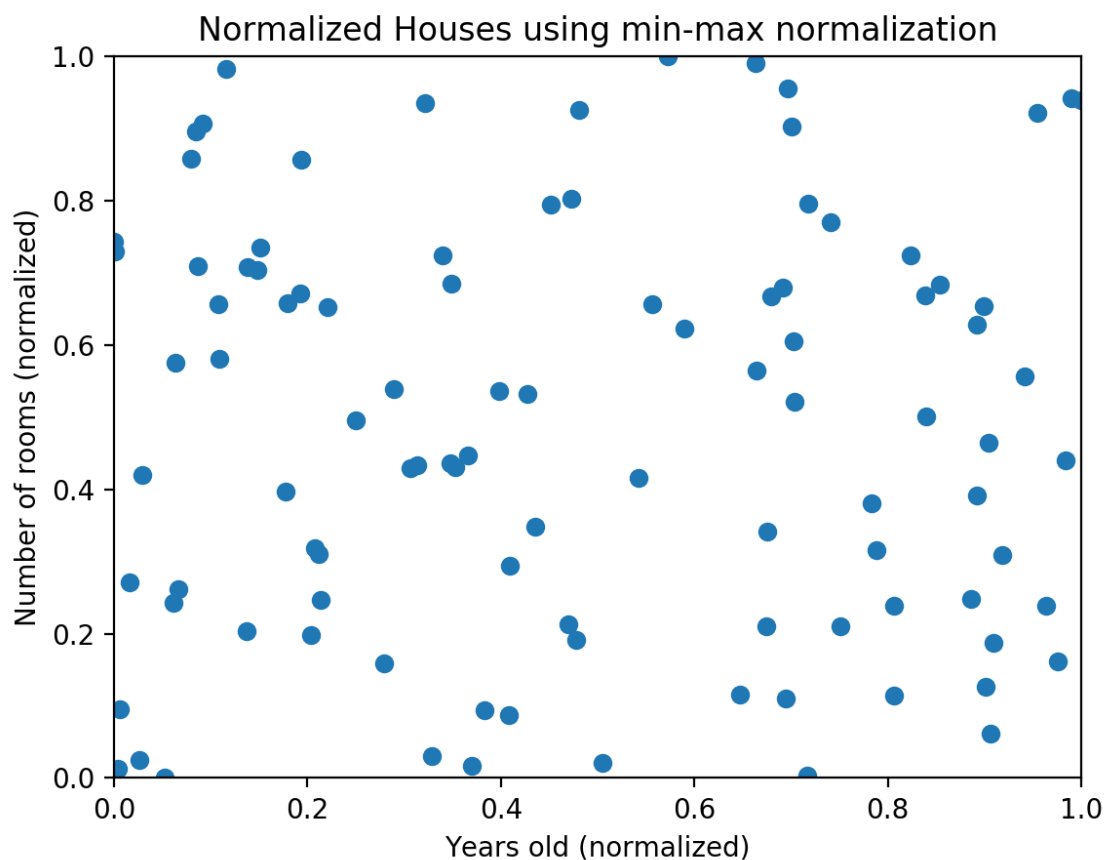
For example, consider a dataset of houses. Two potential features might be the number of rooms in the house, and the total age of the house in years. A machine learning algorithm could try to predict which house would be best for you. However, when the algorithm compares data points, the feature with the larger scale will completely dominate the other. Take a look at the image below:



When the data looks squished like that, we know we have a problem. The machine learning algorithm should realize that there is a huge difference between a house with 2 rooms and a house with 20 rooms. But right now, because two houses can be 100 years apart, the difference in the number of rooms contributes less to the overall difference.

As a more extreme example, imagine what the graph would look like if the x-axis was the cost of the house. The data would look even more squished; the difference in the number of rooms would be even less relevant because the cost of two houses could have a difference of thousands of dollars.

The goal of normalization is to make every datapoint have the same scale so each feature is equally important. The image below shows the same house data normalized using min-max normalization.



Min-Max Normalization

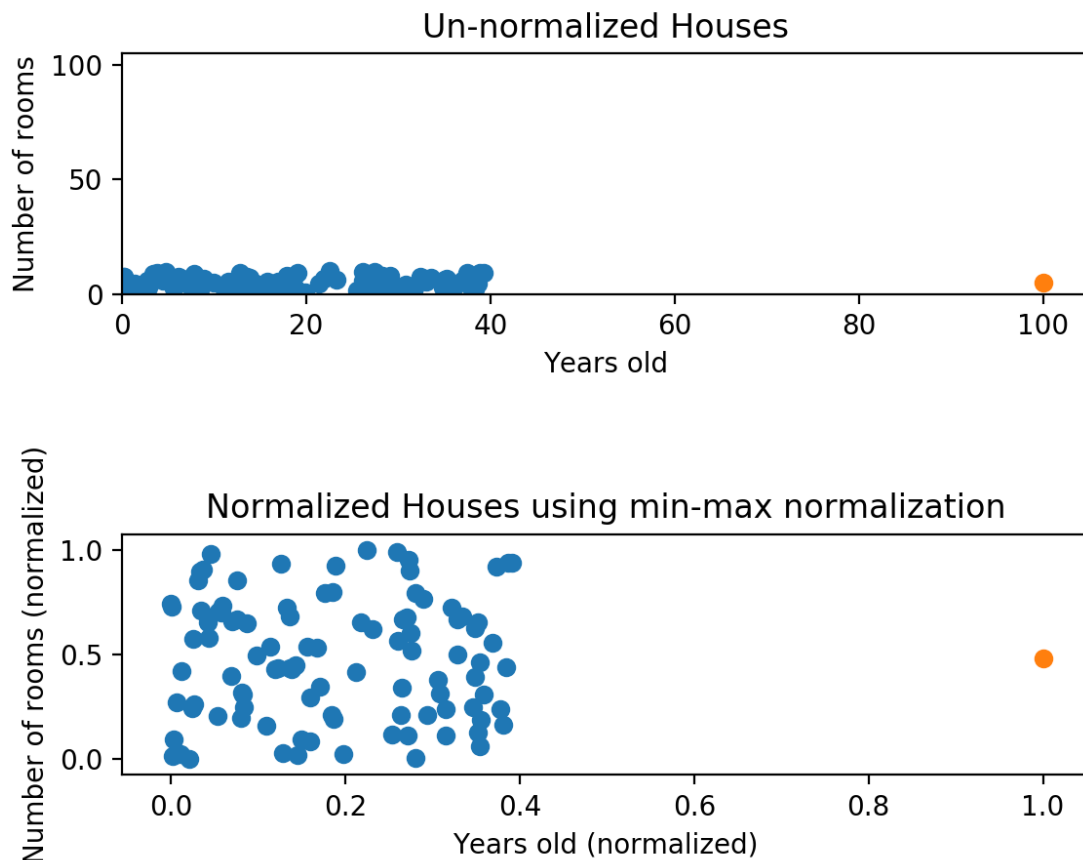
Min-max normalization is one of the most common ways to normalize data. For every feature, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.

For example, if the minimum value of a feature was 20, and the maximum value was 40, then 30 would be transformed to about 0.5 since it is halfway between 20 and 40. The formula is as follows:

$$\frac{\text{value} - \text{min}}{\text{max} - \text{min}}$$

$$\frac{\text{value} - \text{min}}{\text{max} - \text{min}}$$

Min-max normalization has one fairly significant downside: it does not handle outliers very well. For example, if you have 99 values between 0 and 40, and one value is 100, then the 99 values will all be transformed to a value between 0 and 0.4. That data is just as squished as before! Take a look at the image below to see an example of this.



Normalizing fixed the squishing problem on the y-axis, but the x-axis is still problematic. Now if we were to compare these points, the y-axis would dominate; the y-axis can differ by 1, but the x-axis can only differ by 0.4.

Z-Score Normalization

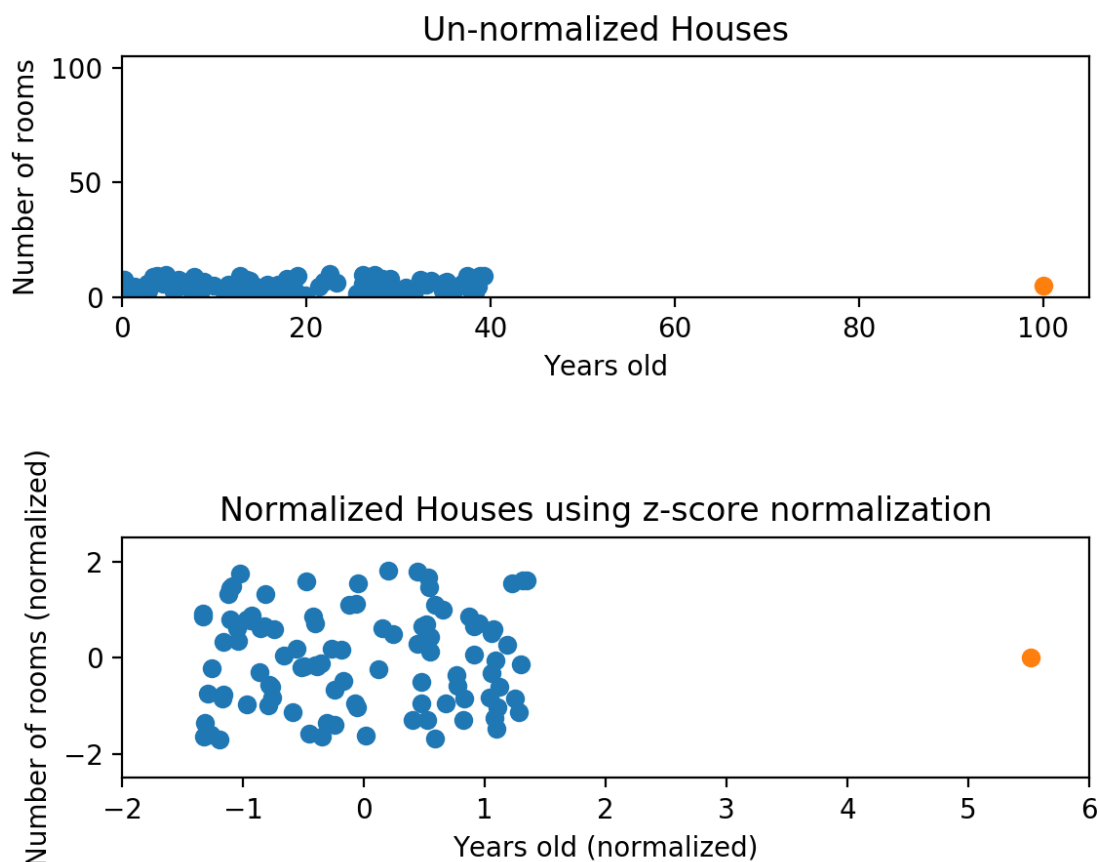
Z-score normalization is a strategy of normalizing data that avoids this outlier issue. The formula for Z-score normalization is below:

$$\frac{\text{value} - \mu}{\sigma}$$

$$\frac{\text{value} - \mu}{\sigma}$$

Here, μ is the mean value of the feature and σ is the standard deviation of the feature. If a value is exactly equal to the mean of all the values of the feature, it will be normalized to 0. If it is below the mean, it will be a negative number, and if it is above the mean it will be a positive number. The size of those negative and positive numbers is determined by the standard deviation of the original feature. If the unnormalized data had a large standard deviation, the normalized values will be closer to 0.

Take a look at the graph below. This is the same data as before, but this time we're using z-score normalization.



While the data still *looks* squished, notice that the points are now on roughly the same scale for both features — almost all points are between -2 and 2 on both the x-axis and y-axis. The only potential downside is that the features aren't on the *exact* same scale.

With min-max normalization, we were guaranteed to reshape both of our features to be between 0 and 1. Using z-score normalization, the x-axis now has a range from about -1.5 to 1.5 while the y-axis has a range from about -2 to 2. This is certainly better than before; the x-axis, which previously had a range of 0 to 40, is no longer dominating the y-axis.

Review

Normalizing your data is an essential part of machine learning. You might have an amazing dataset with many great features, but if you forget to normalize, one of those features might completely dominate the others. It's like you're throwing away almost all of your information! Normalizing solves this problem. In this article, you learned the following techniques to normalize:

Min-max normalization: Guarantees all features will have the exact same scale but does not handle outliers well.

Z-score normalization: Handles outliers, but does not produce normalized data with the exact same scale.

Training, Validation and Test Datasets

Training, Validation and Test Datasets

This article teaches the importance of splitting a dataset into training, validation and test sets.

Supervised machine learning models have the potential to learn patterns from labeled datasets and make predictions. But how do we know if the predictions our models are making are even useful? After all, it is possible that every prediction our email spam classifier makes is actually wrong, or that every housing price prediction our regression model makes is orders of magnitude off!

Luckily, we can leverage the fact that supervised machine learning models require a dataset of pre-labeled observations to help us determine the performance of our model. To do so, we will have to split our labeled dataset into three chunks – a training, validation and test (or holdout) set.

Training-Validation-Test Split

Before we even begin to think about writing the code we will use to fit our machine learning model to the data, we must perform the split of our dataset to create our training set, validation set, and test set.

The *training set* is the data that the model will learn how to make predictions from. Learning looks different depending on which algorithm we are using. With Linear Regression, the observations in the training set are used to fit the parameters of a line of best fit. For choose different model (classification), the observations in the training set are used to fit the parameters being fit.

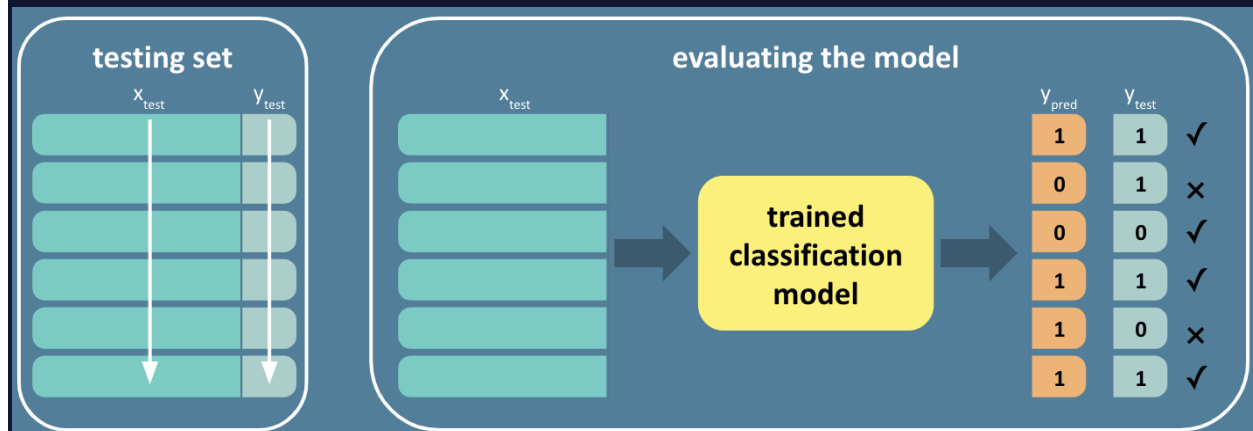
The *validation set* is the data that will be used during the training phase to evaluate the interim performance of the model, guide the tuning of hyper-parameters, and assist in other model improvement capacities (for example, feature selection). Some common metrics used to calculate the performance of machine learning models are [accuracy, recall, precision, and F1-Score](#). The metric we choose to use will vary depending on our particular use case.

The *test set* is the data that will determine the performance of our final model so we can estimate how our model will fare in the real world. To avoid introducing any bias to the final measurements of performance, we do not want the test set anywhere near the model training or tuning processes. That is why the test set is often referred to as the holdout set.

Evaluating the Model

During model fitting, both the features (X) and the true labels (y) of the training set (Xtrain, ytrain) are used to learn. When evaluating the performance of the model with the validation (Xval, yval) or test (Xtest,

ytest) set, we are going to temporarily pretend like we do not know the true label of every observation. If we use the observation features in our validation (X_{val}) or test (X_{test}) sets as inputs to the trained model, we can receive a prediction as output for each observation (y_{pred}). We can now compare each of the true labels (y_{val} or y_{test}) with each of the predicted labels (y_{pred}) and get a quantitative evaluation on the performance of the model.



How to Split

Figuring out how much of our data should be split into training, validation, and test sets is a tricky question and there is no simple answer. If our training set is too small, then the model might not have enough data to effectively learn. On the other hand, if our validation and test sets are too small, then the performance metrics could have a large variance. In general, putting 70% of the data into the training set and 15% of the data into each of the validation and test sets is a good place to start.

N-Fold Cross-Validation

Sometimes our dataset is so small that splitting it into training, validation, and test sets that are appropriate sizes is unfeasible. A potential solution is to perform N-Fold Cross-Validation. While we still first split the dataset into a training and test set, we are going to further split the training set into N chunks. In each iteration (or fold), N-1 of the chunks are treated as the training set and 1 of the chunks is treated as the validation set over which the evaluation metrics are calculated.

This process is repeated N times cycling through each chunk acting as the validation set and the evaluation metrics from each fold are averaged. For example, in 10-fold cross-validation, we'll make the validation set the first 10% of the training set and calculate our evaluation metrics. We'll then make the validation set the second 10% of the data and calculate these statistics once again. We can do this process 10 times, and every time the validation set will be a different chunk of the data. If we then average all of the accuracies, we will have a better sense of how our model does on average.

10 Fold Cross Validation



● Validation Set

● Training Set

Confusion Matrix

When creating a Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

Preview: Docs Loading link description

[algorithm](#)

capable of making predictions, an important step in the process is to measure the model's predictive power. In this lesson we will learn how to calculate some of the more common evaluation metrics for classification problems.

Remember, in order to calculate these

Preview: Docs Loading link description

[statistics](#)

, we need to split our data into a [training, validation, and test set](#) before we start fitting our model.

Let's say we are fitting a machine learning model to try to predict whether or not an email is spam. We can pass the features of our evaluation set through the trained model and get an output list of the predictions our model makes.

We then compare each of those predictions to the actual labels. There are four possible categories that each of the comparisons can fall under:

True Positive (**TP**): The algorithm predicted spam and it was spam

True Negative (**TN**): The algorithm predicted not spam and it was not spam

False Positive (**FP**): The algorithm predicted spam and it was not spam

False Negative (**FN**): The algorithm predicted not spam and it was spam

One common way to visualize these values is in a **confusion matrix**. In a confusion matrix the predicted classes are represented as columns and the actual classes are represented as rows.

	Predicted -	Predicted +
Actual -	TN	FP
Actual +	FN	TP

Let's calculate the number of true positives, true negatives, false positives, and false negatives from the evaluation data of a classification algorithm! Then we will construct a confusion matrix.

Instructions

Checkpoint 1 Failed, try again

1.

We've given you two ordered lists. `actual` represents the true labels of the dataset. Each `1` in the list represents an email that is spam and each `0` represents an email that is not spam. `predicted` represents the classifications that the machine learning algorithm returned. For each email, the classifier predicted whether the email was spam or not spam.

Create four variables to store the results. They should be called `true_positives`, `true_negatives`, `false_positives`, and `false_negatives`. Initialize each of the variables to `0`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Loop through each of the predictions in the list and add 1 to `true_positives` every time the algorithm found a true positive. A true positive is when the actual label and the classifier's predicted label are both 1.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Inside the for loop, count the number of true negatives, false positives, and false negatives.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Outside of the for loop, print the final `true_positives`, `true_negatives`, `false_positives`, and `false_negatives` values.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

We have already imported a function `confusion_matrix` from `scikit-learn`. Create a variable `conf_matrix` and pass the list of `actual` and `predicted` values to the `confusion_matrix` function.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Print `conf_matrix`.

```
from sklearn.metrics import confusion_matrix

actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]

true_positives = 0
true_negatives = 0
false_positives = 0
false_negatives = 0

for i in range(len(predicted)):
    if actual[i] == 1 and predicted[i] == 1:
        true_positives += 1
    if actual[i] == 0 and predicted[i] == 0:
        true_negatives += 1
    if actual[i] == 0 and predicted[i] == 1:
        false_positives += 1
    if actual[i] == 1 and predicted[i] == 0:
        false_negatives += 1

print(true_positives, true_negatives, false_positives, false_negatives)

conf_matrix = confusion_matrix(actual, predicted)

print(conf_matrix)
```

Accuracy

One Loading link description [method](#) for determining the effectiveness of a classification Loading link description [algorithm](#) is by measuring its accuracy statistic. Accuracy is calculated by finding the total number of correctly classified predictions (true positives and true negatives) and dividing by the total number of predictions.

Accuracy is defined as:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{True Positives} + \text{True Negatives} + \text{False Positives} + \text{False Negatives}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Let's calculate the accuracy of the classification algorithm.

Instructions

Checkpoint 1 Enabled

1.

Calculate the accuracy and store it in a variable named `accuracy`. Print `accuracy`.

Accuracy is $(\text{True Positives} + \text{True Negatives}) / (\text{True Positives} + \text{True Negatives} + \text{False Positives} + \text{False Negatives})$.

Your denominator could also be `len(predicted)`.

```
actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]

true_positives = 0
true_negatives = 0
false_positives = 0
false_negatives = 0

for i in range(len(predicted)):
    #True Positives
    if actual[i] == 1 and predicted[i] == 1:
        true_positives += 1
```

```

#True Negatives

if actual[i] == 0 and predicted[i] == 0:

    true_negatives += 1

#False Positives

if actual[i] == 0 and predicted[i] == 1:

    false_positives += 1

#False Negatives

if actual[i] == 1 and predicted[i] == 0:

    false_negatives += 1

accuracy = (true_positives + true_negatives) / len(predicted)

print(accuracy)

```

Recall

Accuracy can be a misleading statistic depending on our data and the problem we are trying to solve. Consider a model tasked with predicting spam in the email inboxes of top secret government employees who never use their work email addresses for online shopping or logging onto their favorite gaming apps. We can write a pretty simple and accurate classifier that always predicts False, the email is not spam. This classifier will be incredibly accurate since there are hardly ever any spam emails sent to those top secret emails, but this classifier will never be able to find the information we are actually interested in, when there is spam.

In this situation, a helpful statistic to consider is **recall**. In our example, recall measures the ratio of correct spam predictions that our classifier found to the total number of spam emails.

Recall is defined as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} = \frac{TP}{TP + FN}$$

Recall is the ratio of correct positive predictions classifications made by the model to all actual positives. For the spam classifier, this would be the number of correctly labeled spam emails divided by all the emails that were actually spam in the dataset.

Our Loading link description [algorithm](#) that always predicts not spam might have a very high accuracy, but it never will find any true positives, so its recall will be 0.

Instructions

Checkpoint 1 Enabled

1.

Calculate the recall and store it in a variable named `recall`. Print the `recall`.

Recall is `True Positives / (True Positives + False Negatives)`.

```
actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]

true_positives = 0
true_negatives = 0
false_positives = 0
false_negatives = 0

for i in range(len(predicted)):
    if actual[i] == 1 and predicted[i] == 1:
        true_positives += 1
    if actual[i] == 0 and predicted[i] == 0:
        true_negatives += 1
    if actual[i] == 0 and predicted[i] == 1:
        false_positives += 1
    if actual[i] == 1 and predicted[i] == 0:
        false_negatives += 1

recall = true_positives / (true_positives + false_negatives)

print(recall)
```

Precision

Unfortunately, recall isn't a perfect statistic either (spoiler alert! There is no perfect statistic). For example, we could create a spam email classifier that always returns `True`, the email is spam. This particular classifier would have low accuracy, but the recall would be 1 because it would be able to accurately find every spam email.

In this situation, a helpful statistic to understand is **precision**. In our email spam classification example, precision is the ratio of correct spam email predictions to the total number of spam predictions.

Precision is defined as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} = \frac{TP}{TP + FP}$$

Precision is the ratio of correct positive classifications to all positive classifications made by the model. For the spam classifier, this would be the number of correctly labeled spam emails divided by all the emails that were correctly or incorrectly labeled spam.

The Loading link description [algorithm](#) that predicts every email is spam will have a recall of 1, but it will have very low precision. It correctly predicts every spam email, but there are tons of false positives as well.

Instructions

Checkpoint 1 Enabled

1.

Calculate the precision and store it in a variable named `precision`. Print the `precision`.

Precision is `True Positives / (True Positives + False Positives)`.

```

actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]

true_positives = 0
true_negatives = 0
false_positives = 0
false_negatives = 0

for i in range(len(predicted)):
    if actual[i] == 1 and predicted[i] == 1:
        true_positives += 1
    if actual[i] == 0 and predicted[i] == 0:
        true_negatives += 1
    if actual[i] == 0 and predicted[i] == 1:
        false_positives += 1
    if actual[i] == 1 and predicted[i] == 0:
        false_negatives += 1

precision = true_positives / (true_positives + false_positives)
print(precision)

```

F1-Score

It is often useful to consider both the precision and recall when attempting to describe the effectiveness of a model. The **F1-score** combines both precision and recall into a single statistic, by determining their harmonic mean. The harmonic mean is a

Preview: Docs Loading link description [method](#) of averaging.
F1-score is defined as:

$$\text{F1-score} = (2 * \text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

We use the harmonic mean rather than the traditional arithmetic mean because we want the F1-score to have a low value when either precision or recall is 0.

For example, consider a classifier where `recall = 1` and `precision = 0.02`. Despite our classifier having an extremely high recall score, there is most likely a problem with this model since the precision is so low. Ideally the F1-score would reflect that.

If we took the arithmetic mean of precision and recall, we get:

$$(1 + 0.022) / 2 = 0.51$$

That performance statistic is misleadingly high for a classifier that has such dismal precision. If we instead calculate the harmonic mean, we get:

$$(2 \times 1 \times 0.021) / (1 + 0.02) = 0.039$$

That is a much better descriptor of the classifier's effectiveness!

Let's calculate the F1-score of the classification Loading link description [algorithm](#)

.

Instructions

Checkpoint 1 Enabled

1.

Calculate and print the F1-score. Store the F1-score in a variable named `f_1`.

F1-score is $2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$.

```
actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]

true_positives = 0
true_negatives = 0
false_positives = 0
false_negatives = 0

for i in range(len(predicted)):
    if actual[i] == 1 and predicted[i] == 1:
        true_positives += 1
    if actual[i] == 0 and predicted[i] == 0:
        true_negatives += 1
    if actual[i] == 0 and predicted[i] == 1:
        false_positives += 1
    if actual[i] == 1 and predicted[i] == 0:
        false_negatives += 1

precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)

f_1 = 2*precision*recall/(precision+recall)

print(f_1)
```

Review

There is no perfect metric. The decision to use accuracy, precision, recall, F1-score, or another metric not covered in this lesson ultimately comes down to the specific context of the classification problem.

Take the email spam problem. We probably want a model that is high precision and do not mind as much if it has a low recall score. This is because we want to make sure the

Preview: Docs Loading link description [Algorithm](#) does not incorrectly send an important email message to the spam folder, while it is not as detrimental to have a few spam emails end up in our inbox.

As long as you have an understanding of what question you're trying to answer, you should be able to determine which statistic is most relevant to you.

The Python library `scikit-learn` has some functions that will calculate these

Loading link description [statistics](#) for you!

You have now learned many different ways to analyze the predictive power of your classification algorithm. Here are some of the key takeaways:

- Classifying a single point can result in a true positive (`actual = 1, predicted = 1`), a true negative (`actual = 0, predicted = 0`), a false positive (`actual = 0, predicted = 1`), or a false negative (`actual = 1, predicted = 0`). These values are often summarized in a confusion matrix.
- Accuracy measures how many classifications your algorithm got correct out of every classification it made.
- Recall is the ratio of correct positive predictions classifications made by the model to all actual positives.
- Precision is the ratio of correct positive classifications to all positive classifications made by the model.
- F1-score is a combination of precision and recall.

- F1-score will be low if either precision or recall is low.

Instructions

Checkpoint 1 Enabled

1.

Python's `scikit-learn` library has functions that will help you calculate accuracy, recall, precision, and F1-score. They all take two parameters — a list of the actual labels and a list of the predicted classifications. Call `accuracy_score()` using the correct parameters and print the results.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Call the three other functions and print the results. The name of those functions are:

```
recall_score()  
precision_score()  
f1_score()
```

```
from sklearn.metrics import accuracy_score, recall_score, precision_score,  
f1_score  
  
actual = [1, 0, 0, 1, 1, 1, 0, 1, 1, 1]  
predicted = [0, 1, 1, 1, 1, 0, 1, 0, 1, 0]  
  
print(accuracy_score(actual, predicted))  
  
print(recall_score(actual, predicted))  
  
print(precision_score(actual, predicted))  
  
print(f1_score(actual, predicted))
```

Feature Engineering for Data Scientists

Introduction to Feature Engineering for Data Scientists

See your future with features!

You've already gotten some basics of feature engineering down. Now it is time to expand your knowledge and skills with even more functions!

What will Feature Engineering for Data Scientists cover?

After this unit, you will be able to:

- Use Filter Methods
- Apply Regularization Methods
- Use Wrapper Methods
- Determine Feature Importance
- Tune your hyperparameters

Introduction to Feature Selection Methods

Learn about the pros and cons of different feature selection methods.

Introduction and motivation

Imagine you want to build a machine learning model for predicting the presence of a disease, and you have a dataset of patient records containing hundreds of different metrics, from demographic characteristics to medical test results to past treatment history. While having access to so many data points is always a good starting point, you most likely would not want to use all of the available features to construct your model, as many of them can be redundant or irrelevant, and would only serve to contribute unnecessary noise. Instead, a more effective approach is to select only a subset of relevant, predictive features to use in your model — this process is known as *feature selection*.

Feature selection is an important step in the machine learning pipeline, and when done right, can optimize the performance and predictive power of your model. The goal is to improve the model's accuracy and efficiency by eliminating extraneous variables that do not contribute any useful information. In addition, simplifying the model through feature selection not only makes the results more easily interpretable, but also reduces the time and resources required to run the model.

There are three broad categories of feature selection that we will discuss in this article: *filter methods*, *wrapper methods*, and *embedded methods*.

Feature Selection Methods

Filter methods

Filter methods are the simplest type of feature selection method. They work by filtering features prior to model building based on some criteria.

Advantages

They are computationally inexpensive, since they do not involve testing the subsetted features using a model.

They can work for any type of machine learning model.

Disadvantages

It is more difficult to take multivariate relationships into account because we are not evaluating model performance. For example, a variable might not have much predictive power on its own, but can be informative when combined with other variables.

They are not tailored toward specific types of models.

Examples

Variance thresholds
Correlation
Mutual information

Wrapper methods

Wrapper methods involve fitting a model and evaluating its performance for a particular subset of features. They work by using a search algorithm to find which combination of features can optimize the performance of a given model.

Advantages

They can determine the optimal set of features that produce the best results for a specific machine learning problem.
They can better account for multivariate relationships because model performance is evaluated.

Disadvantages

They are computationally expensive because the model needs to be re-fitted for each feature set being tested.

Examples

Forward/backward/bidirectional sequential feature selection
Recursive feature elimination

Embedded methods

Embedded methods also involve building and evaluating models for different feature subsets, but their feature selection process happens at the same time as their model fitting step.

Advantages

Like wrapper methods, they can optimize the feature set for a particular model and account for multivariate relationships.
They are also generally less computationally expensive because feature selection happens during model training.

Examples

Regularization (e.g., lasso/ridge regression)
Tree-based feature importance

Conclusion

When it comes to the number of features to keep for your model, more is not always better. It is worth being selective about which features to retain in order to maximize performance and reduce noise. Therefore, finding and implementing the right feature selection method is a key part of developing an effective and reliable model.

Filter Methods

There are many of filter methods, however we'll cover the following filter method:

- *Variance thresholds,*
- *Correlation, and*
- *Mutual information*

<https://www.codecademy.com/paths/data-science/tracks/dsml-feature-engineering-for-ds/modules/dsml-filter-methods/articles/fe-filter-methods>

Learn about the different filter methods for evaluating and selecting features.

Introduction

Filter methods are a type of feature selection method that works by selecting features based on some criteria prior to building the model. Because they don't involve actually testing the subsetted features using a model, they are computationally inexpensive and flexible to use for any type of machine learning algorithm. This makes filter methods an efficient initial step for narrowing down the pool of features to only the most relevant, predictive ones.

There are many different filter methods that can be used for evaluating and selecting features. In this article, we will use *variance thresholds*, *correlation*, and *mutual information* to rank and select the top features. To demonstrate how these methods work in Python, we will use the [feature_selection](#) module from [scikit-learn](#) as well as the [pandas](#) library.

Example dataset

Let's suppose we have the following dataset containing information on a class of middle school students:

```
import pandas as pd

df = pd.DataFrame(data={
    'edu_goal': ['bachelors', 'bachelors', 'bachelors', 'masters', 'masters',
'masters', 'masters', 'phd', 'phd', 'phd'],
    'hours_study': [1, 2, 3, 3, 3, 4, 3, 4, 5, 5],
    'hours_TV': [4, 3, 4, 3, 2, 3, 2, 2, 1, 1],
    'hours_sleep': [10, 10, 8, 8, 6, 6, 8, 8, 10, 10],
```

```

    'height_cm': [155, 151, 160, 160, 156, 150, 164, 151, 158, 152],
    'grade_level': [8, 8, 8, 8, 8, 8, 8, 8, 8, 8],
    'exam_score': [71, 72, 78, 79, 85, 86, 92, 93, 99, 100]
})

print(df)

```

Output:

edu_goal	hours_study	hours_TV	hours_sleep	height_cm	grade_level	exam_score
bachelors	1	4	10	155	8	71
bachelors	2	3	10	151	8	72
bachelors	3	4	8	160	8	78
masters	3	3	8	160	8	79
masters	3	2	6	156	8	85
masters	4	3	6	150	8	86
masters	3	2	8	164	8	92
phd	4	2	8	151	8	93
phd	5	1	10	158	8	99
phd	5	1	10	152	8	100

Our goal is to use the data to predict how well each student will perform on the exam. Thus, our target variable is `exam_score` and the remaining 6 variables are our features. We'll prepare the data by separating the features matrix (`X`) and the target vector (`y`).

10 x 6 features matrix:

```

X = df.drop(columns=['exam_score'])

print(X)

```

Output:

edu_goal	hours_study	hours_TV	hours_sleep	height_cm	grade_level
bachelors	1	4	10	155	8
bachelors	2	3	10	151	8

bachelors	3	4	8	160	8
masters	3	3	8	160	8
masters	3	2	6	156	8
masters	4	3	6	150	8
masters	3	2	8	164	8
phd	4	2	8	151	8
phd	5	1	10	158	8
phd	5	1	10	152	8

10 x 1 target vector:

```
y = df['exam_score']
print(y)
```

Output:

exam_score

71
72
78
79
85
86
92
93
99
100

Variance threshold

One of the most basic filter methods is to use a *variance threshold* to remove any features that have little to no variation in their values. This is because features with low variance do not contribute much information to a model. Since variance can only be calculated on numeric values, this method only works

on quantitative features. That said, we may also want to remove categorical features for which all or a majority of the values are the same. To do that, we would need to dummy code the categorical variables first, but we won't demonstrate that here.

In our example dataset, `edu_goal` is the only feature that is not numeric. We can use the `.drop()` method to remove it from our features DataFrame and store the remaining numeric features in `X_num`:

```
X_num = X.drop(columns=['edu_goal'])
print(X_num)
```

Output:

	hours_study	hours_TV	hours_sleep	height_cm	grade_level
1		4	10	155	8
2		3	10	151	8
3		4	8	160	8
3		3	8	160	8
3		2	6	156	8
4		3	6	150	8
3		2	8	164	8
4		2	8	151	8
5		1	10	158	8
5		1	10	152	8

Now, we'll be able to use the `VarianceThreshold` class from `scikit-learn` to help remove the low-variance features from `X_num`. By default, it drops all features with zero variance, but we can adjust the threshold during class instantiation using the `threshold` parameter if we want to allow some variation. The `.fit_transform()` method returns the filtered features as a numpy array:

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0) # 0 is default

print(selector.fit_transform(X_num))
```

The output will look like this:

```
[[ 1  4 10 155]
 [ 2  3 10 151]
 [ 3  4  8 160]
 [ 3  3  8 160]]
```



```
[ 3  2  6 156]
[ 4  3  6 150]
[ 3  2  8 164]
[ 4  2  8 151]
[ 5  1 10 158]
[ 5  1 10 152]]
```

As we can see, `grade_level` was removed because there is no variation in its values — all students are 8th graders. Since this data is the same across the board, a student's grade level will not be able to provide any useful predictive information about their exam score, so it makes sense to drop `grade_level` as a feature.

Something to note is that [loading datasets with scikit-learn](#) generally works with numpy arrays internally, hence the output type of `.fit_transform()`. However, the methods can also accept other data types that can be converted to numpy arrays, such as Python lists or pandas DataFrames, like the `X_num` we used. From a human perspective, one downside of working with numpy arrays as compared to pandas DataFrame is that we lose information like column headings, making the data harder to visually inspect.

Luckily, `VarianceThreshold` offers another method called `.get_support()` that can return the indices of the selected features, which we can use to manually subset our numeric features DataFrame.

Specify `indices=True` to get indices of selected features:

```
print(selector.get_support(indices=True))
```

Output:

```
[0 1 2 3]
```

Use indices to get the corresponding column names of selected features:

```
num_cols = list(X_num.columns[selector.get_support(indices=True)])
print(num_cols)
```

Output:

```
['hours_study', 'hours_TV', 'hours_sleep', 'height_cm']
```

Subset `X_num` to retain only selected features:

```
X_num = X_num[num_cols]
print(X_num)
```

Output:

`hours_study`

`hours_TV`

`hours_sleep`

`height_cm`

1	4	10	155
2	3	10	151
3	4	8	160
3	3	8	160
3	2	6	156
4	3	6	150
3	2	8	164
4	2	8	151
5	1	10	158
5	1	10	152

Finally, to obtain our entire features DataFrame, including the categorical column `edu_goal`, we could do:

```
X = X[['edu_goal'] + num_cols]
print(X)
```

Output:

edu_goal	hours_study	hours_TV	hours_sleep	height_cm
bachelors	1	4	10	155
bachelors	2	3	10	151
bachelors	3	4	8	160
masters	3	3	8	160
masters	3	2	6	156
masters	4	3	6	150
masters	3	2	8	164
phd	4	2	8	151
phd	5	1	10	158
phd	5	1	10	152

Pearson's correlation

Another type of filter method involves finding the *correlation* between variables. In particular, the [Pearson's correlation coefficient](#) is useful for measuring the linear relationship between two numeric, continuous variables — a coefficient close to `1` represents a positive correlation, `-1` represents a negative correlation, and `0` represents no correlation. Like variance, Pearson's correlation coefficient cannot be calculated for categorical variables. Although, there is a related [point biserial correlation coefficient](#) that can be computed when one variable is dichotomous, but we won't focus on that here.

There are 2 main ways of using correlation for feature selection — to detect correlation between features and to detect correlation between a feature and the target variable.

Correlation between features

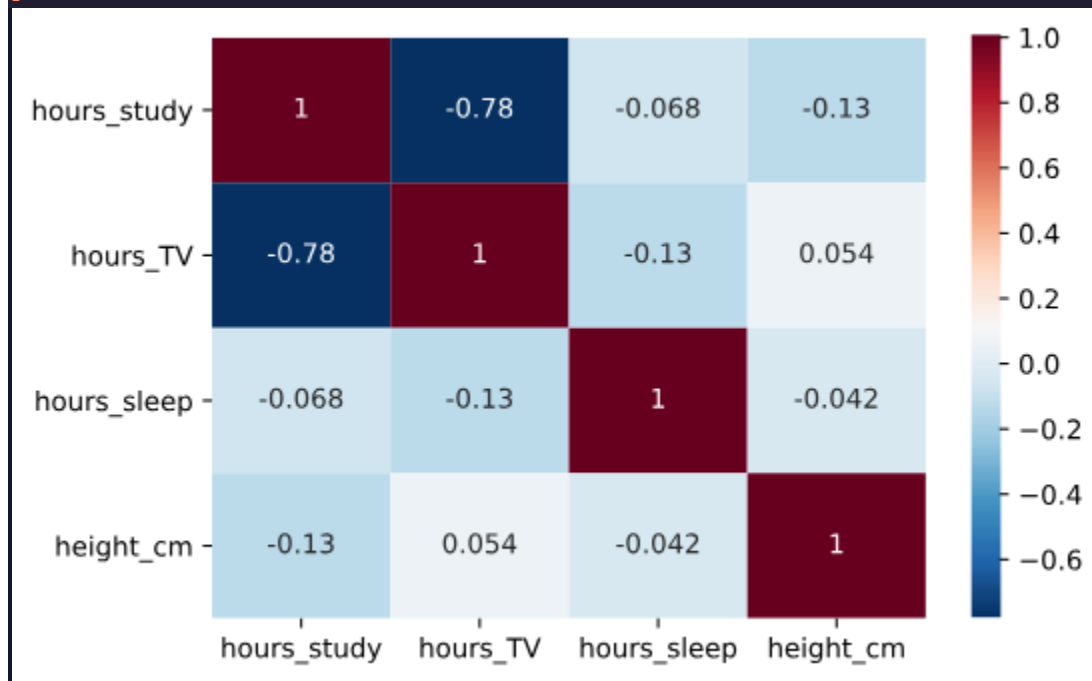
When two features are highly correlated with one another, then keeping just one to be used in the model will be enough because otherwise they provide duplicate information. The second variable would only be redundant and serve to contribute unnecessary noise.

To determine which variables are correlated with one another, we can use the `.corr()` method from `pandas` to find the correlation coefficient between each pair of numeric features in a DataFrame. By default, `.corr()` computes the Pearson's correlation coefficient, but alternative methods can be specified using the `method` parameter. We can visualize the resulting correlation matrix using a heatmap:

```
import matplotlib.pyplot as plt
import seaborn as sns

corr_matrix = X_num.corr(method='pearson') # 'pearson' is default

sns.heatmap(corr_matrix, annot=True, cmap='RdBu_r')
plt.show()
```



Let's define high correlation as having a coefficient of greater than 0.7 or less than -0.7. We can loop through the correlation matrix to identify the highly correlated variables:

```
# Loop over bottom diagonal of correlation matrix
for i in range(len(corr_matrix.columns)):
    for j in range(i):

        # Print variables with high correlation
        if abs(corr_matrix.iloc[i, j]) > 0.7:
            print(corr_matrix.columns[i], corr_matrix.columns[j],
                  corr_matrix.iloc[i, j])
```

The output to our code is:

```
hours_TV hours_study -0.780763315142435
```

As seen, `hours_TV` appears to be highly negatively correlated with `hours_study` — a student who watches a lot of TV tends to spend fewer hours studying, and vice versa. Because they provide redundant information, we can choose to remove one of those variables. To decide which one, we can look at their correlation with the target variable, then remove the one that is less associated with the target. This is explored in the next section.

Correlation between feature and target

As mentioned, the second way correlation can be used is to determine if there is a relationship between a feature and the target variable. In the case of Pearson's correlation, this is especially useful if we intend to fit a linear model, which assumes a linear relationship between the target and predictor variables. If a feature is not very correlated with the target variable, such as having a coefficient of between -0.3 and 0.3, then it may not be very predictive and can potentially be filtered out.

We can use the same `.corr()` method seen previously to obtain the correlation between the target variable and the rest of the features. First, we'll need to create a new DataFrame containing the numeric features with the `exam_score` column:

```
X_y = X_num.copy()
X_y['exam_score'] = y

print(X_y)
```

Output:

	hours_study	hours_TV	hours_sleep	height_cm	exam_score
1		4	10	155	71
2		3	10	151	72
3		4	8	160	78

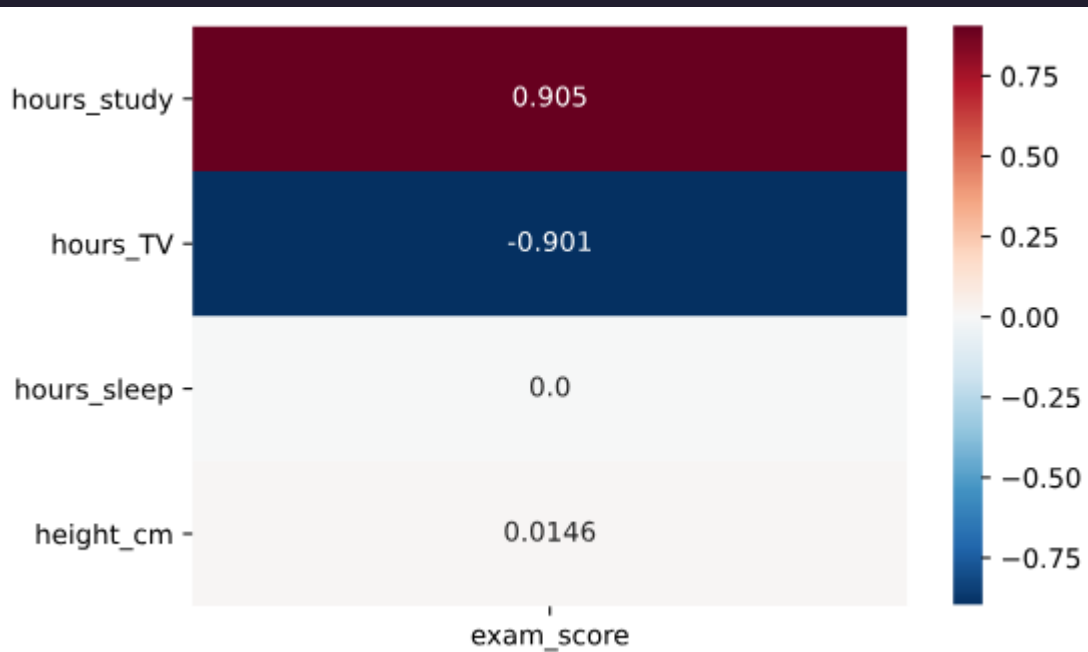
3	3	8	160	79
3	2	6	156	85
4	3	6	150	86
3	2	8	164	92
4	2	8	151	93
5	1	10	158	99
5	1	10	152	100

Then, we can generate the correlation matrix and isolate the column corresponding to the target variable to see how strongly each feature is correlated with it:

```
corr_matrix = X_y.corr()

# Isolate the column corresponding to `exam_score`
corr_target = corr_matrix[['exam_score']].drop(labels=['exam_score'])

sns.heatmap(corr_target, annot=True, fmt='.3', cmap='RdBu_r')
plt.show()
```



Output:

As seen, `hours_study` is positively correlated with `exam_score` and `hours_TV` is negatively correlated with it. It makes sense that `hours_study` and `hours_TV` would be negatively correlated with each other as we saw earlier, and just one of those features would suffice for predicting `exam_score`. Since `hours_study` has a stronger correlation with the target variable, let's remove `hours_TV` as the redundant feature:

```
X = X.drop(columns=['hours_TV'])  
  
print(X)
```

Output:

edu_goal	hours_study	hours_sleep	height_cm
bachelors	1	10	155
bachelors	2	10	151
bachelors	3	8	160
masters	3	8	160
masters	3	6	156
masters	4	6	150
masters	3	8	164
phd	4	8	151
phd	5	10	158
phd	5	10	152

The other two features, `hours_sleep` and `height_cm`, both do not seem to be correlated with `exam_score`, suggesting they would not be very good predictors. We could potentially remove either or both of them as being uninformative. But before we do, it is a good idea to use other methods to double check that the features truly are not predictive. We will do that in the next section by using mutual information to see if there are any non-linear associations between the features and target variable.

To conclude this section, we'll briefly note an alternative approach for assessing the correlation between variables. Instead of generating the full correlation matrix, we could use the `f_regression()` function from `scikit-learn` to find the F-statistic for a model with each predictor on its own. The F-statistic will be larger (and p-value will be smaller) for predictors that are more highly correlated with the target variable, thus it will perform the same filtering:

```
from sklearn.feature_selection import f_regression  
  
print(f_regression(X_num, y))
```

Output:

```
(array([3.61362007e+01, 3.44537037e+01, 0.00000000e+00, 1.70259066e-03]),  
array([3.19334945e-04, 3.74322763e-04, 1.00000000e+00, 9.68097878e-01]))
```

The function returns the F-statistic in the first array and the p-value in the second. As seen, the result is consistent with what we had observed in the correlation matrix — the stronger the correlation (either

positive or negative) between the feature and target, the higher the corresponding F-statistic and lower the p-value. For example, amongst all the features, `hours_study` has the largest correlation coefficient (`0.905`), highest F-statistic (`3.61e+01`), and lowest p-value (`3.19e-04`).

Mutual information

The final filter method we'll look at is using *mutual information* to rank and select the top features. [Mutual information](#) is a measure of dependence between two variables and can be used to gauge how much a feature contributes to the prediction of the target variable. It is similar to Pearson's correlation, but is not limited to detecting linear associations. This makes mutual information useful for more flexible models where a linear functional form is not assumed. Another advantage of mutual information is that it also works on discrete features or target, unlike correlation. Although, categorical variables need to be numerically encoded first.

In our example, we can encode the `edu_goal` column using the [LabelEncoder](#) class from `scikit-learn`'s `preprocessing` module:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Create copy of `X` for encoded version
X_enc = X.copy()
X_enc['edu_goal'] = le.fit_transform(X['edu_goal'])

print(X_enc)
```

Output:

	edu_goal	hours_study	hours_sleep	height_cm
0	1	10	155	
0	2	10	151	
0	3	8	160	
1	3	8	160	
1	3	6	156	
1	4	6	150	
1	3	8	164	
2	4	8	151	
2	5	10	158	

Now, we can compute the mutual information between each feature and `exam_score` using `mutual_info_regression()`. This function is used because our target variable is continuous, but if we had a discrete target variable, we would use `mutual_info_classif()`. We specify the `random_state` in the function in order to obtain reproducible results:

```
from sklearn.feature_selection import mutual_info_regression

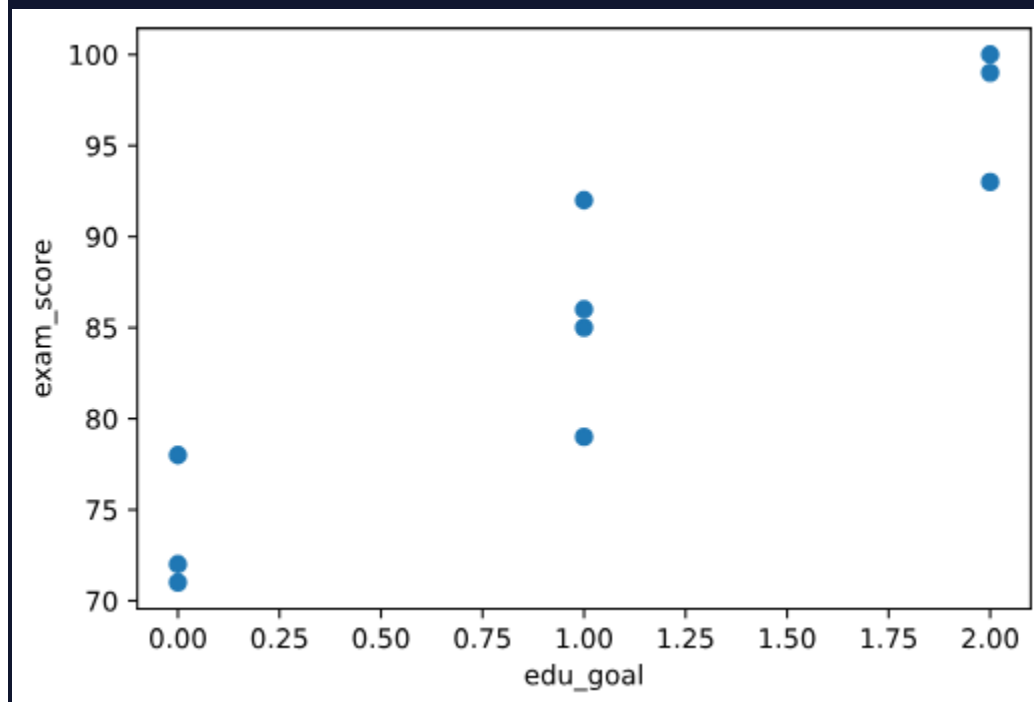
print(mutual_info_regression(X_enc, y, random_state=68))
```

Output:

```
[0.50396825 0.40896825 0.06896825 0.         ]
```

The estimated mutual information between each feature and the target is returned in a numpy array, where each value is a non-negative number – the higher the value, the more predictive power is assumed.

However, we are missing one more important piece here. Earlier, even though we encoded `edu_goal` to be numeric, that does not mean it should be treated as a continuous variable. In other words, the values of `edu_goal` are still discrete and should be interpreted as such. If we plot `edu_goal` against `exam_score` on a graph, we can clearly see the steps between the values of `edu_goal`:



In order to properly calculate the mutual information, we need to tell `mutual_info_regression()` which features are discrete by providing their index positions using the `discrete_features` parameter:

```
print(mutual_info_regression(X_enc, y, discrete_features=[0],
random_state=68))
```

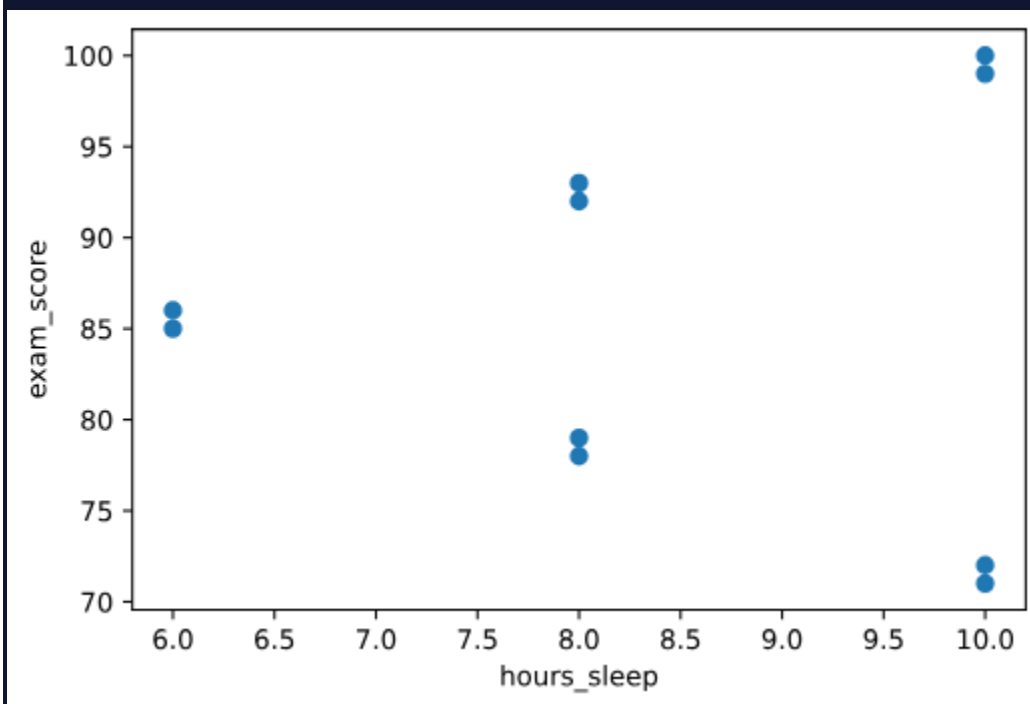

Output:

```
[0.75563492 0.38896825 0.18563492 0.          ]
```

Compared to the earlier results, we now get greater mutual information between `edu_goal` and the target variable once it is correctly interpreted as a discrete feature.

From the results, we can also see that there is 0 mutual information between `height_cm` and `exam_score`, suggesting that these variables are largely independent. This is consistent with what we saw earlier with Pearson's correlation, where the correlation coefficient between them is very close to 0 as well.

What is interesting to note is that the mutual information between `hours_sleep` and `exam_score` is a positive value, even though their Pearson's correlation coefficient is 0. The answer becomes more clear when we plot the relationship between `hours_sleep` and `exam_score`:



As seen, there do seem to be some association between the variables, only it is not a linear one, which is why it was detected using mutual information but not Pearson's correlation coefficient.

Finally, let's look at using the `SelectKBest` class from `scikit-learn` to help pick out the top `k` features with the highest ranked scores. In our case, we are looking to select features that share the most mutual information with the target variable. When we instantiate `SelectKBest`, we'll specify which scoring function to use and how many top features to select. Here, our scoring function is `mutual_info_regression()`, but because we want to specify additional arguments besides the `x` and `y` inputs, we'll need the help of the `partial()` function from Python's built-in `functools` module. Then, the `.fit_transform()` method will return the filtered features as a numpy array:

```
from sklearn.feature_selection import SelectKBest
```

```

from functools import partial

score_func = partial(mutual_info_regression, discrete_features=[0],
random_state=68)

# Select top 3 features with the most mutual information
selection = SelectKBest(score_func=score_func, k=3)

print(selection.fit_transform(X_enc, y))

```

Output:

```

[[ 0  1 10]
 [ 0  2 10]
 [ 0  3  8]
 [ 1  3  8]
 [ 1  3  6]
 [ 1  4  6]
 [ 1  3  8]
 [ 2  4  8]
 [ 2  5 10]
 [ 2  5 10]]

```

As seen above, we selected the top 3 features based on mutual information, thus dropping `height_cm`. Like `VarianceThreshold`, `SelectKBest` also offers the `.get_support()` method that returns the indices of the selected features, so we could subset our original features DataFrame:

```

X = X[X.columns[selection.get_support(indices=True)]]

print(X)

```

Output:

edu_goal	hours_study	hours_sleep
bachelors	1	10
bachelors	2	10
bachelors	3	8
masters	3	8
masters	3	6
masters	4	6
masters	3	8
phd	4	8

phd	5	10
phd	5	10

Conclusion

In our example dataset, we started out with 6 features for predicting the `exam_score` of students. Using various filter methods, we narrowed down that set to just the top most relevant and informative ones. First, we eliminated `grade_level` because it has zero variance and would contribute nothing to the model. Then, we dropped `hours_TV` since it is highly correlated with `hours_study` and is therefore redundant. Lastly, we filtered out `height_cm` based on mutual information, which suggested that it does not have any meaningful association with the target variable, linear or otherwise, and would not have been very predictive.

Phew! That was a lot we were able to accomplish using filter methods. Being the most simple type of feature selection method, they sure do not lack power nor potential. It is certainly worth considering how you might want to incorporate filter methods into your next machine learning project.

Wrapper Methods

Introduction to Wrapper Methods

Machine learning problems often involve datasets with many features. Some of those features might be very important for a specific

Preview: Docs Loading link description

[machine learning](#)

model. Other features might be irrelevant. Given a feature set and a model, we would like to be able to distinguish between important and unimportant features (or even important combinations of features). Wrapper methods do exactly that.

A *wrapper method* for feature selection is an

Preview: Docs Loading link description

[algorithm](#)

that selects features by evaluating the performance of a machine learning model on different subsets of features. These algorithms add or remove features one at a time based on how useful those features are to the model.

Wrapper methods have some advantages over filter methods. The main advantage is that wrapper methods evaluate features based on their performance with a specific model. Filter methods, on the other hand, can't tell how important a feature is to a model.

Another upside of wrapper methods is that they can take into account relationships between features. Sometimes certain features aren't very useful on their own but instead perform well only when combined

with other features. Since wrapper methods test subsets of features, they can account for those situations.

This lesson will explain five different wrapper methods:

- Sequential forward selection
- Sequential backward selection
- Sequential forward floating selection
- Sequential backward floating selection
- Recursive feature elimination

You'll learn how to implement these algorithms in Python and evaluate the results.

Before we get started, let's take a look at a dataset that you'll use throughout this lesson.

Instructions

Checkpoint 1 Enabled

1.

The data in the workspace was taken from the [UCI Machine Learning Repository](#). The outcome variable, `Classification`, is a 1 if a patient has breast cancer and a 0 if not. The features are `Age`, `BMI` (body mass index), `Glucose`, `Insulin`, `HOMA` (homeostatic model assessment, a measure of insulin resistance), as well as four different protein levels: `Leptin`, `Adiponectin`, `Resistin`, and `MCP.1`.

Uncomment `print(health.head())` and take a look at the data.

What are some pros and cons of having this many features?

```
import pandas as pd
import codecademylib3

# Load the data
health = pd.read_csv("dataR2.csv")

print(health.head())
```

Setting Up a Logistic Regression Model

Before we can use a wrapper

Preview: Docs Loading link description

[method](#)

, we need to specify a

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

model. We'll train a logistic regression model on the `health` data and see how well it performs.

We'll prepare the data by splitting it into a pandas DataFrame `x` and a pandas Series `y`. `x` will contain the observations of the independent

Preview: Docs Loading link description

[variables](#)

, and `y` will contain the observations of the dependent variable.

Here's an example of how to do this. The `fire` dataset below was taken from the [UCI Machine Learning Repository](#) and cleaned for our analysis. Its features are `Temperature`, `RH` (relative humidity), `Ws` (wind speed), `Rain`, `DMC` (Duff

Moisture Code), and FWI (Fire Weather Index). The final column, `Classes`, contains a `1` if there is a forest fire at a specific location on a given day and `0` if not.

```
import pandas as pd

# Load the data
fire = pd.read_csv("fire.csv")
# Split independent and dependent variables
X = fire.iloc[:, :-1]
y = fire.iloc[:, -1]
```

Copy to Clipboard

We can create a logistic regression model and fit it to `x` and `y` with scikit-learn using the following code.

```
from sklearn.linear_model import LogisticRegression

# Create and fit the logistic regression model
lr = LogisticRegression()
lr.fit(X, y)
```

Copy to Clipboard

Logistic regression models give a probability that an observation belongs to a category. In the `fire` dataset, probabilities greater than 0.5 are considered predictions that there is a fire, and probabilities less than 0.5 are considered predictions that there is no fire. In the `health` dataset, probabilities greater than 0.5 are considered predictions that a patient has breast cancer.

The *accuracy* of a logistic regression model is the percentage of correct predictions that it makes on a testing set. In scikit-learn, you can check the accuracy of a model with the `.score()` method.

```
print(lr.score(X, y))
```

Copy to Clipboard

This outputs:

```
0.9836065573770492
```

Copy to Clipboard

For our testing set, our logistic regression model correctly predicts whether a fire occurred 98.4% of the time.

Instructions

Checkpoint 1 Failed, try again

1.

The dataset `health` has been loaded and split into features `x` and outcome `y` for you in `script.py`. Use the `.fit()` method to fit `lr` to `x` and `y`.

(Side note: scikit-learn uses an algorithm that goes through many iterations to find optimal logistic regression coefficients. For this particular dataset, the algorithm doesn't converge after the default maximum number of iterations, so we've included `max_iter=1000` as a parameter in the `LogisticRegression` object just to make sure the algorithm converges.)

Recall that to fit the model for the `fire` dataset, we used the following code:

```
from sklearn.linear_model import LogisticRegression

# Create and fit the logistic regression model
lr = LogisticRegression()
lr.fit(X, y)
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `.score()` method to print the accuracy of the model and print it. How often does this model correctly predict whether or not a patient has breast cancer?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Load the data
health = pd.read_csv("dataR2.csv")
# Split independent and dependent variables
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Fit the model
lr.fit(X, y)

# Print the accuracy of the model
print(lr.score(X, y))
```

Sequential Forward Selection

Now that we have a specific

Preview: Docs Loading link description

[machine learning](#)

model, we can use a wrapper

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

to choose a smaller feature subset.

Sequential forward selection is a wrapper method that builds a feature set by starting with no features and then adding one feature at a time until a desired number of features is reached. In the first step, the

Preview: Docs Loading link description

[algorithm](#)

will train and test a model using only one feature at a time. The algorithm keeps the feature that performs best.

In each subsequent step, the algorithm will test the model on each possible new feature addition. Whichever feature improves model performance the most is then added to the feature subset. This process stops once we have the desired number of features.

Let's say we want to use three features, and we have five features to choose from: `age`, `height`, `weight`, `blood_pressure`, and `resting_heart_rate`. Sequential forward selection will train your machine learning model on five different feature subsets: one for each feature.

If the model performs best on the subset `{age}`, the algorithm will then train and test the model on the following four subsets:

```
{age, height}
{age, weight}
{age, blood_pressure}
{age, resting_heart_rate}
```

If the model performs best on `{age, resting_heart_rate}`, the algorithm will test the model on the following three subsets:

```
{age, height, resting_heart_rate}
{age, weight, resting_heart_rate}
{age, blood_pressure, resting_heart_rate}
```

If the model performs best on {age, weight, resting_heart_rate}, it will stop the algorithm and use that feature set.

Sequential forward selection is a *greedy* algorithm: instead of checking every possible feature set by brute force, it adds whichever feature gives the best immediate performance gain.

Instructions

Checkpoint 1 Enabled

1.

If you used sequential forward selection to add a fourth feature to the feature set {age, weight, resting_heart_rate} that you saw in the preceding example, what two feature sets would the algorithm test in the next step? Create sets called `set1` and `set2` that contain the appropriate features.

For example, if the answer was {height, resting_heart_rate} and {weight, resting_heart_rate}, you would type:

```
set1 = {"height", "resting_heart_rate"}
set2 = {"weight", "resting_heart_rate"}
```

Copy to Clipboard

One of the sets is `set1 = {"age", "height", "weight", "resting_heart_rate"}`.

```
set1 = {"age", "height", "weight", "resting_heart_rate"}
set2 = {"age", "weight", "blood_pressure", "resting_heart_rate"}
```

Sequential Forward Selection with mlxtend

Recall from a previous exercise that the logistic regression model was about 80.2% accurate at predicting if a patient had breast cancer. But there were NINE different features. Are all of those features necessary, or is it possible that the model could make accurate predictions with fewer features? That would make the model easier to understand, and it could simplify diagnosis.

We will use the `SFS` class from Python's `mlxtend` library to implement sequential forward selection and choose a subset of just THREE features for the logistic regression model that we used earlier.

```
# Set up SFS parameters
sfs = SFS(lr,
          k_features=3, # number of features to select
          forward=True,
          floating=False,
          scoring='accuracy',
          cv=0)
# Fit SFS to our features X and outcome y
sfs.fit(X, y)
```

Copy to Clipboard

The first

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

parameter

is the name of the model that you're using. In the previous exercise, we called the logistic regression model `lr`.

The parameter `k_features` determines how many features the

Preview: Docs Loading link description

algorithm

will select.

`forward=True` and `floating=False` ensure that we are using sequential forward selection.

`scoring` determines how the algorithm will evaluate each feature subset. It's often okay to use the default value `None` because `mlxtend` will automatically use a metric that is suitable for whatever scikit-learn model you are using. For this lesson, we'll set it to `'accuracy'`.

`cv` allows you to do k-fold cross-validation. We'll leave it at `0` for this lesson and only evaluate performance on the training set.

We'll see which features were selected in the next exercise. For now, we just want to fit the `SFS` model.

Instructions

Checkpoint 1 Enabled

1.

Use `mlxtend` to create an `SFS` object called `sfs`. Set it to select three features, and make sure that you set `forward` and `floating` arguments to ensure that you are using sequential forward selection. Also set `scoring='accuracy'` and `cv=0`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `.fit()` method to fit `sfs` to `X` and `y`.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Sequential forward selection
sfs = SFS(lr,
          k_features=3,
          forward=True,
          floating=False,
          scoring='accuracy',
          cv=0)

# Fit the sequential forward selection model
sfs.fit(X, y)
```


Evaluating the Result of Sequential Forward Selection

The `sfs` object that you fit in the previous exercise contains information about the sequential forward selection that was applied to your feature set. The `.subsets_` attribute allows you to see much of that information, including which feature was chosen at each step and the model's accuracy after each feature addition.

`sfs.subsets_` is a

Preview: Docs Loading link description

[dictionary](#)

that looks something like this:

```
{1: {'feature_idx': (7,),
    'cv_scores': array([0.93852459]),
    'avg_score': 0.9385245901639344,
    'feature_names': ('FWI',)},
 2: {'feature_idx': (4, 7),
    'cv_scores': array([0.97540984]),
    'avg_score': 0.9754098360655737,
    'feature_names': ('DMC', 'FWI')},
 3: {'feature_idx': (1, 4, 7),
    'cv_scores': array([0.9795082]),
    'avg_score': 0.9795081967213115,
    'feature_names': ('RH', 'DMC', 'FWI')}}}
```

Copy to Clipboard

The keys in this dictionary are the numbers of features at each step in the sequential forward selection

Preview: Docs Loading link description

[algorithm](#)

. The values in the dictionary are dictionaries with information about the feature set at each step. `'avg_score'` is the accuracy of the model with the specified number of features.

In this particular example, the model had an accuracy of about 93.9% after the feature `FWI` was added. The accuracy improved to about 97.5% after a second feature, `DMC`, was added. Once three features were added the accuracy improved to about 98.0%.

You can use this dictionary to easily get a

Preview: Docs Loading link description

[tuple](#)

of chosen features or the accuracy of the model after any step.

```
# Print a tuple of feature names after 5 features are added
print(sfs.subsets_[5]['feature_names'])
```

Copy to Clipboard

This outputs:

```
('RH', 'Ws', 'Rain ', 'DMC', 'FWI')
```

Copy to Clipboard

```
# Print the accuracy of the model after 5 features are added
print(sfs.subsets_[5]['avg_score'])
```

Copy to Clipboard

This outputs:

```
0.9836065573770492
```

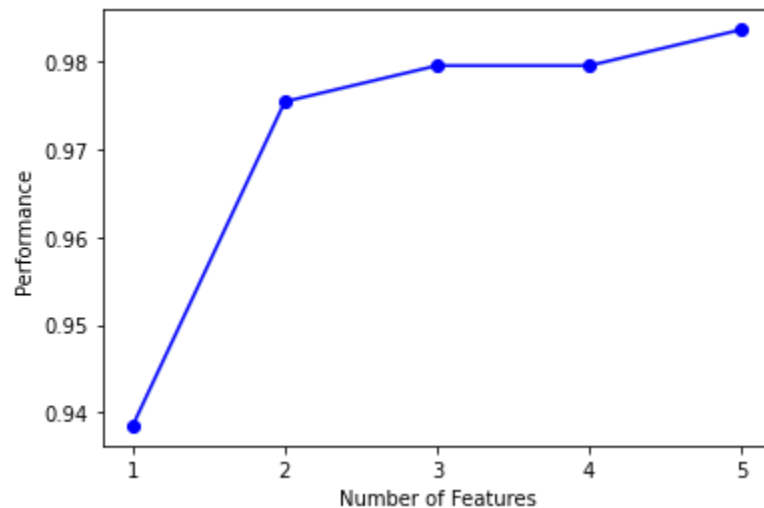
Copy to Clipboard

The `mlxtend` library also makes it easy to visualize how the accuracy of a model changes as sequential forward selection adds features. You can use the code `plot_sfs(sfs.get_metric_dict())` to create a matplotlib figure that plots the model's performance as a function of the number of features used.

```
from mlxtend.plotting import plot_sequential_feature_selection as plot_sfs
import matplotlib.pyplot as plt

# Plot the accuracy of the model as a function of the number of features
plot_sfs(sfs.get_metric_dict())
plt.show()
```

Copy to Clipboard



This plot shows you some of the same information that was in `sfs.subsets_`. The accuracy after one feature was added is about 93.9%, then 97.5% after a second feature is added, and so on.

Instructions

Checkpoint 1 Enabled

1. Use the `.subsets_` attribute of `sfs` to print a tuple that contains the names of the features chosen in the previous exercise (remember that we chose three features).

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Use the `.subsets_` attribute of `sfs` to print the accuracy of the model with three features after doing sequential forward selection.

Recall that the original model was about 80% accurate when it used all nine features. How does the accuracy of the model with only three features compare?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3. Use `plot_sfs(sfs.get_metric_dict())` to plot the accuracy as a function of the number of features. Remember to show your plot.

```
import pandas as pd
import codecademylib3
from sklearn.linear_model import LogisticRegression
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from mlxtend.plotting import plot_sequential_feature_selection as plot_sfs
import matplotlib.pyplot as plt

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
```

```

y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Sequential forward selection
sfs = SFS(lr,
          k_features=3,
          forward=True,
          floating=False,
          scoring='accuracy',
          cv=0)
sfs.fit(X, y)

# Print the chosen feature names
print(sfs.subsets_[3] ['feature_names'])

# Print the accuracy of the model after sequential forward selection
print(sfs.subsets_[3] ['avg_score'])

# Plot the model accuracy
plot_sfs(sfs.get_metric_dict())
plt.show()

```

Sequential Backward Selection with mlxtend

Sequential backward selection is another wrapper

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

for feature selection. It is very similar to sequential forward selection, but there is one key difference. Instead of starting with no features and adding one feature at a time, sequential backward selection starts with all of the available features and removes one feature at a time.

Let's again say we want to use three of the following five features: `age`, `height`, `weight`, `blood_pressure`, and `resting_heart_rate`. Sequential backward selection will start by training whatever

Preview: Docs Loading link description

[machine learning](#)

model you are using on five different feature subsets, one for each possible feature removal:

```

{height, weight, blood_pressure, resting_heart_rate}
{age, weight, blood_pressure, resting_heart_rate}
{age, height, blood_pressure, resting_heart_rate}
{age, height, weight, resting_heart_rate}
{age, height, weight, blood_pressure}

```

Let's say that out of the five subsets, the model performed best on the subset without `blood_pressure`. Then the

Preview: Docs Loading link description

[algorithm](#)

will proceed with the feature set `{age, height, weight, resting_heart_rate}`. It then tries removing each of `age`, `height`, `weight`, and `resting_heart_rate`.

Let's say that of those four subsets, the model performed best without `weight`. Then it will arrive at the subset `{age, height, resting_heart_rate}`. The algorithm will stop there since it arrived at the desired number of features.

To implement sequential backward selection in `mlxtend` you can use the same `SFS` class you used for sequential forward selection. The only difference is that you have to set the

Preview: Docs Loading link description

[parameter](#)

`forward` to `False`.

Instructions

Checkpoint 1 Passed

1.

The object `sbs` is currently set to use sequential forward selection. Change the `forward` parameter so that it uses sequential backward selection instead. You can leave all the other parameters the same.

In the `SFS` object, set `forward=False`.

Checkpoint 2 Passed

2.

Use the `.fit()` method to fit `sbs` to `X` and `y`.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Sequential backward selection
sbs = SFS(lr,
          k_features=3,
          forward=False,
          floating=False,
          scoring='accuracy',
          cv=0)

# Fit the sequential backward selection model
sbs.fit(X, y)
```

Evaluating the Result of Sequential Backward Selection

As you learned in a previous exercise, `model.subsets_` is a

Preview: Docs Loading link description

[dictionary](#)

that contains information about feature subsets from each step of an `SFS` selection model. This works with sequential backward selection just like it did with sequential forward selection.

```
print(sbs.subsets_)
```

Copy to Clipboard

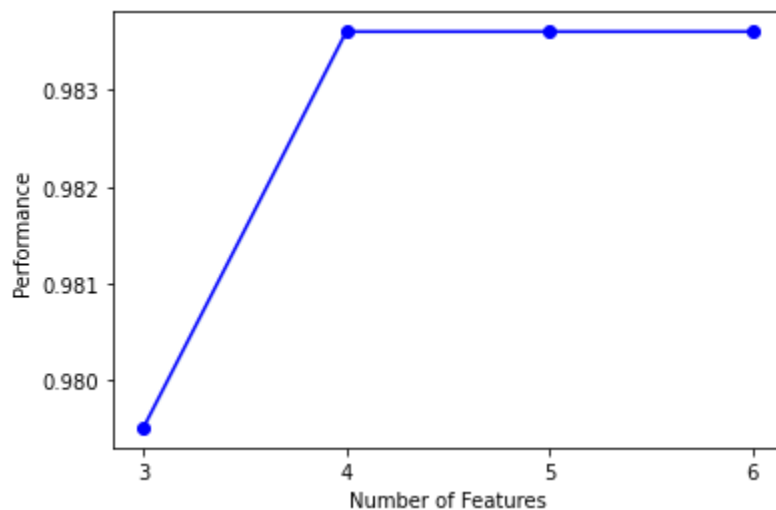
```
{6: {'feature_idx': (0, 1, 2, 3, 4, 5),
      'cv_scores': array([0.98360656]),
      'avg_score': 0.9836065573770492,
      'feature_names': ('Temperature', ' RH', ' Ws', 'Rain ', 'DMC', 'FWI')},
 5: {'feature_idx': (1, 2, 3, 4, 5),
      'cv_scores': array([0.98360656]),
      'avg_score': 0.9836065573770492,
      'feature_names': (' RH', ' Ws', 'Rain ', 'DMC', 'FWI')},
 4: {'feature_idx': (2, 3, 4, 5),
      'cv_scores': array([0.98360656]),
      'avg_score': 0.9836065573770492,
      'feature_names': (' Ws', 'Rain ', 'DMC', 'FWI')},
 3: {'feature_idx': (2, 4, 5),
      'cv_scores': array([0.9795082]),
      'avg_score': 0.9795081967213115,
      'feature_names': (' Ws', 'DMC', 'FWI')}}}
```

Copy to Clipboard

You can also use `plot_sfs(sfs.get_metric_dict())` to visualize the results of sequential backward selection.

```
# Plot the accuracy of the model as a function of the number of features
plot_sfs(sbs.get_metric_dict())
plt.show()
```

Copy to Clipboard



Instructions

Checkpoint 1 Failed, try again

1.

Use the `.subsets` attribute of `sbs` to print a tuple that contains the names of the features chosen in the previous exercise (remember that we chose three features). Did you get the same feature set as you did with sequential forward selection?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `.subsets` attribute of `sbs` to print the accuracy of the model after doing sequential backward selection.

Recall that the original model was about 80% accurate when it used all nine features. How does the accuracy of the model with only three features compare? How does the accuracy of sequential backward selection compare to sequential forward selection?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use `plot_sfs(sbs.get_metric_dict())` to plot the accuracy as a function of the number of features. Notice that the accuracy sometimes decreases when a feature is removed. Do you think that too much accuracy was lost, or is that a reasonable trade-off to have a simpler model?

```
import pandas as pd
import codecademylib3
from sklearn.linear_model import LogisticRegression
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from mlxtend.plotting import plot SequentialFeatureSelector as plot_sfs
import matplotlib.pyplot as plt

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Sequential backward selection
sbs = SFS(lr,
          k_features=3,
          forward=False,
          floating=False,
          scoring='accuracy',
          cv=0)
sbs.fit(X, y)

# Evaluate the result of sequential backward selection
print(sbs.subsets_[3]['feature_names'])
print(sbs.subsets_[3]['avg_score'])

# Plot the model accuracy
plot_sfs(sbs.get_metric_dict())
plt.show()
```

Sequential Forward and Backward Floating Selection

Sequential forward floating selection is a variation of sequential forward selection. It starts with zero features and adds one feature at a time, just like sequential forward selection, but after each addition, it checks to see if we can improve performance by removing a feature.

If performance can't be improved, the floating

Preview: Docs Loading link description

[algorithm](#)

will continue to the next step and add another feature.

If performance can be improved, the algorithm will make the removal that improves performance the most (unless removal of that feature would lead to an infinite loop of adding and removing the same feature over and over again).

For example, let's say that the algorithm has just added `weight` to the feature set `{age, resting_heart_rate}`, resulting in the set `{age, weight, resting_heart_rate}`. The floating algorithm will test whether it can improve performance by removing `age` or `resting_heart_rate`. If the removal of `age` improves performance, then the algorithm will proceed with the set `{weight, resting_heart_rate}`.

Sequential backward floating selection works similarly. Starting with all available features, it removes one feature at a time. After each feature removal, it will check to see if any feature additions will improve performance (but it won't add any features that would result in an infinite loop).

Floating selection algorithms are sometimes preferable to their non-floating counterparts because they test the model on more possible feature subsets. They can detect useful relationships between

Preview: Docs Loading link description

[variables](#)

that plain sequential forward and backward selection might miss.

Instructions

Checkpoint 1 Enabled

1.

Let's say that sequential backward floating selection is being done on the set `{age, height, weight, blood_pressure, resting_heart_rate}`. The algorithm removes `weight, blood_pressure, and resting_heart_rate` in that order. After removing `resting_heart_rate`, the model had an accuracy of 86%. Now it will try some feature additions. The possible feature additions give the following accuracies:

Feature subset	Accuracy
<code>{age, height, weight}</code>	0.95
<code>{age, height, blood_pressure}</code>	0.92

Will it add a feature?

If yes, create a variable called `added_feature`, and set it equal to the name of the feature as a string.

If no, create a variable called `added_feature`, and set it equal to `None`.

Sequential Forward and Backward Floating Selection with mlxtend

We can implement sequential forward or backward floating selection in `mlxtend` by setting the

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

[parameter](#)

`floating` to `True`. The parameter `forward` determines whether `mlxtend` will use sequential forward floating selection or sequential backward floating selection. As usual, the

Preview: Docs Loading link description

[dictionary](#)

`model.subsets_` will contain useful information about the chosen features.

Here's an implementation of sequential backward floating selection.

```
# Sequential backward floating selection
sbfs = SFS(lr,
            k_features=5,
            forward=False,
            floating=True,
            scoring='accuracy',
            cv=0)
sbfs.fit(X, y)
```

Copy to Clipboard

We can use the `.subsets` attribute to look at feature names, just like we did with the non-floating sequential selection algorithms.

```
print(sbfs.subsets_[5]['feature_names'])
```

Copy to Clipboard

This outputs:

```
(' RH', ' Ws', 'Rain ', 'DMC', 'FWI')
```

Copy to Clipboard

Instructions

Checkpoint 1 Failed, try again

1.

Use `mlxtend` to create and fit a sequential forward floating selection model called `sffs` that chooses 3 features. Make sure to set `scoring='accuracy'` and `cv=0`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `.subsets` attribute of `sffs` to print a tuple that contains the names of the features chosen by sequential forward floating selection.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use `mlxtend` to create and fit a sequential backward floating selection model called `sbfs` that chooses 3 features. Make sure to set `scoring='accuracy'` and `cv=0`.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Sequential forward floating selection
sffs = SFS(lr,
            k_features=3,
            forward=True,
            floating=True,
            scoring='accuracy',
            cv=0)
sffs.fit(X, y)

# Print a tuple with the names of the features chosen by sequential forward floating
selection.
print(sffs.subsets_[3]['feature_names'])

# Sequential backward floating selection
```



```

sbfs = SFS(lr,
            k_features=3,
            forward=False,
            floating=True,
            scoring='accuracy',
            cv=0)
sbfs.fit(X, y)

# Print a tuple with the names of the features chosen by sequential backward floating
selection.
print(sbfs.subsets_[3]['feature_names'])

```

Wrapper Methods

Recursive Feature Elimination

Recursive feature elimination is another wrapper method for feature selection. It starts by training a model with all available features. It then ranks each feature according to an importance metric and removes the least important feature. The algorithm then trains the model on the smaller feature set, ranks those features, and removes the least important one. The process stops when the desired number of features is reached.

In regression problems, features are ranked by the size of the absolute value of their coefficients. For example, let's say that we trained a regression model with five features and got the following regression coefficients.

Feature	Regression coefficient
age	2.5
height	7.0
weight	-4.3
blood_pressure	-5.7
resting_heart_rate	-4.6

The regression coefficient for `age` has the smallest absolute value, so it is ranked least important by recursive feature elimination. It will be removed, and the remaining four features will be re-ranked after the model is trained without `age`.

It's important to note that you might need to standardize data before doing recursive feature elimination. In regression problems in particular, it's necessary to standardize data so that the scale of features doesn't affect the size of the coefficients.

Note that recursive feature elimination is different from sequential backward selection. Sequential backward selection removes features by training a model on a collection of subsets (one for each possible feature removal) and greedily proceeding with whatever subset performs best. Recursive feature elimination, on the other hand, only trains a model on one feature subset before deciding which feature to remove next.

This is one advantage of recursive feature elimination. Since it only needs to train and test a model on one feature subset per feature removal, it can be much faster than the sequential selection methods that we've covered.

Instructions

Checkpoint 1 Enabled

1.

Say you want to choose four out of six possible features for a model. If you use recursive feature elimination, how many different feature subsets will you test a model on? Set this number to a variable called `rfe_test_count`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

If you use sequential backward selection to choose four out of six features, how many different feature subsets will you test a model on? Set this number to a variable called `sbs_test_count`. Which method uses fewer tests?

Recursive Feature Elimination with scikit-learn

We can use scikit-learn to implement recursive feature elimination. Since we're using a logistic regression model, it's important to standardize data before we proceed.

We can standardize features using scikit-learn's `StandardScaler()`.

```
from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(X)
```

Copy to Clipboard

Once the data is standardized, you can train the model and do recursive feature elimination using `RFE()` from scikit-learn. As before with the sequential feature selection methods, you have to specify a scikit-learn model for the estimator

Preview: Docs Loading link description

[parameter](#)

(in this case, `lr` for our logistic regression model). `n_features_to_select` is self-explanatory: set it to the number of features you want to select.

```
from sklearn.feature_selection import RFE

# Recursive feature elimination
rfe = RFE(lr, n_features_to_select=2)
rfe.fit(X, y)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Standardize the data by uncommenting the line `X = StandardScaler().fit_transform(X)`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use sklearn to create an `RFE` object called `rfe` that chooses 3 features.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the `.fit()` method on `rfe` to fit the model to `X` and `y`.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
from sklearn.preprocessing import StandardScaler

# Load the data
health = pd.read_csv("dataR2.csv")
X = np.array(health.iloc[:, :-1])
y = np.array(health.iloc[:, -1])

# Standardize the data
X = StandardScaler().fit_transform(X)

# Logistic regression model
lr = LogisticRegression(max_iter=1000)

# Recursive feature elimination
rfe = RFE(estimator=lr, n_features_to_select=3)
rfe.fit(X, y)
```

Evaluating the Result of Recursive Feature Elimination

You can inspect the results of recursive feature elimination by looking at `rfe.ranking_` and `rfe.support_`.

`rfe.ranking_` is an

Preview: Docs Stores elements of various data types in an ordered collection.

[array](#)

that contains the rank of each feature. Here are the features from the `fire` dataset:

```
['Temperature', 'RH', 'Ws', 'Rain', 'DMC', 'FWI']
```

Copy to Clipboard

Here are the feature rankings after recursive feature elimination is done on the `fire` dataset.

```
print(rfe.ranking_)
```

Copy to Clipboard

```
[2 5 4 1 3 1]
```

Copy to Clipboard

A `1` at a certain

Preview: Docs Loading link description

[index](#)

indicates that recursive feature elimination kept the feature at the same index. In this example, the model kept the features at indices 3 and 5: `Rain` and `FWI`. The other numbers indicate at which step a feature was removed. The `5` (the highest rank in the array) at index 1 means that `RH` (the feature at index 1) was removed first. The `4` at index 2 means that `Ws` (the feature at index 2) was removed in the next step, and so on.

`rfe.support_` is an array with `True` and `False` values that indicate which features were chosen. Here's an example of what this looks like, again using the `fire` dataset.

```
print(rfe.support_)
```

Copy to Clipboard

```
[False False False  True False  True]
```

Copy to Clipboard

This array indicates that the features at indices 3 and 5 were chosen. The features at indices 0, 1, 2, and 4 were eliminated.

If you have a list of feature names, you can use a list comprehension and `rfe.support_` to get a list of chosen feature names.

```
# features is a list of feature names
# Get a list of features chosen by rfe
rfe_features = [f for (f, support) in zip(features, rfe.support_) if support]

print(rfe_features)
```

Copy to Clipboard

```
['Rain ', 'FWI']
```

Copy to Clipboard

You can use `rfe.score(X, y)` to check the accuracy of the model.

Instructions

Checkpoint 1 Failed, try again

1.

Create a list called `rfe_features` that contains the names of the features chosen by recursive feature elimination.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the accuracy of the model after doing recursive feature elimination. How does this compare to the accuracy of the original model?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
from sklearn.preprocessing import StandardScaler

# Load the data
health = pd.read_csv("dataR2.csv")
X = health.iloc[:, :-1]
y = health.iloc[:, -1]

# Create a list of feature names
feature_list = list(X.columns)

# Standardize the data
X = StandardScaler().fit_transform(X)

# Logistic regression
lr = LogisticRegression(max_iter=1000)

# Recursive feature elimination
rfe = RFE(estimator=lr, n_features_to_select=3)
rfe.fit(X, y)

# List of features chosen by recursive feature elimination
rfe_features = [f for (f, support) in zip(feature_list, rfe.support_) if support]

print(rfe_features)
```

```
# Print the accuracy of the model with features chosen by recursive feature elimination
print(rfe.score(X, y))
```

```
['BMI', 'Glucose', 'Resistin']
0.7327586206896551
```

Review

Congratulations! You've learned about a few different wrapper methods. You've also learned how to implement them in Python and evaluate the results. Let's recap what we covered.

Wrapper methods for feature selection are algorithms that select features by evaluating the performance of a

Preview: Docs Loading link description

[machine learning](#)

model on different subsets of features. Here are some advantages of wrapper methods.

- They can detect relationships between features that might be relevant to the machine learning model.
- Unlike filter methods, they are designed to choose features that are relevant to whatever machine learning model you are using.

We covered four different greedy wrapper methods and implemented them using `mlex` in Python.

- Sequential forward selection adds one feature at a time.
- Sequential backward selection removes one feature at a time.
- Sequential forward floating selection adds (and sometimes removes) one feature at a time.
- Sequential backward floating selection removes (and sometimes adds) one feature at a time.

We also covered recursive feature elimination, which ranks features by importance and removes the least important feature at every step. We used the `scikit-learn` library to implement that

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

and investigate the results.

The forest fire data for this lesson were taken from the [UCI Machine Learning Repository](#) Faroudja ABID et al., Predicting Forest Fire in Algeria using Data Mining Techniques: Case Study of the Decision Tree Algorithms, International Conference on Advanced Intelligent Systems for Sustainable Development (AI2SD 2019) , 08 - 11 July , 2019, Marrakech, Morocco.

The breast cancer data for this lesson were taken from the [UCI Machine Learning Repository](#). [Patricio, 2018] Patrício, M., Pereira, J., Crisóstomo, J., Matafome, P., Gomes, M., Seíça, R., & Caramelo, F. (2018). Using Resistin, glucose, age and BMI to predict the presence of breast cancer. BMC Cancer, 18(1).

Underfitting | Overfitting

Regularization Methods in Python

Why Regularize?

Regularization plays a very important role in real world implementations of

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

models. It is a statistical technique that minimizes overfitting and is executed during the model fitting step. It is also an embedded feature selection

Preview: Docs Loading link description

[method](#)

because it is implemented while the parameters of the model are being calculated.

Machine learning algorithms are rarely deployed to production without using some form of regularization.

The reason for this is as follows: In practice, every model has to deal with the question of how well it can generalize from known to unknown data. We can train, test and tune models on known data and make them as accurate as possible. However, in deploying models, we are applying them on new data.

Regularization makes sure that our model is still accurate.

In this lesson, we are going to learn how regularization minimizes overfitting and how to use it as a feature selection method. Along the way, we are also going to learn two concepts that are very relevant to regularization and important just as standalone topics within machine learning, namely, *the bias-variance tradeoff* and *hyperparameter tuning*. But first, we're going to answer the question: what is overfitting?

Instructions

Checkpoint 1 Enabled

1.

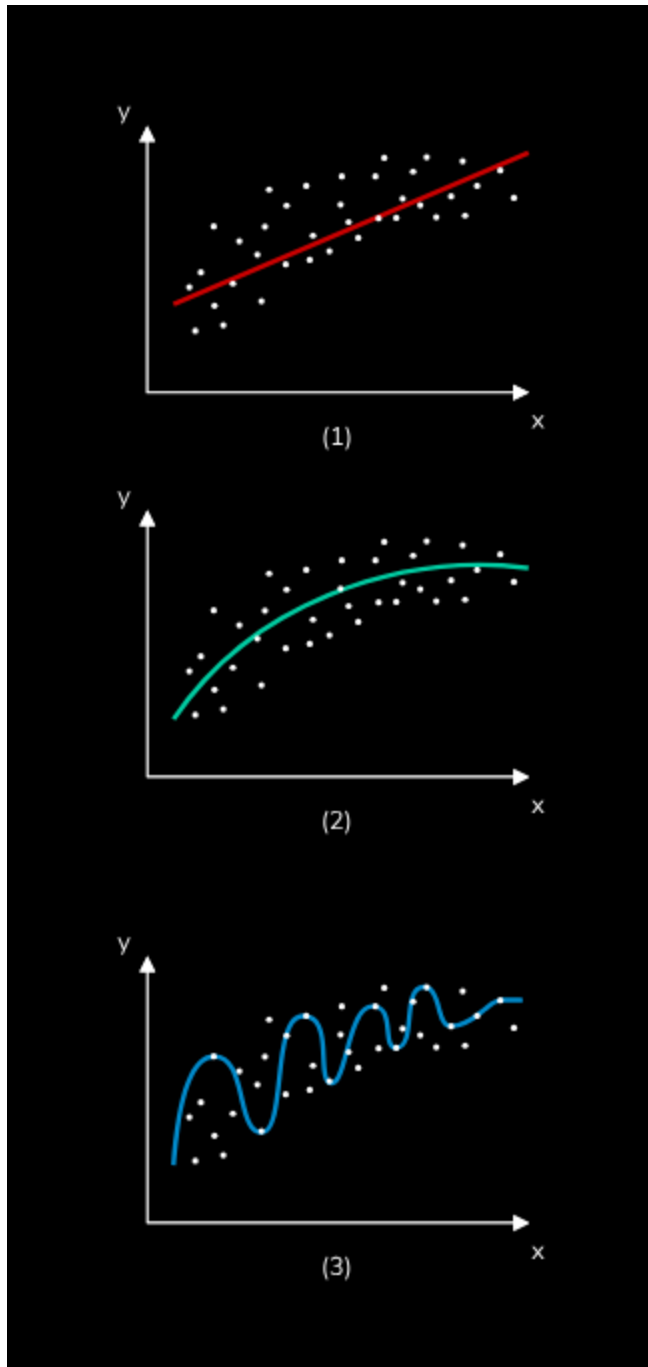
In the image to the right, we have three curves generated by different model fits to the same data. Can you identify which of these might be:

underfitting the data?

overfitting the data?

Fill in the number corresponding to the plots that best matches these on the code editor.

The curve that is least representative of the data (1) is likely underfitting the data and the curve that seems to pass through most of the datapoints but seems to be sensitive to small variations (3) is likely overfitting the data.



What is Overfitting?

It might have been pretty apparent which of the three curves in the previous exercise overfit the set of points. However, often we have too many features to be able to visually assess if our model is overfitting our data or not. And so we need to qualify what we mean by overfitting exactly.

If a model is able to represent a particular set of data points effectively but is not able to fit new data well, it is overfitting the data. Such a model has one or more of the following attributes:

- It fits the training data well but performs significantly worse on test data
- It typically has more parameters than necessary, i.e., it has *high model complexity*
- It might be fitting for features that are multi-collinear (i.e., features that are highly negatively or positively correlated)
- It might be fitting the noise in the data and likely mistaking the noise for features

In practice, we often catch overfitting by comparing the model performance on the training data versus test data. For instance if the R-squared score is high for training data but the model performs poorly on test data, it's a strong indicator of overfitting. We're going to look at one such example now. A dataset has been loaded for you from the [UCI Machine Learning Repository](#) that describes the performance in mathematics of students from two Portuguese schools. We're going to implement a multiple linear regression model to predict the final grade of the students based on a number of features in the dataset.

Instructions

Checkpoint 1 Enabled

1.

We've implemented some code to print the first five rows, set the target variable as `y` and the predictor variables as the matrix `x`. Calculate the number of features in the dataset, save it as `num_features` and press "Run". (Note that the number of features is equal to the number of columns in `x`.)

Set `num_features` equal to `len(X.columns)`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We're about to fit the data to a model with 42 parameters! (Intercept plus 41 coefficients, one for each predictor variable.) This might already make us apprehensive of the model potentially overfitting the data because:

- the model likely has a high degree of complexity
- we haven't checked for collinearity between the features

In order to assess whether we are overfitting, we're going to fit the model. In the workspace, you'll see that we've provided code to split the data into a training and test set, fit a regression on all 41 features, and calculated the mean squared error (MSE) on the training data.

In the section below the comment `# 2. Testing Error`, fill in the code to calculate the MSE for the test set. Save the result as `MSE_test` (currently set to `None`).

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We see that the error on the test data is almost double the error with the training data! This is definitely an indicator of overfitting. We're now going to check the coefficients evaluated by the model to see if we can catch this.

Uncomment the plotting code below the comment `# 3: Plotting the Coefficients` and press "Run" to plot values of the coefficients produced.

We see that there are a few negatively correlated coefficients here (coefficients with roughly similar values mirrored about the axis), which is also a sign of overfitting.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt

df = pd.read_csv("./student_math.csv")
print(df.head())

#setting target and predictor variables
y = df['Final_Grade']
X = df.drop(columns = ['Final_Grade'])
```



```

# 1. Number of features
num_features = None
print("Number of features: ", num_features)

#Performing a Train-Test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)

#Fitting a Linear Regression Model
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)

#Training Error
pred_train = model.predict(X_train)
MSE_train = np.mean((pred_train - y_train)**2)
print("Training Error: ", MSE_train)

# 2. Testing Error
pred_test = model.predict(X_test)
MSE_test = None
print("Testing Error: ", MSE_test)

#Calculating the regression coefficients
predictors = X.columns
coef = pd.Series(model.coef_, predictors).sort_values()

# 3. Plotting the Coefficients

# plt.figure(figsize = (15,10))
# coef.plot(kind='bar', fontsize = 20)
# plt.title ("Regression Coefficients", fontsize = 30)
# plt.show()

```

The Loss Function

Let's revisit how the coefficients (or parameters) of a linear regression model are obtained. Suppose we want to fit a linear regression with two features, x_1 and x_2 :

$$y = b_0 + b_1x_1 + b_2x_2 + e$$

We solve for the intercept b_0 and coefficients b_1 and b_2 by minimizing the mean squared error. This is also referred to as the **loss function** (or alternately, as the *objective function* or *cost function*) in machine learning:

$$\text{Loss function} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1x_{1i} - b_2x_{2i})^2$$

The loss function is minimized by the

Preview: Docs Loading link description

[method](#)

of Ordinary Least Squares (OLS) and for larger datasets, the gradient descent

Preview: Docs Loading link description

[algorithm](#)

is used to arrive at the minimum. As we have seen already however, the “best fit” as obtained by minimizing the loss function might not be the fit that generalizes well to new data. Regularization modifies the loss function in a way that might help with this.

Before going into more detail, we're going to plot the loss function as a function of the coefficients b_1 and b_2 . For this, we're going to generate a dataset for two-feature linear regression using the `datasets` module from `scikit-learn`. The default data generated by this module is centered, meaning that our intercept, b_0 is already set to zero.

Instructions

Checkpoint 1 Enabled

1.

We've written some code here to generate our data for the two feature scenario. The `n_samples` input parameter lets us set how many data points we'd like -- we've set it to 100. `n_features` lets us specify the number of features we want, which would be 2 in our case.

The `coef = True` attribute gives us the coefficients generated by fitting a linear regression model. Print the coefficients given by this model. (Remember that intercept `b0` is zero as the data is centered - so we're printing the best fit values of `b1` and `b2` here!)

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We're going to write a pythonic function to output the value of the loss function at each data point for all possible values of `b1` and `b2`. Fill in the code for the `loss_function()` so that it outputs loss according to the formula below.

$$\text{Loss function} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2$$

Note that we've done some of the work for you, so you just need to fill in the code to calculate the `error` variable inside the function.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

A script `plot_loss.py` has been loaded for you in the workspace. It takes in the values of the coefficients as inputs to a function to output the loss as a contour plot. Uncomment the lines below `# 3. Plot loss function for data` and press "Run" to see what this looks like!

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Each ellipse in this plot represents the points where the loss function has the *same value*. The larger the ellipse, the higher the value of loss for all the points (`b1, b2`) that fall on the ellipse, as we can see from the labels on the contours.

Following this logic, the center of this set of concentric ellipses should represent the minimum value of the loss function and the value of the coefficients (`b1, b2`) at this point should be the solution of the regression problem and represent the best fit for our data. Let's check if this is so by plotting the coefficients we had obtained earlier on! Complete the code under `# 4. Plot the best fit coefficients` to overplot the best fit coefficient values. (You will have to scroll down to see the new plot!)

```
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt

from sklearn import datasets
data, y, coefficients = datasets.make_regression(n_samples = 100, n_features = 2, coef =
True, random_state = 23)
x1 = data[:,0]
x2 = data[:,1]

# 1. Print the coefficients
print(coefficients)

# 2. Loss Function
def loss_function(b1,b2,y,x1,x2):
    error = y - b1*x1 - b2*x2
    loss = np.mean(error**2)
    return loss

# 3. Plot loss function for data
from plot_loss import plot_loss_function
b1 = np.linspace(-150, 150, 501)
b2 = np.linspace(-150, 150, 501)
contour_plot = plot_loss_function(b1,b2,y,x1,x2)
plt.show()
```

```
# 4. Plot the best fit coefficients
best_fit_b1 = coefficients[0]
best_fit_b2 = coefficients[1]
plt.scatter(best_fit_b1, best_fit_b2, s = 50, color = 'green')
plt.show()
```

The Regularization Term

<https://www.codecademy.com/paths/data-science/tracks/dsml-feature-engineering-for-ds/modules/fe-regularization/lessons/fe-intro-regularization-ml/exercises/the-regularization-term>

Regularization penalizes models for overfitting by adding a “penalty term” to the loss function. The two most commonly used ways of doing this are known as *Lasso* (or *L1*) and *Ridge* (or *L2*) regularization. (We’ll explain the meaning behind their names in a moment!)

Both of these rely on penalizing overfitting by controlling how large the coefficients can get in the first place. The penalty term or regularization term is multiplied by a factor `alpha` and added to the old loss function as follows:

New loss function = Old loss function + α * Regularization term

New loss function = Old loss function + α * Regularization term

Because of the additive term, minimizing the new loss function will necessarily mean “true loss” goes up, i.e., the scores on this will be lower on the training data than regression without regularization! But this is what we *want* when we are regularizing. Remember that the reason we’re regularizing is because our model is overfitted to our data (i.e., it is performing well on training data but doesn’t generalize well on test data).

The regularization term is the sum of the absolute values of the coefficients in the case of L1 regularization and the sum of the squares of the coefficients in the case of L2.

L1 Regularization term: $|b_1| + |b_2|$

L2 Regularization term: $b_1^2 + b_2^2$

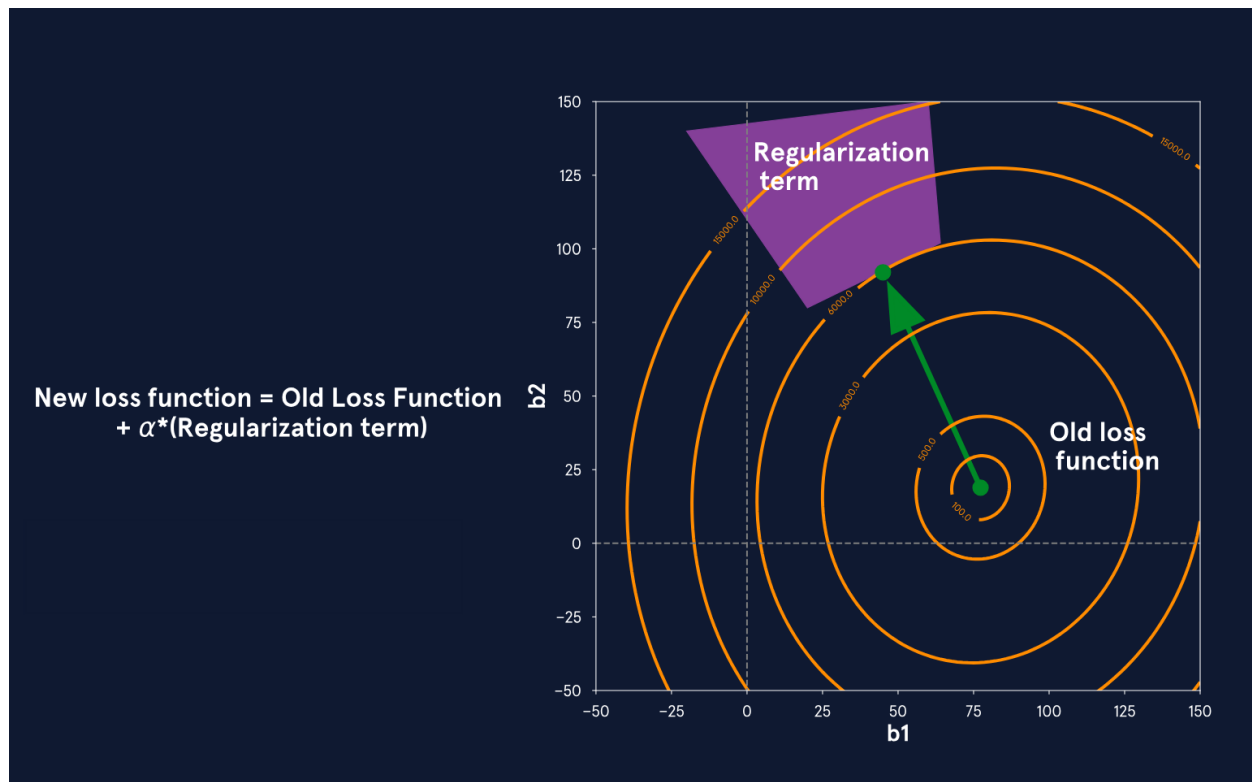
In mathematics, the sum of the magnitudes of a vector is known as its L1 norm (related to “Manhattan distance”) and the square root of the sum of the magnitudes (or the “Euclidean distance” from the origin) is known as its L2 norm - and that is the reason for the names of both methods!

Instructions

Look at the figure next to the narrative panel. The old loss function for the two-variable linear regression scenario is represented by the orange contours. The coefficients (b1,b2) that minimize this loss function is represented by the green dot at the center of the contours.

Suppose the regularization term constrains the coefficients to values within the pink surface. (Typically, the shape of this surface will depend on the exact mathematical expression for the regularization term. The size of the constraint will be determined by α).

The best fit (b1,b2) that minimizes the new loss function has to minimize both the old loss function as well as the regularization term. It thus shifts to a point within the pink trapezium as shown by the arrow. The value of the old loss function at this point is no longer zero as we can see from the contour line closest to it.



L1 or Lasso Regularization

<https://www.codecademy.com/paths/data-science/tracks/dsml-feature-engineering-for-ds/modules/fe-regularization/lessons/fe-intro-regularization-ml/exercises/l1-or-lasso-regularization>

In the case of the two-feature linear regression scenario we were looking at in the previous exercise, our new loss function for L1 regularization looks as follows:

$$L1 \text{ Loss} = \ln \sum_i (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha * (|b_1| + |b_2|)$$

Minimizing this new loss function essentially means restricting the size of the regularization term while minimizing the old loss function. Let's say that our regularization term can take a maximum value, s :

$$|b_1| + |b_2| \leq s$$

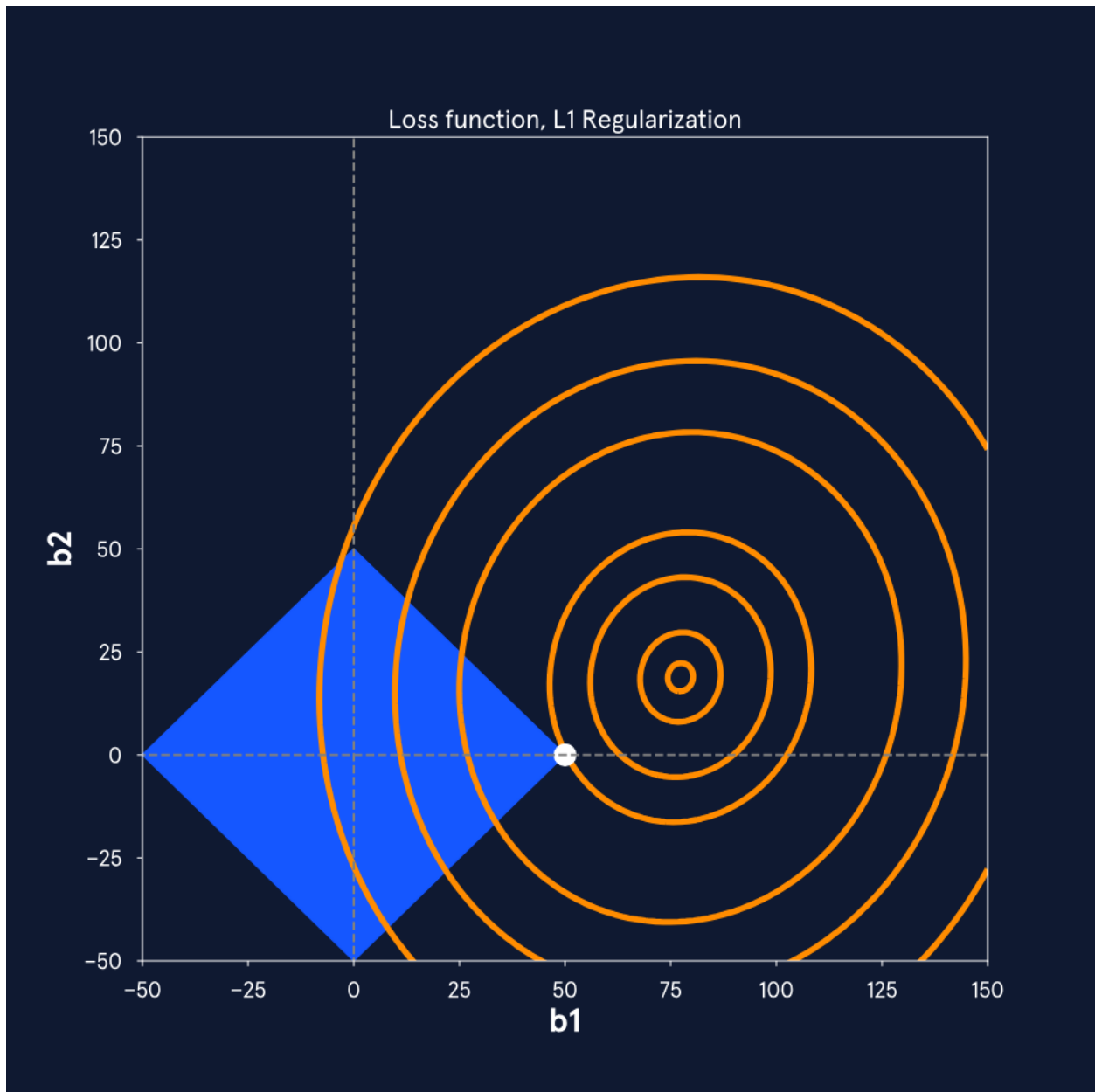
This is equivalent to confining our coefficients to a surface around the origin bounded by 4 lines : $b_1 + b_2 = s$, $b_1 - b_2 = s$, $b_2 - b_1 = s$ and $-b_1 - b_2 = s$.

The figure to the right replicates the elliptical contours for the loss function that we'd plotted earlier in the lesson. If we plot these four lines as with b_1 and b_2 as X and Y axes respectively, we get the diamond shaped blue shaded region that we've shown here. We have chosen a value of $s = 50$ for this figure - this means that either coefficient can have a maximum value of 50. The choice of s is deeply tied to the choice of α as they are inversely related.

The value of (b_1, b_2) that satisfies the regularization constraint while minimizing the loss function is denoted by the white dot. Notice that it is exactly at the tip of the diamond where it intersects with a contour from our loss function. It also happens to fall exactly on the X axis, thus setting the value of b_2 to 0!

Why is this the point that minimizes the new loss function? Remember that the goal here is to minimize the original loss function *while* meeting the regularization constraint. We still want to be as close to the center of the contours (the original loss function minimum) as possible. The point that's closest to the center of the contours *while* simultaneously lying within the regularization surface boundary is the white dot!

The word Lasso is actually an acronym for “Least Absolute Shrinkage and Selection Operator” - it shrinks the absolute value of coefficients and selects parameters by way of setting some of the coefficients to zero. If the coefficient of a regression model is set to zero, it means that we’ve pretty much eliminated the feature associated with it. In models with a high number of features, lasso regularization tends to set a significant fraction of its parameters to zero, thus acting as a feature selection



Lasso Regularization (contd.)

We've covered the basics of how Lasso regularization minimizes overfitting and simultaneously acts as a feature selection

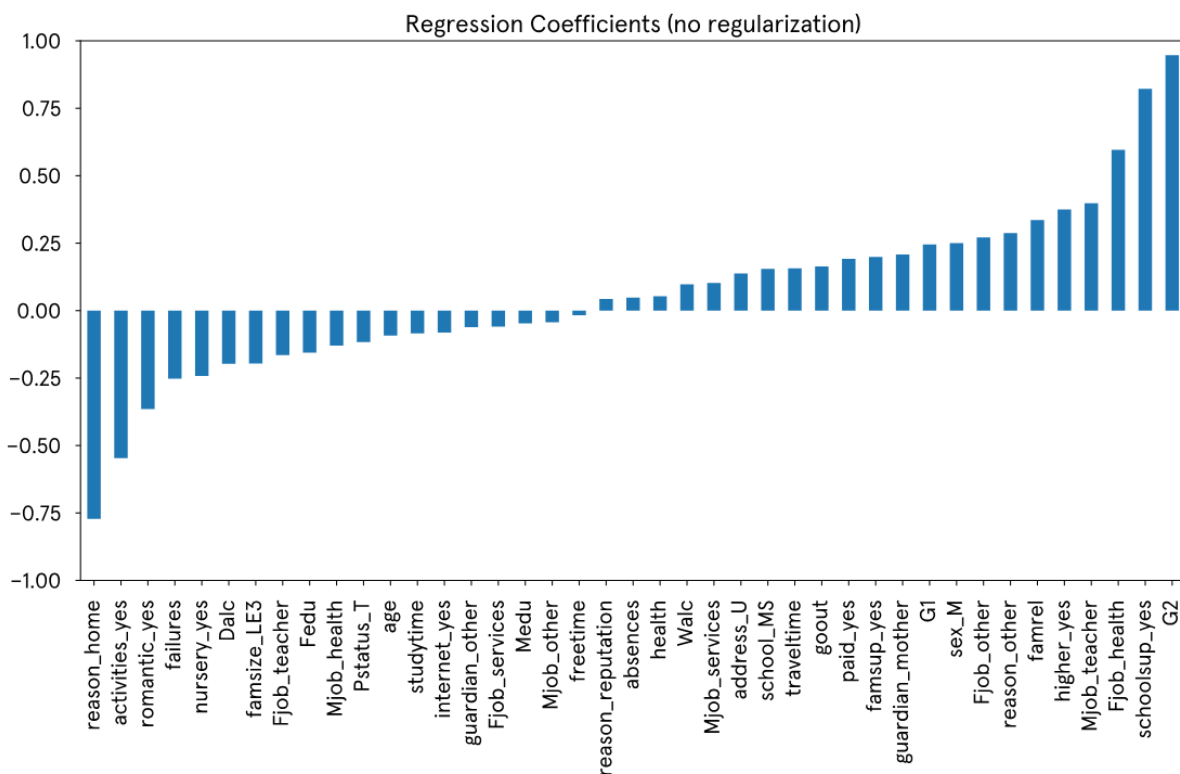
Preview: Docs Loading link description

[method](#)

using a two-feature example. How does this extend to a multiple feature dataset? The loss function for a multiple linear regression case with m features looks as follows: (You can use the horizontal scroll bar here to view the entire equation!)

$$L1 \text{ Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i} - \dots - b_m x_{mi})^2 + \alpha * (|b_1| + |b_2| + |b_3| + \dots + |b_m|)$$

We're going to examine this by reapplying a multiple linear regression model to the student performance dataset we were looking at earlier on in the lesson — only this time, we are going to do this with L1 regularization. A quick reminder that our original unregularized coefficients look as shown in the image below:



We've loaded a file, `script.py` here containing the analysis we did earlier on. Some of the things we'd noted earlier on were:

- too many features possibly (from the number of columns in the DataFrame)
- testing error is higher than training error (by looking at the `MSE`)
- highly negatively correlated features (inferred from the plot of the coefficients)

Instructions

Checkpoint 1 Enabled

1.

We can implement Lasso for this dataset by importing the `Lasso` module within `scikit-learn`'s class of `linear_models`. We're choosing a value of `s` by setting `alpha` to 0.1 - the `scikit-learn` default is 1. We've written some code in `script.py` to do a train-test split, implement regression with Lasso regularization and calculate the training error. Press `Run` to view the result.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Following the syntax for training error, calculate the testing error using the following steps:

set the variable `l1_pred_test` equal to the result of applying the model to `X_test`

set the variable `l1_mse_test` equal to the Mean Squared Error of the test data

print `l1_mse_test`

The training and test Mean Squared Errors obtained from implementing the regression without regularization were 2.627 and 4.987 (rounded to three decimals!) respectively. How do our new errors compare?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We see how our training error went up but our test error went down, which makes sense as our model is performing slightly better on test data at the cost of performing slightly worse on training data. What do our coefficients look like post Lasso regularization? Uncomment the code below `# 3. Plotting the Coefficients`, add a `plt.show()` and press "Run".

Lasso has shrunk more than half our coefficients to zero! Additionally, while the value of all the coefficients have shrunk, Lasso has increased the relative importance of some. An important thing to note here is that we manually set the value of `alpha` to 0.1. The number of features that get eliminated due to Lasso is definitely tied to the value of `alpha` - we're going to examine this more after we cover the basics of L2 or Ridge regularization.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import helpers

df = pd.read_csv("./student_math.csv")
y = df['Final_Grade']
X = df.drop(columns = ['Final_Grade'])

# 1. Train-test split and fitting an l1-regularized regression model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
lasso = Lasso(alpha = 0.1)
lasso.fit(X_train, y_train)

l1_pred_train = lasso.predict(X_train)
l1_mse_train = np.mean((l1_pred_train - y_train)**2)
print("Lasso (L1) Training Error: ", l1_mse_train)

# 2. Calculate testing error
l1_pred_test = lasso.predict(X_test)
l1_mse_test = np.mean((l1_pred_test - y_test)**2)
print("Lasso (L1) Testing Error: ", l1_mse_test)

# 3. Plotting the Coefficients
predictors = X.columns
coef = pd.Series(lasso.coef_,predictors).sort_values()
plt.figure(figsize = (12,8))
plt.ylim(-1.0,1.0)
coef.plot(kind='bar', title='Regression Coefficients with Lasso (L1) Regularization')
plt.show()
```

L2 or Ridge Regularization

L2 regularization is also called “Ridge” regularization as the mathematical formulation of it belongs to a class of analysis methods known as “ridge analysis”. The modified loss function in the case of L2 regularization looks as follows:

$$L2 \text{ Loss} = \ln \sum_i \ln(y_i - b_0 - b_1 x_i - b_2 x_i^2)$$

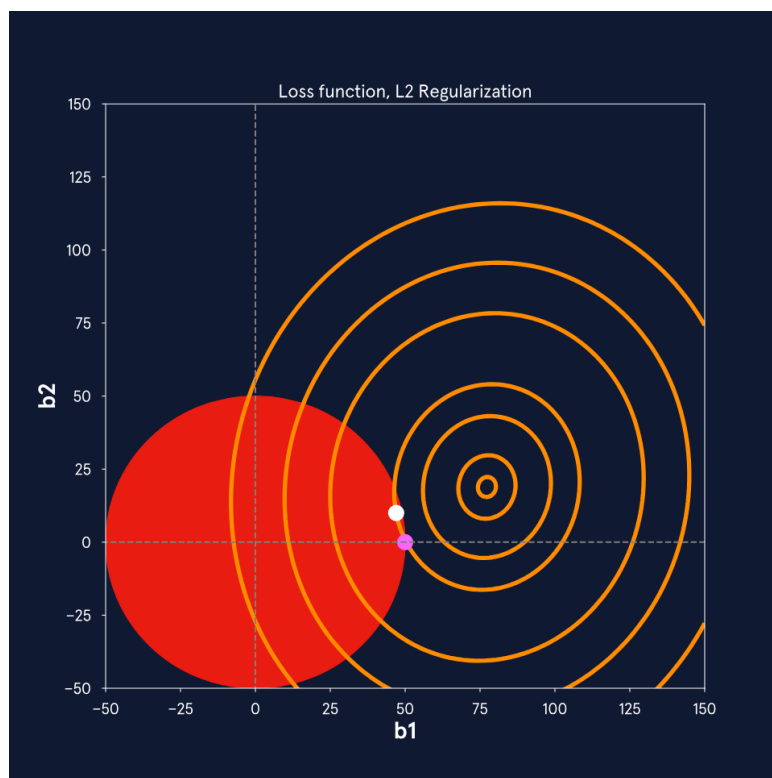
Similar to L1, this would mean constraining the regularization term to the following surface:

$$b_1^2 + b_2^2 \leq s^2$$

The key difference here is that instead of a diamond-shaped area, we’re constraining the coefficients to live within a circle of radius s as shown in the figure here. The general goal is still similar though, we want to minimize the old loss while restricting the values of the coefficients to the boundary of this circle. Once again we want our new coefficient values to be as close to the unregularized best fit solution (i.e., the center of the contours) as possible while falling within the circle.

The value of (b_1, b_2) that minimizes this new loss function almost never lies on either axes. The solution here is **not** the pink dot that lies on the X axis like in L1, but rather the white dot that we have shown. The reason for this is as follows: The circle that contains the white and pink dots represents the smallest value of the old loss function that satisfies the regularization constraint, but while the pink dot makes the regularization term’s value s^2 exactly, the white dot makes it even smaller as it lies *inside* the circle!

Unlike Lasso, our Ridge coefficients can never be exactly zero! L2 regularization is therefore not a feature elimination like L1. The coefficients of L2 get arbitrarily small but never zero. This is particularly useful when we don’t want to get rid of features during modeling but nonetheless want their relative importances emphasized.

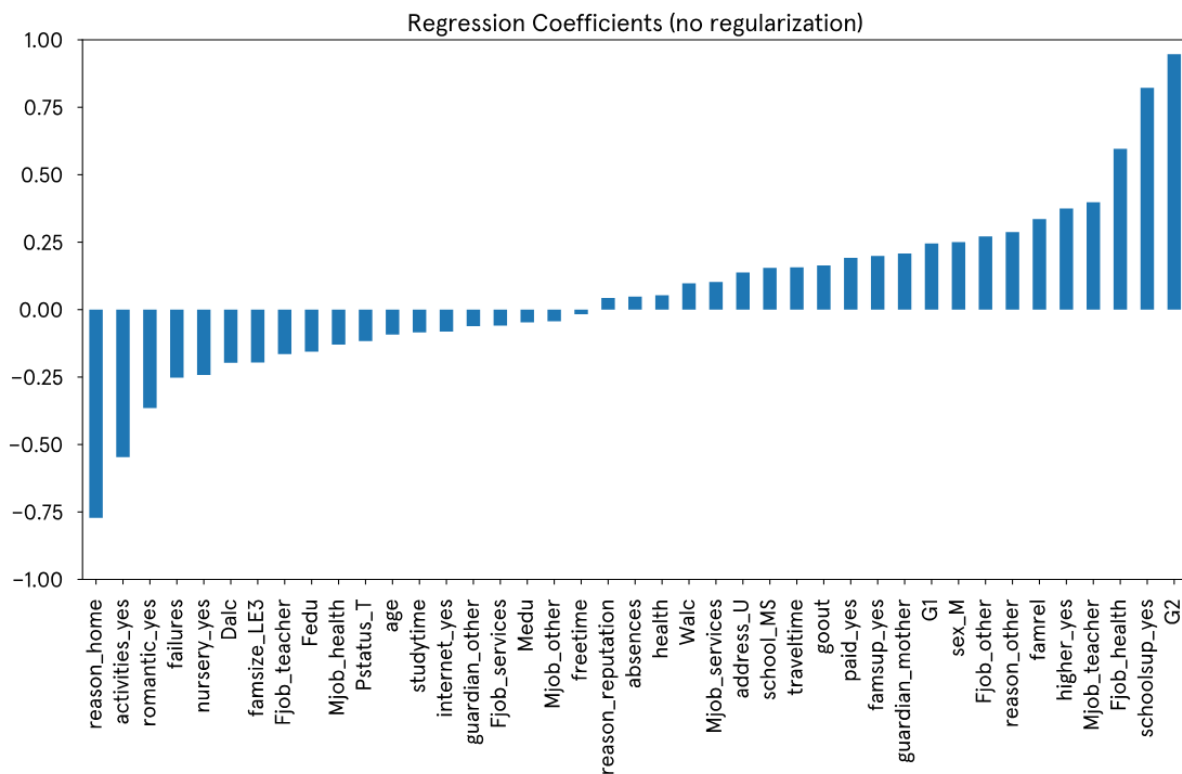


Ridge Regularization (contd.)

We're now going to apply L2 regularization on the student performance dataset we've been working with to see how it works in a many-feature scenario. The loss function for a Multiple Linear Regression model with L2 regularization with m features looks as follows: (You can use the horizontal scroll bar here to view the entire equation!)

$$L2 \text{ Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{i1} - b_2 x_{i2} - \dots - b_m x_{im})^2 + \frac{\lambda}{2} (b_1^2 + b_2^2 + \dots + b_m^2)$$

We're going to examine this by reapplying a multiple linear regression model to the student performance dataset we were looking at earlier on in the lesson – only this time, we are going to do this with L2 regularization. A quick reminder that our original unregularized coefficients look as shown in the image below:



We've loaded a file, `script.py` here containing the analysis we did earlier on. Some of the things we'd noted earlier on were:

- too many features possibly (from the number of columns in the DataFrame)

- testing error is higher than training error (by looking at the `MSE`)

- highly negatively correlated features (inferred from the plot of the coefficients)

Instructions

Checkpoint 1 Enabled

1.

Similar to our L1 exercise, we've loaded a file, `script.py` here containing the analysis we did earlier on. We can implement Ridge regularization for this dataset using `scikit-learn` by importing the `Ridge` module within `linear_models`. We're setting `s` by way of setting `alpha` to 100 - the `scikit-learn` default is 1. (We'll delve deeper into how to choose the right `alpha` in the upcoming exercise!)

We're going to evaluate our training and test error and the error just as we did last time. Following the syntax for training error, calculate the testing error using the following steps:

```
set the variable l2_pred_test equal to the result of applying the model to  
X_test  
set the variable l2_mse_test equal to the Mean Squared Error of the test  
data  
print l2_mse_test
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The training and test Mean Squared Errors obtained from implementing the regression without regularization were 2.627 and 4.987 (rounded to three decimals!) respectively. We see how our training error went up but our test error went down, which makes sense as our model is performing slightly better on test data at the cost of performing slightly worse on training data.

What do our coefficients look like post ridge regularization? Uncomment the code below

3. Plotting the Coefficients, add a `plt.show()` and press "Run".

We see how L2 regularization shrinks the coefficients as well, and while there are some coefficients that are very small, **no coefficient is set to zero**. The amount by which the feature coefficients get shrunk is related to the `alpha` we choose in implementing regularization. We're now ready to learn the things to consider in choosing `alpha`.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import helpers

df = pd.read_csv("./student_math.csv")
y = df['Final_Grade']
X = df.drop(columns = ['Final_Grade'])

# 1. Train-test split and fitting an l2-regularized regression model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
ridge = Ridge(alpha = 100)
```

```

ridge.fit(X_train, y_train)

#Training error
l2_pred_train = ridge.predict(X_train)
l2_mse_train = np.mean((l2_pred_train - y_train)**2)
print("Ridge (L2) Training Error: ", l2_mse_train)

# 2. Calculate testing error
l2_pred_test = ridge.predict(X_test)
l2_mse_test = np.mean((l2_pred_test - y_test)**2)
print("Ridge (L2) Testing Error: ", l2_mse_test)

# 3. Plotting the Coefficients
predictors = X.columns
coef = pd.Series(ridge.coef_,predictors).sort_values()
plt.figure(figsize = (12,8))
plt.ylim(-1.0,1.0)
coef.plot(kind='bar', title='Regression Coefficients with Lasso (L1) Regularization')
plt.show()

```

Hyperparameter Tuning

α is what is known as a *hyperparameter* in [machine learning](#).

It is not learned during the model fitting step the way model parameters are. Rather, it's chosen prior to fitting the model and is used to control the learning process. In regularization, the choice of α determines how much we want to control for overfitting during the model fitting step.

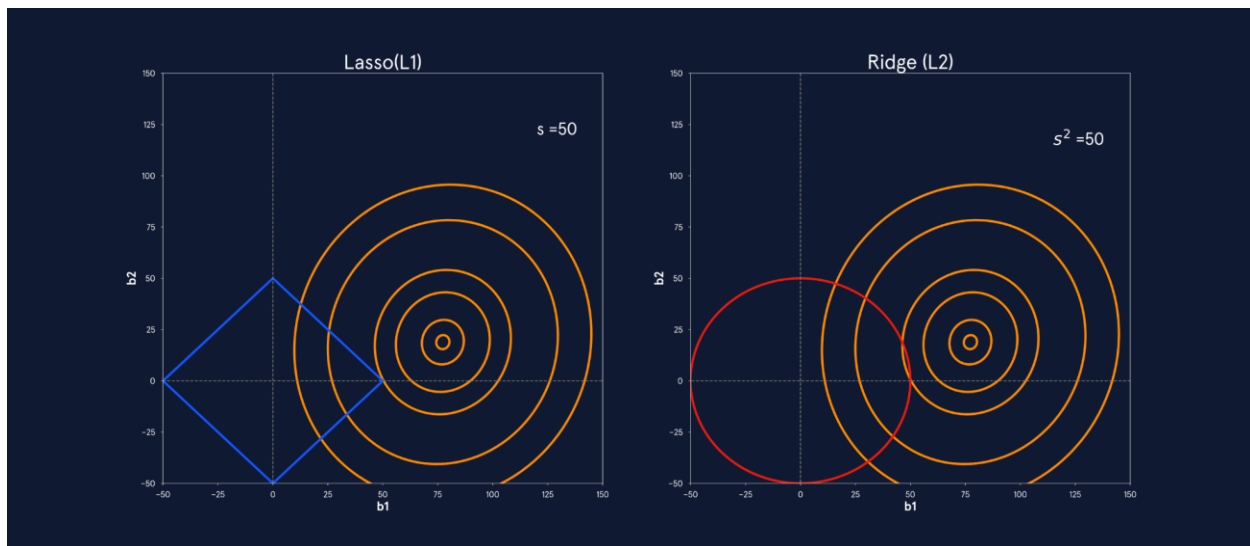
We're able to do this by using α to control the size of the constraint surface and consequently the size of the coefficients themselves. The larger the α , the smaller the size of the allowed coefficients. In essence, α is **inversely proportional** to s in the case of L1 regularization or s^2 in the case of L2 regularization.

Consider all possible choices in terms of the size of the constraint boundaries for both types of regularization. In the figures shown to the right, the blue diamonds and the red circles represent the possible choices for s . As we can see, the button "Less Regularization" increases s and the button "More Regularization" decreases s .

If s is very large (i.e., α is very small), the unregularized loss function minimum can easily fall within this large boundary and thus make it similar to regression without regularization.

If s is very small (i.e., α is very large), the regression coefficients become very small and the loss value for the best fit becomes large making the regression over-regularized.

To avoid either of these extreme scenarios, we need to find the *right amount of regularization* by tuning α and this process is known as **hyperparameter tuning**.



The Bias-Variance Tradeoff

When we add the regularization term to our loss function, we are in essence introducing **bias** into our problem, i.e., we are biasing our model to have coefficients within the regularization boundary. The greater the `alpha`, the smaller the coefficients and the more biased the model.

The animation to the right shows the coefficients obtained from both types of regularization on the student performance dataset we've been working with throughout the lesson. Using the navigation buttons, we can now see what happens to the coefficients when we increase/decrease `alpha`. For very high values of `alpha`, while Ridge begins to make most of the coefficients very small, Lasso ends up eliminating all but one feature!

Recall that the reason we wanted to perform regularization was to prevent our model from overfitting the data. Such a model is said to have high **variance** as it is likely fitting for random errors or noise within the data. We introduced the bias term to minimize the variance, but if we're not careful and allow it to get arbitrarily large, we run the risk of underfitting the model!

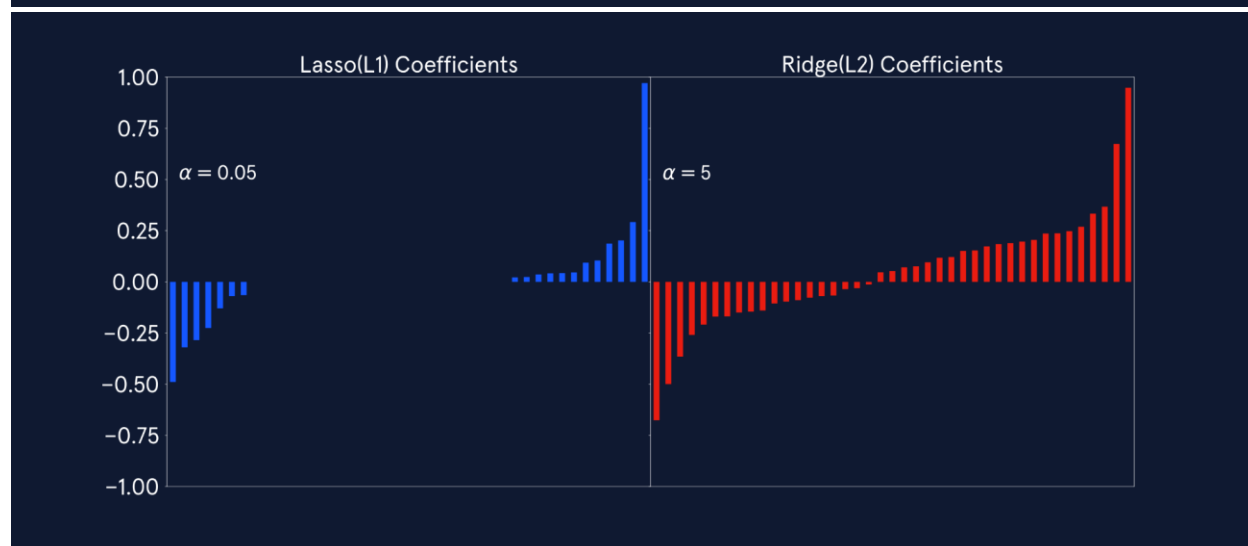
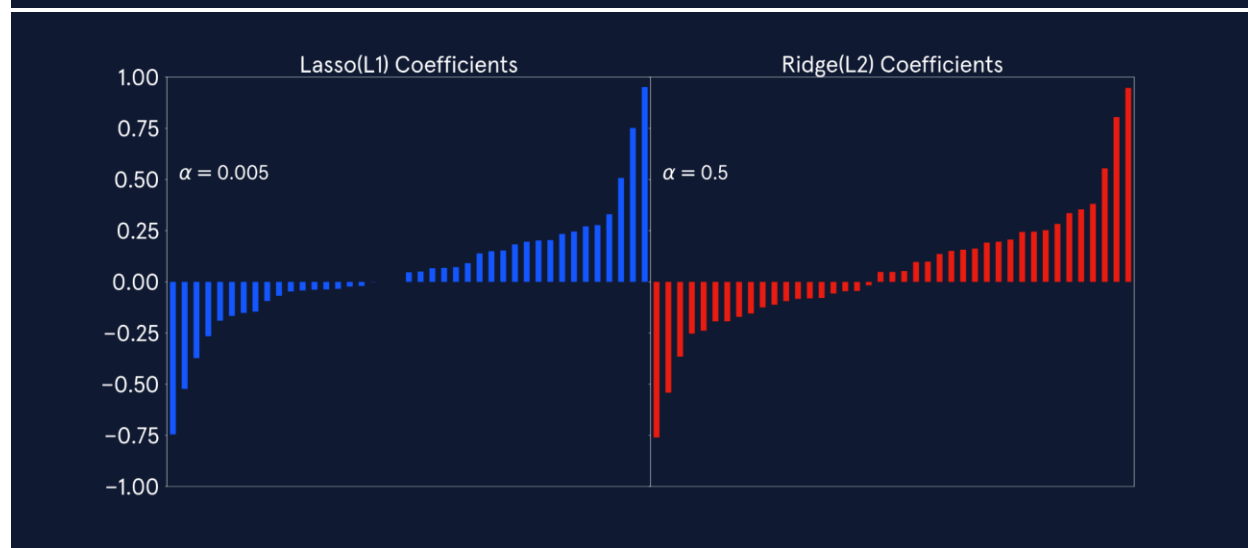
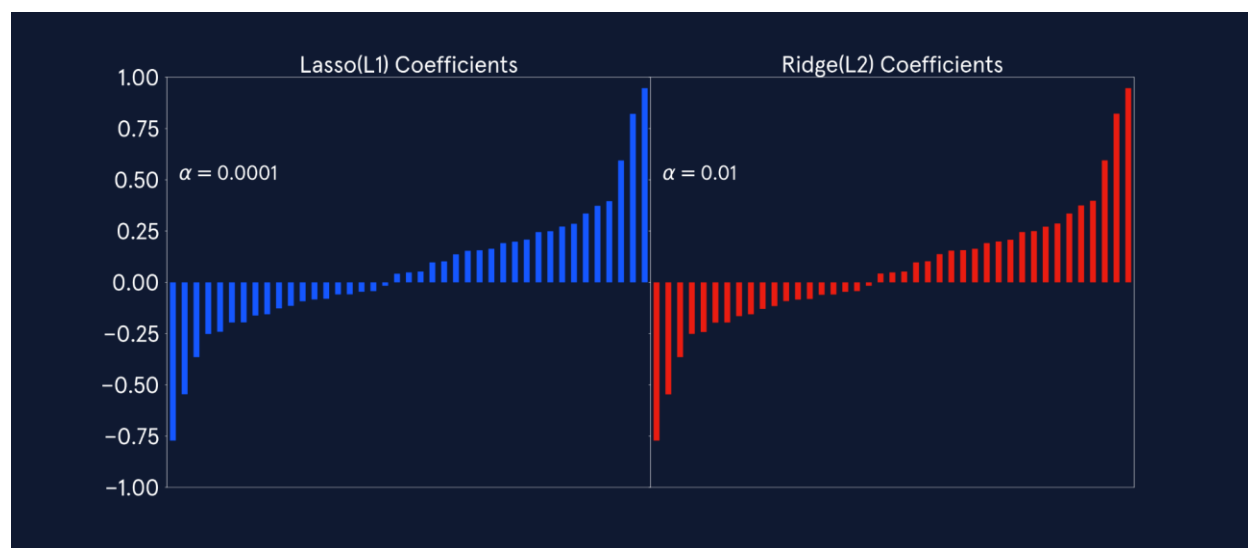
Ideally we want a

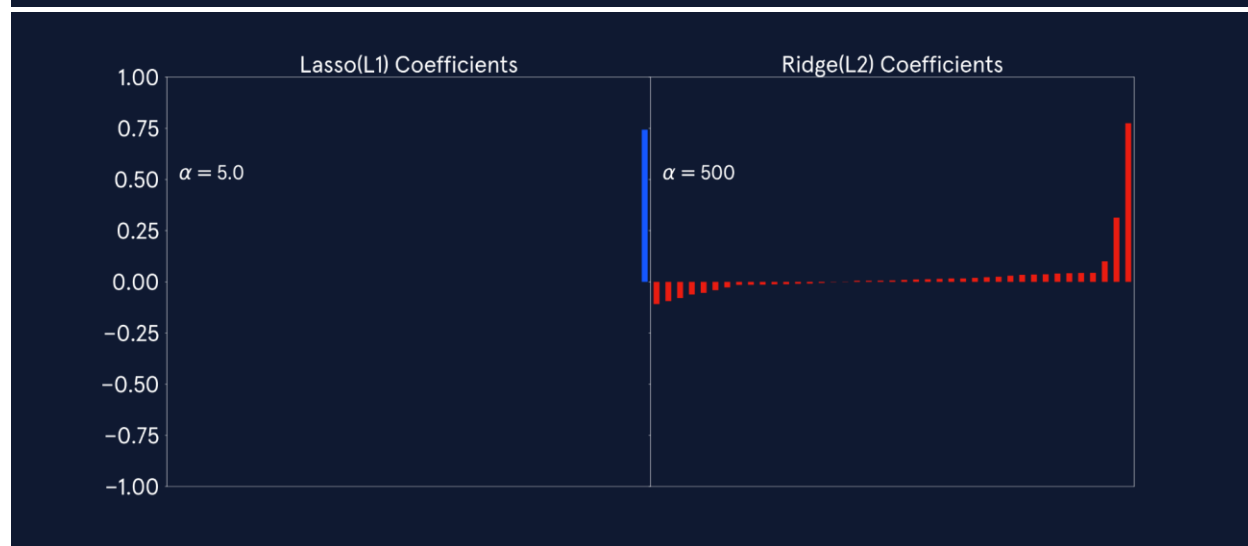
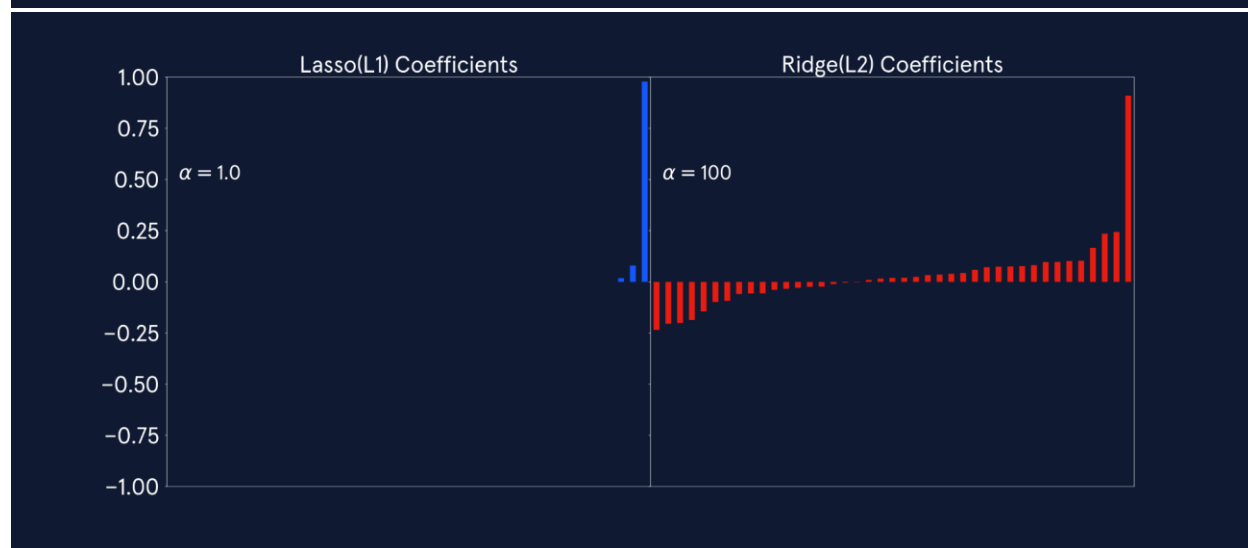
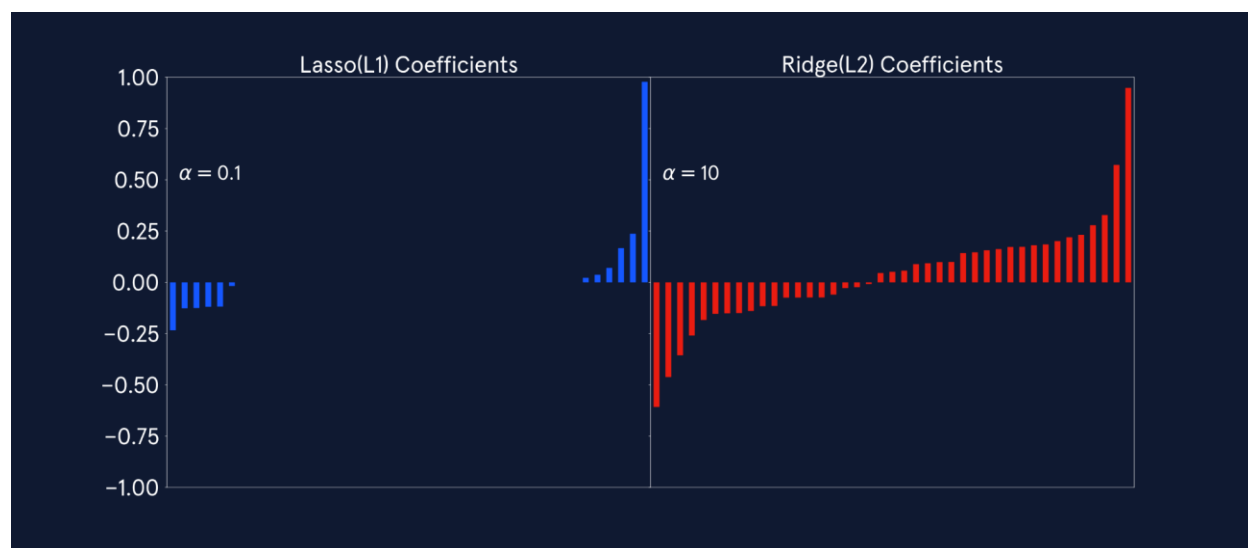
Preview: Docs Loading link description

[machine learning](#)

model to have low bias and low variance, i.e., we want it to perform well on training data as well as test data. However, trying to minimize bias and variance simultaneously is a bit of a conundrum as lowering one raises the other! This dilemma in machine learning models is known as the **bias-variance tradeoff**.

Hyperparameter tuning helps us find a sweet spot in this tradeoff to ensure that neither bias nor variance get too high. Typically, a portion of the data is set aside that is known as the "validation set" or "holdout set" (over and above the usual test-train split) and this is used to perform hyperparameter tuning on. We will delve into the ways we can implement this in `scikit-learn` in the following lesson!





Review

In this lesson we've learned the following topics:

- The importance of regularization in minimizing overfitting
- Lasso and Ridge regularization methods and their associated loss functions
- How to implement Lasso and Ridge with Multiple Linear Regression
- How regularization can be used as a feature selection method
- Hyperparameter tuning and the bias-variance tradeoff

The tables here summarize the key concepts we've learned. These form the basis for understanding regularization in more complex non-linear algorithms such as support vector machines and neural networks. We are now ready to learn how to tune hyperparameters and explore different methods of implementing regularization in Python!

1. Differences between Lasso and Ridge

	Lasso Regularization	Ridge Regularization
Regularization / Penalty term	L1 norm (sum of <u>absolute</u> values) of coefficients	L2 norm (sum of <u>squared</u> values) of coefficients
Constraint surface	<u>Diamond</u> shaped in 2D	<u>Circle</u> shaped in 2D
Coefficients after Regularization	All the coefficients shrink in their magnitude and <u>some</u> can be zero.	All the coefficients shrink in their magnitude but <u>never</u> approach zero.
Feature Selection Method	Yes! Features corresponding to the zero value coefficients are eliminated.	No! But it helps in emphasizing the relative feature importance.

2. Hyperparameter Tuning and the Bias-Variance Tradeoff

Low alpha	High s	Low Bias	High Variance	Low Regularization	Overfitting	Low training error
High alpha	Low s	High Bias	Low Variance	High Regularization	Underfitting	High training error

Implementing Regularization Methods in Python

Learn how to implement L1 and L2 regularization using `scikit-learn` and tune the regularization hyperparameter using `GridSearchCV` for linear and logistic regression.

Implementing Regularization with Linear Regression

In the previous lesson, we saw how implementing Lasso and Ridge regularization methods with linear regression in `scikit-learn` was fairly straightforward. Both `Lasso()` and `Ridge()` functions have an input parameter `alpha` that sets the strength of the regularization. The not-so-straightforward part was the question of how to choose the right `alpha` for the data and question at hand. We're now ready to dive into this!

For starters, we're going to run a quick Lasso implementation on the student performance [dataset](#) that we were working with. The dataset has 41 features relating to various factors that might influence a student's final grade. Let's take a sneak peek first:

```
import pandas as pd
df = pd.read_csv('student_math.csv')
print(df.columns, df.shape)
```

Output:

```
Index(['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel',
      'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2',
      'Final_Grade', 'school_MS', 'sex_M', 'address_U', 'famsize_LE3',
      'Pstatus_T', 'Mjob_health', 'Mjob_other', 'Mjob_services',
      'Mjob_teacher', 'Fjob_health', 'Fjob_other', 'Fjob_services',
      'Fjob_teacher', 'reason_home', 'reason_other', 'reason_reputation',
      'guardian_mother', 'guardian_other', 'schoolsup_yes', 'famsup_yes',
      'paid_yes', 'activities_yes', 'nursery_yes', 'higher_yes',
      'internet_yes', 'romantic_yes'],
      dtype='object') (395, 42)
```

We set our predictor and outcome variables and perform a train-test-split:

```
y = df['Final_Grade']
X = df.drop(columns = ['Final_Grade'])
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)
```

We're now ready to fit a Lasso regularized regression model by using the `Lasso()` function from the `linear_model` module:

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha = 0.05)
lasso.fit(X_train, y_train)
```

We're going to look at the Mean Squared Error (MSE) using a function from the `metrics` module:

```
from sklearn.metrics import mean_squared_error
pred_train = lasso.predict(X_train)
pred_test = lasso.predict(X_test)
training_mse = mean_squared_error(y_train, pred_train)
test_mse = mean_squared_error(y_test, pred_test)
```



```
print('Training Error:', training_mse)
print('Test Error:', test_mse)
```

Output:

```
Training Error: 2.8132075838851405
Test Error: 4.474769444129441
```

We see that the test error is more than 1.5 times that of the training error. This is an indicator that the model is still overfitting the data and that there is room to regularize more.

Tuning the Regularization Hyperparameter

We chose `alpha` to be 0.05 (the `scikit-learn` default is 1.0). It is worth recalling here that an `alpha` of zero is equivalent to no regularization. And when `alpha` is allowed to get too big we have the opposite problem of biasing our model and consequently underfitting the data.

One way to figure out the “Goldilocks zone” in this bias-variance tradeoff is to try multiple values of `alpha`. Iterating over an array of `alpha` values and plotting the resulting training and test errors against the corresponding `alpha`’s gets us the following figure:



We see that the test error goes down as we increase `alpha` until a certain value after which it begins to go up and then seems to slowly but steadily increase. Our optimal `alpha` thus lies somewhere in the blue band shown, likely a number between 0.12 and 0.16.

There is, however, another question to consider: would this be the same for a different train-test split? It is quite possible that our tuned hyperparameter might be different if we did a different train-test split. Luckily, `scikit-learn`

has just the feature for us. `GridSearchCV`, short for “Grid Search Cross-validation” helps us to satisfactorily resolve this while automating and speeding up our hyperparameter search - all within a couple of lines of code!

Automate the Hyperparameter Search with `GridsearchCV`

`GridSearchCV` uses a “k-fold” cross-validation method to search for the optimal hyperparameter in a machine learning algorithm. The k-fold method iteratively splits the dataset into train-test splits “k” times such that every point in the sample gets to be within the test data at least once. For instance, if we wanted to do 5-fold cross-validation, our data would be split into a 80:20 train-test split five times. To visualize this, imagine that our data is contained within this green box as shown below:



Our five “folds” can be represented by the yellow box. Each location of the box shows a specific train-test split. If we iterated our model fit over these five folds, every part of the data gets to be “test data” once. Doing a k-fold cross-validation makes sure that the conclusions we’re drawing about our data are not the result of a sampling effect but are truly representative of all of the data.

`GridSearchCV` takes in the following arguments:

```
GridSearchCV(estimator= Ridge (),  
              param_grid= {  
                  'alpha':  
                      [0.001, 0.01, 0.1, 1, 10]  
              })
```

Since we don’t have an idea of what order of magnitude `alpha` needs to be (i.e., is it between 0.01-0.1 or 0.1-1.0 or 1-10 and so on), using NumPy’s `logspace` function can be very useful. It is similar to `linspace` except that it gets us

numbers that are evenly spaced in the logarithmic scale. (It takes in the powers of ten that we're searching between, so if we wanted to search between 0.01 to 100, we'd set its inputs to -2 and 2 respectively.)

```
import numpy as np
## an array of alpha values between 0.000001 and 1.0
alpha_array = np.logspace(-6, 0, 100)

#dict with key (alpha) and values being alpha_array
tuned_parameters = [{'alpha': alpha_array}]
```

GridSearchCV

scikit-learn's [documentation](#) has an exhaustive list of scoring strategies. The argument for mean squared error, our chosen metric, is `neg_mean_squared_error`. It's a negative value because GridSearchCV *maximizes* the score it's given.

GridSearchCV

False

True

Consider our implementation of Lasso on the student performance dataset. Suppose we wanted to check the result of implementing different values of `alpha` and wanted to do multiple train-test splits to make sure we're covering the entire sample. We could do this in one line as follows:

```
from sklearn.model_selection import GridSearchCV
model = GridSearchCV(estimator = Lasso(), param_grid = tuned_parameters, scoring =
'neg_mean_squared_error', cv = 5, return_train_score = True)
model.fit(X, y)
```

The result of the `fit` object has many attributes, of which we're going to examine the ones that are most important to us:

The `cv_results_` object gives us the details of **every** model fit corresponding to a particular `alpha` value and the train-test split of each fold. (Our `alpha` array had 100 values and we did a 5-fold cross-validated search - so this is essentially equivalent to performing 500 model fits!) We're specifically going to look at the mean train and test scores across the 5 train-test splits:

```
test_scores = model.cv_results_['mean_test_score']
train_scores = model.cv_results_['mean_train_score']
```

Each of the above objects is an array the size of our *param_grid*, i.e., the number of `alpha` values we've specified.

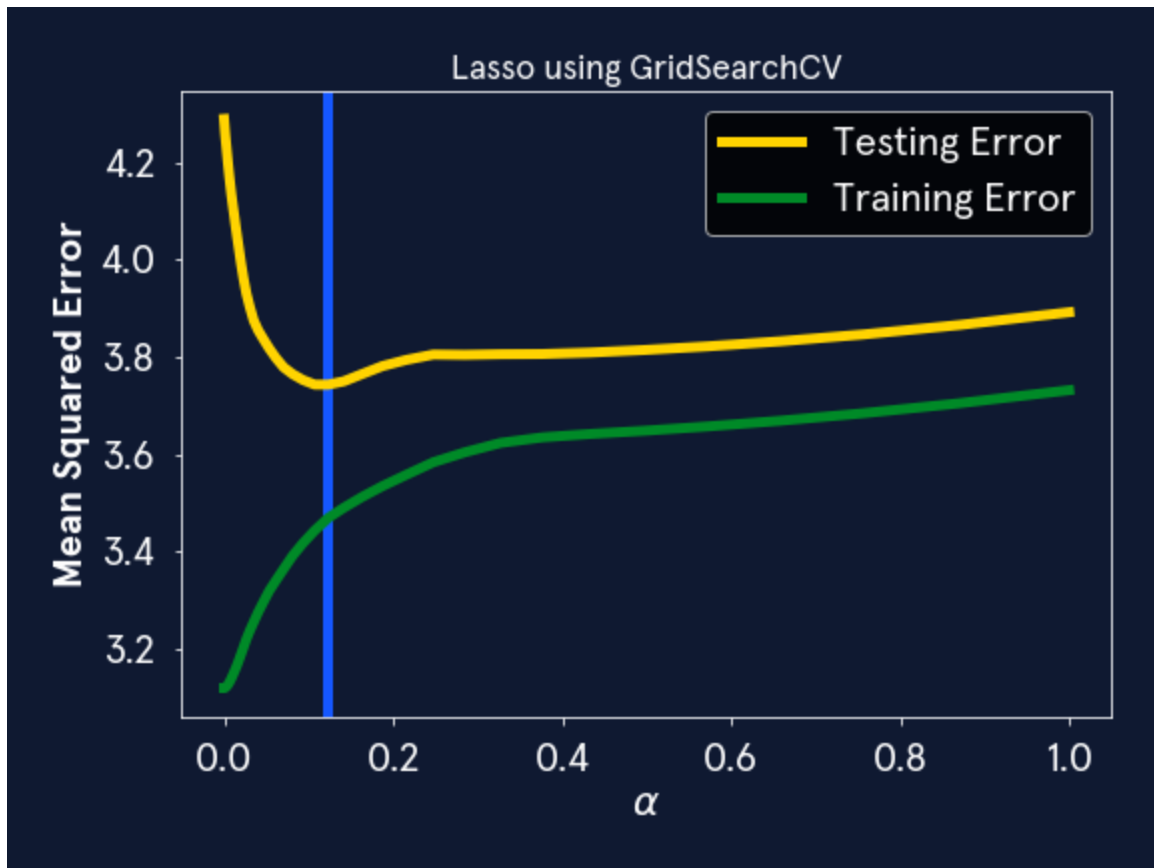
Additionally, the model fit object also gives us: `best_estimator_`, `best_score_` and `best_params_`. To get the `alpha` that is optimal to our scoring strategy, we can do the following:

```
print(model.best_params_, model.best_score_)
```

Output:

```
{'alpha': 0.12328467394420659} -3.7432949015591155
```

(Note that the score is the *negative* of least test mean squared error!) If we were to replicate the previous plot, i.e., plot our new training and test scores as a function of `alpha` and over plot the line corresponding to the "best" `alpha` value, we get the following:



The blue line here is our tuned hyperparameter and this value of α (~0.1233) corresponds to the optimal cross-validated test and training error values.

Coding question

Questions

We've loaded the student performance dataset here. Following the same logic as described above, implement hyperparameter tuning using `GridSearchCV` for Linear Regression with Ridge Regularization. Find the optimal α and the mean squared error corresponding to the best α . `GridSearchCV` might take about a minute to run - it is, after all, performing 5 (folds) times 100 (hyperparameter values) = 500 model fits! :)

Note: We've already specified a range of α values between 0.01 and 10000. These are a few orders of magnitude more than Lasso's α values. Can you think of why this might be the case?

Run your code to check your answer

Note: The α values are orders of magnitude bigger for Ridge for the following reason: α is inversely proportional to the size of the regularization constraint. Recall that for Lasso this means that it's proportional to $1/s$ while for Ridge, this would be $1/s^2$ as the constraint surfaces are different. So reducing s by a factor of 10 means increasing α 10x for Lasso but 100x for Ridge!

Implementing Regularization with a Logistic Regression Classifier

So far we've seen different implementations of regularization with linear regression. What if we were working with a classification problem instead? We're going to discuss a couple of ways of implementing L1 and L2 regularization with a logistic regression classifier.

On α 's, C 's and solver's: some `scikit-learn` pointers!

This might come as a surprise to you (it certainly did to me): the default `scikit-learn` logistic regression [implementation](#) is *already regularized*! It is L2-regularized with an α of 100. To elaborate, let's take a look at a couple of attributes that go into the logistic regression model:

The `LogisticRegression()` function differs from `Lasso()` and `Ridge()` functions for Linear Regression in that its input is the parameter `c`, which is the inverse of `alpha`. This is important to keep in mind, especially while setting up the parameter grid prior to implementing `GridSearchCV`.

Let's take a look at our different options for implementation then:

No regularization:

```
from sklearn.linear_model import LogisticRegression
logistic_no_regularization = LogisticRegression(penalty = 'none')
```

Lasso (L1): The L1 implementation requires setting the `penalty` attribute to 'l1' and also setting the `solver` attribute to 'liblinear'

```
logistic_lasso = LogisticRegression(penalty = 'l1', solver = 'liblinear', C =  
_)
```

It is worth noting here that there is a `solver` attribute in `scikit-learn` for most machine learning algorithms. The default solver for `LogisticRegression()` is `lbfgs`, however the only solver that can be used with Lasso Regularization is the 'liblinear' solver and hence it needs to be explicitly specified. (We're not going to go into further details at the moment but [the docs](#) are always a great place to get some more pointers!)

Ridge (L2): Remember the default is 'l2'! So all we need to do is specify `c` here:

```
logistic_ridge = LogisticRegression(C = _)
```

Elasticnet: The `penalty` options 'l1', 'l2' and 'none' are self-evident but we haven't covered 'elasticnet' in this course. Elasticnet Regularization is a combination of L1 and L2 regularization and has two penalty terms, one for L1 and one for L2!

```
logistic_elasticnet = LogisticRegression(penalty = 'elasticnet', solver =  
'saga', C = _, l1_ratio = _)
```

To implement Elasticnet one needs two hyperparameters: `c`, which specifies the regularization strength and an additional mixing hyperparameter, 'l1_ratio' which specifies how much L1 regularization used relative to L2.

'l1_ratio' takes values between 0 and 1, with 1 being the same as applying just L1 and 0, being equivalent to applying just L2 penalties respectively.

The solver 'saga' needs to be explicitly specified here.

Tuning hyperparameter `c` using `GridSearchCV` and `LogisticRegressionCV`

Implementing Lasso or Ridge with logistic regression in `scikit-learn` is as simple as picking the right arguments for `penalty` and `C`. We've written some sample code here to find the optimal `c` using `GridSearchCV`: (Remember that `c` is the inverse of `alpha` so greater the `c`, the lesser the amount of regularization!)

```
#Making an array of C's; here we're choosing 100 values between 0.001 and 100
C_array = np.logspace(-3,2, 100)
```

```
#Making a dict to enter as an input to param_grid
tuning_C = {'C':C_array}
clf = LogisticRegression(penalty = 'l1', solver = 'liblinear')
gs = GridSearchCV(clf, param_grid = tuning_C, scoring = 'accuracy', cv = 5)
```

Coding question

Questions

A dataset related to different types of candies and their rankings has been loaded for you here. The [data](#) comes from FiveThirtyEight and the article it pertains to is Halloween Candy Rankings from 2017.

But we're going to answer a different question: can we predict whether a candy contains chocolate or not? Build a logistic regression classifier with L2 regularization using `GridSearchCV` to obtain the best `c` as well as the best possible accuracy. Use the parameters `model`, `tuning_C`, `scoring='accuracy'`, and `cv=5` in your `GridSearchCV` model and save your `GridSearchCV` model as `gs`.

Be sure to take a look at the `model` and `tuning_C` variables provided for you, but do not change them.

Since Logistic Regression is such a commonly used algorithm, `scikit-learn` has a function that makes the above faster and neater, the `LogisticRegressionCV()` function. If we were to implement the code challenge using this function, it would look as follows:

```
model = LogisticRegressionCV( Cs=np.logspace(-3,2, 100),
                             penalty='l2',
                             scoring='accuracy', cv=5,
                             random_state=42,max_iter=10000)

model.fit(X, y)
print(model.C_, model.scores_[1].mean(axis=0).max())
```

Output:

```
[2.42012826] 0.8823529411764705
```

Which is the same output as we get from the code challenge! Note that there are slight syntactical differences between the outputs of `GridsearchCV` and `LogisticRegressionCV`. In general, many algorithms in `scikit-learn` have their own special grid search cross-validation function such as `LassoCV()`, `RidgeCV()`, `ElasticnetCV()`, etc.

Conclusion

We have covered multiple ways of implementing regularization with linear and logistic regression in `scikit-learn` in this article. There are a few essential (albeit annoying) differences between these methods. Together they offer a powerful toolkit to find the optimal regularization for regression and classification methods. You can try out all these techniques in the upcoming project!

Feature Importance

Feature Importance

Learn about feature importance and how to calculate it.

Introduction

When we fit a supervised machine learning (ML) model, we often want to understand which features are most associated with our outcome of interest. Features that are highly associated with the outcome are considered more “important.” In this article, we’ll introduce you to the concept of **feature importance** through a discussion of:

- Tree-based feature importance
 - [Gini impurity](#)
 - Implementation in [scikit-learn](#)
- Other methods for estimating feature importance

Feature importance in an ML workflow

There are many reasons why we might be interested in calculating feature importances as part of our machine learning workflow. For example:

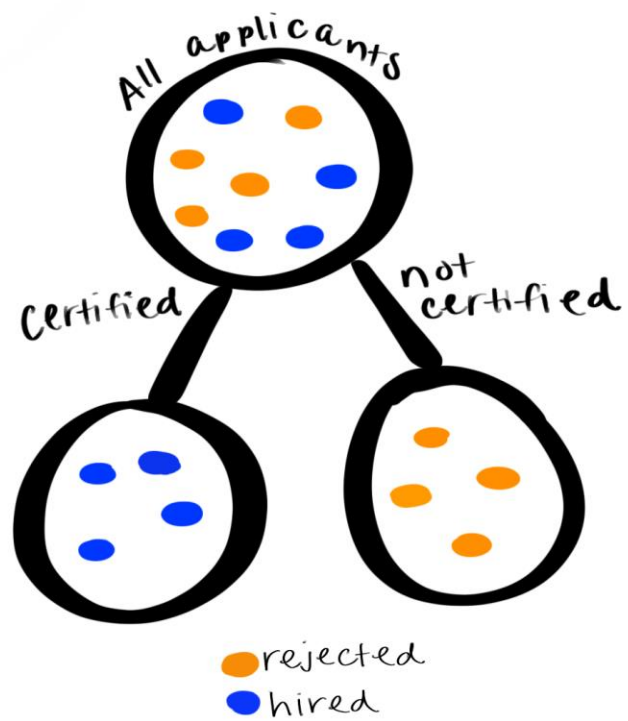
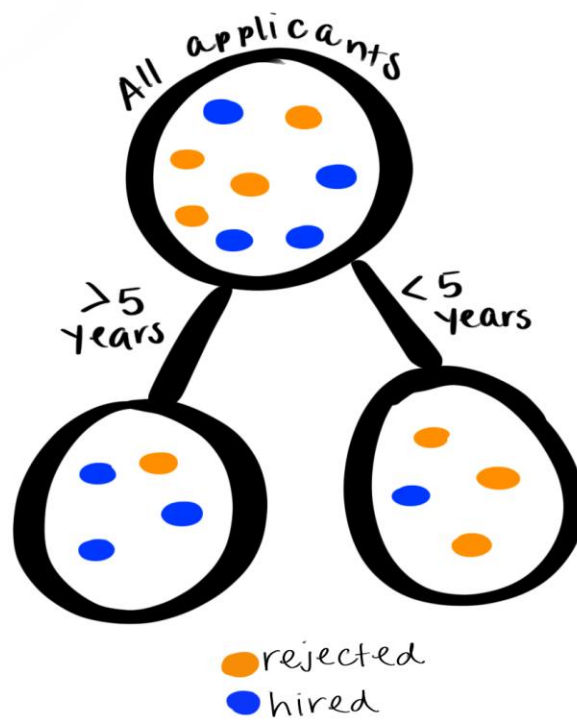
- Feature importance is often used for dimensionality reduction.
 - We can use it as a filter method to remove irrelevant features from our model and only retain the ones that are most highly associated with our outcome of interest.
 - Wrapper methods such as recursive feature elimination use feature importance to more efficiently search the feature space for a model.
- Feature importance may also be used for model inspection and communication. For example, stakeholders may be interested in understanding which features are most important for prediction. Feature importance can help us answer this question.

Calculating feature importance

There are many different ways to calculate feature importance for different kinds of machine learning models. In this section, we’ll investigate one tree-based method in a little more detail: **Gini impurity**.

Gini impurity

Imagine, for a moment, that you’re interested in building a model to screen candidates for a particular job. In order to build this model, you’ve collected some data about candidates who you’ve hired and rejected in the past. For each of these candidates, suppose that you have data on years of experience and certification status. Consider the following two simple decision trees that use these features to predict whether the candidate was hired:



Which of these features seems to be more important for predicting whether a candidate will be hired? In the first example, we saw that *most* candidates who had >5 years of experience were hired and *most* candidates with <5 years were rejected; however, *all* candidates with certifications were hired and *all* candidates without them were rejected.

Gini impurity is related to the extent to which observations are well separated based on the outcome variable at each node of the decision tree. For example, in the two trees above, the Gini impurity is higher in the node with all candidates (where there are an equal number of rejected and hired candidates) and lower in the nodes after the split (where most or all of the candidates in each grouping have the same outcome – either hired or rejected).

To estimate feature importance, we can calculate the Gini gain: the amount of Gini impurity that was eliminated at each branch of the decision tree. In this example, certification status has a higher Gini gain and is therefore considered to be more important based on this metric.

Gini importance in scikit-learn

To demonstrate how we can estimate feature importance using Gini impurity, we'll use the breast cancer dataset from `sklearn`. This dataset contains features related to breast tumors. The outcome variable is the diagnosis: either malignant or benign. To start, we'll load the dataset and split it into a training and test set:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import datasets

dataset = datasets.load_breast_cancer()
X = pd.DataFrame(dataset.data, columns=dataset.feature_names)
y = dataset.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
```

Next, we'll fit a decision tree to predict the diagnosis using `sklearn.tree.DecisionTreeClassifier()`. Note that we're setting `criterion= 'gini'`. This actually tells the function to build the decision tree by splitting each node based on the feature that has the highest Gini gain. By building the tree in this way, we'll be able to access the Gini importances later.

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(criterion='gini')

# Fit the decision tree classifier
clf = clf.fit(X_train, y_train)
```

Next, we can access the feature importances based on Gini impurity as follows:

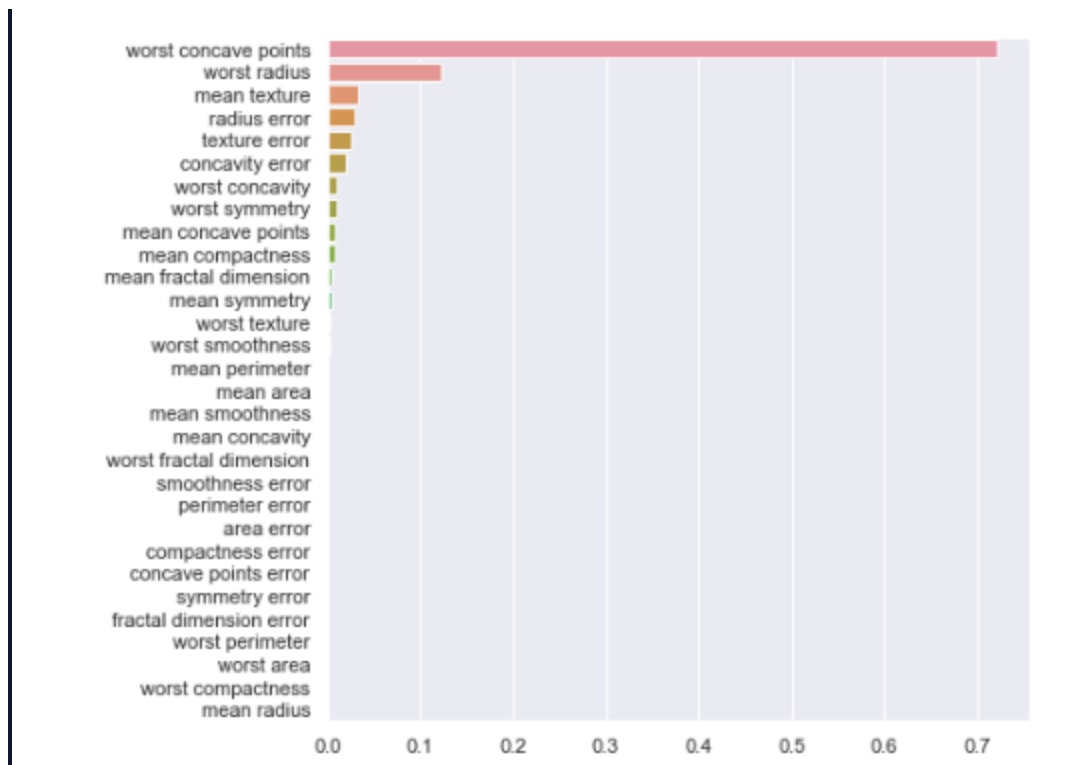
```
# Print the feature importances
feature_importances = clf.feature_importances_
```

Finally, we'll visualize these values using a bar chart:

```
import seaborn as sns

# Sort the feature importances from greatest to least using the sorted indices
sorted_indices = feature_importances.argsort()[::-1]
sorted_feature_names = data.feature_names[sorted_indices]
sorted_importances = feature_importances[sorted_indices]

# Create a bar plot of the feature importances
sns.set(rc={'figure.figsize': (11.7, 8.27)})
sns.barplot(sorted_importances, sorted_feature_names)
```



Based on this output, we could conclude that the features `worst concave points`, `worst radius` and `mean texture` are most predictive of a malignant tumor. There are also many features with importances close to zero which we may want to exclude from our model.

Pros and cons of using Gini importance

Because Gini impurity is used to train the decision tree itself, it is computationally inexpensive to calculate. However, Gini impurity is somewhat biased toward selecting numerical features (rather than categorical features). It also does not take into account the correlation between features. For example, if two highly correlated features are both equally important for predicting the outcome variable, one of those features may have low Gini-based importance because all of its explanatory power was ascribed to the other feature. This issue can be mediated by removing redundant features before fitting the decision tree.

Other measures of feature importance

There are many other methods for estimating feature importance beyond calculating Gini gain for a single decision tree. We'll explore a few of these methods below.

Aggregate methods

Random forests are an ensemble-based machine learning algorithm that utilize many decision trees (each with a subset of features) to predict the outcome variable. Just as we can calculate Gini importance for a single tree, we can calculate average Gini importance across an entire random forest to get a more robust estimate.

Permutation-based methods

Another way to test the importance of particular features is to essentially remove them from the model (one at a time) and see how much predictive accuracy suffers. One way to "remove" a feature is to randomly permute the values for that feature, then refit the model. This can be implemented with any machine learning model, including non-tree-based- methods. However, one potential drawback is that it is computationally expensive because it requires us to refit the model many times.

Coefficients

When we fit a general(ized) linear model (for example, a linear or logistic regression), we estimate coefficients for each predictor. If the original features were standardized, these coefficients can be used to estimate relative feature importance; larger absolute value coefficients are more important. This method is computationally inexpensive because coefficients are calculated when we fit the model. It is also useful for both classification and regression problems (i.e., categorical and continuous outcomes). However, similar to the other methods described above, these coefficients do not take highly correlated features into account.

Conclusion

In this article, we've covered a few different examples of feature importance metrics, including how to interpret and calculate them. We learned about:

- Gini impurity
- How to calculate Gini-based feature importance for a decision tree in `sklearn`
- Other methods for calculating feature importance, including:
 - Aggregate methods
 - Permutation-based methods
 - Coefficients

Feature importance is an important part of the machine learning workflow and is useful for feature engineering and model explanation, alike!

Hyperparameters in Machine Learning Models

Hyperparameters in Machine Learning Models

Learn about hyperparameters in machine learning models.

Introduction

Hyperparameters are an important part of machine learning. We're going to explain what exactly they are along with examples of what they look like for different commonly used ML models. You'll learn why they're important and how they affect model performance. You'll also learn about a few hyperparameter tuning methods that you can implement in your own projects.

What are Hyperparameters?

A *hyperparameter* of a machine learning model is a value that determines part of the learning process and is not affected by training unlike a *parameter*, which is learned during the model training process. You're probably already familiar with some hyperparameters. Let's look at some examples.

In the k-nearest neighbors algorithm, k is a hyperparameter. k determines how many neighbors will be used, and training data does not change k.

Decision trees also use hyperparameters. Before training a decision tree classifier, you might want to specify how deep the tree can go (i.e., how many splits are allowed before arriving at a leaf). You might also want to specify the minimum number of samples that are present in a node in order to split that node. Both of those values are hyperparameters. They determine the structure of the model, but they are not learned during training.

Regularization factors are another example. In linear or logistic regression, regularization is a term that is added to the loss function in order to penalize models with large coefficients. (The coefficients are the *parameters* of the model here! The regularization factor is a hyperparameter. The value of the regularization factor affects how large the coefficients of a regression model will be, but the regularization factor is independent of training data.

Hyperparameter Tuning

Hyperparameters can make or break a machine learning model. Take the k-nearest neighbors algorithm as an example. Recall that in k-nn classification, the class of a sample is determined by the classes of its k nearest neighbors from the training data. If k is too large, classes with a small number of samples might be missed, and the model could underfit the data. If k is too small, it's possible that outliers in the training data could misclassify their neighbors. The model could be overfit to those outliers.

This balance between overfitting and underfitting is called the *bias-variance tradeoff*. It's an important consideration for supervised machine learning.

Bias is the difference between a model's predictions and the correct values. Models with a lot of bias underfit the data and will perform poorly on both testing and training data. In biased models, the structure of the model overpowers the training data.

Variance refers to the dependence of a model on training data. A model has high variance if different training data results in widely varying outcomes. This often leads to overfitting a model to training data.

Overfit, high-variance models generally perform well on training data, but poorly on testing data. In this case, the training data overpowers the structure of the model.

The bias variance tradeoff is called a tradeoff because decreasing the bias usually increases the variance. The converse is also true: decreasing variance usually increases the bias. Good machine learning models strike a balance. We can reduce bias by making a model more sensitive to training data, but we do so at the risk of overfitting. On the other hand, we can reduce variance by restricting a model so that it is less affected by training data. This, however, runs the risk of underfitting the model.

Ultimately, we want models that perform well on testing data. One way to do that is to choose the right hyperparameters. In the case of supervised machine learning, we want hyperparameters that make a good compromise between bias and variance. This can be done by training a model multiple times, each time with different hyperparameter values, and then using the best ones. This process is called *hyperparameter tuning*. There are several different ways of doing this.

Common Hyperparameter Tuning Methods

The *grid search* algorithm for hyperparameter tuning works by training a model on predetermined lists of hyperparameter values. This method tries every hyperparameter value on the list, and then uses the one that makes the model perform best.

The *random search* algorithm works similarly, but instead of using a predetermined list of hyperparameter values, the values are randomly chosen. As with grid search, it selects the hyperparameter that performed the best.

Bayesian optimization is another approach to hyperparameter tuning. It uses ideas from the field of Bayesian statistics to iterate through different hyperparameter values. Each time the Bayesian optimization algorithm evaluates a new hyperparameter value, it gains more information about where it should look for the best hyperparameter value.

Genetic algorithms are another possible hyperparameter tuning method. These work by going through several generations of hyperparameter values. Within each generation, the fittest (i.e., best-performing) hyperparameter values are slightly mutated (i.e., changed) in order to produce the next generation.

Each of these hyperparameter tuning methods will be covered in more detail in a subsequent lesson. You'll also learn how to implement each of them in Python.

When tuning hyperparameters, it's important to split the data into training, testing, and validation data. Training data is used to train the model. Validation data is used to evaluate hyperparameters. After a hyperparameter is tuned, the model can be tested on testing data. This data allows for an estimate of model performance that isn't affected by the hyperparameter tuning process.

Here's an example. Say there is a classification problem and we want to tune a hyperparameter `alpha`. Say we want to determine whether the model performs best with `alpha=1`, `alpha=2`, or `alpha=3`. We'll train the model three times: once with each possible value for `alpha`. (Since we are looking at a predetermined list of hyperparameter values, this is a grid search.) Let's say the accuracy was as follows.

	Accuracy on training set	Accuracy on validation set	Accuracy on testing set
<code>alpha=1</code>	85%	80%	
<code>alpha=2</code>	85%	84%	82%
<code>alpha=3</code>	88%	83%	

Since the model was most accurate on the validation set with `alpha=2`, we will use that value. However, we can't estimate the model's accuracy based on data that was used to determine the model's hyperparameter. This would likely overestimate the actual accuracy. Therefore, we must measure the accuracy of the final model (with `alpha=2`) on the testing set.

Here's a table that lists several machine learning algorithms, some of their hyperparameters, and possible tuning methods. This table is not exhaustive: some machine learning algorithms have many hyperparameters that aren't listed here. If you want to learn more about the hyperparameters for a specific model in scikit-learn, the documentation is a good place to start.

For example, you can read about the details of scikit-learn's support vector machine classifier [here](#). You'll find descriptions of `C` (the inverse of regularization strength), `kernel`, `degree`, and more. Each of these values is a hyperparameter that can be chosen or tuned.

ML Algorithm	scikit-learn package	Hyperparameters	Parameters in sklearn	Notes	How to tune
Linear regression with gradient descent	<code>sklearn.linear_model.SGDRegressor</code>	Learning rate, regularization penalty	<code>learning_rate</code> , <code>penalty</code> , <code>alpha</code>	Set <code>penalty</code> to <code>l1</code> or <code>l2</code> (default). <code>alpha</code> is the regularization strength. Larger <code>alpha</code> will result in stronger regularization.	<code>learning_rate</code> and <code>C</code> can be tuned with a grid search or random search.
Logistic regression	<code>sklearn.linear_model.LogisticRegression</code>	Regularization penalty	<code>penalty</code> , <code>C</code>	Set <code>penalty</code> to <code>l1</code> , <code>l2</code> (default), or <code>None</code> . <code>C</code> is the inverse of regularization strength. Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.
k-nearest neighbors	<code>sklearn.neighbors.KNeighborsClassifier</code>	k (the number of neighbors)	<code>n_neighbors</code>		<code>n_neighbors</code> can be tuned with a grid search or random search.
Decision trees	<code>sklearn.tree.DecisionTreeClassifier</code>	Maximum depth of the tree, minimum number of samples to split an internal node	<code>max_depth</code> , <code>min_samples_split</code>		<code>max_depth</code> and <code>min_samples_split</code> can be tuned with a grid search or random search.
k-means	<code>sklearn.cluster.KMeans</code>	k (the number of clusters)	<code>n_clusters</code>		Use the elbow method.
Support vector machines	<code>sklearn.svm.SVC</code>	Regularization penalty	<code>C</code>	Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.

Introduction

In this lesson you will learn about the different methods one can use to tune hyperparameters in

Preview: Docs Loading link description

[machine learning](#)

models and how to implement them in Python. Specifically we will be diving deep into two methods: grid search (`GridSearchCV`) and random search (`RandomizedSearchCV`).

To understand the implementation of different methods of hyperparameter tuning, we need to choose a dataset, a classification or regression problem we'd like to solve, and a machine learning model to solve it with. The image on the right-hand side lists some of the commonly used machine learning models and their corresponding hyperparameters. Our choices for the rest of the lesson are as follows:

Dataset and Model: We're going to work with the commonly used [breast cancer data set](#) that is available with `scikit-learn`. The prediction task is to classify tumors as benign or malignant and the data has 30 numeric predictor

Preview: Docs Loading link description

[variables](#)

. We're going to use [logistic regression](#) to perform this task.

Hyperparameters: So, which hyperparameters do we tune? There are many arguments to `scikit-learn`'s logistic regression function and many of them can be treated as hyperparameters and tuned. However, we're specifically going to focus on the hyperparameters pertaining to regularization in this exercise.

This lesson assumes familiarity with the implementation of logistic regression models and regularization techniques.

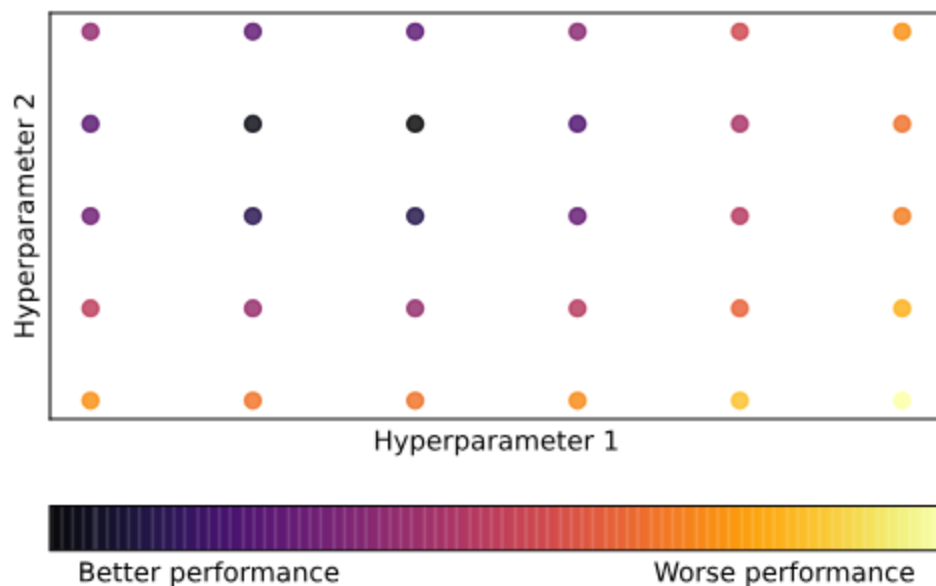
ML Algorithm	scikit-learn package	Hyperparameters	Parameters in sklearn	Notes	How to tune
Linear regression with gradient descent	<code>sklearn.linear_model.SGDRegressor</code>	Learning rate, regularization penalty	<code>learning_rate</code> , <code>penalty</code> , <code>alpha</code>	Set <code>penalty</code> to <code>l1</code> or <code>l2</code> (default). <code>alpha</code> is the regularization strength. Larger <code>alpha</code> will result in stronger regularization.	<code>learning_rate</code> and <code>C</code> can be tuned with a grid search or random search.
Logistic regression	<code>sklearn.linear_model.LogisticRegression</code>	Regularization penalty	<code>penalty</code> , <code>C</code>	Set <code>penalty</code> to <code>l1</code> , <code>l2</code> (default), or <code>None</code> . <code>C</code> is the inverse of regularization strength. Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.
k-nearest neighbors	<code>sklearn.neighbors.KNeighborsClassifier</code>	k (the number of neighbors)	<code>n_neighbors</code>		<code>n_neighbors</code> can be tuned with a grid search or random search.
Decision trees	<code>sklearn.tree.DecisionTreeClassifier</code>	Maximum depth of the tree, minimum number of samples to split an internal node	<code>max_depth</code> , <code>min_samples_split</code>		<code>max_depth</code> and <code>min_samples_split</code> can be tuned with a grid search or random search.
k-means	<code>sklearn.cluster.KMeans</code>	k (the number of clusters)	<code>n_clusters</code>		Use the elbow method.
Support vector machines	<code>sklearn.svm.SVC</code>	Regularization penalty	<code>C</code>	Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.

An Introduction to the Grid Search Method

We'll start with the grid search

Preview: Docs [Loading link description algorithm](#)

. Grid search works by testing a model on a list of hyperparameter values decided upon beforehand. Suppose we had two hyperparameters we wanted to tune and we wanted to choose between 6 values for the first one and 5 values of the second, we'd be searching a grid of thirty values as shown below. Grid search would fit the model and evaluate its performance for each of the values represented by these points. We can then choose the hyperparameter values corresponding to the best performance and conclude our hyperparameter search!



In our case, where we're using a logistic regression model with regularization (`LogisticRegression`), we might want to make decisions on the following using hyperparameter tuning:

The type of regularization to use: The `penalty`

Preview: Docs Loading link description

[parameter](#)

in `scikit-learn` represents this and let's say we'd like to figure out if Lasso ('l1') or Ridge regression ('l2') is the better choice here

The strength of regularization to use: The `scikit-learn` parameter `C` represents the *inverse of regularization strength* and let's say we'd like to test 3 possible values of `C` here: 1, 10 and 100.

This means we have to search a grid of $2 \times 3 = 6$ values. `GridSearchCV` in `scikit-learn` lets us do this! It tests the model performance at each of these values to come up with the best possible set of hyperparameters *from the list given*.

The two most important parameters in `GridSearchCV` that need to be specified are: the name of the model that we are testing and the name of a

Preview: Docs Loading link description

[dictionary](#)

of hyperparameters that we would initialize, represented by the

Preview: Docs Loading link description

[argument](#)

parameters. To tune the hyperparameters, we can use `.fit()`, just as we would for a regular

Preview: Docs Loading link description

[machine learning](#)

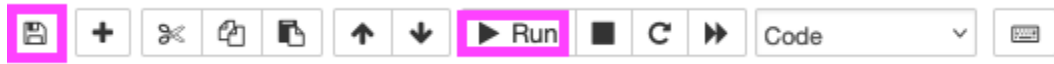
model.

We're going to be working with `scikit-learn`'s breast cancer dataset that has already been loaded in the jupyter notebook. We've also performed a train-test split. Run the Setup cell.

You can run a cell in the Notebook to the right by placing your cursor in the cell and clicking the `Run` button or the `Shift+Enter/Return` keys.

When you are ready to evaluate the code in your Notebook, press the `Save` button at the top of the Notebook or use the `control/command+s` keys before clicking the `Test Work` button at the bottom. Be sure to save your solution code in the cell marked `## YOUR SOLUTION HERE ##` or it will not be evaluated.

When you are ready to move on, click Next.



Instructions

Checkpoint 1 Enabled

1.

After running the setup cell, create a `LogisticRegression` model called `lr`. Since we want to use a solver that is compatible with both L1 and L2 regularization, set the parameter `solver` to `'liblinear'` (the default solver does not support L1 regularization!). To ensure that the model converges, set `max_iter = 1000`.

Write your solution code in the space provided, save the Notebook and click Test Work to move to the next checkpoint.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Define a dictionary `parameters` that you will be performing the grid search over. The dictionary should have two keys:

`'penalty'`, corresponding to the two regularization types, `['l1', 'l2']` (use a lowercase "L" with the number 1 and 2)

`'C'`, the regularization strength to be set to `[1,10,100]`

Write your solution code in the space provided, run the cell, save the Notebook and click Test Work to move to the next checkpoint.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We can now set up grid search! Define a `GridSearchCV()` object named `clf`. The first argument is `estimator`, which corresponds to the logistic regression model you've created and the second argument is `param_grid`, which corresponds to the dictionary you're performing the hyperparameter search on.

Write your solution code in the space provided, run the cell, save the Notebook and click Test Work to move to the next exercise.

Setup

```
In [1]: from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Load the data set
cancer = load_breast_cancer()

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target)
```

1. Initializing model for grid search

```
In [2]: ## YOUR SOLUTION HERE ##
lr = LogisticRegression(solver='liblinear', max_iter=1000)

#check output
print(lr.get_params())

{'C': 1.0, 'class_weight': None, 'dual': False, 'fit_intercept': True, 'intercept_scaling': 1, 'l1_ratio': None, 'max_iter': 1000, 'multi_class': 'auto', 'n_jobs': None, 'penalty': 'l2', 'random_state': None, 'solver': 'liblinear', 'tol': 0.0001, 'verbose': 0, 'warm_start': False}
```

2. Initializing grid search dictionary parameters of hyperparameters to search from

```
In [5]: ## YOUR SOLUTION HERE ##
params = {'penalty': ['l1', 'l2'], 'C': [0.01, .01, 1, 10, 100]}
```

3. Set up grid search using GridSearchCV

```
In [7]: ## YOUR SOLUTION HERE ##
clf = GridSearchCV(lr, params)

#Check output
print(clf.get_params())

{'cv': None, 'error_score': nan, 'estimator__C': 1.0, 'estimator__class_weight': None, 'estimator__dual': False, 'estimator__fit_intercept': True, 'estimator__intercept_scaling': 1, 'estimator__l1_ratio': None, 'estimator__max_iter': 1000, 'estimator__multi_class': 'auto', 'estimator__n_jobs': None, 'estimator__penalty': 'l2', 'estimator__random_state': None, 'estimator__solver': 'liblinear', 'estimator__tol': 0.0001, 'estimator__verbose': 0, 'estimator__warm_start': False, 'estimator': LogisticRegression(max_iter=1000, solver='liblinear'), 'n_jobs': None, 'param_grid': {'penalty': ['l1', 'l2'], 'C': [0.01, 0.01, 1, 10, 100]}, 'pre_dispatch': '2*n_jobs', 'refit': True, 'return_train_score': False, 'scoring': None, 'verbose': 0}
```

Evaluating the Results of GridSearchCV

The “CV” in `GridSearchCV` is an acronym for **cross-validation**. It’s best practice in

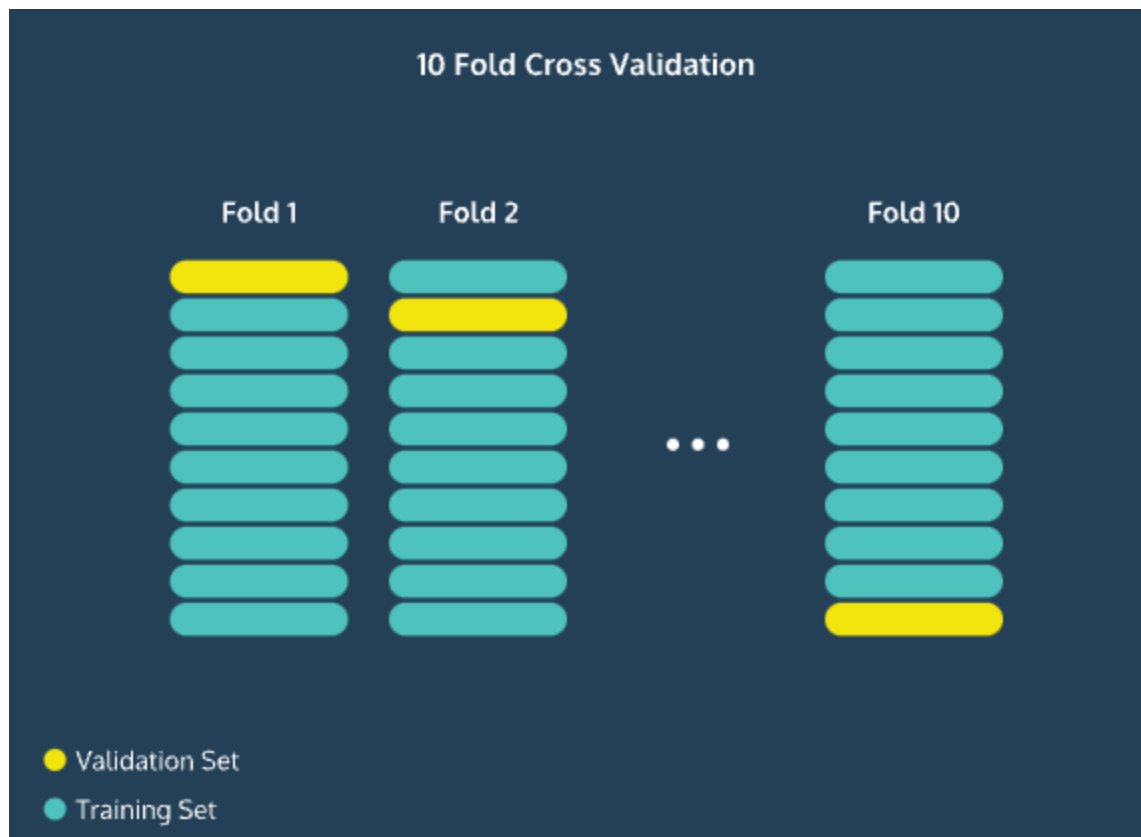
Preview: Docs Loading link description

[machine learning](#)

to go beyond the usual train-test split and have a holdout or validation dataset. Specifically, `GridSearchCV` uses a technique known as **k-fold cross-validation**. This works as follows.

Cross-validation in GridSearchCV

`GridSearchCV` subdivides the training data further into *another training and test data set*. It fits the model on this new training data and evaluates the model on the new test data. But to make sure that we don’t accidentally have good performance in only one part of our dataset, `GridSearchCV` will do this process multiple times on different cross-validation splits so that every point in the data gets to be tested on at least once! The number of times this split happens is the “k” in “k-fold”. For instance, in a 10-fold cross-validation, our data would be split into a 90:10 train-test split 10 times and `GridSearchCV` would evaluate the model on each fold.



In `scikit-learn`, `cv`

Preview: Docs An argument is the actual value of a variable passed into a function.

[argument](#)

in `GridSearchCV` allows us to decide on the number of cross-validation splits we'd like. The default setting for this is 5.

Evaluating `GridSearchCV` results

After fitting a `GridSearchCV` model we can find out the results using the following attributes of the `clf` argument:

```
.best_estimator_ gives us the best estimator  
.best_score_ gives us the mean cross-validated score corresponding to the best estimator  
.best_params_ gives us the set of hyperparameters that correspond to the best estimator
```

Additionally, the `.cv_results_` attribute gives us the scores for each hyperparameter combination in the grid. We're now ready to evaluate the grid search we set up earlier and we've preloaded the code from the previous exercise in the Setup cell.

Instructions

Checkpoint 1 Enabled

1.

After running the setup cell, fit the grid search classifier, `clf` to the training data. Set the best estimator to a variable `best_model` and print it to see which model performed best.

Uncomment the line that prints the best parameters to check if they match up!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Set the score corresponding to the best estimator to a variable `best_score` and print it.
Calculate the accuracy of the best estimator on the test data and store it as the variable `test_score`.

How do the scores of the best model compare on the training and test data?
Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We've successfully performed a grid search over a 2 by 3 grid of hyperparameters! Using the `.cv_results_` attribute, we can look at the scores for each of the values in the hyperparameter grid.

`clf.cv_results_` is a dictionary of values.

Use the key argument `'params'` to get the list of parameters. Convert this to a pandas DataFrame named `hyperparameter_grid`.

Use the key argument `'mean_test_score'` to get the list of mean test scores corresponding to the parameter values. Convert this to a single column pandas DataFrame named `grid_scores` and set the argument `columns` to `['score']`.

Uncomment the lines pertaining to concatenating the two dataframes and printing it to view the results of the grid search for each hyperparameter pair!

Setup

```
In [2]: import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Load the data set
cancer = load_breast_cancer()

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target, random_state = 19)

## Intializaing model and dictionary of hyperparameters
lr = LogisticRegression(solver='liblinear', max_iter=1000)
parameters = {'penalty': ['l1', 'l2'], 'C': [1, 10, 100]}

##Setting up Grid Search
clf = GridSearchCV(lr, parameters)
```

1. Fit `clf` to training data and get best hyperparameters

```
In [3]: ## YOUR SOLUTION HERE ##
clf.fit(X_train, y_train)
best_model = clf.best_estimator_
print(best_model)
print(clf.best_params_)

LogisticRegression(C=10, max_iter=1000, penalty='l1', solver='liblinear')
{'C': 10, 'penalty': 'l1'}
```

2. Calculate training and test scores of the best estimator

```
In [4]: ## YOUR SOLUTION HERE ##
best_score = clf.best_score_
test_score = clf.score(X_test, y_test)
print(best_score)
print(test_score)

0.9671135430916552
0.951048951048951
```

3. Viewing grid search Results

```
In [5]: ## YOUR SOLUTION HERE ##
hyperparameter_grid = pd.DataFrame(clf.cv_results_['params'])
grid_scores = pd.DataFrame(clf.cv_results_['mean_test_score'], columns = ['score'])

df = pd.concat([hyperparameter_grid, grid_scores], axis = 1)
print(df)
```

	C	penalty	score
0	1	11	0.955349
1	1	12	0.952996
2	10	11	0.967114
3	10	12	0.957702
4	100	11	0.957674
5	100	12	0.960027

```
In [ ]:
```

An Introduction to the Random Search Method

We're now going to look at an alternate

Preview: Docs Loading link description

[method](#)

to tune hyperparameters, the *random search method*. With grid search, we made a list of different hyperparameter values and tested each combination of them. What if instead of a pre-selected list, we tested some random values? A random search method that does this might look like the top half of the image on the right as opposed to an orderly grid like in grid search, (which is shown in the bottom half of the image for contrast).

To implement this, one needs to search over hyperparameters by drawing a list of random values from *distributions*.

What would this mean for say, the hyperparameter `c` in logistic regression that we've been working with so far?

Instead of selecting from a pre-determined list (like `[1, 10, 100]` which we've been using for grid search in previous exercises), we would draw 3 random values between say 0 and 100.

In `scikit-learn`, the `RandomSearchCV` function implements random search with cross-validation. `RandomSearchCV` requires three arguments to be specified:

`estimator`: the

Preview: Docs Loading link description

[machine learning](#)

model whose hyperparameters we're tuning; this is exactly the same for `GridSearchCV`

`param_distributions`: a

Preview: Docs Loading link description

[dictionary](#)

which specifies the hyperparameters as keys and corresponding distributions to draw lists of values from for each hyperparameter. In `GridSearchCV`, we instead had `param_grid`, a dictionary representing the grid of hyperparameters to search from

`n_iter`: the number of times the

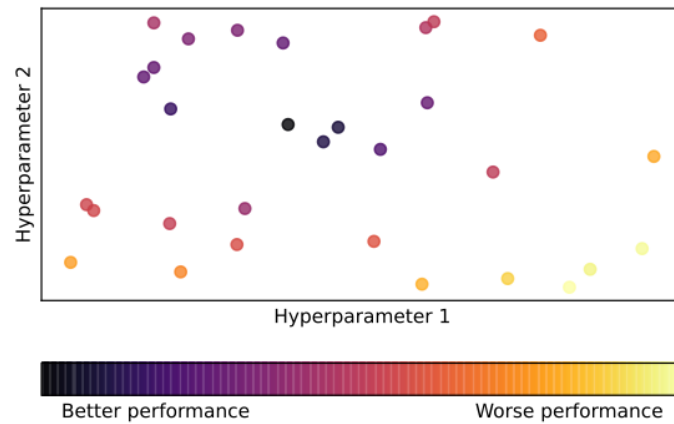
Preview: Docs Loading link description

[algorithm](#)

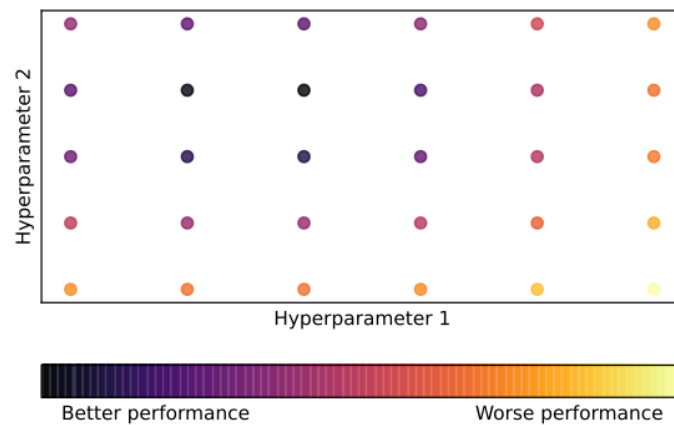
needs to randomly draw from the distributions. The default value for this is 10.

We're now ready to dive into implementing the random search method!

Random Search
Method



Grid Search
Method



Using RandomizedSearchCV

When we used grid search, we made a list of different hyperparameter values. With random search, we don't have to make a list, but we still have to provide some information about how we want to select random numbers. Do we want random numbers between 0 and 100? Between -10 and 10? Do we want the same chance of picking small numbers and picking large numbers?

We can do this by specifying a probability distribution for each hyperparameter.

```
from scipy.stats import uniform
distributions = {'penalty': ['l1', 'l2'], 'C': uniform(loc=0, scale=100)}
```

Copy to Clipboard

Let's take a closer look at each distribution:

The `penalty` hyperparameter of scikit-learn's `LogisticRegression` model has only two possible values: `l1` and `l2`. We list them both. `RandomizedSearchCV` will treat this as a *discrete uniform distribution*. This just means that every item in the list has an equal chance of being selected. In this case, there's a 50% chance of drawing `l1` and a 50% chance of drawing `l2`.

The hyperparameter `c` is the inverse of regularization strength. It can be any positive number, so we have to specify a probability distribution that allows us to randomly select a positive number. The `scipy` library has many probability distributions to choose from (you can see a list of them [here](#)). For this example, we're using the *uniform distribution*. This allows us to randomly select numbers between `loc` and `loc + scale` (in this case, between 0 and 100).

Run the code under the Setup cell to import libraries, load the breast cancer data set, and split the data into training and testing sets.

Instructions

Checkpoint 1 Enabled

1.

Create a dictionary called `distributions` that contains the keys `'penalty'` and `'c'`. The values should be:

```
['l1', 'l2'] for 'penalty'
uniform(loc=0, scale=100) for 'c'
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's do a couple of draws from the distribution for `'c'` to make sure we do get numbers between 0 and 100.

Create an array `first_draw` to use `.rvs()` to generate 10 random variable values for `'c'` from the `distributions` dictionary.

Repeat the same process but with an array named `second_draw`.

Print `first_draw` and `second_draw` to view the outcomes of these draws!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We're now ready to create a `RandomizedSearchCV` model. To do so:

First, define a logistic regression model like we've done in the previous exercises, setting solver to `'liblinear'` and `max_iter` to 1000.

Initialize a random search model `clf` using the parameters `lr`, `distributions`, and `n_iter`, in that order. Set `n_iter` to 8 to test 8 hyperparameter combinations.

Evaluating the Results of `RandomizedSearchCV`

We can now follow a similar process to what we did with `GridSearchCV` to evaluate the results of `RandomizedSearchCV`.

After fitting a `RandomizedSearchCV` model we can find out the results using the following attributes of the `clf` argument:

```
.best_estimator_ gives us the best estimator
.best_score_ gives us the mean cross-validated score corresponding to the best estimator
.best_params_ gives us the set of hyperparameters that correspond to the best estimator
```

Additionally, the `.cv_results_` attribute gives us the scores for each hyperparameter combination in the grid. We're now ready to evaluate the random search we set up earlier and we've preloaded the code from the previous exercise in the Setup cell.

Instructions

Checkpoint 1 Submitting

1.

After running the setup cell, fit the random search classifier, `clf` to the training data. Set the best estimator to a variable `best_model` and print it to see which model performed best.

Uncomment the line that prints the best parameters to check if they match up!

Setup

```
In [1]: import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform

# Load the data set
cancer = load_breast_cancer()

# Split the data into training and testing sets
X = cancer.data
y = cancer.target
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Create distributions to draw hyperparameters from
distributions = {'penalty': ['l1', 'l2'], 'C': uniform(loc=0, scale=100)}

# The logistic regression model
lr = LogisticRegression(solver='liblinear', max_iter=1000)

# Create a RandomizedSearchCV model
clf = RandomizedSearchCV(lr, distributions, n_iter=8)
```

1. Fit `clf` to training data and get best hyperparameters

```
In [2]: ## YOUR SOLUTION HERE ##

clf.fit(X_train, y_train)
best_model = clf.best_estimator_
print(best_model)
print(clf.best_params_)

LogisticRegression(C=31.769718491702726, max_iter=1000, penalty='l1',
                    solver='liblinear')
{'C': 31.769718491702726, 'penalty': 'l1'}
```

2. Calculate training and test scores of the best estimator

```
In [3]: ## YOUR SOLUTION HERE ##

best_score = clf.best_score_
test_score = clf.score(X_test, y_test)
print(best_score)
print(test_score)

0.969466484268126
0.951048951048951
```

3. Viewing random search Results

```
In [4]: ## YOUR SOLUTION HERE ##
hyperparameter_values = pd.DataFrame(clf.cv_results_['params'])
randomsearch_scores = pd.DataFrame(clf.cv_results_['mean_test_score'], columns = ['score'])

df = pd.concat([hyperparameter_values, randomsearch_scores], axis = 1)
print(df)
```

	C	penalty	score
0	87.003879	12	0.955376
1	5.133015	11	0.957729
2	51.976782	11	0.967114
3	71.640591	11	0.967086
4	39.863819	12	0.955376
5	31.769718	11	0.969466
6	90.378809	12	0.960082
7	33.530884	11	0.969466

```
In [ ]:
```

Hyperparameter Tuning Review

Nice work! You've learned how to tune the hyperparameters in a machine-learning model! Let's go over some of the key points to take away.

In

Preview: Docs Loading link description

[machine learning](#)

, hyperparameter tuning is altering the hyperparameter values of the model to get the model that's most suited to answering a question about the data.

Examples of hyperparameters; the learning rate, regularization penalty, number of neighbors (in a k-means model), tree depth (decision tree model), and minimum number of samples to split (decision tree model).

Grid Search is a

Preview: Docs Loading link description

[method](#)

of hyperparameter tuning. It works by testing a model on a list of hyperparameter values decided upon beforehand.

With a grid search, if we want to test 2 different types of regularization penalties and 3 different regularization strengths, this would give us 6 different combinations of hyperparameters on our grid to test ($2 \times 3 = 6$).

The Random Search method is an opposing method to grid search. In a random search model, a

Preview: Docs Loading link description

[dictionary](#)

is given that specifies the distributions from which the model will randomly draw values for each hyperparameter. Additionally, the number of times the

Preview: Docs Loading link description

[algorithm](#)

should randomly draw from these distributions is also specified.

Both `GridSearchCV()` and `RandomSearchCV()` use a technique known as **k-fold cross-validation**. This is when new combinations of the training and testing sets are created (using the same data) to conclude the most accurate model. This ensures a less biased model.

We used the following attributes to evaluate both the `GridSearchCV()` and `RandomSearchCV()` models;

`.best_estimator_`, `.best_score_`, `.best_params_`, and the `.cv_results_`.

ML Algorithm	scikit-learn package	Hyperparameters	Parameters in sklearn	Notes	How to tune
Linear regression with gradient descent	<code>sklearn.linear_model.SGDRegressor</code>	Learning rate, regularization penalty	<code>learning_rate</code> , <code>penalty</code> , <code>alpha</code>	Set <code>penalty</code> to <code>l1</code> or <code>l2</code> (default). <code>alpha</code> is the regularization strength. Larger <code>alpha</code> will result in stronger regularization.	<code>learning_rate</code> and <code>C</code> can be tuned with a grid search or random search.
Logistic regression	<code>sklearn.linear_model.LogisticRegression</code>	Regularization penalty	<code>penalty</code> , <code>C</code>	Set <code>penalty</code> to <code>l1</code> , <code>l2</code> (default), or <code>None</code> . <code>C</code> is the inverse of regularization strength. Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.
k-nearest neighbors	<code>sklearn.neighbors.KNeighborsClassifier</code>	k (the number of neighbors)	<code>n_neighbors</code>		<code>n_neighbors</code> can be tuned with a grid search or random search.
Decision trees	<code>sklearn.tree.DecisionTreeClassifier</code>	Maximum depth of the tree, minimum number of samples to split an internal node	<code>max_depth</code> , <code>min_samples_split</code>		<code>max_depth</code> and <code>min_samples_split</code> can be tuned with a grid search or random search.
k-means	<code>sklearn.cluster.KMeans</code>	k (the number of clusters)	<code>n_clusters</code>		Use the elbow method.
Support vector machines	<code>sklearn.svm.SVC</code>	Regularization penalty	<code>C</code>	Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.

Unsupervised Learning

An Introduction to Unsupervised Learning

Learn about Unsupervised Learning!

Unsupervised Learning describes a class of algorithms that find patterns from unlabelled or untagged data. In Supervised Learning, we deal with data that is labelled or tagged. For example, we predict continuous outcomes in *regression* (for instance, predicting housing prices) or categorical outcomes in *classification* (spam versus not spam, for example). However, often training data isn't labelled in this manner and this is where unsupervised learning comes in! It relies on using the underlying distributions of features within the data to figure out clusters of similarity.

Why did we build this?

Unsupervised learning methods are extremely important to the functioning of many real-world algorithms - think image recognition, ride-shares anticipating demands, snapchat filters and many more! There are three primary ways they're used:

Clustering: Identifying clusters within a dataset like identifying disease outbreak clusters or in natural language processing, in creating word clouds that are semantically related, etc.

Dimensionality Reduction/Feature Extraction: They can be used to condense the number of features in a dataset with a high number of features before applying a supervised learning algorithm.

Automated Labelling/Tagging: Unsupervised learning algorithms are immensely useful in categorizing uncategorized data and one can then perform the familiar classification/regression tasks using supervised learning.

What will you learn?

In this module we will focus on two of the most commonly used unsupervised learning techniques - Principal Component Analysis (PCA) and K-Means Clustering. The former is most often used for dimensionality reduction and the latter is used in clustering problems primarily. After this module you will be able to:

K-Means Clustering

Introduction to Clustering

Often, the data you encounter in the real world won't be sorted into categories and won't have labeled answers to your question. Finding patterns in this type of data, unlabeled data, is a common theme in many

Preview: Docs Loading link description

[machine learning](#)

applications. *Unsupervised Learning* is how we find patterns and structure in these data.

Clustering is the most well-known unsupervised learning technique. It finds structure in unlabeled data by identifying similar groups, or *clusters*. Examples of clustering applications are:

Recommendation engines: group products to personalize the user experience

Search engines: group news topics and search results

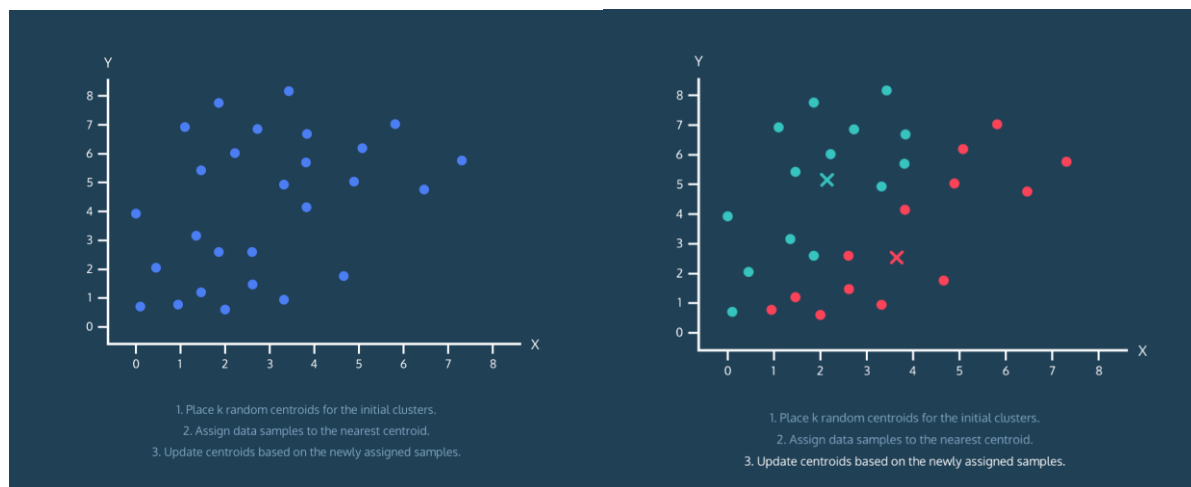
Market segmentation: group customers based on geography, demography, and behaviors

Image segmentation: medical imaging or road scene segmentation on self-driving cars

Text clustering: group similar texts together based on word usage

The *Iris* data set is a famous example of unlabeled data. It consists of measurements of sepals and petals on 50 different iris flowers. Here you can see a visualization of this data set that shows how the flowers naturally form three distinct clusters. We'll learn how to find those clusters in this lesson.

The image for this exercise will be replaced with a visualization of the iris data set.



K-Means Clustering

The goal of clustering is to separate data so that data similar to one another are in the same group, while data different from one another are in different groups. So two questions arise:

How many groups do we choose?

How do we define similarity?

k-means is the most popular and well-known clustering

Preview: Docs Loading link description

[algorithm](#)

, and it tries to address these two questions.

The “k” refers to the number of clusters (groups) we expect to find in a dataset.

The “Means” refers to the average distance of data to each cluster center, also known as the *centroid*, which we are trying to minimize.

It is an iterative approach:

Place k random centroids for the initial clusters.

Assign data samples to the nearest centroid.

Calculate new centroids based on the above-assigned data samples.

Repeat Steps 2 and 3 until convergence.

Convergence occurs when points don’t move between clusters and centroids stabilize. This iterative process of updating clusters and centroids is called *training*.

Once we are happy with our clusters, we can take a new unlabeled datapoint and quickly assign it to the appropriate cluster. This is called *inference*.

In practice it can be tricky to know how many clusters to look for. In the example here, the algorithm is sorting the data into $k=2$ clusters.

Iris Dataset

Before we implement the k-means

Preview: Docs Loading link description

[algorithm](#)

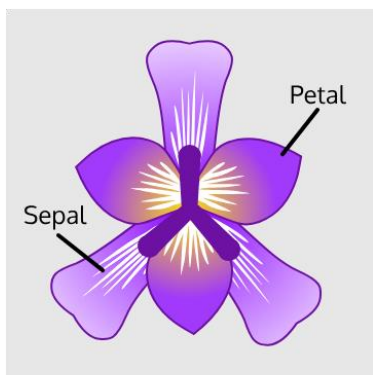
, let’s find a dataset. The `sklearn` package embeds some datasets and sample images. One of them is the [Iris dataset](#).

The Iris dataset consists of measurements of sepals and petals of 3 different plant species:

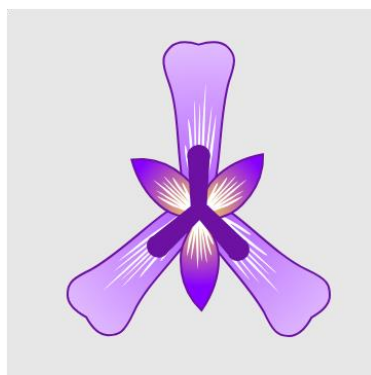
Iris setosa

Iris versicolor

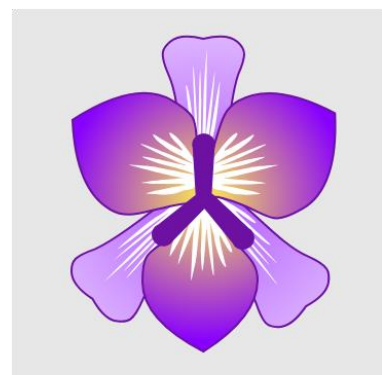
Iris virginica



Iris Versicolor



Iris Setosa



Iris Virginica

The sepal is the part that encases and protects the flower when it is in the bud stage. A petal is a leaflike part that is often colorful.

From `sklearn` library, import the `datasets` module:

```
from sklearn import datasets
```

Copy to Clipboard
To load the Iris dataset:

```
iris = datasets.load_iris()
```

Copy to Clipboard
The Iris dataset looks like:

```
[ [ 5.1  3.5  1.4  0.2 ]  
  [ 4.9  3.   1.4  0.2 ]  
  [ 4.7  3.2  1.3  0.2 ]  
  [ 4.6  3.1  1.5  0.2 ]  
  .   .   .  
  [ 5.9  3.   5.1  1.8 ] ]
```

Copy to Clipboard
We call each row of data a *sample*. For example, each flower is one sample.

Each characteristic we are interested in is a *feature*. For example, petal length is a feature of this dataset.

The features of the dataset are:

Column 0: Sepal length
Column 1: Sepal width
Column 2: Petal length
Column 3: Petal width

The 3 species of Iris plants are what we are going to cluster later in this lesson.

Instructions

Checkpoint 1 Enabled

1.

Import the `datasets` module and load the Iris data.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Every dataset from `sklearn` comes with a bunch of different information (not just the data) and is stored in a similar fashion.

First, let's take a look at the most important thing, the sample data:

```
print(iris.data)
```

Copy to Clipboard

Each row is a plant!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

The iris dataset comes with target values. The target values indicate which cluster each flower belongs to. In real life clustering problems, you will work with unlabeled data sets that don't come with targets. For the sake of practice, we can ignore the targets while we are clustering. After we have clustered the data the targets can be used to check our work.

Take a look at the target values:

```
print(iris.target)
```

Copy to Clipboard

The `iris.target` values give the *ground truth* for the Iris dataset. Ground truth, in this case, is the number corresponding to the flower that we are trying to learn.

```
import codecademylib3_seaborn  
import matplotlib.pyplot as plt
```

```
from sklearn import datasets

iris = datasets.load_iris()

print(iris)

print(iris.data)
print(iris.target)
print(iris.data[0, :], iris.target[0])
print(iris.DESCR)
```

Visualize Before K-Means

To get a better sense of the data in the `iris.data` matrix, let's visualize it!

With Matplotlib, we can create a 2D scatter plot of the Iris dataset using two of its features (sepal length vs. petal length). Of course there are four different features that we could plot, but it's much easier to visualize only two dimensions.

The sepal length measurements are stored in column `0` of the matrix, and the petal length measurements are stored in column `2` of the matrix.

But how do we get these values?

Suppose we only want to retrieve the values that are in column `0` of a matrix, we can use the NumPy/pandas notation `[:,0]` like so:

```
matrix[:,0]
```

Copy to Clipboard

`[:,0]` can be translated to `[all_rows , column_0]`

Once you have the measurements we need, we can make a scatter plot like this:

```
plt.scatter(x, y)
```

Copy to Clipboard

To show the plot:

```
plt.show()
```

Copy to Clipboard

Let's try this! But this time, plot the sepal length (column `0`) vs. sepal width (column `1`) instead.

Instructions

Checkpoint 1 Enabled

1.

Store `iris.data` in a variable named `samples`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a list named `x` that contains the column `0` values of `samples`.

Create a list named `y` that contains the column `1` values of `samples`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the `.scatter()` function to create a scatter plot of `x` and `y`.

Because some of the data samples have the exact same features, let's add `alpha=0.5`:

```
plt.scatter(x, y, alpha=0.5)
```


Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Call the `.show()` function to display the graph.

If you didn't know there are three species of the Iris plant, would you have known just by looking at the visualization?

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt

from sklearn import datasets

iris = datasets.load_iris()

# Store iris.data
samples = iris.data

# Create x and y
x = samples[:,0]
y = samples[:,1]

# Plot x and y
plt.scatter(x, y, alpha=0.5)

plt.xlabel('sepal length (cm)')
plt.ylabel('petal length (cm)')

# Show the plot
plt.show()
```

Implementing K-Means: Step 1

The K-Means algorithm:

Place k random centroids for the initial clusters.

Assign data samples to the nearest centroid.

Update centroids based on the above-assigned data samples.

Repeat Steps 2 and 3 until convergence.

After looking at the scatter plot and having a better understanding of the Iris data, let's start implementing the k-means

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

In this exercise, we will implement Step 1.

Because we expect there to be three clusters (for the three species of flowers), let's implement k-means where the k is 3. In real-life situations you won't always know how many clusters to look for. We'll learn more about how to choose k later.

Using the NumPy library, we will create three *random* initial centroids and plot them along with our samples.

Instructions

Checkpoint 1 Enabled

1.

First, create a variable named `k` and set it to 3.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then, use NumPy's `random.uniform()` function to generate random values in two lists:

a `centroids_x` list that will have `k` random values between `min(x)` and `max(x)`

a `centroids_y` list that will have `k` random values between `min(y)` and `max(y)`

The `random.uniform()` function looks like:

```
np.random.uniform(low, high, size)
```

Copy to Clipboard

The `centroids_x` will have the x-values for our initial random centroids and the `centroids_y` will have the y-values for our initial random centroids.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create an array named `centroids` and use the `zip()` function to add `centroids_x` and `centroids_y` to it.

The `zip()` function looks like:

```
np.array(list(zip(array1, array2)))
```

Copy to Clipboard

Then, print `centroids`.

The `centroids` list should now have all the initial centroids.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Make a scatter plot of `y` vs `x`. (Remember that you're only looking at two out of four dimensions of the data set. Based on that it might not look like there are three distinct clusters, but since we are missing two dimensions this plot won't tell the whole story.)

Make a scatter plot of `centroids_y` vs `centroids_x`.

Show the plots to see your centroids!

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets

iris = datasets.load_iris()

samples = iris.data

x = samples[:,0]
y = samples[:,1]

combined = np.array(list(zip(x, y)))

# Number of clusters
k = 3

# Create x coordinates of k random centroids
centroids_x = np.random.uniform(min(x), max(x), size=k)

# Create y coordinates of k random centroids
centroids_y = np.random.uniform(min(y), max(y), size=k)

# Create centroids array
centroids = np.array(list(zip(centroids_x, centroids_y)))

print(centroids)
```

```
# Make a scatter plot of x, y
plt.scatter(x, y, alpha=0.5)

# Make a scatter plot of the centroids
plt.scatter(centroids_x, centroids_y)

# Display plot
plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')

plt.show()
```

Implementing K-Means: Step 2

The k-means algorithm:

- Place `k` random centroids for the initial clusters.
- Assign data samples to the nearest centroid.**
- Update centroids based on the above-assigned data samples.
- Repeat Steps 2 and 3 until convergence.

In this exercise, we will implement Step 2.

Now we have the three random centroids. Let's assign data points to their nearest centroids.

To do this we're going to use a distance formula to write a `distance()` function.

There are many different kinds of distance formulas. The one you're probably most familiar with is called *Euclidean distance*. To find the Euclidean distance between two points on a 2-d plane, make a right triangle so that the hypotenuse connects the points. The distance between them is the length of the hypotenuse.

Another common distance formula is the *taxicab distance*. The taxicab distance between two points on a 2-d plane is the distance you would travel if you took the long way around the right triangle via the two shorter sides, just like a taxicab would have to do if it wanted to travel to the opposite corner of a city block.

Different distance formulas are useful in different situations. If you're curious, you can learn more about various distance formulas [here](#). For this lesson, we'll use Euclidean distance.

After we write the `distance()` function, we are going to iterate through our data samples and compute the distance from each data point to each of the 3 centroids.

Suppose we have a point and a list of three distances in `distances` and it looks like `[15, 20, 5]`, then we would want to assign the data point to the 3rd centroid. The `argmin(distances)` would return the

Preview: Docs Loading link description

[index](#)

of the lowest corresponding distance, `2`, because the index `2` contains the minimum value.

Instructions

Checkpoint 1 Enabled

1.

Write a `distance()` function.

It should be able to take in `a` and `b` and return the distance between the two points.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create an array called `labels` that will hold the cluster labels for each data point. Its size should be the length of the data sample. It should look something like:

```
[ 0.  0.  0.  0.  0.  0.  ...  0.]
```

Copy to Clipboard

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create a function called `assign_to_centroid()` that assigns the nearest centroid to a sample. You'll need to compute the distance to each centroid to find the closest one.

```
def assign_to_centroid(sample, centroids):  
    # Fill in the code here  
    #  
    #  
    #  
    #  
    return closest_centroid
```

Copy to Clipboard

Then, assign the `cluster` to each index of the `labels` array.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Write a loop that iterates through the whole data sample and assigns each sample's closest centroid to the corresponding index of the `labels` array. Use the function that you created in the previous exercise.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Then, print `labels` (outside of the `for` loop).

Awesome! You have just finished Step 2 of the k-means algorithm.

```
import codecademylib3_seaborn  
import matplotlib.pyplot as plt  
import numpy as np  
from sklearn import datasets  
  
iris = datasets.load_iris()  
  
samples = iris.data  
  
x = samples[:,0]  
y = samples[:,1]  
  
sepal_length_width = np.array(list(zip(x, y)))  
  
# Step 1: Place K random centroids  
  
k = 3  
  
centroids_x = np.random.uniform(min(x), max(x), size=k)  
centroids_y = np.random.uniform(min(y), max(y), size=k)  
  
centroids = np.array(list(zip(centroids_x, centroids_y)))  
  
# Step 2: Assign samples to nearest centroid  
  
# Distance formula  
  
def distance(a, b):  
    one = (a[0] - b[0]) **2  
    two = (a[1] - b[1]) **2  
    distance = (one+two) ** 0.5  
    return distance
```

```
# Cluster labels for each point (either 0, 1, or 2)

labels = np.zeros(len(samples))

# A function that assigns the nearest centroid to a sample

def assign_to_centroid(sample, centroids):
    k = len(centroids)
    distances = np.zeros(k)
    for i in range(k):
        distances[i] = distance(sample, centroids[i])
    closest_centroid = np.argmin(distances)
    return closest_centroid

# Assign the nearest centroid to each sample

for i in range(len(samples)):
    labels[i] = assign_to_centroid(samples[i], centroids)

# Print labels

print(labels)
```

Implementing K-Means: Step 3

The k-means algorithm:

- Place k random centroids for the initial clusters.
- Assign data samples to the nearest centroid.
- Update centroids based on the above-assigned data samples.**
- Repeat Steps 2 and 3 until convergence.

In this exercise, we will implement Step 3.

Find *new* cluster centers by taking the average of the assigned points. To find the average of the assigned points, we can use the `.mean()` function.

Instructions

Checkpoint 1 Failed, try again

1.

- Save the old `centroids` value before updating.
- We have already imported `deepcopy` for you:

```
from copy import deepcopy
```

Copy to Clipboard

Store `centroids` into `centroids_old` using `deepcopy()`:

```
centroids_old = deepcopy(centroids)
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then, create a `for` loop that iterates k times.

Since $k = 3$, as we are iterating through the `for` loop each time, we can calculate the mean of the points that

have the same cluster label.

Inside the `for` loop, create an array named `points` where we get all the data points that have the cluster label `i`.

There are two ways to do this, check the hints to see both!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now that we have assigned each input to its closest centroid, we can update the position of that centroid to the true center. Inside the `for` loop, calculate the mean of those points using `.mean()` to get the new centroid.

Store the new centroid in `centroids[i]`.

The `.mean()` function looks like:

```
np.mean(input, axis=0)
```

Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Outside of the `for` loop, print `centroids_old` and `centroids` to see how centroids changed.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from copy import deepcopy

iris = datasets.load_iris()

samples = iris.data

x = samples[:,0]
y = samples[:,1]

sepal_length_width = np.array(list(zip(x, y)))

# Step 1: Place K random centroids

k = 3

centroids_x = np.random.uniform(min(x), max(x), size=k)
centroids_y = np.random.uniform(min(y), max(y), size=k)

centroids = np.array(list(zip(centroids_x, centroids_y)))

# Step 2: Assign samples to nearest centroid

# Distance formula

def distance(a, b):
    one = (a[0] - b[0]) **2
    two = (a[1] - b[1]) **2
    distance = (one+two) ** 0.5
    return distance

# Cluster labels for each point (either 0, 1, or 2)

labels = np.zeros(len(samples))

# A function that assigns the nearest centroid to a sample

def assign_to_centroid(sample, centroids):
    k = len(centroids)
    distances = np.zeros(k)
    for i in range(k):
        distances[i] = distance(sample, centroids[i])
    closest_centroid = np.argmin(distances)
    return closest_centroid
```

```
# Assign the nearest centroid to each sample

for i in range(len(samples)):
    labels[i] = assign_to_centroid(samples[i], centroids)

# Step 3: Update centroids

centroids_old = deepcopy(centroids)

for i in range(k):
    points = [sepal_length_width[j] for j in range(len(sepal_length_width)) if labels[j] == i]
    centroids[i] = np.mean(points, axis=0)

print(centroids_old)
print("- - - - -")
print(centroids)
```

Implementing K-Means: Step 4

The k-means algorithm:

- Place `k` random centroids for the initial clusters.
- Assign data samples to the nearest centroid.
- Update centroids based on the above-assigned data samples.
- Repeat Steps 2 and 3 until convergence.**

In this exercise, we will implement Step 4.

This is the part of the

Preview: Docs Loading link description

[algorithm](#)

where we repeatedly execute Step 2 and 3 until the centroids stabilize (convergence).

We can do this using a `while` loop. And everything from Step 2 and 3 goes inside the loop.

For the condition of the `while` loop, we need to create an

Preview: Docs Loading link description

[array](#)

named `errors`. In each

Preview: Docs Loading link description

[error](#)

Preview: Docs Loading link description

[index](#)

, we calculate the difference between the updated centroid (`centroids`) and the old centroid (`centroids_old`).

The loop ends when all three values in `errors` are 0.

Instructions

Checkpoint 1 Enabled

1.

On line 49 of `script.py`, initialize `error`:

```
error = np.zeros(3)
```

Copy to Clipboard

Then, use the `distance()` function to calculate the distance between the updated centroids and the old centroids and put them in `error`. Here's how to calculate the error for entry '0'. You can write a loop to compute each distance.

```
error[0] = distance(centroids[0], centroids_old[0])
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

After that, add a `while` loop:

```
while error.all() != 0:
```

Copy to Clipboard

And move *everything* below (from Step 2 and 3) inside.

And recalculate `error` again at the end of each iteration of the `while` loop. You can put this line inside the 'for' loop that computes the new centroids:

```
error[i] = distance(centroids[i], centroids_old[i])
```

Copy to Clipboard

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Awesome, now you have everything, let's visualize it.

After the `while` loop finishes, let's create an array of colors:

```
colors = ['r', 'g', 'b']
```

Copy to Clipboard

Then, create a `for` loop that iterates `k` times.

Inside the `for` loop (similar to what we did in the last exercise), create an array named `points` where we get all the data points that have the cluster label `i`.

Then we are going to make a scatter plot of `points[:, 0]` vs `points[:, 1]` using the `scatter()` function:

```
plt.scatter(points[:, 0], points[:, 1], c=colors[i], alpha=0.5)
```

Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Then, paste the following code at the very end. Here, we are visualizing all the points in each of the `labels` a different color.

```
plt.scatter(centroids[:, 0], centroids[:, 1], marker='D', s=150)
```

```
plt.xlabel('sepal length (cm)')
```

```
plt.ylabel('sepal width (cm)')
```

```
plt.show()
```

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from copy import deepcopy

iris = datasets.load_iris()

samples = iris.data

x = samples[:,0]
y = samples[:,1]

sepal_length_width = np.array(list(zip(x, y)))

# Step 1: Place K random centroids
```



```

k = 3

centroids_x = np.random.uniform(min(x), max(x), size=k)
centroids_y = np.random.uniform(min(y), max(y), size=k)

centroids = np.array(list(zip(centroids_x, centroids_y)))

def distance(a, b):
    one = (a[0] - b[0]) ** 2
    two = (a[1] - b[1]) ** 2
    distance = (one + two) ** 0.5
    return distance

# A function that assigns the nearest centroid to a sample
def assign_to_centroid(sample, centroids):
    k = len(centroids)
    distances = np.zeros(k)
    for i in range(k):
        distances[i] = distance(sample, centroids[i])
    closest_centroid = np.argmin(distances)
    return closest_centroid

# To store the value of centroids when it updates
centroids_old = np.zeros(centroids.shape)

# Cluster labels (either 0, 1, or 2)
labels = np.zeros(len(samples))

distances = np.zeros(3)

# Initialize error:
error = np.zeros(3)

for i in range(k):
    error[i] = distance(centroids[i], centroids_old[i])

# Repeat Steps 2 and 3 until convergence:
while error.all() != 0:

    # Step 2: Assign samples to nearest centroid

    for i in range(len(samples)):
        labels[i] = assign_to_centroid(samples[i], centroids)

    # Step 3: Update centroids

    centroids_old = deepcopy(centroids)

    for i in range(k):
        points = [sepal_length_width[j] for j in range(len(sepal_length_width)) if labels[j] == i]
        centroids[i] = np.mean(points, axis=0)
        error[i] = distance(centroids[i], centroids_old[i])

colors = ['r', 'g', 'b']

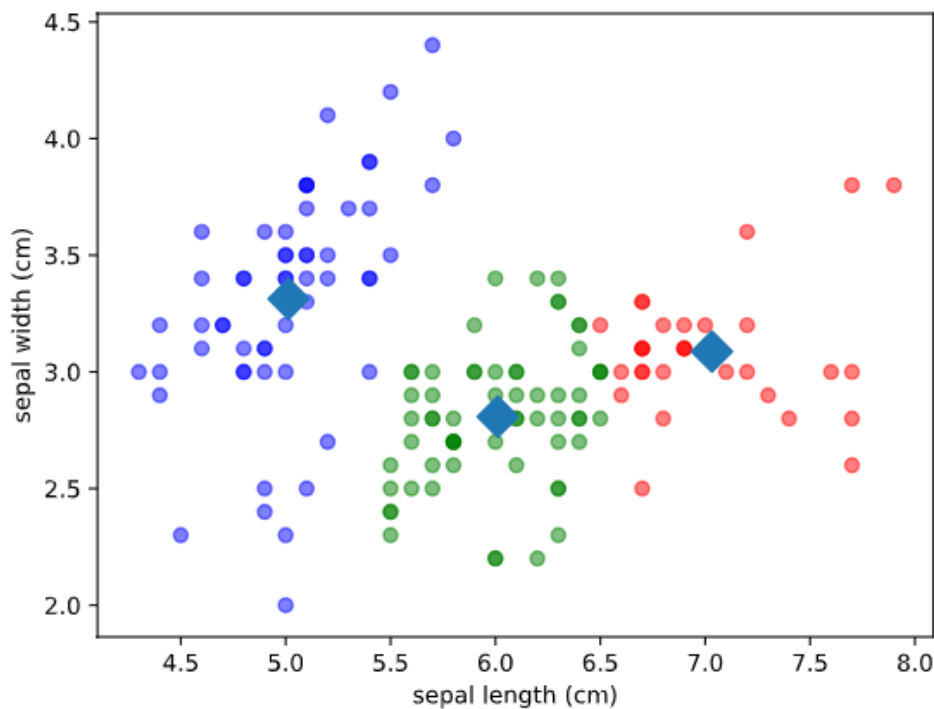
for i in range(k):
    points = np.array([sepal_length_width[j] for j in range(len(samples)) if labels[j] == i])
    plt.scatter(points[:, 0], points[:, 1], c=colors[i], alpha=0.5)

plt.scatter(centroids[:, 0], centroids[:, 1], marker='D', s=150)

plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')

plt.show()

```



Implementing K-Means: Scikit-Learn

Awesome, you have implemented k-means clustering from scratch!

Writing an whenever you need it can be very time-consuming and you might make mistakes and typos along the way. We will now show you how to implement k-means more efficiently – using the [scikit-learn](#) library.

There are many advantages to using scikit-learn. It can run k-means on datasets with as many features as your computer can handle, so it will be easy for us to use all four features of the iris data set instead of the two features that we used in the previous exercises.

Another big advantage of scikit-learn is that it is a widely-used open-source library. It is very well-tested, so it is much less likely to contain mistakes. Since so many people use it, there are many online resources that can help you if you get stuck. If you have a specific question about scikit-learn, it's very likely that other users have already asked and answered your question on public forums.

To import `KMeans` from `sklearn.cluster`:

```
from sklearn.cluster import KMeans
```

Copy to Clipboard

For Step 1, use the `KMeans()`

Preview: Docs Loading link description

[method](#)

to build a model that finds `k` clusters. To specify the number of clusters (`k`), use the `n_clusters` keyword argument:

```
model = KMeans(n_clusters = k)
```

Copy to Clipboard

For Steps 2 and 3, use the `.fit()` method to compute k-means clustering:

```
model.fit(X)
```

Copy to Clipboard

After k-means, we can now predict the closest cluster each sample in `X` belongs to. Use the `.predict()` method to compute cluster centers and predict cluster

Preview: Docs Loading link description

[index](#)

for each sample:

```
model.predict(X)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

First, import `KMeans` from `sklearn.cluster`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Somewhere after `samples = iris.data`, use `KMeans()` to create an instance called `model` to find 3 clusters. To specify the number of clusters, use the `n_clusters` keyword argument.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, use the `.fit()` method of `model` to fit the model to the array of points `samples`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

After you have the “fitted” model, determine the cluster labels of `samples`.

Then, print the `labels`.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans

iris = datasets.load_iris()

samples = iris.data

# Use KMeans() to create a model that finds 3 clusters
model = KMeans(n_clusters=3)

# Use .fit() to fit the model to samples
model.fit(samples)

# Use .predict() to determine the labels of samples
labels = model.predict(samples)

# Print the labels
print(labels)
```

New Data?

You used k-means and found three clusters of the `samples` data. But it gets cooler!

Since you have created a model that computed k-means clustering, you can now feed *new* data samples into it and obtain the cluster labels using the `.predict()`

Preview: Docs Loading link description

[method](#)

So, suppose we went to the florist and bought 3 more Irises with the measurements:

```
[[ 5.1  3.5  1.4  0.2 ]
 [ 3.4  3.1  1.6  0.3 ]
 [ 4.9  3.   1.4  0.2 ]]
```

Copy to Clipboard

We can feed this new data into the model and obtain the labels for them.

Instructions

Checkpoint 1 Failed, try again

1.

First, store the 2D matrix:

```
new_samples = np.array([[5.7, 4.4, 1.5, 0.4],
                        [6.5, 3. , 5.5, 0.4],
                        [5.8, 2.7, 5.1, 1.9]])
```

Copy to Clipboard

To test if it worked, print the `new_samples`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the model to predict labels for the `new_samples`, and print the predictions.

The output might look like:

```
[0 2 2]
```

Copy to Clipboard

Those are the predicted labels for our three new flowers. If you are seeing different labels, don't worry! Since the cluster centroids are randomly initialized, running the model repeatedly can produce different clusters with the same input data.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

You may want to see the names of the predicted classes instead of just the numerical targets.

Create a list called `new_names`. Use `iris.target_names` to convert the target numbers to text names and print the result.

You can use a list comprehension something like this:

```
new_names = [iris.target_names[label] for label in labels]
```

Copy to Clipboard

The output might look like:

```
['setosa', 'virginica', 'virginica']
```

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from sklearn.cluster import KMeans

iris = datasets.load_iris()
samples = iris.data
```

```

model = KMeans(n_clusters=3)

model.fit(samples)

# Store the new Iris measurements
new_samples = np.array([[5.7, 4.4, 1.5, 0.4],
                        [6.5, 3. , 5.5, 0.4],
                        [5.8, 2.7, 5.1, 1.9]])

# Predict labels for the new_samples
new_labels = model.predict(new_samples)

print(new_labels)

new_names = [iris.target_names[label] for label in new_labels]

print(new_names)

```

Visualize After K-Means

We have done the following using `sklearn` library:

- Load the embedded dataset
- Compute k-means on the dataset (where `k` is 3)
- Predict the labels of the data samples

And the labels resulted in either `0`, `1`, or `2`.

Let's finish it by making a scatter plot of the data again!

This time, however, use the `labels` numbers as the colors.

To edit colors of the scatter plot, we can set `c = labels`:

```

plt.scatter(x, y, c=labels, alpha=0.5)

plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')

```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Create an array called `x` that contains the Column `0` of `samples`.

Create an array called `y` that contains the Column `1` of `samples`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Make a scatter plot of `x` and `y`, using labels to define the colors.

```

import codecademylib3_seaborn
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans

iris = datasets.load_iris()

```

```

samples = iris.data

model = KMeans(n_clusters=3)

model.fit(samples)

labels = model.predict(samples)

print(labels)

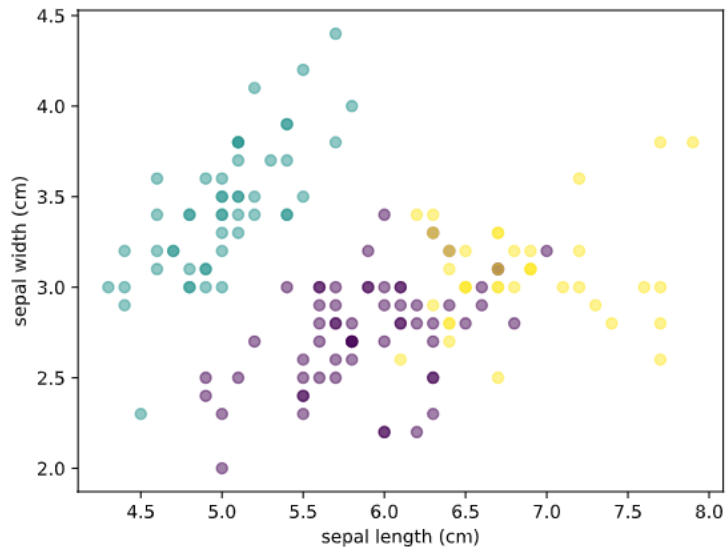
# Make a scatter plot of x and y and using labels to define the colors
x = samples[:,0]
y = samples[:,1]

plt.scatter(x, y, c=labels, alpha=0.5)

plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')

plt.show()

```



Evaluation

At this point, we have clustered the Iris data into 3 different groups (implemented using Python and using scikit-learn). But do the clusters correspond to the actual species? Let's find out!

First, remember that the Iris dataset comes with target values:

```
target = iris.target
```

Copy to Clipboard
It looks like:

```
[ 0 0 0 0 0 ... 2 2 2]
```

Copy to Clipboard
According to the metadata:

All the 0's are *Iris-setosa*
All the 1's are *Iris-versicolor*
All the 2's are *Iris-virginica*

Let's change these values into the corresponding species using the following code:

```
species = [iris.target_names[t] for t in list(target)]
```

Copy to Clipboard
Then we are going to use the Pandas library to perform a *cross-tabulation*.

Cross-tabulations enable you to examine relationships within the data that might not be readily apparent when analyzing total survey responses.

The result should look something like:

labels	setosa	versicolor	virginica
0	50	0	0
1	0	2	36
2	0	48	14

Copy to Clipboard
(You might need to expand this narrative panel in order to read the table better.)

The first column has the cluster labels. The second to fourth columns have the Iris species that are clustered into each of the labels.

By looking at this, you can conclude that:

Iris-setosa was clustered with 100% accuracy.
Iris-versicolor was clustered with 96% accuracy.
Iris-virginica didn't do so well.

Follow the instructions below to learn how to do a cross-tabulation.

Instructions

Checkpoint 1 Enabled

1.

`pandas` is already imported for you:

```
import pandas as pd
```

Copy to Clipboard

Add the code from the narrative to get the `species` list.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then create and print a DataFrame:

```
df = pd.DataFrame({'labels': labels, 'species': species})  
  
print(df)
```

Copy to Clipboard

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, use the `crosstab()` method to perform cross-tabulation:

```
ct = pd.crosstab(df['labels'], df['species'])  
print(ct)
```

Copy to Clipboard
Expand the right panel (output terminal).
How accurate are the clusters?

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from sklearn.cluster import KMeans
import pandas as pd

iris = datasets.load_iris()

samples = iris.data

target = iris.target

model = KMeans(n_clusters=3)

model.fit(samples)

labels = [iris.target_names[s] for s in model.predict(samples)]

# Code starts here:

species = [iris.target_names[t] for t in list(target)]

df = pd.DataFrame({'labels': labels, 'species': species})

ct = pd.crosstab(df['labels'], df['species'])

print(ct)
```

Elbow Method- find the Number of Clusters

At this point, we have grouped the Iris plants into 3 clusters. But suppose we didn't know there are three species of Iris in the dataset, what is the best number of clusters? And how do we determine that?

Before we answer that, we need to define what is a *good* cluster?

Good clustering results in tight clusters, meaning that the samples in each cluster are bunched together. How spread out the clusters are is measured by *inertia*. Inertia is the distance from each sample to the centroid of its cluster. The lower the inertia is, the better our model has done.

You can check the inertia of a model by:

```
print(model.inertia_)
```

Copy to Clipboard

For the Iris dataset, if we graph all the `k`s (number of clusters) with their inertias:

Notice how the graph keeps decreasing.

Ultimately, this will always be a trade-off. If the inertia is too large, then the clusters probably aren't clumped close together. On the other hand, if there are too many clusters, the individual clusters might not be different enough from each other. The goal is to have low inertia *and* a small number of clusters.

One of the ways to interpret this graph is to use the *elbow method*: choose an "elbow" in the inertia plot - when inertia begins to decrease more slowly.

In the graph above, 3 is the optimal number of clusters.

Instructions

Checkpoint 1 Failed, try again

1.

First, create two lists:

`num_clusters` that has values from 1, 2, 3, ... 8

`inertias` that is empty

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then, iterate through `num_clusters` and calculate k-means for each number of clusters.

Add each of their inertias into the `inertias` list.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Plot the `inertias` vs `num_clusters`:

```
plt.plot(num_clusters, inertias, '-o')

plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')

plt.show()
```

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.cluster import KMeans

iris = datasets.load_iris()

samples = iris.data

# Code Start here:

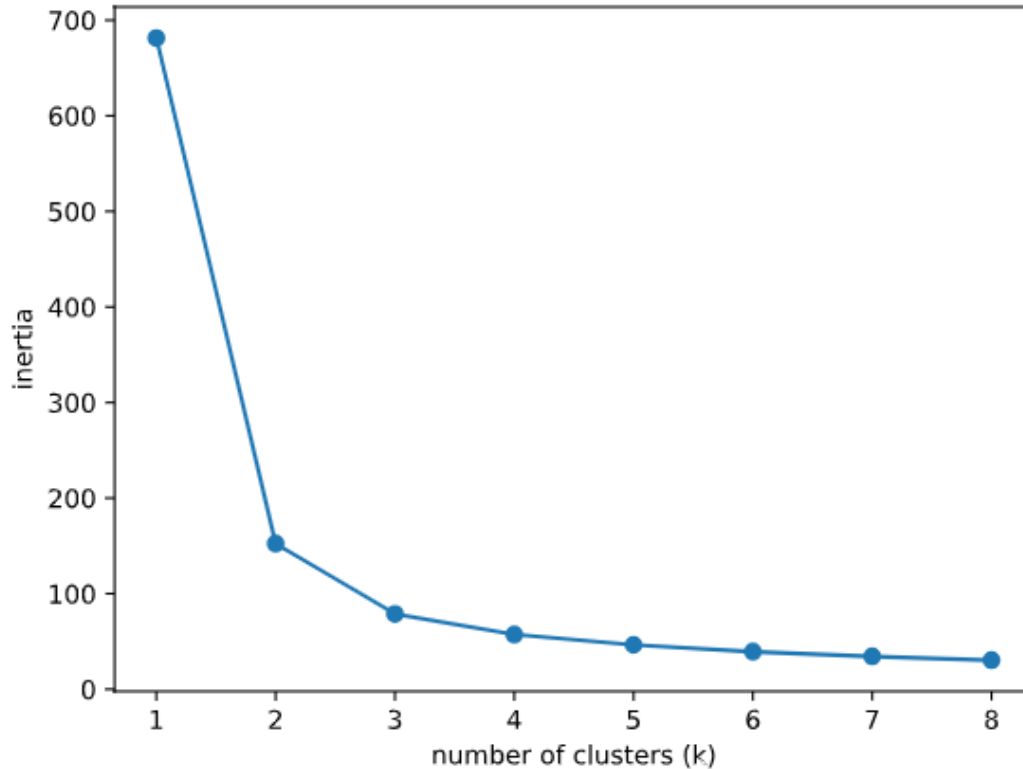
num_clusters = list(range(1, 9))
inertias = []

for k in num_clusters:
    model = KMeans(n_clusters=k)
    model.fit(samples)
    inertias.append(model.inertia_)

plt.plot(num_clusters, inertias, '-o')

plt.xlabel('number of clusters (k)')
plt.ylabel('inertia')

plt.show()
```



PCA

What is PCA?

Learn how, when, and why to use Principal Component Analysis (PCA) as part of the Machine Learning lifecycle.

Introduction to PCA

In the world of Machine Learning, any model that we implement will be more valuable when the features are engineered to suit the question we're trying to answer. With many datasets, we can simply include all available features, which gives us the full picture about our observations. For example, it's straightforward to see a correlation between height and weight for a patient dataset. Some datasets, however, have very large numbers of features. If our example patient dataset expanded to include 20 different features, how would we visualize and correlate this data? When it comes time to actually process the data and train the model, we often hit computational or complexity limits.

How do we leverage correlations within the data to make fewer, better features without losing the information included in the dataset?

Situations like this are a great use case for implementing Principal Component Analysis. PCA is a technique where we can reduce the number of features in a dataset without losing any of the information we have. Sounds pretty great right? This article will cover various aspects of PCA, so let's dive in.

Laying the groundwork for PCA

Before we dive into the specifics of PCA, we need to understand the importance of information. In particular, we need to understand how variance plays into the level of information in a dataset. For the purposes of this article, we will be looking at a synthetic dataset about local pizza stores. Let's see what data we have:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pizza.csv')

print(df.columns)
```

Output:

```
Index(['revenue', 'total_customers', 'amt_flour', 'amt_tomatoes',
      'amt_cheese'],
      dtype='object')
```

Each row of data pertains to an individual store, and gives information about how the store is doing overall with inventory and sales. Suppose we look at just `revenue` and `total_customers`, and we see the following information:

In this scenario, the value for `total_customers` has a value of 500 for every row. While every row has a value, and therefore has *data*, this column does not provide a lot of *information*, due to the lack of variance in the values. While we could include this feature in our downstream analytics, it doesn't provide any additional value, because each row would have the same data.

Now let's look at what the real data shows us for these two columns:

```
df[['revenue', 'total_customers']].head()
```

Output:

	revenue	total_customers
0	9931.860710	615.336682
1	12397.798907	725.440590
2	11983.079340	630.987797
3	13910.984353	746.264763
4	13083.859701	689.060436

As we can see, the real dataset has far more variance in these two columns. Since each feature has significant variance, these features provide valuable information about our observations, and should therefore be included in our analysis.

Variance alone is one indicator of the level of information in a dataset, but is not the only factor. To expand on the idea of variance within a dataset, we will look at the Coefficient of Variance, or CV for short. The premise here is that variance must be taken into context with the central tendencies of that dataset. For example, if a dataset has a variance of 5, that will mean very different things if the mean is 2 vs. a dataset with a mean of 100.

Now, let's actually calculate the Coefficient of Variance for each of our columns.

```
import numpy as np
```

```
#define function to calculate cv
cv = lambda x: np.std(x, ddof=1) / np.mean(x) * 100

print(df.apply(cv))
```

Output:

```
revenue      10.001034
total_customers  5.138628
amt_flour      9.128946
amt_tomatoes   9.926973
amt_cheese     6.401035
dtype: float64
```

All of the features in this dataset have enough variance where they will be useful in analysis. Since variance is an important factor to PCA, these features will ultimately be ordered by the level of information (i.e. variance) they have. For this dataset, that means, in order of importance, PCA will look at `revenue`, `amt_tomatoes`, `amt_flour`, `amt_cheese`, and then `total_customers`. While the results of PCA won't resemble our original features, they will be a mathematical representation of the information contained in the original features, which has value for analytical purposes.

Coding question

Questions

The kind of information we have can vary from dataset to dataset, and thus can the Coefficient of Variance. Use what you just learned on a new set of synthetic pizza store data, `pizza_new.csv`. Calculate the Coefficients of Variance for each feature in the dataset. Then, create a ranked order Python list for the features in the dataset in terms of information for PCA, from most important to least important.

The Math Behind PCA

At this point, we need to address how we can actually take information from multiple features and distill it down into a smaller number of features. Let's dive deeper into each of the steps that lead to PCA.

Data Matrix

First, we need to isolate a *Data Matrix*, another name for a dataset. This data matrix holds all of the features and information that we are interested in. Many datasets will have columns that hold information (i.e. features), and other columns that we want to predict (i.e. labels). Using our previous pizza dataset, we have 5 features in our data matrix.

revenue	total_customers	amt_flour	amt_tomatoes	amt_cheese
9931.860710	615.336682	37.662830	174.102712	139.402208
12397.798907	725.440590	44.424509	239.119556	168.425842
11983.079340	630.987797	40.259276	224.084121	146.612426
13910.984353	746.264763	43.633485	227.096619	170.726464
13083.859701	689.060436	48.964844	221.383478	154.786070

Covariance Matrix

From here, the next step of PCA is to calculate a *covariance matrix*. Essentially, a covariance matrix is calculating how much a feature changes with changes in every other feature, i.e., we're looking at the relative variance between any two features. Mathematically, the formula for covariance between two features x and y is:

$$\text{Cov}(X,Y) = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

We will do this equation for the relationship between each of our features, ultimately resulting in a covariance matrix that shows relationships for the entire dataset. Simplifying our example dataset, we could think about our pizza dataset having five individual features with the names a , b , c , d , and e . Our ultimate covariance matrix, thus, would end up looking like this:

```
[Cova,aCova,bCova,cCova,dCova,eCovb,aCovb,bCovb,cCovb,dCovb,eCovc,aCovc,bCovc,cCovc,dCovc,eCovd,aCovd,bCovd,cCovd,dCovd,eCove,aCove,bCove,cCove,dCove,e]
```

Luckily, with the `pandas` package, we can calculate a covariance matrix with the `.cov()` method. For our pizza dataset, this results in the following:

```
revenue    total_customers  amt_flour  amt_tomatoes  amt_cheese
revenue      1.563517e+06    31853.053820    3713.664277  19980.869509    9152.568482
total_customers  3.185305e+04    1295.096885    105.909335    577.443015    256.641069
amt_flour      3.713664e+03    105.909335    16.894551     66.165738     29.130750
amt_tomatoes    1.998087e+04    577.443015    66.165738    500.715936    162.221734
amt_cheese      9.152568e+03    256.641069    29.130750    162.221734    105.370280
```

One important point to note is that along the primary diagonal (from top-left to bottom-right), we see the same variance values that we calculated for each individual column earlier on.

Matrix Factorization, Eigenvalues, and Eigenvectors

We now have a matrix of variance values for our features. The next step in PCA revolves around *matrix factorization*. Without going into too much detail, our goal with matrix factorization is to find a pair of smaller matrices whose product would equal our covariance matrix. Another way of thinking about it: we want to find a smaller matrix that captures the majority of our information.

An important part of this matrix factorization are *Eigenvectors*. Eigenvectors are vectors (mathematical concepts that have direction and magnitude) that do not change direction when a transformation is applied to them. In the context of data matrices, these eigenvectors give us a direction to "rotate" the dataset in n -dimensional space so we can look at the entire dataset from a simplified perspective. The *eigenvalues* are related to the relative variation described by each principal component.

For a matrix A , the eigenvectors and eigenvalues are the solutions to the following equation:

$$\det(A - \lambda I)$$

$$\det(A - \lambda I)$$

After some linear algebra, for our covariance matrix, we are looking for the solution to the following matrix, which will be our eigenvectors and eigenvalues.

```
det[1.563517e+06-λ31853.0538203713.66427719980.8695099152.5684823.185305e+041295.096885-λ105.909335577.443015
256.6410693.713664e+03105.90933516.894551-λ66.16573829.1307501.998087e+04577.44301566.165738500.715936-λ162.2
217349.152568e+03256.64106929.130750162.221734105.370280-λ]
```

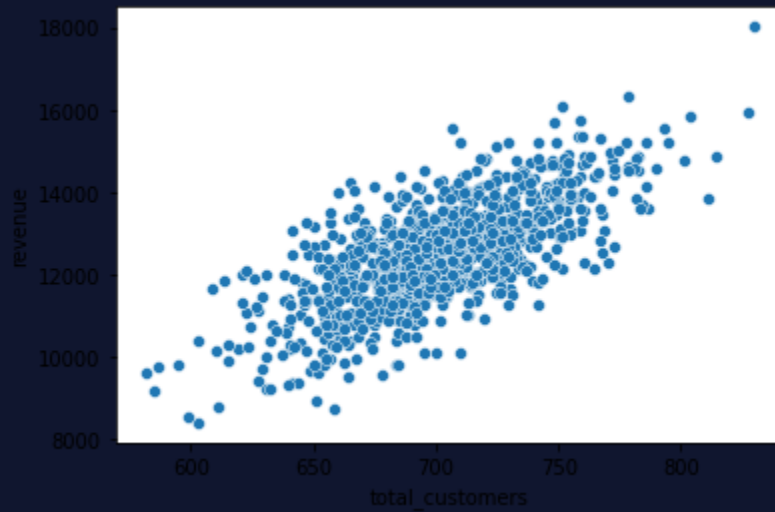
Principal Components

All of the underlying math behind PCA results in *principal components*, but what exactly are they? Principal components are a linear combination of all the input features from the original dataset. By using the eigenvectors we

calculated earlier, we can “rotate” our dataset features from an n-dimensional space into a 2-dimensional space, which is easier for us to understand and analyze.

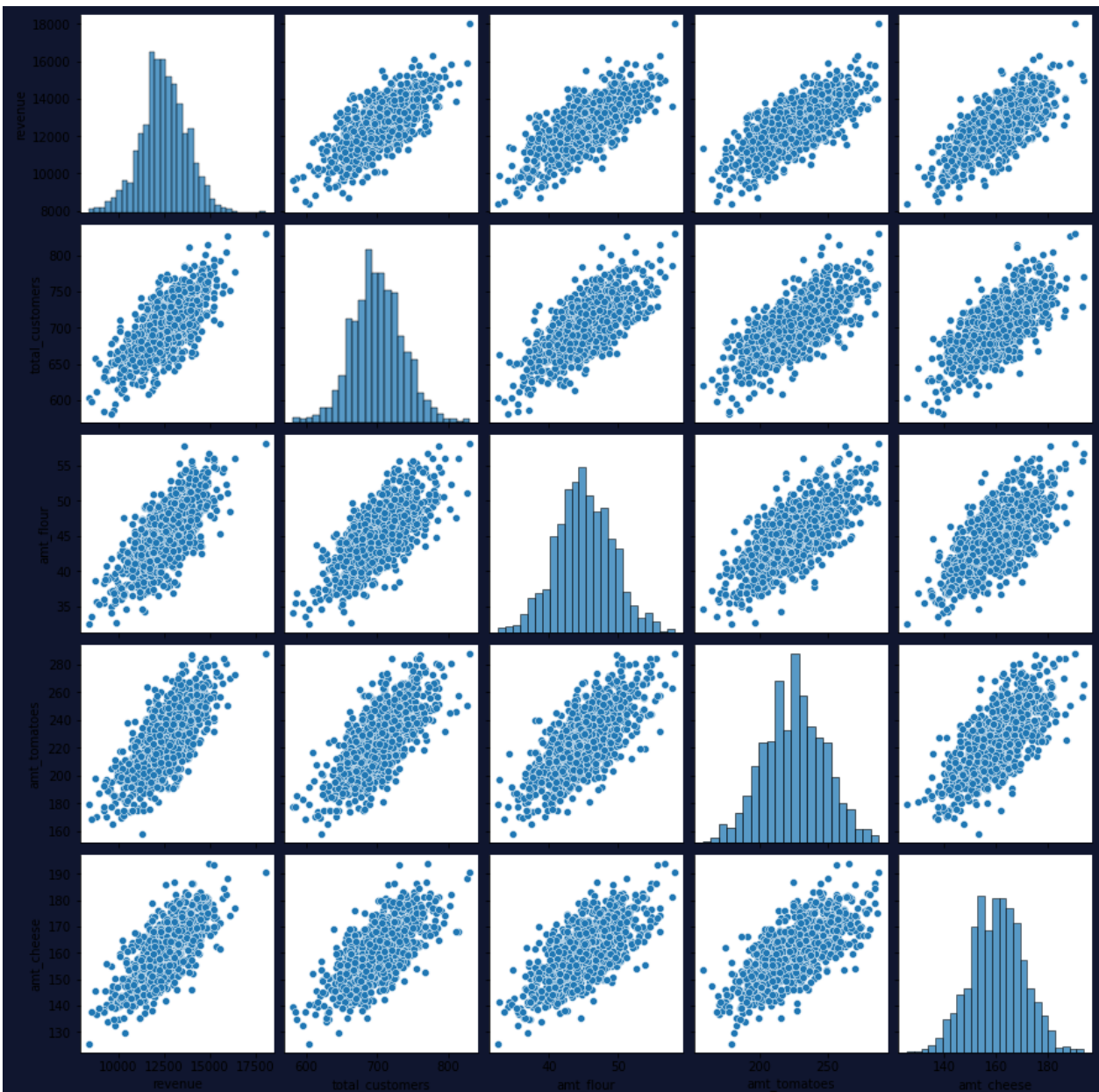
To illustrate this point, let’s return to our pizza dataset. We can observe the correlation between our `revenue` and `total_customers` features.

```
sns.scatterplot(x='total_customers',  
                y='revenue',  
                data=df)
```



We can see a positive correlation between these two features, and could use that information to guide any analysis we perform. We can also do correlation plots for every combination of features, like so:

```
sns.pairplot(df)
```



Each individual combination of features will have its own correlation and variance, both of which provide valuable information about that relationship. When comparing two features at a time, these relationships are more understandable. If we wanted to, however, look at all of the feature relationships and information at once, it would be very difficult to decipher, as we cannot visualize data in a 5-dimensional space.

By using PCA, however, we can reduce the dimensionality of our dataset into a 2-dimensional dataset, allowing for better visualization. Let's see the result.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_array = pca.fit_transform(df)
```

Output:

```
array([[ -2572.64663126,  -37.43225524],
       [ -104.21066949,   31.54952593],
       [ -521.0563251 ,  -54.19045713],
       ...,
       [ 1429.58053669,   -5.98229122],
       [-3550.23561932,   23.8935932 ],
       [ -481.85213117,  -34.14891261]])
```

As we can see, by running PCA on our original dataset, we were able to take our 5 features and reduce the dimensions down to 2 principal components. With 2 dimensions, we can now plot the data on a single scatterplot:

```
sns.scatterplot(pca_array[:,0], pca_array[:,1])
```



While it can be difficult to interpret what this new data matrix is showing us, it does hold valuable information that can be used in a variety of contexts. The axes of this chart are the two most impactful principal components as part of our analysis, and were the two that we decided to keep.

Fill in the blank

Questions

The below statements outline the overall process of PCA. Fill in the blanks with the correct terms to validate your understanding.

Code

```
For a given dataset, we start by calculating a
blank 1
for all of our features.
```

```
Afterwards, we perform
blank 2
```

```
, which will separate out the dataset and give us two results:
```

```
1)
```

```
blank 3
```

```
, also known as Principal Components, which define the direction, or "rotation", of
our new data space
```

```
2)
```

```
blank 4
```

```
, which determine the magnitude of that new data space
```


Answer Choices

principal values
covariance matrix
correlation matrix
principal vectors
eigenvalues
eigenvectors
matrix factorization
matrix decomposition

Click or drag and drop to fill in the blank

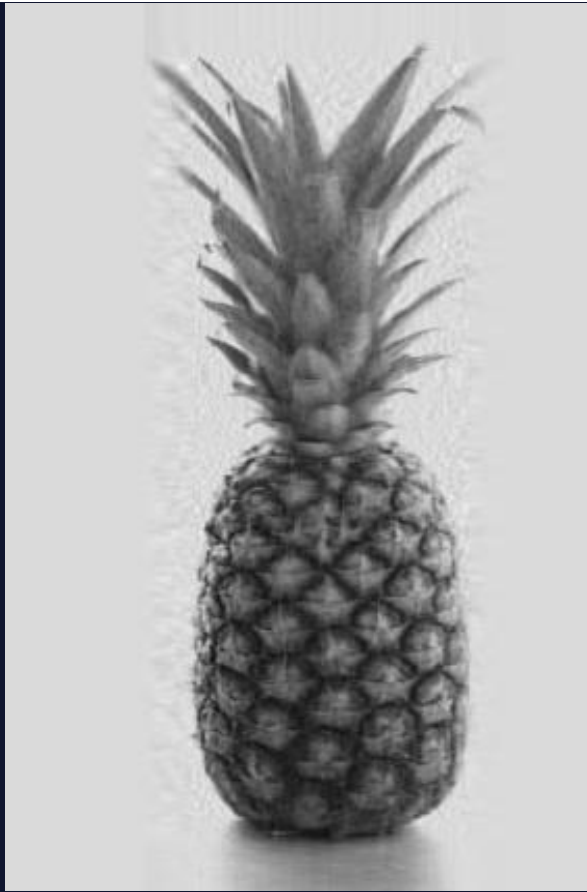
The How, Where, and Why of PCA

PCA serves an important role in many different parts of data science and analytics in general, as this process allows us to maximize the amount of information we can extract from data while reducing computational time down the line. We just saw a common use case for PCA with our pizza dataset. We took a higher dimensional dataset (5 dimensions in our case), and reduced it down to 2 dimensions. This two-dimensional dataset can now be an input to a variety of Machine Learning models. For example, we could use this new dataset as part of a forecasting model, or perform linear regression. These techniques would have been much more difficult prior to performing PCA.

PCA is also inherently an unsupervised learning algorithm and can be used to identify clusters in data on its own. Very similar to the popular `k-means` algorithms, PCA will look at overall similarities between the different features in a dataset. When we set the number of principal components to keep, we are defining the number of similar “rotations” of our dataset, which will act very much like a cluster of their own. Typically, many practitioners will implement PCA as a precursor to other clustering algorithms to augment the accuracy, but it is an interesting application to do clustering with PCA alone!

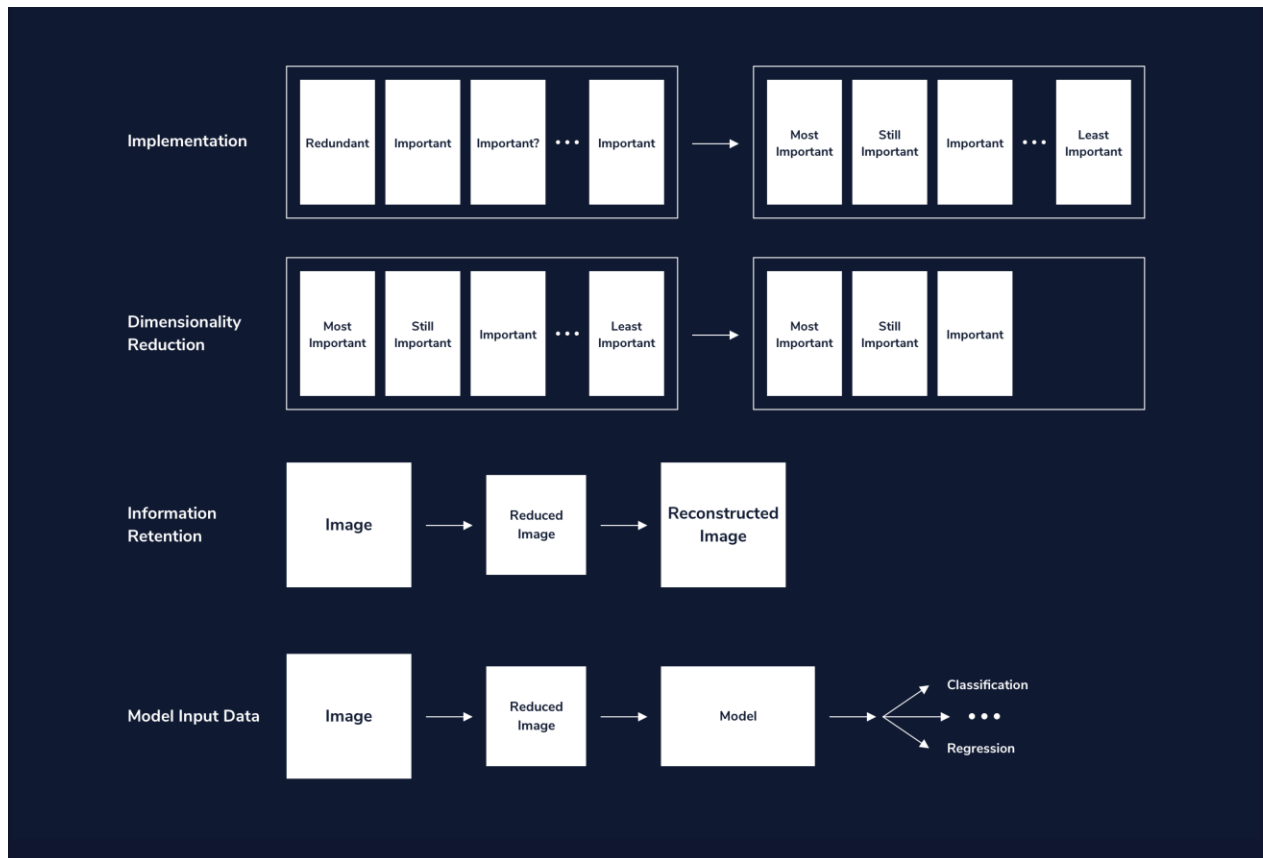
Another, very powerful, application of PCA is with image processing. Images hold a vast amount of information in each file, and analyzing this information can have very useful applications. Image classification, for example, uses algorithms to detect the subject of an image, or find a particular object within the image. Overall, it can be very costly to process image data, due to the high dimensionality it has. By applying PCA, however, practitioners can reduce the number of features for the image with minimal information loss and continue their processing.

Below are two images from a Fruit and Vegetable Image dataset. The first image is the original image, and the second is the reconstructed image using the calculated eigenvectors after performing PCA. Note that, in this very quick example, there is minimal information loss, meaning that data could be effectively used for analytics.



Summary

In this article, you learned the underlying mathematics behind Principal Component Analysis (PCA) and got a bird's eye view of how, when, and where to implement PCA. Principal Component Analysis is a powerful tool that provides a mechanism to reduce dimensionality and simplify datasets without losing the valuable information they inherently contain. The following image summarizes the different applications of PCA and we are now ready to delve into these in detail!



Implementing PCA in NumPy I

In this exercise, we will perform PCA using the NumPy

Preview: Docs Loading link description

[method](#)

`np.linalg.eig`, which performs eigendecomposition and outputs the eigenvalues and eigenvectors.

The **eigenvalues** are related to the relative variation described by each principal component. The **eigenvectors** are also known as the principal axes. They tell us how to transform (rotate) our data into new features that capture this variation.

To implement this in Python:

```
correlation_matrix = data_matrix.corr()
eigenvalues, eigenvectors = np.linalg.eig(correlation_matrix).
```

Copy to Clipboard

First, we generate a correlation matrix using `.corr()`

Next, we use `np.linalg.eig()` to perform eigendecomposition on the correlation matrix. This gives us two outputs – the eigenvalues and eigenvectors.

Instructions

Checkpoint 1 Passed

1.

The DataFrame that you created in the previous exercise, `data_matrix`, is loaded for you. Find the correlation matrix for the features in `data_matrix` and save the result as `correlation_matrix`.

Uncomment the lines that show the heatmap and notice that there are pairs of features with very high correlations.

Checkpoint 2 Passed

2.

Using the `correlation_matrix`, find the eigenvalues and eigenvectors using the NumPy method `np.linalg.eig()`. Save them as `eigenvalues` and `eigenvectors`, respectively, then print them out.

You should see that `eigenvalues` contains 16 numbers, while `eigenvectors` contains 16 vectors with 16 values, each.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import codecademylib3

data_matrix = pd.read_csv('./data_matrix.csv')

# 1. Use the `.corr()` method on `data_matrix` to get the correlation matrix
correlation_matrix = data_matrix.corr()

## Heatmap code:
red_blue = sns.diverging_palette(220, 20, as_cmap=True)
sns.heatmap(correlation_matrix, vmin = -1, vmax = 1, cmap=red_blue)
plt.show()

# 2. Perform eigendecomposition using `np.linalg.eig`
eigenvalues, eigenvectors = np.linalg.eig(correlation_matrix)

# 3. Print out the eigenvectors and eigenvalues
print('eigenvectors: ')
print(eigenvectors)

print('eigenvalues: ')
print(eigenvalues)
```

Implementing PCA in NumPy II - Analysis

After performing PCA, we generally want to know how useful the new features are. One way to visualize this is to create a scree plot, which shows the proportion of information described by each principal component.

The proportion of information explained is equal to the relative size of each eigenvalue:

```
info_prop = eigenvalues / eigenvalues.sum()
print(info_prop)
```

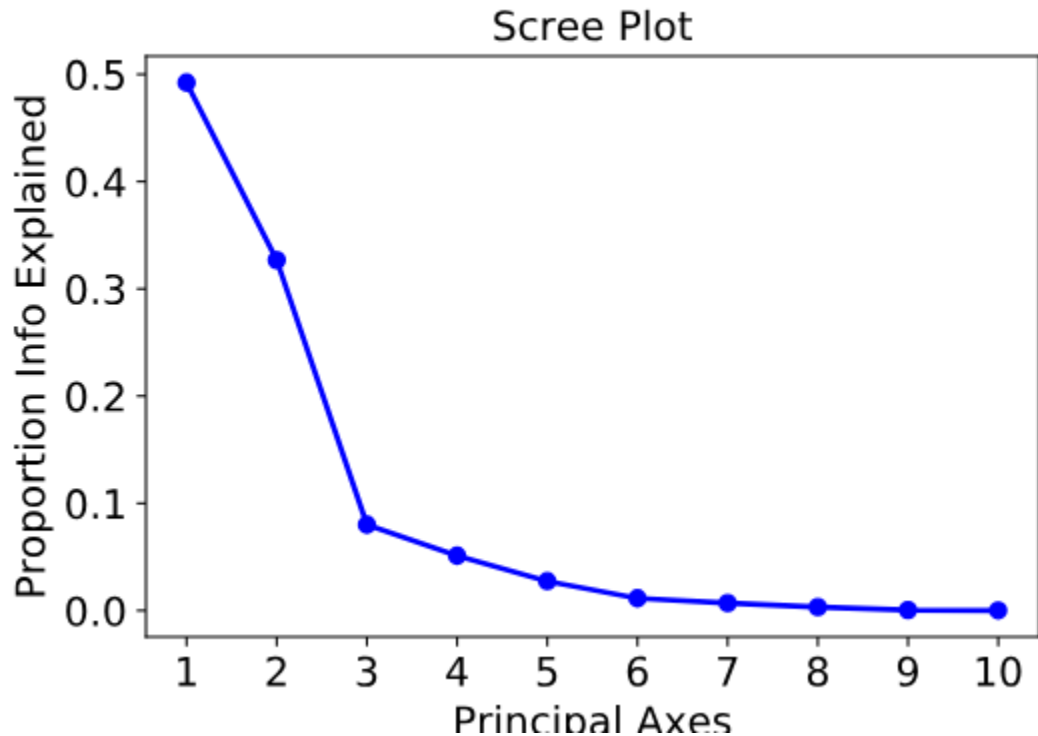
Copy to Clipboard

To create a scree plot, we can then plot these relative proportions:

```
plt.plot(np.arange(1, len(info_prop)+1),
```

```
info_prop,  
'bo-')  
plt.show()
```

Copy to Clipboard



From this plot, we see that the first principal component explains about 50% of the variation in the data, the second explains about 30%, and so on.

Another way to view this is to see how many principal axes it takes to reach around 95% of the total amount of information. Ideally, we'd like to retain as few features as possible while still reaching this threshold.

To do this, we need to calculate the cumulative sum of the `info_prop` vector we created earlier:

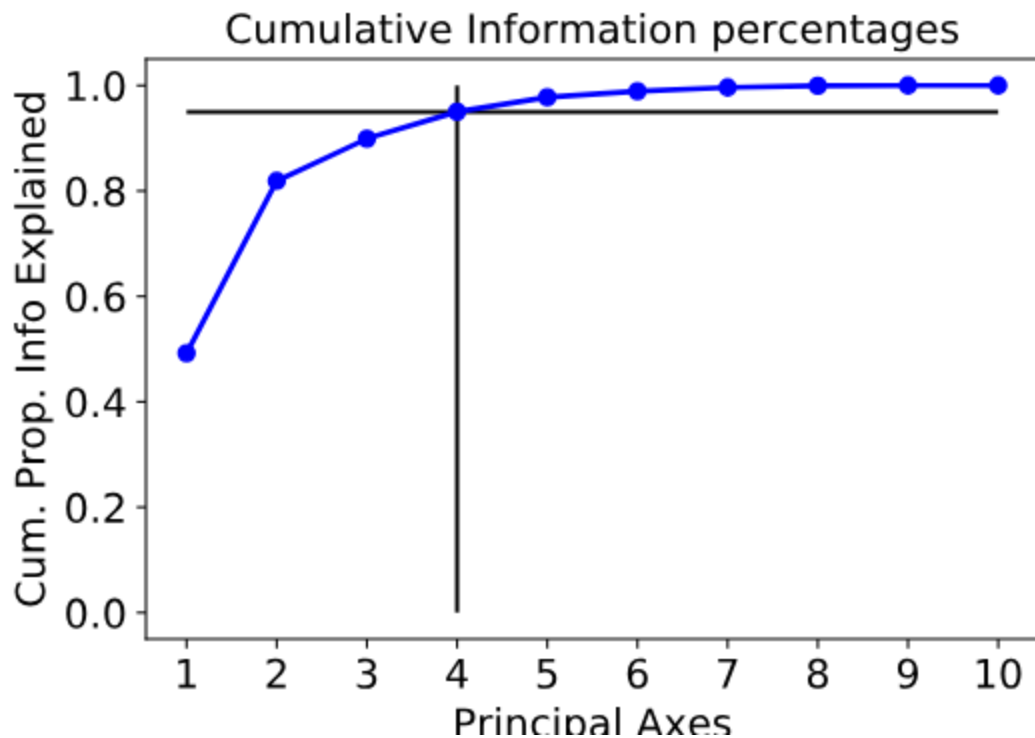
```
cum_info_prop = np.cumsum(info_prop)
```

Copy to Clipboard

We can then plot these values using matplotlib:

```
plt.plot(np.arange(1, len(cum_info_prop)+1),  
         cum_info_prop,  
         'bo-')  
plt.hlines(y=.95, xmin=0, xmax=15)  
plt.vlines(x=4, ymin=0, ymax=1)  
plt.show()
```

Copy to Clipboard



From this plot, we see that four principal axes account for 95% of the variation in the data.

Instructions

Checkpoint 1 Failed, try again

1.

The array of `eigenvalues` is loaded for you. Find the proportion of each eigenvalue compared to the sum of the eigenvalues. Save the results as an array named `info_prop`.

Then, uncomment the plotting code below to generate a scree plot. Notice that the first principal axes contain nearly 50% of the information!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using `info_prop`, calculate the cumulative sum of the information proportions. Save the resulting array as `cum_info_prop`.

Then, uncomment the plotting code, which generates a plot of the cumulative proportion of information explained. Notice how four principal components describe about 95% of the information in the data.

```
import numpy as np
import pandas as pd
import codecademylib3
import matplotlib.pyplot as plt

eigenvalues = pd.read_csv('eigenvalues.csv')['eigenvalues'].values

# 1. Find the proportion of information for each eigenvector, which is equal to the
# eigenvalues divided by the sum of all eigenvalues
info_prop = eigenvalues / eigenvalues.sum()

## Plot the principal axes vs the information proportions for each principal axis

plt.plot(np.arange(1,len(info_prop)+1),info_prop, 'bo-', linewidth=2)
plt.title('Scree Plot')
plt.xlabel('Principal Axes')
plt.xticks(np.arange(1,len(info_prop)+1))
plt.ylabel('Proportion of Information Explained')
plt.show()
```

```
plt.clf()

# 2. Find the cumulative sum of the proportions
cum_info_prop = np.cumsum(info_prop)

## Plot the cumulative proportions array

plt.plot(np.arange(1,len(info_prop)+1), cum_info_prop, 'bo-', linewidth=2)
plt.hlines(y=.95, xmin=1, xmax=15)
plt.vlines(x=4, ymin=0, ymax=1)
plt.title('Cumulative Information Proportions')
plt.xlabel('Principal Axes')
plt.xticks(np.arange(1,len(info_prop)+1))
plt.ylabel('Cumulative Proportion of Information Explained')
plt.show()
```

Implementing PCA using Scikit-Learn

Another way to perform PCA is using the scikit-learn module `sklearn.decomposition.PCA`.

The steps to perform PCA using this

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

are:

Standardize the data matrix. This is done by subtracting the mean and dividing by the standard deviation of each column vector.

```
mean = data.mean(axis=0)
sttd = data.std(axis=0)
data_standardized = (data - mean) / sttd
```

Copy to Clipboard

Perform eigendecomposition by fitting the standardized data. We can access the eigenvectors using the `components_` attribute and the proportional sizes of the eigenvalues using the `explained_variance_ratio_` attribute.

```
pca = PCA()
components = pca.fit(data_standardized).components_
components = pd.DataFrame(components).transpose()
components.index = data_matrix.columns
print(components)
```

Copy to Clipboard

	Comp1	Comp2	Comp3	Comp4
bill_length_mm	0.455250	0.597031	0.644301	0.145523
bill_depth_mm	-0.400335	0.797767	-0.418427	-0.167986
flipper_length_mm	0.576013	0.002282	-0.232084	-0.783799
body_mass_g	0.548350	0.084363	-0.596600	0.579882

```
var_ratio = pca.explained_variance_ratio_
var_ratio = pd.DataFrame(var_ratio).transpose()
print(var_ratio)
```

Copy to Clipboard

	Comp1	Comp2	Comp3	Comp4
Proportion of Variance	0.688439	0.193129	0.091309	0.027123

This module has many advantages over the NumPy method, including a number of different solvers to calculate the principal axes. This can greatly improve the quality of the results.

Instructions

Checkpoint 1 Enabled

1.

The data matrix is loaded for you as `data_matrix`. Standardize all of the columns in the data matrix and save the result as `data_matrix_standardized`, then print out the first 5 rows to inspect it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Will we now use the `PCA` module from `sklearn.decomposition.PCA` to calculate the principal components (eigenvectors) for the standardized data matrix.

Fit and calculate the principal components using the `fit()` method and `components_` attribute. Save the results as `components` and then uncomment the code to transpose, index, and print the components/eigenvectors.

You should see 16 eigenvectors. Each one should have a value (or weight) for the each of the 16 original features.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Using the same `pca` object from the previous checkpoint, we can retrieve the variance (information) ratios for each principal component using the `explained_variance_ratio_` property.

Save the variance ratios as `var_ratio` and then uncomment the code to re-save the values as a DataFrame and print them out.

Are they consistent with our original finding that the first four components account for about 95% of the information in the data?

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import codecademylib3

data_matrix = pd.read_csv('./data_matrix.csv')

# 1. Standardize the data matrix
mean = data_matrix.mean(axis=0)
```



```

std = data_matrix.std(axis=0)
data_matrix_standardized = (data_matrix - mean) / std
print(data_matrix_standardized.head())

# 2. Find the principal components
pca = PCA()
components = pca.fit(data_matrix_standardized).components_
components = pd.DataFrame(components).transpose()
components.index = data_matrix.columns
print(components)

# 3. Calculate the variance/info ratios
var_ratio = pca.explained_variance_ratio_
var_ratio = pd.DataFrame(var_ratio).transpose()
print(var_ratio)

```

Projecting the Data onto the principal Axes

Once we have performed PCA and obtained the eigenvectors, we can use them to project the data onto the first few principal axes. We can do this by taking the dot product of the data and eigenvectors, or by using the `sklearn.decomposition.PCA` module as follows:

```

from sklearn.decomposition import PCA

# only keep 3 PCs
pca = PCA(n_components = 3)

# transform the data using the first 3 PCs
data_pcomp = pca.fit_transform(data_standardized)

# transform into a dataframe
data_pcomp = pd.DataFrame(data_pcomp)

# rename columns
data_pcomp.columns = ['PC1', 'PC2', 'PC3']

# print the transformed data
print(data_pcomp.head())

```

Copy to Clipboard

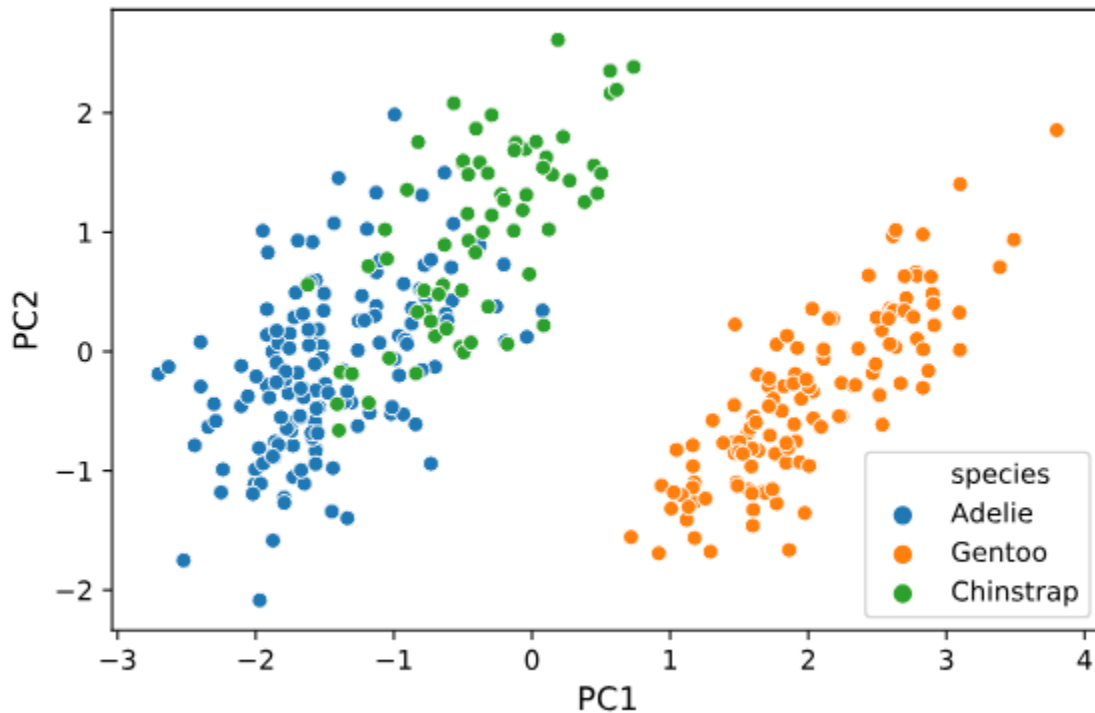
	Comp1	Comp2	Comp3
0	-1.843445	0.047702	-0.232794
1	-1.306762	-0.428348	-0.029562
2	-1.369181	-0.154476	0.198672
3	-1.878827	-0.002048	-0.618596
4	-1.911748	0.829210	-0.686584

Once we have the transformed data, we can look at a scatter plot of the first two transformed features using seaborn or matplotlib. This allows us to view relationships between multiple features at once in 2D or 3D space. Often, the the first 2-3 principal components result in clustering of the data.

Below, we've plotted the first two principal components for a dataset of measurements for three different penguin species:

```
sns.lmplot(x='PC1', y='PC2', data=data_pcomp, hue='species', fit_reg=False)
plt.show()
```

Copy to Clipboard



Instructions

Checkpoint 1 Enabled

1.

Let's only keep the first four principal components because they account for 95% of the information in the data! Use the `.fit_transform()` method to transform the standardized data matrix into the new features and save the result as `data_pcomp`. Then uncomment the code to save as a dataframe, re-label, and print the transformed data.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment the lines that create a seaborn scatter plot of the first two components. Do you notice any clustering occurring?

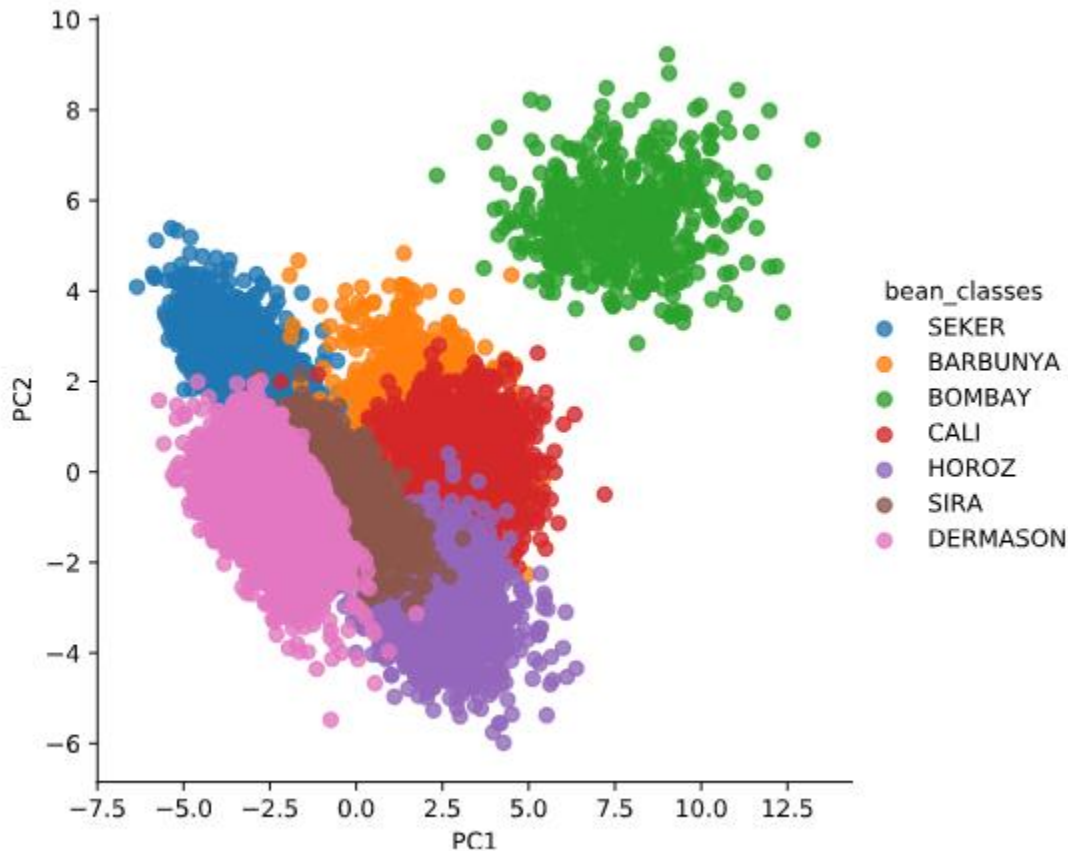
```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

data_matrix_standardized = pd.read_csv('./data matrix standardized.csv')
classes = pd.read_csv('./classes.csv')['Class']

# 1. Transform the data into 4 new features using the first PCs
pca = PCA(n_components = 4)
data_pcomp = pca.fit_transform(data_matrix_standardized)
data_pcomp = pd.DataFrame(data_pcomp)
data_pcomp.columns = ['PC1', 'PC2', 'PC3', 'PC4']
print(data_pcomp.head())

## 2. Plot the first two principal components colored by the bean classes
```

```
data_pcomp['bean_classes'] = classes
sns.lmplot(x='PC1', y='PC2', data=data_pcomp, hue='bean_classes', fit_reg=False)
plt.show()
```



PCA as Features

So far we have used PCA to find principal axes and project the data onto them. We can use a subset of the projected data for modeling, while retaining most of the information in the original (and higher-dimensional) dataset.

For example, recall in the previous exercise that the first four principal axes already contained 95% of the total amount of variance (or information) in the original data. We can use the first four components to train a model, just like we would on the original 16 features.

Because of the lower dimensionality, we should expect training times to be faster. Furthermore, the principal axes ensure that each new feature has no correlation with any other, which can result in better model performance.

In this checkpoint, we will be using the first four principal components as our training data for a Support Vector Classifier (SVC). We will compare this to a model fit with the entire dataset (16 features) using the average likelihood score. Average likelihood is a model evaluation metric; the higher the average likelihood, the better the fit.

Instructions

Checkpoint 1 Enabled

1.

Read through the code to make sure that you understand what's happening. Here are the steps:

- Transform the original data by projecting it onto the first four principal axes. We chose four PCs because we previously found that they contain 95% of the variance in the original data

- Split the data into 67% training and 33% testing sets

- Use the transformed training data to fit an SVM model

- Print out the average likelihood score for the testing data

- Re-split the original 16 standardized features into training and test sets

- Fit the same SVM model on the training set with all 16 features

- Print out the average likelihood score for the test data

Notice that the score for the model using the first 4 principal components is higher than for the model that was fit with the 16 original features. We only needed 1/4 of the data to get even better model performance!

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split

data_matrix_standardized = pd.read_csv('./data_matrix_standardized.csv')
classes = pd.read_csv('./classes.csv')

# We will use the classes as y
y = classes.Class.astype('category').cat.codes

# Get principal components with 4 features and save as X
pca_1 = PCA(n_components=4)
X = pca_1.fit_transform(data_matrix_standardized)

# Split the data into 33% testing and the rest training
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)

# Create a Linear Support Vector Classifier
svc_1 = LinearSVC(random_state=0, tol=1e-5)
svc_1.fit(X_train, y_train)

# Generate a score for the testing data
score_1 = svc_1.score(X_test, y_test)
print(f'Score for model with 4 PCA features: {score_1}')

# Split the original data into 33% testing and the rest training
X_train, X_test, y_train, y_test = train_test_split(data_matrix_standardized, y,
test_size=0.33, random_state=42)

# Create a Linear Support Vector Classifier
svc_2 = LinearSVC(random_state=0)
svc_2.fit(X_train, y_train)

# Generate a score for the testing data
score_2 = svc_2.score(X_test, y_test)
print(f'Score for model with original features: {score_2}')
```

PCA for Images I

Another way to show the inner workings of PCA is to use an image dataset. An image can be represented as a row in a data matrix, where each feature corresponds to the intensity of a pixel.

In this and the following exercise, we will be using the [Olivetti Faces](#) image dataset. We will begin by standardizing the images, and then observing the images of faces themselves.

In the next exercise, we will then transform the original data using PCA and re-plot the images using a subset of the principal components. This will allow us to visualize the mechanism by which PCA retains information in the data while reducing the dimensionality.

Instructions

Checkpoint 1 Failed, try again

1.

Let's start by downloading the Olivetti Faces dataset from scikit-learn. Standardize the image dataset by subtracting the mean and dividing by the standard deviation.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Each row from the faces dataset is a flattened image. Each feature is a pixel in the image. Find the side length of each square image by taking the square root of the number of features (pixels). Uncomment the code to print the number of pixels per image and per side-length.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Copy and paste the following plotting code to the bottom of **script.py** and press run to see the first 15 faces in the dataset.

```
import numpy as np
from sklearn import datasets
import codecademylib3
import matplotlib.pyplot as plt

# Download the data from sklearn's datasets
faces = datasets.fetch_olivetti_faces(['data'])

# 1. Standardize the images using the mean and standard deviation
faces_mean = faces.mean(axis=0)
faces_std = faces.std(axis=0)
faces_standardized = (faces - faces_mean) / faces_std

# 2. Find the number of features per image
n_images, n_features = faces_standardized.shape
side_length = int(np.sqrt(n_features))
print(f'Number of features(pixels) per image: {n_features}')
print(f'Square image side length: {side_length}')

# 3. Plot the images
# Create an empty 10x8 plot
fig = plt.figure(figsize=(10, 8))

# Observe the first 15 images.
for i in range(15):

    # Create subplot, remove x and y ticks, and add title
    ax = fig.add_subplot(3, 5, i + 1, xticks=[], yticks=[])
    ax.set_title(f'Image of Face: #{i}')

    # Get an image from a row based on the current value of i
    face_image = faces_standardized[i]

    # Reshape this image into side_length x side_length
    face_image_resized = face_image.reshape(side_length, side_length)

    # Show the image
    ax.imshow(face_image_resized, cmap=plt.cm.bone)
plt.show()
```

Image of Face: #0 Image of Face: #1 Image of Face: #2 Image of Face: #3 Image of Face: #4



Image of Face: #5 Image of Face: #6 Image of Face: #7 Image of Face: #8 Image of Face: #9



Image of Face: #10 Image of Face: #11 Image of Face: #12 Image of Face: #13 Image of Face: #14



PCA for Images II

Now that we have cleaned up the data, we can perform PCA to retrieve the eigenvalues and eigenvectors.

This can be useful in understanding how PCA works! We can visualize the eigenvectors by plotting them. They actually have a name: **eigenfaces**. The eigenfaces are the building blocks for all the other faces in the data.

We can also visualize the dimensionality reduction that occurs when we transform the original data using a smaller number of principal components. In the

Preview: Docs A code editor is a program designed for writing software and utilizing human-readable text to make code easier to parse and understand.

[code editor](#)

, we've provided you with code to:

Plot the eigenfaces

Plot the reconstructed faces using a smaller number of transformed features. To start, we've used 400 principal components — only 0.9% of the original number of features (pixels)!

Instructions

Checkpoint 1 Enabled

1.

Press "Run". First, inspect the eigenfaces. Notice how they each depict some of the common distinctive features in the various faces.

Now, scroll down in the plotting output to view the reconstructed faces. These were constructed using only 400 transformed features. Notice that they look almost identical to the original faces, despite requiring only .9% as much data to store them!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, change the number of components to 40 instead of 400 and press "Run". Scroll down to view the reconstructed faces. Notice how we get a decent reconstruction, but lose some detail and clarity by reducing the feature space this much (only 0.09% of the original).

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import codecademylib3
import matplotlib.pyplot as plt

faces_standardized = pd.read_csv('./faces_standardized.csv').values

# 1. Instantiate a PCA object and fit the standardized faces dataset
pca = PCA(n_components=40)
pca.fit(faces_standardized)

# 2. Retrieve and plot eigenvectors (eigenfaces)
eigenfaces = pca.components_

fig = plt.figure(figsize=(10, 8))
fig.suptitle('Eigenvectors of Images (Eigenfaces)')
for i in range(15):
    # Create subplot, remove x and y ticks, and add title
    ax = fig.add_subplot(3, 5, i + 1, xticks=[], yticks=[])
    ax.set_title(f'Eigenface: #{i}')

    # Get an eigenvector from the current value of i
    eigenface = eigenfaces[i]

    # Reshape this image into 64x64 since the flattened shape was 4096
    eigenface_resaped = eigenface.reshape(64, 64)

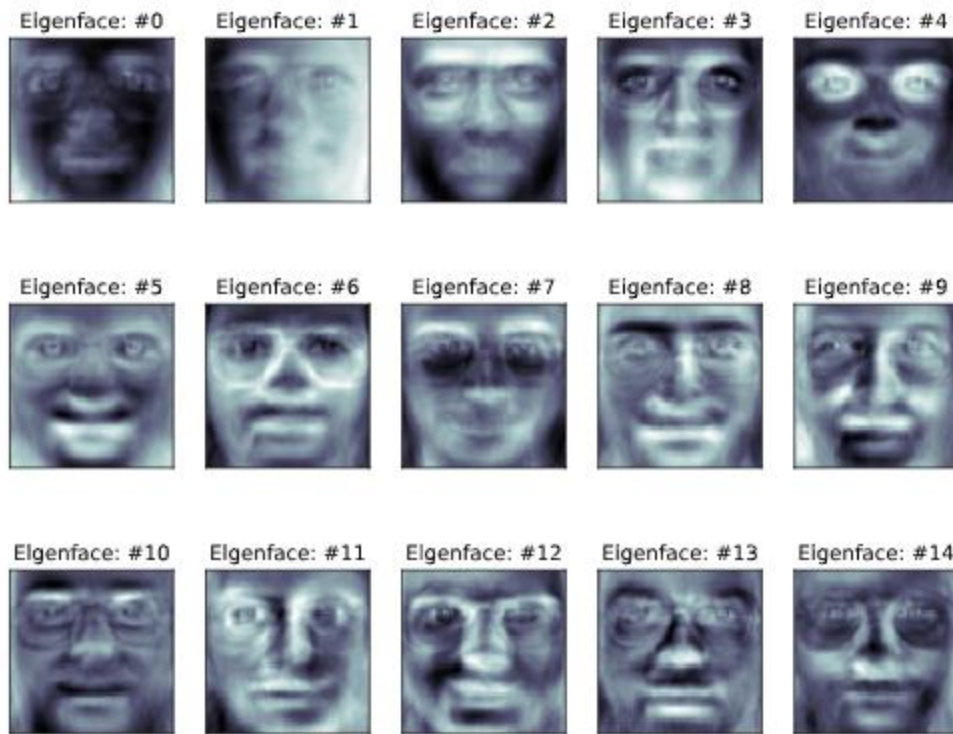
    # Show the image
    ax.imshow(eigenface_resaped, cmap=plt.cm.bone)
plt.show()

# 3. Reconstruct images from the compressed principal components
# The principal components are usually calculated using `faces_standardized @
principal_axes` or the `.transform` method
principal_components = pca.transform(faces_standardized)

# The `inverse_transform` method allows for reconstruction of images in the original size
faces_reconstructed = pca.inverse_transform(principal_components)

# Plot the reconstructed images
fig = plt.figure(figsize=(10, 8))
fig.suptitle('Reconstructed Images from Principal Components')
for i in range(15):
    ax = fig.add_subplot(3, 5, i + 1, xticks=[], yticks=[])
    ax.set_title(f'Reconstructed: {i}')

    reconstructed_face = faces_reconstructed[i]
    reconstructed_face_resaped = reconstructed_face.reshape(64, 64)
    ax.imshow(reconstructed_face_resaped, cmap=plt.cm.bone)
plt.show()
```

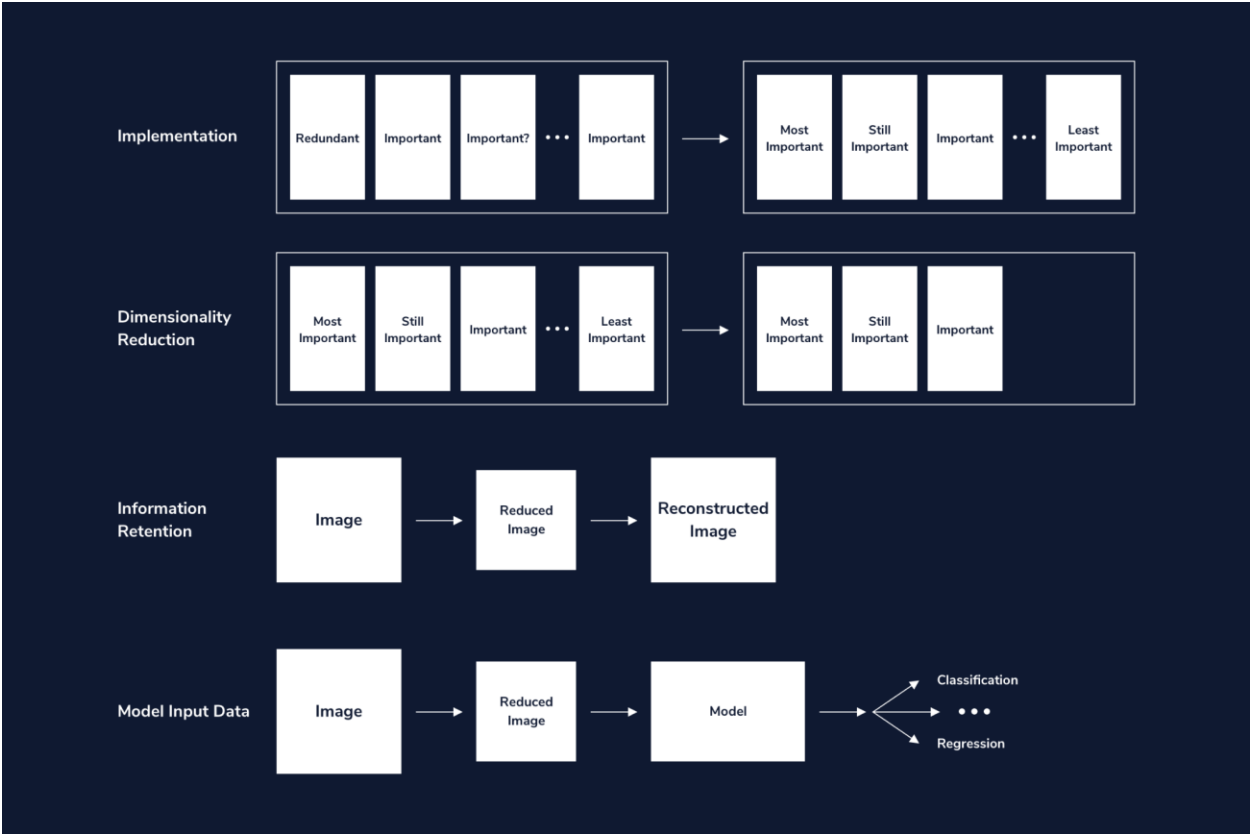
Exercise # 9: Review

In this lesson, we have seen how PCA can be implemented using NumPy and scikit-learn. In particular, we have seen how:

Implementation: scikit-learn provides a more in-depth set of methods and attributes that extend the number of ways to perform PCA or display the percentage of variance for each principal axis.
 Dimensionality reduction: We visualized the data projected onto the principal axes, known as principal components.

Image classification: We performed PCA on images of faces to visually understand how principal components still retain nearly all the information in the original dataset.

Improved algorithmic speed/accuracy: Using principal components as input to a classifier, we observed how we could achieve equal or better results with lower dimensional data. Having lower dimensionality also speeds the training.



Ensembling Methods

Introduction to Ensembling Methods

An introduction to Ensembling Methods in Machine Learning.

Introduction

Why use one machine learning model to solve a problem when you can use many at the same time? **Ensemble learning** is the process of building a complex machine learning model by combining multiple base estimators as building blocks. The main goals for employing ensemble methods are to end up with a resultant machine learning model that is more performant and more robust than its components.

Base estimators

In ensembling, often the base models that are chosen tend to underachieve on their own and are typically referred to as **weak learners**. The reason this is preferable is to have the model implementation be computationally efficient. Combining strong learners doesn't necessarily make the resultant ensemble model more performant so one might as well choose learners that cost less computationally.

What makes a model a "weak learner"? There are many ways in which a base model can be considered weak. For example, it can have high bias or high variance. The nature of the weakness of the base model is typically taken into consideration and is a design choice when determining the best ensembling method to use.

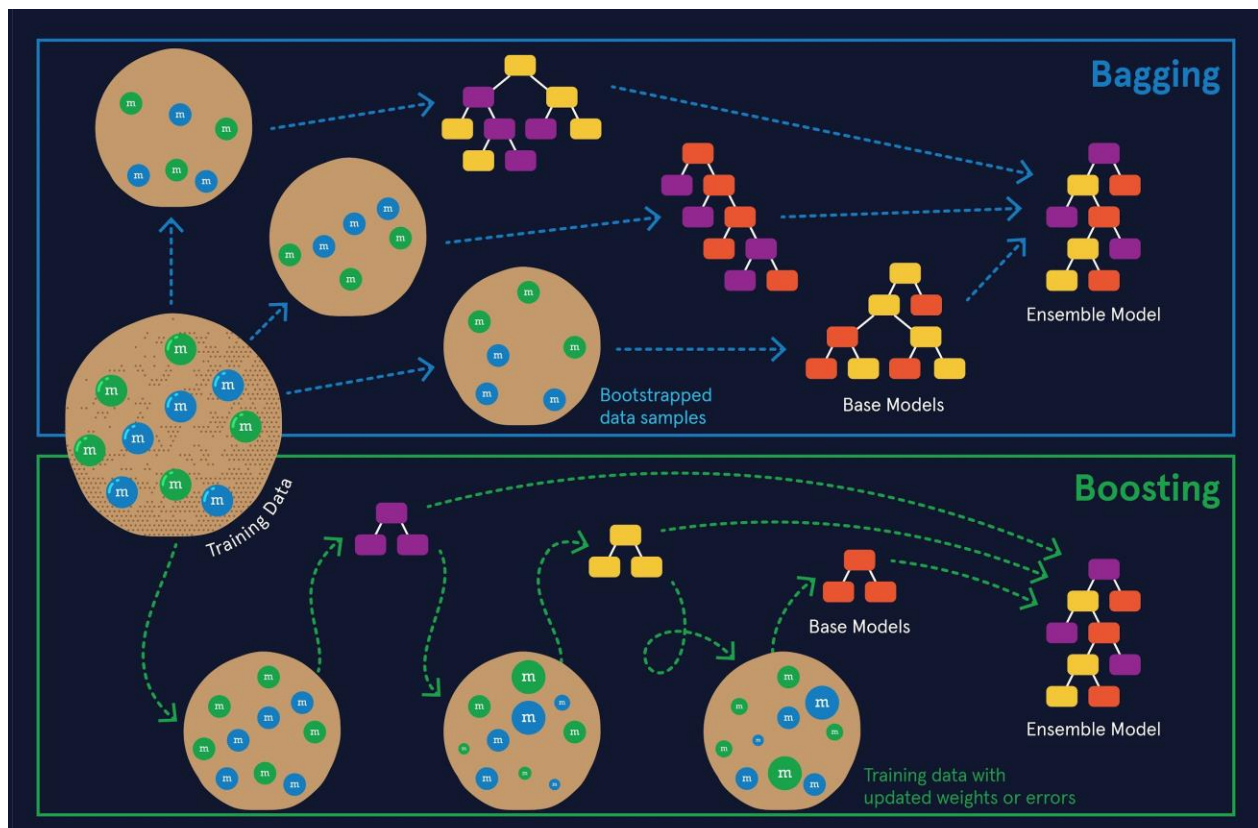
The basic components of ensemble learning are:

- Base models that are weak learners

- An ensembling method that combines the base models to improve performance and robustness

It is possible to have an ensemble model that performs worse than any one of its contributing base estimators. To circumvent this outcome it is important that the base estimators are uncorrelated and independent. Having a higher diversity among the trained base estimators leads to a stronger ensemble model.

Three common ensembling methods in machine learning that will be covered briefly in this article and later on in separate modules are Bagging, Boosting, and Stacking.



Bagging

As the name suggests, **Bootstrap AGGREGATING** (also known as **Bagging**) is an ensemble method that combines the concepts of bootstrapping and aggregation. Bagging can be used for both classification and regression problems. Bagging methods use weak learners as base models that are complex and tend to suffer from high variance. Their weakness as models is due to the fact that they are built with only a subset of the available features and on a subset of the training data due to bootstrapping.

If the full dataset is represented by the larger cookie in the figure above, each of the candies atop the cookie represents an individual training data instance. The smaller cookies in the Bagging panel represent a **bootstrapped** sample of the full training dataset. Bootstrapping refers to the method of sampling data with replacement.

Bagging is a learning technique that is done in parallel. Each of the base models is trained independently of the others. Additionally, each base model is trained using only a subset of the original features. Even though the base models tend to be complex, they overfit to both a subset of the available training data and a subset of the available features. This allows them to be diverse from one another, often leading to a very strong ensemble model when aggregated. In the Bagging panel of the figure, the base models are decision trees that are relatively large and overfit to the bootstrapped subset of data provided to each of them.

Once each of the base models is trained, the method for ensembling tends to be a simple aggregation technique over each of the models; a majority vote for classification problems and averaging for regression problems.

A common implementation of a bagging algorithm that uses decision trees as their base model is the Random Forest. You can learn more about Bagging and Random Forests in the [Random Forests module](#).

Boosting

Boosting is an ensemble learning technique where the weak learners are too simple and tend to suffer from high bias. In the Boosting panel of the figure above, the base models are decision trees with only one level, a decision

stump. Decision stumps can only make a decision based off of one feature at a time, causing them to underfit the data substantially.

Boosting is a sequential learning technique where each of the base models builds off the previous model. Each subsequent model aims to improve the performance of the final ensemble model by attempting to fix the errors in the previous stage.

In the Boosting panel of the figure, you may notice that some of the candies atop the cookies are larger than the others. These particular training instances were misclassified by the previous decision stump and are therefore given more weight by the next decision stump. This is one method in which boosting methods may learn from their mistakes.

Two common implementations of the boosting algorithm are Adaptive Boosting and Gradient Boosting. You can learn more about Boosting, including these two implementations, in the [Boosting module](#).

Stacking

Stacking is an extremely flexible ensemble technique where a final model is trained to learn how to best combine a set of base models to make strong predictions. In contrast to bagging and boosting, the base models in stacking do not need to be the same type of learning algorithm.

While bagging and boosting are built with base models that are weak learners, that does not necessarily have to be the case for stacking. A stacking algorithm can be used to combine decently performing learners as well.

You can learn more about Stacking in the [Stacking article](#).

Basics of a Random Forest

In this lesson, you'll learn what random forests are, how the random forest algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

works and how to implement it in `scikit-learn`.

We've seen that [decision trees](#)** can be powerful supervised

Preview: Docs Loading link description

[machine learning](#)

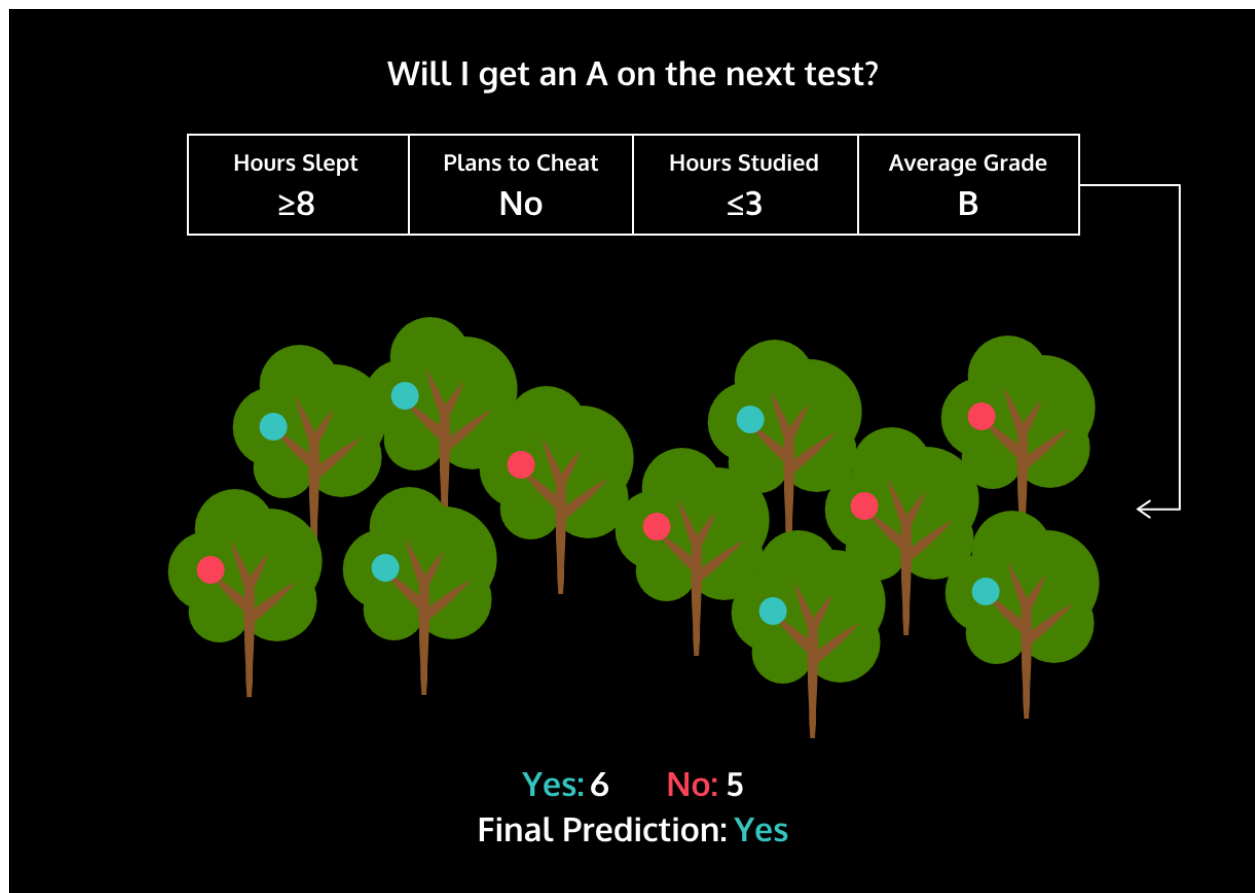
models. However, they're not without their weaknesses — decision trees are often prone to overfitting. We've discussed some strategies to minimize this problem, like pruning, but sometimes that isn't enough. We need to find another way to generalize our trees. This is where the concept of a random forest comes in handy.

A random forest is an *ensemble machine learning technique*. A random forest contains many decision trees that all work together to classify new points. When a random forest is asked to classify a new point, the random forest gives that point to each of the decision trees. Each of those trees reports their classification and the random forest returns the most popular classification. It's like every tree gets a vote, and the most popular classification wins. Some of the trees in the random forest may be overfit, but by making the prediction based on a large number of trees, overfitting will have less of an impact.

**** A prerequisite for this lesson is the understanding of decision trees. Be sure to revisit the decision trees module if needed!**

Instructions

The illustration to the right depicts a random forest used to predict if a student will get an A in a test or not based on an anonymous survey. The survey fields (the features here!) include hours the student slept, whether they expressed an intention to cheat or not, hours studied and their average grade so far. Here, the random forest is made up of eleven decision trees — the prediction outcome of A is outvoted 6:5!



Bootstrapping

You might be wondering how the trees in the random forest get created. After all, right now, our

Preview: Docs Loading link description
[algorithm](#)

for creating a decision tree is deterministic — given a training set, the same tree will be made every time. To make a random forest, we use a technique called *bagging*, which is short for *bootstrap aggregating*. This exercise will explain bootstrapping, which is a type of sampling

Preview: Docs Loading link description

[method](#)

done with replacement.

How it works is as follows: every time a decision tree is made, it is created using a different subset of the points in the training set. For example, if our training set had 1000 rows in it, we could make a decision tree by picking 100 of those rows at random to build the tree. This way, every tree is different, but all trees will still be created from a portion of the training data.

In bootstrapping, we're doing this process *with replacement*. Picture putting all 100 rows in a bag and reaching in and grabbing one row at random. After writing down what row we picked, we put that row back in our bag. This means that when we're picking our 100 random rows, we could pick the same row more than once. In fact, it's very unlikely, but all 100 randomly picked rows could all be the same row! Because we're picking these rows with replacement, there's no need to shrink our bagged training set from 1000 rows to 100. We can pick 1000 rows at random, and because we can get the same row more than once, we'll still end up with a unique data set.

We've loaded a dataset about cars here. An important field within the dataset is the safety rating, which tells us how crash/rollover resistant a car is, in other words, how safe the car is. The `safety` variable can be either "low", "med", or "high." We're going to implement bootstrapping and estimate the average safety rating across the different bootstrapped samples.

Here is a list of the

Preview: Docs Loading link description
[variables](#)
in the car evaluation dataset.

Variable	Description
safety	estimated safety of the car (low, med, or high)
buying	buying price
maint	price of the maintenance
doors	number of doors
persons	capacity in terms of persons to carry
lug_boot	the size of luggage boot
accep	evaluation level (unacceptable, acceptable, good, very good)

Instructions

Checkpoint 1 Failed, try again

1.

We've written some code to print the number of rows in the data set and the distribution of safety ratings of the entire dataset. Uncomment the relevant lines (under `## 1.`) and press Run to view the results of this.

Are the safety classes distributed equally in the dataset?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now that we know the `safety` variable classes are equally distributed, we're going to create a bootstrapped sample using `.sample()`.

`.sample()` takes two arguments:

`number of rows`: which is the same size as the dataset

`replace`: set to `True` because bootstrapping is sampling with replacement.

Uncomment the lines under `## 2.` and then print the distribution of safety ratings of the new sampled dataset.

What is the `safety` ratings distribution of the bootstrapped data?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Note that the distribution has now shifted! Using the same process, write a for loop to create 1000 bootstrapped samples of the same size as the original dataset. Save the percentage of "low" ratings into an array called `low_perc`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We've written some code to plot a histogram of the low percentage values. Uncomment the lines under `## 4.` and press Run.

We see that the average value of the low safety proportion of vehicles spans a range centered around the true mean.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Now uncomment the lines under `## 5.` to print the average low percentage and the 95% confidence range.

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

#Models from scikit learn module:
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)
y = df['accep']

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)
nrows = df.shape[0]

## 1. Print number of rows and distribution of safety ratings
print(nrows)
print(f'Distribution of safety ratings in {nrows} of data:')
print(df.safety.value_counts(normalize=True))

## 2. Create bootstrapped sample
boot_sample = df.sample(nrows, replace=True)
print(f'Distribution of safety ratings in bootstrapped sample data:')
print(boot_sample.safety.value_counts(normalize=True))

## 3. Create 1000 bootstrapped samples
low_perc = []
for i in range(1000):
```

```

boot_sample = df.sample(nrows, replace=True)
low_perc.append(boot_sample.safety.value_counts(normalize=True) ['low'])

## 4. Plot a histogram of the low percentage values
mean_lp = np.mean(low_perc)
print(mean_lp)
plt.hist(low_perc, bins=20);
plt.xlabel('Low Percentage')
plt.show()

## 5. What are the 2.5 and 97.5 percentiles?
print(f'Average low percentage: {np.mean(low_perc).round(4)}')

low_perc.sort()
print(f'95% Confidence Interval for low percentage:
({low_perc[25].round(4)}, {low_perc[975].round(4)})')

```

Bagging

Random forests create different trees using a process known as bagging, which is short for bootstrapped aggregating. As we already covered bootstrapping, the process starts with creating a single decision tree on a bootstrapped sample of data points in the training set. Then after many trees have been made, the results are “aggregated” together. In the case of a classification task, often the aggregation is taking the majority vote of the individual classifiers. For regression tasks, often the aggregation is the average of the individual regressors.

We will dive into this process for the cars dataset we used in the previous exercise. The dataset has six features:

```

buying: car price as a categorical variable: “vhigh”, “high”, “med”, or “low”
maint: cost of maintaining the car; can be “vhigh”, “high”, “med”, or “low”.
doors: number of doors; can be “2”, “3”, “4”, “5more”.
persons: number of people the car can hold; can be “2”, “4”, or “more”.
lugboot: size of the trunk; can be “small”, “med”, or “big”.
safety: safety rating of the car; can be “low”, “med”, or “high”

```

We’ve already loaded the dataset and done the train-test split. Our target variable for prediction is an acceptability rating, `accep`, that’s either `True` or `False`.

Instructions

Checkpoint 1 Enabled

1.

Train a decision tree with `max_depth` set to 5. Evaluate the `accuracy_score` on the `test` data.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We’ve written some code to get a new set of indices, `ids`, to generate a bootstrapped set of row indices.

We’ve set the `random_state` argument to 0 for reproducibility. Using these indices, fit another decision tree to training data pertaining to these rows. What is the accuracy score on the test set for the new classifier?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Repeat a decision tree build on 10 different bootstrapped samples using a for loop. Save the results, `y_pred` of all 10 predictions on the test set in an array, `preds`. Take the average of the 10 results and save it as `ba_pred`

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We have just performed bagging! Calculate the accuracy score on the bagged predictions and save it as `ba_accuracy`. (Note that the predictions are averaged and will no longer be binary as a bunch of zeroes and ones have been averaged.)


```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.metrics import accuracy_score

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-
databases/car/car.data', names=['buying', 'maint', 'doors', 'persons', 'lug boot',
'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)
y = df['accep']
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0,
test_size=0.25)

#original decision tree trained on full training set
dt = DecisionTreeClassifier(max_depth=5)
dt.fit(x_train, y_train)
print(f'Accuracy score of DT on test set (trained using full set):
{dt.score(x_test, y_test).round(4)}')

#2. New decision tree trained on bootstrapped sample
dt2 = DecisionTreeClassifier(max_depth=5)
#ids are the indices of the bootstrapped sample
ids = x_train.sample(x_train.shape[0], replace=True, random_state=0).index
dt2.fit(x_train.loc[ids], y_train[ids]) #max_depth=50,criterion='gini'
print(f'Accuracy score of DT on test set (trained using bootstrapped sample):
{dt2.score(x_test, y_test).round(4)}')

## 3. Bootstrapping ten samples and aggregating the results:
preds = []
random_state = 0
for i in range(10):
    ids = x_train.sample(x_train.shape[0], replace=True,
random_state=random_state+i).index
    dt2.fit(x_train.loc[ids], y_train[ids])
    preds.append(dt2.predict(x_test))
ba_pred = np.array(preds).mean(0)

# 4. Calculate accuracy of the bagged sample
ba_accuracy = accuracy_score(ba_pred>=0.5, y_test)
print(f'Accuracy score of aggregated 10 bootstrapped
samples:{ba_accuracy.round(4)}')

```

Random Feature Selection

In addition to using bootstrapped samples of our dataset, we can continue to add variety to the ways our trees are created by randomly selecting the features that are used.

Recall that for our car data set, the original features were the following:

The price of the car which can be "vhigh", "high", "med", or "low".
The cost of maintaining the car which can be "vhigh", "high", "med", or "low".
The number of doors which can be "2", "3", "4", "5more".
The number of people the car can hold which can be "2", "4", or "more".
The size of the trunk which can be "small", "med", or "big".
The safety rating of the car which can be "low", "med", or "high"

Our target variable for prediction is an acceptability rating, `accep`, that's either `True` or `False`. For our final features sets, `x_train` and `x_test`, the categorical features have been dummy encoded, giving us 15 features in total.

When we use a decision tree, all the features are used and the split is chosen as the one that increases the information gain the most. While it may seem counter-intuitive, selecting a random subset of features can help in the performance of an ensemble model. In the following example, we will use a random selection of features prior to model building to add additional variance to the individual trees. While an individual tree may perform worse, sometimes the increases in variance can help model performance of the ensemble model as a whole.

Instructions

Checkpoint 1 Enabled

1.

We've written some code that create a random sample of 10 features, `rand_features` from the 15 original features by using NumPy's `random.choice` method and defined a decision tree classifier `dt2`. Train the new decision tree model (without specifying any pre-selected parameters!), `dt2` on training data *that contains only these ten columns*. Calculate the new accuracy on the test set, store it as `accuracy_dt2` and print it to compare it to the tree built using the entire training set.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Every time we use `np.random.choice`, we can generate a new subset of random features. We're now going to obtain predictions from ten decision tree classifiers by doing the following. We've initialized an empty array called `predictions` and created a `for` loop with ten iterations with the following steps:

The first line selects ten random features using `rand_features = np.random.choice(x_train.columns,10)`

Fit the decision tree `dt2` to training data pertaining to these features.

Append the predictions on test data (with only `rand_features` columns!) to the array `predictions`

Uncomment the relevant lines and press Run.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We have ten decision trees' worth of predictions now! To meaningfully combine their predictions, let's use the following system. If more than 5 classifiers predict that a datapoint belong to a certain class, we assign an aggregate prediction to that class. To do this:

Create an array `prob_predictions` that is the mean value of the predictions stored in `predictions`. This is similar to a probability on each datapoint since this is a binary classification problem.

Create an array `agg_predictions` that assigns a value of False or True to each element in `prob_predictions` based on whether the value is ≤ 0.5 or > 0.5 . This gives us the *aggregate* prediction from the 10 decision tree classifiers.

Calculate the accuracy score on this aggregate prediction array. Store it as `agg_accuracy` and print it to compare it to the previous accuracy scores.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)
y = df['accep']
```

```

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)
dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
print("Accuracy score of DT on test set (trained using full feature set):")
accuracy_dt = dt.score(x_test, y_test)
print(accuracy_dt)

# 1. Create rand_features, random samples from the set of features
rand_features = np.random.choice(x_train.columns,10)

# Make new decision tree trained on random sample of 10 features and calculate the new
accuracy score
dt2 = DecisionTreeClassifier()

dt2.fit(x_train[rand_features], y_train)
print("Accuracy score of DT on test set (trained using random feature sample):")
accuracy_dt2 = dt2.score(x_test[rand_features], y_test)
print(accuracy_dt2)

# 2. Build decision trees on 10 different random samples
predictions = []
for i in range(10):
    rand_features = np.random.choice(x_train.columns,10)
    dt2.fit(x_train[rand_features], y_train)
    predictions.append(dt2.predict(x_test[rand_features]))

## 3. Get aggregate predictions and accuracy score
prob_predictions = np.array(predictions).mean(0)
agg_predictions = (prob_predictions>0.5)
agg_accuracy = accuracy_score(agg_predictions, y_test)
print('Accuracy score of aggregated 10 samples:')
print(agg_accuracy)

```

Bagging in `scikit-learn`

The two steps we walked through above created trees on bootstrapped samples and randomly selecting features. These can be combined together and implemented at the same time! Combining them adds an additional variation to the base learners for the ensemble model. This in turn increases the ability of the model to generalize to new and unseen data, i.e., it minimizes bias and increases variance. Rather than re-doing this process manually, we will use `scikit-learn`'s bagging implementation, `BaggingClassifier()`, to do so.

Much like other models we have used in `scikit-learn`, we instantiate a instance of `BaggingClassifier()` and specify the parameters. The first

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`:
 js function `tripleThis(x)`
 { return x * 3; } `tripleThis(6);`

[parameter](#)

,`base_estimator` refers to the

Preview: Docs Loading link description

[machine learning](#)

model *that is being bagged*. In the case of random forests, the *base estimator* would be a *decision tree*. We are going to use a decision tree classifier WITH a `max_depth` of 5, this will be instantiated with `BaggingClassifier(DecisionTreeClassifier(max_depth=5))`.

After the model has been defined, methods `.fit()`, `.predict()`, `.score()` can be used as expected. Additional hyperparameters specific to bagging include the number of estimators (`n_estimators`) we want to use and the maximum number of features we'd like to keep (`max_features`).

Note: While we have focused on decision tree classifiers (as this is the base learner for a random forest classifier), this procedure of bagging is not specific to decision trees, and in fact can be used for any base classifier or regression model. The `scikit-learn` implementation is generalizable and can be used for other base models!

Instructions

Checkpoint 1 Enabled

1.

Create an instance of `BaggingClassifier`, `bag_dt` with the following arguments:

```
DecisionTreeClassifier (with max_depth=5) base estimator
n_estimators=10.
```

Fit the model on the training set and evaluate the model on the test set by calculating the accuracy score. Save the score as `bag_accuracy` and print it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a different bagging classifier, `bag_dt_10` that includes the parameter `max_features=10` over and above the parameters in `bag_dt`. Fit the model on the training set and evaluate the model on the test set by calculating the accuracy score. Save the score as `bag_accuracy_10` and print it.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Change the base estimator of the bagged classifier to be logistic regression and call this instance `bag_lr` as follows:

Set `base_estimator` to be `LogisticRegression()`

Set `n_estimators` and `max_features` to be 10 like in the previous checkpoint

Refit on the training set and calculate the accuracy on the test set. Store this as `bag_accuracy_lr` and print it.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)
y = df['accep']
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)

# 1. Bagging classifier with 10 Decision Tree base estimators
bag_dt = BaggingClassifier(base_estimator=DecisionTreeClassifier(max_depth=5),
n_estimators=10)
bag_dt.fit(x_train, y_train)

print('Accuracy score of Bagged Classifier, 10 estimators:')
bag_accuracy = bag_dt.score(x_test, y_test)
print(bag_accuracy)

# 2. Set `max_features` to 10.
```

```

bag_dt_10 = BaggingClassifier(base_estimator=DecisionTreeClassifier(max_depth=5),
                             n_estimators=10, max_features=10)
bag_dt_10.fit(x_train, y_train)

print('Accuracy score of Bagged Classifier, 10 estimators, 10 max features:')
bag_accuracy_10 = bag_dt_10.score(x_test, y_test)
print(bag_accuracy_10)

# 3. Change base estimator to Logistic Regression
from sklearn.linear_model import LogisticRegression
bag_lr = BaggingClassifier(base_estimator=LogisticRegression(),
                           n_estimators=10, max_features=10)
bag_lr.fit(x_train, y_train)

print('Accuracy score of Logistic Regression, 10 estimators:')
bag_accuracy_lr = bag_lr.score(x_test, y_test)
print(bag_accuracy_lr)

```

Train and Predict using `scikit-learn`

Now that we have covered two major ways to combine trees, both in terms of samples and features, we are ready to get to the implementation of random forests! This will be similar to what we covered in the previous exercises, but the random forest

Preview: Docs Loading link description

[algorithm](#)

has a slightly different way of randomly choosing features. Rather than choosing a single random set at the onset, each split chooses a different random set.

For example, when finding which feature to split the data on the first time, we might randomly choose to only consider the price of the car, the number of doors, and the safety rating. After splitting the data on the best feature from that subset, we'll likely want to split again. For this next split, we'll randomly select three features again to consider. This time those features might be the cost of maintenance, the number of doors, and the size of the trunk. We'll continue this process until the tree is complete.

One question to consider is how to choose the number of features to randomly select. Why did we choose 3 in this example? A good rule of thumb is select as many features as *the square root of the total number of features*. Our car dataset doesn't have a lot of features, so in this example, it's difficult to follow this rule. But if we had a dataset with 25 features, we'd want to randomly select 5 features to consider at every split point.

You now have the ability to make a random forest using your own decision trees. However, `scikit-learn` has a `RandomForestClassifier()` class that will do all of this work for you! `RandomForestClassifier` is in the `sklearn.ensemble` module.

`RandomForestClassifier()` works almost identically to `DecisionTreeClassifier()` — the `.fit()`, `.predict()`, and `.score()` methods work in the exact same way.

Instructions

Checkpoint 1 Enabled

1.

Create a random forest classification model defined as `rf` with default parameters. Use the `.get_params()` method to get the parameters of the random forest. Store it as `rf_params` and print it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Fit `rf` using the training data set and labels. Predict the classes of the test data set (`x_test`) and save this as an array `y_pred`. Calculate the accuracy of model on the test set (either using `.score()` or `accuracy_score()`) and save it as `rf_accuracy`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Implement additional classification evaluation metrics — calculate and print the precision, recall, and confusion matrix on the test set. Store them as `rf_precision`, `rf_recall` and `rf_confusion_matrix` respectively.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)
y = df['accep']
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)

# 1. Create a Random Forest Classifier and print its parameters
rf = RandomForestClassifier()
print('Random Forest parameters:')
rf_params = rf.get_params()
print(rf_params)

# 2. Fit the Random Forest Classifier to training data and calculate accuracy score on the
test data
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)
print('Test set accuracy:')
rf_accuracy = rf.score(x_test, y_test)
print(rf_accuracy)

# 3. Calculate Precision and Recall scores and the Confusion Matrix
rf_precision = precision_score(y_test, y_pred)
print(f'Test set precision: {rf_precision}')
rf_recall = recall_score(y_test, y_pred)
print(f'Test set recall: {rf_recall}')
rf_confusion_matrix = confusion_matrix(y_test, y_pred)
print(f'Test set confusion matrix:\n{rf_confusion_matrix}')
```

Random Forest Regressor

Just like in decision trees, we can use random forests for regression as well! It is important to know when to use regression or classification — this usually comes down to what type of variable your target is. Previously, we were using a

Preview: Docs Loading link description

[binary](#)

categorical variable (acceptable versus not), so a classification model was used.

We will now consider a hypothetical new target variable, price, for this data set, which is a continuous variable. We've generated some fake prices in the dataset so that we have numerical values instead of the previous categorical

Preview: Docs Loading link description

[variables](#)

. (Please note that these are not reflective of the previous categories of high and low prices - we just wanted some numeric values so we can perform regression! :))

Now, instead of a classification task, we will use `scikit-learn`'s `RandomForestRegressor()` to carry out a regression task.

Note: Recall that the [default evaluation score](#) for regressors in `scikit-learn` is the [R-squared score](#).

Instructions

Checkpoint 1 Enabled

1.

Initialize and fit a `RandomForestRegressor()` model named `rfr` on the training data. Calculate the default scores (the R^2 values here) on the train and test sets, store them as `r_squared_train` and `r_squared_test` respectively, and print them.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Calculate the average price of a car, store it as `avg_price`. Calculate the MAE (Mean Absolute Error) for the train and test sets and store the values as `mae_train` and `mae_test`. Print all three values to see how the errors compare to the mean.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data',
names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep'])
df['accep'] = ~(df['accep']=='unacc') #1 is acceptable, 0 if not acceptable
X = pd.get_dummies(df.iloc[:,0:6], drop_first=True)

## Generating some fake prices for regression! :)
fake_prices = (15000 + 25*df.index.values)+np.random.normal(size=df.shape[0])*5000
df['price'] = fake_prices
print(df.price.describe())
y = df['price']

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)

# 1. Create a Random Regressor and print `R^2` scores on training and test data
rfr = RandomForestRegressor()
rfr.fit(x_train, y_train)

r_squared_train = rfr.score(x_train, y_train)
print(f'Train set R^2: {r_squared_train}')

r_squared_test = rfr.score(x_test, y_test)
print(f'Test set R^2: {r_squared_test}')

# 2. Print Mean Absolute Error on training and test data

avg_price = y.mean()
print(f'Avg Price Train/Test: {avg_price}')
```

```

y_pred_train = rfr.predict(x_train)
y_pred_test = rfr.predict(x_test)

mae_train = mean_absolute_error(y_train, y_pred_train)
print(f'Train set MAE: {mae_train}')

mae_test = mean_absolute_error(y_test, y_pred_test)
print(f'Test set MAE: {mae_test}')

```

Review

Nice work! Here are some of the major takeaways about random forests:

A random forest is an ensemble

Preview: Docs Loading link description

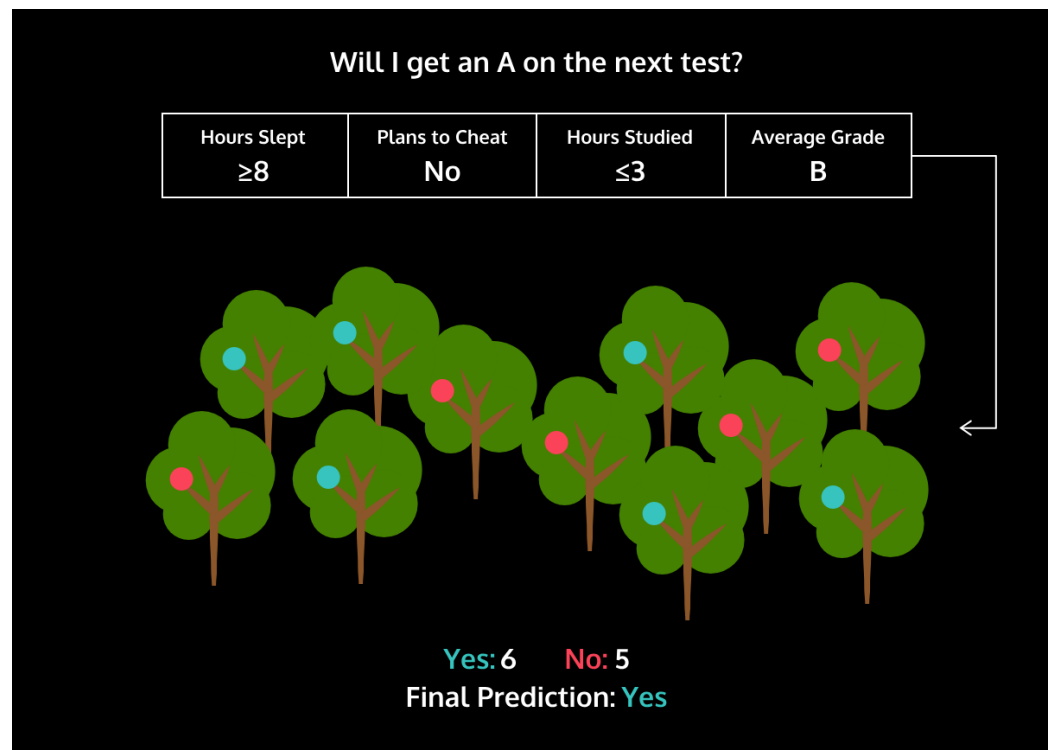
[machine learning](#)

model. It makes a classification by aggregating the classifications of many decision trees.

Random forests are used to avoid overfitting. By aggregating the classification of multiple trees, having overfitted trees in a random forest is less impactful.

Every decision tree in a random forest is created by using a different subset of data points from the training set. Those data points are chosen at random with replacement, which means a single data point can be chosen more than once. This process is known as bagging.

When creating a tree in a random forest, a randomly selected subset of features are considered as candidates for the best splitting feature. If your dataset has n features, it is common practice to randomly select the square root of n features.



Boosting

In this module we will cover a powerful [ensemble method](#) called Boosting. Boosted ensemble methods use weak learners as base models that are simple and tend to suffer from high bias. The weak learners underfit the data.

Boosting is a sequential learning technique where each of the base models builds off of the previous model. Each subsequent model aims to improve the performance of the final ensemble model by attempting to fix the errors in the previous stage.

There are two important decisions that need to be made to perform boosted ensembling:

Sequential Fitting Method
Aggregation Method

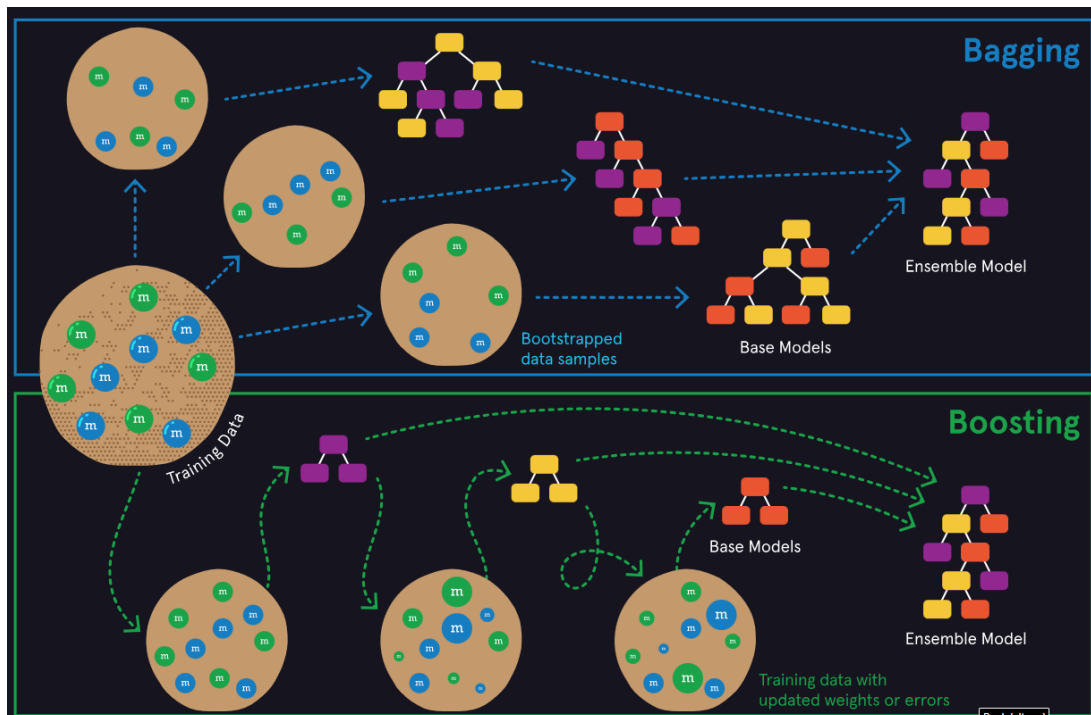
Two boosting algorithms that will be covered in detail in this module are **Adaptive Boosting** and **Gradient Boosting**.

While boosting can be applied to any base

Preview: Docs Loading link description
[machine learning](#)

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.
[algorithm](#)

, we will demonstrate with an extremely popular choice as a base estimator, the decision tree. Recall that [Decision Trees](#) are a commonly used and powerful machine learning algorithm because they are easy to interpret. Additionally, the training data requires very little manipulation (no need standardization, removal of collinearity, etc.). The major limitation to decision trees is that they tend to suffer from high variance and are therefore prone to overfitting. They are good at making a series of decisions which cause them to memorize the training data, so they do not generalize well to unseen data. In the following exercises we will explore how to work past these limitations while using decision trees for boosting.



Adaptive Boosting Overview

Adaptive Boosting (or AdaBoost) is a sequential ensembling

Preview: Docs Loading link description

[method](#)

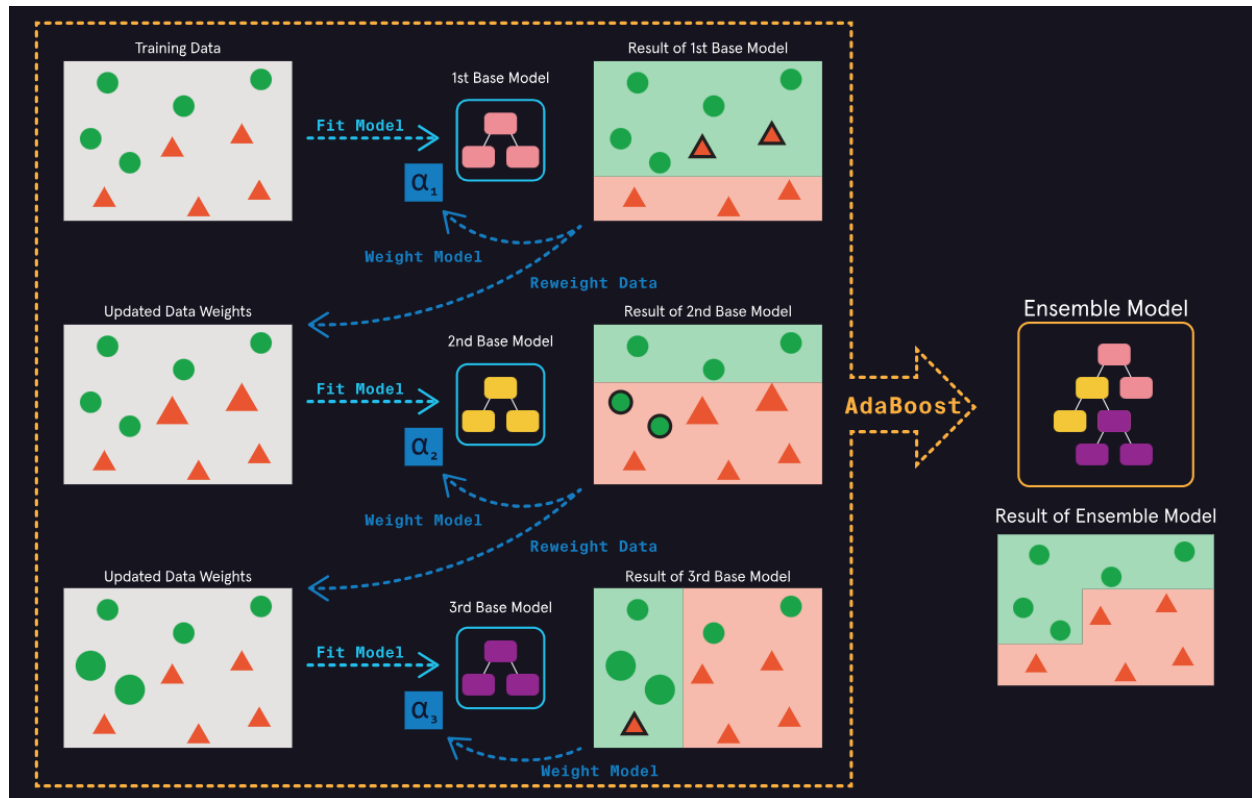
that can be used for both classification and regression. It can use any base link description [machine learning](#) model, though it is most commonly used with decision trees.

For AdaBoost, the **Sequential Fitting Method** is accomplished by updating the weight attached to each of the training dataset observations as we proceed from one base model to the next.

The **Aggregation Method** is a weighted sum of those base models where the model weight is dependent on the error of that particular estimator.

The training of an AdaBoost model is the process of determining the training dataset observation weights at each step as well as the final weight for each base model for aggregation.

In the next exercise we will dive into the details of AdaBoost!



Adaptive Boosting

Let's take a deeper look at how AdaBoost works! AdaBoost can be used for both regression and classification, but in this example we will be solving a classification problem. We begin with the full Training Dataset. You will see that it consists of green circles and red triangles. The goal of our AdaBoost classifier will be to form a decision boundary that separates these two classes. Initially, the training data instances are all given the same weight. This is indicated by the size of shapes all being the same.

Our first step is to fit an estimator, the 1st Base Model. While boosting can be applied to any base

Preview: Docs Loading link description

[machine learning](#)

model, we will use decision trees. But aren't decision trees prone to overfitting? We already said that the base models for boosting are supposed to be very simple and tend to underfit. That is correct, and for this reason we use the simplest version of a decision tree, known as a decision stump. A decision stump only makes a single decision, so the resultant estimator only has two leaf nodes.

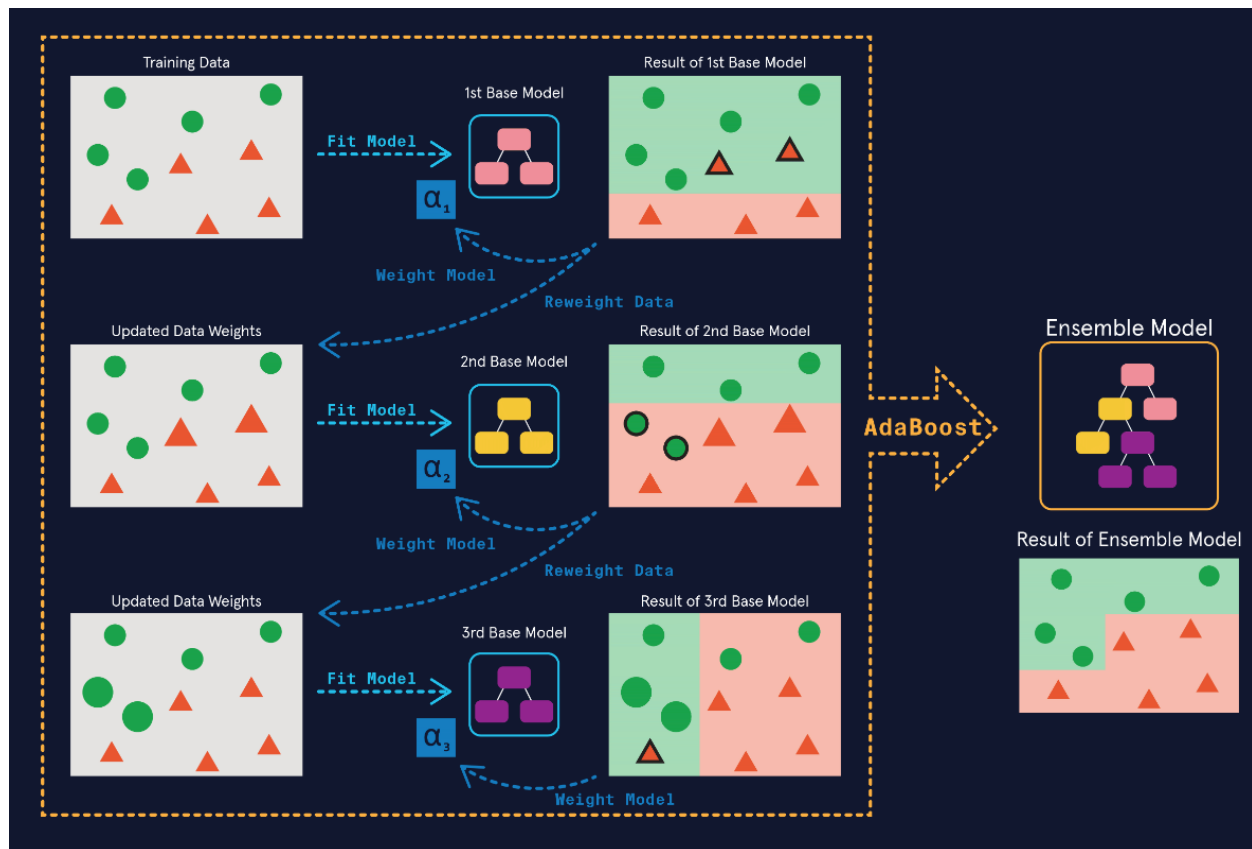
Taking a look at the Result of the 1st Base Model, we see that the decision boundary, that is the border between the lighter green and lighter red regions, does a decent job of separating the green circles from the red triangles. However we do notice that there are two red triangles in the light green region. This indicates that they have been classified incorrectly by the decision stump.

Each of the base models will contribute a different amount to the final ensemble model. The influence that a particular base model contributes is going to be dependent on the number of errors it makes, or for regression, the magnitude of the errors it makes. We do not want a decision stump that does a terrible job of classifying the data to have the same say as a decision stump that does a great job. Once we are able to evaluate the Result of the 1st Base Model, we can Weight the Model and assign it a value, here indicated by `alpha_1`.

To prepare for the next stage of the sequential learning process, we need to Reweight the Data. The instances of the training data that were classified incorrectly by the 1st Base Model, the two red triangles in the middle right, are given a larger weight than the other data instances indicated by their larger size. By assigning those misclassified points a larger weight, we are asking the the 2nd Base Model to give them preferential treatment during the Model Fitting.

Taking a look at the Result of the 2nd Base Model, we see that is exactly what happens. The two larger red triangles are classified correctly by the 2nd Base Model. Once again we assign the base model a weight, `alpha_2` proportional to the errors it makes and prepare for the next stage of the sequential learning by reweighting the training data. The instances that were incorrectly classified by the 2nd Base Model, the two green circles in on the center right, are given a larger weight.

Once we have reached the predefined number of estimators for our AdaBoost model, the base models are ready to aggregate. In this example we have chosen `n_estimators = 3`. The influence of each base model in the final ensemble model will be proportional to the `alpha` it was assigned during the training process.



Adaptive Boosting Implementation

Wow, that was a lot to take in! Let's take this opportunity to implement AdaBoost on a real dataset and solve a classification problem.

We will be using a dataset from [UCI's Machine Learning Repository](https://ml.cs.ucb.edu/mlcourse/UCI's%20Machine%20Learning%20Repository) to evaluate the acceptability of a car based on a set of features that encompasses their price and technical characteristics.

Instructions

Checkpoint 1 Enabled

1.

Create the base estimator for the AdaBoost classifier in the form a decision stump using `DecisionTreeClassifier()` and store it in a variable named `decision_stump`. Recall, that a decision stump is a decision tree with only two leaf nodes.

Print the parameters of the decision stump using the `.get_params()` method.

A decision stump is a decision tree with `max_depth=1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using `AdaBoostClassifier()`, create an AdaBoost classification model with the `base_estimator` parameter set to `decision_stump` and `n_estimators` set to 5. Store the model in a variable named `ada_classifier`.

Print the parameters of the AdaBoost model using the `.get_params()` method.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Fit `ada_classifier` using the training features (`x_train`) and corresponding labels (`y_train`).
Predict the classes of the testing dataset (`x_test`) and store them as an array in a variable named `y_pred`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Now we will explore some of the most common evaluation metrics for classification on our trained AdaBoost model.

Calculate the accuracy and store it in a variable named `accuracy`.

Calculate the precision and store it in a variable named `precision`.

Calculate the recall and store it in a variable named `recall`.

Calculate the f1-score and store it in a variable named `f1`.

Remove the comments from the code block to print the evaluation metrics you just stored.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Take a look at the confusion matrix by uncommenting the relevant lines.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Load dataset to a pandas DataFrame
path_to_data = 'https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data'
column_names = ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep']
df = pd.read_csv(path_to_data, names=column_names)

target_column = 'accep'
raw_feature_columns = [col for col in column_names if col != target_column]

# Create dummy variables from the feature columns
X = pd.get_dummies(df[raw_feature_columns], drop_first=True)

# Convert target column to binary variable; 0 if 'unacc', 1 otherwise
df[target_column] = np.where(df[target_column] == 'unacc', 0, 1)
y = df[target_column]

# Split the full dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=123, test_size=0.3)

# 1. Create a decision stump base model using the Decision Tree Classifier and print its
parameters
decision_stump = DecisionTreeClassifier(max_depth=1)
print(decision_stump.get_params())

# 2. Create an Adaptive Boost Classifier and print its parameters
ada_classifier = AdaBoostClassifier(base_estimator=decision_stump, n_estimators=5)
print(ada_classifier.get_params())

# 3. Fit the Adaptive Boost Classifier to the training data and get the list of predictions
ada_classifier.fit(X_train, y_train)
y_pred = ada_classifier.predict(X_test)

# 4. Calculate the accuracy, precision, recall, and f1-score on the testing data
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)

print(f'Test set accuracy:\t{accuracy}')
print(f'Test set precision:\t{precision}')
print(f'Test set recall:\t{recall}')
print(f'Test set f1-score:\t{f1}')

# 5. Remove the comments from the following code block to print the confusion matrix
test_conf_matrix = pd.DataFrame(
    confusion_matrix(y_test, y_pred, labels=[1, 0]),
    index=['actual yes', 'actual no'],
    columns=['predicted yes', 'predicted no']
)

print(f'Confusion Matrix:\n{test_conf_matrix.to_string()}')
```

Gradient Boosting Overview

Gradient Boosting is a sequential ensembling

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

that can be used for both classification and regression. It can use any base

Preview: Docs Loading link description

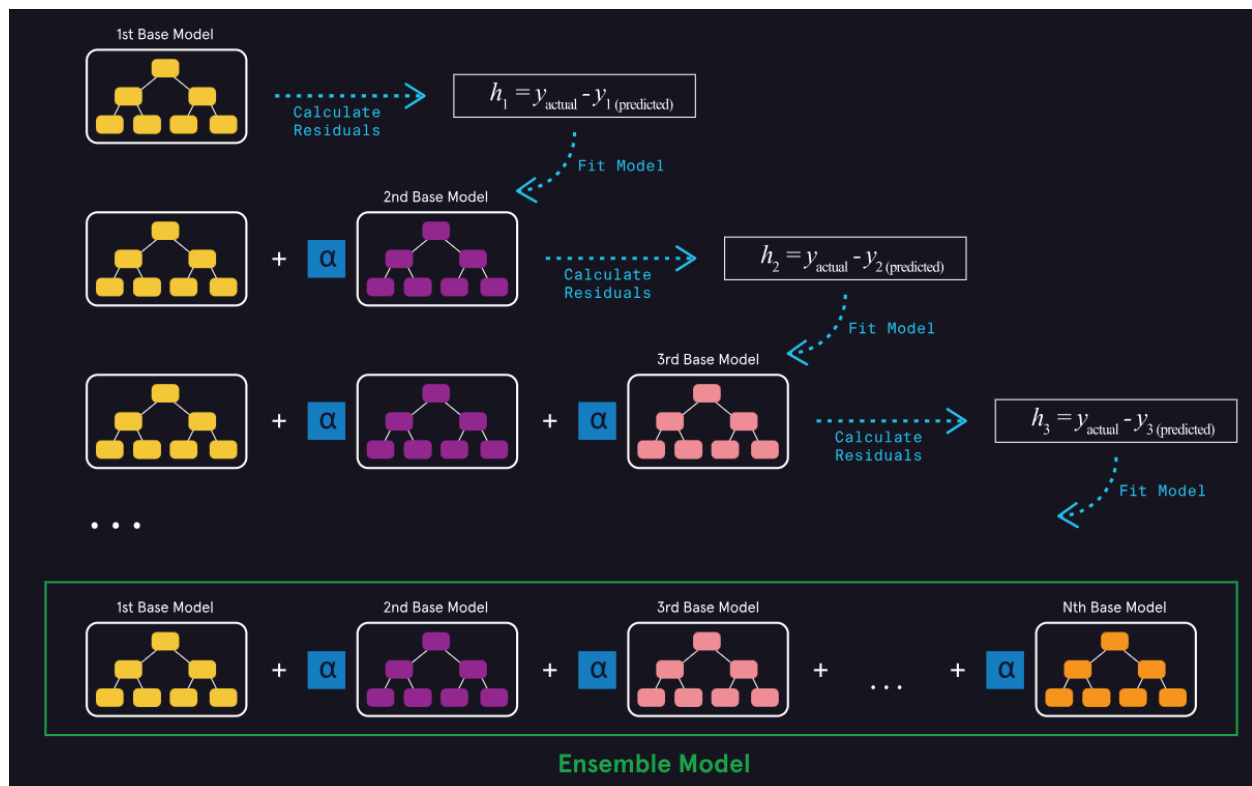
[machine learning](#)

model, though it is most commonly used with decision trees, known as Gradient Boosted Trees.

For Gradient Boost, the **Sequential Fitting Method** is accomplished by fitting a base model to the negative gradient of the error in the previous stage. The **Aggregation Method** is a weighted sum of those base models where the model weight is constant.

The training of a Gradient Boosted model is the process of determining the base model error at each step and using those to determine how to best formulate the subsequent base model.

In the next exercise we will dive into the details of Gradient Boosting!



Gradient Boosting

Now we will take a deeper look at how Gradient Boosting works. While Gradient Boosting can be applied to any base

Preview: Docs Loading link description

[machine learning](#)

model, decision trees are commonly used in practice. In this example we will be focusing on a Gradient Boosted Trees model.

Our first step is to fit an estimator, the 1st Base Model. Recall that the base estimators for boosting algorithms tend to be simple and high bias. In contrast to AdaBoost which leveraged the simplest form of decision trees, the decision stump with only 1 level, gradient boosted trees can and actually do tend to include a few more decision branches. Often gradient boosted trees will have up to 32 leaf nodes, which corresponds to a tree depth of 5 levels. In this example, we are limiting the depth of the base estimators to 2, corresponding to 4 leaf nodes.

Once the 1st Base Model is trained, the residual errors (h_1), of the model given the training data are determined. The residual error is the difference between the actual and predicted values for each of the training data instances.

$$h_1 = y_{\text{actual}} - y_1(\text{predicted})$$

The errors will be greater for the training data instances where the model did not do as good of a job with its prediction and will be lower on training data instances where the model fit the data well.

In the next stage of the sequential learning process, we fit the 2nd Base Model. Here is where the interesting part comes in. Instead of fitting the model to the target values y_{actual} as we are typically used to doing in machine learning, we actually fit the model on the errors of the previous stage, in this case h_1 . The 2nd Base Model is literally learning from the mistakes of the 1st Base Model through those residuals that were calculated.

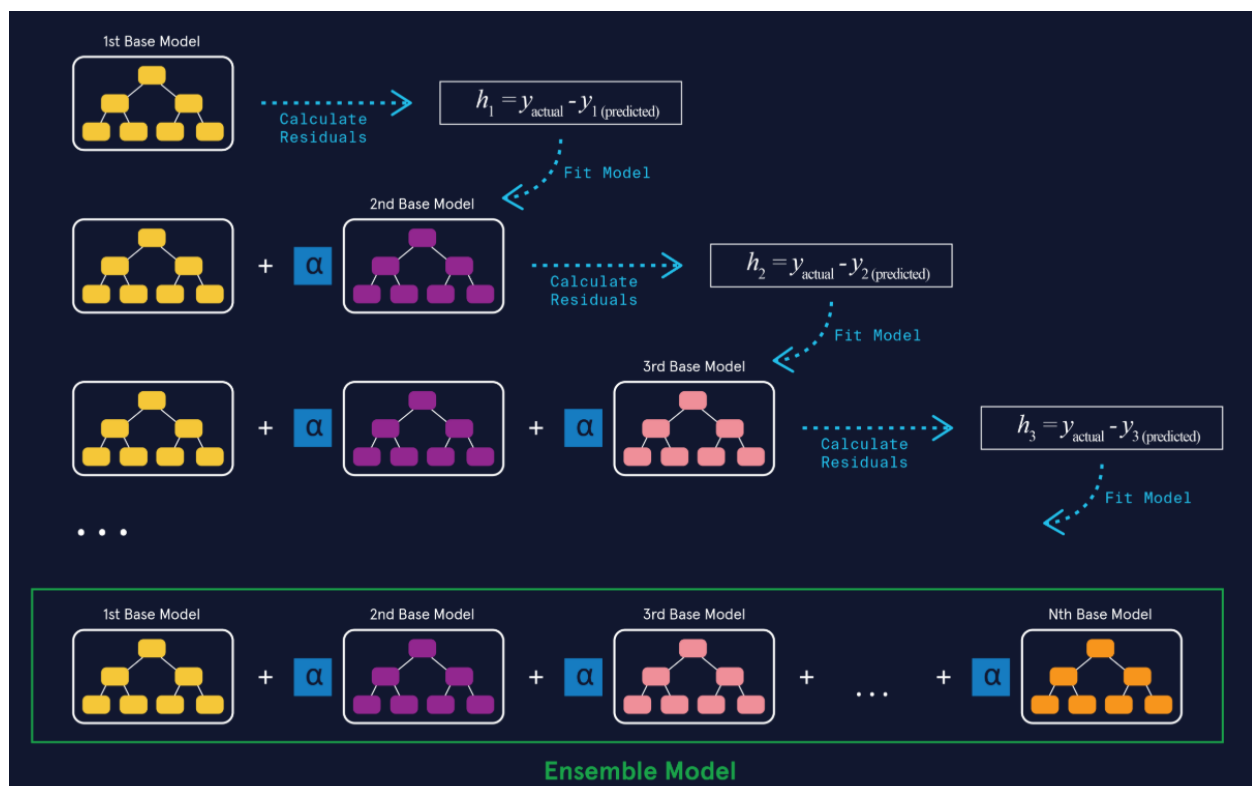
The results of the 2nd Base Model are multiplied by a constant learning rate, α , and added to the results of the 1st Base Model to give the set of updated predictions. The results of the second base model, which was tasked with fitting the errors of the first base model are multiplied by a constant learning rate, α and added to the results of the first base model to give us a set of updated predictions, $y_2(\text{predicted})$.

The residual errors of the 2nd stage are calculated using the updated predictions to get,

$$h_2 = y_{\text{actual}} - y_2(\text{predicted})$$

The subsequent stages repeat the same steps. At stage N , the base model is fit on the errors calculated at the previous stage $h_{(N-1)}$. The new model that is fit is multiplied by the constant learning rate α and added to the predictions of the previous stage.

Once we have reached the predefined number of estimators for our Gradient Boosting model or the residual errors are not changing between iterations, the model will stop training and we end up with the resultant ensemble model.



Gradient Boosting Implementation

Now that we have taken a look at what is going on under the hood, we are ready to implement Gradient Boosting on a real dataset and solve a classification problem.

We will be using a dataset from [UCI's Machine Learning Repository](#) to evaluate the acceptability of a car based on a set of features that encompasses their price and technical characteristics.

Instructions

Checkpoint 1 Enabled

1.

Create a Gradient Boosted Trees classification model using

`GradientBoostingClassifier()` with the `n_estimators` set to 15. Leave all other parameters to their default values. Store the model in a variable named `grad_classifier`.

Print the parameters of the GradientBoostedTrees model using the `.get_params()` method.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Fit `grad_classifier` using the training features (`x_train`) and corresponding labels (`y_train`).

Predict the classes of the testing dataset (`x_test`) and store them as an array in a variable named `y_pred`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now we will explore some of the most common evaluation metrics for classification on our trained Gradient Boosted Trees model.

Calculate the accuracy and store it in a variable named `accuracy`.

Calculate the precision and store it in a variable named `precision`.

Calculate the recall and store it in a variable named `recall`.

Calculate the f1-score and store it in a variable named `f1`.

Remove the comments from the code block to print the evaluation metrics you just stored.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Take a look at the confusion matrix by removing the comments in the relevant code block and pressing Run.

```
import pandas as pd
```

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Load dataset to a pandas DataFrame
path_to_data = 'https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data'
column_names = ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'accep']

df = pd.read_csv(path_to_data, names=column_names)
target_column = 'accep'
raw_feature_columns = [col for col in column_names if col != target_column]

# Create dummy variables from the feature columns
X = pd.get_dummies(df[raw_feature_columns], drop_first=True)

# Convert target column to binary variable; 0 if 'unacc', 1 otherwise
df[target_column] = np.where(df[target_column] == 'unacc', 0, 1)
y = df[target_column]

# Split the full dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=123, test_size=0.3)

# 1. Create a Gradient Boosting Classifier and print its parameters
grad_classifier = GradientBoostingClassifier(n_estimators=15)

print(grad_classifier.get_params())

# 2. Fit the Gradient Boosted Trees Classifier to the training data and get the list of
predictions
grad_classifier.fit(X_train, y_train)
y_pred = grad_classifier.predict(X_test)

# 3. Calculate the accuracy, precision, recall, and f1-score on the testing data
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f'Test set accuracy:\t{accuracy}')
print(f'Test set precision:\t{precision}')
print(f'Test set recall:\t{recall}')
print(f'Test set f1-score:\t{f1}')

# 4. Remove the comments from the following code block to print the confusion matrix
test_conf_matrix = pd.DataFrame(
    confusion_matrix(y_test, y_pred, labels=[1, 0]),
    index=['actual yes', 'actual no'],
    columns=['predicted yes', 'predicted no']
)

print(f'Confusion Matrix:\n{test_conf_matrix.to_string()}')

```

Output-only Terminal

Output:

```

{'ccp_alpha': 0.0, 'criterion': 'friedman_mse', 'init': None,
'learning_rate': 0.1, 'loss': 'log_loss', 'max_depth': 3, 'max_features':
None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0,
'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf':
0.0, 'n_estimators': 15, 'n_iter_no_change': None, 'random_state': None,

```

```
'subsample': 1.0, 'tol': 0.0001, 'validation_fraction': 0.1, 'verbose': 0,
'warm_start': False}
```

```
Test set accuracy:      0.8978805394990366
Test set precision:     0.7885714285714286
Test set recall:       0.8961038961038961
Test set f1-score:     0.8389057750759877
Confusion Matrix:
```

	predicted yes	predicted no
actual yes	138	16
actual no	37	328

Stacking

[Learn about the stacking ensemble learning method.](#)

Introduction

We've seen how ensembling methods such as bagging or boosting can be used to manage bias and variance of a machine learning model and improve performance. In this article, we're going to learn about another ensemble method: Stacking! Stacking fundamentally operates differently from bagging and boosting in two ways:

Stacking is a proven technique for improving prediction accuracy of standalone models. How does this work? Let's take a look!

How does stacking work?

Consider the scenario in which we are handling a classification problem where we are exploring different models. When using a decision tree we find that our model has poor generalization performance as a result of overfitting. Additionally, when using a logistic regression classification model, we discover our model parameters are hard to tune due to highly-correlated features. We might discover that while each model has a unique advantage over the others, each may also have a distinct drawback such as poor generalization accuracy or the inability to predict a specific class. As it turns out, there's no rule stating we can't take the best of both worlds!

Stacking can be thought of as a democracy of machine learning models, where different models are trained and subsequently cast their vote through their predictions. A majority-rules approach can be used for determining the final model prediction if we weighted each estimator equally. In practice, each base estimator may need to be

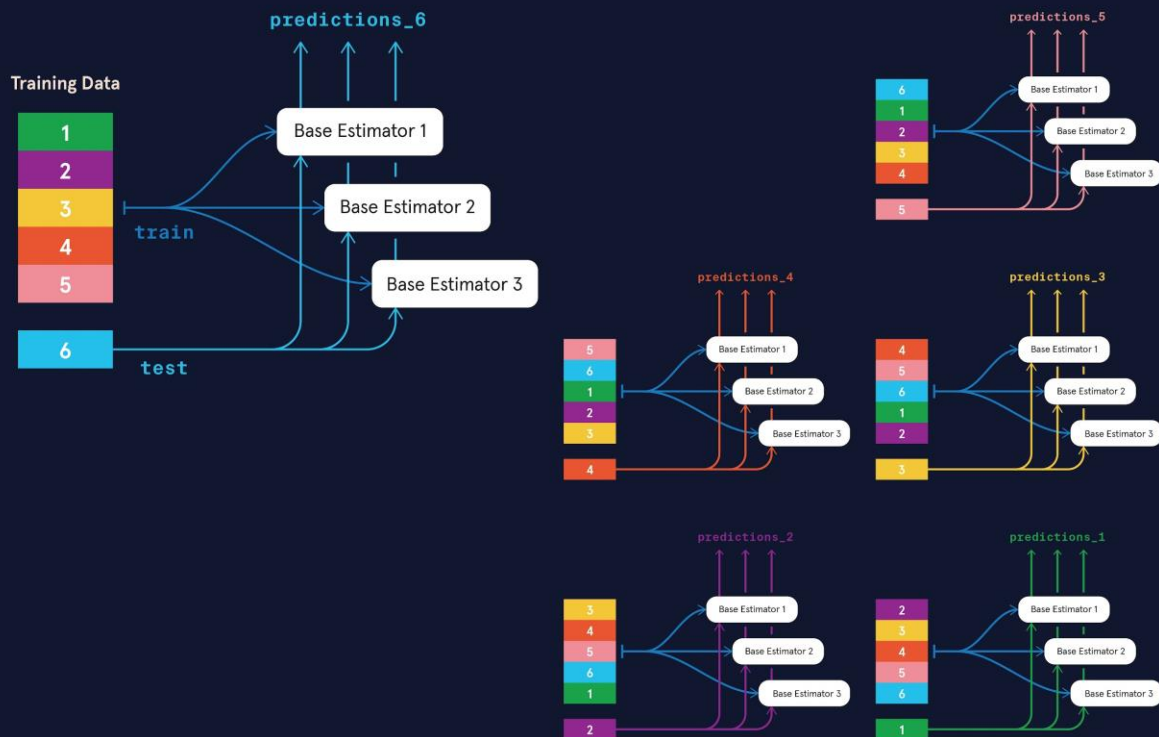
weighted differently, so we have a later-stage model to learn how to appropriately weigh the predictions of all the prior base estimators.

Let's talk about creating a stacking model and how to set up our training data for it.

Training Base Estimators

We can select from combinations of different base estimators, such as a logistic regression model in combination with a decision tree. We could additionally select models of the same learning algorithms, but with different parameters, such as multiple decision trees with varying depths. The number of estimators is arbitrary, so it's good practice to explore how different combinations behave.

This introduces a problem, however. The estimators would be making predictions on data used in training. This puts our model at risk of overfitting. To avoid this, we use K-Fold Cross Validation as described next.



K-Fold Cross Validation

Consider 10 segments (or folds) and a stacking model that uses a logistic regression model and a decision tree model. Each estimator can be trained using data from 9 of the segments, and make predictions on the excluded 10th segment. We then append the predictions as new features to that 10th segment. Now 1/10th of the training data has two new features: one is the prediction made by the logistic regression model and the other is the prediction made by the decision tree model.

We want to do the same with the other 9 segments, so we rotate the excluded segment and repeat this process until all training data points are augmented with new features. The end result is a prediction made on each training sample, without having seen the sample during the training process.

Feature Augmentation

In our stacking setup, the base estimators need to be trained to make predictions on our training data. The prediction of each estimator will be appended to the corresponding data sample as a new feature. We thus augment the training

data set with this additional information. The augmented training set is used by our later-stage stacking model to make the final prediction.



```
graph LR; A[Training Data] --> B[Base Estimators]; B --> C[Stacking Model];
```

Training Data

So, in summary, say our training dataset has 10,000 samples, 10 features, and we select 3 base estimators. We would train each base estimator on the training set and make predictions on the training set. Each estimator would make a prediction on each sample, therefore each sample will have 3 predictions. These 3 predictions are appended to the pre-existing 10 features. This leaves us with 10,000 training samples with 13 features each.

Training the stacking model

With our augmented training set, we're ready to prepare the final model that will make the official prediction from our stacking setup.

This later-stage model is a learning algorithm that we select just like the base estimators. We may even reuse an algorithm from an earlier stage here. The purpose of this model is to learn the proper weighting of the earlier estimator given our training samples now include a data point from each estimator. Some estimators may perform better than others, so our overall model should account for this.

The only difference in the training process is that the model will be designed to accept samples of the augmented size rather than the given size. This means if the given data set has n features and m base estimators, this model will require $n + m$ features on each sample.

And that's it! Once the later-stage model is trained, we feed testing data samples into the base estimators, which will append their predictions to the data sample. This sample is then given to the later-stage model for our final prediction.

Example

In the associated code for this article, we'll be referencing a water quality dataset. The features of this dataset are ones that can be used to determine how safe a supply of water is to drink or its "potability" represented by a 1 for drinkable and a 0 for not drinkable. For simplicity, let's take a look at a subset of the data with three of its features:

Sample	pH	Conductivity	Turbidity	Potability
1	8.316766	363.266516	4.628771	1
2	9.092223	398.410813	4.075075	0
3	5.584087	280.467916	2.559708	1

For this example, we'll select a Logistic Regression model and a Decision Tree model as our base estimators. The table below shows how the training dataset expands with new features provided by the predictions of the trained base estimators.

Sample	Logistic Regression prediction	Decision Tree prediction	pH	Conductivity	Turbidity	Potability
1	0	1	8.316766	363.266516	4.628771	1
2	0	0	9.092223	398.410813	4.075075	0
3	1	1	5.584087	280.467916	2.559708	1

Implementing stacking with `scikit-learn`

```
import pandas as pd
df = pd.read_csv('water_potability')
print(df.columns, df.shape)
```

Output:

```
Index(['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity',
      'Organic_carbon', 'Trihalomethanes', 'Turbidity', 'Potability'],
      dtype='object') (2011, 10)
```

We'll take a train-test split of our data to handle the training process.

```
X = water_potability.drop(['Potability'], axis=1)
y = water_potability['Potability']
```

[illegible]

To assemble our ensemble, we'll make a dictionary of base estimators. This will be contained within the `level_0_estimators` dict. Also, our final estimator will be a Random Forest as represented with `level_1_estimator`. Notice also how we prepare to add new features to our training dataset (as columns) in `level_0_columns`.

```
level_0_estimators = dict()
level_0_estimators["logreg"] = LogisticRegression(random_state=rand_state)
level_0_estimators["forest"] = RandomForestClassifier(random_state=rand_state)

level_0_columns = [f"{name}_prediction" for name in level_0_estimators.keys()]

level_1_estimator = RandomForestClassifier(random_state=rand_state)
```

Handling our k-fold cross-validation is fairly straightforward using `sklearn.model_selection.StratifiedKFold` from the `scikit-learn` library. The `kfold` is then given to the instantiated `StackingClassifier`.

```
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=rand_state)
stacking_clf = StackingClassifier(estimators=list(level_0_estimators.items()),
                                final_estimator=level_1_estimator,
                                passthrough=True, cv=kfold,
                                stack_method="predict_proba")
```

Calling `fit_transform` on our classifier manages a lot of the heavy lifting. It will handle the training of our `level_0` base estimators along with the `level_1_estimator`, make cross-validated predictions on the training set, and augment the training set with predictions from each estimator. Let's see how the resulting training dataset looks:

```
df = pd.DataFrame(stacking_clf.fit_transform(X_train, y_train),
                  columns=level_0_columns + list(X_train.columns))
```

	logreg_prediction	forest_prediction	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity
0	0.417408	0.17	9.927024	208.490738	19666.992792	8.008618	340.237824	482.842435	11.360427	85.829113	4.051733
1	0.379019	0.26	8.769676	215.368742	13969.438863	7.548543	322.799070	369.016667	18.919188	54.755214	3.776718
2	0.409582	0.87	8.077261	125.302719	23931.282833	8.773162	317.693331	398.328789	15.279583	62.668356	4.279871
3	0.407893	0.73	9.739562	166.948864	13623.160063	7.235922	385.059134	369.591289	12.322604	68.505852	2.568080
4	0.399452	0.04	5.343075	211.662091	45166.912141	6.651801	279.767500	485.959717	19.682337	70.546862	4.240032

Finally, with our full model trained, we can make predictions. Let's compare how our Stacking classifier performed with how a lone linear model and a lone decision tree model perform!

```
y_val_pred = stacking_clf.predict(X_test)
stacking_accuracy = accuracy_score(y_test, y_val_pred)

vanilla_logistic_regression = LogisticRegression(random_state=rand_state).fit(X_train,
y_train)
lr_accuracy = accuracy_score(y_test, vanilla_logistic_regression.predict(X_test))

vanilla_decision_tree = RandomForestClassifier(random_state=rand_state).fit(X_train,
y_train)
dt_accuracy = accuracy_score(y_test, vanilla_decision_tree.predict(X_test))

print(f'Stacking accuracy: {stacking_accuracy:.4f}')
print(f'Logistic Regression accuracy: {lr_accuracy:.4f}')
print(f'Decision Tree accuracy: {dt_accuracy:.4f}')
```

Output:

```
Stacking accuracy: 0.6576
Logistic Regression accuracy: 0.5732
Decision Tree accuracy: 0.6526
```

Limitations of stacking

Stacking is very powerful in that we remove the occasionally difficult choice of which learning algorithm to use for our problem. Depending on the use case, this benefit does come with some limitations worth noting:

Additional Considerations

How many layers can a stacking model have?

Stacking offers some creativity and openendedness in how we want to build our model. A vanilla stacking model may have one tier of models to contribute to the final prediction. We could alternatively construct a multi-tier approach in which one level of models feeds into a later stage of models before making a final prediction.

As with other machine learning models, we also have many parameters to tune and select from. We can enhance model diversity by using different training algorithms, different training sets, different feature subsets, and different hyperparameters. In the next section, we'll use a k-fold cross-validation to preprocess the training data, where $k=10$. In reality, the value for k is arbitrary, and a different k folds may lead to performance improvement or decline.

Conclusion

We have covered in-depth how stacking can combine the predictive power of various machine learning models. This property distinguishes stacking from other ensemble methods like bagging or boosting, while not being mutually exclusive in application. Further, we discussed how a k-fold cross-validation setup of the training data can help our stacking model avoid overfitting and augment the data to leverage all of its predictive strengths. In a coded example, we see this in action using the `scikit-learn` library where we assembled a stacking model with logistic regression estimators and decision tree estimators. We found that using these estimators in a combination yielded higher prediction accuracy compared to using them independently.

Practice

Multiple choice

Questions

How is stacking different from other ensemble methods like bagging or boosting?

Answer Choices

Bagging and boosting models can only be used with decision trees, while Stacking can be used with other classifiers like logistic regression.

Bagging and boosting models can only be used for regression problems and not for classification problems. Stacking models can be used for either.

Stacking models cannot reduce variance in predictions but only bias. Boosting and bagging models can reduce both bias and variance.

A Stacking model can combine the predictive strengths of different types of estimators whereas bagging and boosting models typically use one estimator.

Multiple choice

Questions

In what way(s) can we fine-tune a Stacking model?

Answer Choices

Selecting a variety of level-0 estimator combinations (e.g. logistic regression, Decision trees, K-nearest neighbors).
Using different parameters in base estimators.
All techniques mentioned can be used in fine-tuning a Stacking model.
Experimenting with data pre-processing (e.g. using different values of k in k-fold cross-validation).
Varying the parameters of our final estimator (the model that combines the results of the base estimators).

Introduction to Supervised Learning Part II

More supervised machine learning fun!

You've already built a foundation in supervised machine learning, now it's time to go deeper and learn about another set of algorithms that build on the algorithms you've already learned about!

What will Supervised Learning for Data Science (Part II) Cover?

After this unit, you will be able to use and apply:

- Support Vector Machines (SVMs)
- Recommender Systems
- Naive Bayes Classification

Support Vector Machines

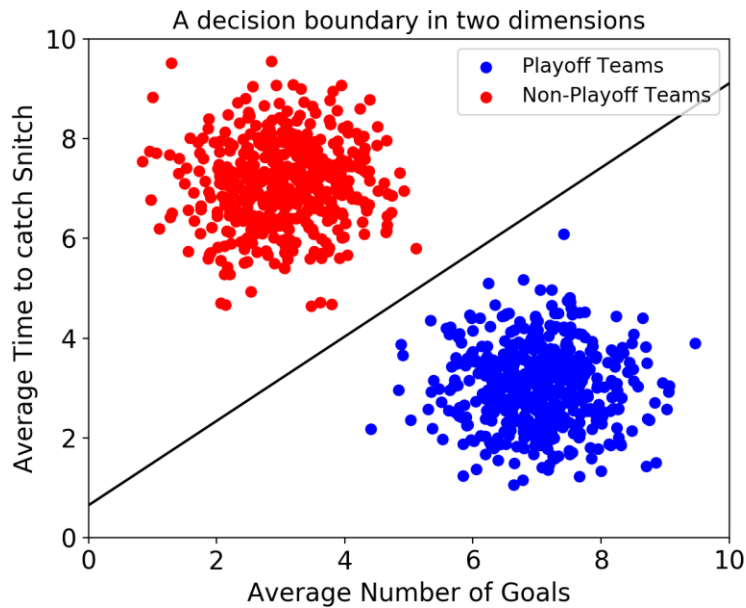
A **Support Vector Machine** (SVM) is a powerful supervised

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

model used for classification. An SVM makes classifications by defining a decision boundary and then seeing what side of the boundary an unclassified point falls on. In the next few exercises, we'll learn how these decision boundaries get defined, but for now, know that they're defined by using a training set of classified points. That's why SVMs are *supervised* machine learning models.

Decision boundaries are easiest to wrap your head around when the data has two features. In this case, the decision boundary is a line. Take a look at the example below.



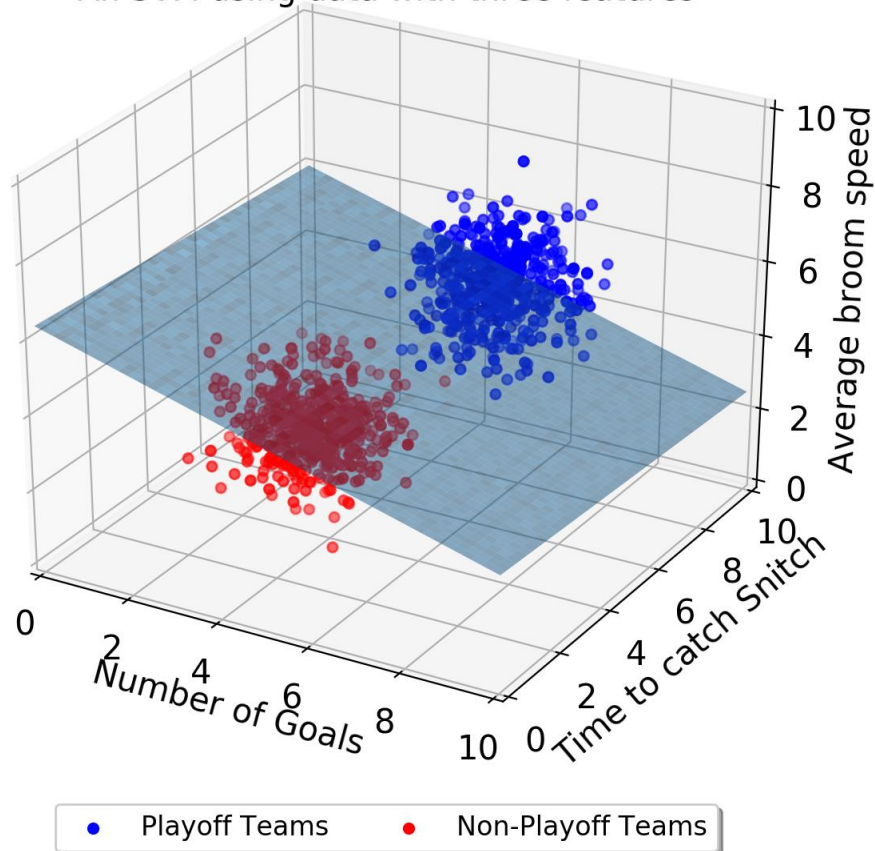
Note that if the labels on the figures in this lesson are too small to read, you can resize this pane to increase the size of the images.

This SVM is using data about fictional games of Quidditch from the Harry Potter universe! The classifier is trying to predict whether a team will make the playoffs or not. Every point in the training set represents a "historical" Quidditch team. Each point has two features — the average number of goals the team scores and the average number of minutes it takes the team to catch the Golden Snitch.

After finding a decision boundary using the training set, you could give the SVM an unlabeled data point, and it will predict whether or not that team will make the playoffs.

Decision boundaries exist even when your data has more than two features. If there are three features, the decision boundary is now a plane rather than a line.

An SVM using data with three features



As the number of dimensions grows past 3, it becomes very difficult to visualize these points in space. Nonetheless, SVMs can still find a decision boundary. However, rather than being a separating line, or a separating plane, the decision boundary is called a *separating hyperplane*.

Instructions

Checkpoint 1 Enabled

1.

Run the code to see two graphs appear. Right now they should be identical. We're going to fix the bottom graph so it has a good decision boundary. Why is this decision boundary bad?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's shift the line on the bottom graph to make it separate the two clusters. The slope of the line looks pretty good, so let's keep that at `-2`.

We want to move the boundary up, so change `intercept_two` so the line separates the two clusters.

```

import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from graph import ax, x_1, y_1, x_2, y_2

#Top graph intercept and slope
intercept_one = 8
slope_one = -2

x_vals = np.array(ax.get_xlim())
y_vals = intercept_one + slope_one * x_vals
plt.plot(x_vals, y_vals, '-')

#Bottom Graph
ax = plt.subplot(2, 1, 2)
plt.title('Good Decision Boundary')
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)

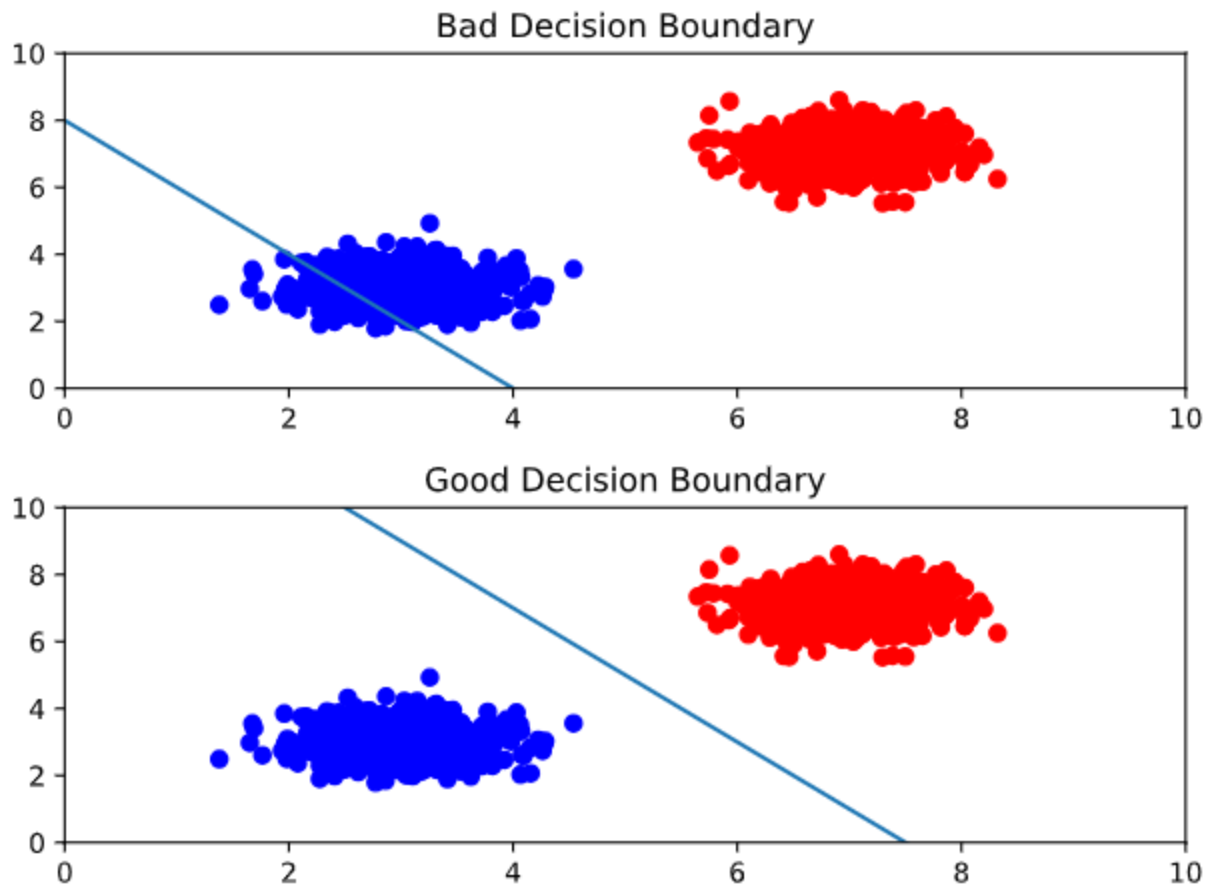
plt.scatter(x_1, y_1, color = "b")
plt.scatter(x_2, y_2, color = "r")

#Change the intercept to separate the clusters
intercept_two = 15
slope_two = -2

x_vals = np.array(ax.get_xlim())
y_vals = intercept_two + slope_two * x_vals
plt.plot(x_vals, y_vals, '-')

plt.tight_layout()
plt.show()

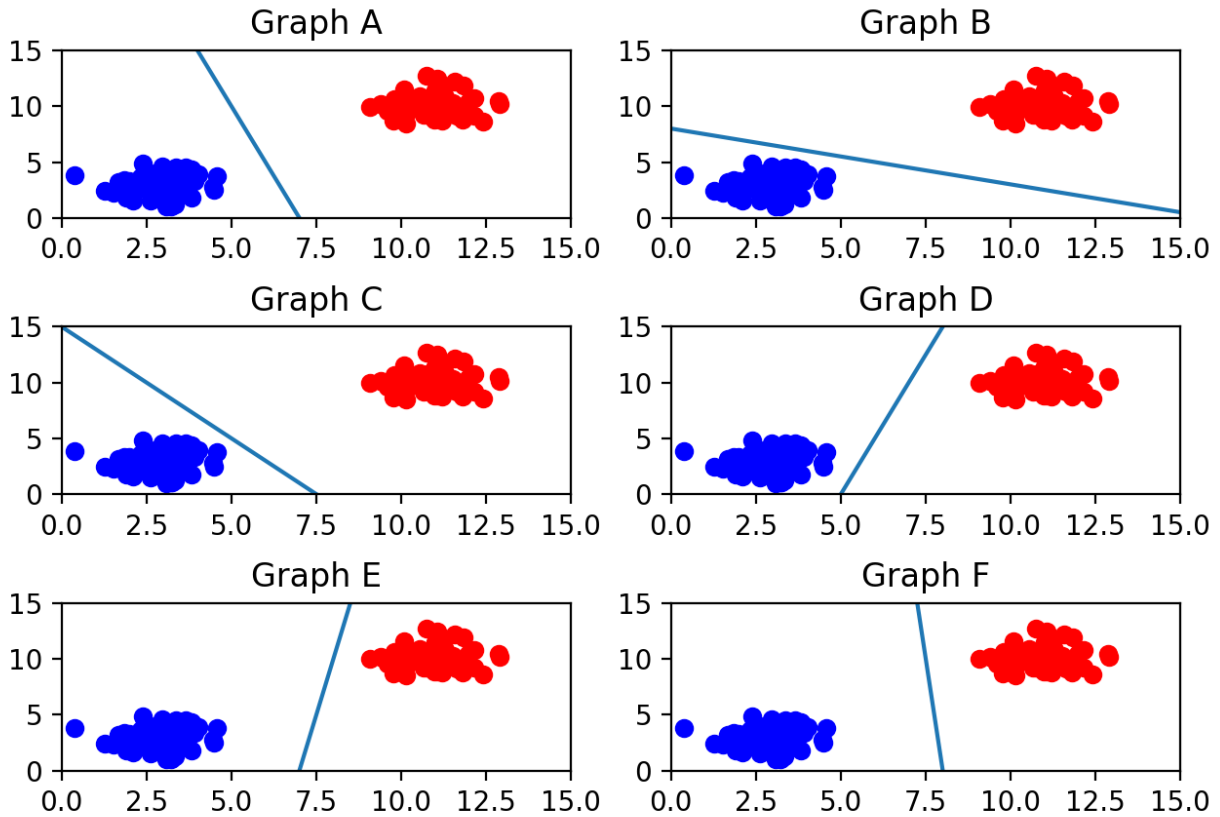
```



Optimal Decision Boundaries

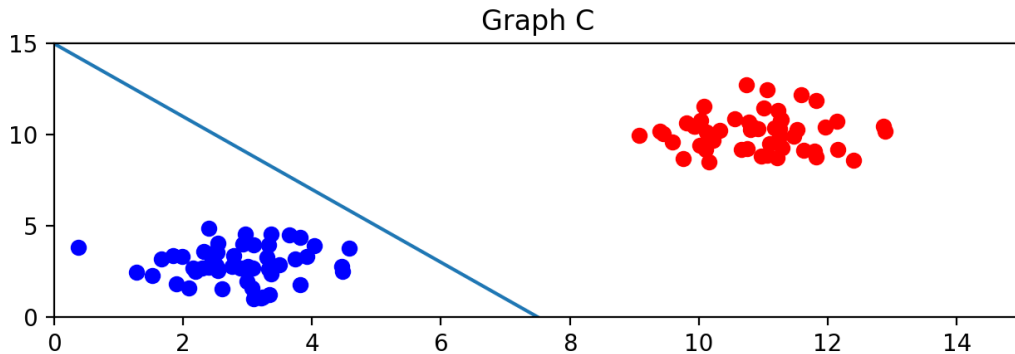
One problem that SVMs need to solve is figuring out what decision boundary to use. After all, there could be an infinite number of decision boundaries that correctly separate the two classes. Take a look at the image below:

Different Decision Boundaries



There are so many valid decision boundaries, but which one is best? In general, we want our decision boundary to be as far away from training points as possible.

Maximizing the distance between the decision boundary and points in each class will decrease the chance of false classification. Take graph C for example.



The decision boundary is close to the blue class, so it is possible that a new point close to the blue cluster would fall on the red side of the line.

Out of all the
 Preview: Docs Loading link description
[graphs](#)
 shown here, graph F has the best decision boundary.

Instructions

Checkpoint 1 Enabled

1.

Run the code. Both graphs have suboptimal decision boundaries. Why? We're going to fix the bottom graph.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We're going to have to make the decision boundary much flatter, which means we first need to lower its y-intercept. Change `intercept_two` to be 8.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, we want the slope to be pretty flat. Change the value of `slope_two`. The resulting line should split the two clusters.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from graph import ax, x_1, y_1, x_2, y_2

#Top graph intercept and slope
intercept_one = 98
slope_one = -20

x_vals = np.array(ax.get_xlim())
y_vals = intercept_one + slope_one * x_vals
plt.plot(x_vals, y_vals, '-')

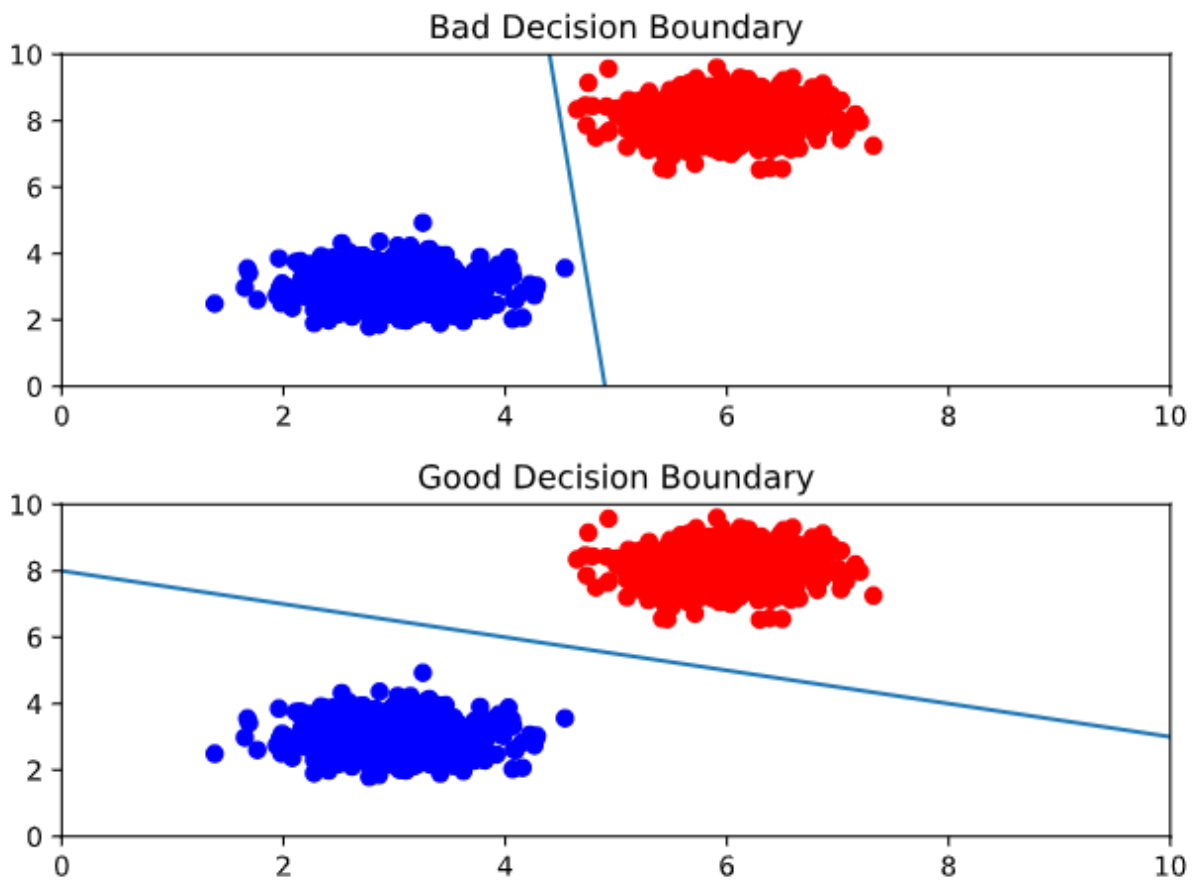
#Bottom graph
ax = plt.subplot(2, 1, 2)
plt.title('Good Decision Boundary')
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)

plt.scatter(x_1, y_1, color = "b")
plt.scatter(x_2, y_2, color = "r")

#Bottom graph intercept and slope
intercept_two = 8
slope_two = -0.5

x_vals = np.array(ax.get_xlim())
y_vals = intercept_two + slope_two * x_vals
plt.plot(x_vals, y_vals, '-')

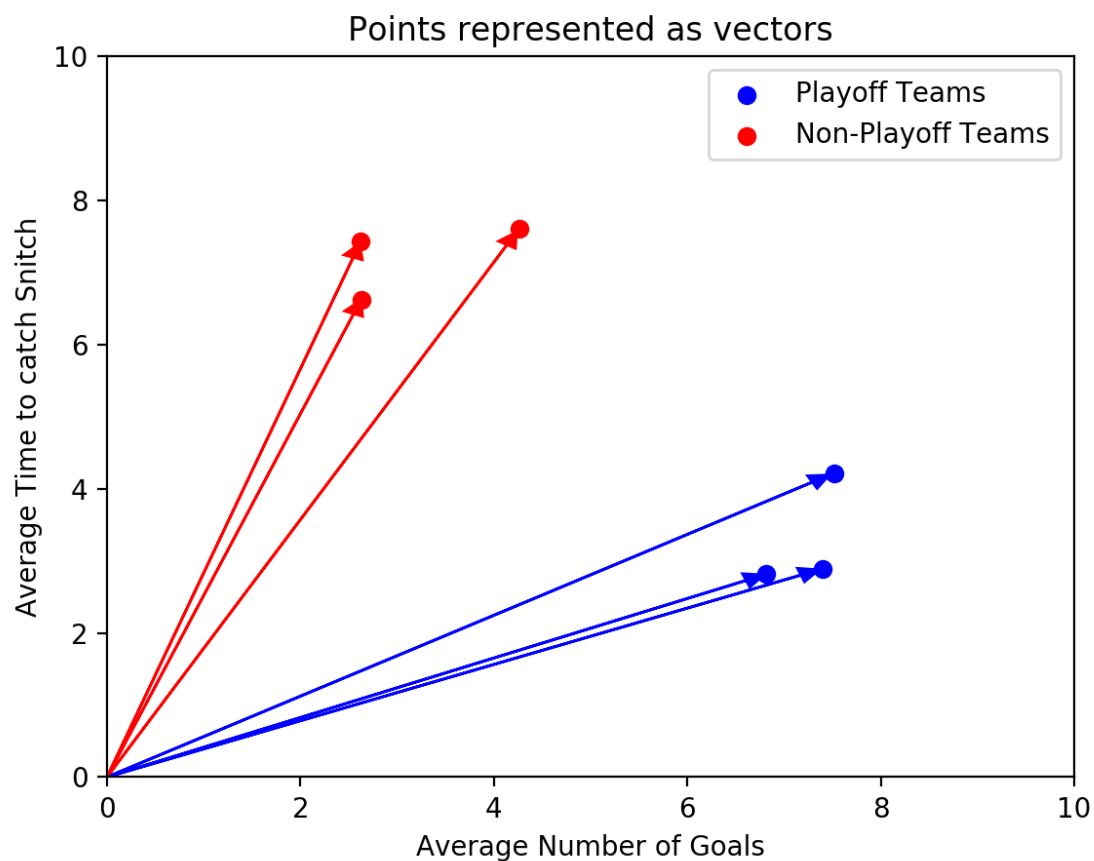
plt.tight_layout()
plt.show()
```



Support Vectors and Margins

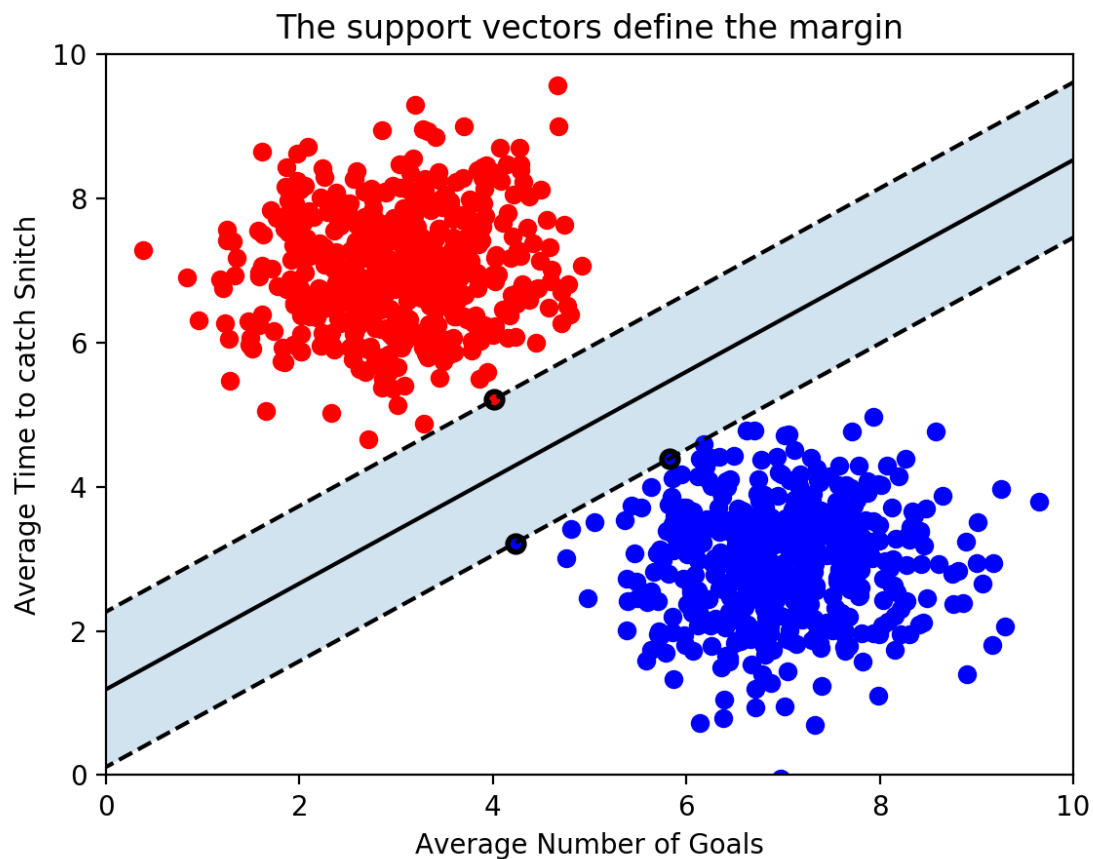
We now know that we want our decision boundary to be as far away from our training points as possible. Let's introduce some new terms that can help explain this idea.

The *support vectors* are the points in the training set closest to the decision boundary. In fact, these vectors are what define the decision boundary. But why are they called vectors? Instead of thinking about the training data as points, we can think of them as vectors coming from the origin.



These vectors are crucial in defining the decision boundary – that’s where the “support” comes from. If you are using n features, there are at least $n+1$ support vectors.

The distance between a support vector and the decision boundary is called the *margin*. We want to make the margin as large as possible. The support vectors are highlighted in the image below:



Because the support vectors are so critical in defining the decision boundary, many of the other training points can be ignored. This is one of the advantages of SVMs. Many supervised

Preview: Docs Loading link description

[machine learning](#)

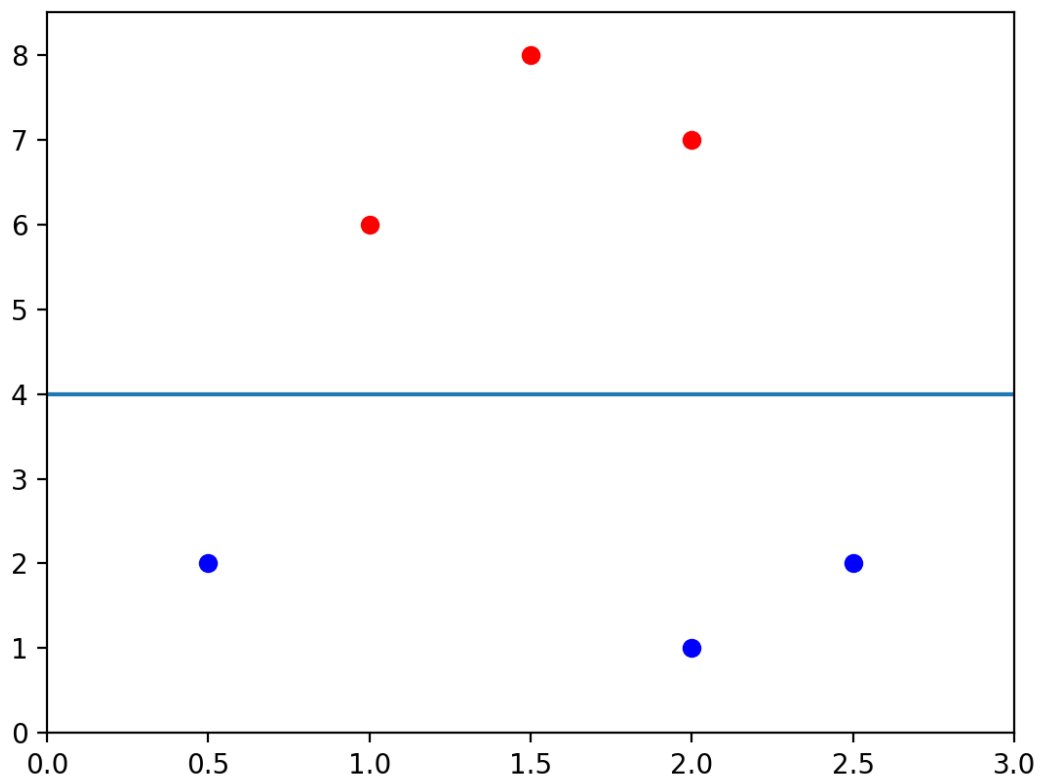
algorithms use every training point in order to make a prediction, even though many of those training points aren't relevant. SVMs are fast because they only use the support vectors!

Instructions

Checkpoint 1 Enabled

1.

What are the support vectors for the SVM pictured below? There should be 2 blue support vectors and 1 red support vector.



Finish defining `red_support_vector`, `blue_support_vector_one`, and `blue_support_vector_two`. Set them equal to the correct points. The point should be represented as a list like `[1, 0.5]`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed
2.

What is the size of the margin? Find the total distance between a support vector and the line by looking at the graph. Create a variable named `margin_size` and set it equal to the correct number.

scikit-learn

Now that we know the concepts behind SVMs we need to write the code that will find the decision boundary that maximizes the margin. All of the code that we've written so far has been guessing and checking — we don't actually know if we've found the best line. Unfortunately, calculating the parameters of the best decision boundary is a fairly complex optimization problem. Luckily, Python's scikit-learn library has implemented an SVM that will do this for us.

Note that while it is not important to understand how the optimal parameters are found, you should have a strong conceptual understanding of what the model is optimizing.

To use scikit-learn's SVM we first need to create an SVC object. It is called an SVC because scikit-learn is calling the model a Support Vector Classifier rather than a Support Vector Machine.

```
classifier = SVC(kernel = 'linear')
```

Copy to Clipboard

We'll soon go into what the `kernel`

Preview: Docs Loading link description

[parameter](#)

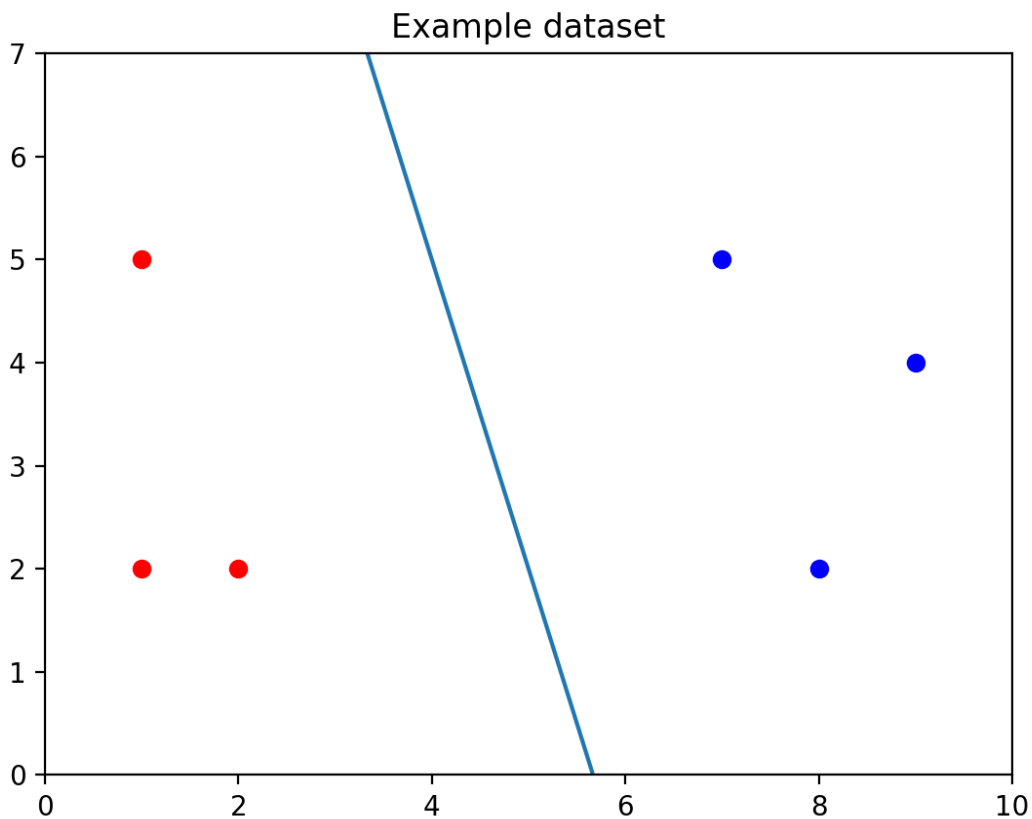
is doing, but for now, let's use a `'linear'` kernel.

Next, the model needs to be trained on a list of data points and a list of labels associated with those data points. The labels are analogous to the color of the point — you can think of a `1` as a red point and a `0` as a blue point. The training is done using the `.fit()` method:

```
training_points = [[1, 2], [1, 5], [2, 2], [7, 5], [9, 4], [8, 2]]
labels = [1, 1, 1, 0, 0, 0]
classifier.fit(training_points, labels)
```

Copy to Clipboard

The graph of this dataset would look like this:



Calling `.fit()` creates the line between the points.

Finally, the classifier predicts the label of new points using the `.predict()`

Preview: Docs Loading link description

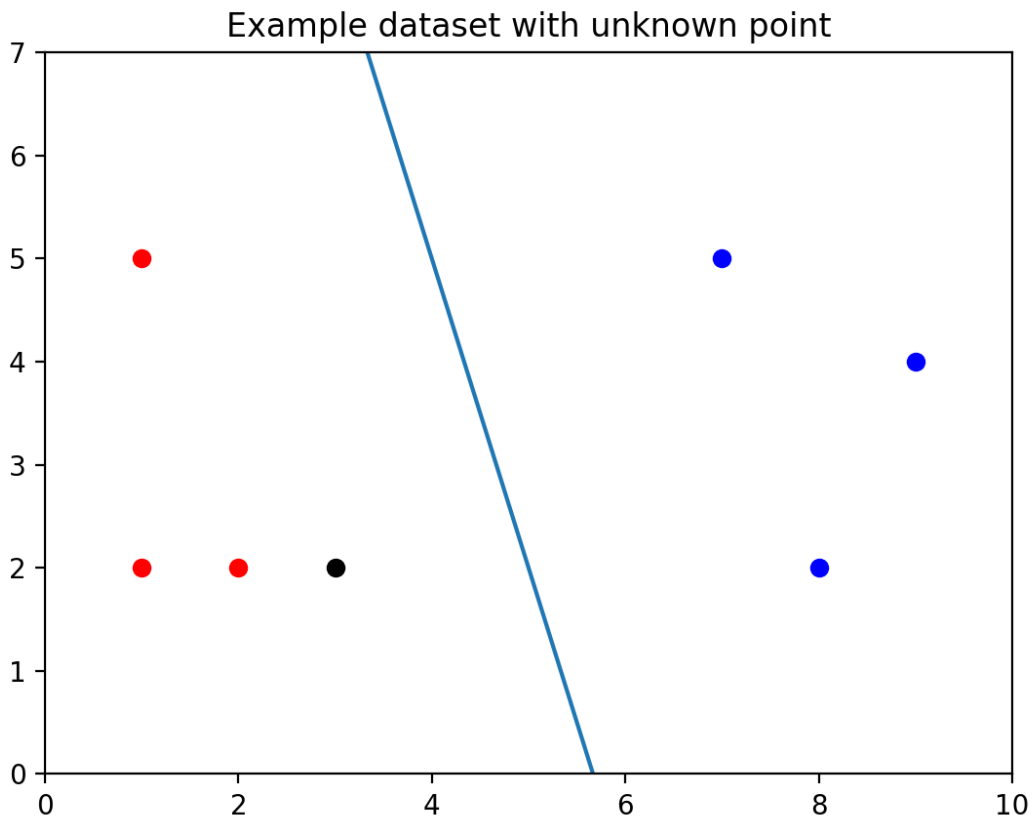
[method](#)

. The `.predict()` method takes a list of points you want to classify. Even if you only want to classify one point, make sure it is in a list:

```
print(classifier.predict([[3, 2]]))
```

Copy to Clipboard

In the image below, you can see the unclassified point `[3, 2]` as a black dot. It falls on the red side of the line, so the SVM would predict it is red.



In addition to using the SVM to make predictions, you can inspect some of its attributes. For example, if you can print `classifier.support_vectors_` to see which points from the training set are the support vectors.

In this case, the support vectors look like this:

```
[[7, 5],  
 [8, 2],  
 [2, 2]]
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Let's start by making a `SVC` object with `kernel = 'linear'`. Name the object `classifier`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We've imported the training set and labels for you. Call `classifier's .fit()` method using `points` and `labels` as parameters.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

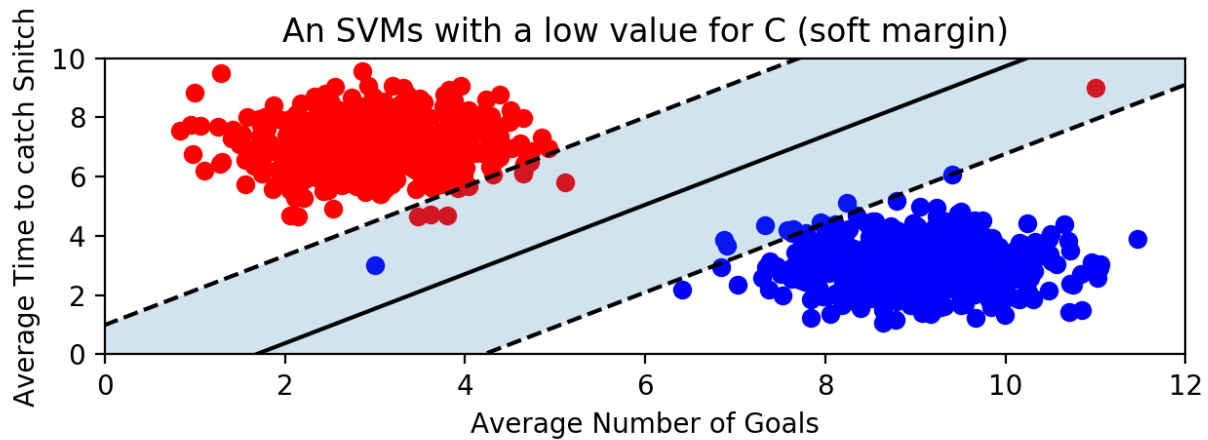
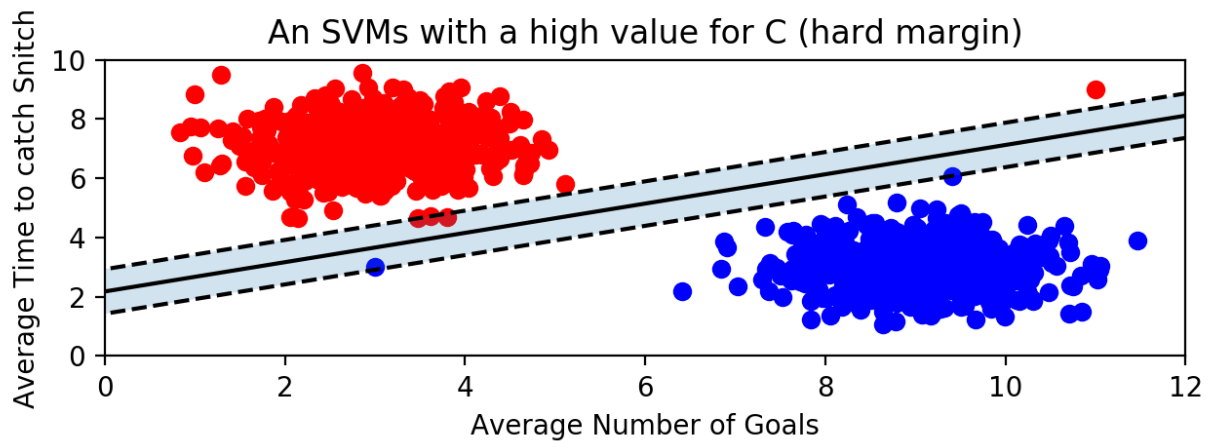
We can now classify new points. Try classifying both `[3, 4]` and `[6, 7]`. Remember, the `.predict()` function expects a list of points to predict. Print the results.

```
from sklearn.svm import SVC
from graph import points, labels

classifier = SVC(kernel = 'linear')
classifier.fit(points, labels)
print(classifier.predict([[3, 4], [6, 7]]))
```

Outliers

SVMs try to maximize the size of the margin while still correctly separating the points of each class. As a result, outliers can be a problem. Consider the image below.



The size of the margin decreases when a single outlier is present, and as a result, the decision boundary changes as well. However, if we allowed the decision boundary to have some error, we could still use the original line.

SVMs have a

Preview: Docs Loading link description

parameter

C that determines how much error the SVM will allow for. If C is large, then the SVM has a hard margin — it won't allow for many misclassifications, and as a result, the margin could be fairly small. If C is too large, the model runs the risk of overfitting. It relies too heavily on the training data, including the outliers.

On the other hand, if C is small, the SVM has a soft margin. Some points might fall on the wrong side of the line, but the margin will be large. This is resistant to outliers, but if C gets too small, you run the risk of underfitting. The SVM will allow for so much error that the training data won't be represented.

When using scikit-learn's SVM, you can set the value of C when you create the object:

```
classifier = SVC(C = 0.01)
```

Copy to Clipboard

The optimal value of C will depend on your data. Don't always maximize margin size at the expense of error. Don't always minimize error at the expense of margin size. The best strategy is to validate your model by testing many different values for C .

Instructions

Checkpoint 1 Enabled

1.

Run the code to see the SVM's current boundary line. Note that we've imported some helper functions we wrote named `draw_points` and `draw_margins` to help visualize the SVM.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's add an outlier! Before calling `.fit()`, append `[3, 3]` to `points` and append `0` to `labels`. This will add a blue point at `[3, 3]`

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Right now, our classifier has hard margins because $C = 1$. Change the value of C to `0.01` to see what the SVM looks like with soft margins.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

append at least two more points to `points`. If you want the points to appear on the graph, make sure their `x` and `y` values are between `0` and `12`.

Make sure to also append a label to `labels` for every point you add. A `0` will make the point blue and a `1` will make the point red.

Make sure to add the points before training the SVM.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Play around with the C variable to see how the decision boundary changes with your new points added.

Change C to be a value between `0.01` and `1`.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from graph import points, labels, draw_points, draw_margin

points.append([3, 3])
labels.append(0)

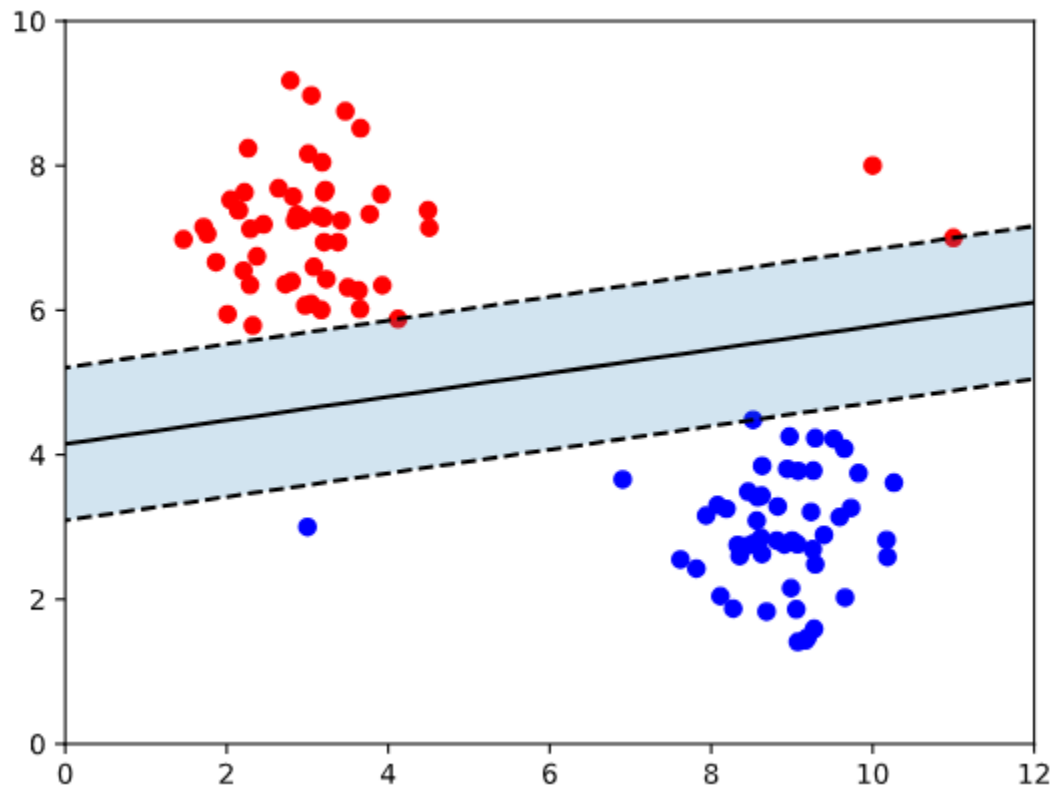
points.append([10, 8])
labels.append(1)

points.append([11, 7])
labels.append(1)

classifier = SVC(kernel='linear', C = 0.5)
classifier.fit(points, labels)

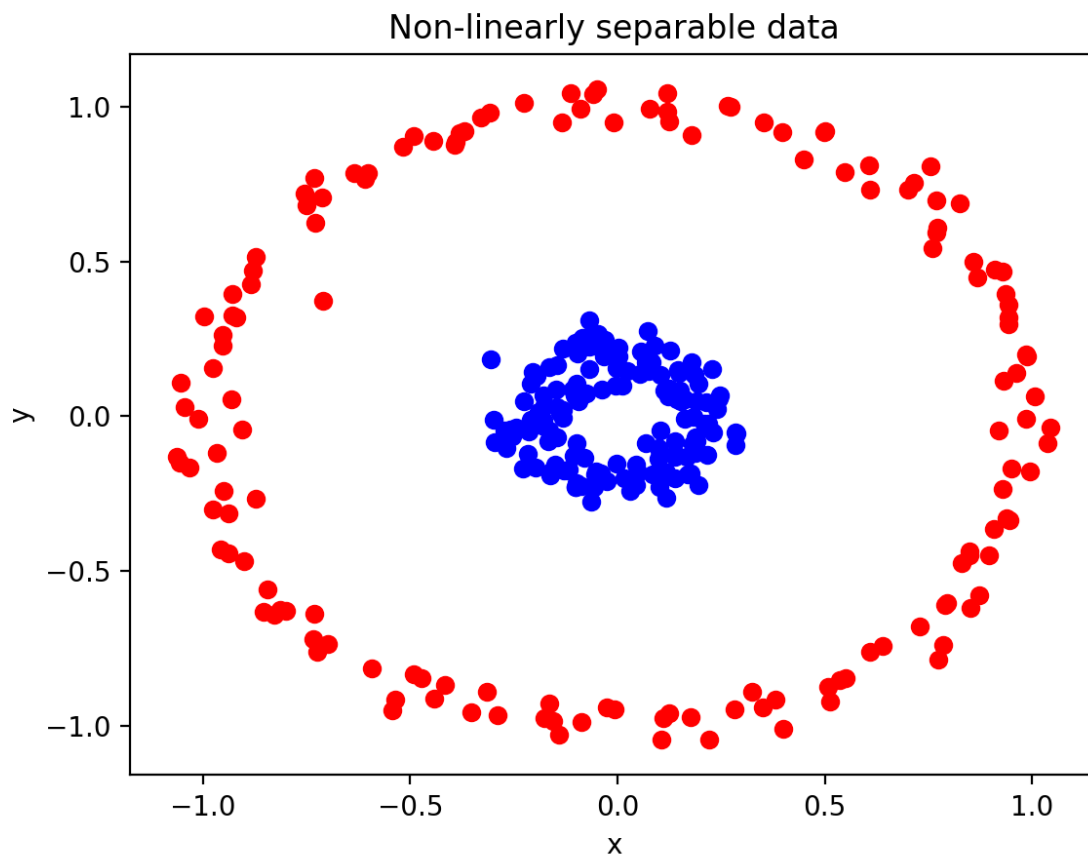
draw_points(points, labels)
draw_margin(classifier)
```

```
plt.show()
```



Kernels

Up to this point, we have been using data sets that are linearly separable. This means that it's possible to draw a straight decision boundary between the two classes. However, what would happen if an SVM came along a dataset that wasn't linearly separable?



It's impossible to draw a straight line to separate the red points from the blue points!

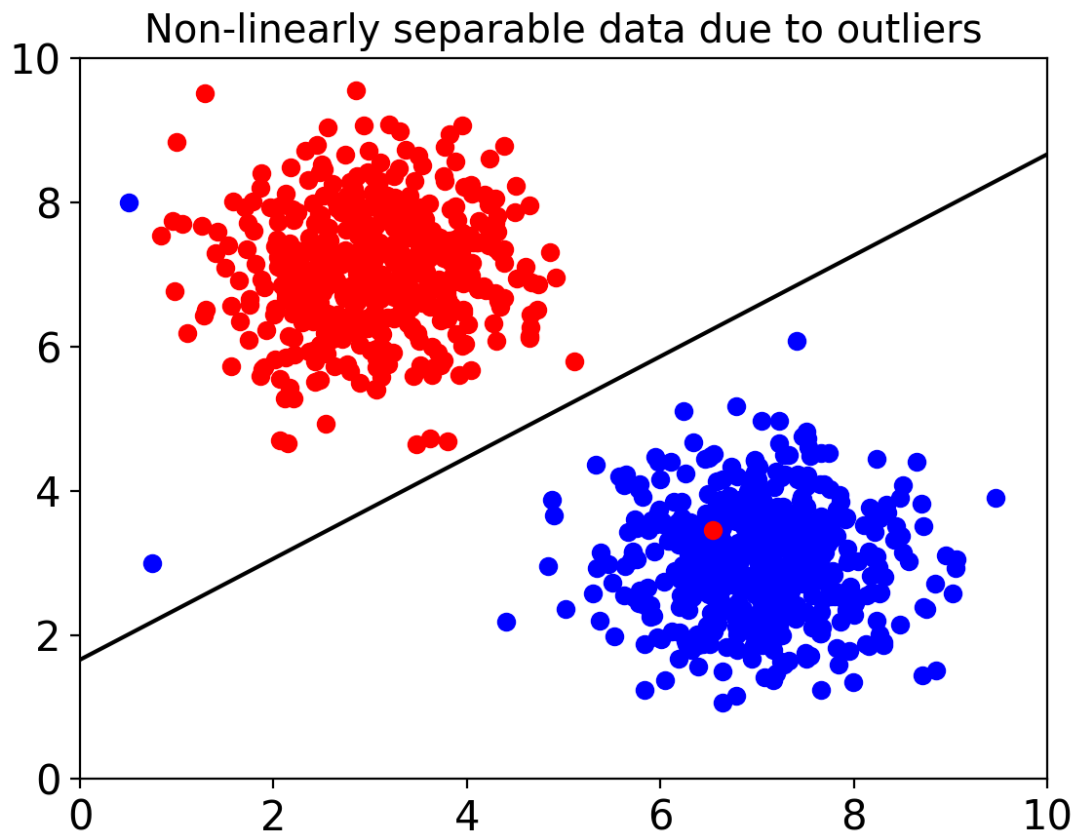
Luckily, SVMs have a way of handling these data sets. Remember when we set `kernel = 'linear'` when creating our SVM? Kernels are the key to creating a decision boundary between data points that are not linearly separable.

Note that most

Preview: Docs Loading link description

[machine learning](#)

models should allow for some error. For example, the image below shows data that isn't linearly separable. However, it is not linearly separable due to a few outliers. We can still draw a straight line that, for the most part, separates the two classes. You shouldn't need to create a non-linear decision boundary just to fit some outliers. Drawing a line that correctly separates every point would be drastically overfitting the model to the data.



Instructions

Checkpoint 1 Failed, try again

1.

Let's take a look at the power of kernels. We've created a dataset that isn't linearly separable and split it into a training set and a validation set.

Create an `SVC` named `classifier` with a `'linear'` kernel.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Call the `.fit()` method using `training_data` and `training_labels` as parameters.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's see how accurate our classifier is using a linear kernel.

Call `classifier's .score()` function using `validation_data` and `validation_labels` as parameters. Print the results.

This will print the average accuracy of the model.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

That's pretty bad! The classifier is getting it right less than 50% of the time! Change `'linear'` to `'poly'` and add the parameter `degree = 2`. Run the program again and see what happens to the score.

```
import codecademylib3_seaborn
from sklearn.svm import SVC
from graph import points, labels
from sklearn.model_selection import train_test_split
```

```

training_data, validation_data, training_labels, validation_labels =
train_test_split(points, labels, train_size = 0.8, test_size = 0.2, random_state =
100)

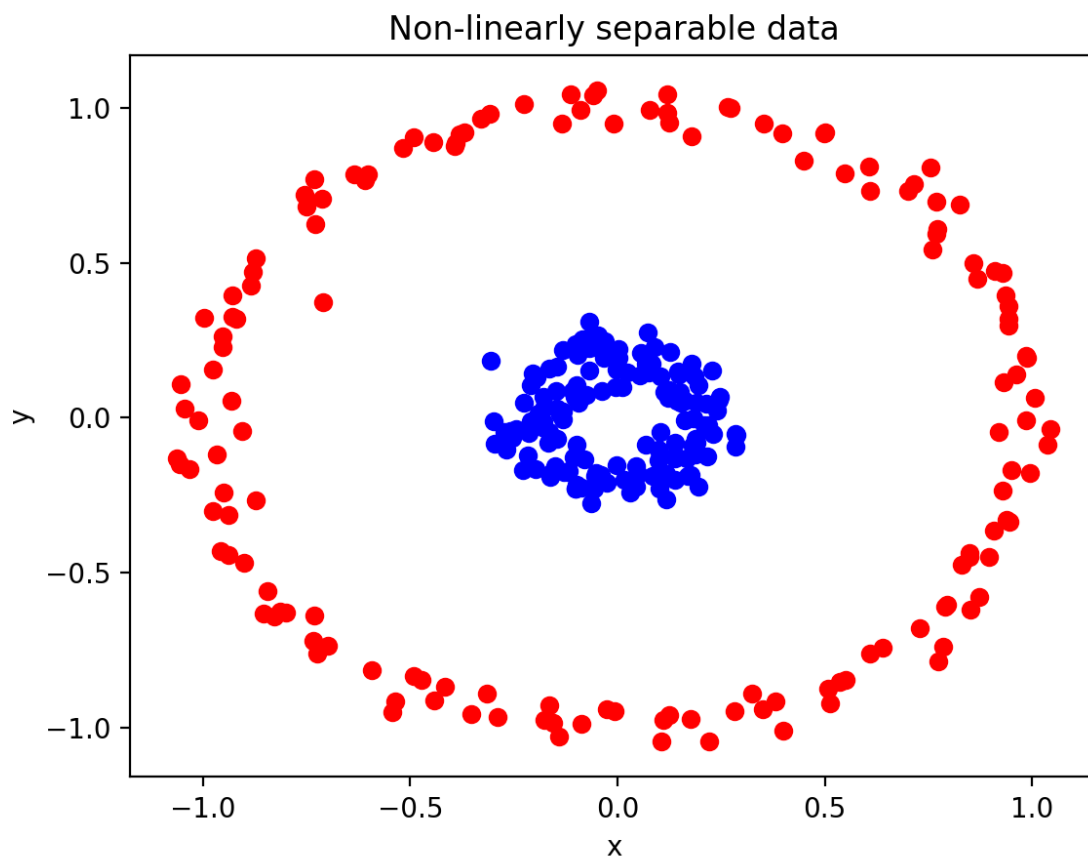
classifier = SVC(kernel = "poly", degree = 2)
classifier.fit(training_data, training_labels)
print(classifier.score(validation_data, validation_labels))

```

Polynomial Kernel

That kernel seems pretty magical. It is able to correctly classify every point! Let's take a deeper look at what it was really doing.

We start with a group of non-linearly separable points that looked like this:



The kernel transforms the data in a clever way to make it linearly separable. We used a polynomial kernel which transforms every point in the following way:

$$(x, y) \rightarrow (2xy, x^2, y^2)$$

$$(x, y) \rightarrow (2xy, y, x)$$

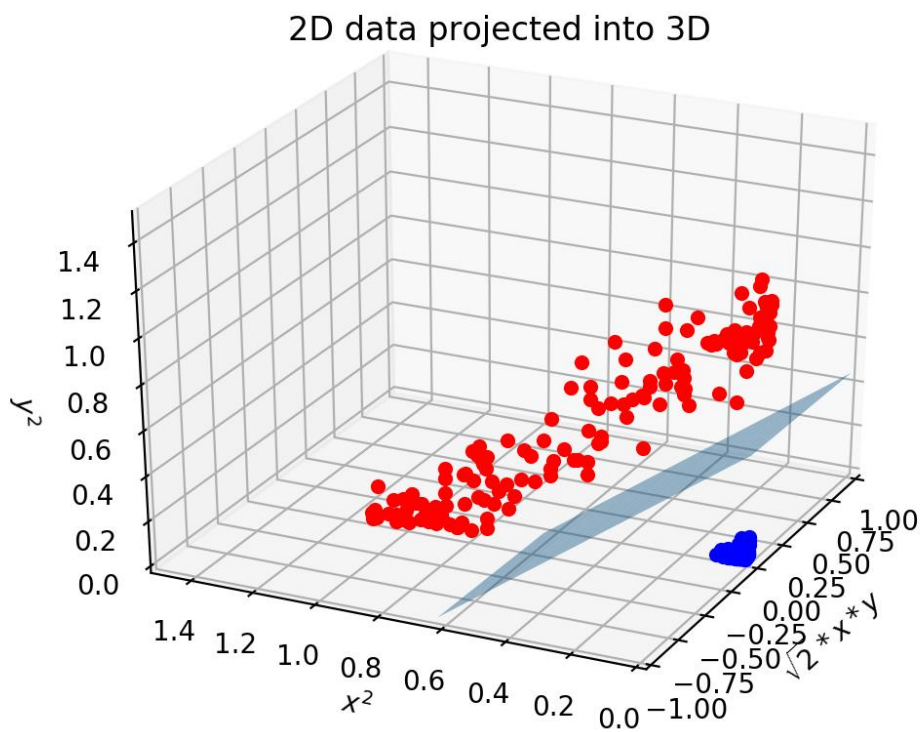
,y
2
)

The kernel has added a new dimension to each point! For example, the kernel transforms the point `[1, 2]` like this:

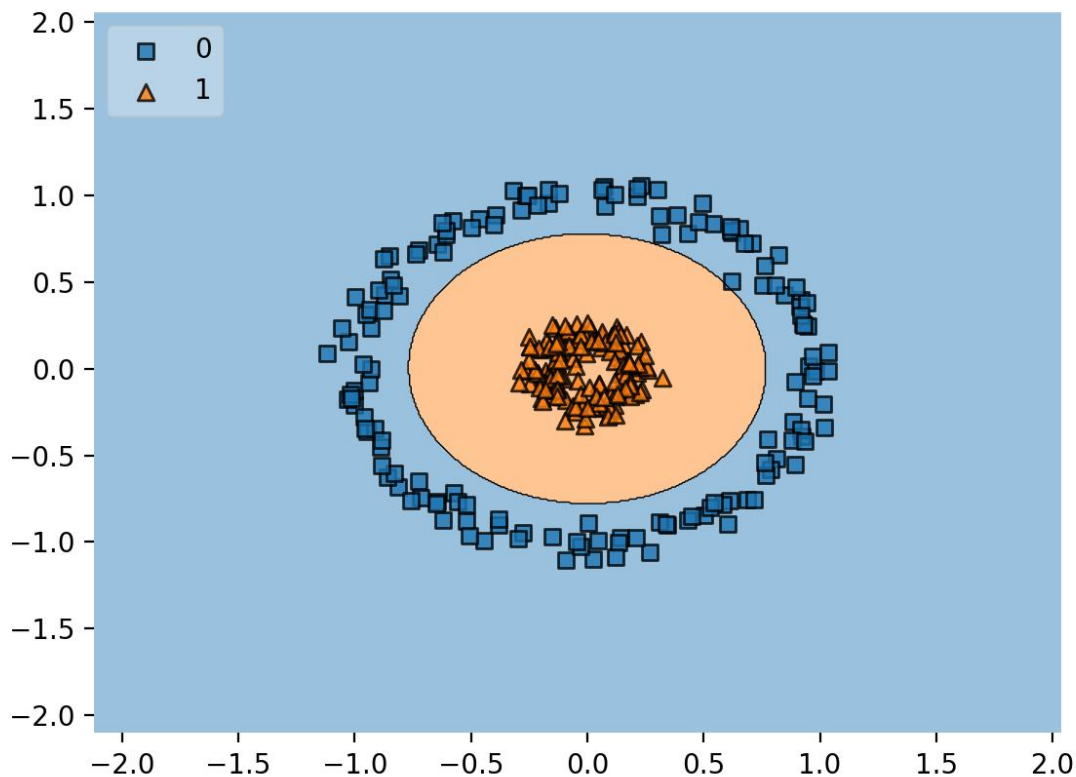
$$(1, 2) \rightarrow (22, 1, 4)$$

$$(1, 2) \rightarrow (22, 1, 4)$$

If we plot these new three dimensional points, we get the following graph:



Look at that! All of the blue points have scooted away from the red ones. By projecting the data into a higher dimension, the two classes are now linearly separable by a plane. We could visualize what this plane would look like in two dimensions to get the following decision boundary.



Instructions

Checkpoint 1 Enabled

1.

In this exercise, we will be using a non-linearly separable dataset similar to the concentric circles above. Rather than using a polynomial kernel, we're going to stick with a linear kernel and do the transformation ourselves. The SVM running a linear kernel on the transformed points should perform identically to the SVM running a polynomial kernel on the original points.

To begin, at the bottom of your code, print `training_data[0]` to see the first data point. You will also see the accuracy of the SVM when the data is not projected into 3 dimensions.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's transform the data into three dimensions! Begin by creating two empty lists called `new_training` and `new_validation`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Loop through every point in `training_data`. For every point, append a list to `new_training`. The list should contain three numbers:

The square root of 2 times `point[0]` times `point[1]`.

`point[0]` squared.

`point[1]` squared.

Remember, to square a number in Python do `number ** 2`. To take the square root, do `number ** 0.5`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Do the same for every point in `validation_data`. For every point in `validation_data`, add the new list to `new_validation`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Retrain `classifier` by calling the `.fit()` method using `new_training` and `training_labels` as parameters.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Finally, run `classifier`'s `.score()` method using `new_validation` and `validation_labels` as parameters. Print the results. How did the SVM do when the data was projected to three dimensions?

```
from sklearn.datasets import make_circles
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split

#Makes concentric circles
points, labels = make_circles(n_samples=300, factor=.2, noise=.05, random_state = 1)

#Makes training set and validation set.
training_data, validation_data, training_labels, validation_labels =
train_test_split(points, labels, train_size = 0.8, test_size = 0.2, random_state =
100)

classifier = SVC(kernel = "linear", random_state = 1)
classifier.fit(training_data, training_labels)
print(classifier.score(validation_data, validation_labels))
print(training_data[0])

new_training = [[2 ** 0.5 * pt[0] * pt[1], pt[0] ** 2, pt[1] ** 2] for pt in
training_data]
new_validation = [[2 ** 0.5 * pt[0] * pt[1], pt[0] ** 2, pt[1] ** 2] for pt in
validation_data]

classifier.fit(new_training, training_labels)
print(classifier.score(new_validation, validation_labels))
```

Radial Basis Function Kernel

The most commonly used kernel in SVMs is a radial basis function (**rbf**) kernel. This is the default kernel used in scikit-learn's `SVC` object. If you don't specifically set the kernel to `"linear"`, `"poly"` the `SVC` object will use an `rbf` kernel. If you want to be explicit, you can set `kernel = "rbf"`, although that is redundant.

It is very tricky to visualize how an `rbf` kernel "transforms" the data. The polynomial kernel we used transformed two-dimensional points into three-dimensional points. An `rbf` kernel transforms two-dimensional points into points with an infinite number of dimensions!

We won't get into how the kernel does this — it involves some fairly complicated linear algebra. However, it is important to know about the `rbf` kernel's `gamma`

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`:
`js function tripleThis(x) { return x * 3; } tripleThis(6);`
[parameter](#)

```
classifier = SVC(kernel = "rbf", gamma = 0.5, C = 2)
```

Copy to Clipboard

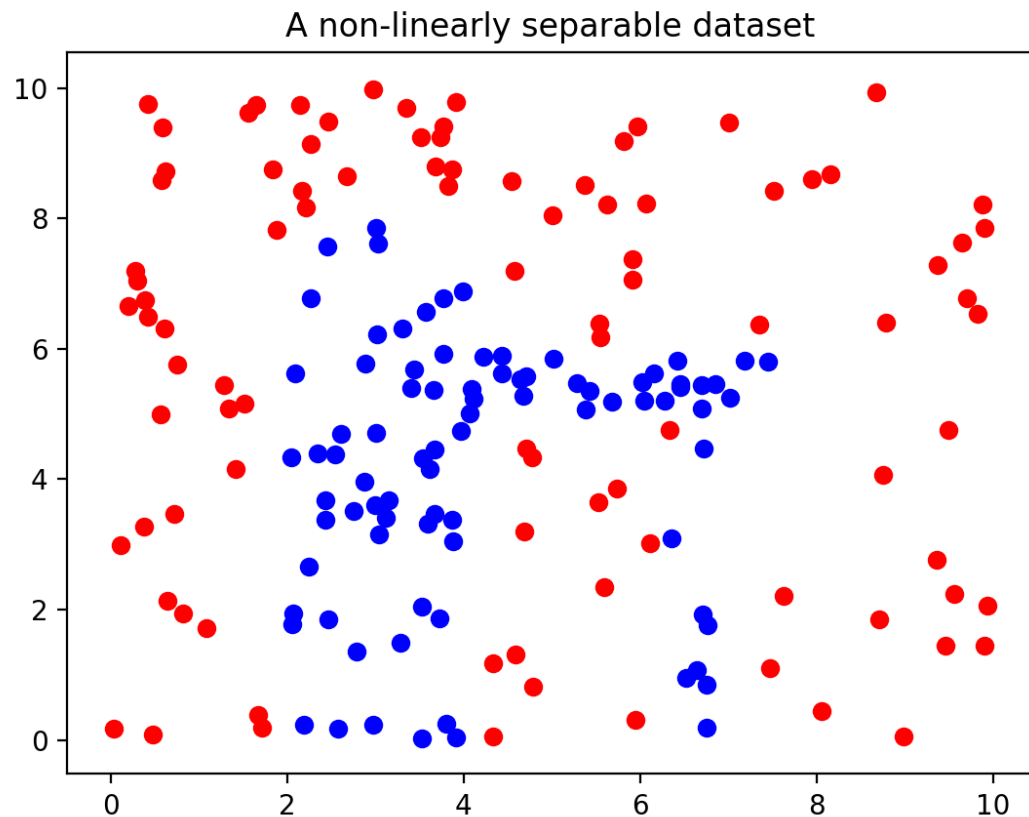
`gamma` is similar to the `C` parameter. You can essentially tune the model to be more or less sensitive to the training data. A higher `gamma`, say `100`, will put more importance on the training data and could result in overfitting. Conversely, A lower `gamma` like `0.01` makes the points in the training data less relevant and can result in underfitting.

Instructions

Checkpoint 1 Failed, try again

1.

We're going to be using a rbf kernel to draw a decision boundary for the following points:



We've imported the data for you and split it into `training_data`, `validation_data`, `training_labels`, and `validation_labels`.

Begin by creating an `SVC` named `classifier` with an "rbf" kernel. Set the kernel's `gamma` equal to `1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Next, train the model using the `.fit()` method using `training_data` and `training_labels` as parameters.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's test the classifier's accuracy when its `gamma` is `1`. Print the result of the `.score()` function using `validation_data` and `validation_labels` as parameters.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Let's see what happens if we increase `gamma`. Change `gamma` to `10`. What happens to the accuracy of our model?

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

The accuracy went down. We overfit our model. Change `gamma` to `0.1`. What happens to the accuracy of our model this time?

```

from data import points, labels
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

training_data, validation_data, training_labels, validation_labels =
train_test_split(points, labels, train_size = 0.8, test_size = 0.2, random_state =
100)

classifier = SVC(kernel = "rbf", gamma = 0.1)
classifier.fit(training_data, training_labels)
print(classifier.score(validation_data, validation_labels))

```

Review

Great work! Here are some of the major takeaways from this lesson on SVMs:

SVMs are supervised

Preview: Docs Loading link description

[machine learning](#)

models used for classification.

An SVM uses support vectors to define a decision boundary. Classifications are made by comparing unlabeled points to that decision boundary.

Support vectors are the points of each class closest to the decision boundary. The distance between the support vectors and the decision boundary is called the margin.

SVMs attempt to create the largest margin possible while staying within an acceptable amount of error.

The C

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called tripleThis(): js function tripleThis(x) { return x * 3; } tripleThis(6);

[parameter](#)

controls how much error is allowed. A large C allows for little error and creates a hard margin. A small C allows for more error and creates a soft margin.

SVMs use kernels to classify points that aren't linearly separable.

Kernels transform points into higher dimensional space. A polynomial kernel transforms points into three dimensions while an rbf kernel transforms points into infinite dimensions.

An rbf kernel has a γ parameter. If γ is large, the training data is more relevant, and as a result overfitting can occur.

Recommender Systems

What are Recommender Systems?

With the growth of the internet, the number of options people have for everything from watching videos, to buying clothes, to finding a date has increased dramatically. Having potentially thousands of options, how are we to find and choose the best options for us?

Many internet companies have provided us tools to help us navigate these seemingly infinite number of choices. For example, when you visit a product page on an ecommerce site, you may have noticed a section of the page that suggests other products you might like based on things you have purchased in the past. Perhaps you have received a push notification on your mobile phone recommending a video to watch. The closer you look, the more likely you are to find all sorts of recommendations being made in the websites and mobile apps you use.

All these tools are powered by **recommender systems**. Recommender systems are algorithms that use data about products and users' preferences to make recommendations to users about the best options to choose from a set of options.

In this lesson, we will learn about the properties of recommender systems, the ways they differ from traditional supervised learning, and the different types of recommender systems.



Recommender Systems versus Supervised Learning

As you learned about recommender systems in the previous exercise, you may be wondering how recommender systems differ from traditional supervised learning techniques you have learned about in previous sections. In general, recommender systems do often utilize

Preview: Docs Loading link description

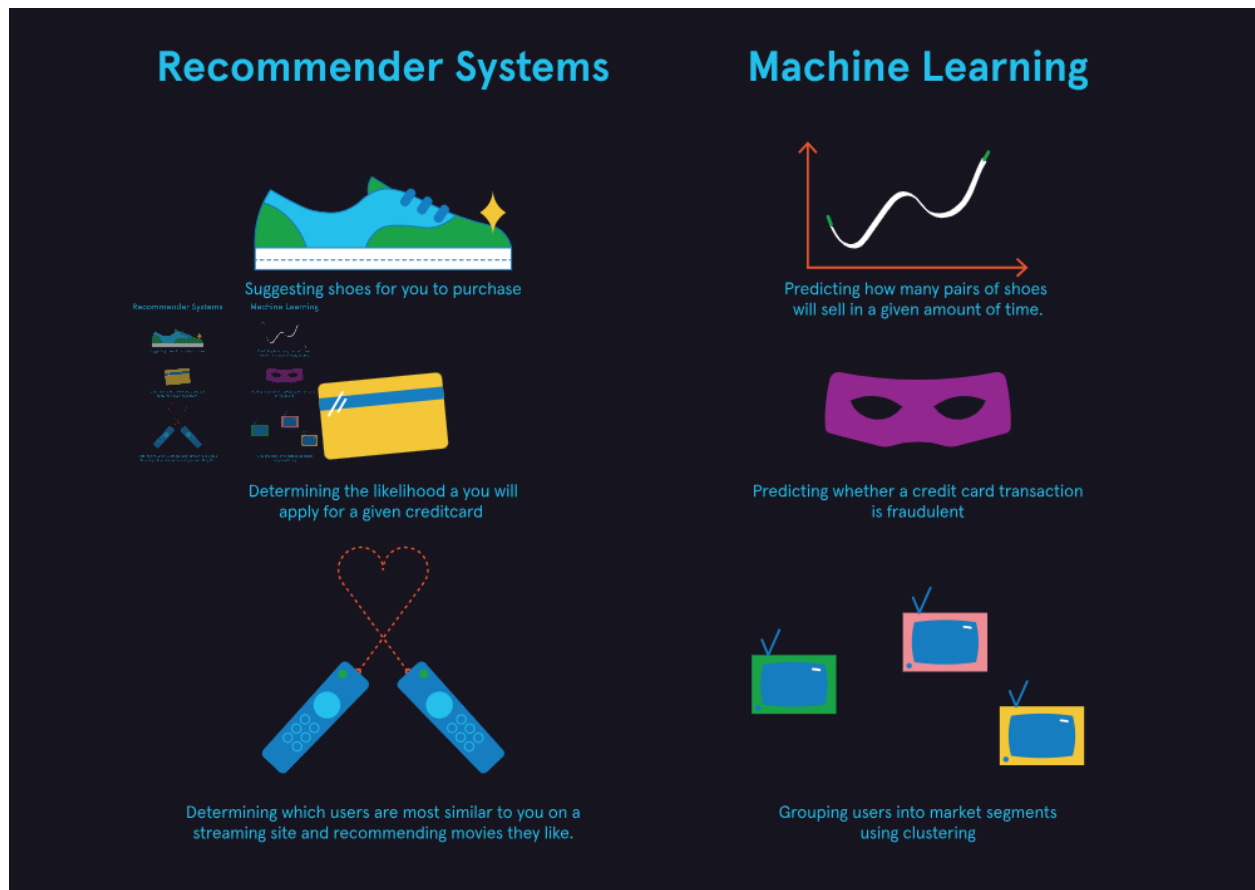
[machine learning](#)

techniques. So what is the difference between the two terms?

The difference is not in technique, but in purpose. Recommender systems are built to address problems of determining the best action for a user to take given a set of options. Supervised learning is generally used to describe machine learning to predict outcomes.

Let's take the example of an E-Commerce website selling shoes. If the website built a machine learning model that was designed to determine how much each user would like each shoe, that model would be considered a recommender system. However, if the purpose of the machine learning model is built to determine how many shoes the site would sell next month, that would be considered just an application of supervised learning.

Recommender systems also differ from traditional supervised learning in the specific properties are important for measuring their performance, as we shall see in the following section.



Properties of Good Recommender Systems

When considering the performance of a recommender system, there are a couple of different dimensions we should consider:

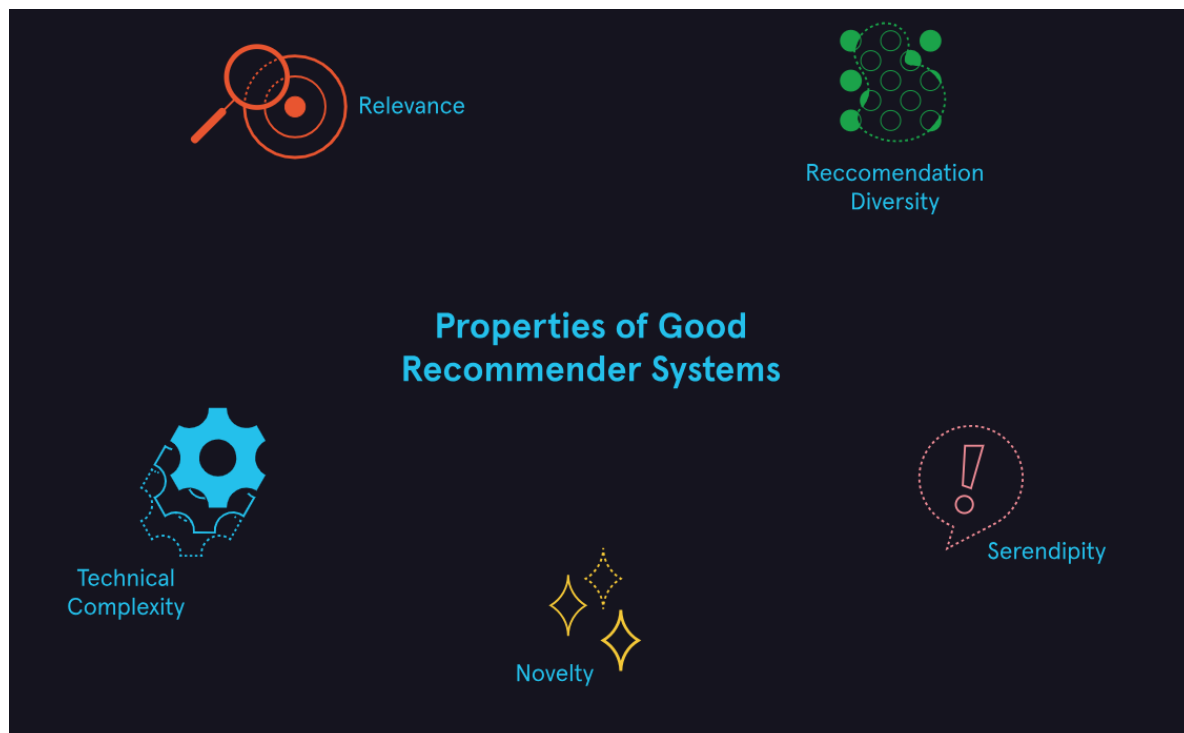
Relevance: When a recommender system makes a recommendation, it should be relevant to the user. More specifically, these recommendations should be ones the user would likely rate highly. For example, a female user who only looks at and buys women's shoes on an ecommerce website should generally only receive recommendations for women's shoes. For many data practitioners, this is the most obvious and important property to optimize. However, it is critical to understand this is not the only important property of a recommender system.

Novelty: Recommender systems should be ideally making recommendations the user has not seen before. For example, a recommender system that consistently recommends only the most popular women's shoes to a female user may be considered a poor recommender system.

Serendipity: Good recommender systems generally make recommendations that are somewhat unexpected. These sorts of recommendations (assuming they are relevant) can often delight users. Going back to our shoe ecommerce site example, a user that often purchases running shoes might get recommended sandals that are good for running from a serendipitous recommender system.

Recommendation Diversity: Recommender systems that recommend many different types of items are more likely to have at least one item liked by the user. Looking at our shoe store site example, a recommender system that suggests to users not only shoes, but socks and shoe polish may be preferable to one that only recommends shoes.

Technical Complexity: Recommender systems may often consist of many complex algorithms and parts. As a result, these systems require maintenance and some level of interpretability by technical staff. Therefore, recommender systems that are less complex and easier to understand are preferable from a cost and risk perspective.



Types of Recommender Systems

Recommender systems can generally be classified into one of three different groups depending on the algorithmic approach they take to make recommendations:

Collaborative Filtering: Collaborative filtering is a recommender system technique that makes recommendations for a target user by using ratings information from other users. The driving principle behind collaborative filtering is that users that have similar ratings for items have similar tastes.

Content-based Filtering: Content-based Filtering is a recommender system technique that uses data about user preferences and attributes of items to model the likelihood a given user will like a specific item. This type of recommender systems tends to look more like the traditional

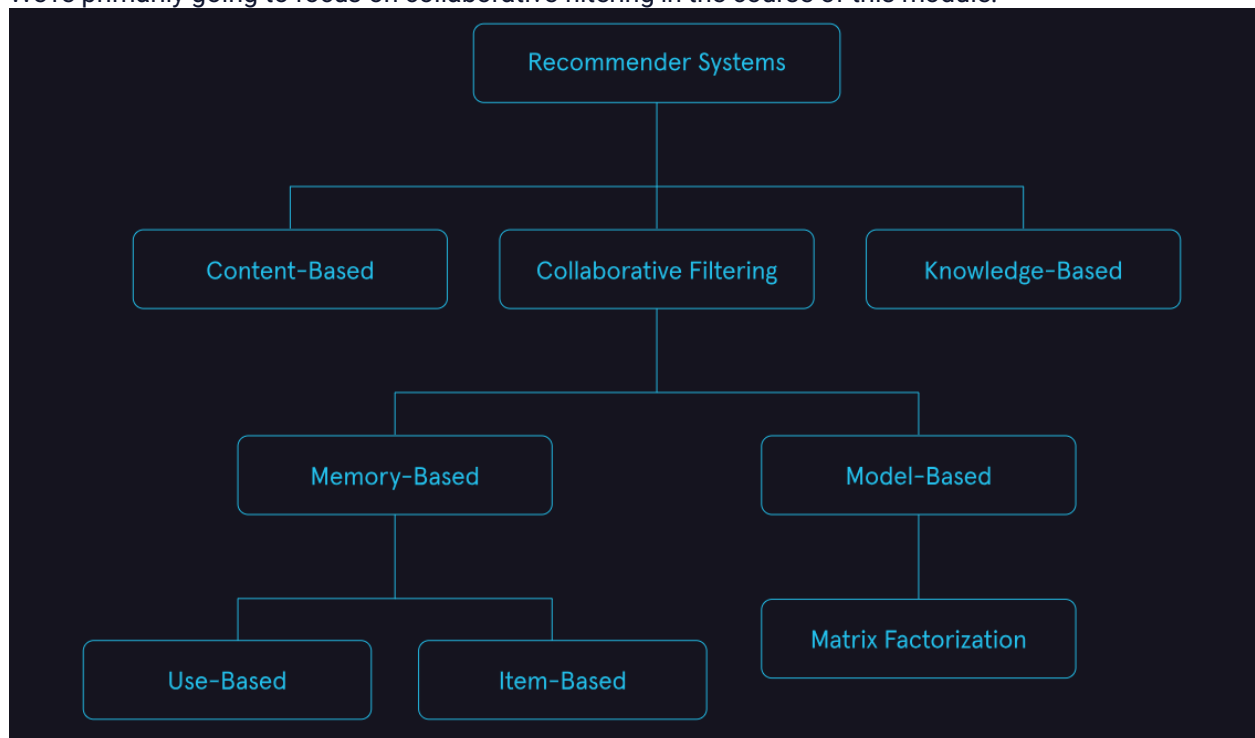
Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

models used in supervised learning.

Knowledge-Based: Knowledge-based recommender systems are a class of recommender systems used when there is not a lot of data available. Rules are explicitly programmed based on user preferences and domain knowledge. While an important class of recommender systems, we will not be discussing them in detail in this module.

We're primarily going to focus on collaborative filtering in the course of this module.



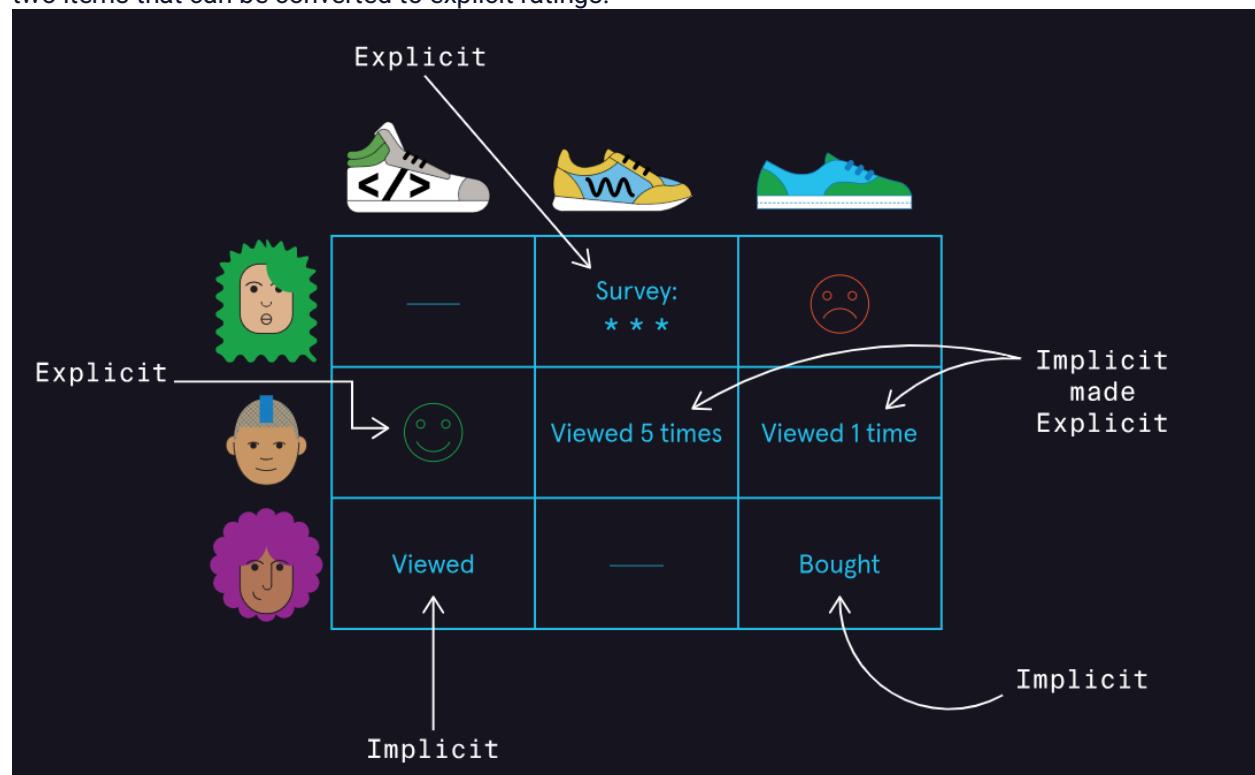
Ratings Matrix: Representing User Preferences

The first step in building a recommender system is to have a mathematical representation of data relating to user's preferences. Often, the representation used is a matrix of numbers called a **ratings matrix**, where each row represents a user, each column represents an item, and the intersection of a row and a column contains the rating for an item given by a user. Ratings inside of a ratings matrix can generally be represented as either **explicit ratings** or **implicit ratings**.

Explicit ratings involve using the ratings given by users for items they have rated. Items that are not rated by users are left blank. Often they are normalized to help model performance. The major downside of this representation is explicit data may be scarce. Often, users skip rating items in an application. Often, rating data is not available at all.

On the other hand, implicit ratings do not require users to submit ratings. Instead, user events on the app or website are viewed as endorsements of an item. For example, purchasing an item on an E-commerce website could be viewed as an endorsement of an item. Any item that a user purchases could then be represented as a 1 in a ratings matrix, and anything they do not purchase can be represented as a 0. The main advantage of implicit ratings is that data is much more readily available. The major downside is the data is not as granular as that of explicit ratings, and therefore recommendations can degrade accordingly.

And sometimes an implicit rating can be converted into an explicit rating according to the data scientist's discretion. For instance, a user may have listened to one song fifty times on an audio streaming service and another one, only five times. There is relative information here as regards the user's preference on two items that can be converted to explicit ratings.



Collaborative Filtering

In the previous exercise, we introduced collaborative filtering. However collaborative filtering can be further classified into two major subclasses: **memory-based methods** (also called **neighborhood-based methods**) and **model-based methods**.

Memory-based methods work through the concept of similarity. Fundamentally, memory-based methods work in one of two ways:

The

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

finds similar users to the target users, and recommends items those similar users liked. This approach is known as **user-user collaborative filtering**.

The algorithm finds similar items to ones the target user liked by measuring the similarity of how users rated items. This approach is known as **item-item** collaborative filtering.

In contrast, model-based methods work by building models that attempt to predict a rating for a user-item pair by using ratings as features. One particular

Preview: Docs Loading link description

[method](#)

that is often used in practice is *matrix factorization*. This method models the user-item ratings matrix as the product of a set of users vector and product vectors. The rating of any user-item pair can then be predicted by multiplying the relevant user vector by the relevant product vector.

After creating a ratings matrix, various data transformations may be performed on the ratings matrix. These transformations are done generally to improve model performance, similar to how normalizing features in a

Preview: Docs Loading link description

[machine learning](#)

model can help improve performance.

One such transformation is ratings normalization. Ratings normalization is a technique where the value of each rating for a given row is adjusted based on the statistical properties of that row. The primary reason this transformation is done is because different users may have different approaches to rating something. For example, some users may give a 5-star rating to any positive experience they have. Other users may be more selective, and only give 5 star ratings very rarely. Ratings normalization provides a way to control for these differences.

Instructions

One of two approaches is usually used for normalizing ratings. The first, mean centering, involves subtracting the mean rating of a row from every value of the row. The other method, z-score normalization, involves using the mean and standard deviation of each row to calculate a z-score for each element of the row. Can you think of other ways to make sure that the ratings between different users and items can be meaningfully compared?

Will E like Shoe 3?

Users		Shoes				
		1	2	3	4	5
A		✓	×	✓	✓	×
B		✓	✓	×	×	✓
C		✓	✓	×	×	✓
D		×	×	✓	✓	×
E		✓	✓	?	×	✓

Users		Shoes				
		1	2	3	4	5
A		3	2	3	4	1
B		4	3	3	1	5
C		3	3	3	2	5
D		1	2	3	3	1
E		4	4	?	1	5

User-User

B & C are most similar to E. They don't like Shoe 3.

A prediction:
E won't like Shoe 3

Item-Item

Shoe 4 is most similar to Shoe 3. E didn't like Shoe 4

A prediction:
E won't like Shoe 3

Matrix Factorization

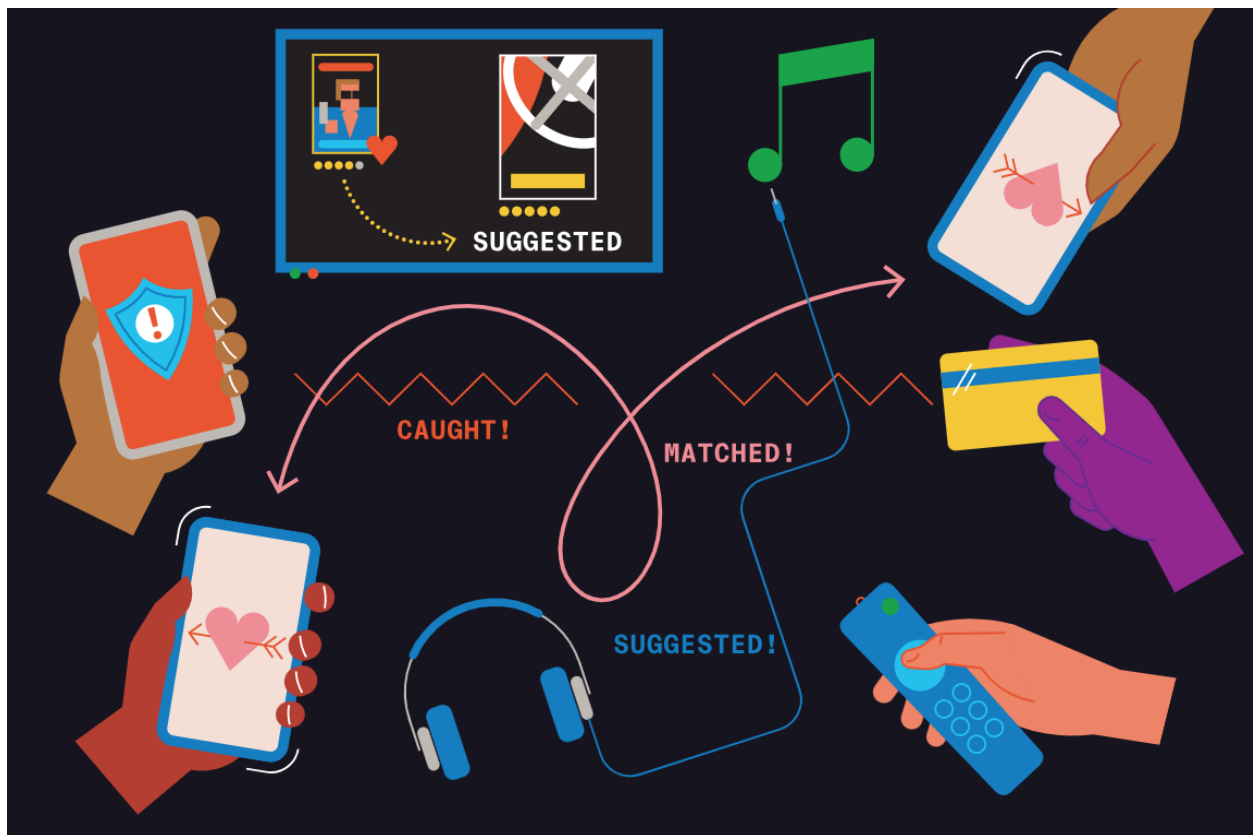
User Vector for E

Product Vector for Shoe 3

A value:
Rating(E,3)

Summary

In this lesson, we learned about recommender systems. We learned about how they compare to traditional supervised learning and what the properties of good recommender systems. We also discussed the different types of recommender systems, as well as how we can transform data for use in recommender systems. We are now ready to learn how we can use Python to implement and build recommender systems.



Simple Python Recommender Engine (Surprise!)

Learn how to build a recommender system using the Python library [surprise](#)!

The engineering and development of a recommender system requires work across data science, software engineering, infrastructure, product, and design. With so many things that need to be executed in order to produce a good recommender system, having tools that make writing and testing the code for recommender systems is critical to success.

In the specific case of machine learning engineering, one group of tools that makes development easier is open-source software libraries. These standardized and well-tested libraries that are shared across the machine learning community help increase the speed of writing code without an explicit need to develop and maintain one's own libraries. If you have worked with [scikit-learn](#), then you are familiar with how this open-source library makes it very easy to build sophisticated machine learning models using only a few lines of code. It would be ideal to have a similar library for recommender systems. Surprise was designed to solve this problem.

What is Surprise?

Surprise (an abbreviation of *Simple Python Recommendation System Engine*) is a `scikit`, a software library built as an add-on to the numerical computation library `SciPy`. Much like how `scikit-learn` makes developing and testing different machine learning models easy, Surprise makes developing and testing various recommender system algorithms easy. Surprise comes with several modules that make it easy to transform data, train recommender systems, and measure recommender system performance. It also comes with a solid base of documentation that makes it easy to understand and explore the library's capabilities.

Building A Simple Recommender System

We're now going to use Surprise to build a basic recommender system. We will be using a classic dataset, the [MovieLens Dataset](#), to build a recommender system that suggests movies to users based on the ratings they gave several movies. The MovieLens dataset is a set of 100,000 movie ratings for 9,000 movies provided by 600 users. The ratings come from the website [movielens.org](#), a non-commercial site dedicated to giving users personalized movie recommendations. This data is very similar to the type of data you would find at large commercial streaming services.

Installation

To install Surprise on your computer, simply use the following pip command:

```
pip install surprise
```

To test that the command worked, open up your terminal or `Jupyter` notebook or IDE and verify the following code runs:

```
import surprise
```

Loading Data

Surprise comes preloaded with datasets for building and testing recommender systems, including the MovieLens dataset. To load the MovieLens dataset, run the following code:

```
from surprise import Dataset

movie_data = Dataset.load_builtin('ml-100k')
```

Train/Test Split of Data

Like other machine learning problems, recommender systems can be modeled as a supervised machine learning problem where the movie ratings of users are being utilized as features to predict ratings for movies users have not watched yet. Viewing recommender systems in this way, we can split the data into training and testing sets to help validate the performance of our recommender system. The code below will split the MovieLens dataset into a 75%/25% train and test set:

```
from surprise.model_selection import train_test_split
trainset, testset = train_test_split(movie_data, test_size=.2,
                                     random_state=42)
```

To understand what the data looks like, let's look at the ratings. To look at the ratings, we will use the `ur` method of `trainset`. The method `ur` returns a dictionary where the keys are user ids, and the values are a list of tuples, where each tuple is in the form `((item_id), {rating})`. For example, to see the ratings of user 590:

```
trainset.ur[590]
```

```
[(699, 4.0),  
(114, 4.0),  
(339, 5.0),  
(769, 5.0),  
(230, 4.0),  
(212, 5.0),  
(21, 5.0),  
(33, 3.0),  
(94, 4.0),  
(74, 2.0),  
(22, 4.0),  
(471, 5.0),  
(8, 3.0),  
(305, 3.0),  
(645, 3.0),  
(558, 2.0),  
(166, 4.0),  
(129, 4.0),  
(100, 5.0),  
(232, 3.0),  
(391, 3.0),  
(204, 3.0),  
(508, 4.0),  
(79, 3.0),  
(83, 4.0),  
(426, 1.0),  
(6, 3.0),  
(58, 3.0),  
(1119, 5.0),  
(76, 5.0)]
```

Training A Recommender System

Now that we have proper datasets for training and testing, we will now train a basic recommender system. Specifically, we will be training a simple user-user collaborative filter using Surprise's `KNNBasic` algorithm. `KNNBasic` is a very simple implementation of user-user *collaborative filtering*. This algorithm works by using the k-nearest Neighbors (KNN) algorithm to determine the similarity of users based on how they rated movies (For more information on the K-nearest neighbors algorithm, check out this [lesson](#)). The average of the movie ratings from the most similar users are then used as the prediction for the target user. The recommender system is initialized and trained like so:

```
from surprise import KNNBasic  
  
movie_recommender = KNNBasic()  
movie_recommender.fit(trainset)
```

The output should look like:

```
Computing the msd similarity matrix...  
Done computing similarity matrix.  
  
<surprise.prediction_algorithms.knns.KNNBasic at 0x10b5c2fa0>
```

Evaluating Recommender System Performance

Finally, we will evaluate the performance of the `KNNBasic` recommender system. First, let's create some predictions on the test set.

```
predictions = movie_recommender.test(testset)
```

The `predictions` are a list of `Prediction` objects. Each `Prediction` object contains a user id (`uid`), an item id representing the movie (`iid`), and actual rating given by the user for that movie (`r_ui`), an estimated rating from the recommender system (`est`) and some additional metadata about each prediction (`details`). An example of a `Prediction` object from our `predictions` can be seen below.

```
predictions[0]
```

Output:

```
Prediction(uid='907', iid='143', r_ui=5.0, est=4.039960584359155,
details={'actual_k': 40, 'was_impossible': False})
```

This `Prediction` object contains a prediction for the movie 'The Sound of Music' (to see which movie each id corresponds to, take a look at this [movie data file](#)). As seen above, the recommender system predicted a rating of about 4 stars, which is not very far from the actual prediction of 5 stars.

We can measure one aspect of the model's performance by looking at the root-mean square error, or RMSE. The RMSE is an average measure of how far off predictions will be from their actual values. The closer the RMSE is to 0, the more accurate the model. The RMSE can also be thought of as a measure of **relevance**, or how likely the items recommended will be something the user would be interested in based on their previous behavior. It is important to note that while relevance is one important aspect of a recommender system, it is not the only important thing to optimize in a recommender system.

As seen below, Surprise makes it very easy to measure model RMSE.

```
from surprise import accuracy
accuracy.rmse(predictions)
```

Output

```
RMSE: 0.9802
0.980150596704479
```

As seen above, the trained recommender system has an RMSE of about 0.98. We can now use this number as a baseline to see if there is a gain or loss in model performance versus other algorithms.

Conclusion

And that's it! With just 10 lines of code, Surprise allowed us to train and test a basic recommender system with ease. Surprise also has several other built-in recommender systems algorithms, including alternative forms of collaborative filtering, Singular Value Decomposition (SVD), and other algorithms that will be explored in the course. Additionally, much like `scikit-learn` models, each of these models have hyperparameters that can be adjusted to help further optimize the performance of recommender systems.

This article has hopefully given you a taste of the power and usefulness of Surprise. While Surprise by itself is not sufficient to build a scalable and well-performing recommender system for large-scale use, it

is an excellent starting point for developing recommender systems, and a powerful addition to a machine learning practitioner's toolkit.

Assessment

Now it's your turn! Using what you have learned, train a simple SVD recommender system using Surprise's `SVD` algorithm (documentation can be found [here](#)) and the provided training data. Then calculate its RMSE. Is this an improvement over the `KNNBasic` algorithm above? How can you tell?

```
from surprise import SVD
from surprise import accuracy

svd_recommender = SVD()
svd_recommender.fit(trainset)

svd_predictions = svd_recommender.test(testset)

accuracy.rmse(svd_predictions)
```

```
RMSE: 0.9407
0.9406680154918938
```

Bayes' Theorem

Introduction to Bayes' Theorem

In this lesson, we'll learn about **Bayes' Theorem**. Bayes' Theorem is the basis of a branch of

[statistics](#)

Preview: Docs Loading link description

called *Bayesian Statistics*, where we take prior knowledge into account before calculating new probabilities.

This allows us to find narrow solutions from a huge universe of possibilities. British mathematician

[Alan Turing](#)

Preview: Docs Loading link description

used it to crack the German Enigma code during WWII. And now it is used in:

Machine Learning

Statistical Modeling

A/B Testing

Robotics

By the end of this lesson, you'll be able to solve simple problems involving prior knowledge.

Instructions

Before we learn this theorem, we'll need to review independence and conditional probability.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Independent Events

The ability to determine whether two events are *independent* is an important skill for

[statistics](#)

Preview: Docs Loading link description

If two events are **independent**, then the occurrence of one event does not affect the probability of the other event. Here are some examples of independent events:

I wear a blue shirt; my coworker wears a blue shirt
I take the subway to work; I eat sushi for lunch
The NY Giants win their football game; the NY Rangers win their hockey game

If two events are **dependent**, then when one event occurs, the probability of the other event occurring changes in a predictable way.

Here are some examples of dependent events:

It rains on Tuesday; I carry an umbrella on Tuesday
I eat spaghetti; I have a red stain on my shirt
I wear sunglasses; I go to the beach

Instructions

Checkpoint 1 Enabled

1.

A certain family plans to have three children. Is the event that the couple's third child is a girl independent of the event that the couple's first two children are girls?

Save your answer ('independent' or 'not independent') to the variable `third_child`.

Bayes' Theorem

Conditional Probability

Conditional probability is the probability that two events happen. It's easiest to calculate conditional probability when the two events are independent.

Note: For the rest of this lesson, we'll be using the statistical convention that the probability of an event is written as $P(\text{event})$.

If the probability of event A is $P(A)$ and the probability of event B is $P(B)$ and the two events are independent, then the probability of both events occurring is the product of the probabilities:

$$P(A \cap B) = P(A) \times P(B)$$

The symbol $\cap$ just means "and", so $P(A \cap B)$ means the probability that both A and B happen.

For instance, suppose we are rolling a pair of dice, and want to know the probability of rolling two sixes.



Each die has six sides, so the probability of rolling a six is $1/6$. Each die is independent (i.e., rolling one six does not increase or decrease our chance of rolling a second six), so:

$$P(6 \cap 6) = P(6) \times P(6) = 1/6 \times 1/6 = 1/36$$

Instructions

Checkpoint 1 Enabled

1.

This week, there is a 30% probability that it will rain on any given day. At a certain high school, gym class is held on three days out of the five day school week.

On a school day, what is the probability that it is raining and the students have gym class?

Save your answer to the variable `p_rain_and_gym`.

```
import numpy as np

p_rain_and_gym = 0.3 * 0.6
```

Bayes' Theorem

Testing for a Rare Disease

Suppose you are a doctor and you need to test if a patient has a certain rare disease. The test is very accurate: it's correct 99% of the time. The disease is very rare: only 1 in 100,000 patients have it.

You administer the test and it comes back positive, so your patient must have the disease, right?

Not necessarily. If we just consider the test, there is only a 1% chance that it is wrong, but we actually have more information: we know how rare the disease is.

Given that the test came back positive, there are two possibilities:

The patient had the disease, and the test correctly diagnosed the disease.

The patient didn't have the disease and the test incorrectly diagnosed that they had the disease.

Instructions

Checkpoint 1 Enabled

1.

What is the probability that the patient had the disease **and** the test correctly diagnosed the disease?

Save your answer to the variable `p_disease_and_correct`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

What is the probability that the patient *does not* have the disease **and** the test incorrectly diagnosed the disease?

Save your answer to the variable `p_no_disease_and_incorrect`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print both `p_disease_and_correct` and `p_no_disease_and_incorrect`.

```
import numpy as np

# P(test correct) = 1 - 0.01
# P(disease) = 1.0 / 100000
p_disease_and_correct = (1 - 0.01) * (1.0 / 100000)

print(p_disease_and_correct)

# P(test incorrect) = 0.01
# P(no disease) = 1 - (1.0 / 100000)
p_no_disease_and_incorrect = 0.01 * (1 - (1.0 / 100000))

print(p_no_disease_and_incorrect)
```

Bayes' Theorem

Bayes' Theorem

In the previous exercise, we determined two probabilities:

The patient had the disease, and the test correctly diagnosed the disease ≈ 0.00001

The patient didn't have the disease and the test incorrectly diagnosed that they had the disease ≈ 0.01

Both events are rare, but we can see that it was about 1,000 times more likely that the test was incorrect than that the patient had this rare disease.

We're able to come to this conclusion because we had more information than just the accuracy of the test; we also knew the prevalence of this disease.

In

[statistics](#)

Preview: Docs Loading link description

, if we have two events (A and B), we write the probability that event A will happen, given that event B already happened as $P(A|B)$. In our example, we want to find $P(\text{rare disease} | \text{positive result})$. In other words, we want to find the probability that the patient has the disease *given* the test came back positive. We can calculate $P(A|B)$ using **Bayes' Theorem**, which states:

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$

So in this case, we'd say:

$$P(\text{rare disease} \mid \text{positive result}) = P(\text{positive result} \mid \text{rare disease}) \cdot P(\text{rare disease}) / P(\text{positive result})$$

$$P(\text{rare disease} \mid \text{positive result}) =$$

$$P(\text{positive result})$$

$$P(\text{positive result} \mid \text{rare disease}) \cdot P(\text{rare disease})$$

It is important to note that on the right side of the equation, we have the term $P(B|A)$. This is the probability that event B will happen given that event A has already happened. This is very different from $P(A|B)$, which is the probability we are trying to solve for. The order matters!

Instructions

Checkpoint 1 Enabled

1.

Calculate $P(\text{positive result} \mid \text{rare disease})$, or the probability of a positive test result, given that a patient really has this rare disease.

Save your answer (as a decimal) to `p_positive_given_disease`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

What is $P(\text{rare disease})$, the probability that a randomly selected patient has the rare disease?

Save your answer to `p_disease`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We now need to compute the denominator; we need to find $P(\text{positive result})$.

As we discussed previously, there are two ways to get a positive result:

The patient had the disease, **and** the test correctly diagnosed the disease.

The patient didn't have the disease **and** the test incorrectly diagnosed that they had the disease.

Using these two probabilities, calculate the total probability that a randomly selected patient receives a positive test result, $P(\text{positive result})$.

Save your answer to the variable `p_positive`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Substitute all three of these values into Bayes' Theorem and calculate $P(\text{rare disease} \mid \text{positive result})$.

Save your result as `p_disease_given_positive`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Print `p_disease_given_positive`.

Is it likely that your patient has this disease?

```
import numpy as np

p_positive_given_disease = (0.99 * (.00001)) / (1./100000.)
print(p_positive_given_disease)

p_disease = 1./100000.
print(p_disease)

p_positive = (0.00001) + (0.01)
print(p_positive)

p_disease_given_positive = (p_positive_given_disease) * (p_disease) / (p_positive)
print(p_disease_given_positive)
```

Bayes' Theorem

Spam Filters

Let's explore a different example. Email spam filters use Bayes' Theorem to determine if certain words indicate that an email is [spam](#).

Let's take a word that often appears in spam: "enhancement".

With just 3 facts, we can make some preliminary steps towards a good spam filter:

"enhancement" appears in just 0.1% of non-spam emails

"enhancement" appears in 5% of spam emails

Spam emails make up about 20% of total emails

Given that an email contains "enhancement", what is the probability that the email is spam?

Instructions

Checkpoint 1 Failed, try again

1.

In this example, we are dealing with two probabilities:

$P(\text{enhancement})$ - the probability that the word "enhancement" appears in an email.

$P(\text{spam})$ - the probability that an email is spam.

Using Bayes' Theorem to answer our question means that we want to calculate $P(A|B)$.

But what are A and B referring to in this case?

Save the string 'spam' to the variable a .

Save the string 'enhancement' to the variable b .

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

What is $P(\text{spam})$?

Save your answer to p_spam .

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

What is $P(\text{enhancement} | \text{spam})$?

Save your answer to $p_enhancement_given_spam$.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We want to know the overall probability that any email (spam or non-spam) contains "enhancement".

Because we know the probability of "enhancement" occurring in both spam (0.05) and non-spam (0.001) emails, we can use a *weighted average* to calculate the probability of "enhancement" occurring in an email:

$P(\text{enhancement}) = P(\text{enhancement} | \text{spam}) \times P(\text{spam}) + P(\text{enhancement} | \text{not spam}) \times P(\text{not spam})$

$P(\text{enhancement}) = P(\text{enhancement} | \text{spam}) \times P(\text{spam}) + P(\text{enhancement} | \text{not spam}) \times P(\text{not spam})$

Save your answer to $p_enhancement$.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Now that we know:

$$\frac{P(\text{spam}) \cdot P(\text{enhancement} \mid \text{spam})}{P(\text{enhancement})}$$

We can plug this into Bayes' Theorem:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

$$P(A \mid B) =$$

$$\frac{P(B \mid A) \cdot P(A)}{P(B)}$$

$$P(B \mid A) \cdot P(A)$$

Save your answer as `p_spam_enhancement`.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Print `p_spam_enhancement`. This is the probability that an email is spam given that it contains the word "enhancement".

Should we block all emails that contain "enhancement"?

How much non-spam email would we block?

```
import numpy as np

a = 'spam'
b = 'enhancement'

p_spam = 0.2
p_enhancement_given_spam = 0.05
p_enhancement = 0.05 * 0.2 + 0.001 * (1 - 0.2)
p_spam_enhancement = p_enhancement_given_spam * p_spam / p_enhancement

print(p_spam_enhancement)
```

BAYES' THEOREM

Review

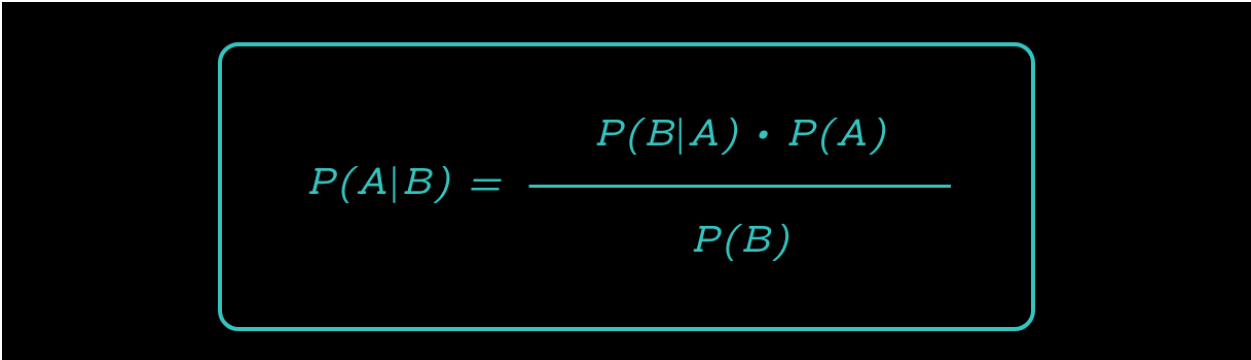
In this course, we learned several new definitions:

- Two events are *independent* if the occurrence of one event does not affect the probability of the second event
- If two events are independent then:

$$P(A \cap B) = P(A) \times P(B)$$

- Bayes' Theorem is the following:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$


$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Naive Bayes Classifier

The Naive Bayes Classifier

A Naive Bayes classifier is a supervised

[machine learning](#)

Preview: Docs Loading link description

[algorithm](#)

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

that leverages Bayes' Theorem to make predictions and classifications. Recall Bayes' Theorem:

$$P(A | B) = P(B | A) \cdot P(A) / P(B)$$

This equation is finding the probability of A given B . This can be turned into a classifier if we replace B with a data point and A with a class. For example, let's say we're trying to classify an email as either spam or not spam. We could calculate $P(\text{spam} | \text{email})$ and $P(\text{not spam} | \text{email})$. Whichever probability is higher will be the classifier's prediction. Naive Bayes classifiers are often used for text classification.

So why is this a supervised machine learning algorithm? In order to compute the probabilities used in Bayes' theorem, we need previous data points. For example, in the spam example, we'll need to compute $P(\text{spam})$. This can be found by looking at a tagged dataset of emails and finding the ratio of spam to non-spam emails.

	Percent Spam	Percent Not Spam
Urgent: limited time offer	0.93	0.07

Naive Bayes Classifier

Investigate the Data

In this lesson, we are going to create a Naive Bayes classifier that can predict whether a review for a product is positive or negative. This type of classifier could be extremely helpful for a company that is curious about the public reaction to a new product. Rather than reading thousands of reviews or tweets about the product, you could feed those documents into the Naive Bayes classifier and instantly find out how many are positive and how many are negative.

The dataset we will be using for this lesson contains Amazon product reviews for baby products. The original dataset contained many different features including the reviewer's name, the date the review was made, and the overall score. We've removed many of those features; the only features that we're interested in are the text of the review and whether the review was "positive" or "negative". We labeled all reviews with a score less than 4 as a negative review.

Note that in the next two lessons, we've only imported a small percentage of the data to help the code run faster. We'll import the full dataset later when we put everything together!

Instructions

Checkpoint 1 Passed

1.

Let's look at the data given to us. Print `pos_list[0]`. Would you classify this review as positive or negative?

Checkpoint 2 Passed

2.

Take a look at the first review in `neg_list` as well. Does that one look negative?

Checkpoint 3 Passed

3.

We've also created a [Counter object](#) for all of the positive reviews and one for all of the negative reviews. These counters are like Python dictionaries — you could find the number of times the word "baby" was used in the positive reviews by printing `pos_counter['baby']`.

Print the number of times the word "crib" was used in the positive and negative reviews. In which set was it used more often?

```
from reviews import neg_list, pos_list, neg_counter, pos_counter

from reviews import neg_list, pos_list, neg_counter, pos_counter

print(pos_list[0])
print()

print(neg_list[0])
print()

print(pos_counter['crib'])
print(neg_counter['crib'])
```

Bayes Theorem I

For the rest of this lesson, we're going to write a classifier that can predict whether the review "This crib was amazing" is a positive or negative review. We want to compute both $P(\text{positive} \mid \text{review})$ and $P(\text{negative} \mid \text{review})$ and find which probability is larger. To do this, we'll be using Bayes' Theorem. Let's look at Bayes' Theorem for $P(\text{positive} \mid \text{review})$.

$$P(\text{positive} \mid \text{review}) = [P(\text{review} \mid \text{positive}) \cdot P(\text{positive})] / P(\text{review})$$

The first part of Bayes' Theorem that we are going to tackle is $P(\text{positive})$. This is the probability that any review is positive. To find this, we need to look at all of our reviews in our dataset - both positive and negative - and find the percentage of reviews that are positive.

We've bolded the part of Bayes' Theorem we're working on.

$$P(\text{positive} \mid \text{review}) = [P(\text{review} \mid \text{positive}) \cdot P(\text{positive})] / P(\text{review})$$

Instructions

Checkpoint 1 Enabled

1.

Find the total number of positive reviews by finding the length of `pos_list`. Do the same for `neg_list`.

Add those two numbers together and save the sum in a variable called `total_reviews`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create variables named `percent_pos` and `percent_neg`. `percent_pos` should be the number of positive reviews divided by `total_reviews`. Do the same for `percent_neg`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print `percent_pos` and `percent_neg`. They should add up to 1!

```
from reviews import neg_list, pos_list, neg_counter, pos_counter

total_reviews = len(pos_list) + len(neg_list)
percent_pos = len(pos_list) / total_reviews
percent_neg = len(neg_list) / total_reviews

print(percent_pos)
print(percent_neg)
```

Naive Bayes Classifier

Bayes Theorem II

Let's continue to try to classify the review "This crib was amazing".

The second part of Bayes' Theorem is a bit more extensive. We now want to compute $P(\text{review} \mid \text{positive})$.

$$P(\text{positive} \mid \text{review}) = [P(\text{review} \mid \text{positive}) \cdot P(\text{positive})] / P(\text{review})$$

In other words, if we assume that the review is positive, what is the probability that the words “This”, “crib”, “was”, and “amazing” are the only words in the review?

To find this, we have to assume that each word is conditionally independent. This means that one word appearing doesn’t affect the probability of another word from showing up. This is a pretty big assumption!

We now have this equation. You can scroll to the right to see the full equation.

$$P(\text{“This crib was amazing”} \mid \text{positive}) = P(\text{“This”} \mid \text{positive}) \cdot P(\text{“crib”} \mid \text{positive}) \cdot P(\text{“was”} \mid \text{positive}) \cdot P(\text{“amazing”} \mid \text{positive})$$

$$P(\text{“This crib was amazing”} \mid \text{positive}) = P(\text{“This”} \mid \text{positive}) \cdot P(\text{“crib”} \mid \text{positive}) \cdot P(\text{“was”} \mid \text{positive}) \cdot P(\text{“amazing”} \mid \text{positive})$$

Let’s break this down even further by looking at one of these terms. $P(\text{“crib”} \mid \text{positive})$ is the probability that the word “crib” appears in a positive review. To find this, we need to count up the total number of times “crib” appeared in our dataset of positive reviews. If we take that number and divide it by the total number of words in our positive review dataset, we will end up with the probability of “crib” appearing in a positive review.

$$P(\text{“crib”} \mid \text{positive}) = \text{\#of “crib” in positive} / \text{\#of words in positive}$$

If we do this for every word in our review and multiply the results together, we have $P(\text{review} \mid \text{positive})$.

Instructions

Checkpoint 1 Enabled

1.

Let’s first find the total number of words in all positive reviews and store that number in a variable named `total_pos`.

To do this, we can use the built-in Python `sum()` function. `sum()` takes a list as a parameter. The list that you want to sum is the `values` of the dictionary `pos_counter`, which you can get by using `pos_counter.values()`.

Do the same for `total_neg`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create two variables named `pos_probability` and `neg_probability`. Each of these variables should start at 1. These are the variables we are going to use to keep track of the probabilities.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create a list of the words in `review` and store it in a variable named `review_words`. You can do this by using Python’s `.split()` function.

For example if the string `test` contained “Hello there”, then `test.split()` would return `["Hello", "there"]`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Loop through every `word` in `review_words`. Find the number of times `word` appears in `pos_counter` and `neg_counter`. Store those values in variables named `word_in_pos` and `word_in_neg`.

In the next steps, we’ll use this variable inside the for loop to do a series of multiplications.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Inside the for loop, set `pos_probability` to be `pos_probability` multiplied by `word_in_pos / total_pos`.

This step is finding each term to be multiplied together. For example, when `word` is `"crib"`, you're calculating the following:

$P(\text{"crib"} \mid \text{positive}) = \frac{\text{\# of "crib" in positive}}{\text{\# of words in positive}}$

$P(\text{"crib"} \mid \text{positive}) =$

$\frac{\text{\# of words in positive}}{\text{\# of "crib" in positive}}$

$\frac{\text{\# of "crib" in positive}}{\text{\# of words in positive}}$

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Do the same multiplication for `neg_probability`.

Outside the for loop, print both `pos_probability` and `neg_probability`. Those values are $P(\text{"This crib was amazing"} \mid \text{positive})$ and $P(\text{"This crib was amazing"} \mid \text{negative})$.

```
from reviews import neg_counter, pos_counter

review = "This crib was amazing"

percent_pos = 0.5
percent_neg = 0.5

total_pos = sum(pos_counter.values())
total_neg = sum(neg_counter.values())

pos_probability = 1
neg_probability = 1

review_words = review.split()

for word in review_words:
    word_in_pos = pos_counter[word]
    word_in_neg = neg_counter[word]

    pos_probability *= word_in_pos / total_pos
    neg_probability *= word_in_neg / total_neg

print(pos_probability)
print(neg_probability)
```

Naive Bayes Classifier

Smoothing

In the last exercise, one of the probabilities that we computed was the following:

$$P(\text{"crib"} \mid \text{positive}) = \frac{\text{\# of "crib" in positive}}{\text{\# of words in positive}}$$

But what happens if `"crib"` was *never* in any of the positive reviews in our dataset? This fraction would then be 0, and since everything is multiplied together, the entire probability $P(\text{review} \mid \text{positive})$ would become 0.

This is especially problematic if there are typos in the review we are trying to classify. If the unclassified review has a typo in it, it is very unlikely that that same exact typo will be in the dataset, and the entire probability will be 0. To solve this problem, we will use a technique called *smoothing*.

In this case, we smooth by adding 1 to the numerator of each probability and N to the denominator of each probability. N is the number of unique words in our review dataset.

For example, $P(\text{"crib"} \mid \text{positive})$ goes from this:

$$P(\text{"crib"} \mid \text{positive}) = \# \text{ of "crib" in positive} / \# \text{ of words in positive}$$

To this:

$$P(\text{"crib"} \mid \text{positive}) = (\# \text{ of "crib" in positive} + 1) / (\# \text{ of words in positive} + N)$$

Instructions

Checkpoint 1 Enabled

1.

Let's demonstrate how these probabilities break if there's a word that never appears in the given datasets.

Change `review` to `"This cribb was amazing"`. Notice the second `b` in `cribb`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Inside your `for` loop, when you multiply `pos_probability` and `neg_probability` by a fraction, add 1 to the numerator.

Make sure to include parentheses around the numerator!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

In the denominator of those fractions, add the number of unique words in the appropriate dataset.

For the positive probability, this should be the length of `pos_counter` which can be found using `len()`.

Again, make sure to put parentheses around your denominator so the division happens after the addition!

Did smoothing fix the problem?

```
from reviews import neg_counter, pos_counter

review = "This cribb was amazing"

percent_pos = 0.5
percent_neg = 0.5

total_pos = sum(pos_counter.values())
total_neg = sum(neg_counter.values())

pos_probability = 1
neg_probability = 1

review_words = review.split()

for word in review_words:
    word_in_pos = pos_counter[word]
    word_in_neg = neg_counter[word]

    pos_probability *= (word_in_pos + 1) / (total_pos + len(pos_counter))
    neg_probability *= (word_in_neg + 1) / (total_neg + len(neg_counter))
```

```
print(pos_probability)
print(neg_probability)
```

Classify

If we look back to Bayes' Theorem, we've now completed both parts of the numerator. We now need to multiply them together.

$$P(\text{positive} \mid \text{review}) = P(\text{review} \mid \text{positive}) \cdot P(\text{positive}) / P(\text{review})$$

Let's now consider the denominator $P(\text{review})$. In our small example, this is the probability that "This", "crib", "was", and "amazing" are the only words in the review. Notice that this is extremely similar to $P(\text{review} \mid \text{positive})$. The only difference is that we don't assume that the review is positive.

However, before we start to compute the denominator, let's think about what our ultimate question is. We want to predict whether the review "This crib was amazing" is a positive or negative review. In other words, we're asking whether $P(\text{positive} \mid \text{review})$ is greater than $P(\text{negative} \mid \text{review})$. If we expand those two probabilities, we end up with the following equations.

$$P(\text{positive} \mid \text{review}) = P(\text{review} \mid \text{positive}) \cdot P(\text{positive}) / P(\text{review})$$

$$P(\text{negative} \mid \text{review}) = P(\text{review} \mid \text{negative}) \cdot P(\text{negative}) / P(\text{review})$$

Notice that $P(\text{review})$ is in the denominator of each. That value will be the same in both cases! Since we're only interested in comparing these two probabilities, there's no reason why we need to divide them by the same value. We can completely ignore the denominator!

Let's see if our review was more likely to be positive or negative!

Instructions

Checkpoint 1 Enabled

1.

After the for loop, multiply `pos_probability` by `percent_pos` and `neg_probability` by `percent_neg`. Store the two values in `final_pos` and `final_neg` and print both.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Compare `final_pos` to `final_neg`:

If `final_pos` was greater than `final_neg`, print "The review is positive".

Otherwise print "The review is negative".

Did our Naive Bayes Classifier get it right for the review "This crib was amazing"?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Replace the review "This crib was amazing" with one that you think should be classified as negative. Run your program again.

Did your classifier correctly classify the new review?

```
from reviews import neg_counter, pos_counter

review = "This crib was amazing"
```

```

percent_pos = 0.5
percent_neg = 0.5

total_pos = sum(pos_counter.values())
total_neg = sum(neg_counter.values())

pos_probability = 1
neg_probability = 1

review_words = review.split()

for word in review_words:
    word_in_pos = pos_counter[word]
    word_in_neg = neg_counter[word]

    pos_probability *= (word_in_pos + 1) / (total_pos + len(pos_counter))
    neg_probability *= (word_in_neg + 1) / (total_neg + len(neg_counter))

```

Formatting the Data for scikit-learn

Congratulations! You've made your own Naive Bayes text classifier. If you have a dataset of text that has been tagged with different classes, you can give your classifier a brand new document and it will predict what class it belongs to.

We're now going to look at how Python's scikit-learn library can do all of that work for us!

In order to use scikit-learn's Naive Bayes classifier, we need to first transform our data into a format that scikit-learn can use. To do so, we're going to use scikit-learn's `CountVectorizer` object.

To begin, we need to create a `CountVectorizer` and teach it the vocabulary of the training set. This is done by calling the `.fit()`

[method](#)

Preview: Docs Loading link description

.

For example, in the code below, we've created a `CountVectorizer` that has been trained on the vocabulary "Training", "review", "one", and "Second".

```

vectorizer = CountVectorizer()

vectorizer.fit(["Training review one", "Second review"])

```

Copy to Clipboard

After fitting the vectorizer, we can now call its `.transform()` method. The `.transform()` method takes a list of strings and will transform those strings into counts of the trained words. Take a look at the code below.

```

counts = vectorizer.transform(["one review two review"])

```

Copy to Clipboard

`counts` now stores the

[array](#)

Preview: Docs Loading link description

`[[1 2 0 0]]`. The word "review" appeared twice, the word "one" appeared once, and neither "Training" nor "Second" appeared at all.

But how did we know that the 2 corresponded to review? You can print `vectorizer.vocabulary_` to see the

[index](#)

Preview: Docs Loading link description

that each word corresponds to. It might look something like this:

```
{'training': 3, 'review': 1, 'one': 0, 'second': 2}
```

Copy to Clipboard

Finally, notice that even though the word "two" was in our new review, there wasn't an index for it in the vocabulary. This is because "two" wasn't in any of the strings used in the `.fit()` method.

We can now use `counts` as input to our Naive Bayes Classifier.

Note that in the code in the editor, we've imported only a small percentage of our review dataset to make load times faster. We'll import the full dataset later when we put all of the pieces together!

Instructions

Checkpoint 1 Enabled

1.

Create a `CountVectorizer` and name it `counter`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Call `counter's .fit()` method. `.fit()` takes a list of strings and it will learn the vocabulary of those strings. We want our counter to learn the vocabulary from both `neg_list` and `pos_list`.

Call `.fit()` using `neg_list + pos_list` as a parameter.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print `counter.vocabulary_`. This is the vocabulary that your counter just learned. The numbers associated with each word are the indices of each word when you `transform` a review.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Let's transform our brand new review. Create a variable named `review_counts` and set it equal to `counter's .transform()` function. Remember, `.transform()` takes a *list* of strings to transform. So call `.transform()` using `[review]` as a parameter.

Print `review_counts.toarray()`. If you don't include the `toarray()`, `review_counts` won't print in a readable format.

It looks like this is an array of all 0s, but the indices that correspond to the words "this", "crib", "was", and "amazing" should all be 1.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

We'll use `review_counts` as the test point for our Naive Bayes Classifier, but we also need to transform our training set.

Our training set is `neg_list + pos_list`. Call `.transform()` using that as a parameter. Store the results in a variable named `training_counts`. We'll use these variables in the next exercise.

```
from reviews import neg_list, pos_list
from sklearn.feature_extraction.text import CountVectorizer

review = "This crib was amazing"

counter = CountVectorizer()
counter.fit(neg_list + pos_list)
```

```
print(counter.vocabulary_)

review_counts = counter.transform([review])
print(review_counts.toarray())

training_counts = counter.transform(neg_list + pos_list)
```

Using scikit-learn

Now that we've formatted our data correctly, we can use it using scikit-learn's `MultinomialNB` classifier.

This classifier can be trained using the `.fit()`

[method](#)

Preview: Docs Loading link description

`.fit()` takes two parameters: The

[array](#)

Preview: Docs Loading link description

of data points (which we just made) and an array of labels corresponding to each data point.

Finally, once the model has been trained, we can use the `.predict()` method to predict the labels of new points. `.predict()` takes a list of points that you want to classify and it returns the predicted labels of those points.

Finally, `.predict_proba()` will return the probability of each label given a point. Instead of just returning whether the review was good or bad, it will return the likelihood of a good or bad review.

Note that in the

[code editor](#)

Preview: Docs A code editor is a program designed for writing software and utilizing human-readable text to make code easier to parse and understand.

, we've imported some of the

[variables](#)

Preview: Docs Loading link description

you created last time. Specifically, we've imported the `counter` object, `training_counts` and then make `review_counts`. This means the program won't have to re-create those variables and should help the

[runtime](#)

Preview: Docs Runtime is the period of time during which a program is running.
of your program.

Instructions

Checkpoint 1 Enabled

1.

Begin by making a `MultinomialNB` object called `classifier`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We now want to fit the classifier. We have the transformed points (found in `training_counts`), but we don't have the labels associated with those points.

We made the training points by combining `neg_list` and `pos_list`. So the first half of the labels should be `0` (for negative) and the second half should be `1` (for positive).

Create a list named `training_labels` that has 1000 `0`s followed by 1000 `1`s.

Note that there are 1000 negative and 1000 positive reviews. Normally you could find this out by asking for the length of your dataset — in this example, we haven't included the dataset because it takes so long to load!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Call `classifier's .fit()` function. Fit takes two parameters: the training set and the training labels.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Call `classifier's .predict()` method and print the results. This method takes a list of the points that you want to test.

Was your review classified as a positive or negative review?

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

After printing `predict`, print a call to the `predict_proba` method. The parameter to `predict_proba` should be the same as `predict`.

The first number printed is the probability that the review was a `0` (bad) and the second number is the probability the review was a `1` (good).

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Change the text `review` to see the probabilities change.

Can you create a review that the algorithm is *really* confident about being positive?

The review "This crib was great amazing and wonderful" had the following probabilities:

```
[[ 0.04977729 0.95022271]]
```

Can you create a review that is even *more* positive?

Another interesting challenge is to create a clearly negative review that our classifier thinks is positive.

```
from reviews import counter, training_counts
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

review = "This crib was amazing"
review_counts = counter.transform([review])
```

Review

In this lesson, you've learned how to leverage Bayes' Theorem to create a supervised

[machine learning](#)

Preview: Docs Loading link description

[algorithm](#)

Preview: Docs Loading link description

. Here are some of the major takeaways from the lesson:

- A tagged dataset is necessary to calculate the probabilities used in Bayes' Theorem.
- In this example, the features of our dataset are the words used in a product review. In order to apply Bayes' Theorem, we assume that these features are independent.

- Using Bayes' Theorem, we can find $P(\text{class}|\text{data point})$ for every possible class. In this example, there were two classes — positive and negative. The class with the highest probability will be the algorithm's prediction.

Even though our algorithm is running smoothly, there's always more that we can add to try to improve performance. The following techniques are focused on ways in which we process data before feeding it into the Naive Bayes classifier:

- Remove punctuation from the training set. Right now in our dataset, there are 702 instances of "great!" and 2322 instances of "great.". We should probably combine those into 3024 instances of "great".
- Lowercase every word in the training set. We do this for the same reason why we remove punctuation. We want "Great" and "great" to be the same.
- Use a bigram or trigram model. Right now, the features of a review are individual words. For example, the features of the point "This crib is great" are "This", "crib", "is", and "great". If we used a bigram model, the features would be "This crib", "crib is", and "is great". Using a bigram model makes the assumption of independence more reasonable.

These three improvements would all be considered part of the field *Natural Language Processing*.

You can find the baby product review dataset, along with many others, on [Dr. Julian McAuley's website](#).

Instructions

Checkpoint 1 Passed

1.

In the code editor, we've included three Naive Bayes classifiers that have been trained on different datasets. The training sets used are the baby product reviews, reviews for Amazon Instant Videos, and reviews about video games.

Try changing `review` again and see how the different classifiers react!

```
from reviews import baby_counter, baby_training, instant_video_counter,
instant_video_training, video_game_counter, video_game_training
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

review = "this game was violent"

baby_review_counts = baby_counter.transform([review])
instant_video_review_counts = instant_video_counter.transform([review])
video_game_review_counts = video_game_counter.transform([review])

baby_classifier = MultinomialNB()
instant_video_classifier = MultinomialNB()
video_game_classifier = MultinomialNB()

baby_labels = [0] * 1000 + [1] * 1000
instant_video_labels = [0] * 1000 + [1] * 1000
video_game_labels = [0] * 1000 + [1] * 1000

baby_classifier.fit(baby_training, baby_labels)
instant_video_classifier.fit(instant_video_training, instant_video_labels)
video_game_classifier.fit(video_game_training, video_game_labels)

print("Baby training set: " + str(baby_classifier.predict_proba(baby_review_counts)))
print("Amazon Instant Video training set: " +
      str(instant_video_classifier.predict_proba(instant_video_review_counts)))
print("Video Games training set: " +
      str(video_game_classifier.predict_proba(video_game_review_counts)))
```




NEURAL NETWORK
Deep Learning

NEURAL NETWORK

Deep Learning

What Is Deep Learning?

A quick overview of deep learning and its applications

What is Deep Learning?

Have you ever wondered what powers ChatGPT? What technology developers are using to create self-driving cars? How your phone can recognize faces? Why it seems like your photos app is better at recognizing your friends' faces than even you can? What is behind all of this? Is it magic? Well, not exactly; it is a powerful technology called *deep learning* (DL). Let's dive in and see what this is all about!



Deep Learning vs. Machine Learning

First, let's focus on *learning*. If you have come across [*machine learning*](#) before, you might be familiar with the concept of a *learning model*. Learning describes the process by which models analyze data and finds patterns. A machine learning algorithm learns from patterns to find the best representation of this data, which it then uses to make predictions about new data that it has never seen before.

Deep learning is a subfield of machine learning, and the concept of *learning* is pretty much the same.

Deep learning models are used with many different types of data, such as text, images, audio, and more, making them applicable to many different domains.

What does “deep” mean?

That leaves the question: what is this “deep” aspect of deep learning? It separates deep learning from typical machine learning models and why it is a powerful tool that is becoming more prevalent in today’s society.

The deep part of deep learning refers to the numerous “layers” that transform data. This architecture mimics the structure of the brain, where each successive layer attempts to learn progressively complex patterns from the data fed into the model. This may seem a bit abstract, so let’s look at a concrete example, such as facial recognition. With facial recognition, a deep learning model takes in a photo as an input, and numerous layers perform specific steps to identify whose face is in the picture. The steps taken by each layer might be the following:

Find the face within the image using edge detection.
Analyze the facial features (eyes, nose, mouth, etc.).
Compare against faces within a repository.
Output a prediction!



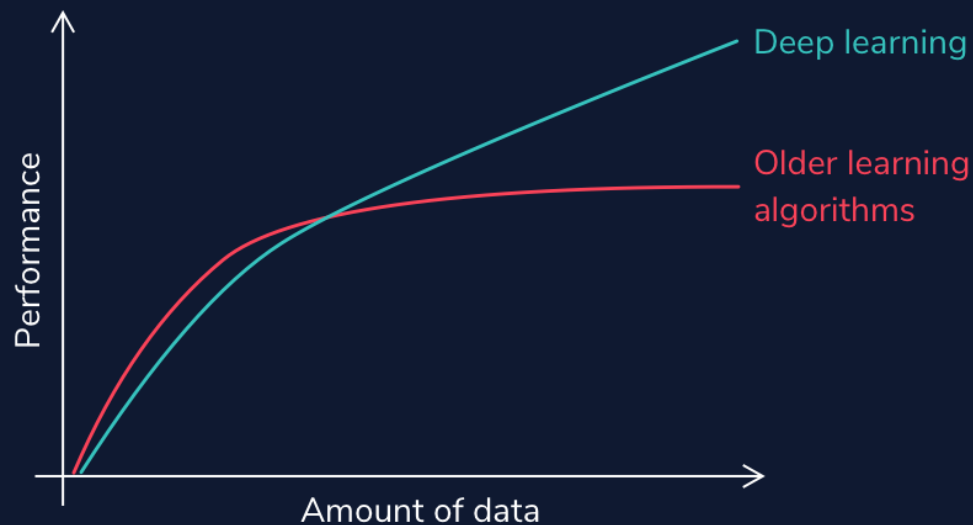
1. Find face in the image using edge detection
2. Analyze facial features
3. Compare against known faces
4. Make a prediction

This structure of many abstract layers makes deep learning incredibly powerful. Feeding high volumes of data into the model makes the connections between layers more intricate. Deep learning models tend to perform better with more massive amounts of data than other learning algorithms.

High Volume of Data

Notice that without large amounts of data, deep learning models are no more powerful (and maybe even less accurate) than less complex learning models. However, with large amounts of data, deep learning

models can improve performance to the point that they outperform humans in tasks such as classifying objects or faces in images or driving. Deep learning is fundamentally a future learning system (also known as representation learning). It learns from raw data without human intervention. Hence, given massive amounts of data, a deep learning system will perform better than traditional machine learning systems that rely on feature extractions from developers.



Autonomous vehicles, such as Tesla, have to process thousands, even millions of stop signs, to understand that cars are supposed to stop when they see a red hexagon with “Stop” written on them (and this is only for U.S. stop signs!). Think about the enormous number of situations a self-driving vehicle must train on to ensure safety.

With Deep Learning Comes Deep Responsibility

Even beyond identifying objects, deep learning models can generate audio and visual content that is deceptively real. They can modify existing images, such as in [this cool applet](#) that allows you to add in trees or alter buildings in a set of photos. However, they can have a darker side. DL models can produce artificial media in which the identity of someone in an image, video, or audio is replaced with someone else. These are known as [deepfakes](#), and they can have scary implications, such as [financial fraud](#) and the distribution of fake news and hoaxes.



Graphics Processing Units

One final thing to note about deep learning is that with large amounts of data and layers of complexity, you may imagine that this takes a lot of time and processing power. That intuition would be correct. These models often require high-performance *GPUs* (graphics processing units) to run in a reasonable amount of time. These processors have a large memory bandwidth and can process multiple computations simultaneously. CPUs can run deep learning models as well; however, they will be much slower.

The development of GPUs has been critical to the success of deep learning. It is interesting to note that one of the driving factors for this development was not the need for better deep learning tools, but the [demand for better video game graphics](#). It just so happened that GPUs are perfect for processing large datasets. This makes them a perfect tool for learning models and has put deep learning specifically at the forefront of machine learning conversations.

We have only just begun our dive into deep learning. Let's keep digging in and see where our journey takes us!

Deep Learning Math

Introduction

Before we dive into creating our deep learning models, let's take a step back and unbox the mechanisms of these models. In this lesson, we will investigate the foundations that run through the inner workings of neural networks. Hopefully, after reading about the steps our data takes on its deep learning journey, you will have a clearer picture of the overall process and feel ready to get your hands on some code!

We are not going to assume that you have a deep understanding of linear algebra, and you will not need a high-level math background to follow along.

Instructions

Let's take a look at the cartoon in the learning environment. Deep learning is commonly viewed as a jumbled mess of linear algebra which this cartoon pokes fun at.

In this lesson, we hope to clarify the fundamental concepts that make deep learning possible, so you can build your own learning models with more precision than "pour the data into this big pile of linear algebra."

Source: [xkcd](#)

THIS IS YOUR MACHINE LEARNING SYSTEM?

YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.

WHAT IF THE ANSWERS ARE WRONG?

JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.



Scalars, Vectors, and Matrices

To start, let us go over a couple of topics that will be integral to understanding the mathematical operations that are present in deep learning, including how data is represented:

Scalars: A scalar is a single quantity that you can think of as a number. In machine learning models, we can use scalar quantities to manipulate data, and we often modify them to improve our model's accuracy. We can also represent data as scalar values depending on what dataset we are working with.

Code example:

```
x = 5
```

Copy to Clipboard

Vectors: Vectors are arrays of numbers. In Python, we often denote vectors as NumPy arrays. Each value in the array can be identified by its index (location within the array).

Code example:

```
x = np.array([1,2,3])
```

Copy to Clipboard

Matrices: Matrices are grids of information with rows and columns. We can index a matrix just like an array; however, when indexing on a matrix, we need two arguments: one for the row and one for the column.

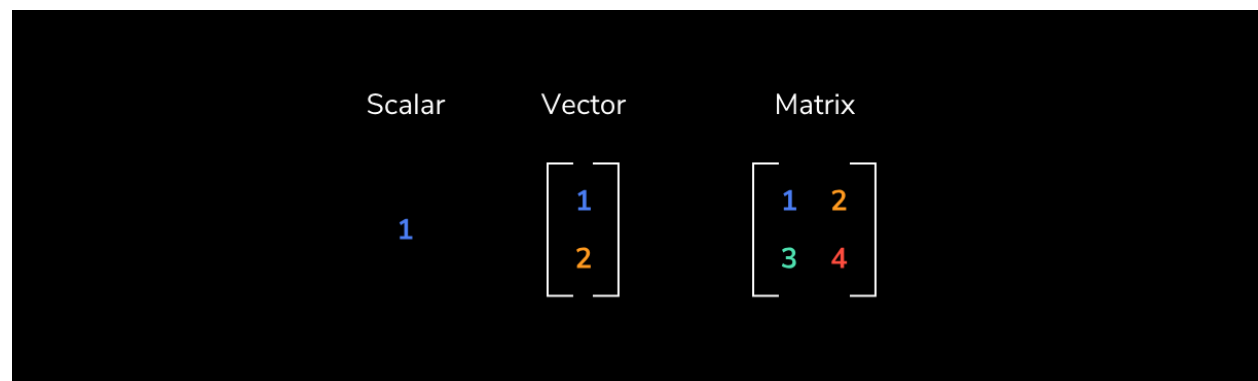
Code example:

```
x = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

Copy to Clipboard

Instructions

Take a look at the visual example of each of these objects.



Tensors

Scalars, *vectors*, and *matrices* are foundational objects in linear algebra. Understanding the different ways they interact with each other and can be manipulated through matrix algebra is integral before diving into deep learning. This is because the data structure we use in deep learning is called a *tensor*, which is a generalized form of a vector and matrix: a multidimensional

[array](#)

Preview: Docs Stores elements of various data types in an ordered collection.

A tensor allows for more flexibility with the type of data you are using and how you can manipulate that data.

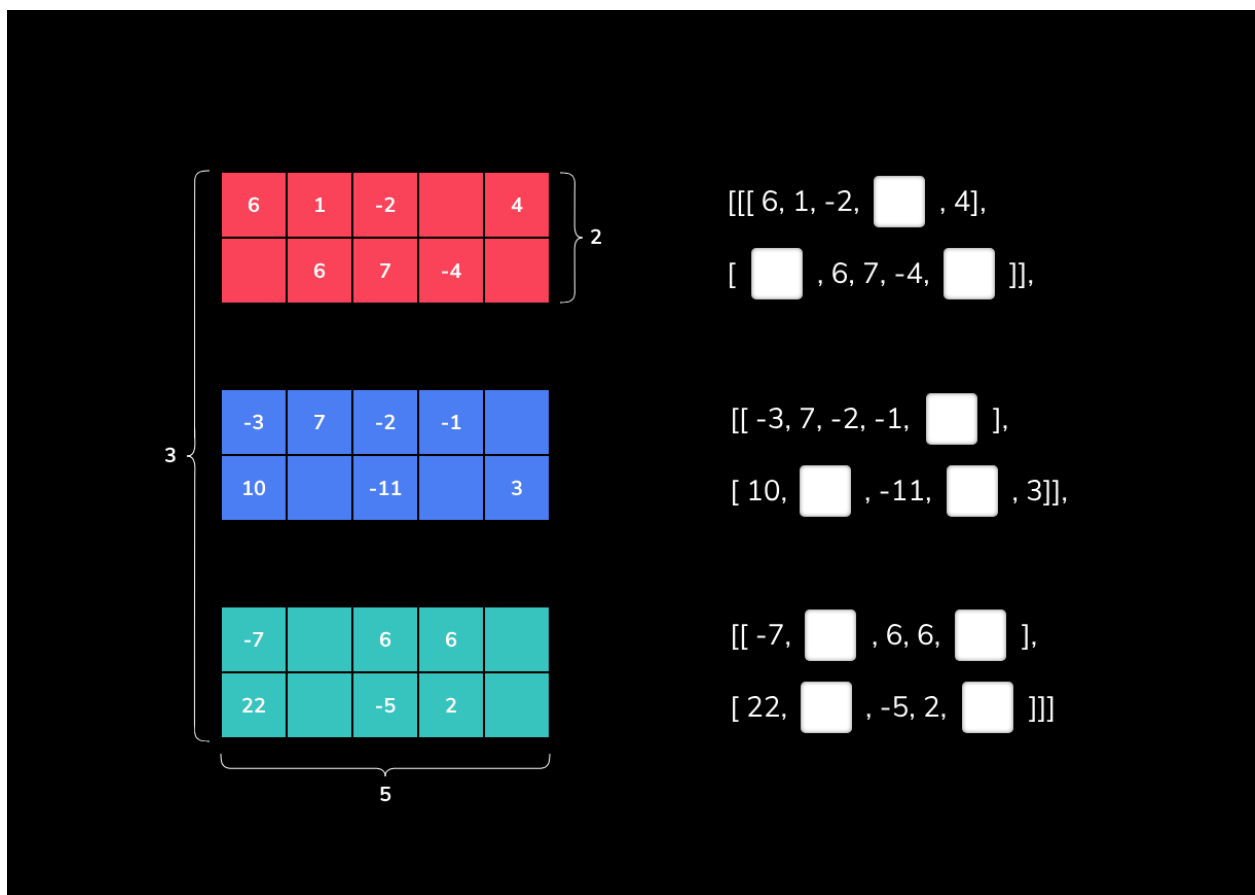
Instructions

Let us visualize what they look like and how they can be expressed using NumPy arrays. Use the applet on this page to get a feel for a tensor's structure and how it is represented!

In this applet, you are given boxes where you can fill in negative and positive whole numbers. On the right side is a set of nested arrays. These nested arrays form a 3-dimensional tensor that can be viewed as a "stack" or "layer" of grids, as shown on the left.

The *shape* of this tensor is (3, 2, 5), as outlined on the diagram. The shape of our data is an important factor when we are feeding it into our neural network. It affects the way our model interacts with our inputs. This is something you will see in future lessons!

If you would like to read more about tensors and see more ways to visualize tensors, feel free to do so [here](#)!



Matrix Algebra

The following gifs walkthrough matrix multiplication, addition, and transpose. You can perform element-wise operations on tensors using matrix algebra as well, which you can read more about [here](#).

$$\begin{bmatrix} 3 & 8 \\ 4 & 6 \end{bmatrix} + \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 7 & 8 \\ 5 & -3 \end{bmatrix}$$

Diagram illustrating Matrix Addition. The operation shows two 2x2 matrices being added element-wise. The first matrix has elements 3, 8, 4, 6. The second matrix has elements 4, 0, 1, -9. The resulting matrix has elements 7, 8, 5, -3. Red arrows and circles highlight the addition of the top-left elements: $3 + 4 = 7$.

Matrix Addition:

$$2 \times \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 8 & 0 \\ 2 & -18 \end{bmatrix}$$

Diagram illustrating Scalar Multiplication. A scalar value 2 is multiplied by a 2x2 matrix. The matrix has elements 4, 0, 1, -9. The resulting matrix has elements 8, 0, 2, -18. Red arrows and circles highlight the multiplication of the top-left element: $2 \times 4 = 8$.

Scalar Multiplication:

Matrix Multiplication:

This is the most complicated, so spend time understanding how it is done in the applet in the Learning Environment. Feel free to pause the animation if you would like to spend more time making sense of each slide!

Transposing a Matrix turns rows into columns

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}^T = \begin{bmatrix} 6 & 1 \\ 4 & -9 \\ 24 & 8 \end{bmatrix}$$

Transpose:

This is all of the matrix algebra we need to proceed with the rest of our deep learning investigation! These concepts are the fundamental building blocks of why deep learning models are so powerful. When we are training our models, we are performing operations on tensors. This data is analyzed, manipulated, and shaped by the matrix algebra we have quickly gone over.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

$$(4, 5, 6) \cdot (7, 9, 11) = 4 \times 7 + 5 \times 9 + 6 \times 11 = 139$$

Neural Networks

Concept Overview

Let's take a look at the journey our *inputs* take inside of a neural network! By an input, we mean a data point from our dataset. Our input can have many different features, so in our *input layer*, each node represents a different input feature. For example, if we were working with a dataset of different types of food, some of our features might be size, shape, nutrition, etc., where the value for each of these features would be held in an input node.

Besides an input layer, our neural network has two other different types of layers:

Hidden layers are layers that come between the input layer and the output layer. They introduce complexity into our neural network and help with the learning process. You can have as many hidden layers as you want in a neural network (including zero of them).

The *output layer* is the final layer in our neural network. It produces the final result, so every neural network must have only one output layer.

Each layer in a neural network contains nodes. Nodes between each layer are connected by *weights*. These are the learning parameters of our neural network, determining the strength of the connection between each linked node.

The weighted sum between nodes and weights is calculated between each layer. For example, from our input layer, we take the weighted sum of the inputs and our weights with the following equation:

$$\text{weighted_sum} = (\text{inputs} \cdot \text{weight_transpose}) + \text{bias}$$

$$\text{weighted_sum} = (\text{inputs} \cdot \text{weight_transpose}) + \text{bias}$$

We then apply an *activation function* to it.

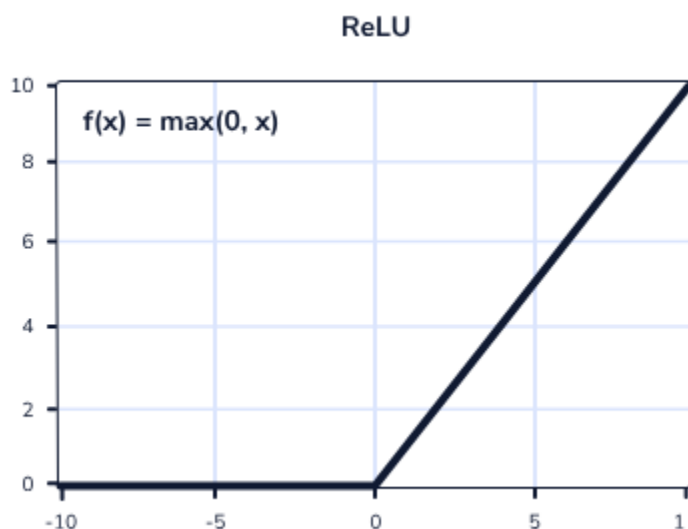
$$\text{Activation}(\text{weighted_sum})$$

$$\text{Activation}(\text{weighted_sum})$$

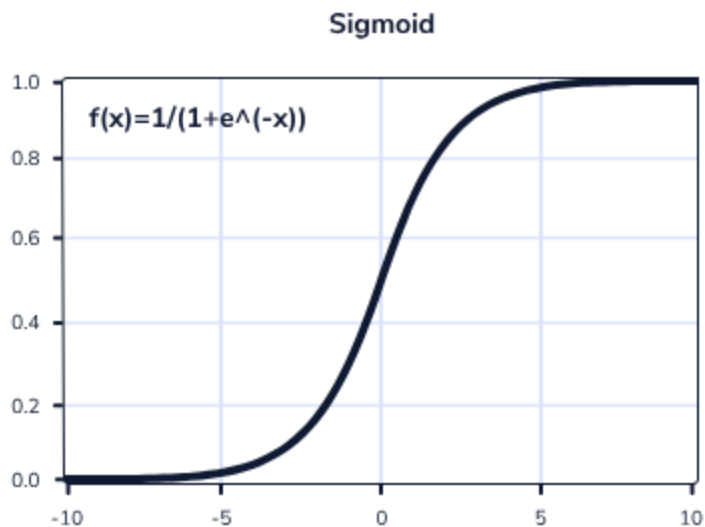
The two formulas we have gone over take all the inputs through one layer of a neural network. Aside from the activation function, all of the transformations we have done so far are linear. Activation functions introduce nonlinearity in our learning model, creating more complexity during the learning process.

This is what makes activation functions important. A neural network with many hidden layers but no activation functions would just be a series of successive layers that would be no more effective or accurate than simple linear regression.

An activation function decides what is fired to the next neuron based on its calculation for the weighted sums. Various types of activation functions can be applied at each layer. The most popular one for hidden layers is *ReLU*.



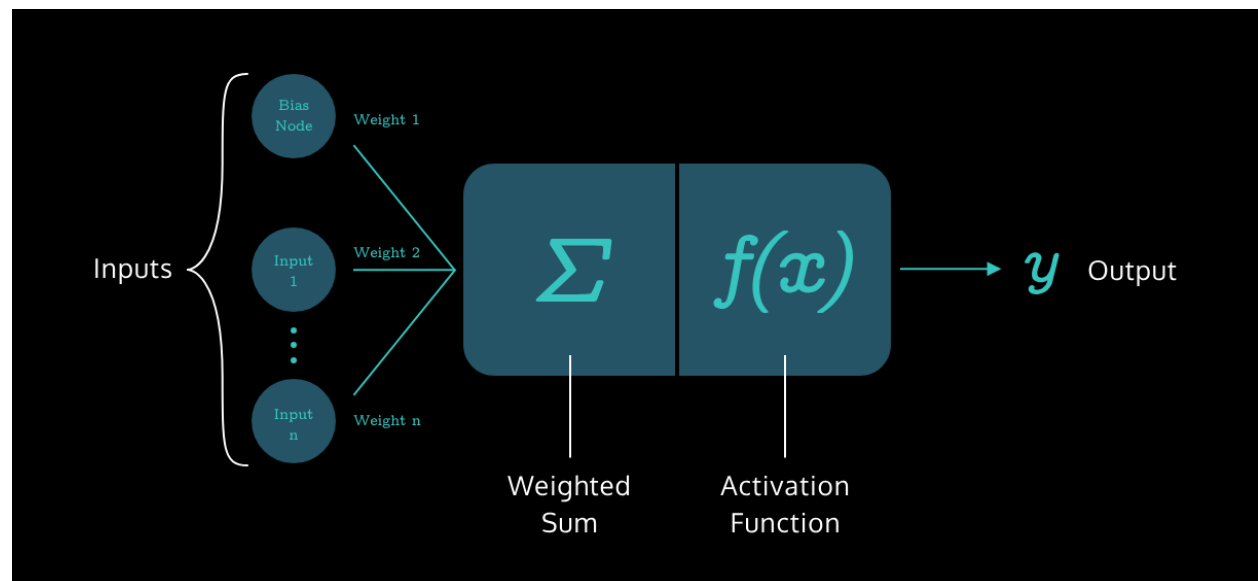
Others commonly used, often for the output layer, are *sigmoid* and *softmax*. You will learn more about these functions as you use them later in this course.



Instructions

In this diagram, we see a basic neural network with no hidden layers. Use your mouse to hover over each section of the image to get a feel for how each step of the neural network works. Each part of the diagram contains a description to indicate its role in a neural network.

(Note: on some browsers, you may need to scroll down to see the descriptions)



The Math Behind the Journey!

Let's bring all of these concepts together and see how they function in a neural network with one hidden layer. As you scroll over each section, you will see the inputs/weights/calculations associated with it and see how inputs get from the starting point and make their way to the end!

The process we have been going through is known as *forward propagation*. Inputs are moved forward from the input layer through the hidden layer(s) until they reach the output layer.

Instructions

In this applet, you can scroll over each part of the neural network and observe the mathematics behind the diagram.

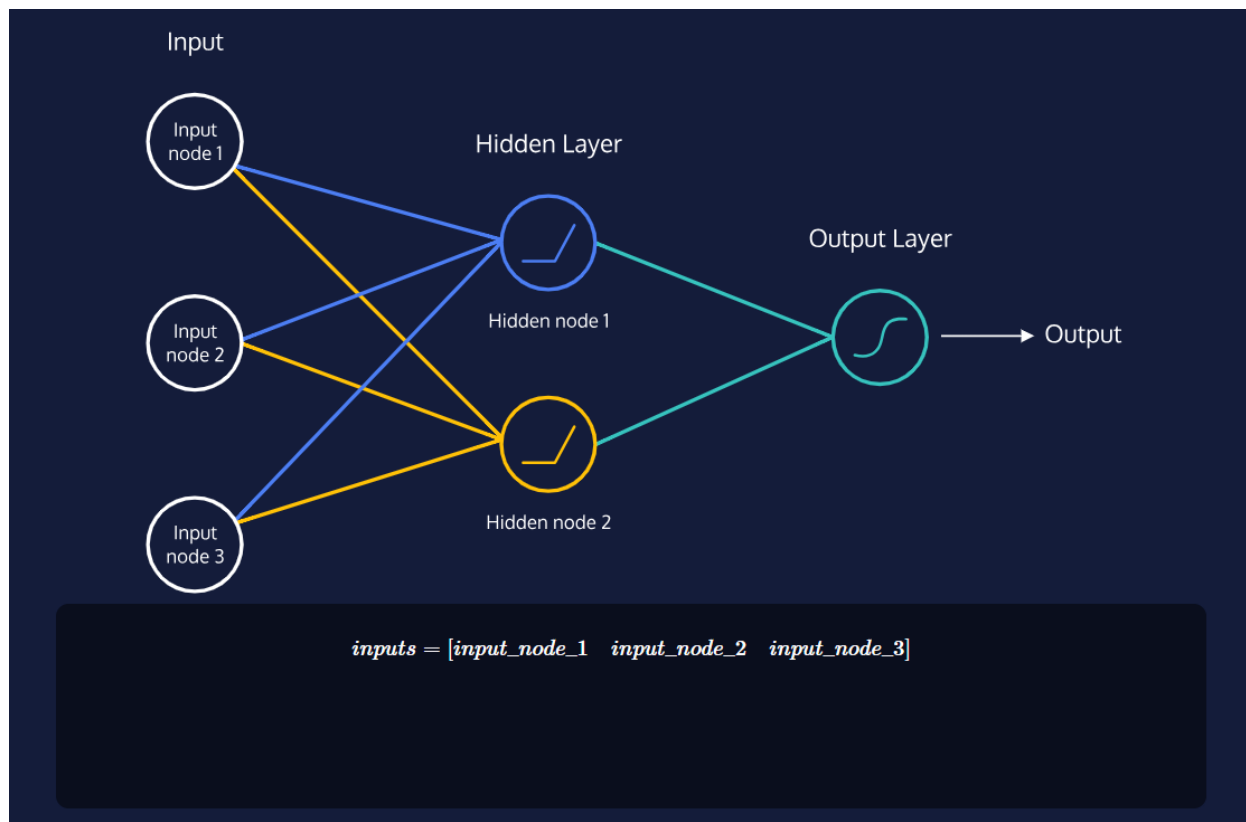
When you scroll over the input section, you should see how the input is represented as a vector. Scrolling over the weights, we see how each set of weights (blue and yellow) is represented as a vector. When brought together, they make up the `weights_matrix` and `weights_matrix.transpose`.

When scrolling through the hidden nodes sections, you will notice that there are two parts. In the first step, we take the weighted sum of our data using the `weights_matrix.transpose`. From this, we end up with a vector and apply our ReLU activation function to it.

This takes us to our teal weights. These are represented as a vector. The `weights_teal.transpose` turns our row vector into a column vector. Then we take another weighted sum in our output layer, this time between our `hidden_nodes` and our `weights_teal.transpose`. Following this, we have a sigmoid activation function, which gives us our output.

Feel free to open this [applet](#) in a separate window for a larger viewing screen.

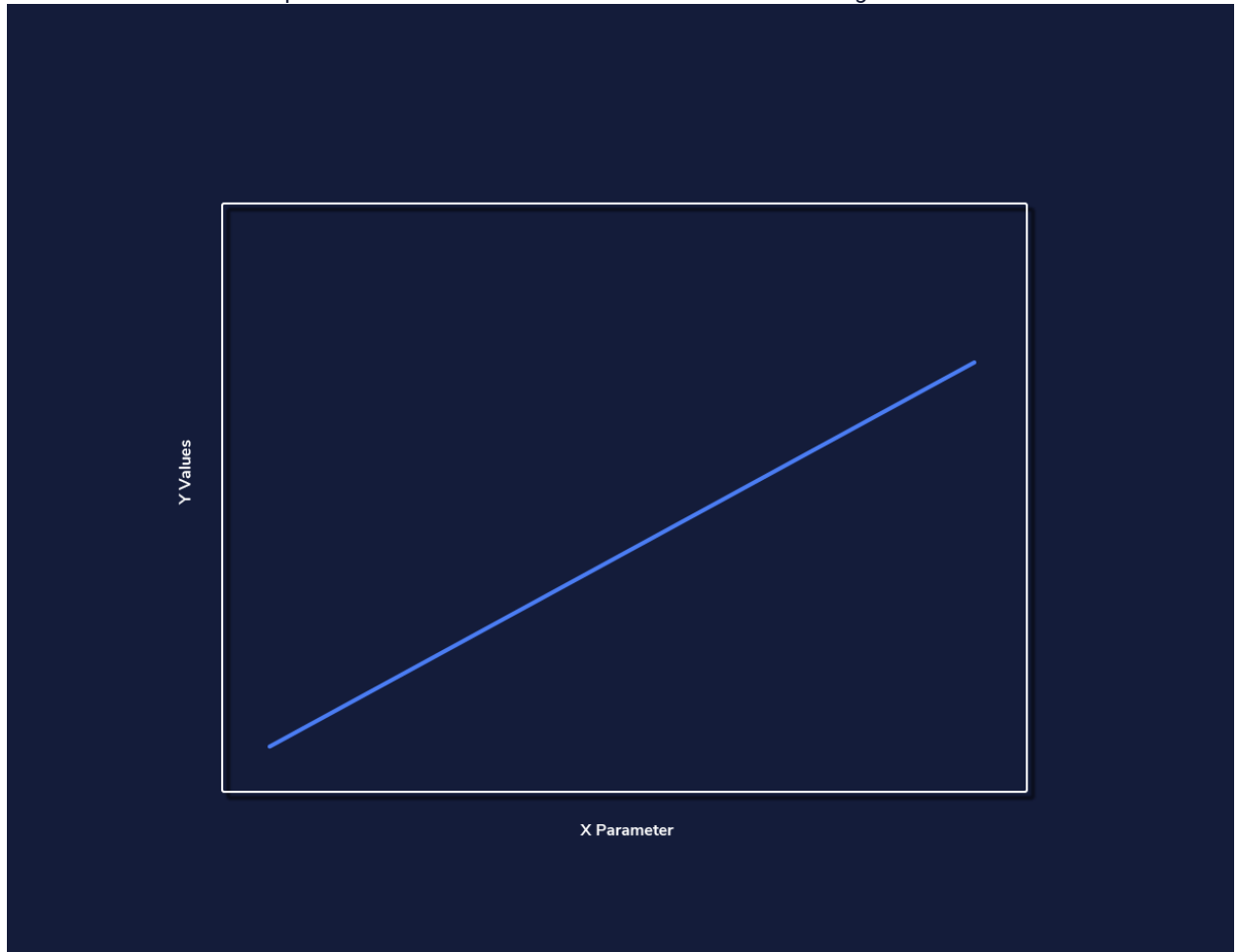
We now understand the adventure our data takes on one journey through our neural network. We are not quite finished yet, though. Let's keep exploring!



Loss Functions

We have seen how we get to an output! Now, what do we do with it? When a value is outputted, we calculate its error using a loss function. Our predicted values are compared with the actual values within the training data. There are two commonly used loss calculation formulas:

Mean squared error, which is most likely familiar to you if you have come across linear regression. This gif below shows how mean squared error is calculated for a *line of best fit* in linear regression.



Click [here](#) to view the gif in a full-sized page.

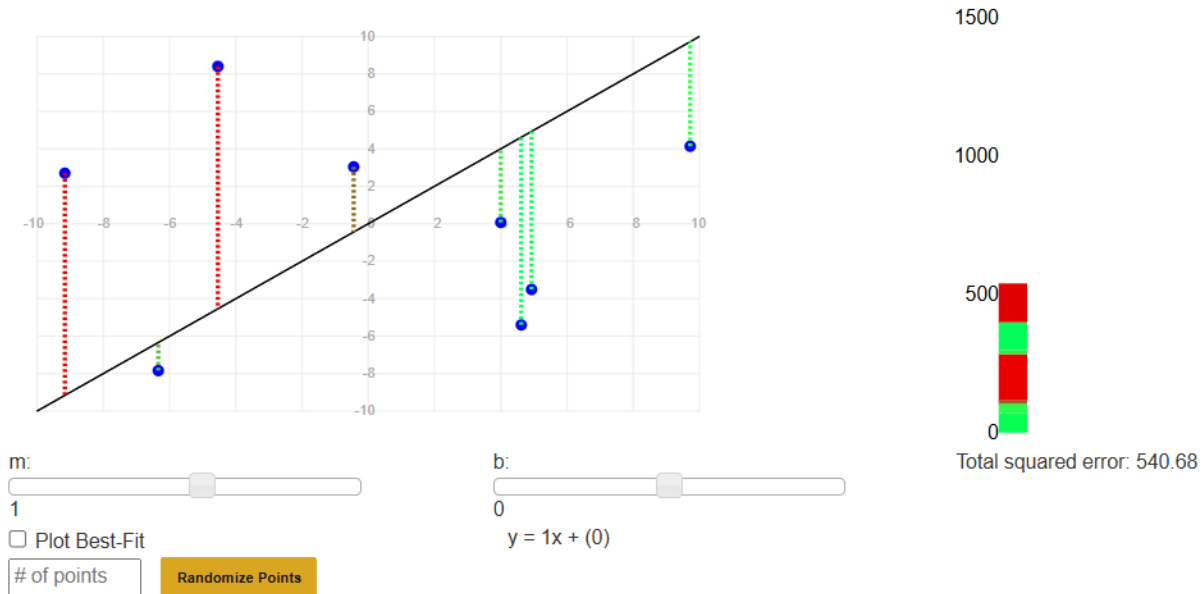
Cross-entropy loss, which is used for classification learning models rather than regression.

You will learn more about this as you use loss functions in your deep learning models.

Instructions

1. The interactive visualization in the browser lets you try to find the line of best fit for a random set of data points:
 - The slider on the left controls the m (slope)
 - The slider on the right controls the b (intercept)
 - You can see the total squared error on the right side of the visualization. To get the line of best fit, we want this loss to be as small as possible.
2. To check if you got the best line, check the "Plot Best-Fit" box.
Randomize a new set of points and try to fit a new line by entering the number of points you want (try 8!) in the textbox and pressing Randomize Points.
NOTE: This visualization works best with fewer than 50 points but will continue to work up to 1000 points. Attempting to add more than 1000 points may result in significant lag or the applet crashing.
Play around with the interactive applet, and notice what method you use to minimize loss:

Do you first get the slope to where it produces lowest loss, and then move the intercept to where it produces lowest loss?
 Do you create a rough idea in your mind where the line should be first, and then enter the parameters to match that image?



Backpropagation

This all seems fine and dandy so far. However, what if our output values are inaccurate? Do we cry? Try harder next time? Well, we can do that, but the good news is that there is more to our deep learning models.

This is where *backpropagation* and *gradient descent* come into play. Forward propagation deals with feeding the input values through hidden layers to the final output layer. Backpropagation refers to the computation of gradients with an

[algorithm](#)

Preview: Docs Loading link description

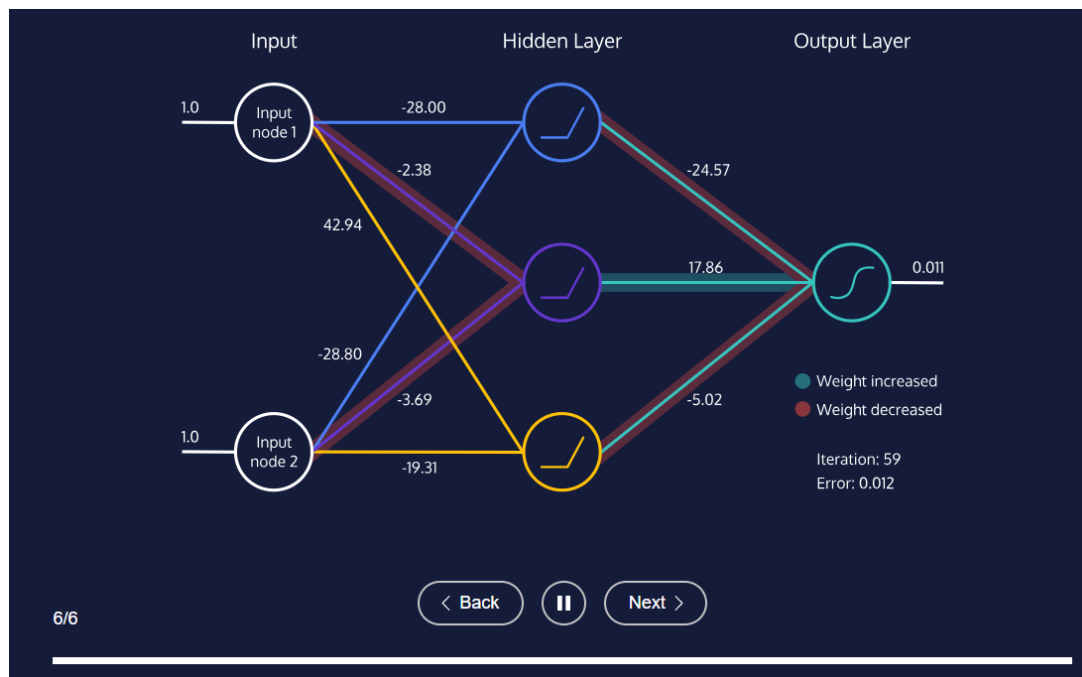
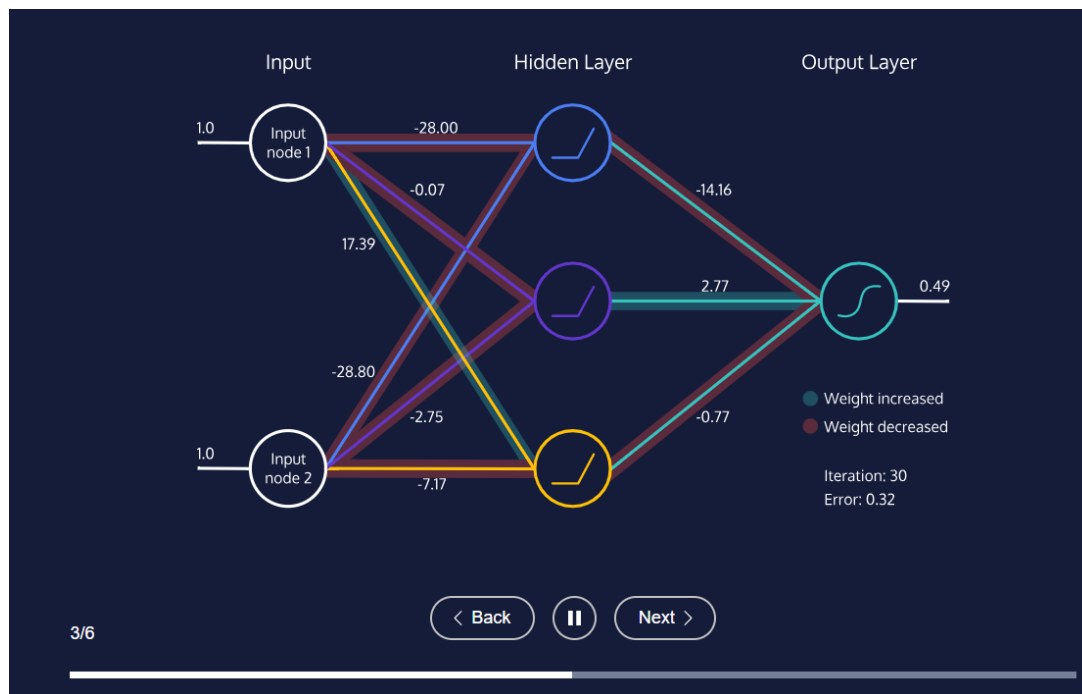
known as gradient descent. This algorithm continuously updates and refines the weights between neurons to minimize our loss function.

By gradient, we mean the rate of change with respect to the parameters of our loss function. From this, backpropagation determines how much each weight is contributing to the error in our loss function, and gradient descent will update our weight values accordingly to decrease this error.

This is a conceptual overview of backpropagation. If you would like to engage with the gritty mathematics of it, you can do so [here](#). However, for this course, we will not need this level of detailed understanding.

Instructions

Let's take a look at what happens with backpropagation and gradient descent on a neural network directly. In the applet in the learning environment, watch as weights are updated and error is decreased after each iteration. Without backpropagation, neural networks would be much less accurate.



Gradient Descent

We have the overall process of backpropagation down! Now, let's zoom in on what is happening during gradient descent.

If we think about the concept graphically, we want to look for the minimum point of our loss function because this will yield us the highest accuracy. If we start at a random point on our loss function, gradient descent will take "steps" in the "downhill direction" towards the negative gradient. The size of the "step" taken is dependent on our learning rate. Choosing the optimal learning rate is important because it affects both the efficiency and accuracy of our results.

The formula used with learning rate to update our weight parameters is the following:

$$\text{parameter_new} = \text{parameter_old} + \text{learning_rate} \cdot \text{gradient}(\text{loss_function}(\text{parameter_old}))$$

$$\text{parameter_new} = \text{parameter_old} + \text{learning_rate} \cdot \text{gradient}(\text{loss_function}(\text{parameter_old}))$$

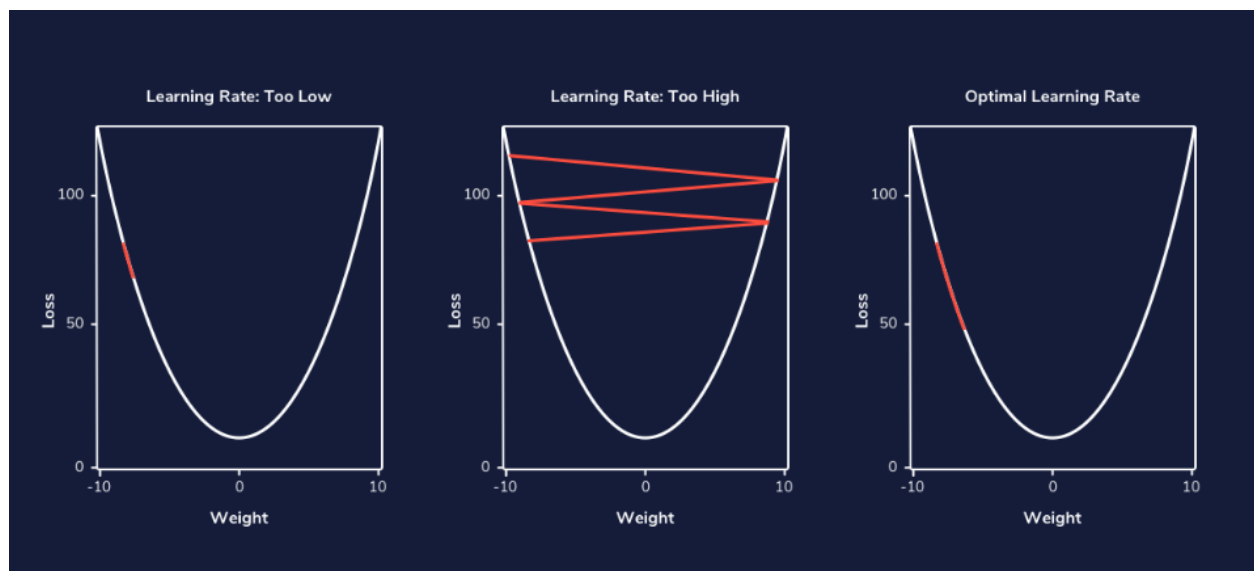
The learning rate we choose affects how large the "steps" our pointer takes when trying to optimize our error function. Initial intuition might indicate that you should choose a large learning rate; however, as shown above, this can lead you to overshoot the value we are looking for and cause a divergent search.

Now you might think that you should choose an incredibly small learning rate; however, if it is too small, it could cause your model to be unbearably inefficient or get stuck in a local minimum and never find the optimum value. It is a tricky game of finding the correct combination of efficiency and accuracy.

Instructions

Take a look at the graphs, which depict some common issues you may run into when selecting a learning rate. If you select a learning rate that is too small, it will take a very long time to find the minimum value. However, if you choose a learning rate that is too large, it might overshoot the minimum value and end up with a divergent algorithm

Choosing an ideal learning rate means it should find the ideal loss value efficiently and accurately.



Stochastic Gradient Descent

This leads us to the final point about gradient descent. In deep learning models, we are often dealing with extremely large datasets. Because of this, performing backpropagation and gradient descent calculations on all of our data may be inefficient and computationally exhaustive no matter what learning rate we choose.

To solve this problem, a variation of gradient descent known as *Stochastic Gradient Descent (SGD)* was developed. Let's say we have 100,000 data points and 5 parameters. If we did 1000 iterations (also known as

[epochs](#)

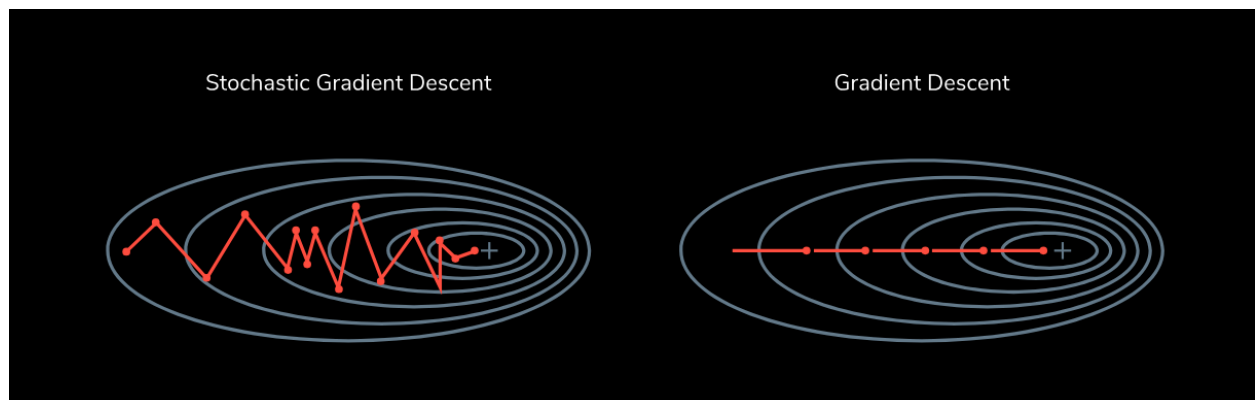
Preview: Docs Loading link description

in Deep Learning) we would end up with $100000 \cdot 5 \cdot 1000 = 500,000,000$ computations. We do not want our computer to do that many computations on top of the rest of the learning model; it will take forever.

This is where SGD comes to play. Instead of performing gradient descent on our entire dataset, we pick out a random data point to use at each iteration. This cuts back on computation time immensely while still yielding accurate results.

Instructions

The diagram below shows the performance differences between SGD and GD. You may notice that the SGD graph is a bit more sporadic. There is a reason for this, and we will address it in the next exercise. The main point is that both will reach the ideal parameters, and SGD will be easier and more efficient for your computer processor. Because of this, SGD is almost universally used in favor of normal GD. However, as we will see next, there are even more variants of gradient descent.



More Variants of Gradient Descent

Just when you thought SGD solved all our problems, even more options come into the picture!

There are also other variants of gradient descent such as *Adam optimization algorithm* and *mini-batch gradient descent*. Adam is an adaptive learning

[algorithm](#)

Preview: Docs Loading link description
that finds individual learning rates for each

[parameter](#)

Preview: Docs Loading link description

. Mini-batch gradient descent is similar to SGD except instead of iterating on one data point at a time, we iterate on small batches of fixed size.

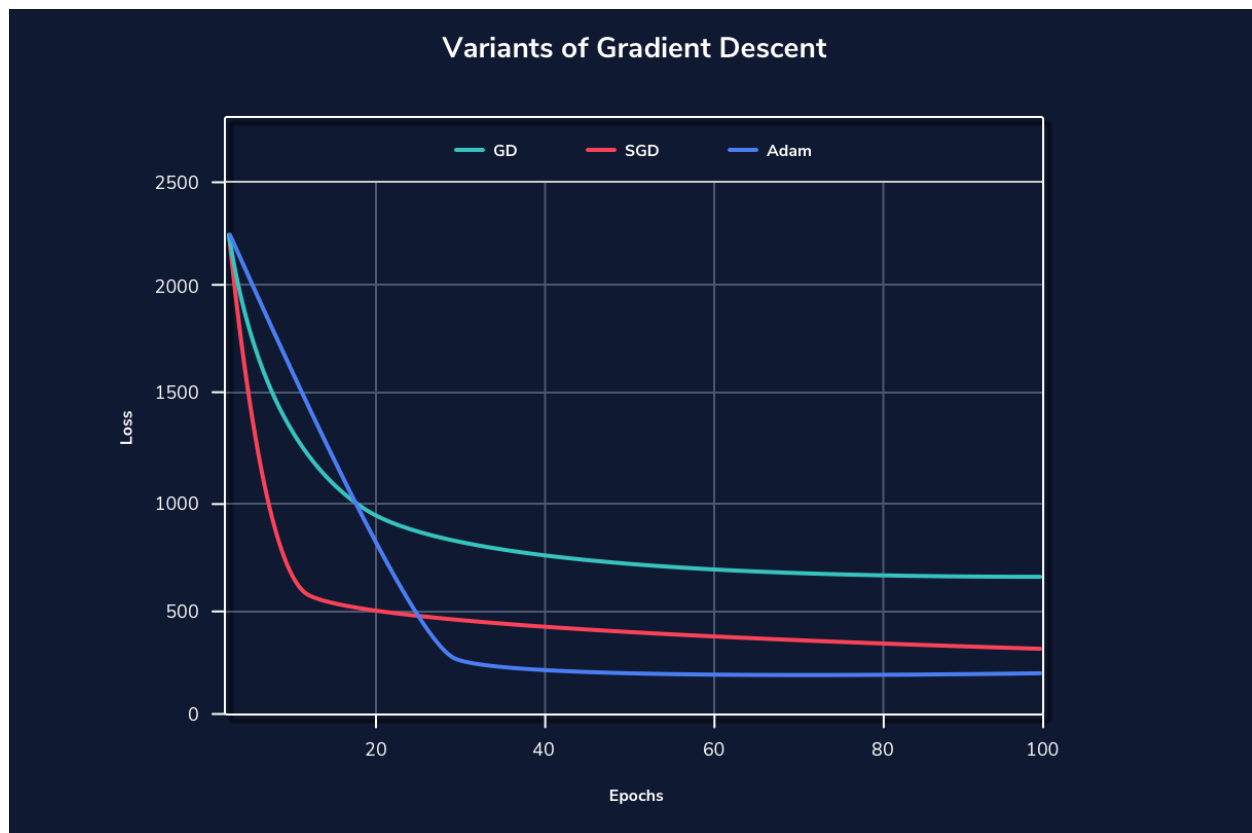
Adam optimizer's ability to have an adaptive learning rate has made it an ideal variant of gradient descent and is commonly used in deep learning models. Mini-batch gradient descent was developed as an ideal trade-off between GD and SGD. Since mini-batch does not depend on just one training sample, it has a much smoother curve and is less affected by outliers and [noisy data](#) making it a more optimal algorithm for gradient descent than SGD.

These are just some quick notes! You can read more about Adam [here](#) and more about mini-batch [here](#). Experts in deep learning are constantly coming up with ways to improve these algorithms to make them more efficient and accurate, so the ability to adapt and build upon what you learn as you dive into this domain will be key!

Instructions

This is a diagram you will become accustomed to in future lessons. It shows us our loss function performance over many *epochs* (iterations) of our deep learning model with different types of gradient descent.

These graphs are extremely useful when creating learning models because they offer us a detailed view of their performance.



Review

This overview completes the necessary mathematical intuition you need to move forward and begin coding your own learning models! To recap all the things we have learned (so many things!!!):

- Scalars, vectors, matrices, and tensors
 - A *scalar* is a singular quantity like a number.
 - A *vector* is an [array](#)
 - Preview: Docs Loading link description of numbers (scalar values).
 - A *matrix* is a grid of information with rows and columns.
 - A *tensor* is a multidimensional array and is a generalized version of a vector and matrix.
- Matrix Algebra
 - In *scalar multiplication*, every entry of the matrix is multiplied by a scalar value.
 - In *matrix addition*, corresponding matrix entries are added together.
 - In *matrix multiplication*, the dot product between the corresponding rows of the first matrix and columns of the second matrix is calculated.
 - A *matrix transpose* turns the rows of a matrix into columns.
- In *forward propagation*, data is sent through a neural network to get initial outputs and error values.
 - *Weights* are the learning parameters of a deep learning model that determine the strength of the connection between two nodes.
 - A *bias node* shifts the activation function either left or right to create the best fit for the given data in a deep learning model.
 - *Activation Functions* are used in each layer of a neural network and determine whether neurons should be “fired” or not based on output from a weighted sum.
 - *Loss functions* are used to calculate the error between the predicted values and actual values of our training set in a learning model.
- In *backpropagation*, the gradient of the loss function is calculated with respect to the weight parameters within a neural network.
 - *Gradient descent* updates our weight parameters by iteratively minimizing our loss function to increase our model’s accuracy.
 - *Stochastic gradient descent* is a variant of gradient descent, where instead of using all data points to update parameters, a random data point is selected.
 - *Adam optimization* is a variant of SGD that allows for adaptive learning rates.
 - *Mini-batch gradient descent* is a variant of GD that uses random batches of data to update parameters instead of a random datapoint.

This has been a ton of material, so do not fret if you are confused or still wrapping your head around some of the concepts. Many of this will be reviewed as you begin coding models. Get excited to see how these concepts are powered with TensorFlow!

Black PEOPLE CLASSIFIED AS APE - MONKEY OR GORILLA

<https://web.archive.org/web/20201004111743/https://www.wnycstudios.org/podcasts/notetotself/episodes/deep-problem-deep-learning>

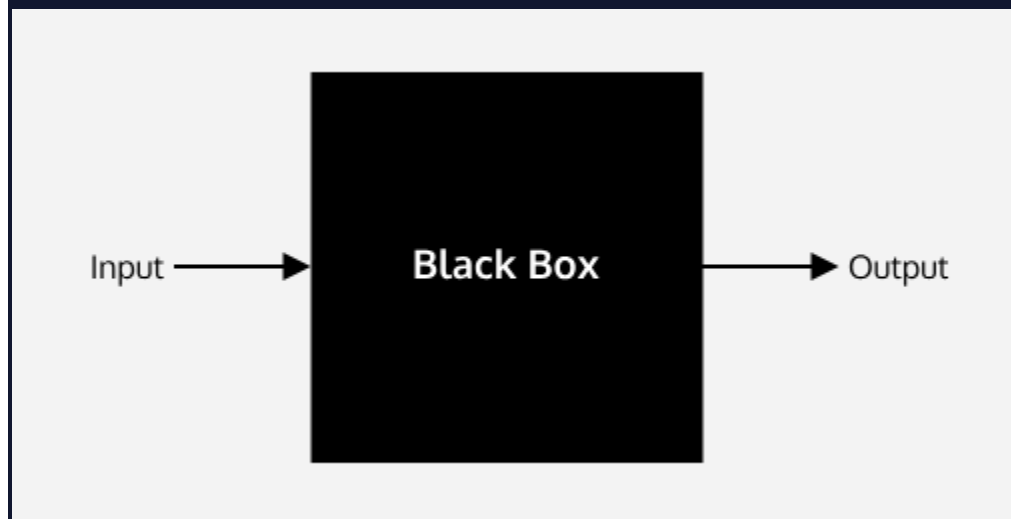
Dangers of the Black Box

Deep learning models have deep implications.

What Makes Deep Learning Models Dangerous?

When talking about machine learning, deep learning, and artificial intelligence, people tend to focus on the progress and amazing feats we could potentially achieve. While it is true that these disciplines have the potential to change the world we live in and allow us to perform otherwise impossible feats, there are often unintended consequences.

We live in an imperfect world, and the learning algorithms we design are not immune to these imperfections. Before we dive into creating our models, we must review some of the issues and implications that they can have on people's lives. Deep Learning models can be especially scary as they are some of the most powerful, and they are becoming more commonplace in society. They are also most often *black boxes* (hard for users to understand as to why and how specific results occur).



When to Use Them

Due to this nature of deep learning models, we should only ever use them if there is an apparent, significant reason for using one. If there is not a clear reason, we can use basic machine learning or statistical analysis approaches depending on what suffices.

When choosing solutions to a problem, we have to juggle many numerous factors, including

along with even more before beginning to prototype. Asking questions along these lines is vital because we design learning algorithms that can harm individuals or even entire communities in their daily lives if we do not craft them with care and awareness.

Misconceptions

There are often misconceptions about AI and deep learning models and what implications they have on our world. Science fiction has illustrated the dangers as some sort of robot apocalypse where humans are outsmarted and overpowered. This depiction does not reflect the real risks that are already present from the growing dependence on learning models in our everyday lives. Healthcare, banking, and the criminal justice system have all seen a massive rise in the reliance on learning algorithms to make decisions in recent years. While these have led to increased efficiency and developments, trusting these systems to make high-risk decisions has also led to various risks.

Key Issues

There are some key things to address about machine learning models that can lead to potentially problematic implications. Let's dive into this through the lens of healthcare.

Machine learning algorithms can only be as good as the data it is trained on. If we train the data on a model that does not match environmental data well, accuracy will not translate into the real world. A good example is this [study](#) on personalized risk assessments for acute kidney injury. While patient data evolved and disease patterns changed, the predictive model became less accurate and, therefore, much less useful. It is crucial for developers to monitor outputs periodically and account for data-shift problems over time. The work is not complete after we deploy a model; it must be managed and continuously refined to account for continuous environmental developments.

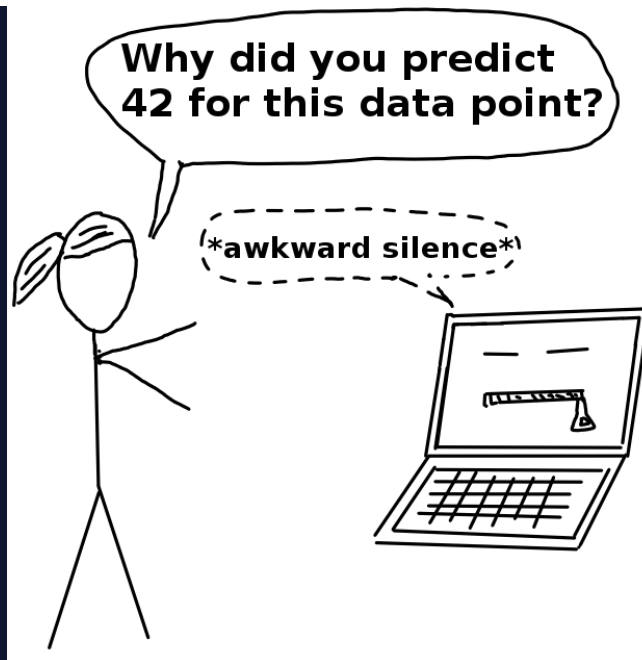
There is also a history of the health industry not including enough women and people of color in medical studies. Different demographics can have unique manifestations and risk factors with diseases. If training data is not diverse and representative of all potential sample individuals, inaccuracies and misdiagnoses could occur. It is vital to have orthogonal data in that it is both high in volume and diversity. Without attention to this, social health disparities that are already present could become widened.

Machine learning models do not understand the impact of a false negative vs. a false positive diagnostic (at least not like humans can). When diagnosing patients, doctor's often "err on the side of caution." For example, being falsely diagnosed with a malignant tumor (false positive) is much less dangerous than being incorrectly diagnosed with a benign tumor (false negative). Further testing would occur in the former situation, whereas the latter could cause much scarier consequences with a malignant tumor being unaddressed.

Models may not have this "err on the side of caution" attitude, especially if we do not design them with this implication in mind. If we solely train it to be as accurate as possible, it is at the expense of missing malignant diagnoses. Developers can create custom loss functions that can account for the implications of false negatives vs. false positives. However, to do this, they must understand the domain well.

For many of the clinicians and the patients, the models are a black box. Stakeholders only can make decisions based on the outcome, and the predictions are not open to inspection. Therefore, it is hard for anyone to determine that there are patterns of inaccurate predictions until prolonged use has already occurred. For example, imagine an X-ray analysis model that is inaccurate under certain conditions because it was not present in the training data. The doctor would not identify

this until continuously observing incorrect diagnoses because the only aspect of the model focuses on is the outcome.

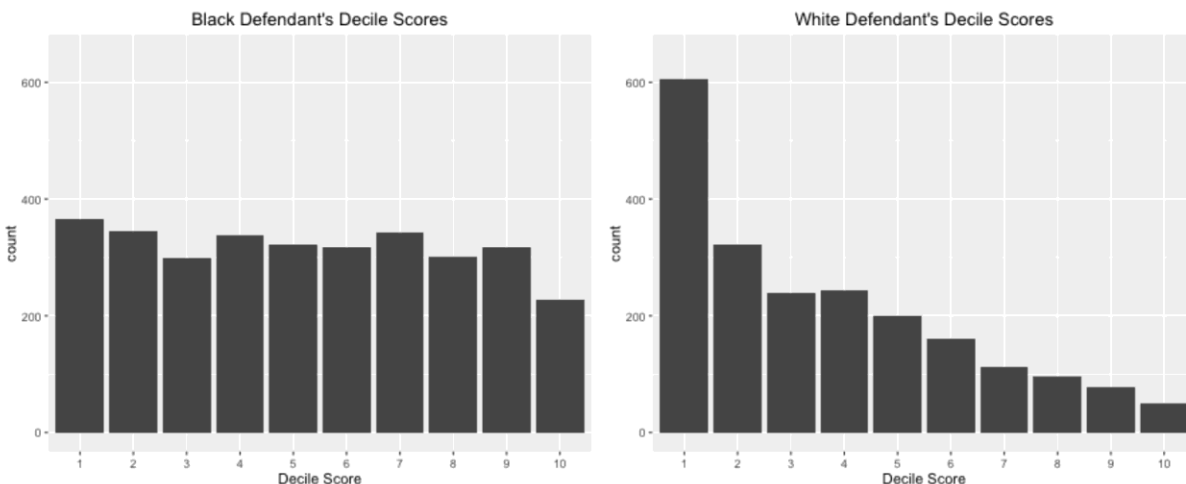


(Source: [Christoph Molnar](#))

More Examples

This is just the tip of the iceberg for concerns about the use of learning models in the healthcare industry, and we have yet to even go into implications within other sectors. Facial recognition has shown implicit bias within Google's algorithm, which [incorrectly identified a woman and her friend as gorillas](#). An error like this leads to emotional harm, and it shows that learning models are not immune to social issues and can reproduce or even exacerbate them.

Employing black box technology becomes more of an issue when used in contexts without transparency. For example, in criminal justice or banking, biased data is used to deny people of color loans at a higher rate or label them as "high risk" repeat offenders. A real-life example of this is a machine learning questionnaire algorithm known as COMPAS (Correctional Offender Management Profiling for Alternative Sanctions). It was used to determine the risk of whether an arrestee would be a repeat offender. A [study](#) showed that this algorithm was racially-biased despite never explicitly asking about race. Individuals were asked questions that modeled existing social inequalities, and minorities, particularly blacks, were more likely to be labeled "high-risk" repeat offenders. The graph below shows the distribution of risk scores for black defendants and white defendants. You can read more about the analysis of this report [here](#).



(Source: [ProPublica](#))

Models like these can even go beyond mirroring existing inequity. They can go on to perpetuate them and contribute to vicious cycles. For example, if we were to use systems like these to determine patrol routes, this could bias crime data for individuals in those communities. As we add more data to the learning model, the problem is exacerbated. What is even scarier about these models is that they are often beyond dispute or appeal. Given the result of a mathematical formula, we usually take it as fact. Someone who ends up negatively affected by this cannot argue against it or reason with a machine. They cannot explain the full reality to a computer. Instead, a machine will define its own reality to justify its results. While an outside observer can question, “why did this individual get a high-risk score despite only a minor crime?” a machine is merely operating under its historical data and findings.

Personal Responsibility

This is not to say that we should not use machine learning models in ways that impact everyday lives. It is meant to outline some of the issues that can arise when used to make high-stakes decisions and how they can cause harm. As you move forward to developing your own neural networks, consider the social implications of what you have made. As developers, we must work to ensure that our models are free from bias and will not misrepresent certain populations. The Institute for Ethical AI and Machine Learning has a [framework for developing responsible machine learning](#) that we encourage you to read up on.

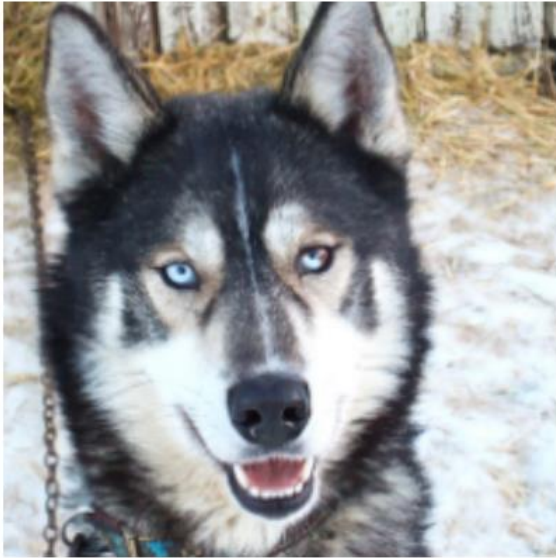
Interpretability and Transparency

It is also imperative that the models we build are used in ways that are transparent to potential stakeholders. If someone is impacted due to the output of a learning model, they should understand:

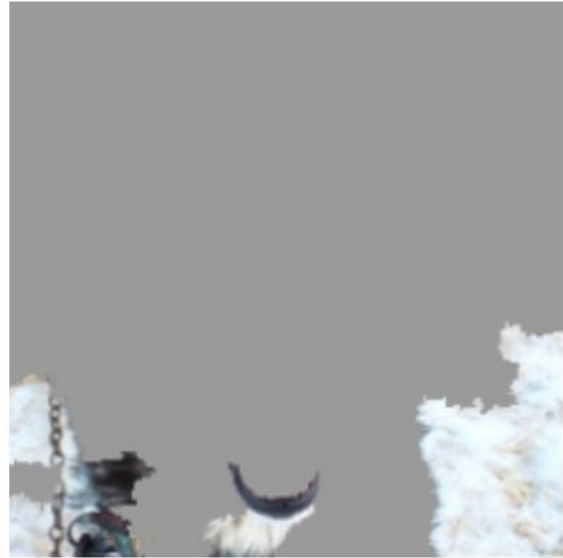
- why the model is being used
- how their personal data is being used
- what personal data is being used

The final thing to consider is making your model's inner workings understandable for your users, also known as *interpretable machine learning*. Making the inner workings of a model understandable allows users to identify inaccuracies earlier and explain results to potential stakeholders. A great example of this is a simple classifier model that identifies huskies vs. wolves. In the case below, a husky is incorrectly classified as a wolf. With the implementation of a system called [LIME](#), predictions are explained and can be understood by users. In this case, the explanation is that if a picture contains snow, it will be classified

as a wolf. This information gives developers an indicator of why their model contains inaccuracies and clarifies how to improve it.



(a) Husky classified as wolf



(b) Explanation

(Source: [arXiv.org](https://arxiv.org))

In Conclusion

There is no doubt that machine learning can help us make waves of progress. However, in a world riddled with inequity, developers must attempt design systems so that no one is left behind. By designing learning models that account for limitations of interpretability and likelihood of bias, we can take strides to ensure that individuals and communities are not unjustly harmed.

What are Neural Networks?

An artificial neural network is an interconnected group of nodes, an attempt to mimic to the vast network of neurons in a brain.

Understanding Neural Networks

As you are reading this article, the very same brain that sometimes forgets why you walked into a room is magically translating these pixels into letters, words, and sentences — a feat that puts the world's fastest supercomputers to shame. Within the brain, thousands of neurons are firing at incredible speed and accuracy to help us recognize text, images, and the world at large.

The image you are
requesting does not exist
or is no longer available.

imgur.com

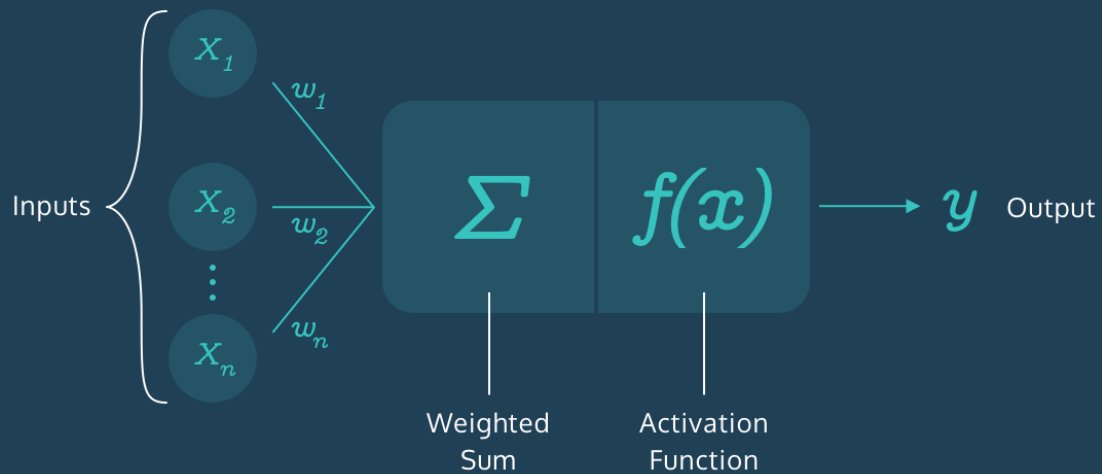
A **neural network** is a programming model inspired by the human brain. Let's explore how it came into existence.

The Birth of an Artificial Neuron

Computers have been designed to excel at number-crunching tasks, something that most humans find terrifying. On the other hand, humans are naturally wired to effortlessly recognize objects and patterns, something that computers find difficult.

This juxtaposition brought up two important questions in the 1950s:

In 1957, [Frank Rosenblatt](#) explored the second question and invented the **Perceptron** algorithm that allowed an artificial neuron to simulate a biological neuron! The artificial neuron could take in an input, process it based on some rules, and fire a result. But computers had been doing this for years — what was so remarkable?



There was a final step in the Perceptron algorithm that would give rise to the incredibly mysterious world of Neural Networks — the artificial neuron could *train itself based on its own results, and fire better results in the future*. In other words, it could learn by trial and error, just like a biological neuron.

More Neurons

The Perceptron Algorithm used multiple artificial neurons, or perceptrons, for image recognition tasks and opened up a whole new way to solve computational problems. However, as it turns out, this wasn't enough to solve a wide range of problems, and interest in the Perceptron Algorithm along with Neural Networks waned for many years.

But many years later, the neurons fired back.

The image you are requesting does not exist or is no longer available.

imgur.com

It was found out that creating multiple layers of neurons — with one layer feeding its output to the next layer as input — could process a wide range of inputs, make complex decisions, and still produce meaningful results. With some tweaks, the algorithm became known as the **Multilayer Perceptron**, which led to the rise of Feedforward Neural Networks.

60 Years Later...

With Feedforward Networks, the results improved. But it was only recently, with the development of high-speed processors, that neural networks finally got the necessary computing power to seamlessly integrate into daily human life.

Today, the applications of neural networks have become widespread — from simple tasks like speech recognition to more complicated tasks like self-driving vehicles.

In 2012, [Alex Krizhevsky](#) and his team at University of Toronto entered the ImageNet competition (the annual Olympics of computer vision) and trained a **deep convolutional neural network** [\[pdf\]](#). No one truly understood how it made the decisions it did, but it worked better than any other traditional classifier, by a huge 10.8% margin.

Summary

The neurons have come a long way. They have braved the AI winter and remained patient amidst the lack of computing power in the 20th century. They have now taken the world by storm and deservedly so.

Neural networks are ridiculously good at generating results but also mysteriously complex; the apparent complexity of the decision-making process makes it difficult to say exactly how neural networks arrive at their superhuman level of accuracy.

Let's dive into the world of Neural Networks and relish in all its mystery!

What is a Perceptron?

Similar to how atoms are the building blocks of matter and how microprocessors are the building blocks of a computer, perceptrons are the building blocks of [Neural Networks](#).

If you look closely, you might notice that the word “perceptron” is a combination of two words:

Perception (noun) the ability to sense something

Neuron (noun) a nerve cell in the human brain that turns sensory input into meaningful information

Therefore, the perceptron is an artificial neuron that simulates the task of a biological neuron to solve problems through its own “sense” of the world.

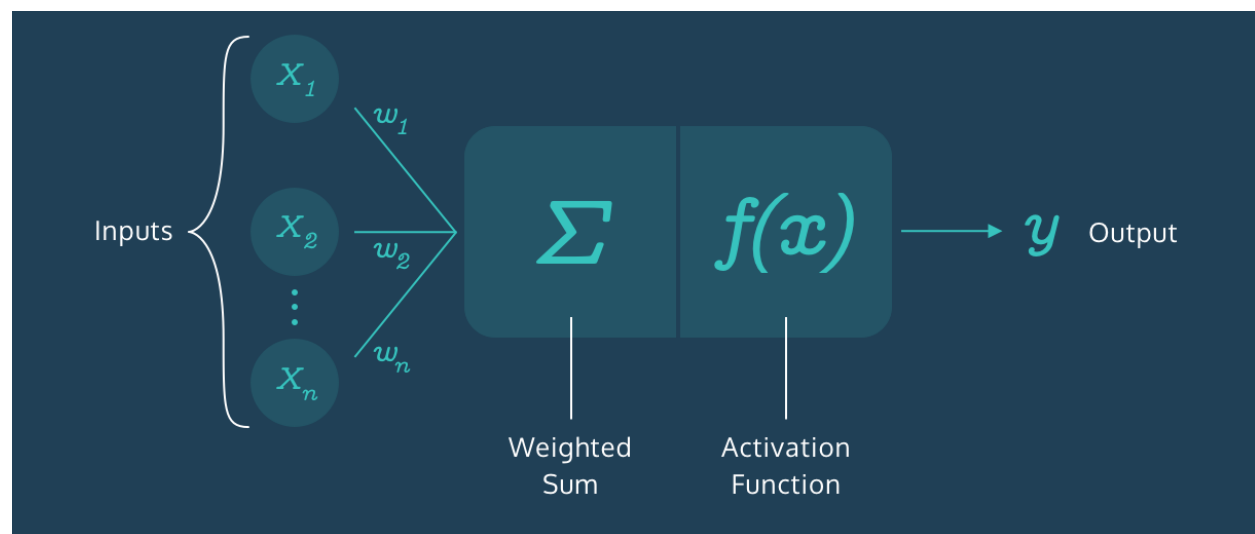
Although the perceptron comes with its own artificial design and set of parameters, at its core, a single perceptron is trying to make a simple decision.

Let's take the example a simple self-driving car that is based on a perceptron. If there's an obstacle on the left, the car would have to steer right. Similarly, if there's an obstacle on the right, the car would have to steer left.

For this example, a perceptron could take the position of the obstacle as inputs, and produce a decision — left turn or right turn — based on those inputs.

And here's the cool part — the perceptron can correct itself based on the result of its decision to make better decisions in the future!

Of course, the real world scenario isn't that simple. But if you combine a bunch of such perceptrons, you will get a neural network that can even make better decisions on your behalf!



Representing a Perceptron

So the perceptron is an artificial neuron that can make a simple decision. Let's implement one from scratch in Python!

The perceptron has three main components:

Inputs: Each input corresponds to a feature. For example, in the case of a person, features could be age, height, weight, college degree, etc.

Weights: Each input also has a weight which assigns a certain amount of importance to the input. If an input's weight is large, it means this input plays a bigger role in determining the output. For example, a team's skill level will have a bigger weight than the average age of players in determining the outcome of a match.

Output: Finally, the perceptron uses the inputs and weights to produce an output. The type of the output varies depending on the nature of the problem. For example, to predict whether or not it's going to rain, the output has to be

[binary](#)

Preview: Docs Binary is a number system of 1s and 0s which serves as a textual representation method for giving instructions to a computer's CPU. In computing and telecommunications, binary codes are used for various methods of encoding data such as turning the character strings of source code, into bit strings or instruction sets for the CPU to execute.

— 1 for Yes and 0 for No. However, to predict the temperature for the next day, the range of the output has to be larger — say a number from 70 to 90.

Instructions

Checkpoint 1 Failed, try again

1.

Our `Perceptron` [class](#) by default takes two inputs and a pre-defined weight for each input.

Complete the `__init__()` method inside the `Perceptron` class by creating instance variables `self.num_inputs` and `self.weights` that represent the attributes of a `Perceptron` object.

Assign the parameters `num_inputs` and `weights` to `self.num_inputs` and `self.weights` respectively.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a `Perceptron` object called `cool_perceptron` (without any arguments) and print it out to see what it looks like.

```
class Perceptron:
    def __init__(self, num_inputs=2, weights=[1,1]):
        self.num_inputs = num_inputs
        self.weights = weights

cool_perceptron = Perceptron()
print(cool_perceptron)
```

Step 1: Weighted Sum

Great! Now that you understand the structure of the perceptron, here's an important question — how are the inputs and weights magically turned into an output? This is a two-step process, and the first step is *finding the weighted sum of the inputs*.

What is the **weighted sum**? This is just a number that gives a reasonable representation of the inputs:

$$\text{weighted sum} = x_1w_1 + x_2w_2 + \dots + x_nw_n$$

The x 's are the inputs and the w 's are the weights.

Here's how we can implement it:

Start with a weighted sum of 0. Let's call it `weighted_sum`.

Start with the first input and multiply it by its corresponding weight. Add this result to `weighted_sum`.

Go to the next input and multiply it by its corresponding weight. Add this result to `weighted_sum`. Repeat this process for all inputs.

Instructions

Checkpoint 1 Enabled

1.

Create a variable called `weighted_sum` to hold the value of the weighted sum and set its starting value to 0.

Return `weighted_sum` outside the `for` loop.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's go through each input-weight pair and find the weighted sum using indexing.

Delete the `pass` statement inside the `for` loop. For each iteration in the loop, find the product of the input and weight at index `i`, add the result to `weighted_sum`, and store it back in `weighted_sum` to update the value of `weighted_sum`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Outside the `Perceptron` class, after the `Perceptron` object `cool_perceptron` has been created, print out the weighted sum for the inputs `[24, 55]`.

What is the weighted sum?

```
class Perceptron:
    def __init__(self, num_inputs=2, weights=[2,1]):
        self.num_inputs = num_inputs
        self.weights = weights

    def weighted_sum(self, inputs):
        weighted_sum = 0
        for i in range(self.num_inputs):
            weighted_sum += self.weights[i]*inputs[i]
            #complete this loop
        return weighted_sum

cool_perceptron = Perceptron()
print(cool_perceptron.weighted_sum([24, 55]))
```

Step 2: Activation Function

After finding the weighted sum, the second step is to *constrain the weighted sum to produce a desired output*.

Why is that important? Imagine if a perceptron had inputs in the range of 100-1000 but the goal was to simply predict whether or not something would occur — `1` for “Yes” and `0` for “No”. This would result in a very large weighted sum.

How can the perceptron produce a meaningful output in this case? This is exactly where **activation functions** come in! These are special functions that transform the weighted sum into a desired and constrained output.

For example, if you want to train a perceptron to detect whether a point is above or below a line (which we will be doing in this lesson!), you might want the output to be a `+1` or `-1` label. For this task, you can use the “sign activation function” to help the perceptron make the decision:

If weighted sum is positive, return `+1`
If weighted sum is negative, return `-1`

In this lesson, we will focus on using the sign activation function because it is the simplest way to get started with perceptrons and eventually visualize one in action.

Instructions

Checkpoint 1 Enabled

1.

Inside the `.activation()` method, return `1` if the `weighted_sum` is greater than or equal to `0`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Inside the `.activation()` method, return `-1` if the `weighted_sum` is less than `0`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Try it out for yourself!

Print out the result of the method `.activation()` called on `cool_perceptron` if the weighted sum is `52`.

```
class Perceptron:
    def __init__(self, num_inputs=2, weights=[1,1]):
        self.num_inputs = num_inputs
        self.weights = weights

    def weighted_sum(self, inputs):
        weighted_sum = 0
        for i in range(self.num_inputs):
            weighted_sum += self.weights[i]*inputs[i]
        return weighted_sum

    def activation(self, weighted_sum):
        if weighted_sum >= 0:
            return 1
        if weighted_sum < 0:
            return -1

cool_perceptron = Perceptron()
print(cool_perceptron.weighted_sum([24, 55]))
print(cool_perceptron.activation(52))
```

Training the Perceptron

Our perceptron can now make a prediction given inputs, but how do we know if it gets those predictions right?

Right now we expect the perceptron to be very bad because it has random weights. We haven't taught it anything yet, so we can't expect it to get classifications correct! The good news is that we can train the perceptron to produce better and better results! In order to do this, we provide the perceptron a training set — a collection of random inputs with correctly predicted outputs.

On the right, you can see a plot of scattered points with positive and negative labels. This is a simple training set.

In the code, the training set has been represented as a

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3

with coordinates as keys and labels as values. For example:

```
training_set = {(18, 49): -1, (2, 17): 1, (24, 35): -1, (14, 26): 1, (17, 34): -1}
```

Copy to Clipboard

We can measure the perceptron's actual performance against this training set. By doing so, we get a sense of "how bad" the perceptron is. The goal is to gradually nudge the perceptron — by slightly changing its weights — towards a better version of itself that correctly matches all the input-output pairs in the training set.

We will use these points to train the perceptron to correctly separate the positive labels from the negative labels by visualizing the perceptron as a line. Stay tuned!

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import random

def generate_training_set(num_points):
    x_coordinates = [random.randint(0, 50) for i in range(num_points)]
    y_coordinates = [random.randint(0, 50) for i in range(num_points)]
    training_set = dict()
    for x, y in zip(x_coordinates, y_coordinates):
        if x <= 45-y:
            training_set[(x,y)] = 1
        elif x > 45-y:
            training_set[(x,y)] = -1
    return training_set

training_set = generate_training_set(30)

x_plus = []
y_plus = []
x_minus = []
y_minus = []

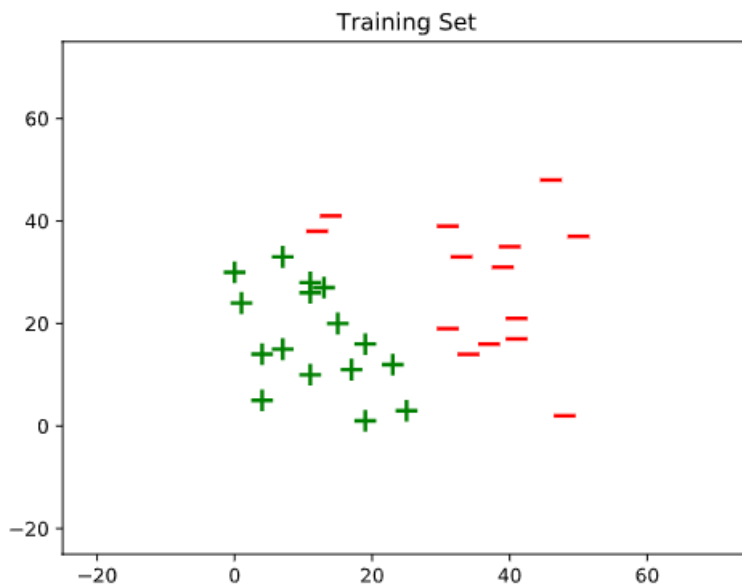
for data in training_set:
    if training_set[data] == 1:
        x_plus.append(data[0])
        y_plus.append(data[1])
    elif training_set[data] == -1:
        x_minus.append(data[0])
        y_minus.append(data[1])

fig = plt.figure()
ax = plt.axes(xlim=(-25, 75), ylim=(-25, 75))

plt.scatter(x_plus, y_plus, marker = '+', c = 'green', s = 128, linewidth = 2)
plt.scatter(x_minus, y_minus, marker = '_', c = 'red', s = 128, linewidth = 2)

plt.title("Training Set")

plt.show()
```



Training Error

Now that we have our training set, we can start feeding inputs into the perceptron and comparing the actual outputs against the expected labels!

Every time the output mismatches the expected label, we say that the perceptron has made a **training error** – a quantity that measures “how bad” the perceptron is performing.

As mentioned in the last exercise, the goal is to nudge the perceptron towards zero training error. The training error is calculated by subtracting the predicted label value from the actual label value.

$$\text{training error} = \text{actual label} - \text{predicted label}$$

$$\text{training error} = \text{actual label} - \text{predicted label}$$

For each point in the training set, the perceptron either produces a `+1` or a `-1` (as we are using the Sign Activation Function). Since the labels are also a `+1` or a `-1`, there are four different possibilities for the error the perceptron makes:

Actual	Predicted	Training Error
+1	+1	0
+1	-1	2

-1	-1	0
-1	+1	-2

These training error values will be crucial in improving the perceptron's performance as we will see in the upcoming exercises.

Instructions

Checkpoint 1 Enabled

1.

In the `.training()` method, let's find the perceptron's error on each `inputs` in `training_set`.

First, we need the perceptron's predicted output for a point. Inside the `for` loop, create a variable called `prediction` and assign it the correct label value using `.activation()`, `.weighted_sum()`, and `inputs` in a single statement.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable named `actual` and assign it the actual label for each `inputs` in `training_set`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create a variable called `error` and assign it the value of `actual - prediction`.

```
class Perceptron:
    def __init__(self, num_inputs=2, weights=[1,1]):
        self.num_inputs = num_inputs
        self.weights = weights

    def weighted_sum(self, inputs):
        weighted_sum = 0
        for i in range(self.num_inputs):
            weighted_sum += self.weights[i]*inputs[i]
        return weighted_sum

    def activation(self, weighted_sum):
        if weighted_sum >= 0:
            return 1
        if weighted_sum < 0:
            return -1

    def training(self, training_set):
        for inputs in training_set:
            prediction = self.activation(self.weighted_sum(inputs))
            actual = training_set[inputs]
            error = actual - prediction

cool_perceptron = Perceptron()
print(cool_perceptron.weighted_sum([24, 55]))
print(cool_perceptron.activation(52))
```

Tweaking the Weights

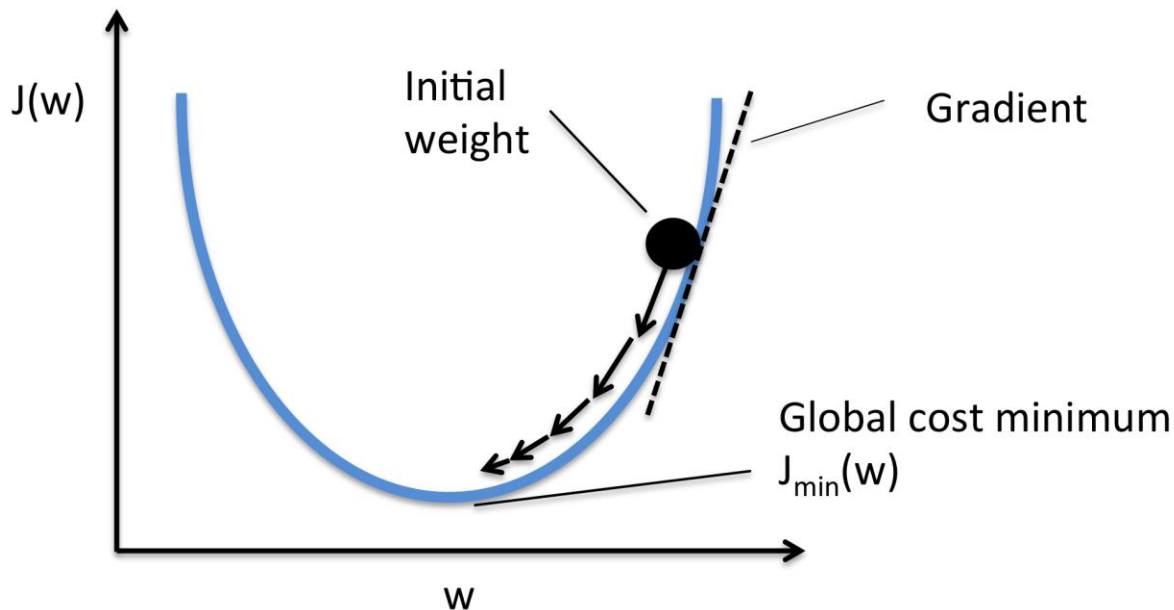
What do we do once we have the errors for the perceptron? We slowly nudge the perceptron towards a better version of itself that eventually has zero error.

The only way to do that is to change the parameters that define the perceptron. We can't change the inputs so the only thing that can be tweaked are the weights. As we change the weights, the outputs change as well.

The goal is to find the optimal combination of weights that will produce the correct output for as many points as possible in the dataset.

Instructions

Take a look at the visual on the right.



The Perceptron Algorithm

But one question still remains — how do we tweak the weights optimally? We can't just play around randomly with the weights until the correct combination magically pops up. There needs to be a way to guarantee that the perceptron improves its performance over time.

This is where the **Perceptron Algorithm** comes in. The math behind why this works is outside the

[scope](#)

Preview: Docs Loading link description
of this lesson, so we'll directly apply the

[algorithm](#)

Preview: Docs Loading link description
to optimally tweak the weights and nudge the perceptron towards zero error.

The most important part of the algorithm is the update rule where the weights get updated:

```
weight=weight+(error* input)
```

```
weight=weight+(error* input)
```

We keep on tweaking the weights until all possible labels are correctly predicted by the perceptron. This means that multiple passes might need to be made through the `training_set` before the Perceptron Algorithm comes to a halt.

In this exercise, you will continue to work on the `.training()`

[method](#)

Preview: Docs Loading link description

. We have made the following changes to this method from the last exercise:

```
foundLine = False (a
```

[boolean](#)

Preview: Docs Loading link description

that indicates whether the perceptron has found a line to separate the positive and negative labels)

while not foundLine: (a while loop that continues to train the perceptron until the line is found)

total_error = 0 (to count the total error the perceptron makes in each round)

total_error += abs(error) (to update the total error the perceptron makes in each round)

Instructions

Checkpoint 1 Enabled

1.

If the algorithm doesn't find an error, the perceptron must have correctly predicted the labels for all points.

Outside the for loop (but inside the while loop), change the value of foundLine to True if total_error equals 0.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

In order to update the weight for each inputs, create another for loop (inside the existing for loop) that iterates a loop variable i through a range of self.num_inputs.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Inside the second for loop, update each weight self.weights[i] by applying the update rule:

weight=weight+(error* inputs)

$weight = weight + (error * inputs)$

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Great job! Now give it a try for yourself.

Train cool_perceptron using small_training_set.

You can also print out the optimal weights to see for yourself!

```
class Perceptron:
    def __init__(self, num_inputs=2, weights=[1,1]):
        self.num_inputs = num_inputs
        self.weights = weights

    def weighted_sum(self, inputs):
        weighted_sum = 0
        for i in range(self.num_inputs):
            weighted_sum += self.weights[i]*inputs[i]
        return weighted_sum

    def activation(self, weighted_sum):
        if weighted_sum >= 0:
            return 1
        if weighted_sum < 0:
            return -1

    def training(self, training_set):
        foundLine = False
        while not foundLine:
            total_error = 0
            for inputs in training_set:
                prediction = self.activation(self.weighted_sum(inputs))
                actual = training_set[inputs]
                error = actual - prediction
                total_error += abs(error)
                for i in range(self.num_inputs):
                    self.weights[i] += error*inputs[i]
            if total_error == 0:
                foundLine = True
```

```
cool_perceptron = Perceptron()
small_training_set = {(0,3):1, (3,0):-1, (0,-3):-1, (-3,0):1}
cool_perceptron.training(small_training_set)
print(cool_perceptron.weights)
```

The Bias Weight

You have understood that the perceptron can be trained to produce correct outputs by tweaking the regular weights.

However, there are times when a minor adjustment is needed for the perceptron to be more accurate. This supporting role is played by the bias weight. It takes a default input value of 1 and some random weight value.

So now the weighted sum equation should look like:

$$\text{weighted sum} = x_1w_1 + x_2w_2 + \dots + x_nw_n + 1w_b$$

How does this change the code so far? You only have to consider two small changes:

- Add a 1 to the set of inputs (now there are 3 inputs instead of 2)
- Add a bias weight to the list of weights (now there are 3 weights instead of 2)

We'll automatically make these replacements in the code so you should be good to go!

```
class Perceptron:
    def __init__(self, num_inputs=3, weights=[1,1,1]):
        self.num_inputs = num_inputs
        self.weights = weights

    def weighted_sum(self, inputs):
        weighted_sum = 0
        for i in range(self.num_inputs):
            weighted_sum += self.weights[i]*inputs[i]
        return weighted_sum

    def activation(self, weighted_sum):
        if weighted_sum >= 0:
            return 1
        if weighted_sum < 0:
            return -1

    def training(self, training_set):
        foundLine = False
        while not foundLine:
            total_error = 0
            for inputs in training_set:
                prediction = self.activation(self.weighted_sum(inputs))
                actual = training_set[inputs]
                error = actual - prediction
                total_error += abs(error)
                for i in range(self.num_inputs):
                    self.weights[i] += error*inputs[i]
            if total_error == 0:
                foundLine = True

cool_perceptron = Perceptron()
```

Representing a Line

So far so good! The perceptron works as expected, but everything seems to be taking place behind the scenes. What if we could visualize the perceptron's training process to gain a better understanding of what's going on?

The weights change throughout the training process so if only we could meaningfully visualize those weights...

Turns out we can! In fact, it gets better. The weights can actually be used to represent a line! This greatly simplifies our visualization.

You might know that a line can be represented using the slope-intercept form. A perceptron's weights can be used to find the slope and intercept of the line that the perceptron represents.

```
slope = -self.weights[0]/self.weights[1]
intercept = -self.weights[2]/self.weights[1]
```

The explanation for these equations is beyond the

[scope](#)

Preview: Docs Loading link description

of this lesson, so we'll just use them to visualize the perceptron for now.

In the plot on your right, you should be able to see a line that represents the perceptron in its first iteration of the training process.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import random
from perceptron import Perceptron, lines

def generate_training_set(num_points):
    x_coordinates = [random.randint(0, 50) for i in range(num_points)]
    y_coordinates = [random.randint(0, 50) for i in range(num_points)]
    training_set = dict()
    for x, y in zip(x_coordinates, y_coordinates):
        if x <= 45-y:
            training_set[(x,y,1)] = 1
        elif x > 45-y:
            training_set[(x,y,1)] = -1
    return training_set

training_set = generate_training_set(30)

x_plus = []
y_plus = []
x_minus = []
y_minus = []

for data in training_set:
    if training_set[data] == 1:
        x_plus.append(data[0])
        y_plus.append(data[1])
    elif training_set[data] == -1:
        x_minus.append(data[0])
        y_minus.append(data[1])
```

```

perceptron = Perceptron()
perceptron.training(training_set)

fig = plt.figure()
ax = plt.axes(xlim=(-25, 75), ylim=(-25, 75))
line, = ax.plot([], [], lw=2)

fig.patch.set facecolor('#ffc107')

plt.scatter(x_plus, y_plus, marker = '+', c = 'green', s = 128, linewidth = 2)
plt.scatter(x_minus, y_minus, marker = '_', c = 'red', s = 128, linewidth = 2)

plt.title('Iteration: 0')

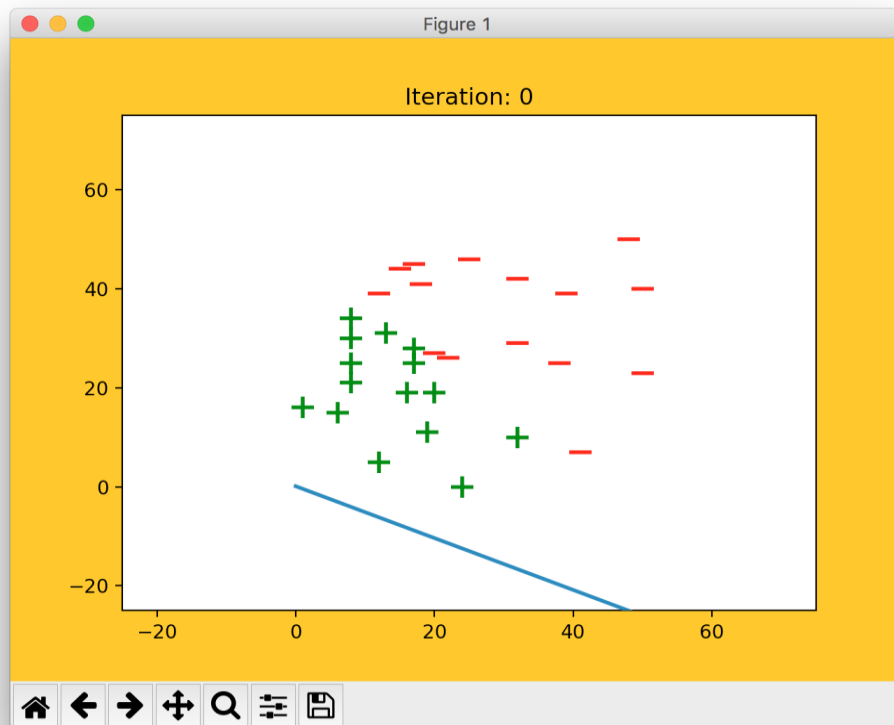
def animate(i):
    print(i)
    line.set_xdata(lines[i][0]) # update the data
    line.set_ydata(lines[i][1]) # update the data
    return line,

def init():
    line.set_data([], [])
    return line,

ani = animation.FuncAnimation(fig, animate, frames=1, init_func=init, interval=50,
blit=True, repeat=False)

plt.show()

```

Finding a Linear Classifier

Let's recap what you just learned!

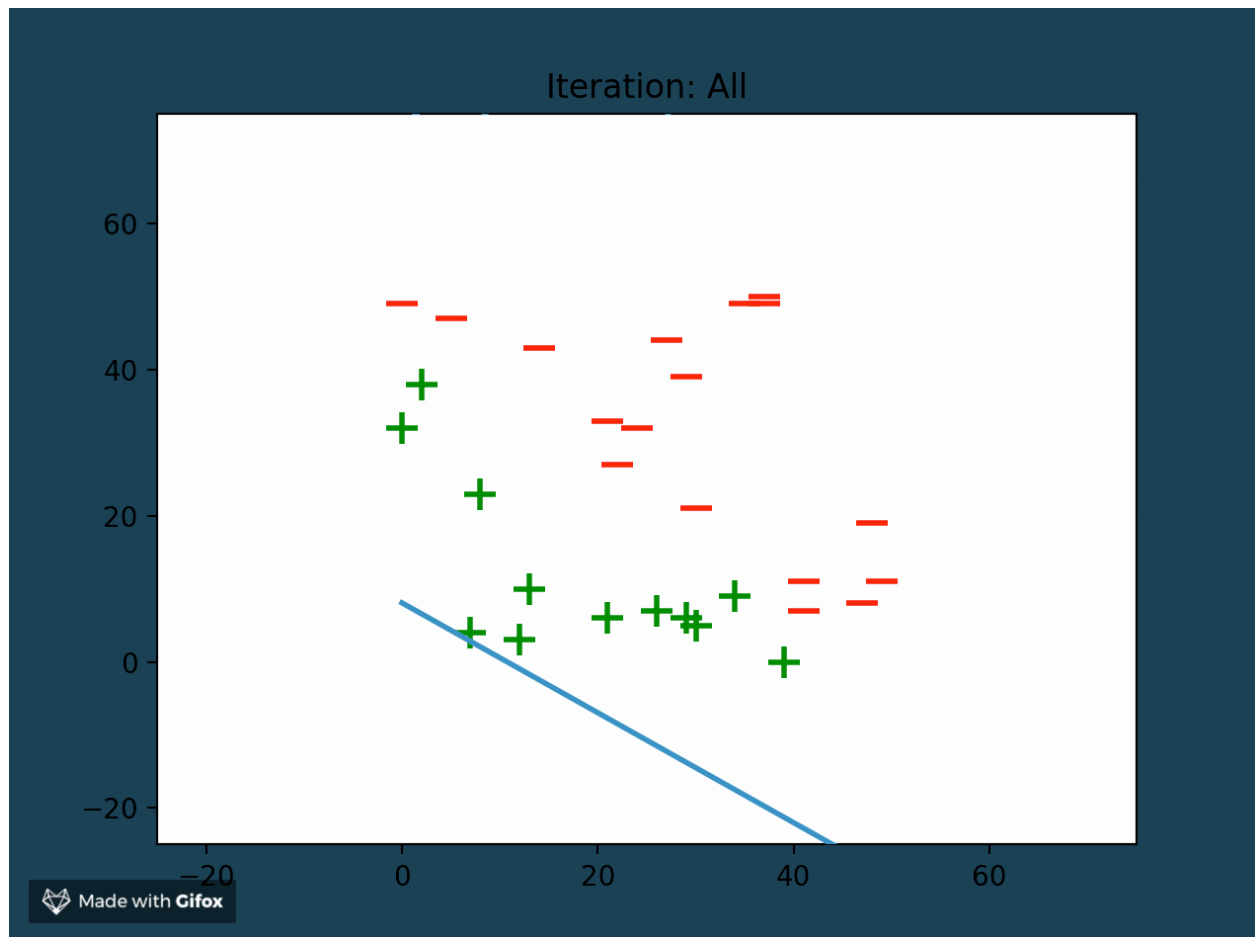
The perceptron has inputs, weights, and an output. The weights are parameters that define the perceptron and they can be used to represent a line. In other words, the perceptron can be visualized as a line.

What does it mean for the perceptron to correctly classify every point in the training set?

Theoretically, it means that the perceptron predicted every label correctly.

Visually, it means that the perceptron found a *linear classifier*, or a *decision boundary*, that separates the two distinct set of points in the training set.

In the plot on the right, you should be able to see the *linear classifier* that was found by the perceptron in the last iteration of the training process.



What's Next? Neural Networks

Congratulations! You have now built your own perceptron from scratch.

Let's step back and think about what you just accomplished and see if there are any limits to a single perceptron.

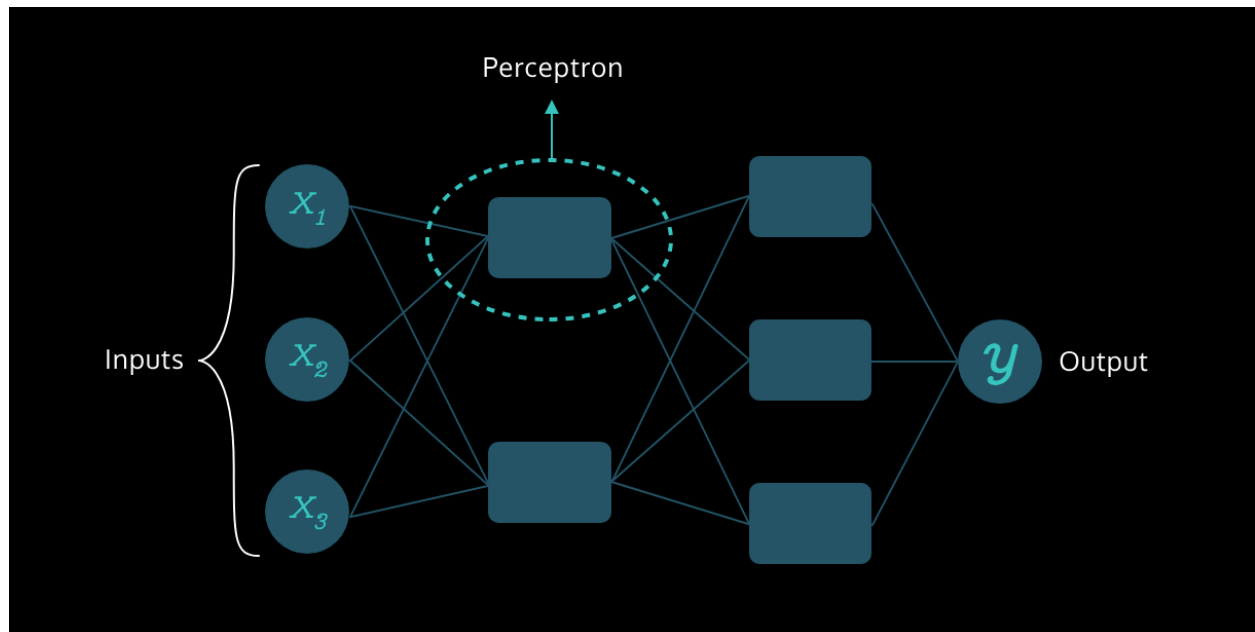
Earlier, the data points in the training set were *linearly separable* i.e. a single line could easily separate the two dissimilar sets of points.

What would happen if the data points were scattered in such a way that a line could no longer classify the points? A single perceptron with only two inputs wouldn't work for such a scenario because it cannot represent a *non-linear decision boundary*.

That's when more perceptrons and features come into play!

By increasing the number of features and perceptrons, we can give rise to the Multilayer Perceptrons, also known as Neural Networks, which can solve much more complicated problems.

With a solid understanding of perceptrons, you are now ready to dive into the incredible world of Neural Networks!



Group Project: OKCupid Date-A-Scientist

Group Project: OKCupid Date-A- Scientist

Group Project: OKCupid Date-A-Scientist

You can learn a lot on your own, but you can learn even more when you collaborate with others!

Group Projects in the Data Scientist Path

As part of this Career Path, you will have the opportunity to connect with other learners to practice and apply your new skills. This is optional! Throughout the career path, we will sometimes prompt you to work with peers, but you should feel free to work with others as often as you like to gain practice!

How to Conduct a Group Project

Step One – Understand the Project

Select what you would like to work on together. We suggest [OKCupid Date-A-Scientist](#), which will help you to collaborate on creating a full machine learning project with scikit-learn and a real-world dataset. Alternatively, come up with your own project that uses scikit-learn to implement machine learning algorithms or NLP.

Assess the prompt or project requirements:

- What technical skill sets do you need to complete the project? Will you need data analysis, data engineering, visualization and subject matter expertise?
- How advanced a skillset in each is required and how much work is there to go around?

From here you'll know which roles your project needs and how many people are necessary. Your project may only need one other person or could have four or more collaborators, depending on the size and complexity of the project.

Step Two – Find Your Teammates

Find collaborators! [Visit the projects forum](#). You can also find collaborators in the [data-science channel in Codecademy's Discord server](#). If still looking for others to work with, check out the Codecademy [Facebook group](#), in a [Codecademy Chapter](#), or at local meetups.

When you arrive, be sure to introduce yourself with the following information:

- What % of the path you have completed
- What timezone or country you're in
- A link to your Codecademy Profile
- One fun fact about yourself

Step Three – Build Your Team

Now that you've found people to work with, the next step is to figure out how each of you would like to contribute to this project.

These are some different roles that you and your teammates could take. Also, one person can have more than one role and more than one person could have the same role.

- Product Manager who also codes. This person makes decisions about what features will be built and in which order. Data Science Product Managers keep the team organized and on track to be sure everything gets done. To learn a lot more about being a team leader, [please see this post](#).
- Data Engineer. This person manages the data, making sure it is clean, reliable, and accessible. This role typically maintains the data standards and ensures consistency across a project.
- Data Scientist. This person draws on one or more data sets to generate insights, and uses hypothesis testing to answer questions. They may create visualizations, conduct exploratory analyses, or use summary statistics to describe data. They may also use predictive statistics and machine learning to make predictions from the data.
- Subject Matter Expert who also codes. This person situates datasets within their real-world context to ensure that the team is asking the right questions and interpreting the results accurately.

Think you may want to build something more complex? [Check out this article to see what a more specialized team looks like.](#)

Step Four – Get to Work

Get to it! Have a kick-off meeting, ideally over a video chat platform, so you can all meet one another and discuss the project.

Remember that if and when you get stuck, you can lean on each other in order to get through problems. In the real world, data scientists troubleshoot their own work first with tried and true methods, but they often ask teammates for help too. If your teammates can't help, reach out in the forums or in [the Discord server](#).

Sometimes, life happens. Even in the "real world" workplace, people get sick or leave projects. If a learner leaves and you need a replacement, post in the Discord server and the community will try to help you out.

Step Five – Share!

If you completed the [OKCupid Date-A-Scientist](#) Project, share it with fellow learners on its [forum page](#). If you choose to do another project, post your project in [the Projects category](#) and share it with the other learners in the forums or on Discord! Getting feedback is a vital part of the development process and in growing your skills. Remember to give back to other people wanting feedback or guidance, not just because it's a good thing to do, but you'll also benefit by explaining to others what you've learned.

If you're feeling generous, don't just share your work but share your insights and guidance for the next team tackling something similar. Paying it forward and giving back is how the developer world works – [entire systems have been built this way](#).

What is a Portfolio Project?

This project is a little different from other Codecademy projects you've encountered. In this project, you will bring together what you have learned in previous lessons to build a project in your own development environment and publish it to the web!



Create your own files
outside of the
Codecademy platform



Write your own code
with less guided
instructions



Use common project
management processes to
track your progress

How Do Portfolio Projects Work?

We'll provide you with high-level tasks to guide your project to completion, but you will be responsible for deciding how to implement them in your code.

There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Note that there are hints that can assist you, but they will only provide one potential implementation. Do not worry if your program looks different from ours!



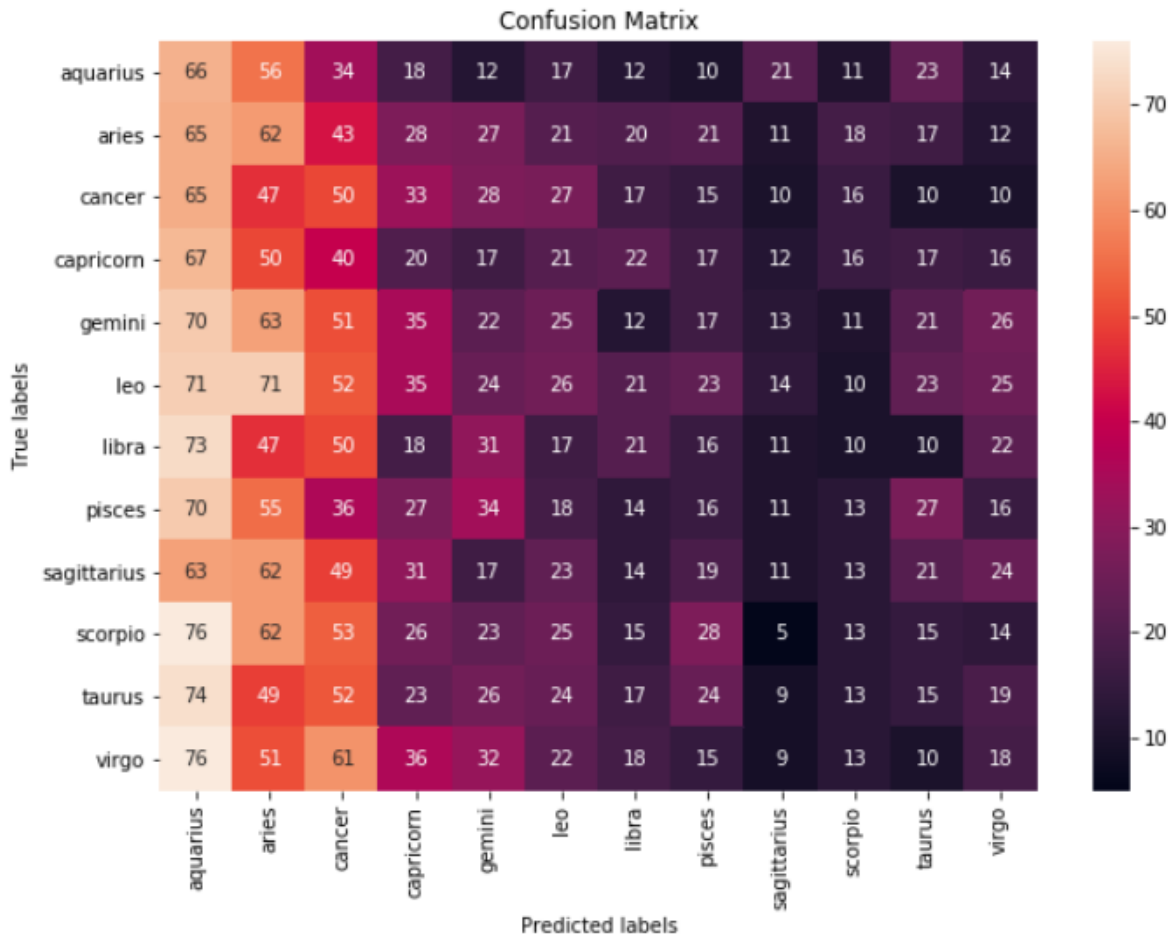
Project Prompt

OKCupid Date-A-Scientist

In recent years, there has been a massive rise in the usage of dating apps to find love. Many of these apps use sophisticated data science techniques to recommend possible matches to users and to optimize the user experience. These apps give us access to a wealth of information that we've never had before about how different people experience romance.

Click [Begin](#) to get started!

Project



Project Objectives:

- Complete a project to add to your portfolio
- Use Jupyter Notebook to communicate findings
- Build, train, and evaluate a machine learning model

Prerequisites:

- *Natural Language Processing*

- *Supervised Machine Learning*
- *Unsupervised Machine Learning*

IN PROGRESS

Project Setup

First we will have to download our materials

1. Download the [Date-A-Scientist project zip folder](#).

Double-click on it to “unzip” the folder. It should contain two items:

`date-a-scientist.ipnyb`

`profiles.csv`

- 2.
3. Type the following into the command line: `jupyter notebook` This should open a browser tab.
4. Click on **date-a-scientist.ipnyb** in the browser tab. This will open up your Jupyter Notebook.
5. Build your project in this file.

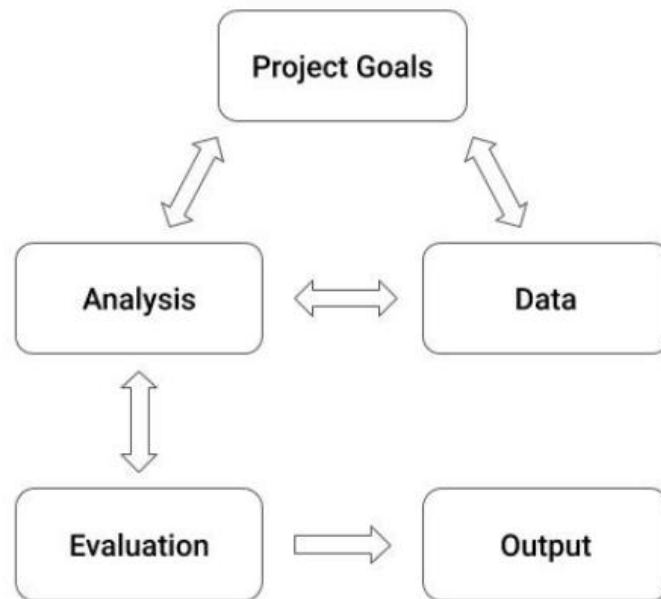
Setting up your Git Repository

Create a new Git repository for this project.

Project Scoping

Properly scoping your project will greatly benefit you; scoping creates structure while requiring you to think through your entire project before you begin. You should start with stating the goals for your project, then gathering the data, and considering the analytical steps required. A proper project scope can be a great road map for your project, but keep in mind that some down-

stream tasks may become dead ends which will require adjustment to the scope.



Select ML-Solvable Problem

First select a problem that you can answer using a machine learning approach. Make sure that this problem can be solved in a reasonable amount of time and with available data. Also, if the problem is too complex, it might be worth breaking it down into smaller units.

Load and Check Data

If supervised learning will be used, check that the data has a label or response variable. If the data is missing a crucial piece for analysis, go back to the previous step to select a different problem to solve.

Explore and Explain Data

Once you have your data, it's a good idea to get acquainted with your data. You should show some summary statistics and visually examine your data. Don't forget to also write out some insights that you have gained along with your analysis.

Preprocess Data

Once you have a good understanding of your data, it is time to get it prepared for modeling. These steps can involve joining data, standardizing variables, removing variables, imputation, and splitting data into train and test sets.

Build Model(s)

Once the data is preprocessed, it is ready to model. Select the appropriate type(s) of models to solve your problem. Once you have your models, run your data through each model and store your results.

Evaluate Model(s)

Once the model(s) are created, you need to use the metric of success that was defined to evaluate the model performance(s). Find the accuracy, precision, and recall of each model you used, and create graphs showing how they changed. If you aren't satisfied with your results you can repeat the previous step.

Create a Slide Deck

Once you've performed your analysis, you're ready to create your slide deck. One of the most important parts of Data Analysis is being able to communicate your findings.

Next Steps

Lastly, being a data scientist is about making continual progress. What are future-looking takeaways and limitations? If you were to do this project again, how could you extend this analysis?

Conclusions

Finally we can wrap up the project. You can write a conclusion about your process and findings.

Data Scientist: Machine Learning Specialist Career Path Review

Data Scientist: Machine Learning Specialist Career Path Review

Review what you learned in the Data Scientist: Machine Learning Career Path!

Congratulations on all your hard work building your data science knowledge and machine learning fluency. Everything you've learned in this Career Path will continue to serve you for years to come!

At this point, you have the technical knowledge, skills, and abilities you need to get your start as a Data Scientist with a specialty in Machine Learning.

You are now able to:

- Create programs with Python 3
- Query and manipulate data with SQL and Python pandas
- Create data visualizations with Python matplotlib and seaborn
- Summarize and analyze datasets
- Conduct hypothesis testing
- Clean and tidy datasets
- Communicate your findings clearly
- Run experiments and interpret the results
- Build, train, and test supervised and unsupervised machine learning models
- Evaluate model performance
- Decide when to apply a given model
- Conduct Feature Engineering
- Leverage Jupyter Notebooks for experimentation and communication

Plus, you now have several portfolio projects that showcase what you've learned in this Career Path and can share these projects with your community and future employers.

Once again, congratulations on finishing a challenging learning journey! We hope that you continue to build on your skills for years to come.

Next Steps after Data Scientist: Machine Learning

You've finished the Data Scientist: Machine Learning Career Path – what will you learn next?

Congratulations (again) on completing the Data Scientist: Machine Learning Career Path! There are a lot of places you can go with your new skills: You could work as a data scientist, in AI or Machine Learning Research, or create hobby projects. As you know, there is always more to learn.

So, now that you have finished this learning journey, you may be asking, “What’s next?”.

Interview Preparation

Now that you have completed the Career Path, you are ready to start applying to jobs (though we realize you may have already started this). There is a lot that goes into interviewing for data scientist roles, from developing a portfolio to passing take home, as well as multiple rounds of interviews. We have you covered with a [Data Scientist Interview Preparation](#) Skill Path, where you will learn about the stages in the interview process, practice sample take-home projects, and review everything you need to know to pass the interview and earn a job as a Data Scientist.

Other Areas of Data Science

While you've completed your Data Scientist Career Path with an Inference specialization, there is still a lot more to learn about data science as a field. From here, you can build your visualization skills with [Tableau](#), your Causal Inference Skills, or move into [Natural Language Processing](#).

R

R is the other major programming language in Data Science. You can get a start with the [R for Programmers] Course. If you want to dig deeper, the [Learn R](#) course covers the basics of the

language, and there's the [Analyze Data with R](#) Skill Path will help you get started with R for data analysis.

Data Science Events

Events give you a chance to connect with other data scientists and learn about the latest topics in the field. There are a lot of events out there including [PyData](#), [rstudio](#), [Women in Analytics](#), [Gartner Data & Analytics Summit](#), [The Data Science Conference](#), and the list goes on! You can also join or create a [Codecademy meetup](#) to connect with other new data scientists.

Once again, congratulations! We're excited to see what you accomplish next.

clean, transform, and sometimes manipulate the data structure to gain any insights. This process is often called data wrangling or data munging.

DATA WRANGLING - MUNGING

- `.read_csv()`
- `.head()`
- `.shape`

Preliminary data cleaning

- `.drop_duplicates()` / `drop_duplicates()`
- Column name (lowercase): `df.columns = map(str.lower, df.columns)`
- `df.rename({'dba': 'name', 'cuisine description': 'cuisine'}, axis=1)`

Data Types

object | float64 | int64 | bool | datetime64

- df.dtypes
- df.nunique()

Missing Data

- df.isna().sum()

- We will use the .where() method from the pandas library to replace the invalid coordinates. The .where() method keeps the values specified in its first argument, and replaces all other values with NaN.

```
# here our .where() function replaces latitude values less than 40 with NaN values
restaurants['latitude'] = restaurants['latitude'].where(restaurants['latitude'] > 40)
```